

昇腾 310B 实战

从入门到精通边缘计算与人工智能

周贤中

2026 年 6 月 12 日

前言

书名中的“实战”，核心是“在编程实践中学习”。本书作为昇腾 310B 芯片的入门教材，跳出单纯的理论讲解，通过真实的 AI 推理部署案例，带读者直观理解昇腾 310B 的硬件架构特性、Atlas 工具链使用逻辑与端侧 AI 项目开发流程，包括模型适配、量化优化、媒体处理、算子开发、性能分析与推理服务部署等环节。每一个知识点都配套可落地的代码示例，使读者在动手编码的过程中逐步掌握昇腾 310B 的工程应用能力，实现从“了解芯片”到“能用芯片落地项目”的跨越。

本书定位与目标读者

本书面向以下三类读者：

- **高校学生 / 科研新人**：希望通过一套系统化路径快速理解边缘 AI 硬件与部署流程。
- **嵌入式 / IoT 工程师**：已有一定 Linux / C / Python 基础，希望把 AI 模型真正跑在边缘端并做性能调优。
- **AI 应用开发者 / 创客**：已经能使用主流深度学习框架，希望将训练好的模型迁移到昇腾 310B 进行高效推理与产品化落地。

阅读预期：

- 零基础读者可依照“快速起步路径”完成第 1~3 章 + 精选案例；
- 进阶读者可继续深入算子优化、系统整合与复杂多模型协同部署；
- 有项目诉求的团队可参考“方法论 + 附录模板”直接搭建属于自己的边缘 AI 应用。

学习路径速览（建议路线）

1. 环境与工具链：硬件认知、开发环境装配、CANN 工具初试
2. 基础模型部署：图像分类、目标检测、语义分割、OCR 与 NLP
3. 媒体处理与底层能力：PyACL、DVPP、算子开发与内存管理
4. 性能优化：模型结构裁剪、精度与性能权衡、Profiling 与流水线优化
5. 系统构建：多进程、多模型协同、任务调度、监控与日志体系
6. 实战案例：需求分析、方案设计、模型适配、部署脚本与交付验收

全书结构（初版规划）

本书分为 **Part I：理论教程**与 **Part II：实验教程**两大部分。

0.1. Part I：理论教程

章节	标题	内容聚焦	读者收益
Chapter 1	昇腾 310B 边缘计算基础	边缘计算概念、竞品对比、310B 规格	建立边缘计算认知与选型能力
Chapter 2	CANN 软件栈核心组件解析	异构架构、ATC 转换工具、msame 推理	掌握 CANN 基础与模型转换流程
Chapter 3	昇腾 PyTorch 扩展迁移基础	环境搭建、torch_npu 插件、经典网络迁移	学会在昇腾上运行 PyTorch 模型
Chapter 4	PyACL 应用开发基础	ACL 运行时、内存管理、模型推理流程	掌握 C/C++ 底层应用开发能力
Chapter 5	DVPP 视频处理基础	VENC、VDEC、VPC、JPEG 媒体处理	掌握 310B 媒体硬件调用与调试方法
Chapter 6	算子开发实战	TBE DSL 流程、310B 计算架构、自定义算子	理解底层计算原理，具备算子扩展能力
Chapter 7	性能分析与优化案例教学	Profiling 分析、瓶颈定位、AIPP/DVPP、流水线优化	掌握性能调优与系统工程化技能
Chapter 8	模型量化与精度性能权衡	FP16、INT8、校准数据、精度对齐与性能对比	学会在精度与性能之间做可验证取舍
Chapter 9	项目交付、评测与回归方法论	需求分析、评测体系、标准化交付流程	建立规范的 AI 项目交付方法论
Appendix A	工具箱、FAQ 与参数模板	错误速查、参数模板、Checklist、FAQ	提供高效查阅的工具与参考资料

0.2. Part II：实验教程

包含从 **Case 0** 到 **Case 9** 的十个动手实验，涵盖环境搭建与九大端侧 AI 实战案例（人脸打卡机、实时跟踪、智能电子琴、掌纹识别、数据采集仪、智能小车、智能相册、手势识别、聊天机器人）。

各模块核心要点概述

1. 昇腾 310B 边缘计算基础

- 边缘计算 vs 云计算：概念辨析与应用场景
- 边缘计算卡生态：NVIDIA、华为、瑞芯微对比
- 昇腾 310B 技术规格详解

2. CANN 软件栈核心组件解析

- CANN 异构计算架构与 CUDA 对比
- ATC 模型转换工具流程、参数详解与注意事项
- msame 推理工具使用概览

3. 昇腾 PyTorch 扩展迁移基础

- PyTorch (torch_npu) 环境安装与配置
- 基础网络实现：Linear, MLP, LeNet
- 现代 CNN 移植实战：AlexNet, VGG, ResNet

4. PyACL 应用开发基础

- 快速入门：Hello World 与运行模式
- 初始化与运行时管理：Device, Context, Stream, Memory
- 模型推理完整流程：加载、Dataset 准备、执行
- Host 与 Device 之间的数据传输、同步与资源释放

5. DVPP 视频处理基础

- DVPP 编程模型、内存约束与数据格式
- VENC、VDEC、VPC、JPEG 的调用流程
- 视频编解码与图像处理案例、参数调试和性能测试

6. 算子开发实战

- 为什么需要自定义算子：TBE vs AICPU
- 昇腾 310B 硬件架构：存储层级与向量计算单元
- TBE DSL 开发流程与基础算子实例

7. 性能分析与优化案例教学

- 可运行 ResNet18-TinyImageNet 性能基线
- Profiling 性能分析：采集、分段计时与关键视图解读
- 核心优化技术：Buffer 复用、AIPP、DVPP 与 Queue Pipeline

8. 模型量化与精度性能权衡

- FP32、FP16、INT8 的区别与适用场景
- PTQ、QAT、校准数据集与精度对齐
- ATC 精度模式、量化失败模式与性能对比流程

9. 项目交付、评测与回归方法论

- 需求澄清 Canvas 与指标分层
- Baseline 策略、评测集设计与迭代看板
- 资产沉淀、交付目录与上线 Checklist
- 验收、回归与风险管理

附录 A. 工具箱、FAQ 与参数模板

- 常见报错速查表
- 模型转换参数模板合集 (ResNet, YOLO, INT8)
- 性能与质量 Checklist
- 术语表、FAQ 与推荐资源

10. 实验教程 (Case 0 - 9)

- **Case 0**: 开发板硬件介绍、系统安装 (刷写/启动/网络)、环境验证
- **Case 1**: 智能打卡机 (Web 应用, 人脸识别)
- **Case 2**: 边缘端实时目标跟踪 (检测 + 关联)
- **Case 3**: 智能电子琴 (手势识别 + 音频合成)
- **Case 4**: 智能掌纹识别机 (预处理, 特征提取)
- **Case 5**: 智能数据采集仪 (传感器, 数据分析)
- **Case 6**: 智能小车 (视觉, 路径规划, 运动控制)
- **Case 7**: 智能相册 (识别, 分类, 检索)
- **Case 8**: 手势识别 (数据采集, 实时系统)
- **Case 9**: 智能聊天机器人 (LLM, 语音交互, RAG)

实践驱动与开源协作

本书所有示例代码、脚本、案例与附录工具均开源托管于本仓库。欢迎通过 Issue / PR 反馈问题、提交改进、补充案例或翻译。我们鼓励读者参与以下工作：

- 增补新模型 / 新任务的部署范式
- 分享 DVPP、PyACL、自定义算子与性能优化经验

- 提交性能测试报告（含硬件信息 + 指标）
- 翻译与文档校对

如何使用本书

读者类型	推荐阅读路径	目标	补充建议
零基础学生	Chapter 1、Chapter 2、Case 0、Case 1	能跑通首个模型	结合案例做改动实验
嵌入式工程师	Chapter 4、Chapter 5、Chapter 6、Chapter 7	掌握底层开发与优化	关注资源管理、媒体处理、算子开发与性能调优
AI 应用开发者	Chapter 2、Chapter 3、Chapter 8、选读 Case 案例	快速场景落地	重点关注 PyTorch 迁移、模型部署与量化评估
技术负责人	Chapter 1、Chapter 7、Chapter 8、Chapter 9	构建团队方法论	制定内部量化评估、性能验收和交付标准

许可证与引用

本书内容采用 Apache 2.0 许可证。引用本书内容请注明：

《昇腾 310B 实战：从入门到精通边缘计算与人工智能》（GitHub: [zhouxzh/Ascend310](#)）

目录

0.1.	Part I: 理论教程	ii
0.2.	Part II: 实验教程	ii
I.	Part I: 理论教程	1
1.	昇腾 310B 边缘计算基础	2
1.1.	边缘计算简介	2
1.1.1.	什么是云计算?	2
1.1.2.	什么是边缘计算?	2
1.1.3.	边缘计算 vs 云计算?	3
1.1.4.	何时采用边缘计算?	4
1.2.	边缘计算卡的发展历史	5
1.3.	常见的边缘计算卡	7
1.3.1.	英伟达 - 边缘 AI 的性能与生态王者	7
1.3.2.	华为 - 全栈全场景的国产化标杆	8
1.3.3.	瑞芯微 - 高性价比的 SoC 芯片专家	8
1.4.	昇腾 310B 简介	9
1.4.1.	昇腾 310B 技术规格详解	9
1.4.2.	其他常见的边缘计算卡的对比	10
1.5.	实践任务	12
2.	CANN 软件栈核心组件解析	13
2.1.	CANN 异构计算架构	13
2.1.1.	CANN 异构计算与人工智能	13
2.1.2.	CANN 与 CUDA	14
2.1.3.	CANN 的架构	14
2.1.4.	CANN 的快速安装	17
2.2.	ATC 模型转换详解	19
2.2.1.	ATC 工具介绍	19
2.2.2.	ATC 工具使用流程	21
2.2.3.	模型推理工具 msame 工具概览	25

2.3. 章节小结	30
2.4. 实践任务	30
3. 昇腾 PyTorch 扩展迁移基础	32
3.1. 架构速览	32
3.2. 核心能力纵览	32
3.3. PyTorch 安装教程	33
3.3.1. CANN 环境准备	33
3.3.2. PyTorch 二进制软件包安装方法	33
3.3.3. torch_npu 安装后测试	35
3.4. torch_npu 插件的使用	36
3.4.1. 线性神经网络实现	36
3.4.2. 多层感知机实现	46
3.4.3. LeNet-5	50
3.5. torch_npu 插件兼容性测试套件	57
3.5.1. 测试目标与验证维度	57
3.5.2. 测试套件结构详解	57
3.5.3. 测试结果摘要	58
3.6. 现代卷积神经网络的移植	60
3.6.1. AlexNet	60
3.6.2. VGG	69
3.6.3. ResNet	74
3.7. 总结	80
4. PyACL 应用开发基础	82
4.1. PyACL 的基本概念	82
4.1.1. PyACL 的逻辑架构	82
4.1.2. 基本概念与运行模型	84
4.1.3. PyACL 应用开发环境	86
4.1.4. PyACL 接口调用流程	87
4.1.5. 运行管理资源生命周期	89
4.1.6. 异构内存管理	91
4.1.7. 同步等待机制	95
4.2. 模型推理流水线	100
4.2.1. 模型推理开发流程解析	100
4.2.2. 实例分析 (ResNet-18)	102
4.3. 目标检测推理	119
4.3.1. SSD300 (ResNet-50 主干)	119

4.3.2.	SSDLite320 (MobileNet 主干)	119
4.3.3.	推理流程对比	123
4.4.	总结	123
5.	DVPP 视频处理基础	124
5.1.	DVPP 基础概念与编程模型	124
5.1.1.	DVPP 在芯片中的位置	124
5.1.2.	DVPP 数据流	125
5.1.3.	子模块概览	125
5.1.4.	AIPP vs DVPP (何时用哪个)	125
5.1.5.	H.264 与 H.265 编解码基础	126
5.1.6.	ACL 初始化——四步必要步骤	127
5.1.7.	通道模型	127
5.1.8.	回调线程模型	128
5.1.9.	DVPP 内存管理	129
5.1.10.	描述符模型	129
5.1.11.	NV12——DVPP 的通用货币	130
5.1.12.	DVPP V1 与 himpi V2 — 两套 API 体系	131
5.1.13.	典型 DVPP 使用模式	131
5.1.14.	开发注意事项	131
5.2.	VENC — 硬件视频编码	132
5.2.1.	理论背景	132
5.2.2.	环境与架构	133
5.2.3.	VENC API 详解	134
5.2.4.	开发过程与常见问题与调试	135
5.2.5.	练习脚本	139
5.2.6.	集成到 aiortc	143
5.2.7.	性能对比与基准测试	144
5.2.8.	附录	148
5.3.	VDEC — 硬件视频解码	149
5.3.1.	VDEC 简介	149
5.3.2.	VDEC 与 VENC 的关键区别	152
5.3.3.	VDEC API 详解	153
5.3.4.	练习脚本走读	154
5.3.5.	常见问题与调试	159
5.3.6.	性能实测	162
5.3.7.	附录	164

5.4. VPC — 硬件图像处理	165
5.4.1. VPC 简介	165
5.4.2. VPC 与 VENC/VDEC 的关键区别	166
5.4.3. VPC API 详解	167
5.4.4. 练习脚本走读	169
5.4.5. 性能基准	171
5.4.6. 常见问题与调试	172
5.4.7. 场景推荐	173
5.4.8. 附录	174
5.5. JPEG — 硬件编解码	174
5.5.1. JPEG 编解码简介	175
5.5.2. 与 VENC/VDEC/VPC 的关键区别	175
5.5.3. JPEGE API 详解	176
5.5.4. JPEGD API 详解	177
5.5.5. 练习脚本走读	179
5.5.6. 场景推荐	180
5.5.7. 常见问题与调试	180
5.5.8. 附录	181
5.6. 集成实战：WebRTC 推流性能对比	182
5.6.1. 项目定位	182
5.6.2. 目录结构与角色分工	183
5.6.3. 关键模块走读	184
5.6.4. 性能实测（1920×1080）	187
5.6.5. 依赖说明	188
5.6.6. 关键结论	189
5.7. 应用调试与常见 FAQ	189
5.7.1. 调试技巧	189
5.7.2. 常见问题	189
5.7.3. 使用约束	190
6. 算子开发实战	191
6.1. 算子开发概述：昇腾 310B 硬件架构与开发路径	191
6.1.1. 昇腾 310B 处理器与达芬奇架构	191
6.1.2. 算子开发的两条主流路径	193
6.1.3. 小结	195
6.2. 初体验：使用 TBE DSL 快速实现第一个算子（向量加法）	195
6.2.1. TBE DSL 快速概览	195
6.2.2. 算子分析：明确我们要做什么	198

6.2.3.	完整代码实现	199
6.2.4.	代码逐段详解	201
6.2.5.	编译算子	202
6.2.6.	算子验证：在 NPU 上运行测试	202
6.2.7.	TBE DSL 开发的优势与局限	205
6.2.8.	小结与展望	206
6.3.	理解算子开发的核心概念	206
6.3.1.	算子原型定义	206
6.3.2.	算子信息库 (.ini 文件)	207
6.3.3.	Tiling (分片计算) 的基本思想	208
6.3.4.	小结	209
6.4.	深入底层：Ascend C 算子开发入门	209
6.4.1.	Ascend C 的编程模型	209
6.4.2.	工程结构	210
6.4.3.	向量加法的 Ascend C 实现	211
6.4.4.	编译与运行	212
6.4.5.	Ascend C 与 TBE DSL 的对比回顾	213
6.5.	从简单到复杂：实现一个需要 Tiling 的算子（如矩阵加法）	213
6.5.1.	场景设定	213
6.5.2.	Tiling 策略设计	213
6.5.3.	带 Tiling 的 Kernel 伪代码	214
6.5.4.	调试技巧	215
6.5.5.	小结与展望	215
7.	性能分析与优化基础	216
7.1.	7.1 性能优化先从可测量开始	216
7.2.	7.2 第 7 章样例目录	217
7.3.	7.3 工具箱：从轻量计时到系统 Profiling	218
7.3.1.	7.3.1 Python 分段计时	218
7.3.2.	7.3.2 msprof：观察 ACL API、Memcpy 和 NPU Timeline	218
7.3.3.	7.3.3 npu-smi：确认设备状态	218
7.3.4.	7.3.4 msame / ais_bench：独立测模型上限	219
7.3.5.	7.3.5 ATC/AIPP 与 DVPP：把图像工作下沉到设备侧	219
7.4.	7.4 案例一：建立同步推理基线	219
7.5.	7.5 案例二：复用 ACL Buffer 和 Dataset	220
7.6.	7.6 案例三：CPU 预处理为什么容易成为瓶颈	221
7.7.	7.7 案例四：DVPP VPC resize 与 CPU resize	221
7.8.	7.8 案例五：AIPP 把归一化下沉到模型输入侧	222

7.9. 7.9 案例六：串行流程与 Queue Pipeline	223
7.10.7.10 作者实测结果	223
7.11.7.11 调优 Checklist	227
7.12.7.12 练习任务	227
7.13.7.13 小结	227
8. 模型量化与精度性能权衡	229
8.1. 8.1 什么时候需要量化	229
8.2. 8.2 FP32、FP16 与 INT8 的区别	229
8.3. 8.3 PTQ 与 QAT	230
8.4. 8.4 校准数据集怎么准备	230
8.5. 8.5 昇腾模型转换中的精度模式	231
8.6. 8.6 精度对齐流程	233
8.7. 8.7 性能对比流程	233
8.8. 8.8 常见失败模式	235
8.9. 8.9 与第 7 章的关系	235
8.10.8.10 练习任务	235
8.11.8.11 小结	236
9. 项目交付、评测与回归方法论	237
9.1. 9.1 章节总览	237
9.2. 9.2 需求澄清 Canvas	237
9.3. 9.3 指标分层与优先级	237
9.4. 9.4 Baseline 策略与控制变量法	238
9.5. 9.5 评测集设计原则	238
9.6. 9.6 迭代计划与看板	239
9.7. 9.7 资产沉淀文档体系	239
9.8. 9.8 交付目录与不可变产物	239
9.9. 9.9 上线前综合 Checklist	240
9.10.9.10 验收、回归与漂移监测	240
9.11.9.11 风险管理与决策日志	241
9.12.9.12 章节小结	241
9.13.9.13 实践任务	241

II. Part II: 实验教程	242
0. 案例 0: 初步使用开发板	243
0.1. 昇腾 310B 开发板介绍	243
0.1.1. 开发板详细视图	243
0.1.2. 开发板硬件规格	243
0.2. 配件准备	243
0.3. 操作系统安装	250
0.3.1. 镜像下载	251
0.3.2. 校验下载文件 (MD5)	253
0.3.3. 刷写系统到 TF 卡	253
0.4. 启动开发板	260
0.4.1. 方式一: 图形化界面启动	260
0.4.2. 串口界面	261
0.5. 网络连接	264
0.5.1. 无线网络连接 (WiFi)	264
0.5.2. 有线网络连接	264
0.6. 远程连接 (SSH)	265
0.7. 验证开发环境	265
0.7.1. 设置 SWAP 交换分区	266
0.8. 结语	266
1. 案例 1: 智能人脸识别考勤系统	267
1.1. 教程定位	267
1.2. 实验硬件与运行条件	267
1.3. 本案例支持的模型结构	269
1.4. 人脸识别系统的核心流水线	269
1.5. RetinaFace 深度剖析: 从“看见”到“看懂”人脸	269
1.5.1. RetinaFace 的发展历史与作者信息	269
1.5.2. RetinaFace 到底在解决什么问题?	270
1.5.3. 核心技术解析	270
1.5.4. 在本案例中的应用	271
1.6. ArcFace 深度剖析: 让特征在角度空间中更具区分度	271
1.6.1. 从 Softmax 到度量学习: 损失函数的演进之路	271
1.6.2. ArcFace 的核心创新: 加性角度间隔	272
1.6.3. 在本案例中的应用	273
1.7. 数据存储与检索深度剖析: 边缘场景下的智慧与权衡	273
1.7.1. 为什么选择 SQLite?	273

1.7.2.	特征向量的存储: BLOB 格式的优与劣	273
1.7.3.	当前的检索方案: 简单遍历	274
1.7.4.	性能瓶颈与未来展望	274
1.8.	PyACL 深度剖析: 与昇腾 NPU 对话的艺术	275
1.8.1.	什么是 ACL? 为什么需要它?	275
1.8.2.	昇腾计算语言 (ACL) 的标准工作流	275
1.8.3.	为什么需要区分 Host 和 Device?	279
1.8.4.	从 ONNX 到 OM: 模型转换的必要性	279
1.9.	Anchor-based 检测深度解析: RetinaFace 如何“看见”人脸	280
1.9.1.	目标检测的根本问题	280
1.9.2.	Anchor-based 方法的智慧	281
1.9.3.	边界框解码: 从偏移量到坐标	282
1.9.4.	NMS (非极大值抑制): 去除重复检测	282
1.10.	余弦相似度: 如何判断“两张脸是同一个人”	283
1.10.1.	特征向量的几何意义	283
1.10.2.	方法 1: 欧氏距离 (不推荐)	283
1.10.3.	方法 2: 余弦相似度 (推荐)	284
1.10.4.	为什么 ArcFace 适合用余弦相似度?	284
1.10.5.	阈值的选择: 0.5 的含义	284
1.11.	多线程与实时性: 摄像头后台线程的设计	285
1.11.1.	为什么需要多线程?	285
1.11.2.	多线程的优势	286
1.11.3.	线程安全: Lock 的作用	286
1.11.4.	daemon 线程的含义	286
1.12.	Flask Web 框架: 构建轻量级 API 服务	287
1.12.1.	什么是 Web 框架?	287
1.12.2.	Flask 的核心概念	287
1.12.3.	MJPEG 视频流: 实时传输的技巧	288
1.13.	图像预处理: 为什么需要归一化?	289
1.13.1.	第 1 步: 缩放到固定尺寸	289
1.13.2.	第 2 步: 归一化 (Normalization)	289
1.13.3.	第 3 步: 通道顺序转换	290
1.13.4.	第 4 步: 添加 Batch 维度	290
1.14.	性能优化策略: 让系统跑得更快	291
1.14.1.	1. 内存复用: 避免重复分配	291
1.14.2.	2. 批处理: 一次处理多张图片	291
1.14.3.	3. 异步推理: 让 CPU 和 NPU 并行工作	292
1.14.4.	4. 模型量化: 用更少的位数表示权重	292

1.15. 实际部署建议	292
1.15.1. 1. 用户规模与检索策略	292
1.15.2. 2. 防止重复打卡	293
1.15.3. 3. 安全性考虑	293
1.15.4. 4. 日志与监控	294
1.15.5. 5. 错误处理与降级策略	294
1.16. 常见问题深度解答	294
1.16.1. Q1: 为什么摄像头打开失败?	294
1.16.2. Q2: 识别率低怎么办?	295
1.16.3. Q3: 推理速度慢怎么办?	296
1.17. 扩展实验建议	296
1.17.1. 实验 1: 添加情绪识别	296
1.17.2. 实验 2: 多摄像头支持	296
1.17.3. 实验 3: 移动端集成	296
1.18. Web 界面详细介绍	297
1.19. Web 界面详细介绍	297
1.19.1. 1. 主页 (/)	297
1.19.2. 2. 用户管理 (/users_page)	297
1.19.3. 3. 考勤记录 (/attendance_page)	298
1.19.4. 4. 实时视频流 (/video_feed)	298
1.20. 系统架构总结	298
1.20.1. 数据流向	298
1.20.2. 关键技术栈	299
1.20.3. 性能指标	299
1.21. 学习路径建议	299
1.21.1. 初学者 (第 1-2 周)	299
1.21.2. 进阶学习 (第 3-4 周)	300
1.21.3. 高级应用 (第 5-8 周)	300
1.22. 总结	301
2. 案例 2: 目标跟踪检测	302
2.1. 教程定位	302
2.2. 实验硬件与运行条件	302
2.3. 本案例支持的骨干网络	304
2.4. 目标跟踪对时序关联的要求	304
2.5. MobileNet-SSD 作为检测前端的依据	305
2.5.1. 边缘侧场景的基本要求	305
2.5.2. MobileNet 的网络结构	305

2.5.3.	MobileNet v1 至 v4 的版本演进	306
2.5.4.	MobileNet v1-v4 的结构对比总结	308
2.5.5.	MobileNet 与 ResNet 骨干在本案例中的取向差异	308
2.5.6.	SSD 的检测特点	309
2.5.7.	MobileNet-SSD 用于目标跟踪的优缺点	309
2.6.	DeepSORT 作为跟踪后端的选择依据	310
2.6.1.	从检测到跟踪, 还缺少什么	310
2.6.2.	DeepSORT 为什么适合作为入门算法	310
2.7.	本案例的代码结构与总体流程	310
2.7.1.	入口层	311
2.7.2.	检测层	311
2.7.3.	跟踪层	311
2.8.	检测部分代码解析	311
2.8.1.	detection_app.py 如何组织检测主流程	311
2.8.2.	backend_base.py 如何统一检测后端	312
2.8.3.	preprocess_frame 体现了部署输入规范	313
2.8.4.	decoder.py 如何体现 SSD 的核心思想	313
2.8.5.	CPU 与 NPU 后端如何被统一接入	318
2.9.	从检测到跟踪: tracking_app.py 的桥梁作用	318
2.10.	跟踪部分代码解析	319
2.10.1.	Track 类如何表示一条轨迹	319
2.10.2.	predict 和 update 对应卡尔曼滤波两阶段	319
2.10.3.	kalman_filter.py 如何实现线性状态估计	320
2.10.4.	卡尔曼滤波器的发展历史与作者信息	320
2.10.5.	卡尔曼滤波器到底在解决什么问题	321
2.10.6.	本案例中的 predict 与 update 如何对应算法公式	321
2.10.7.	卡尔曼滤波在本案例中的适用性	323
2.10.8.	DeepSORT.update 如何体现多目标跟踪主线	323
2.10.9.	数据关联为什么使用 IOU + 匈牙利算法	324
2.10.10.	匈牙利算法的发展历史与作者信息	324
2.10.11.	匈牙利算法到底解决什么问题	325
2.10.12.	本案例中匈牙利算法是如何被调用的	325
2.10.13.	匈牙利算法在本案例中的完整上下文	325
2.10.14.	匈牙利算法相对于贪心匹配的优势	326
2.10.15.	匈牙利算法在多目标跟踪中的局限	326
2.10.16.	轨迹生命周期参数如何影响最终效果	327
2.11.	可视化代码如何帮助理解算法结果	327

2.12. 如何评价这个案例方案	328
2.12.1. 这个方案的优点	328
2.12.2. 这个方案的不足	328
2.12.3. 作为入门案例的合理性	328
2.13. 实验设计	328
2.13.1. 只运行检测链路	329
2.13.2. 检测链路性能拆分实验	329
2.13.3. 在检测基础上启用跟踪	330
2.13.4. 类别过滤与解码开销实验	330
2.13.5. 调节 DeepSORT 参数	331
2.13.6. 调节中心距离与平滑参数	331
2.13.7. 组合实验建议	332
2.14. 当前工程版本的优化说明	332
2.14.1. 检测链路中的工程优化	332
2.14.2. 进一步提高帧率的建议	335
2.14.3. 跟踪链路中的工程优化	335
2.14.4. 卡尔曼滤波器中的工程优化	337
2.14.5. tracking_app.py 当前支持的实用参数	337
2.14.6. 如何理解“教材版”和“工程版”的关系	338
2.15. 章节总结	338
3. 案例 3：智能电子琴	340
3.1. 1. 项目简介	340
3.2. 2. 内容大纲	340
3.2.1. 2.1. 硬件准备	340
3.2.2. 2.2. 软件环境	341
3.2.3. 2.3. 手势识别与音符映射	342
3.2.4. 2.4. 模型训练与优化	342
3.2.5. 2.5. 音频合成与处理	343
3.2.6. 2.6. 3D 打印结构件	343
3.2.7. 2.7. 用户手册	344
3.3. 3. 源代码结构	344
3.4. 4. 效果演示	345
4. 案例 4：智能掌纹识别机	346
4.1. 1. 项目简介	346
4.1.1. 与已有案例的关系	346

4.2.	2. 内容大纲	347
4.2.1.	2.1. 硬件准备	347
4.2.2.	2.2. 软件环境	347
4.2.3.	2.3. 掌纹图像预处理	349
4.2.4.	2.4. GhostNet 特征提取	350
4.2.5.	2.5. 对比学习训练	352
4.2.6.	2.6. 模型转换与昇腾部署	354
4.2.7.	2.7. 开集验证系统	355
4.2.8.	2.8. Web 仪表盘	356
4.2.9.	2.9. 用户手册	357
4.3.	3. 源代码结构	358
4.4.	4. 效果演示	360
4.4.1.	预期效果	360
4.4.2.	性能指标	360
4.4.3.	浏览器中的效果	360
4.4.4.	如何验证系统正常工作	361
5.	案例 5：智能数据采集仪	362
5.1.	1. 项目简介	362
5.1.1.	与已有案例的关系	362
5.2.	2. 内容大纲	362
5.2.1.	2.1. 硬件准备	362
5.2.2.	2.2. 软件环境	364
5.2.3.	2.3. STM32 低速传感器采集	365
5.2.4.	2.4. FPGA 高速振动采集	365
5.2.5.	2.5. 振动频谱分析原理	366
5.2.6.	2.6. NPU 故障分类	367
5.2.7.	2.7. CPU 趋势分析与异常检测	368
5.2.8.	2.8. 模型转换与昇腾部署	369
5.2.9.	2.9. Web 仪表盘	370
5.2.10.	2.10. 用户手册	371
5.3.	3. 源代码结构	372
5.4.	4. 效果演示	373
5.4.1.	预期效果	373
5.4.2.	性能指标	374
5.4.3.	浏览器中的效果	374
5.4.4.	如何验证系统正常工作	375

6. 案例 6：智能小车视觉感知	376
6.1. 1. 项目简介	376
6.1.1. 与已有案例的关系	376
6.2. 2. 内容大纲	376
6.2.1. 2.1. 硬件准备	376
6.2.2. 2.2. 软件环境	377
6.2.3. 2.3. 车道线检测原理	378
6.2.4. 2.4. 驾驶场景分类	379
6.2.5. 2.5. 模型转换与昇腾部署	380
6.2.6. 2.6. Web 界面	381
6.2.7. 2.7. 用户手册	382
6.3. 3. 源代码结构	384
6.4. 4. 效果演示	385
6.4.1. 预期效果	385
6.4.2. 性能指标	385
6.4.3. 浏览器中的效果	385
6.4.4. 如何验证系统正常工作	386
7. 案例 7：智能相册	387
7.1. 1. 项目简介	387
7.1.1. 三个核心功能	387
7.1.2. 与已有案例的关系	387
7.2. 2. 内容大纲	388
7.2.1. 2.1. 硬件准备	388
7.2.2. 2.2. 软件环境	388
7.2.3. 2.3. 图像特征提取原理	390
7.2.4. 2.4. 模型转换与昇腾部署	391
7.2.5. 2.5. 照片索引与向量检索	393
7.2.6. 2.6. Web 界面	395
7.2.7. 2.7. 用户手册	396
7.3. 3. 源代码结构	398
7.4. 4. 效果演示	399
7.4.1. 预期效果	399
7.4.2. 性能指标	399
7.4.3. 浏览器中的效果	400
7.4.4. 如何验证系统正常工作	400

8. 案例 8：实时手势识别	401
8.1. 项目简介	401
8.2. 学习目标	401
8.3. 实验环境	402
8.4. HaGRID 手势检测任务	402
8.5. YOLOv10 与本案例模型	405
8.6. 模型导出与 ATC 转换	409
8.7. OM 推理与代码解析	411
8.8. WebRTC 远程推流	413
8.9. OpenCV 与 DVPP 采集后端	414
8.10. 完整实验流程	414
8.11. 性能分析与优化	415
8.12. 常见问题	416
8.13. 维护建议	417
8.14. 参考资料	417
9. 案例 9：边缘智能聊天机器人	418
9.1. 1. 项目简介	418
9.2. 2. 内容大纲	418
9.2.1. 2.1. 硬件准备	418
9.2.2. 2.2. 软件环境	419
9.2.3. 2.3. 系统架构设计	420
9.2.4. 2.4. 文本嵌入模型与昇腾部署	423
9.2.5. 2.5. 知识库与向量检索	426
9.2.6. 2.6. 对话管理与响应生成	428
9.2.7. 2.7. 语音交互系统	430
9.2.8. 2.8. Web 界面与 API 服务	431
9.2.9. 2.9. 用户手册	432
9.3. 3. 源代码结构	433
9.4. 4. 效果演示	434
9.4.1. 基础对话 — 离线模板模式	434
9.4.2. RAG 检索 — 专业问题	435
9.4.3. 语音交互	435
9.4.4. 性能指标	435
A. 工具箱、FAQ 与参数模板	436
A.1. A.1 附录总览	436
A.2. A.2 常见报错速查	436

A.3. A.3 模型转换参数模板合集	437
A.3.1. A.3.1 分类模型 (ResNet)	437
A.3.2. A.3.2 YOLO 动态分辨率	437
A.3.3. A.3.3 INT8 量化 (示例)	437
A.4. A.4 性能与质量 Checklist (执行勾项)	437
A.5. A.5 术语表 (扩展)	438
A.6. A.6 推荐资源与外部引用	439
A.7. A.7 贡献指南摘要	439
A.8. A.8 FAQ	439
A.9. A.9 License 与引用	439
A.10.A.10 版本路线回顾	440
A.11.A.11 实践任务	440

Part I.

Part I: 理论教程

1. 昇腾 310B 边缘计算基础

1.1. 边缘计算简介

华为云等公共云计算平台允许企业以全国的云服务器作为其私有的数据与 AI 计算中心，将其基础设施扩展到任意地点，并根据需要向上或向下扩展计算资源。然而遍布全球实时运行的 AI 应用可能需要显著的本地处理能力，且往往位于距离集中式云服务器过远的偏远位置。而且出于低时延或数据驻留要求，一些工作负载需要保留在本地或特定地点，就是为什么许多企业使用边缘计算来部署其 AI 应用。边缘计算指的是在数据产生的位置进行处理，边缘计算在边缘设备中本地处理与存储数据。由于边缘计算设备无需依赖互联网连接，可以作为独立的网络节点运行。

1.1.1. 什么是云计算？

云计算（Cloud Computing）是一种计算风格，其可扩展与弹性能力通过互联网技术以服务形式交付。云计算依托集中化数据中心，通过互联网按需提供弹性、可扩展、托管式 IT 资源与服务（计算 / 存储 / 网络 / 数据 / AI 平台）。

云计算的好处是什么？- 较低的前期成本：购买硬件、软件、IT 管理以及全天候电力与制冷的资本支出被消除。云计算使组织能够以较低的财务进入门槛快速将应用推向市场。灵活的定价 – 企业只为其使用的计算资源付费，从而对成本有更多控制并减少意外。- 按需无限计算：云服务可以通过自动配置与撤销资源即时对不断变化的需求做出反应与适配。这可以降低成本并提升组织整体效率。- 简化的 IT 管理：云提供商为其客户提供访问 IT 管理专家的渠道，使员工可以专注于企业的核心需求。- 便捷更新：可一键访问最新的硬件、软件与服务。- 可靠性：数据备份、灾难恢复与业务连续性更容易且更便宜，因为数据可以在云提供商网络的多个冗余站点镜像。- 节省时间：企业可能在配置私有服务器与网络上耗费时间。借助按需云基础设施，它们可以在更短时间内部署应用并更快进入市场。

1.1.2. 什么是边缘计算？

边缘计算（Edge Computing）是一种分布式计算框架，旨在将计算、存储和网络能力部署在靠近数据源或终端设备的网络边缘位置。通过在本地或近端节点处理数据，减少向远端数据中心/云的集中回传，获得更低的时延、更好的带宽利用与更高的数据主权与隐私保障。边缘计算是在距离数据产生源（传感器、摄像头、终端设备、工业控制点）物理更近的位置部署计算与存

储，使数据在本地/近端被快速处理与筛选，减少长距离回传，保障低时延、带宽节省与数据主权。边缘计算是将计算能力在物理上靠近数据生成位置（通常是物联网设备或传感器）的实践。因为计算能力被带到网络或设备的边缘，边缘计算能够实现更快的数据处理、增加的带宽以及确保的数据主权。

通过在网络边缘处理数据，边缘计算减少了大量数据在服务器、云与设备或边缘位置之间往返传输以被处理的需求。这对诸如数据科学与 AI 等现代应用尤为重要。许多高计算应用（如深度学习与推理、数据处理与分析、仿真与视频流）已成为现代生活的支柱。随着企业日益意识到这些应用由边缘计算驱动，生产中的边缘用例数量应会增加。

边缘计算的特点是什么？ - 靠近数据源：在物联网设备、网关、工业控制器或本地微型服务器附近直-成初级或核心推理逻辑，降低往返延迟。 - 分布式架构：区别于云的集中式调度，采用多节点协同与局部自治，适合-化、地理分散与对实时性敏感的任务。 - 实时响应：适配无人驾驶、工业控制、视频安防、AR/VR 等对毫秒级响应-的场景。 - 隐私与安全：敏感原始数据（面部特征、生产工艺、地理轨迹）在本地预-或结构化提取后再上云，降低泄露与合规风险。 - 带宽优化：仅上传事件/特征/聚合指标，显著降低原始全量视频/传感流量-路的占用。 - 弹性与容错：弱网/离线时保持关键功能脱网运行，网络恢复后再同步（延迟一致性）。

边缘计算的好处是什么？ - 更低时延：在边缘进行数据处理会消除或减少数据传输。这可为需要低时延的复杂 AI 模型用例（如完全自动驾驶车辆与增强现实）加速洞察。 - 降低成本：使用局域网进行数据处理相比云计算可为组织提供更高带宽与更低成本的存储。此外，由于处理发生在边缘，需要发送到云或数据中心进一步处理的数据更少。这导致需要传输的数据量减少，成本也降低。 - 模型精度：AI 依赖高精度模型，尤其是需要实时响应的边缘用例。当网络带宽过低时，通常通过降低输入模型的数据尺寸来缓解。这会导致图像尺寸缩小、视频跳帧、音频采样率降低。部署在边缘时，数据反馈回路可用于提升 AI 模型精度，并且可同时运行多个模型。 - 更广覆盖：传统云计算必须依赖互联网接入。而边缘计算可在本地处理数据，无需互联网接入。这将计算范围扩展到以前无法访问或偏远的位置。 - 数据主权：当数据在其采集位置被处理时，边缘计算允许组织将所有敏感数据与计算保留在局域网和公司防火墙内。这减少了暴露于云端网络安全攻击的风险，并在不断变化的严格数据法律下具备更好合规性。

1.1.3. 边缘计算 vs 云计算？

维度	边缘计算	云计算	典型取舍 / 典型场景
处理位置	近端（本地/网关/边缘节点）	远端数据中心	边缘降低时延并就近处理高带宽原始数据（如视频）；云用于集中算力与全局聚合。
时延特性	低（本地判决 20~几十 ms）	受网络往返影响（>100ms 场景常见）	实时/控制闭环优先边缘；非实时批处理、训练优先云。
带宽占用	上行压缩（只传事件/特征/聚合）	常上传原始数据到云	当传输成本高或链路受限，将预处理放到边缘；带宽充足且需保留原始数据则上云。

维度	边缘计算	云计算	典型取舍 / 典型场景
部署/运维	分布式，需节点管理（异构、离线可用）	集中化，统一维护与弹性伸缩	节点数多时运维复杂度上升（需探针/模板）；云侧适合动态弹性与大规模训练/批处理。
隐私合规	本地脱敏/保留数据主权易控	数据集中存储需合规审核	高度敏感或受监管数据优先边缘；低合规风险且需统一治理优先云。
伸缩弹性	受制于本地硬件与现场成本	云端资源弹性丰富	现场扩展（CAPEX）与运维（OPEX）成本较高；云适合突发/动态扩展工作负载。
典型适用场景（示例对照）	实时推理、低延迟控制、弱网/离线场所	非实时批处理、模型训练、集中化数据湖	云：非时间敏感数据、可靠互联网、已在云存储的数据；边缘：实时数据处理、受限或无联网地点、传输成本过高的大规模本地数据、受严格法规约束的数据。

一个边缘计算优于云计算的示例是医疗机器人，外科医生需要实时数据访问。这些系统包含大量可在云中执行的软件，但手术室中日益出现的智能分析与机器人控制无法容忍时延、网络可靠性问题或带宽限制。在此示例中，边缘计算为患者提供了生死攸关的益处。

1.1.4. 何时采用边缘计算？

边缘节点的硬件选择，正如您所指出的，本质上是功耗、体积和成本之间的权衡。像华为 Ascend 310B 这样的处理器正是这一理念的产物：它不追求极致的算力，而是在满足边缘场景基本 AI 推理需求的前提下，尽可能实现能效比和成本的最优解。这种硬件特性直接决定了其所能支撑的应用边界。边缘计算的价值在多种场景中得以凸显。在物联网网关中，它扮演着“区域数据枢纽”的角色，对海量传感数据进行本地预处理、协议转换与降噪聚合，大幅减轻上行带宽压力。工业自动化场景，如产线质量检测与能耗分析，依赖边缘节点的毫秒级响应能力，实现控制闭环的实时性与可靠性。在智慧城市的交通流量或环境监测中，边缘计算使得低延时告警成为可能，同时有效节省了持续视频流上传所需的巨大带宽。

安防监控是边缘计算的经典应用，通过实时视频结构化分析，将原始视频转化为结构化的事件信息（如“有人越界”、“车辆违章”）再上传，既保护了个人隐私，又提升了事件处理效率。智能零售中的客流与货架分析，则体现了边缘的设备自治特性，即使在网络不佳时也能独立运行。而对于车路协同中的路侧单元，超低时延与本地协同决策是保障行车安全的核心，任何云端的往返延迟都是不可接受的。

边缘计算并非云的替代品，而是与云共同构成了“端 - 边 - 云”三层协同的健壮体系。在这个体系中，端侧负责产生原始数据，边缘侧专注于低时延的智能决策、实时控制与数据“提纯”，而云端则依托其强大的算力与存储资源，负责全局模型的训练、长周期的大数据分析与跨区域的调度协同。

一个合理的功能切分策略至关重要，它直接决定了整个系统的成本结构、响应性能与可持

续演进能力。判断一个功能更适合放在云还是边缘，可以遵循一些基本原则：适合上云的任务通常是非实时的批处理、模型训练、需要动态弹性伸缩的业务，或者数据本就存储在云数据湖中的场景，且对合规和延迟不敏感。相反，更适合下沉到边缘的任务则对实时推理和控制、稳定持续的低时延有强需求，其数据来源密集且分散，或者涉及高敏感数据受合规监管，同时面临着带宽受限或成本高昂的现实约束。

在具体的工程实现上，云和边缘的偏好存在显著差异，这要求开发者采用不同的技术策略。

在日志策略上，云侧倾向于全量集中收集以方便分析，而边缘侧由于带宽和存储限制，必须采用采样与本地环形缓存截断等轻量化手段。模型分发时，云侧可以利用 CDN 轻松推送大文件，边缘侧则需要差分升级、分块传输与严格的哈希校验机制，以确保在不稳定网络下的更新成功率。配置管理方面，边缘服务需支持配置嵌入版本与增量下发，并必须具备离线安全回滚能力，以应对网络中断的极端情况。

监控系统的设计上，云侧普遍采用 Prometheus 等集中式时序数据库，而边缘侧则需要轻量的代理，并优先在本地缓存指标，在资源冲突时甚至需要丢弃低优先级数据。对于安全补丁，云上可以做到自动批量推送，但在边缘侧，为了避免对关键业务造成意外中断，通常需要设定计划维护窗口或要求手动确认，实施更为谨慎的更新策略。

总而言之，边云协同的主旋律是“边缘强化实时响应与局部自治，云强化全局优化与集中智能”。在设计系统时，一个有效的思路是以“放在云端的必要性”为拷问，反向审视每一个功能模块。任何不具备必须上云理由的功能，都应优先考虑下沉到边缘。最终的功能切分边界，应当由可观测的量化指标来界定，包括时延、带宽消耗、总体拥有成本、处理精度以及合规等级等。

在实际的资源与任务编排中，需要精细化管理。例如，将高 I/O 密度的任务与计算密集型任务错峰执行，以避免资源争抢。将热点算子进行分组，防止它们在短时间内同时提交导致带宽剧烈抖动。同时，必须高度重视热设计，通过持续监控硬件温度（如每 5 秒采样），在超过安全阈值（如 85°C）时自动触发降频或任务降级等保护性负载策略，确保边缘设备在复杂现场环境下的长期稳定运行。

1.2. 边缘计算卡的发展历史

边缘计算卡的发展历程，是一场为了将“智能”从遥远的云端数据中心不断推向现实生活前沿的“迁徙史”。在早期，所谓的边缘节点大多是基于通用的低功耗 CPU 构建，处理能力有限，难以胜任复杂的实时计算任务。那时可以说是“有边缘，而无计算”。随着自动驾驶和安防监控等应用对视觉处理的需求爆发，市场开始探索专用的加速方案。FPGA 因其可重构性和低延迟成为重要选择，同时，像英特尔 Movidius 这类专为视觉优化的低功耗芯片也开始出现，让设备真正具备了本地处理 AI 的能力。真正的转折点来自深度学习浪潮。英伟达将其 GPU 技术下沉，推出了划时代的 Jetson 系列。这不仅带来了强大的算力，更重要的是建立了成熟的 CUDA 软件生态，让边缘 AI 开发从硬件调优变成了软件工程，极大地降低了开发门槛。近年来，边缘计算卡的发展进入了百家争鸣的时代。英特尔通过“CPU+VPU+FPGA”组合和 OpenVINO 工具链提供灵活方案；高通将移动端的高能效 SoC 设计经验引入边缘；谷歌则推出了为 TensorFlow 量

身定制的 Edge TPU，追求极致能效。同时，国产力量快速崛起，华为的昇腾系列、寒武纪的思元系列等，都在安防、自动驾驶等特定领域展现出强大竞争力。如今，边缘计算卡已不再是单纯的算力竞赛，而是进入了**算力、功耗、成本、软件栈和行业生态**的综合竞争阶段。它的演进史，核心始终是为了让智能在物理世界的每个角落都能实时、可靠地运行。

当前，海外边缘计算市场格局鲜明，由几家技术路径各异的科技巨头共同引领。

英伟达凭借其 Jetson 系列，被视为边缘设备中的“微型超算”。该系列覆盖从入门到旗舰的全场景需求，不仅以强劲性能著称，更以成熟的 CUDA 和 TensorRT 软件生态构筑起护城河。无论是复杂的机器人视觉系统还是自动驾驶车辆，Jetson 往往是开发者实现高性能边缘 AI 的首选平台。

英特尔则展现出“全能型选手”的特质。它不仅以 CPU 处理通用计算任务，还通过 Movidius 视觉处理单元等专用芯片强化 AI 视觉推理。其核心优势在于 OpenVINO 软件工具包——能够将 AI 模型高效适配并优化至英特尔各类硬件平台，使其在工业自动化、智能零售等需要灵活部署的领域中广受青睐。

AMD 在整合赛灵思之后，实力显著提升。其杀手锏在于“自适应计算”，尤其是 FPGA 芯片的可重构特性，能够通过硬件级定制实现超低延迟加速。这一特性让 AMD 在专业机器视觉、通信基础设施等需要快速算法迭代的特定场景中脱颖而出。

高通将移动芯片领域积累的低功耗与高集成度经验成功复用于边缘侧。其芯片方案集成了专用 AI 引擎与 5G 连接能力，特别适合无人机、扩展现实设备等对续航和体积敏感的移动场景，为电池供电的边缘设备提供了持久智能的可能。

谷歌选择了一条差异化的技术路径，推出专为 TensorFlow 模型推理设计的 Edge TPU 芯片。这款芯片以极低的功耗和紧凑的体积，在兼容的 AI 任务中提供出色的推理速度。对于深度融入谷歌 AI 生态的开发者而言，基于 Edge TPU 的 Coral 开发板成为一个高效、节能的部署选择。

国内边缘计算卡的发展，是一部从“跟跑”到“并跑”的奋斗史。大约在 2016 年前后，随着 AI 浪潮兴起，国内边缘计算开始萌芽。最初阶段，大多数企业还是采用国外芯片做集成开发，或者在通用芯片上进行适配优化。这个时期可说是“用起来就好”的实用主义阶段。2018 年左右，随着“中兴事件”的爆发，自主可控成为国家战略，真正推动了国产边缘计算芯片的研发热潮。以**华为昇腾 310** 芯片为代表的第一代国产边缘 AI 芯片正式亮相，标志着我们开始拥有自己的“算力底座”。这款芯片不仅在性能上对标国际产品，更重要的是实现了从芯片到框架的全栈自主创新。几乎在同一时期，**寒武纪**的思元系列、**地平线**的征程系列等专用 AI 芯片也纷纷落地。这些芯片不再是简单的替代品，而是在特定场景下展现出了独特优势——比如寒武纪在算力密度上的突破，地平线在自动驾驶领域的专注深耕。2020 年之后，国产边缘计算进入了“场景深化”阶段。各家企业不再追求单纯的算力指标，而是更注重实际应用效果。华为 Atlas 系列在智慧城市项目中大规模部署，地平线的征程芯片在多家车企实现前装量产，黑芝麻智能则专注于大算力自动驾驶芯片的研发。这些成就显示，国产边缘计算卡已经从“能用”进化到了“好用”的阶段。如今，国产边缘计算卡正在形成自己独特的发展路径——不仅在性能上紧追国际先进水平，更在能效比、场景适配度和成本控制上展现出越来越强的竞争力。从最初的艰难起步，到现在的百花齐放，这段发展历程见证了中国在高科技领域的快速崛起。

近年来，国内科技公司在自研 AI 芯片和计算卡领域进展迅速，形成了具有鲜明特色的产品体系。

华为是国内边缘计算领域的领头羊，其 **Atlas 系列**是基于自研的“昇腾”（Ascend）AI 处理器。它不仅仅是一张卡，更是一个完整的解决方案。从面向开发者的 **Atlas 200I DK** 套件，到集成了多张加速卡的 **Atlas 500** 智能边缘服务器，华为提供了覆盖多种场景的产品形态。其最大优势在于“软硬件协同”，通过自家的 CANN 驱动和昇思（MindSpore）AI 框架，能充分发挥芯片性能，特别在安防、智慧城市等要求自主可控的项目中成为首选。

寒武纪是国内专业的 AI 芯片上市公司，其“思元”（MLU）系列边缘计算卡在业界拥有很高的知名度。寒武纪的芯片以专业化的 **AI 算力**和**高计算密度**见长，在智能安防、智慧交通等领域实现了规模化应用。它的产品是纯计算加速卡形态，可以灵活地插入到标准的服务器中，为数据中心和边缘机房提供强大的推理能力。

地平线是另一家极具特色的公司，其强项在于**自动驾驶和智能驾驶舱**领域。它提供的不是单一的计算卡，而是以“芯片 + 工具链”为核心的解决方案。其**征程系列**车规级 AI 芯片，功耗控制非常出色，专门为处理智能汽车上的视觉感知、环境建模等任务而优化。通过与众多车企和零部件供应商合作，地平线的技术已经广泛应用在众多量产车型中。

此外，像**瑞芯微**这样的传统 SoC 设计公司，其 **RK3588** 等高端芯片也具备了可观的边缘 AI 算力。虽然它们通常以核心板的形式出现，但因其极高的**性价比**和强大的**多媒体处理能力**，被广泛集成到各种边缘设备中，如智能 NVR、商显广告机和工业控制设备，是中低算力需求场景中非常普遍的选择。

总的来说，国内边缘计算卡市场呈现出百花齐放的态势：华为提供全栈式方案，寒武纪提供专业的算力卡，地平线和黑芝麻则深耕自动驾驶这一垂直领域，而瑞芯微等则在更广泛的 AIoT 市场占据重要地位。这些国产力量共同为各行各业的智能化转型提供了坚实且多样化的算力基础。

1.3. 常见的边缘计算卡

好的，我们重点来介绍一下**英伟达**、**华为**和**瑞芯微**这三家在边缘计算领域颇具代表性的产品。这三家的产品定位和优势各有侧重，形成了从高性能到高性价比的全面覆盖。

1.3.1. 英伟达 - 边缘 AI 的性能与生态王者

英伟达凭借其强大的 GPU 架构和成熟的软件栈（CUDA, TensorRT 等），在边缘 AI 领域占据了主导地位，尤其是在需要复杂 AI 推理的场景中。英伟达的 Jetson 系列构成了一个完整的嵌入式 AI 计算平台，涵盖了从核心计算模块到载板设计，再到软件栈的全套解决方案。

当前产品线的中流砥柱是 Jetson Orin 系列。其中的旗舰型号 Jetson AGX Orin 堪称性能猛兽，拥有高达 275 TOPS 的 AI 算力，性能足以媲美小型服务器，能够胜任自主机器、高级机器人、医疗影像等最严苛的边缘应用。对于需要高性能但空间受限的场景，Jetson Orin NX 提

供了理想的解决方案，它在紧凑的体积内实现了最高 100 TOPS 的算力，成为多路视频分析设备和边缘服务器的首选。而 Jetson Orin Nano 则重新定义了入门级边缘 AI 的标准，以 20-40 TOPS 的算力为新一代 AI 产品和原型开发打开了新的可能。

除了新一代产品，一些经典机型仍然在市场上发挥着重要作用。Jetson Xavier NX 凭借其在性能和价格之间的出色平衡，至今仍在许多项目中广泛使用。更早的 Jetson Nano 虽然性能已经不及新产品，但其低廉的成本和庞大的开发者社区，使其仍然是教育入门和简单项目的绝佳选择。

这个平台最突出的优势在于其极致的性能表现和无比成熟的软件生态。CUDA-X AI 为开发者提供了完善的工具链，加上活跃的社区支持和丰富的学习资料，大大降低了开发门槛。不过，这些优势也伴随着相应的代价——Jetson 产品的价格相对较高，功耗表现也不算特别出色。正因如此，它们主要被应用于自动驾驶、高端机器人、复杂视频分析等对计算性能要求极高的场景中。

1.3.2. 华为 - 全栈全场景的国产化标杆

华为基于其自主研发的昇腾 AI 处理器与昇思 AI 框架，构建了一套从底层芯片到顶层应用的“全栈式”人工智能解决方案。这一完整的技术布局使华为在安防、智慧交通等对自主可控要求较高的领域展现出独特优势。

在边缘计算领域，华为通过 Atlas 系列产品提供了丰富的形态选择。对于开发者而言，Atlas 200I DK A2 是一个理想的入门平台，这款小巧的开发板搭载昇腾 310 处理器，为初学者提供了体验昇腾生态的便捷途径。而在实际部署中，Atlas 500 Pro 作为集成的智能边缘服务器，以其丰富的接口和稳定的性能，成为智慧交通、园区管理等场景的常见选择。此外，像 Atlas 300I 这样的 PCIe 加速模块，则为企业现有的工控设备和服务器提供了灵活高效的 AI 能力扩展方案。

这套边缘计算体系的核心优势在于完全的国产化技术栈，通过 CANN 驱动实现了深度的软硬件协同优化，同时与华为云服务形成了良好的端边云协同能力。不过，与英伟达等国际巨头相比，昇腾生态的成熟度和社区资源仍处在追赶阶段。目前，华为 Atlas 系列特别适合应用于安防监控、智慧城市、工业质检等对数据安全和国产化有明确要求的项目场景。

1.3.3. 瑞芯微 - 高性价比的 SoC 芯片专家

瑞芯微作为国内领先的集成电路设计企业，以其高集成度与出色性价比在 AIoT 芯片领域占据重要地位。与提供完整计算卡的厂商不同，瑞芯微专注于核心 SoC 系统级芯片的设计，由下游合作伙伴基于其芯片开发出形态丰富的开发板与整机产品。

在其 AIoT 芯片系列中，RK3588 系列堪称旗舰代表。这款芯片不仅搭载了八核高性能处理器，还集成了算力达 6 TOPS 的自研 NPU，支持多种精度量化方式。特别值得一提的是其卓越的多媒体处理能力，支持 8K 高清解码与多路摄像头输入，使其成为边缘计算盒子、智能 NVR 及商显设备等高端应用的理想选择。

面向主流市场，RK3568 系列则展现出强大的市场号召力。这款芯片在算力与功耗之间取得了良好平衡，集成的 1 TOPS NPU 为各类 AI 应用提供了基础算力保障。目前该芯片已被广泛应用于工业控制、智能门禁、网络存储设备等场景，更是成为了 Firefly、Radxa 等知名开发板平台的核心选择。

总体而言，瑞芯微方案的最大优势在于极致的性价比与高度的集成化——单颗芯片即融合了 CPU、GPU、NPU 及多媒体处理单元。虽然在绝对 AI 算力上无法与英伟达、华为的高端产品直接竞争，其软件生态也仍在持续完善中，但在智能门禁、商显广告机、网络录像机等中低算力需求的 AIoT 应用场景中，瑞芯微无疑提供了一个经济实用且性能均衡的优质选择。

1.4. 昇腾 310B 简介

昇腾 310 是华为昇腾（Ascend）AI 计算产品线的关键入门级芯片，它的发展史与华为全栈 AI 战略的落地紧密相连。

在 2018 年的华为全联接大会上，华为首次发布了昇腾 AI 芯片系列，其中**昇腾 310** 作为面向**边缘和端侧**场景的处理器正式亮相。它的设计目标非常明确：在**极低的功耗（仅 8W）**下，提供足以在边缘端进行实时 AI 推理的算力。这与面向数据中心的昇腾 910（训练芯片）形成了“云边协同”的搭配。

昇腾 310 并非追求极致算力，而是追求**极致的能效比和通用 AI 能力**。它采用了华为自研的达芬奇架构（关于达芬奇架构的 AI Core 计算单元、存储层次等深度解析请参见第 6 章），在一张芯片上集成了 CPU、DVPP（数字视觉预处理）和 AI 计算核心，使其特别适合处理视频、图像等非结构化数据。早期，它被集成在 Atlas 200/300 系列的加速模块中，用于安防摄像头的视频结构化、智慧交通的车辆识别等场景，初步展现了其在实际部署中的价值。

“310B”可以被理解为昇腾 310 的一个**优化和增强版本**。虽然官方没有大肆宣传其细节改进，但业界普遍认为，B 版本在算力、稳定性和可靠性上进行了提升，以更好地满足严苛的工业和商业环境需求。

1.4.1. 昇腾 310B 技术规格详解

华为昇腾 310B 系列提供了两个不同算力等级的 AI 加速模块，分别是 20T 算力版本跟 8T 算力版本。它们在核心架构、接口和外形上高度统一，主要差异在于性能配置。详细的技术规格如下表所示：

规格项	昇腾 310B（20 TOPS）	昇腾 310B（8 TOPS）
AI 算力	20 TOPS INT8 10 TFLOPS FP16	8 TOPS INT8 4 TFLOPS FP16
内存规格	LPDDR4X 12 / 8 / 4 GB 总带宽 51.2 / 34.1 / 34.1 GB/s 支持 ECC	LPDDR4X 4GB 总带宽 25.6 GB/s 支持 ECC
CPU 算力	4 核 × 1.6 GHz	4 核 × 1.0 GHz

规格项	昇腾 310B (20 TOPS)	昇腾 310B (8 TOPS)
编解码能力	解码: H.264/H.265 硬件解码 40 路 1080P 30FPS 4 路 4K (3840×2160) 75FPS 编码: H.264/H.265 硬件编码 20 路 1080P 30FPS 3 路 4K (3840×2160) 50FPS JPEG: 解码 1080P 512FPS 编码 1080P 256FPS 最大分辨率: 16384×16384	解码: H.264/H.265 硬件解码 20 路 1080P 30FPS 2 路 4K (3840×2160) 75FPS 编码: H.264/H.265 硬件编码 12 路 1080P 30FPS 2 路 4K (3840×2160) 50FPS JPEG: 解码 1080P 512FPS 编码 1080P 256FPS 最大分辨率: 16384×16384
高速接口	8 lane, 支持: SGMII (1.25Gbps) 1000BASE-R (1.25Gbps) USB3.0 (5Gbps) SATA 3.0 (6Gbps) PCIe Gen3 (8Gbps) 向下兼容各接口旧版本	同左
低速接口	UART × 5 I2C × 4 SPI × 2 CAN × 4	同左
典型功耗	25W	21W
工作环境温度	-20°C ~ +103°C	同左
结构尺寸	MXM 连接器 82mm × 60mm × 7mm	同左

20 TOPS 版本在整体性能上全面领先, 其 AI 算力 (20 TOPS INT8) 和 CPU 频率 (1.6GHz) 均高于 **8 TOPS 版本** (8 TOPS INT8, 1.0GHz), 并配备了更灵活、带宽更高的内存 (最高 12GB, 51.2GB/s)。在视频处理能力上, 20 TOPS 版本也能支持更多的视频解码与编码路数。

两款产品都集成了丰富的接口并支持宽温工作, 确保了在边缘环境下的连接性和可靠性。**8 TOPS 版本**则以其稍低的 21W 功耗和适中的配置, 提供了一个更具性价比的解决方案。综上所述, 20 TOPS 版本定位为高性能选择, 适用于计算密集型任务; 而 8 TOPS 版本则面向算力需求适中、对成本更敏感的应用场景。

1.4.2. 其他常见的边缘计算卡的对比

华为昇腾 310B、瑞芯微 Rk3588 与英伟达 Jetson Orin Nano 三者之间的定位各有不同。华为昇腾 310B 是一款专为边缘 AI 推理场景设计的人工智能处理器。它不追求极致的通用算力, 而是专注于在特定功耗下提供高效、稳定的 AI 推理能力, 是华为全栈 AI 解决方案在边缘侧的关键载体, 强调自主可控与场景落地。瑞芯微 RK3588 是瑞芯微的旗舰级的高端系统级芯片, 其定位是一个“全能战士”。它集成了强大的 CPU、GPU、NPU 以及顶尖的多媒体处理单元, 旨在为各类 AIoT 设备提供一个高度集成、高性价比的“大脑”, 核心优势在于多功能集成与极致性价比。Jetson Orin Nano 是英伟达面向入门级边缘 AI 市场推出的嵌入式系统模块。它继承了英伟达在 GPU 计算领域的绝对优势, 在小型封装内提供了惊人的 AI 算力, 其核心价值在于强大的性能和无与伦比的成熟软件生态, 是 AI 开发者的首选平台之一。它们之间的详细性能参数

对比如下表所示：

规格类别	昇腾 310B (20T)	Rockchip RK3588	Jetson Orin Nano 8GB
AI 性能	20 TOPS INT8 10 TFLOPS FP16	6 TOPS	67 TOPS
CPU 配置	4 核 ARM × 1.6 GHz	4 核 Cortex-A76 @2.4GHz 4 核 Cortex-A55 @1.8GHz	6 核 Arm Cortex-A78AE @1.7GHz
GPU 配置	-	Mali-G610 MP4 @850MHz	32 个 Tensor Core 的 1024 核 NVIDIA Ampere 架构 GPU
内存	LPDDR4X 12/8/4GB	LPDDR4/LPDDR4x 最大 16GB	LPDDR5 8GB
视频解码	H.265/H.264 8 路 4K@30fps 或者 40 路 1080p@30fps	H.265 8K@60fps 或者 AV1 4K@60fps 或者 VP9 4K@120fps	H.265 4K@60fps 或者 2×4K@30fps 或者 5×1080p@60fps
视频编码	H.265/H.264 3 路 4K@50fps 或者 20 路 1080p@30fps	H.265 4K@60fps 或者 H.264 1080p@60fps	1080p@30fps (CPU 软件编码)
存储接口	eMMC 5.1 SD 3.0 NVMe	eMMC 5.1 SD 3.0 NVMe	支持外部 NVMe
PCIe 接口	PCIe Gen3	PCIe 3.0	PCIe 3.0 (1×4 + 3×1)
USB 接口	USB 3.0	USB 2.0/3.0/3.1	3×USB 3.2 (10Gbps) 3×USB 2.0
网络接口	支持 1GbE/10GbE	支持 1GbE	1×GbE
摄像头接口	MIPI-CSI x2	MIPI-CSI ×4	8 通道 MIPI CSI-2 最多 4 个摄像头
显示输出	HDMI x2	HDMI 2.1/eDP/LVDS 最多 4 路显示输出 8K 分辨率	DP 1.2/HDMI 1.4 4K@30fps
其他 I/O	UART×5, I2C×4 SPI×2, CAN×4	I2S/PCM/SPDIF	UART×3, SPI×2 I2S×2, I2C×4 CAN×1, PWM, GPIO
典型功耗	25W	≤ 15W	7-10W
工作温度	-20°C ~ +103°C	工业级温控支持	-
封装尺寸	MXM 82×60×7mm	BGA 27×27mm	69.6×45mm SO-DIMM 连接器
工艺制程	12nm	8nm	-

在 AI 性能方面, Jetson Orin Nano 凭借其基于 Ampere 架构的 GPU, 提供了三者中最强的 AI 算力; 昇腾 310B 则凭借其专用 AI 加速器, 提供了坚实的中等算力; 而 RK3588 集成的 NPU 则提供了相对基础的 AI 加速能力。

视频处理能力上, 三者各有侧重: RK3588 拥有最强的编解码能力, 原生支持 8K 视频; 昇腾 310B 的优势在于强大的多路视频并发处理能力; 而 Jetson Orin Nano 虽然视频解码能力不俗, 但其编码任务则需要更多地依赖 CPU 进行。

从功耗效率来看, Jetson Orin Nano 的功耗控制最为出色, 展现了优异的能效比; RK3588 则处于中等功耗水平; 相比之下, 昇腾 310B 为了提供更高的算力和视频路数, 功耗也相对较高。

综合来看, 这三款平台因其不同的技术特点, 各自面向的应用场景也有所区分: 昇腾 310B 更适合工业视觉检测和多路视频分析; RK3588 在高清多媒体播放、工业平板及显示设备中表现出色; 而 Jetson Orin Nano 则主要瞄准高性能 AI 推理、边缘计算和机器人等前沿领域。

1.5. 实践任务

1. 基于你的目标场景输出一份协同模式决策表 (含放弃理由)。
2. 编写数据生命周期图 (ASCII 或 Mermaid)。
3. 实现一个队列背压示例: 当处理时延 > 阈值时自动丢弃旧帧。
4. 采集 10 分钟温度与时延数据, 绘制相关性 (是否热导致抖动)。
5. 设计一份故障降级矩阵并评审可行性。

2. CANN 软件栈核心组件解析

本章系统阐述 Ascend CANN 软件栈的分层结构、模型从框架格式到 OM 的转换原理、转换工具 ATC 的关键参数、OM 文件组织结构、AscendCL (ACL) 推理编程模型、精度与性能验证方法以及工程级质量保障流水线建设。

2.1. CANN 异构计算架构

昇腾 AI 异构计算架构 CANN (Compute Architecture for Neural Networks) 是面向基于达芬奇架构的昇腾处理器的全栈软件体系。作为承接主流 AI 框架与通用模型格式、向下对接异构硬件与运行时的中枢层，CANN 构建了覆盖模型表示、转换编译、图优化、调度执行与可观测分析的端到端技术链路，实现语义一致性、性能最优化与诊断可控性的协同统一，从而系统地释放昇腾处理器的算力与能效潜力。

2.1.1. CANN 异构计算与人工智能

异构计算与人工智能是相互促进的关系。深度学习训练与推理负载由高密度线性代数（向量/矩阵乘）、张量算子融合链、非规整控制流（条件/循环）、以及高带宽低延迟的数据搬移共同构成，表现出算子类型多样性、数据复用模式差异与访存/算力不均衡等特征。单一通用处理器在算力利用率、内存层次效率与能效上难以同时达到最优。针对边缘计算场景的典型 SoC（如昇腾 310B、RK3588、Jetson Nano 等）普遍采用低功耗约束下设计的 ARM 通用核，其单核与整体算力在受限功耗窗口内更偏向控制与轻量任务，难以在纯 CPU 路径上满足大规模张量运算所需的高吞吐与低时延深度学习推理需求；因此需要通过集成专用 NPU/加速器进行算子下沉与数据流协同，以弥补通用核在向量化宽度、片上带宽与能效曲线上的结构性不足。异构体系通过在同一系统中组合通用 CPU 与专用 NPU，辅以分层内存与高吞吐互连，依据算子粒度、数据局部性与精度需求执行跨设备调度：密集算子下沉至矩阵/张量专用单元，控制与调度逻辑保留在通用核，数据流经由优化的流水与对齐策略减少跨域搬运成本。结果是在单位功耗与芯片面积约束下提升有效吞吐、降低端到端推理时延并改进能效曲线的可扩展性。该协同范式构成 CANN 设计的理论基础：通过抽象统一语义表示与针对性后端优化，将模型图中不同类别算子映射到最合适执行单元，实现性能、能效与可诊断性三者的统筹平衡。

2.1.2. CANN 与 CUDA

CANN 是面向华为昇腾达芬奇架构昇腾处理器，聚焦深度学习推理与训练的算子图优化、编译调度及可观测性闭环，而 CUDA (Compute Unified Device Architecture) 则是针对 NVIDIA GPU 的通用并行计算平台与编程模型，覆盖 GPGPU 与 AI 复合场景。二者均体现“软硬件协同加速”范式，但在抽象层次、硬件耦合与生态策略上形成差异化路径。CANN 与 CUDA 二者的共同点主要体现在，它们都是通过软硬件协同，把模型或并行程序转化为底层设备能理解的指令，从而提升运行速度和效率。同时这两种计算平台都提供了内存管理、算子或内核调用、性能分析、调试和日志等工具，帮助开发者更方便地开发和诊断问题。它们都支持自定义算子或内核插件，方便针对特定需求进行优化或替换实现。此外，CANN 与 CUDA 都能通过性能分析、数据导出和分级日志等方式，降低了定位性能瓶颈和对齐精度的难度。在设计理念上，CANN 与 CUDA 展现出显著的差异。CUDA 作为面向广义并行计算与 HPC/图形及 AI 统一生态的平台，强调线程块与网格 (Block/Grid) 以及 SIMT 并行范式，开发者需显式组织核函数、流 (Stream)、事件 (Event) 和内存管理，实现灵活的并行与资源重叠。而 CANN 则专注于模型图语义，突出图级算子融合、AIPP 预处理下沉和全局内存复用，通过任务列表与算子调度策略有效治理动态形状与内存复用，降低推理工程的不确定性。在精度策略方面，CANN 内置 FP16 与 INT8 的混合精度及校准链路，适应端到端推理与训练的工程质量要求；而 CUDA 虽然基础类型丰富，但精度管理更多依赖于上层库如 cuDNN 和 TensorRT 的组合。硬件结构上，CANN 紧密耦合于达芬奇架构的矩阵单元、向量单元及片上分层内存，优化模型推理的能效与性能；CUDA 则绑定于 NVIDIA 的 SM、Tensor Core 及多级缓存体系，着重提升访存效率与线程调度。生态成熟度上，CUDA 自 2007 年以来积累了庞大的社区与库支持，形成了全球主流的开发生态；CANN 虽处于快速迭代阶段，但聚焦国产化替代与特定行业应用，推动本地生态建设。编程灵活度方面，CUDA 需要开发者具备底层并行划分与核函数开发能力，提供极高的灵活性；CANN 则通过贴近主流框架的算子开发工具 (TBE) 与高层接口，降低了入门门槛，但在底层调度上牺牲了一定的自由度。整体而言，二者在抽象层次、硬件耦合与生态策略上形成了各自鲜明的技术路径，反映了其服务对象与应用场景的根本差异。

2.1.3. CANN 的架构

CANN 通过提供分层清晰的编程与运行时结构，以覆盖训练与推理的“全场景”、贴近主流框架的“低门槛”以及面向昇腾硬件深度优化的“高性能”为核心特性，支撑用户在昇腾平台上快速构建与部署各类 AI 应用与业务。从整体上看，CANN 可以抽象为一个自上而下递进的五层架构 (计算语言接口、计算服务层、计算编译引擎、计算执行引擎与计算基础层)，如图 2.1 所示。各层通过稳定的接口与数据流协议进行解耦协同，在保证语义一致性的前提下最大化发挥底层算力与能效潜力。

在分层结构上，CANN 可抽象为五个层次：上层的计算语言接口——AscendCL 接口负责设备与上下文管理、流与内存控制、模型与算子的加载执行，以及媒体与图管理等通用 API，为应用提供稳定的编程入口；其下的昇腾计算服务层汇聚神经网络与线性代数库，承载算子与子

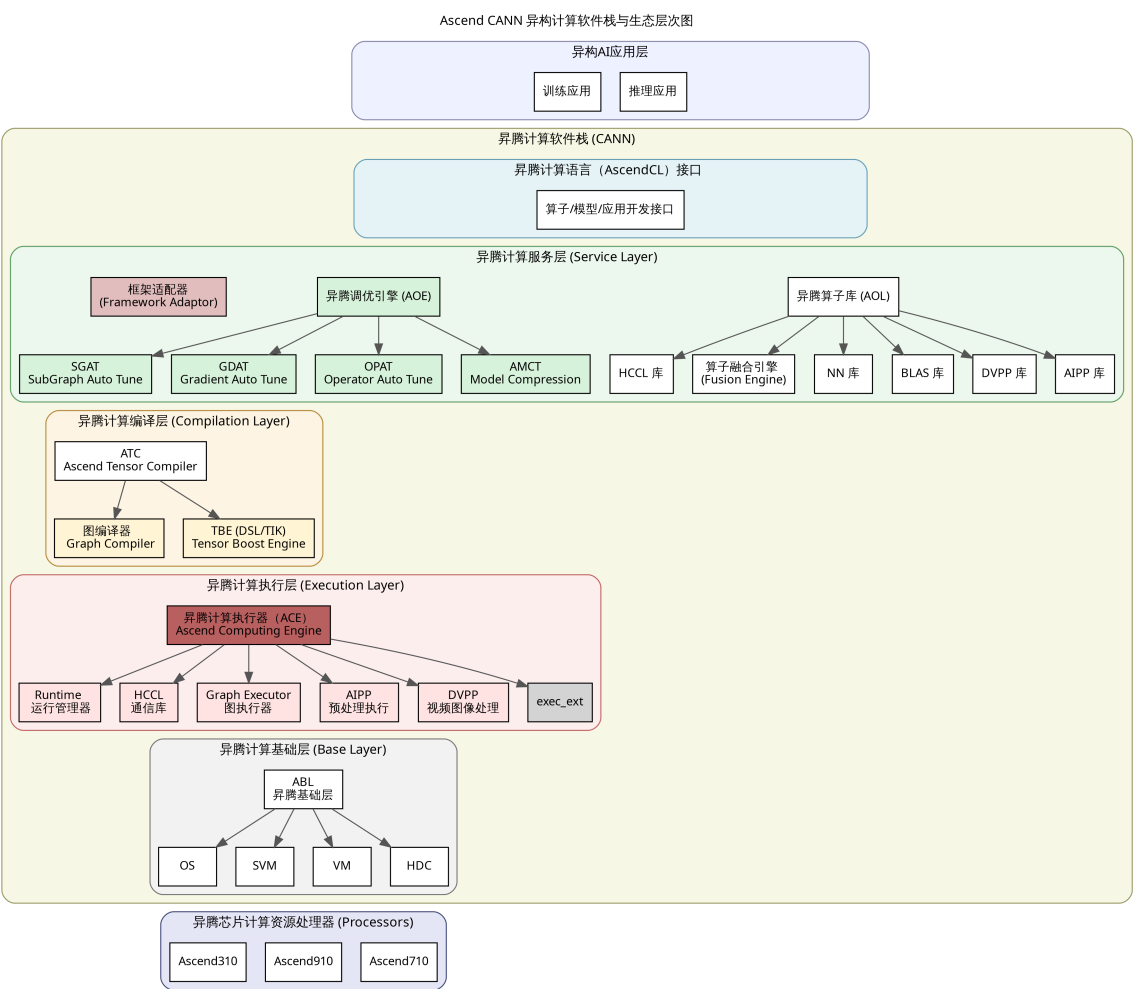


Figure 2.1.: CANN 架构图

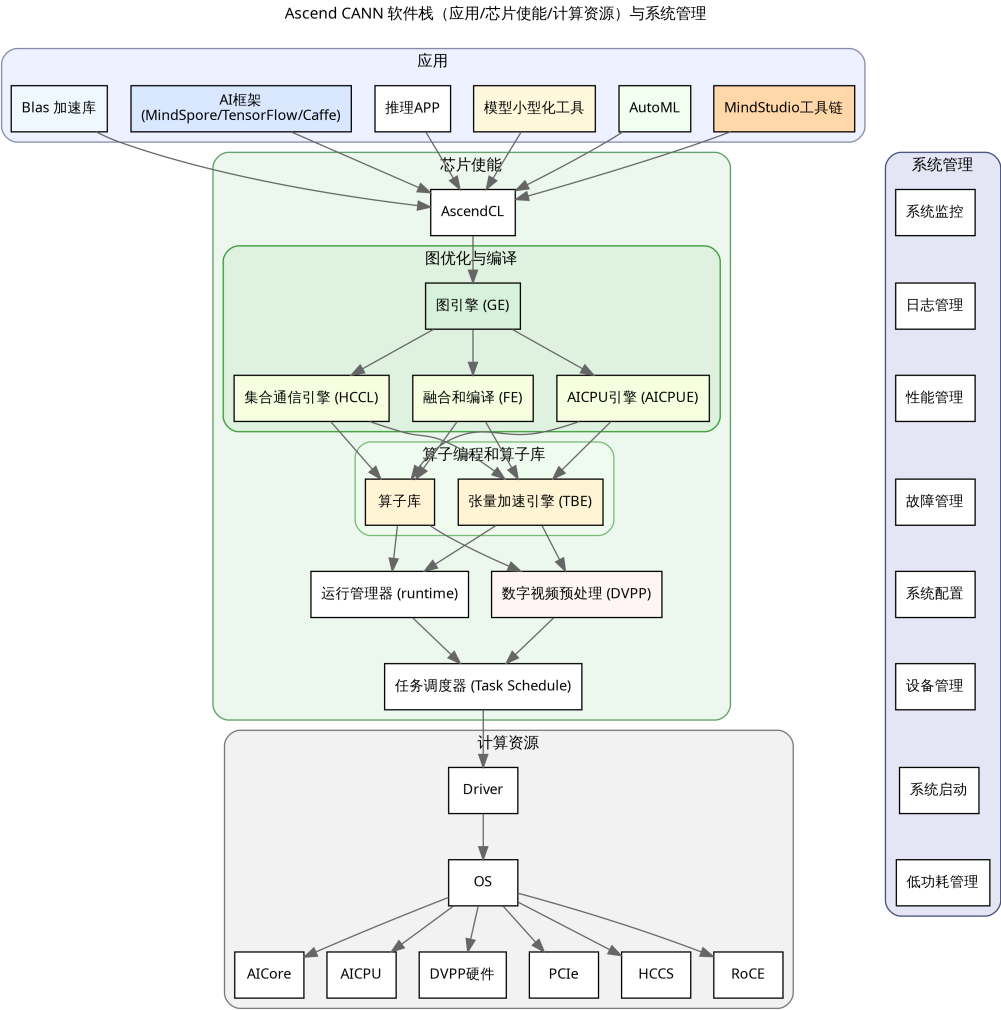


Figure 2.2.: CANN 逻辑图

图的自动调优、梯度优化与模型压缩，并通过框架适配器降低迁移成本；昇腾计算编译层的编译引擎通过图编译器与 TBE 算子开发支持，把前端计算图转化为在 NPU 可执行的模型与内核，实现图级语义到后端实现的精准映射；下一层的昇腾执行引擎面向运行时，负责模型与算子的调度执行，并内置数字视觉与 AI 预处理、集合通信等能力以提升端到端效率；最下层的昇腾计算基础层提供共享虚拟内存、设备虚拟化与主机—设备通信等底座服务，保证跨设备数据流与资源管理的可靠性与可扩展性。

与此相辅的是三层逻辑架构：应用层承载具体业务与开发者工具，芯片使能层开放解决方案能力并驱动基于计算图的业务流运行，计算资源层则聚焦数据处理与运算执行，形成从业务到硬件的清晰闭环。这个 CANN 的三层逻辑结构如下图2.2所示：

在技术特性上，CANN 通过对计算图的编译与优化，将密集算子与数据流合理分配到异构单元上执行，显著提升吞吐与时延表现，并在能效上取得可工程化的优势；同时提供贴近主流

框架的易用接口与完善的工具链，降低入门与迁移门槛，使开发者能够快速完成部署与调优。生态方面，CANN 构建了面向个人、高校、科研与企业的赋能体系，并与开源社区协同，支持多种框架与推理引擎在异构硬件上的高效运行。依托这些能力，开发者可以深入调用运行时与图编译能力，释放底层硬件潜力，在性能与成本维度形成差异化竞争力。

在工程实践中，CANN 的核心价值可以概括为在“模型—编译—执行—诊断”的完整链路上实现软硬件协同与语义稳定：通过构建对主流深度学习框架高度兼容的前端接口，最大限度降低模型迁移与语义对齐成本，在图级优化阶段则依托算子融合、内存复用以及混合精度策略的协同设计，系统性提升吞吐能力、推理时延以及功耗效率；与此同时，借助自定义算子机制与实现优先级调度策略，使新结构能够快速接入并在多实现版本之间灵活切换，从而在不同硬件代际与不同业务场景下充分挖掘底层计算资源的潜力，并在运行时通过分级日志、Profiling 时间线与精度 Dump 等手段构建起闭环的可观测体系。在大规模、复杂模型的工程化路径中，首先需要完成面向硬件架构的感知建模，在导出阶段显式固化张量形状与算子语义，为后续编译与部署奠定稳定的语义基础；随后在模型转换阶段，以“转换即优化”为原则，围绕 precision_mode、op_select_implmode 与 AIPP 等关键配置项进行显式调优，使图优化与算子选型在编译期前置完成，其中精度策略以 FP16 为主，并辅以具有代表性的校准数据集对 INT8 量化误差进行严格控制，以在性能与精度之间取得可工程化的平衡。对于动态形状问题，可以采用分桶与 Padding 结合的方式，并在必要时生成多份 OM 模型，以在一定范围内换取更可控的峰值资源占用与时延方差；在内存与带宽层面，则通过将预处理逻辑尽可能下沉到设备侧、合并数据搬移操作，并配合 Pinned 内存与多 Stream 并行传输技术，显著降低 H2D/D2H 的时间占比，从系统视角优化整体流水线的调度效率。针对在实际 Profiling 中暴露出的性能瓶颈算子，需要开展有针对性的算子重写与图重构，并充分利用实现优先级机制在不同 Kernel 之间进行策略化选择，以进一步榨取底层硬件的计算与访存潜力；最终，通过围绕“转换—精度对齐—Benchmark—回归监测”构建自动化流水线，将模型转换、性能评估与精度验证纳入统一的持续反馈闭环，在 Ascend 310B 等专用加速平台上沉淀出可复用的优化经验与差异化性能优势，不仅在算力利用率与综合成本上形成工程级竞争力，也为大模型与行业应用场景的规模化加速落地提供坚实的基础设施支撑。

2.1.4. CANN 的快速安装

本节给出 CANN 软件的极速安装示例。若需更完整的操作指导，请按实际环境选择对应安装场景并参考[官方说明](#)，推荐优先使用离线安装。

安装前准备

- 驱动与固件检查：在目标设备上执行 `npu-smi info`，如果能够正常显示设备信息，则表明 NPU 驱动和固件已经正确安装。对于昇腾 310B 产品或开发板，系统一般会预装所需的驱动和固件，无需额外手动配置。
- 环境准备：推荐采用离线安装方式，安装前需确保系统已准备好 Python 环境并可使用

pip3, 当前 CANN 支持 Python 3.7.x 至 3.11.4 版本。对于昇腾 310B 开发板, 通常已预装 Miniconda 和 pip3, 可直接满足依赖, 无需额外配置。

- 安装介质获取: 在安装前, 需先从华为昇腾官网下载所需的 CANN 软件包, 并将其上传到目标设备的可访问路径。以 CANN 8.3.RC1 版本为例, 可通过[官网](#) 获取相应安装包, 其他版本可在页面切换选择。请确保下载的 CANN Toolkit 和 CANN Kernels 版本号完全一致 (即同一发行号), 以避免兼容性问题。推荐优先选择.run 格式的安装包, 且昇腾 310B 开发板为 AArch64 架构, 需下载 aarch64 对应的包。例如 8.3.RC1 版本包括 Ascend-cann-toolkit_8.3.RC1_linux-aarch64.run 和 Ascend-cann-kernels-310b_8.3.RC1_linux-aarch64.run。如果实际下载的版本号不同, 请将文件名中的版本号替换为对应的实际版本。

安装命令与依赖说明

安装 CANN 可使用 root 或默认账户 HwHiAiUser。推荐以 root 安装, 不推荐用其他用户进行安装, 因为容易发生权限问题。下面以 root 用户为例, 详细介绍 CANN 的安装过程:

- 切换到 root:

```
1 su -
2 # 输入 root 密码后继续执行安装
```

- 命令示例, 以 root 执行:

```
1 ./Ascend-cann-toolkit_8.3.RC1_linux-aarch64.run --install
```

- 安装 Toolkit 开发套件:

```
1 chmod +x Ascend-cann-toolkit_8.3.RC1_linux-aarch64.run
2 ./Ascend-cann-toolkit_8.3.RC1_linux-aarch64.run --install
```

整个安装过程大概会持续十几分钟, 甚至更长, 请耐心等待。

- 配置环境变量:

```
1 source /usr/local/Ascend/ascend-toolkit/set_env.sh
```

- 安装 Kernels 算子包 (310B 平台):

```
1 chmod +x Ascend-cann-kernels-310b_8.3.RC1_linux-aarch64.run
2 ./Ascend-cann-kernels-310b_8.3.RC1_linux-aarch64.run --install
```

- 安装业务运行时所需的 Python 第三方库:

```
1 pip3 install attrs cython 'numpy>=1.19.2,<=1.24.0' \
2 decorator sympy cffi pyyaml pathlib2 \
3 psutil protobuf==3.20.0 scipy requests absl-py
```

- 说明: 依赖的版本范围已经覆盖了大多数常见环境, 如果遇到依赖冲突, 请根据实际的 Python 或操作系统版本进行相应调整。安装完成后, 建议将 CANN 的环境变量配置追

加到 `~/.bashrc` 文件中，以便每次登录时自动加载。需要注意的是，昇腾 310B 开发板的系统镜像通常已经预置了这些环境变量配置，无需手动设置。

2.2. ATC 模型转换详解

2.2.1. ATC 工具介绍

昇腾张量编译器（Ascend Tensor Compiler, ATC）是 CANN 异构计算体系中的模型转换组件，支持将主流开源框架导出的网络模型以及基于 Ascend IR 的单算子描述文件（JSON）编译为昇腾 AI 处理器可执行的离线模型（.om）。如图下图所示，ATC 在转换过程中会执行算子调度优化、权重重排与内存复用等关键步骤，对原始深度学习模型进行面向部署场景的系统化调优，从而在昇腾硬件上实现高吞吐、低时延的高效执行。

从上面的流程图我们可以看到，ATC 工具聚焦模型到设备可执行体的“转换即优化”闭环：一方面，它能够将开源框架导出的网络模型解析为中间态 IR Graph，经由图准备、拆分与融合、形状推理、内存复用与算子选型等编译步骤，生成适配昇腾处理器的离线模型（OM），并在板端通过 AscendCL 接口加载执行，从语义到性能实现端到端落地；另一方面，ATC 也支持基于 Ascend IR 的单算子 JSON 直接编译为离线 OM，用于在设备侧进行算子级功能验证与精度对齐，帮助开发者快速定位与迭代关键算子实现，在图级与算子级两条路径上形成统一的工程化编译能力。

onnx 格式介绍

ONNX（Open Neural Network Exchange）是一种针对深度学习所设计的开放式文件格式，用于存储训练好的模型。它使得不同的人工智能框架（如 PyTorch、TensorFlow、mindspore 等）可以采用相同格式存储模型数据并交互。ONNX 的规范包含计算图模型的定义，以及内置运算符的定义和标准数据类型。

ONNX 的核心价值在于解决了 AI 生态系统中“碎片化”的问题。在没有 ONNX 之前，开发者如果想将一个在 PyTorch 中训练好的模型部署到移动端推理引擎上，通常需要进行繁琐的代码重写或复杂的格式转换。ONNX 提供了一种中间表达（Intermediate Representation, IR），充当了不同框架之间的“通用语”。

一个标准的 ONNX 模型文件（通常以 .onnx 为后缀）主要包含以下几个部分：

1. **Model Proto**: 顶层结构, 包含模型的元数据(如版本信息、生产者信息)和计算图(Graph)。
2. **Graph Proto**: 描述了模型的计算逻辑, 由一系列节点(Node)、输入(Input)、输出(Output)和初始化器(Initializer, 即权重数据)组成。
3. **Node Proto**: 代表计算图中的一个操作(Operator), 如 Conv、Relu、Add 等。每个节点包含操作类型、输入输出张量的名称以及属性(Attribute, 如卷积的步长、填充等)。
4. **Tensor Proto**: 用于存储权重数据或常量的具体数值和形状信息。

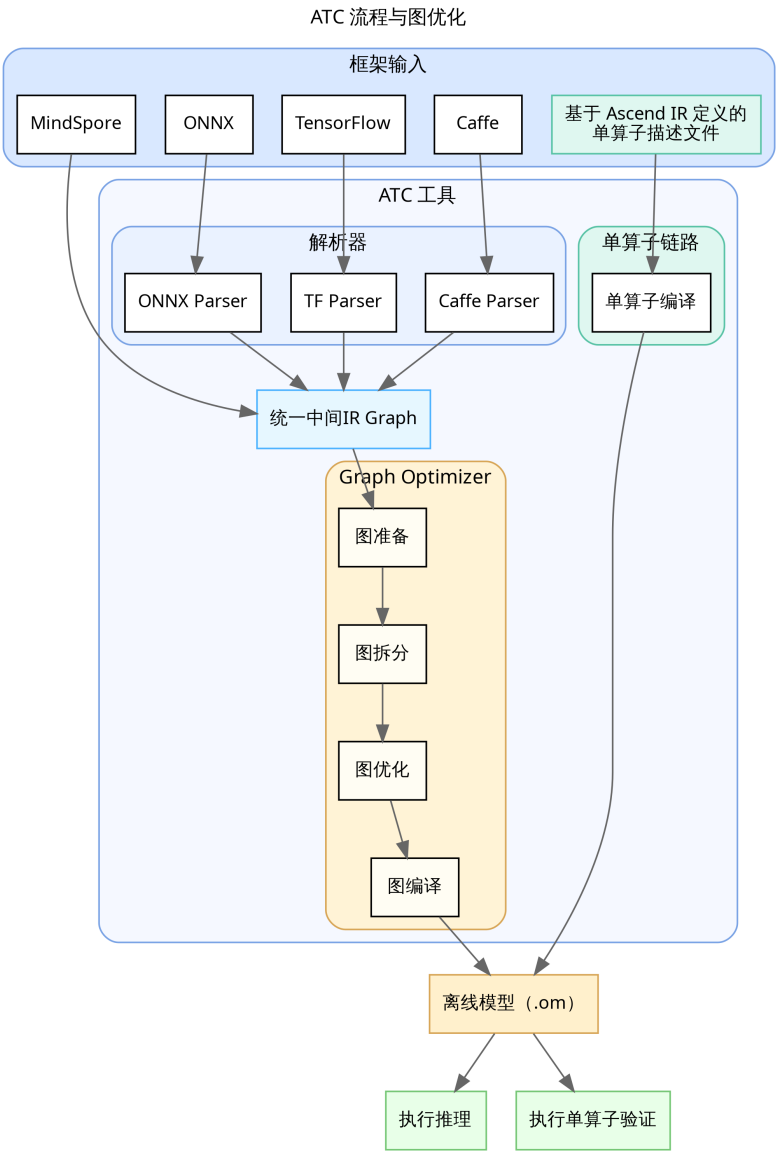


Figure 2.3.: ATC 框架图

ATC 对 ONNX 的支持情况虽然 ONNX 旨在成为通用的 AI 模型交换标准，但 CANN 的 ATC 工具并非支持 ONNX 规范中的所有算子和特性。ATC 对 ONNX 的支持主要集中在 CV（计算机视觉）和 NLP（自然语言处理）领域的常用算子，对于一些较新或较冷门的算子，可能会出现不支持的情况。

具体来说，ATC 对 ONNX 的支持限制主要体现在以下几个方面：

1. **算子覆盖率**：ATC 支持绝大多数主流的 CNN 和 Transformer 类算子（如 Conv, MatMul, Relu, Softmax 等）。但对于部分非标准或特定框架自定义的算子（Custom Ops），ATC 无法直接解析，需要开发者通过 TBE（Tensor Boost Engine）开发自定义算子并注册到 CANN 中。
2. **动态控制流**：ONNX 中的动态控制流（如 If, Loop, Scan）在静态图编译中处理较为复杂。虽然 ATC 支持部分控制流算子，但在某些复杂嵌套场景下可能会导致编译失败或性能下降，通常建议在导出模型时尽量将控制流展开（Unroll）或转为静态图结构。
3. **数据类型限制**：虽然 ONNX 支持多种数据类型（如 Double, Int64 等），但昇腾 AI 处理器主要针对 FP16 和 INT8 进行了硬件加速优化。对于 FP64（Double）类型，ATC 通常不支持或需要强制转为 FP32/FP16 处理；对于 Int64 类型的索引或计数，部分算子可能要求转为 Int32。
4. **动态 Shape 支持**：虽然 ATC 支持动态 Batch 和动态分辨率，但对于模型内部中间层出现的完全动态且无法推导的 Shape，可能会导致编译报错。

因此，在进行模型转换前，建议开发者查阅华为昇腾社区发布的《CANN 算子清单》，确认模型中使用的 ONNX 算子版本（Opset Version）是否在当前 CANN 版本的支持范围内。如果遇到不支持的算子，通常的解决路径包括：修改模型结构以替换为等价的支持算子、在 ONNX 导出阶段进行算子简化（使用 onnx-simplifier 工具）、或者开发自定义算子。

2.2.2. ATC 工具使用流程

使用 ATC 工具进行模型转换的使用流程如下图所示：

在开始模型转换之前，首先需要在开发环境中安装与 CANN 软件包版本相匹配的版本，并确保 ATC 可执行文件的路径可用。接下来，准备待转换的模型文件或基于 Ascend IR 的单算子 JSON，并将其上传到开发环境中可访问的目录。最后，使用 ATC 执行转换，并根据实际业务需求和输入规范配置相关参数。如果需要将预处理步骤（如色彩空间转换、归一化和尺度调整）下沉到设备侧，可以同时提供并启用 AIPP（Artificial Intelligence Pre-Processing）配置。AIPP 是昇腾处理器内置的硬件级图像预处理模块，负责将上游（如 DVPP）输出的对齐后 YUV420SP 图像在设备侧完成色域转换（如 YUV 图像格式转换为 RGB 或者 BGR 图像格式）、归一化（减均值/乘系数/尺度放大）与抠图（在指定起始点裁剪到模型输入尺寸），从而把原始图像规范化为模型所需的输入格式与数值范围。由于昇腾 310B 在推理及训练中常以 DVPP 输出 YUV420SP 图片格式，这种图像格式是有损图像颜色编码格式，常用为 YUV420SP_UV、YUV420SP_VU 两种格式，不直接提供 RGB 图片。AIPP 能将 YUV420SP 类型图像无缝转化为模型期望的 RGB/BGR

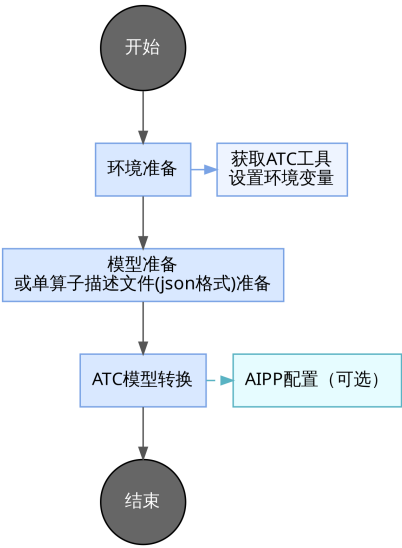


Figure 2.4.: ATC 流程图

图像格式，并在同一数据流中完成裁剪与数值处理，避免将预处理放在 CPU 侧导致的多次拷贝与额外时延，提升端到端吞吐与能效。

模型转换-以 ResNet50 为例

ResNet-50 由多级残差单元堆叠形成 50 层卷积网络,通过跨层跳连缓解深层退化与梯度消失,使其在 ImageNet 等大规模数据集上具备稳定的特征表达与分类精度，因此常被选作各类视觉任务的通用骨干。要在昇腾 310B 上部署该模型,需要借助 ATC 将 ONNX 版本转换为 OM,可参考华为昇腾官方示例仓库(<https://github.com/Ascend/samples/tree/master/inference/modelInference/sampleResNet50>)。为方便快速复现，可按以下步骤从零搭建工程：

- 1. 在昇腾 310B 的 Documents 目录创建 sample_resnet_quick_start, 并划分 data、model、out、src 四个子目录用于存放测试图片、模型、脚本与源码：

```
1 cd ~/Documents
2 mkdir -p sample_resnet_quick_start/{data,model,out,src}
3 cd sample_resnet_quick_start
```

- 2. 下载示例图片至 data 目录：

```
1 cd ~/Documents/sample_resnet_quick_start/data
2 wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/aclsample/
   ↪ dog1_1024_683.jpg
```

- 3. 进入 model 目录获取 ResNet-50 ONNX 模型：

```
1 cd ~/Documents/sample_resnet_quick_start/model
2 wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/003_Atc_Models/resnet50/
   ↪ resnet50.onnx
```

4. 设置编译并行度后执行典型 ATC 命令，生成适配昇腾 310B4 的 OM 文件：

```
1 export TE_PARALLEL_COMPILER=1
2 export MAX_COMPILE_CORE_NUMBER=1
3 atc \
4   --model=resnet50.onnx \
5   --framework=5 \
6   --output=resnet50 \
7   --input_shape="actual_input_1:1,3,224,224" \
8   --soc_version=Ascend310B4
```

关键参数说明

参数	作用	注意事项
--framework	指定原始模型框架	0=Caffe, 1=MindSpore, 3=TensorFlow, 5=ONNX, 需与导出框架一致
--input_shape	定义静态输入 Shape	按 "name:n,c,h,w" 写法，字符串需加引号；动态改用区间或配合动态参数
--dynamic_batch_size	设置可选 Batch 列表	以 "1,4,8" 形式填写；与同一输入的完全静态 shape 互斥
--dynamic_image_size	配置多分辨率输入	"h1,w1;h2,w2"，常用于多尺度检测；需与模型实际支持一致
--precision_mode	控制混合精度策略	常用值：force_fp16、allow_mix_precision、allow_fp32_to_fp16，需兼顾精度
--soc_version	选择目标芯片	例：Ascend310B4，可通过 npu-smi info 查询；310B/P 区分清楚
--insert_op_conf	下沉 AIPP/自定义算子	指定 JSON/YAML，支持色域转换、归一化等预处理
--op_select_implmode	算子实现优先级	支持 high_performance(默认)、high_precision 等，可配合 --optypelist_for_implmode
--input_format	声明输入数据排布	如 NCHW、NHWC；需与模型导出及 --input_shape 保持一致

参数	作用	注意事项
--output_type	重指定输出 dtype	可设全局 FP16/FP32，或按 node:idx:FP32 精细配置，便于后处理
--enable_small_channel	小通道优化开关	取值 0/1，轻量网络或低通道数场景启用可减时延
--model	指定待转换模型文件	支持 .onnx、.pb、.prototxt 等类型，路径需可访问
--output	设置 OM 输出前缀	不需写后缀；默认位置在当前目录，可结合 --output_type 使用
--dynamic_dims	定义多维动态组合	"d1_1,d1_2;d2_1,d2_2" 格式；用于多输入或非 Batch 维变动
--enable_single_stream	强制单流执行	true/false；在推理序列化或调试稳定性时启用
--dump_mode	导出模型结构 JSON	配合 --mode=1 使用，1 开启；便于排查 Shape 与算子信息

如果对于参数有任何的疑惑，可以通过命令查询 atc 的参数作用：

```
1 atc --help
```

成果转换模型后，我们可以看到命令窗口有如下输出：

```
1 ATC start working now, please wait for a moment.
2 ...
3 ATC run success, welcome to the next use.
```

注意事项

- 1. 在昇腾 310B 上将 ONNX 模型转换为 OM 时若出现 BrokenPipeError: [Errno 32] Broken pipe，通常是编译阶段内存不足导致进程被系统终止。可参照以下两类方案缓解：
 - 扩展可用内存通过创建 Swap 交换文件临时扩充虚拟内存，以缓解编译期间的内存压力：

```
1 dd if=/dev/zero of=/swapfile bs=1M count=8192
2 chmod 600 /swapfile
3 mkswap /swapfile
4 swapon /swapfile
```

若需永久生效，请在 /etc/fstab 文件中追加以下配置：

```
1 /swapfile swap swap defaults 0 0
```

注意：如果设备使用 TF (MicroSD) 卡作为主要存储介质，不建议在 TF 卡上频繁使用 Swap。Swap 分区涉及大量高频读写操作，会极大加速闪存磨损，导致 TF 卡寿命缩短或损坏。建议仅在必要时临时开启，或使用 SSD 等更耐用的存储介质。

• 降低编译资源占用

```
1 export TE_PARALLEL_COMPILER=1      # 限制算子最大并行编译进程数
2 export MAX_COMPILE_CORE_NUMBER=1  # 限制图编译占用的 CPU 核数
```

方便起见，我们可以采用第二种方案，可以有效解决内存不足的问题。

- 2. 若无法确认当前设备的 soc_version，可在安装驱动的昇腾 310B 上执行 npu-smi info 查询，示例如下：

```
1 npu-smi 23.0.0 Version: 23.0.0
2 +-----+-----+-----+-----+-----+
3 | NPU Name | Health | Power | Temp | Hugepages-Usage |
4 | Chip    | Bus-Id | AICore | Mem  |                  |
5 +-----+-----+-----+-----+-----+
6 | 0 310B4 | Alarm  | 0.0W  | 58C  | 15 / 15          |
7 | 0 0     | NA     | 0%    | 2500/7545MB |
8 +-----+-----+-----+-----+-----+
```

请在查询结果的 Name 值前追加 Ascend 前缀：若 Name 为 310B4，则需配置 soc_version=Ascend310B4。

- 3. 模型转换完成后会生成一份 JSON 文件，用作 ATC 编译日志的结构化摘要，集中记录当前会话 (session_and_graph_id=0_0) 内图级与 UB 级融合规则的触发统计。具体而言，graph_fusion 字段逐条给出各图融合 Pass 的匹配次数与实际生效次数 (match_times 与 effect_times)，可据此识别潜在的优化空档；ub_fusion 字段在上述统计基础上新增 repository_hit_times，用以衡量算子库模板的命中情况。实践中，可重点关注 effect_times 低于 match_times 的 Pass，分析是否因算子属性、精度模式或实现优先级等因素造成融合未落地，并据此调整 ATC 参数或模型图结构，以复现并提升性能优化效果。

2.2.3. 模型推理工具 msame 工具概览

在完成 ATC 转换得到 .om 模型后，最快验证部署链路是否畅通的方式就是调用华为昇腾提供的模型推理工具 msame。该工具封装了 OM 加载、设备内存初始化、输入二进制数据映射以及推理调度等通用流程，无需额外编写 AscendCL 程序即可直接对接 ATC 输出的模型，只需准备好与模型输入规格匹配的 .bin 文件和基础运行时环境，即可在命令行下完成端到端推理验证。通过 msame，开发者能够快速检查模型是否成功落地、输出是否合理，并以最小成本排除输入格式或环境配置类问题，为后续集成到自研应用或高层框架之前提供低门槛的功能性回归手段。

msame 的设计目标是将 OM 模型的快速验证能力标准化，覆盖单输入、多输入和循环执行等常见推理形态，支持在相同模型与不同输入数据组合之间做对比试验，同时允许选择 TXT 或 BIN 格式导出结果，以便于人工审查或自动化脚本解析。在已正确安装 CANN Runtime 的

环境中，msame 会自动完成与设备的上下文绑定、内存申请和任务提交，将推理过程抽象成简单的命令行参数；这不仅适用于初步确认模型可运行，也可用于小批量性能评估、数据一致性校验以及在定位性能或精度问题时的基准对照，从而在模型转换与实际业务部署之间搭建起轻量而高效的桥梁。

msame 工具的获取与构建

- 1. 通过 git clone https://gitee.com/ascend/tools.git 拉取仓库，或下载 ZIP 包后解压到开发环境。
- 2. 进入 msame 子目录：

```
1 cd ./tools/msame
```

- 3. 配置 CANN 环境变量（以下路径请按实际安装位置调整）：

```
1 export DDK_PATH=/usr/local/Ascend/ascend-toolkit/latest
2 export NPU_HOST_LIB=/usr/local/Ascend/ascend-toolkit/latest/runtime/lib64/stub
```

- 4. 运行构建脚本生成可执行文件：

```
1 # 若脚本缺少执行权限，请先执行 chmod +x build.sh
2 ./build.sh g++
```

如果在执行时遇到 Permission denied 报错，说明脚本文件缺少可执行权限，需先运行 chmod +x build.sh 进行赋权，随后再次执行编译命令。

- 5. 构建成功后会在 out 目录生成二进制文件 msame,将其复制到工程目录 sample_resnet_quick_start 中备用。

• 常用参数速览

参数	说明
--model	OM 文件路径
--input	输入 bin 文件或目录，支持多输入
--output	推理结果存放目录
--outfmt	输出格式：TXT / BIN
--loop	重复推理次数（1-100）
--profiler / --dump	性能分析或 Dump，互斥
--device	设备 ID，默认 0
--dymBatch / --dymHW / --dymDims /	对应 ATC 动态配置的实际取值
--dymShape	
--outputSize	动态 Shape 场景下手动申请输出内存



Figure 2.5.: resnet 测试图片

参数	说明
--debug	打印模型描述信息
--help	查看全部参数

- 使用注意运行 msame 工具时，需确保当前账号拥有目录的执行和写入权限。对于部分动态 Shape 场景，建议选择 BIN 格式输出而非 TXT，以保证兼容性。配置 NPU_HOST_LIB 时应指向 runtime/lib64/stub 目录，并在运行阶段通过 LD_LIBRARY_PATH 正确链接所需依赖库。此外，profiler 和 dump 功能不可同时启用；如涉及动态 Shape，需同步设置合适的输出内存大小以确保推理顺利进行。

图片预处理

由于 msame 工具不具备图像预处理能力，输入必须是已转换好的 .bin 文件。因为之前在模型转换时未显式指定精度，ResNet50 默认接受 FP32 格式的输入；同样地，未设置输入内存布局时 ATC 会采用默认的 NCHW。NCHW 表示批次 N、通道 C、空间高度 H 与宽度 W，意味着同一张图片的三个颜色通道会按通道优先的顺序依次排布，再展开到空间维度。这与 TensorFlow 常见的 NHWC（通道在最后）相对。sampleResnetQuickStart.py 中的 image_rgb.transpose([2, 0, 1]) 正是将 OpenCV 读取的 HWC 布局转换为 CHW，并配合外层批次维得到符合要求的 NCHW。是否必须使用 NCHW 取决于模型导出时的约定。PyTorch 与 CANN 默认使用 NCHW，因此常见的 resnet50.onnx/resnet50.om 都在此布局下训练与推理。只要导出模型为 NCHW，推理阶段同样必须以 NCHW 喂入数据，否则通道错位会导致结果异常。当然，也可以导出为 NHWC，只需在模型与预处理两端同时调整即可。由于刚才我们从华为云哪里获取的测试图片 “dog1_1024_683.jpg” 如下：

由于这个图片是 jpg 格式的，因此为了可以利用这个图片推理，我们第一步需要对图片进行预处理,具体过程如何:1. 在 src 文件目录创建一个预处理的 python 文件 “make_bin_resnet224_float32.py” 如下：

```

1 from PIL import Image
2 import numpy as np
3
4 # 加载图像并转换为 RGB
5 img = Image.open('data/dog1_1024_683.jpg').convert('RGB')
6 # 使用双线性插值缩放到 256x256
7 img = img.resize((256, 256), Image.BILINEAR)
8 # 计算 224x224 中心裁剪的坐标
9 left = (256 - 224) // 2
10 top = (256 - 224) // 2
11 # 执行中心裁剪
12 img = img.crop((left, top, left + 224, top + 224))
13
14 # 转换为 [0, 1] 范围的 float32 数组
15 arr = np.array(img).astype(np.float32) / 255.0
16 # 定义归一化常量
17 mean = np.array([0.485, 0.456, 0.406], dtype=np.float32)
18 std = np.array([0.229, 0.224, 0.225], dtype=np.float32)
19 # 按通道进行归一化
20 arr = (arr - mean) / std
21
22 # 调整为 NCHW 格式并添加批次维度
23 arr = arr.transpose(2, 0, 1)[None, :, :, :]
24 # 保存为二进制文件
25 arr.tofile('data/dog1_224_float32.bin')
26 # 打印缓冲区大小 (字节数)
27 print('bytes:', arr.nbytes)

```

2. 在 shell 中运行这个 python 文件:

```
1 python src/make_bin_resnet224_float32.py
```

该脚本在导出 **.bin** 输入前会完成: 将示例图像缩放到 256×256 后再做 224×224 中心裁剪; 把像素转换为 NCHW 排布的 float32 缓冲区并写入文件; 整个过程中先除以 255 将数值归一化到 [0,1], 再按通道减去 [0.485, 0.456, 0.406]、除以 [0.229, 0.224, 0.225] 标准差, 以对齐 ResNet 在 ImageNet 上的训练预处理, 从而避免因均值或方差失配导致的输出偏移, 最终在 data 文件夹内得到 dog1_224_float32.**bin**。

3. 快速运行——单输入文件运行下面这个命令:

```
1 ./msame --model "./model/resnet50.om" --input "./data/dog1_224_float32.bin" --
   ↪ output "./out" --outfmt BIN
```

运行成功后, 我们会得到以下的信息:

```

1 [INFO] acl init success
2 [INFO] open device 0 success
3 [INFO] create context success
4 [INFO] create stream success
5 [INFO] get run mode success
6 [INFO] load model ./model/resnet50.om success
7 [INFO] create model description success
8 [INFO] get input dynamic gear count success

```

```

9 [INFO] create model output success
10 ./out/20251213_11_6_52_564388
11 [INFO] start to process file:./data/dog1_224_float32.bin
12 [INFO] model execute success
13 Inference time: 6.811ms
14 [INFO] get max dynamic batch size success
15 [INFO] output data success
16 Inference average time: 6.811000 ms
17 [INFO] destroy model input success
18 [INFO] unload model success, model Id is 1
19 [INFO] pid: 98299 Execute sample success
20 [INFO] end to destroy stream
21 [INFO] end to destroy context
22 [INFO] end to reset device is 0
23 [INFO] end to finalize acl

```

- 该日志显示推理流程已完整走通：ACL 初始化、设备上下文、模型加载、输入输出申请均返回 success，Inference time/Inference average time 给出单次与平均耗时，路径 ./out/20251213_11_6_52_564388 指向本次推理结果目录。该输出目录遵循 “YYYYM-MDD_H_M_S_microsec” 命名：20251213 表示日期（2025-12-13），11_6_52 为时分秒，564388 为微秒级序列号，用于区分同日多次推理结果。
- 若选择 --outfmt BIN，可用 `numpy.fromfile('./out/.../resnet50_output_0.bin', dtype=np.float32).reshape(1, 1000)` 读取并执行 `np.argsort` 获取 TopK；使用 softmax 还原概率后结合 ImageNet label 映射即可解读分类结果。
- 若选择 --outfmt TXT，直接 `cat ./out/.../resnet50_output_0.txt | head` 快速查看前若干输出值，同样可以配合脚本解析为 TopK 概率。

4. 后处理

无论 msame 的 --outfmt 选择 BIN 还是 TXT，得到的都是形如 1×1000 的分类 logits，需要结合 ImageNet 标签映射再做一次后处理才能输出可读结果。以下示例脚本演示如何解析 BIN 文件、执行 softmax 并打印 Top-K 概率。

- 先下载 imagenet_class_index.json（可来自 Kaggle 或 GitHub）放到 src 目录，然后创建 src/postprocess_resnet50.py：

```

1 # src/postprocess_resnet50.py
2 import os
3 import argparse
4 import json
5 import numpy as np
6
7 parser = argparse.ArgumentParser(description="将ResNet50的 BIN 输出解码为
8     ↳ ImageNet标签。")
9 parser.add_argument("--bin", required=True, help="msame 输出BIN文件的路径。")
10 parser.add_argument("--topk", type=int, default=5, help="需要打印的最高置信度
11     ↳ 量。")
12 args = parser.parse_args()

```

```

12 logits = np.fromfile(args.bin, dtype=np.float32).reshape(1, -1)
13 probs = np.exp(logits - logits.max(axis=-1, keepdims=True))
14 probs = probs / probs.sum(axis=-1, keepdims=True)
15 probs = probs[0]
16
17 labels_path = os.path.join("src", "imagenet_class_index.json")
18 if not os.path.exists(labels_path):
19     raise FileNotFoundError("请先将imagenet_class_index.json下载到src目录。")
20
21 with open(labels_path, "r", encoding="utf-8") as f:
22     data = json.load(f)
23 labels = [data[str(i)][1] for i in range(len(data))]
24
25 topk = np.argsort(probs)[::-1][:args.topk]
26 print(f"Decoded {args.bin} (Top-{args.topk})")
27 for rank, idx in enumerate(topk, start=1):
28     label = labels[idx] if idx < len(labels) else f"cls_{idx}"
29     print(f"{rank:>2d}: class={idx:<4d} prob={probs[idx]:.6f} label={label}")

```

- 执行脚本：

```

1 python ./src/postprocess_resnet50.py --bin out/20251213_11_6_52_564388/
   ↪ resnet50_output_0.bin

```

- 示例输出：

```

1 Decoded out/20251213_11_6_52_564388/resnet50_output_0.bin (Top-5)
2 1: class=162 prob=0.964885 label=beagle
3 2: class=161 prob=0.023230 label=basset
4 3: class=166 prob=0.006107 label=Walker_hound
5 4: class=167 prob=0.004985 label=English_foxhound
6 5: class=164 prob=0.000334 label=bluetick

```

2.3. 章节小结

本章从宏观分层、转换编译、OM 结构、ACL 编程、性能与精度保障、调试工具、自动化流水线到动态 Shape 与安全实践建立了闭环。掌握这些内容后即可进入后续“边缘系统架构与部署实践”章节，扩展到多模型、多进程及系统级优化。

2.4. 实践任务

1. 任选一个公开 ONNX 分类模型（如 ResNet50）完成 ATC 转换，提交：命令 + atc.log。
2. 以 C 或 Python 实现最小推理程序，输出前 5 TopK 结果与 softmax 概率。
3. 编写对齐脚本比较 50 张图片 ONNX vs OM 输出差异（报告 L1/Top1 差异）。

4. 收集 Profiling Timeline, 列出前 3 耗时算子类型及优化建议。
5. 输出 signature.json、metrics.json、conversion_meta.yaml 并归档。

3. 昇腾 PyTorch 扩展迁移基础

PyTorch 是以动态计算图著称的深度学习框架，核心由灵活的张量运算与自动微分系统构成。`eager execution` 让调试、可视化和原型迭代更直观，而 `TorchScript` 则提供图模式部署以兼顾性能与可移植性。配合丰富的 `TorchVision`、`TorchAudio`、`TorchText` 等生态包，开发者可以在视觉、语音、自然语言处理等任务上快速搭建端到端方案。

`Ascend Extension for PyTorch` 是华为昇腾为 PyTorch 用户提供的深度适配插件，使 PyTorch 能无缝调用昇腾 AI 处理器算力，沿用原生接口并针对算子、通信与调度做深度优化。项目源码可在官方仓库获取，更多动态可关注昇腾社区。

虽然当前已有 `CANN`、`MindSpore` 等优秀深度学习框架，但在 PyTorch 广泛应用的背景下，先基于 PyTorch 学习并借助昇腾插件迁移至昇腾 310B，既能降低学习门槛，又可减少适配成本、缩短开发周期。

由于 `torch_npu` 对昇腾 310B 的支持仍有限，适配与迁移存在诸多问题，因此这里只做 PyTorch 在 310B 上的基础示例与简要介绍。

3.1. 架构速览

整体架构可概括为“PyTorch 前端 + `torch_npu` 中间层 + 昇腾算子后端”。图 1 展示了其分层设计：PyTorch 前端将动态图、自动微分、优化器等能力暴露给开发者；`torch_npu` 负责对接 `CANN` 算子库、通信库以及运行时；底层由昇腾 AI 处理器提供硬件加速。理解这一层次关系有助于在故障排查时迅速定位问题。

3.2. 核心能力纵览

插件在多个维度增强了 PyTorch 在昇腾平台的体验：

- **算子与生态适配**：基于开源 PyTorch 实现对昇腾 AI 处理器的深度定制，持续补齐主流算子与第三方库，保证常见模型脚本可直接迁移。
- **训练基础设施**：保持动态图、自动微分、优化器与 `Profiling` 等特性，与原生 PyTorch 保持一致，降低学习成本。
- **自定义算子扩展**：为算子开发者提供接口，可在 PyTorch 框架内快速集成自研算子。

- **分布式通信**: 内置 Broadcast、AllReduce 等集合通信原语, 覆盖单机多卡与多机多卡场景。
- **推理链路**: 模型可导出 ONNX, 再借助离线转换工具生成适配昇腾的推理模型, 便于训练—推理一体化交付。

3.3. PyTorch 安装教程

3.3.1. CANN 环境准备

在昇腾 310B 开发板上安装 PyTorch 之前, 必须安装匹配 CANN 版本的驱动与固件。如果没有安装 CANN 相关的工具包与驱动, 请参考《[CANN 软件安装指南](#)》或者本教程的[第二章](#)进行离线安装。如果已安装 CANN 相关工具包与驱动, 可按执行以下命令获取版本信息:

```
1 cat /usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/ascend_toolkit_install
   ↪ .info
```

例如刚刚完成 CANN 8.3.RC1 版本的安装后, 会输出以下信息:

```
1 package_name=Ascend-cann-toolkit
2 version=8.3.RC1
3 innerversion=V100R001C23SPC001B235
4 compatible_version=[V100R001C15],[V100R001C18],[V100R001C19],[V100R001C20],[
   ↪ V100R001C21],[V100R001C23]
5 arch=aarch64
6 os=linux
7 path=/usr/local/Ascend/ascend-toolkit/8.3.RC1/aarch64-linux
```

在安装 PyTorch 框架与 torch_npu 插件之前, 请先确认 CANN 环境变量已正确加载。若尚未将加载命令写入 ~/.bashrc, 可在当前会话执行:

```
1 source /usr/local/Ascend/ascend-toolkit/set_env.sh
```

如需持久生效, 请将上述命令追加到 ~/.bashrc 后执行 source ~/.bashrc。

3.3.2. PyTorch 二进制软件包安装方法

完成基础环境后, 可以参考按照[Ascend Extension for PyTorch 软件安装指南](#)或者昇腾的 [Py-Torch 框架的 Github 仓库](#)部署 PyTorch 及 torch_npu 插件。在昇腾 310B 开发板上安装 PyTorch 框架和 torch_npu 插件的方法有两种, 一种是二进制软件包安装, 另外一种源码编译安装。对于初学者来说, 推荐使用二进制软件包进行安装。PyTorch 框架和 torch_npu 插件二进制软件包安装的具体方法如下:

安装 PyTorch 框架

由于出厂的 Ubuntu 22.04 系统已预装 miniconda, 我们可以直接利用该环境。预装的 miniconda 通常带有 bash, 在 base 下直接安装 PyTorch 与 torch_npu 容易产生冲突, 因此建议

新建一个 conda 环境，例如使用下面这个命令创建名为 npu 的环境。

```
1 conda create -n npu
```

然后利用下面这个命令进入 npu 虚拟环境：

```
1 conda activate npu
```

若按照本教程第二章在昇腾 310B 开发板上安装了 CANN 8.3.RC1，则可用的 Python 版本为 3.8–3.11，对应支持的 PyTorch 版本为 2.1.0、2.6.0、2.7.1、2.8.0。这里选择安装 Python 3.11 与 PyTorch 2.8.0，先在新建的 npu 环境安装 Python 编译器：

```
1 conda install python=3.11
```

下载 PyTorch 二进制包时需与当前 Python 版本一致，可用 `python -V` 确认版本，输出示例为 Python 3.11.14。该环境还需安装 CANN 依赖的第三方库，可执行：

```
1 pip3 install attrs cython 'numpy>=1.19.2,<=1.24.0' decorator \
2 sympy cffi pyyaml pathlib2 psutil protobuf==3.20.0 scipy requests absl-py \
3 cloudpickle ml-dtypes tornado jinja2 matplotlib tqdm
```

在昇腾开发板上安装 PyTorch 可手动下载二进制包或直接用 pip 自动安装。若手动下载，需确保与 Python 和 CANN 版本严格匹配，可在[华为昇腾 PyTorch 官方安装指南](#)选择对应的包。假设 Python 3.11、CANN 8.3.RC1，可参考以下命令下载：

```
1 wget https://download.pytorch.org/whl/cpu/torch-2.8.0%2Bcpu-cp311-cp311-
  ↪ manylinux_2_28_aarch64.whl
```

然后使用 pip3 安装：

```
1 pip3 install torch-2.8.0+cpu-cp311-cp311-manylinux_2_28_aarch64.whl
```

安装完成后，可通过以下命令验证：

```
1 python -c "import torch; print(torch.__version__)"
```

若安装正确，输出示例为：

```
1 2.8.0+cpu
```

安装昇腾 torch_npu 插件

安装 torch_npu 时，版本需要与 PyTorch、Python、系统架构以及 CANN 保持一致。可前往[华为昇腾 PyTorch 官方安装指南](#)选择匹配的二进制包。例如在 Python 3.11、PyTorch 2.8.0、CANN 8.3.RC1 场景下，选择 torch_npu 7.2.0，对应示例下载命令为：

```
1 wget https://gitcode.com/Ascend/pytorch/releases/download/v7.2.0-pytorch2.8.0/
  ↪ torch_npu-2.8.0-cp311-cp311-manylinux_2_28_aarch64.whl
```

下载后使用 pip3 安装：

```
1 pip3 install torch_npu-2.8.0-cp311-cp311-manylinux_2_28_aarch64.whl
```

注意：直接使用 `pip install torch_npu` 进行简易安装，可能导致程序与当前环境不兼容，或与已安装的 CANN 版本不完全适配。因此，建议谨慎采用该方式安装 `torch_npu` 插件。

3.3.3. torch_npu 安装后测试

成功安装 `torch_npu` 后，需要对安装后的 `torch_npu` 进行简单的测试，以此验证是否安装成功。其中最简单的方法，是直接在命令窗口中输入以下命令：

```
1 python3 -c "import torch;import torch_npu; a = torch.randn(3, 4).npu(); print(a
    ↪ + a);"
```

成功运行后可见如下输出：

```
1 .[W compiler_depend.ts:164] Warning: Device do not support double dtype now,
    ↪ dtype cast replace with float. (function operator())
2 ...tensor([[ 0.0879,  0.5805, -0.3437, -0.0229],
3           [-1.7557,  0.0712,  2.7342,  1.8566],
4           [-1.2092,  1.4896,  0.2234,  0.4621]], device='npu:0')
```

上述 warning 来源于昇腾 310B 暂不支持双精度浮点 (double)，运行时会自动将双精度浮点 double 转为单精度浮点 float。若在 HwHiAiUser (非 root) 用户下执行同一命令，输出与下述示例类似：

```
1 /usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/collect_env.py
    ↪ :59: UserWarning: Warning: The /usr/local/Ascend/ascend-toolkit/latest
    ↪ owner does not match the current owner.
2 warnings.warn(f"Warning: The {path} owner does not match the current owner.")
3 /usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/collect_env.py
    ↪ :59: UserWarning: Warning: The /usr/local/Ascend/ascend-toolkit/8.3.RC1/
    ↪ aarch64-linux/ascend_toolkit_install.info owner does not match the
    ↪ current owner.
4 warnings.warn(f"Warning: The {path} owner does not match the current owner.")
5 /usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/_path_manager.
    ↪ py:67: UserWarning: Permission mismatch: The owner of /usr/local/
    ↪ miniconda3/lib/python3.9/site-packages/torch_npu/lib/libop_plugin_atb.so
    ↪ does not match.
6 warnings.warn(f"Permission mismatch: The owner of {path} does not match.")
7 /usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/collect_env.py
    ↪ :59: UserWarning: Warning: The /usr/local/Ascend/ascend-toolkit/latest
    ↪ owner does not match the current owner.
8 warnings.warn(f"Warning: The {path} owner does not match the current owner.")
9 /usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/collect_env.py
    ↪ :59: UserWarning: Warning: The /usr/local/Ascend/ascend-toolkit/8.3.RC1/
    ↪ aarch64-linux/ascend_toolkit_install.info owner does not match the
    ↪ current owner.
10 warnings.warn(f"Warning: The {path} owner does not match the current owner.")
11 [W compiler_depend.ts:164] Warning: Device do not support double dtype now,
    ↪ dtype cast replace with float. (function operator())
12 tensor([[ 0.4831,  0.6161,  0.2278, -0.1595],
13         [ 1.3375,  0.2916, -0.3860, -2.3874],
14         [ 1.3766, -1.6611,  2.7217,  2.0813]], device='npu:0')
```

尽管会出现较多 warning，这是因为 `torch_npu` 插件用 root 账户安装所致，但结果依然是正确的。

注意事项：多数情况下，已成功安装后仍在运行上文命令时报错，原因是内存不足，尤其常见于 8GB 内存的昇腾 310B 开发板。首次运行 PyTorch+torch_npu 时，会触发将计算图交给 CANN 进行编译与优化（含算子/图的并行编译），默认并行度较高，容易在低内存设备上触发 OOM。

解决办法：降低编译并行度

- 临时生效（仅针对本次运行）：

```
1 TE_PARALLEL_COMPILER=1 MAX_COMPILE_CORE_NUMBER=1 python your_script.py
```

- 会话/永久生效：

```
1 export TE_PARALLEL_COMPILER=1
2 export MAX_COMPILE_CORE_NUMBER=1
3 # 建议追加到 ~/.bashrc 后执行：source ~/.bashrc
```

说明 - 该问题与使用 atc 工具时的并行编译内存占用现象一致，降低并行度可显著缓解。- 首次运行的编译开销较大，通过以上设置可提高在 8GB 设备上的稳定性。

3.4. torch_npu 插件的使用

PyTorch 是广泛使用的深度学习框架，基于 Python，入门友好且生态完善。系统学习可参考李沐的《动手学深度学习》第二版（开源电子书，含配套代码）：[动手学深度学习](#)。

关于 torch_npu：目前在昇腾 310B 上自动迁移工具不可用（实测无法跑通），因此本章默认不使用自动迁移，统一采用手工迁移路径，并提供常见替换与调优对策。迁移与调优可参考官方文档《[PyTorch 训练模型迁移调优](#)》。但是该文档主要面向训练服务器，细节与 310B 存在差异。

下文将结合若干实例，演示 torch_npu 在昇腾 310B 上的实践要点。

3.4.1. 线性神经网络实现

为了更清晰地展示基于 CUDA 的 PyTorch 使用与基于 torch_npu 的使用之间的区别，我们将从经典的线性神经网络入手。线性回归和 softmax 回归作为经典统计学习技术，可以视为线性神经网络的实例。这些知识将为在昇腾 310B 上进行代码移植的其他部分奠定基础。

一元线性回归

线性回归可用于刻画变量间的线性关系。在最简单的一元情形中，大学物理实验的电阻测量就是典型案例。根据欧姆定律可得：

$$V = IR$$

为适应实际中的接触电势/表计零漂，常用含偏置模型：

$$V_i \approx R I_i + b$$

以均方误差为目标函数：

$$L(R, b) = \frac{1}{n} \sum_{i=1}^n (R I_i + b - V_i)^2$$

其梯度为：

$$\frac{\partial L}{\partial R} = \frac{2}{n} \sum_{i=1}^n I_i (R I_i + b - V_i), \quad \frac{\partial L}{\partial b} = \frac{2}{n} \sum_{i=1}^n (R I_i + b - V_i)$$

采用小批量 SGD（批大小为 m ，索引集合 B ）：

$$g_R = \frac{2}{m} \sum_{i \in B} I_i (R I_i + b - V_i), \quad g_b = \frac{2}{m} \sum_{i \in B} (R I_i + b - V_i)$$

按学习率更新：

$$R \leftarrow R - \eta g_R, \quad b \leftarrow b - \eta g_b$$

实践流程（单位统一为 A、V，建议 float32，NPU 上双精度会自动降为 float32）：- 采集多组 (I_i, V_i) ，若存在接触电势选用含偏置模型。- 初始化参数（启发式： $R \approx (\sum I_i V_i) / (\sum I_i^2)$, $b \approx 0$ ）。- 随机抽取小批量，计算 g_R, g_b ，按学习率迭代至收敛。- 用 MSE 或 R^2 评估拟合，必要时剔除离群点。

下方示例代码在 PyTorch+torch_npu 上实现上述流程。

```

1 """线性回归 ( $V \approx R \cdot I + b$ ) 示例，使用华为昇腾 NPU 设备训练。
2 - 数据：合成电压/电流数据，单位 A/V，含高斯噪声（约 5mV）。
3 - 目标：拟合电阻 R（斜率）与偏置 b（截距）。
4 - 设备：默认 npu:0，可切换为 CPU。
5 - 训练：SGD 优化，MSE 损失，小批量训练并通过 tqdm 展示进度。
6 - 输出：打印估计的 R、b，并保存训练损失曲线为 training_loss.png。
7 """
8
9 import torch # PyTorch 主库
10 from tqdm import tqdm # 进度条显示
11 import matplotlib.pyplot as plt # 绘图
12 import torch_npu # NPU 支持库（确保已安装与环境就绪）
13
14 device = torch.device('npu:0') # 选择 NPU 设备（Ascend），需要正确的驱动/环境
15 torch.manual_seed(0) # 固定随机种子以确保可复现
16
17 # 使用合成数据（无自有数据）
18 N = 1024 # 样本数
19 R_true, b_true = 120.0, 0.02 # 真实电阻 (Ω) 与偏置 (V)，用于生成数据的“地真
    ↳ 值”
20 I = torch.linspace(0.0, 1.0, N).unsqueeze(1) # 电流张量 [N,1]，范围 0~1 A
21 noise_std = 0.005 # 噪声标准差  $\approx$  5 mV
22 noise = noise_std * torch.randn(N, 1) # 加性高斯噪声，与 V 同形状
23 V = R_true * I + b_true + noise # 生成电压张量 [N,1]，单位 V
24

```

```

25 I = I.to(device) # 将数据移至计算设备 (NPU/CPU)
26 V = V.to(device)
27 batch_size = 16 # 小批量大小, 影响梯度估计方差与训练稳定性
28
29 # 线性模型  $V \approx R \cdot I + b$  (1 输入 -> 1 输出, 带偏置)
30 model = torch.nn.Linear(1, 1, bias=True).to(device) # 权重即电阻 R, 偏置即 b
31 opt = torch.optim.SGD(model.parameters(), lr=1e-2) # 随机梯度下降, 学习率 1e-2
32 loss_fn = torch.nn.MSELoss() # 均方误差: 衡量预测电压与真实电压的差异
33
34 # 使用小批量训练循环, 加入 tqdm 进度条并记录 loss
35 num_epochs = 50 # 训练轮数
36 epoch_losses = [] # 记录每个 epoch 的平均损失, 便于绘图
37
38 with tqdm(range(num_epochs), desc="Epochs") as pbar: # 进度条显示每个 epoch
39     for _ in pbar:
40         epoch_loss = 0.0 # 累计当前 epoch 的损失
41         n_batches = 0 # 实际批次数 (用于计算平均损失)
42         perm = torch.randperm(I.shape[0], device=device) # 打乱索引以实现随机小
43             ↳ 批量
44         for start in range(0, I.shape[0], batch_size): # 遍历每个批次起点
45             end = start + batch_size # 批次终点
46             idx = perm[start:end] # 当前批次的随机样本索引
47             I_b = I[idx].to(torch.float32) # NPU 通常以 float32 计算, 确保类型
48                 ↳ 一致
49             V_b = V[idx].to(torch.float32)
50             pred = model(I_b) # 前向计算:  $V = R \cdot I + b$ 
51             loss = loss_fn(pred, V_b) # 计算当前批次的 MSE 损失
52             opt.zero_grad() # 清空上一轮梯度
53             loss.backward() # 反向传播, 计算参数梯度
54             opt.step() # 参数更新 (SGD)
55             epoch_loss += loss.item() # 累加标量损失
56             n_batches += 1 # 批次数 +1
57         avg_loss = epoch_loss / max(n_batches, 1) # 当前 epoch 的平均损失
58         epoch_losses.append(avg_loss) # 记录以便后续绘图
59         pbar.set_postfix(loss=f"{avg_loss:.6f}") # 在进度条尾部显示当前损失
60
61 R = model.weight.detach().item() # 提取斜率 (电阻, 单位  $\Omega$ ), 不跟踪梯度
62 b = model.bias.detach().item() # 提取截距 (偏置, 单位 V), 不跟踪梯度
63
64 print("device =", device) # 打印使用的设备
65 print("R( $\Omega$ ) =", R, " b(V) =", b) # 打印拟合结果, 便于与真值对比
66
67 # 绘制训练损失下降曲线并保存图片
68 plt.figure(figsize=(6, 4)) # 设定图像尺寸
69 plt.plot(epoch_losses, label="Train MSE") # 绘制每个 epoch 的平均损失
70 plt.xlabel("Epoch") # 横轴: 训练轮数
71 plt.ylabel("Loss") # 纵轴: 均方误差
72 plt.title("Training Loss") # 标题
73 plt.grid(True) # 开启网格, 便于阅读
74 plt.legend() # 图例显示
75 plt.tight_layout() # 自动布局, 防止标签遮挡
76 plt.savefig("linear_regression_training_loss.png") # 保存为 PNG 文件

```

要运行上述示例, 请先在当前 (npu) 环境中安装依赖:

```
1 pip install tqdm matplotlib
```

示例运行后可见如下输出:

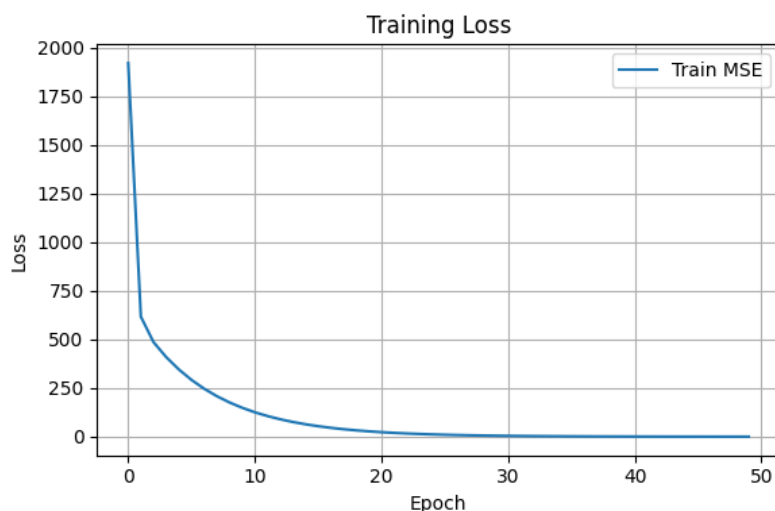


Figure 3.1.: 线性回归的损失函数关系图

```

1 Epochs: 100%|
  ↳ 50/50 [00:28<00:00, 1.77it/s, loss=0.173204]
2 device = npu:0
3 R(Ω) = 118.63302612304688  b(V) = 0.7558640241622925

```

同时给出训练结果的可视化：下图展示训练损失（MSE）随 epoch 变化的曲线。

提示 - 数据需统一单位 (A、V), 使用 float32 更稳妥。- 无明显偏置时可偏置项置为 0; 存在测量/接触偏差时保留偏置并做鲁棒处理 (如剔除离群点)。- 拟合优劣可用 MSE 或 R^2 评估, 必要时清理异常样本。- 在部分环境中, PyTorch 2.1 运行上述代码可能报错, 通常与 `nn.Linear` 的输出维度设为 1 的配置触发兼容性问题有关; 建议升级至更高版本或将输出维度调整为大于 1 以规避。- 将代码中的 `device = torch.device('npu:0')` 改为 `device = torch.device('cpu')`, 即可在纯 CPU 上运行。该示例在 CPU 与 NPU 上耗时接近, 主要因为未涉及大规模矩阵/张量计算, 计算密度较低, NPU 的并行优势难以发挥。

多元线性回归

为了验证昇腾 310B 上 `torch_npu` 与 PyTorch 的兼容性, 下面以加州房价数据集的多元线性回归示例继续演示 `torch_npu` 的使用。加州房价数据 (California Housing Dataset) 可通过 `fetch_california_housing()` 在 `scikit-learn` 获取, 或在 [Kaggle](#) 等平台下载 CSV。该数据集包含经纬度 (longitude/latitude)、房屋中位年龄 (housing_median_age)、总房间数与卧室数 (total_rooms/total_bedrooms)、人口与家庭数 (population/households)、收入中位数 (median_income)、房价中位数 (median_house_value) 以及与海距离类别 (ocean_proximity) 等字段, 可用于房价预测 (如线性回归、随机森林、XGBoost 等回归模型)、结合经纬度进行地

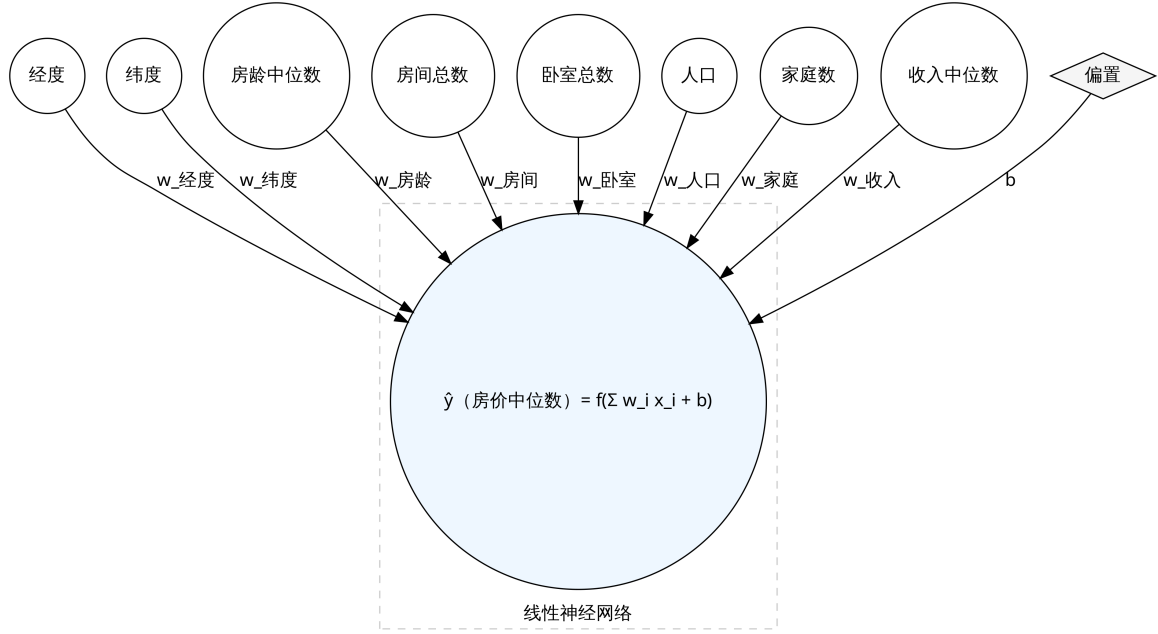


Figure 3.2.: 线性神经网络

理可视化绘制房价热力图以分析区域差异，并通过“每户平均房间数”等衍生特征进行特征工程以提升效果。需注意，该数据为历史数据，不能直接反映当前价格；使用时应遵守隐私与数据保护规定，尤其在与其他数据源结合时；该数据集结构清晰、特征丰富，是机器学习回归入门的经典案例。对于线性神经网络（多元线性回归），可表示为：

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + \mathbf{b}, \quad \mathbf{X} \in \mathbb{R}^{m \times d}, \quad \mathbf{w} \in \mathbb{R}^{d \times k}, \quad \mathbf{b} \in \mathbb{R}^k$$

其中 m 为样本数， d 为特征维度， k 为输出维度（回归时常取 $k = 1$ ）；对于单样本 $\mathbf{x} \in \mathbb{R}^d$ ，有 $\hat{\mathbf{y}} = \mathbf{x}^\top \mathbf{w} + b$ 。加州房价数据集的输入特征维度为 8，输出为房价估计值，其线性神经网络结构示意图如下：

损失函数通常选用均方误差（MSE）：

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m \|\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)}\|_2^2$$

在训练模型的时候，我们希望寻找出一组参数 $(\mathbf{w}^*, b^*)$ ，使得训练样品的损失函数最小：

$$\mathbf{w}^*, b^* = \underset{\mathbf{w}, b}{\operatorname{argmin}} \mathcal{L}(\mathbf{w}, b)$$

在实际训练中，通常采用小批量随机梯度下降法来求解最优参数。其算法流程如下：

1. **初始化**：随机初始化模型参数 $\mathbf{w}$ 和 b ，并设定学习率 η 。
2. **小批量采样**：从训练数据中随机抽取包含 m 个样本的小批量 $B = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(m)}, y^{(m)})\}$ 。

3. **计算梯度**：计算损失函数关于参数的梯度估计值：

$$\mathbf{g}_w = \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{2}{m} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} (\mathbf{x}^{(i)\top} \mathbf{w} + b - y^{(i)})$$

$$g_b = \frac{\partial \mathcal{L}}{\partial b} = \frac{2}{m} \sum_{i \in \mathcal{B}} (\mathbf{x}^{(i)\top} \mathbf{w} + b - y^{(i)})$$

4. **更新参数**：利用计算出的梯度更新当前参数：

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \mathbf{g}_w$$

$$b \leftarrow b - \eta \cdot g_b$$

5. **迭代**：重复步骤 2-4，直到满足停止准则（如达到最大迭代次数或损失函数收敛）。

在上述算法中，学习率（Learning Rate, η ）与训练轮数（Epoch）是影响模型性能的关键超参数。学习率决定了参数沿梯度反方向更新的步长：若设置过大，模型可能在最优解附近震荡甚至发散；若设置过小，则收敛速度过慢且易陷入局部最优。通常学习率的取值范围在 0.01 到 0.00001 之间。训练轮数是指完整遍历一次训练数据集的过程。由于单次迭代仅利用小批量数据更新梯度，通常需要多个 Epoch 才能使模型参数充分拟合数据特征并趋于稳定。一般建议将 Epoch 设置在 10 到 100 之间。

利用 torch_npu 插件，可以在昇腾 310B 上进行这个模型的训练，合理配置这些超参数不仅影响模型的最终精度，还直接关系到有限算力资源下的训练效率。详细的代码如下：

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from sklearn.datasets import fetch_california_housing
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.model_selection import train_test_split
7 from torch.utils.data import DataLoader, TensorDataset
8 import numpy as np
9 from tqdm.auto import tqdm, trange
10
11 # =====
12 # 1. 环境配置与随机种子设置
13 # =====
14 # 设置随机种子以确保实验结果的可复现性
15 np.random.seed(42)
16 torch.manual_seed(42)
17
18 # 指定计算设备。'npu' 表示使用华为昇腾 AI 处理器
19 # 如果在没有 NPU 的环境下运行，可改为 'cuda' 或 'cpu'
20 device = torch.device('npu')
21
22 # =====
23 # 2. 数据加载与预处理
24 # =====
25 # 加载加州房价数据集（回归任务）
26 california = fetch_california_housing()
27 X = california.data # 特征矩阵：包含 8 个特征（如人均收入、房龄等）
28 y = california.target.reshape(-1, 1) # 目标值：房价，调整为 (N, 1) 形状
```

```

29
30 # 数据标准化：神经网络对输入特征的量级很敏感
31 # 使用 StandardScaler 使数据符合均值为 0，方差为 1 的分布
32 scaler_X = StandardScaler()
33 scaler_y = StandardScaler()
34 X_scaled = scaler_X.fit_transform(X)
35 y_scaled = scaler_y.fit_transform(y)
36
37 # 划分数据集：80% 用于训练，20% 用于验证
38 X_train, X_val, y_train, y_val = train_test_split(
39     X_scaled, y_scaled, test_size=0.2, random_state=42
40 )
41
42 # =====
43 # 3. 构建 PyTorch 数据加载器 (DataLoader)
44 # =====
45 batch_size = 32
46
47 # 将 NumPy 数组转换为 PyTorch 张量 (FloatTensor)
48 train_dataset = TensorDataset(
49     torch.FloatTensor(X_train),
50     torch.FloatTensor(y_train)
51 )
52 val_dataset = TensorDataset(
53     torch.FloatTensor(X_val),
54     torch.FloatTensor(y_val)
55 )
56
57 # DataLoader 负责按批次 (Batch) 提供数据，并在训练集上开启打乱 (Shuffle)
58 train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
59 val_loader = DataLoader(val_dataset, batch_size=batch_size)
60
61 # =====
62 # 4. 定义模型结构
63 # =====
64 # 单层感知机：本质上是一个线性回归模型 ( $y = Wx + b$ )
65 class SingleLayerPerceptron(nn.Module):
66     def __init__(self, input_dim):
67         super().__init__()
68         # 定义一个全连接层，输入维度为特征数，输出维度为 1
69         self.fc = nn.Linear(input_dim, 1)
70
71     def forward(self, x):
72         # 前向传播逻辑
73         return self.fc(x)
74
75 # 实例化模型并迁移到指定的计算设备 (NPU)
76 model = SingleLayerPerceptron(X_train.shape[1]).to(device)
77
78 # 定义损失函数：均方误差 (MSE)，适用于回归任务
79 criterion = nn.MSELoss()
80
81 # 定义优化器：随机梯度下降 (SGD)，学习率设为 0.01
82 optimizer = optim.SGD(model.parameters(), lr=0.01)
83
84 # =====
85 # 5. 训练与验证函数定义
86 # =====

```

```

87 def train_epoch(model, loader, criterion, optimizer):
88     """单轮训练逻辑"""
89     model.train() # 设置为训练模式
90     total_loss = 0
91     for batch_x, batch_y in tqdm(loader, desc="Train", leave=False):
92         # 将数据迁移到 NPU
93         batch_x = batch_x.to(device)
94         batch_y = batch_y.to(device)
95
96         # 梯度清零
97         optimizer.zero_grad()
98         # 前向传播
99         outputs = model(batch_x)
100        # 计算损失
101        loss = criterion(outputs, batch_y)
102        # 反向传播
103        loss.backward()
104        # 梯度裁剪：防止梯度爆炸
105        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
106        # 更新参数
107        optimizer.step()
108
109        total_loss += loss.item() * batch_x.size(0)
110    return total_loss / len(loader.dataset)
111
112 def validate(model, loader, criterion):
113     """验证逻辑"""
114     model.eval() # 设置为评估模式
115     total_loss = 0
116     with torch.no_grad(): # 验证阶段不需要计算梯度
117         for batch_x, batch_y in tqdm(loader, desc="Val", leave=False):
118             batch_x = batch_x.to(device)
119             batch_y = batch_y.to(device)
120             outputs = model(batch_x)
121             loss = criterion(outputs, batch_y)
122             total_loss += loss.item() * batch_x.size(0)
123     return total_loss / len(loader.dataset)
124
125 # =====
126 # 6. 执行训练循环
127 # =====
128 epochs = 20
129 train_losses = []
130 val_losses = []
131
132 # 使用 trange 展示总进度条
133 t = trange(epochs, desc="Epochs")
134 for epoch in t:
135     train_loss = train_epoch(model, train_loader, criterion, optimizer)
136     val_loss = validate(model, val_loader, criterion)
137
138     train_losses.append(train_loss)
139     val_losses.append(val_loss)
140
141     # 在进度条右侧实时显示当前损失和学习率
142     t.set_postfix(train_loss=f"{train_loss:.4f}", val_loss=f"{val_loss:.4f}", lr
143                   ↳ =f'{optimizer.param_groups[0]["lr"]:.6f}')

```

```

144 print(f"\nFinal Validation Loss: {val_losses[-1]:.4f}")
145
146 # =====
147 # 7. 模型评估与指标计算
148 # =====
149 model.eval()
150 all_predictions = []
151 all_targets = []
152
153 # 获取验证集上的所有预测结果
154 with torch.no_grad():
155     for batch_x, batch_y in val_loader:
156         batch_x = batch_x.to(device)
157         predictions = model(batch_x)
158         # 将结果移回 CPU 并转为 NumPy
159         all_predictions.extend(predictions.cpu().numpy())
160         all_targets.extend(batch_y.cpu().numpy())
161
162 # 逆标准化: 将预测值和真实值还原回原始的房价单位 (10万美元)
163 all_predictions = scaler_y.inverse_transform(np.array(all_predictions))
164 all_targets = scaler_y.inverse_transform(np.array(all_targets))
165
166 # 计算回归常用指标
167 from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
168
169 r2 = r2_score(all_targets, all_predictions) # 决定系数, 越接近 1 越好
170 mae = mean_absolute_error(all_targets, all_predictions) # 平均绝对误差
171 rmse = np.sqrt(mean_squared_error(all_targets, all_predictions)) # 均方根误差
172
173 print(f"\n最终评估指标:")
174 print(f"R2 Score: {r2:.4f}")
175 print(f"MAE: ${mae:.2f}")
176 print(f"RMSE: ${rmse:.2f}")
177
178 # =====
179 # 8. 结果可视化
180 # =====
181 import matplotlib.pyplot as plt
182
183 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
184
185 # 1. 损失曲线: 观察模型是否收敛
186 axes[0, 0].plot(train_losses, label='Train Loss')
187 axes[0, 0].plot(val_losses, label='Val Loss')
188 axes[0, 0].set_xlabel('Epoch')
189 axes[0, 0].set_ylabel('Loss')
190 axes[0, 0].set_title('Training and Validation Loss')
191 axes[0, 0].legend()
192 axes[0, 0].grid(True)
193
194 # 2. 预测值 vs 真实值: 散点越接近对角线表示预测越准
195 axes[0, 1].scatter(all_targets, all_predictions, alpha=0.3)
196 axes[0, 1].set_xlabel('True Price')
197 axes[0, 1].set_ylabel('Predicted Price')
198 axes[0, 1].set_title('True vs Predicted Prices')
199 axes[0, 1].grid(True)
200
201 # 3. 残差图: 检查误差是否随机分布 (理想状态下应无明显模式)

```

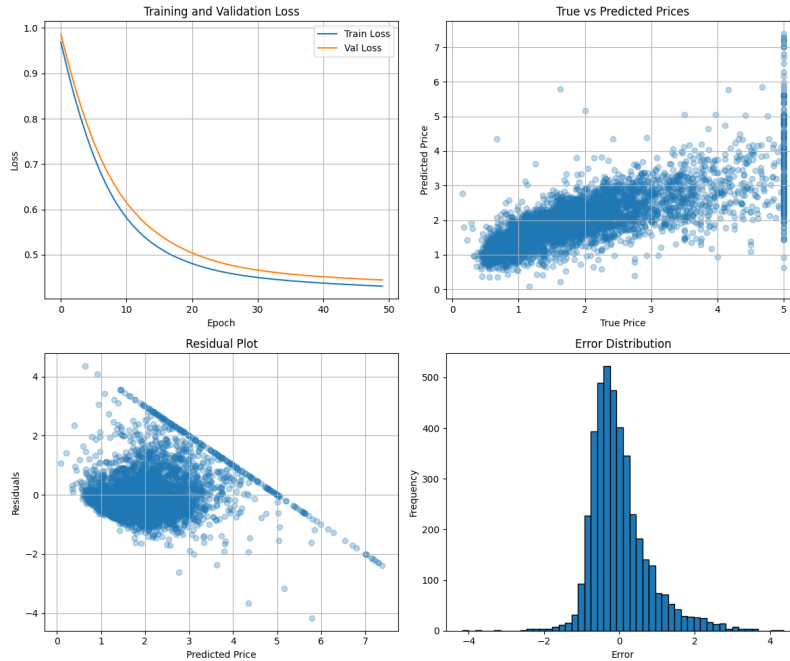


Figure 3.3.: 线性神经网络训练结果图

```

202 residuals = all_targets - all_predictions
203 axes[1, 0].scatter(all_predictions, residuals, alpha=0.3)
204 axes[1, 0].set_xlabel('Predicted Price')
205 axes[1, 0].set_ylabel('Residuals')
206 axes[1, 0].set_title('Residual Plot')
207 axes[1, 0].grid(True)
208
209 # 4. 误差分布直方图：检查误差是否符合正态分布
210 axes[1, 1].hist(residuals, bins=50, edgecolor='black')
211 axes[1, 1].set_xlabel('Error')
212 axes[1, 1].set_ylabel('Frequency')
213 axes[1, 1].set_title('Error Distribution')
214
215 plt.tight_layout()
216 plt.savefig('california_housing_linear_network_results.png')

```

在配置好 torch_npu 环境的昇腾 310B 开发板上运行以上的代码，可以得到以下的结果：

```

1 Epochs: 100%|██████████| 50/50 [02:14<00:00, 2.68s/it, lr=0.000100, train_loss
  ↳ =0.4308, val_loss=0.4444]
2
3 Final Validation Loss: 0.4444
4
5 最终评估指标：
6 R2 Score: 0.5484
7 MAE: $0.56
8 RMSE: $0.77

```

同时，我们可以得到训练过程的损失收敛及评估指标图如下图所示：

3.4.2. 多层感知机实现

我们之所以需要多层感知机 (Multilayer Perceptron, MPL)，是因为线性神经网络的输出仅为输入的加权和，难以刻画现实世界中复杂的非线性关系。以房价预测为例，房屋面积对价格的影响往往存在边际效应，而非简单的线性增长。为了突破这一局限，我们需要构建更深的网络结构。

如果我们仅仅通过堆叠多个线性层来构建网络，其数学表达为

$$\hat{\mathbf{y}} = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

展开后可见

$$\hat{\mathbf{y}} = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2) = \mathbf{W}_{new}\mathbf{x} + \mathbf{b}_{new}$$

这意味着多个线性层的组合在数学上仍然等价于单个线性层。无论网络深度如何增加，若缺乏非线性因素，它都无法拟合复杂的非线性函数。

多层感知机通过在层间引入激活函数打破了这种线性限制。以常用的 ReLU 为例，其定义为

$$\text{ReLU}(x) = \max(0, x)$$

另一个经典的激活函数是 Sigmoid 函数，它将任意实数映射到 (0, 1) 区间，曾广泛应用于早期神经网络。其定义为

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

这种非线性变换允许网络通过组合不同的线性分段来逼近任意复杂的连续函数。一个含隐藏层的多层感知机可以表示为

$$\mathbf{H} = \sigma(\mathbf{XW}_1 + \mathbf{b}_1)$$

与

$$\mathbf{O} = \mathbf{HW}_2 + \mathbf{b}_2$$

其中 σ 即为非线性激活函数。

相比于只能处理线性相关问题的线性网络，多层感知机通过增加隐藏层的深度与宽度，能够捕捉特征之间的高阶交互作用。在加州房价数据集中，这种结构可以学习到经纬度与收入水平之间复杂的空间关联。对于加州房价数据集的多层感知机的模型如下图所示：

在昇腾 310B 上实现多层感知机，不仅能验证 torch_npu 对多层算子序列的调度能力，也能通过实验直观观察到非线性映射带来的精度提升。具体的代码如下：

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from sklearn.datasets import fetch_california_housing
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.model_selection import train_test_split
7 from torch.utils.data import DataLoader, TensorDataset
8 import numpy as np
9 from tqdm.auto import tqdm, trange
```

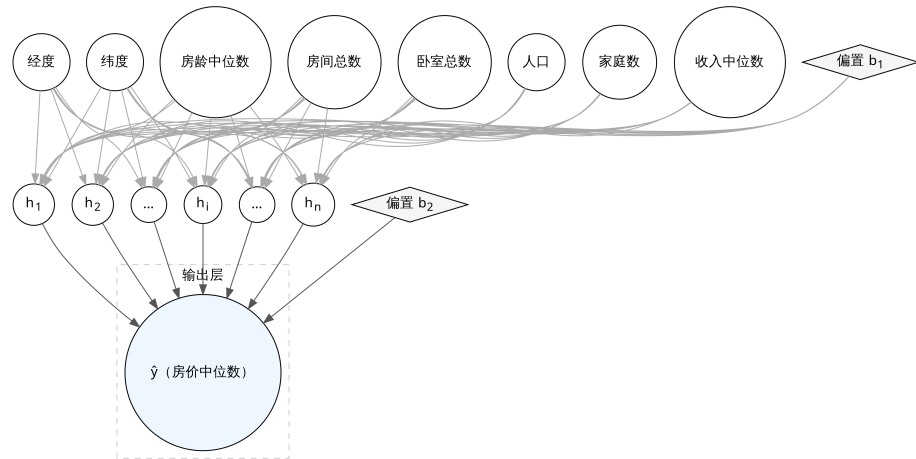


Figure 3.4.: 多层神经网络

```

10 import random
11
12 # 设置随机种子以确保实验结果的可重现性
13 SEED = 42
14 random.seed(SEED)
15 np.random.seed(SEED)
16 torch.manual_seed(SEED)
17 # 指定使用昇腾 NPU 设备
18 device = torch.device('npu:0')
19
20 # 加载加州房价数据集：包含8个特征，目标值为房屋中位数价格
21 california = fetch_california_housing()
22 X = california.data # 特征矩阵 (样本数, 8)
23 y = california.target.reshape(-1, 1) # 目标值 (样本数, 1)
24
25 # 数据标准化：将特征和目标值缩放到均值为0、方差为1的分布，有助于模型收敛
26 scaler_X = StandardScaler()
27 scaler_y = StandardScaler()
28 X_scaled = scaler_X.fit_transform(X)
29 y_scaled = scaler_y.fit_transform(y)
30
31 # 划分数据集：80% 用于训练，20% 用于验证
32 X_train, X_val, y_train, y_val = train_test_split(
33     X_scaled, y_scaled, test_size=0.2, random_state=42
34 )
35
36 # 构建 PyTorch 数据管道
37 batch_size = 32
38 train_dataset = TensorDataset(
39     torch.FloatTensor(X_train),
40     torch.FloatTensor(y_train)
41 )
42 val_dataset = TensorDataset(
43     torch.FloatTensor(X_val),
44     torch.FloatTensor(y_val)
45 )
46

```



```

47 # DataLoader 负责按批次(Batch)提供数据并支持洗牌(Shuffle)
48 train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
49 val_loader = DataLoader(val_dataset, batch_size=batch_size)
50
51 # 定义双层感知机模型
52 class DoubleLayerPerceptron(nn.Module):
53     def __init__(self, input_dim, hidden_dim=64):
54         super().__init__()
55         self.net = nn.Sequential(
56             nn.Linear(input_dim, hidden_dim), # 输入层到隐藏层
57             nn.ReLU(), # 激活函数, 引入非线性
58             nn.Linear(hidden_dim, 1) # 隐藏层到输出层 (回归任务输出维度
59                                     ↪ 为1)
60         )
61     def forward(self, x):
62         return self.net(x)
63
64 # 实例化模型并迁移至 NPU
65 model = DoubleLayerPerceptron(X_train.shape[1]).to(device)
66 # 回归任务常用的均方误差损失函数
67 criterion = nn.MSELoss()
68 # 随机梯度下降优化器
69 optimizer = optim.SGD(model.parameters(), lr=0.0001)
70
71 # 单个Epoch 的训练逻辑
72 def train_epoch(model, loader, criterion, optimizer):
73     model.train()
74     total_loss = 0
75     for batch_x, batch_y in tqdm(loader, desc="Train", leave=False):
76         # 将数据搬运到 NPU 显存
77         batch_x = batch_x.to(device)
78         batch_y = batch_y.to(device)
79
80         optimizer.zero_grad() # 清空梯度
81         outputs = model(batch_x) # 前向传播
82         loss = criterion(outputs, batch_y) # 计算损失
83         loss.backward() # 反向传播计算梯度
84         torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0) # 梯度
85                                     ↪ 裁剪防止梯度爆炸
86         optimizer.step() # 更新权重
87         total_loss += loss.item() * batch_x.size(0)
88     return total_loss / len(loader.dataset)
89
90 # 验证逻辑
91 def validate(model, loader, criterion):
92     model.eval() # 设置为评估模式
93     total_loss = 0
94     with torch.no_grad(): # 验证阶段不需要计算梯度
95         for batch_x, batch_y in tqdm(loader, desc="Val", leave=False):
96             batch_x = batch_x.to(device)
97             batch_y = batch_y.to(device)
98             outputs = model(batch_x)
99             loss = criterion(outputs, batch_y)
100             total_loss += loss.item() * batch_x.size(0)
101         return total_loss / len(loader.dataset)
102
103 # 执行训练过程
104 epochs = 50

```

```

103 train_losses = []
104 val_losses = []
105
106 t = trange(epochs, desc="Epochs")
107 for epoch in t:
108     train_loss = train_epoch(model, train_loader, criterion, optimizer)
109     val_loss = validate(model, val_loader, criterion)
110     train_losses.append(train_loss)
111     val_losses.append(val_loss)
112     # 在进度条右侧实时显示当前损失和学习率
113     t.set_postfix(train_loss=f"{train_loss:.4f}", val_loss=f"{val_loss:.4f}", lr
        ↳ =f'{optimizer.param_groups[0]["lr"]:.6f}')
114
115 print(f"\nValidation Loss: {val_losses[-1]:.4f}")
116
117 # 评估阶段：获取验证集的预测值并还原标准化
118 model.eval()
119 all_predictions = []
120 all_targets = []
121
122 with torch.no_grad():
123     for batch_x, batch_y in val_loader:
124         batch_x = batch_x.to(device)
125         predictions = model(batch_x)
126         all_predictions.extend(predictions.cpu().numpy())
127         all_targets.extend(batch_y.cpu().numpy())
128
129 # 将标准化后的数据逆转回原始价格单位
130 all_predictions = scaler_y.inverse_transform(np.array(all_predictions))
131 all_targets = scaler_y.inverse_transform(np.array(all_targets))
132
133 # 计算回归评估指标
134 from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
135
136 r2 = r2_score(all_targets, all_predictions) # 决定系数，越接近 1 越好
137 mae = mean_absolute_error(all_targets, all_predictions) # 平均绝对误差
138 rmse = np.sqrt(mean_squared_error(all_targets, all_predictions)) # 均方根误差
139
140 print(f"\n最终评估指标:")
141 print(f"R2 Score: {r2:.4f}")
142 print(f"MAE: ${mae:.2f}")
143 print(f"RMSE: ${rmse:.2f}")
144
145 # 结果可视化
146 import matplotlib.pyplot as plt
147
148 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
149
150 # 1. 损失下降曲线：观察模型是否收敛或过拟合
151 axes[0, 0].plot(train_losses, label='Train Loss')
152 axes[0, 0].plot(val_losses, label='Val Loss')
153 axes[0, 0].set_xlabel('Epoch')
154 axes[0, 0].set_ylabel('Loss')
155 axes[0, 0].set_title('Training and Validation Loss')
156 axes[0, 0].legend()
157 axes[0, 0].grid(True)
158
159 # 2. 预测值 vs 真实值散点图：点越靠近红虚线表示预测越准

```

```

160 axes[0, 1].scatter(all_targets, all_predictions, alpha=0.3)
161 axes[0, 1].plot([all_targets.min(), all_targets.max()],
162                [all_targets.min(), all_targets.max()], 'r--', lw=2)
163 axes[0, 1].set_xlabel('True Price')
164 axes[0, 1].set_ylabel('Predicted Price')
165 axes[0, 1].set_title('True vs Predicted Prices')
166 axes[0, 1].grid(True)
167
168 # 3. 残差图：检查误差是否随机分布（理想状态应均匀分布在0附近）
169 residuals = all_targets - all_predictions
170 axes[1, 0].scatter(all_predictions, residuals, alpha=0.3)
171 axes[1, 0].axhline(y=0, color='r', linestyle='--')
172 axes[1, 0].set_xlabel('Predicted Price')
173 axes[1, 0].set_ylabel('Residuals')
174 axes[1, 0].set_title('Residual Plot')
175 axes[1, 0].grid(True)
176
177 # 4. 误差分布直方图：观察误差是否符合正态分布
178 axes[1, 1].hist(residuals, bins=50, edgecolor='black')
179 axes[1, 1].set_xlabel('Error')
180 axes[1, 1].set_ylabel('Frequency')
181 axes[1, 1].set_title('Error Distribution')
182 axes[1, 1].grid(True)
183
184 plt.tight_layout()
185 plt.savefig('california_housing_mlp_results.png')

```

在昇腾 310B 开发板上运行以上代码，可以得到以下的结果：

```

1 Epochs: 100%|
  ↳ 50/50 [02:39<00:00, 3.19s/it, lr=0.000100, train_loss=0.4509, val_loss
  ↳ =0.4563]
2
3 Validation Loss: 0.4563
4
5 最终评估指标：
6 R2 Score: 0.5363
7 MAE: $0.57
8 RMSE: $0.78

```

同时，我们可以得到训练过程的损失收敛及评估指标图如下图所示：

3.4.3. LeNet-5

LeNet-5 是由 Yann LeCun 等人在 1998 年提出的经典卷积神经网络（CNN），最初用于手写数字识别（MNIST 数据集）。它是深度学习领域的里程碑，奠定了现代卷积神经网络的基础架构。其名称中的“Le”取自第一作者 Yann LeCun 的姓氏，而数字“5”则代表该网络包含 5 层具有可训练参数的权重层（包括 3 个卷积层和 2 个全连接层）。在原始设计中，C5 层虽然在逻辑上起到了全连接的作用，但由于其卷积核尺寸与输入特征图完全一致，因此被归类为卷积层。

LeNet-5 共有 7 层（不含输入层），其标准结构如下：

1. **输入层 (Input)**: 接收 32×32 的灰度图像。

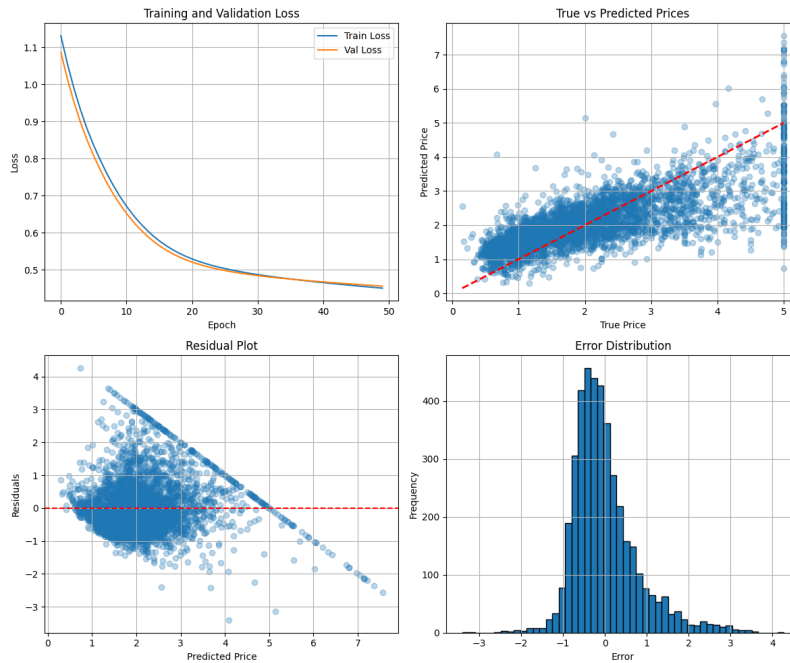


Figure 3.5.: 多层神经网络训练结果图

2. 卷积层 (C1): 使用 6 个 5×5 的卷积核, 输出特征图大小为 28×28 。
3. 池化层 (S2): 2×2 平均池化, 步长为 2, 输出大小为 14×14 。
4. 卷积层 (C3): 使用 16 个 5×5 的卷积核, 输出大小为 10×10 。
5. 池化层 (S4): 2×2 平均池化, 步长为 2, 输出大小为 5×5 。
6. 全连接层 (C5): 包含 120 个神经元, 将卷积特征展开。
7. 全连接层 (F6): 包含 84 个神经元, 激活函数通常使用 Sigmoid 或 Tanh。
8. 输出层 (Output): 10 个神经元, 对应数字 0-9。

LeNet-5 的核心设计理念在于通过局部感受野使卷积核仅关注图像的局部区域, 从而有效提取边缘和角点等局部特征。配合权值共享机制, 同一特征图上的神经元能够共享卷积参数, 这不仅极大减少了模型的参数总量, 还显著降低了过拟合的风险。此外, 通过下采样技术, 池化层在降低特征图分辨率、减少计算量的同时, 增强了模型对图像形变和平移的鲁棒性。这种交替使用卷积与池化的层级特征提取方式, 使网络能够从基础线条逐步抽象出复杂的数字轮廓。在昇腾 310B 上实现 LeNet-5, 可以进一步验证 torch_npu 对卷积算子 (Conv2d) 和池化算子 (AvgPool2d) 的硬件加速支持。具体代码如下:

```
1 import os
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from torch.utils.data import TensorDataset, DataLoader
```

```

7 from torchvision import datasets
8 import matplotlib.pyplot as plt
9 from tqdm.auto import tqdm, trange
10
11 # 指定使用昇腾 NPU 设备
12 # 在昇腾 AI 处理器上运行 PyTorch 代码时，需要指定设备为 "npu"
13 device = torch.device("npu")
14
15 class LeNet(nn.Module):
16     """
17     经典的 LeNet-5 网络结构实现
18     LeNet-5 是一个简单的卷积神经网络，包含两个卷积层和三个全连接层
19     """
20     def __init__(self):
21         super().__init__()
22         # 第一层卷积：输入通道1（灰度图），输出通道6，卷积核5x5
23         # padding默认是0，stride默认是1
24         self.conv1 = nn.Conv2d(1, 6, kernel_size=5)
25         # 池化层：2x2 窗口，步长为2，用于下采样，减少特征图尺寸
26         self.pool = nn.AvgPool2d(2, 2)
27         # 第二层卷积：输入通道6，输出通道16，卷积核5x5
28         self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
29         # 全连接层：输入特征 16*4*4，输出120
30         # 这里的 16*4*4 是经过两次卷积和池化后的特征图大小展平后的维度
31         # 原始 28x28 -> conv1(5x5) -> 24x24 -> pool(2x2) -> 12x12
32         # -> conv2(5x5) -> 8x8 -> pool(2x2) -> 4x4
33         self.fc1 = nn.Linear(16 * 4 * 4, 120)
34         # 全连接层：输入120，输出84
35         self.fc2 = nn.Linear(120, 84)
36         # 输出层：输入84，输出10（对应 MNIST 的10个类别 0-9）
37         self.fc3 = nn.Linear(84, 10)
38
39     def forward(self, x):
40         # 卷积 -> ReLU 激活 -> 池化
41         # x shape: [batch, 1, 28, 28] -> [batch, 6, 12, 12]
42         x = torch.relu(self.conv1(x))
43         x = self.pool(x)
44         # 卷积 -> ReLU 激活 -> 池化
45         # x shape: [batch, 6, 12, 12] -> [batch, 16, 4, 4]
46         x = torch.relu(self.conv2(x))
47         x = self.pool(x)
48         # 展平特征图为一维向量，用于全连接层
49         # x shape: [batch, 16, 4, 4] -> [batch, 16*4*4]
50         x = x.view(x.size(0), -1)
51         # 全连接层 -> ReLU
52         x = torch.relu(self.fc1(x))
53         x = torch.relu(self.fc2(x))
54         # 输出层，通常配合 CrossEntropyLoss 使用，不需要 softmax
55         x = self.fc3(x)
56         return x
57
58 def plot_metrics(script_dir, train_acc_history, test_acc_history,
59                 train_loss_history, test_loss_history):
60     """
61     绘制并保存训练过程中的精度和损失曲线
62     """
63     plt.figure(figsize=(10, 4))
64     # Accuracy subplot (左图：精度曲线)

```

```

64 plt.subplot(1, 2, 1)
65 plt.plot(train_acc_history, label="Train Acc")
66 plt.plot(test_acc_history, label="Test Acc")
67 plt.xlabel("Epoch"); plt.ylabel("Accuracy"); plt.title("Accuracy"); plt.
    ↳ legend()
68 # Loss subplot (右图: 损失曲线)
69 plt.subplot(1, 2, 2)
70 plt.plot(train_loss_history, label="Train Loss")
71 plt.plot(test_loss_history, label="Test Loss")
72 plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.title("Loss"); plt.legend()
73 plt.tight_layout()
74 # 保存图片到脚本所在目录
75 out_path = os.path.join(script_dir, "accuracy_loss_curve.png")
76 plt.savefig(out_path, bbox_inches="tight")
77 print(f"Saved accuracy & loss curve to: {out_path}")
78
79 def load_data(script_dir, batch_size=256):
80     """
81     加载 MNIST 数据集并进行预处理
82     """
83     # 数据存储路径
84     data_root = os.path.join(script_dir, "data")
85     # 使用 torchvision.datasets 下载并加载 MNIST 数据
86     train_ds = datasets.MNIST(data_root, train=True, download=True)
87     test_ds = datasets.MNIST(data_root, train=False, download=True)
88
89     # 预处理:
90     # 1. unsqueeze(1): 增加通道维度 [N, H, W] -> [N, 1, H, W]
91     # 2. to(torch.float16): 转换为半精度浮点数, NPU 上 float16 计算性能更好
92     # 3. div_(255.0): 像素值归一化到 [0, 1] 区间
93     x_train = train_ds.data.unsqueeze(1).to(torch.float16).div_(255.0)
94     y_train = train_ds.targets.to(torch.int64)
95     x_test = test_ds.data.unsqueeze(1).to(torch.float16).div_(255.0)
96     y_test = test_ds.targets.to(torch.int64)
97
98     # 构建 DataLoader, 用于批量加载数据
99     # shuffle=True 表示训练时打乱数据顺序
100    train_loader = DataLoader(TensorDataset(x_train, y_train), batch_size=
        ↳ batch_size, shuffle=True)
101    test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=
        ↳ batch_size, shuffle=False)
102    return train_loader, test_loader
103
104 def train_and_eval(train_loader, test_loader, epochs=10, lr=0.01):
105     """
106     执行模型训练与验证循环
107     """
108     # 初始化模型并迁移至 NPU 设备
109     # .half() 将模型参数转换为 float16, 利用 NPU 的 Tensor Core 加速
110    model = LeNet().half().to(device)
111    # 定义损失函数: 交叉熵损失, 适用于多分类问题
112    criterion = nn.CrossEntropyLoss()
113    # 定义优化器: 随机梯度下降 (SGD)
114    optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9)
115
116    train_acc_history = []
117    test_acc_history = []
118    train_loss_history = []

```

```

119 test_loss_history = []
120
121 # 使用 trange 展示总进度条
122 t = trange(epochs, desc="Epochs")
123 for epoch in t:
124     # --- 训练阶段 ---
125     model.train() # 设置模型为训练模式 (启用 Dropout, BatchNorm 等)
126     correct = 0; total = 0
127     running_loss = 0.0
128     for inputs, labels in train_loader:
129         # 数据迁移至 NPU 并转为半精度, 标签保持 int64 或 int32
130         inputs = inputs.to(device)
131         labels = labels.to(device)
132
133         # 前向传播
134         outputs = model(inputs)
135         # 计算损失时将输出转回 float32 以保证数值稳定性 (混合精度训练的常见
136         # 做法)
137         loss = criterion(outputs.float(), labels)
138
139         # 反向传播与优化
140         optimizer.zero_grad() # 清空梯度
141         loss.backward()        # 计算梯度
142         optimizer.step()       # 更新参数
143
144         # 统计损失与精度
145         running_loss += loss.item() * labels.size(0)
146         # 获取预测结果 (最大概率对应的索引)
147         preds = outputs.float().argmax(dim=1)
148         correct += (preds == labels).sum().item()
149         total += labels.size(0)
150
151     # 计算本 epoch 的平均训练精度和损失
152     train_acc = correct / total
153     train_loss = running_loss / total
154     train_acc_history.append(train_acc)
155     train_loss_history.append(train_loss)
156
157     # --- 测试阶段 ---
158     model.eval() # 设置模型为评估模式
159     correct_t = 0; total_t = 0
160     running_loss_t = 0.0
161     with torch.no_grad(): # 验证时不计算梯度, 节省显存和计算资源
162         for inputs, labels in test_loader:
163             inputs = inputs.to(device).half(); labels = labels.to(device)
164             outputs = model(inputs)
165             loss_t = criterion(outputs.float(), labels)
166             running_loss_t += loss_t.item() * labels.size(0)
167             preds = outputs.float().argmax(dim=1)
168             correct_t += (preds == labels).sum().item()
169             total_t += labels.size(0)
170
171     # 计算本 epoch 的平均测试精度和损失
172     test_acc = correct_t / total_t
173     test_loss = running_loss_t / total_t
174     test_acc_history.append(test_acc)
175     test_loss_history.append(test_loss)
176     t.set_postfix(train_acc=f"{train_acc:.4f}", val_loss=f"{test_acc:.4f}",

```



```

176         lr=f'{optimizer.param_groups[0]["lr"]:.6f}')
177
178     # 返回训练/测试指标历史
179     return train_acc_history, test_acc_history, train_loss_history,
180           test_loss_history
181
182 if __name__ == "__main__":
183     # 获取当前脚本目录，方便保存文件
184     script_dir = os.path.dirname(os.path.abspath(__file__))
185     # 加载数据
186     train_loader, test_loader = load_data(script_dir, batch_size=256)
187     # 开始训练
188     train_acc_history, test_acc_history, train_loss_history, test_loss_history =
189         train_and_eval(
190             train_loader, test_loader, epochs=10, lr=0.01
191         )
192     # 绘制结果
193     plot_metrics(script_dir, train_acc_history, test_acc_history,
194                 train_loss_history, test_loss_history)

```

注意：运行此代码前，请确保已安装 torchvision 库。安装时必须严格注意 torchvision 与 PyTorch 的版本对应关系，版本不匹配可能导致无法导入或运行时错误。

以下是昇腾 PyTorch 插件支持的 PyTorch 版本与其对应的 TorchVision 版本对照表：

PyTorch 版本	TorchVision 版本
2.1.0	0.16.0
2.6.0	0.21.0
2.7.1	0.22.0
2.8.0	0.23.0

例如，若当前环境安装的是 PyTorch 2.8.0，则需安装 TorchVision 0.19.0：

```
1 pip install torchvision==0.23.0
```

在昇腾 310B 开发板上运行以上代码，可以得到以下的结果：

```

1 warnings.warn(f"Warning: The {path} owner does not match the current owner.")
2 Epochs: 100%|████████████████████████████████████████| 10/10
   ↳ [01:51<00:00, 11.15s/it, lr=0.010000, train_acc=0.9782, val_loss=0.9800]
3 Saved accuracy & loss curve to: /home/HwHiAiUser/Documents/samples/chapter3/
   ↳ LeNet/accuracy_loss_curve.png

```

并生成如下图所示的部分预测结果可视化：

在本示例中，尽管使用了昇腾 NPU 进行加速，但观察到的训练速度与 CPU 相比并未展现出量级上的优势，这主要是由任务规模与硬件特性共同决定的。MNIST 数据集中的图像分辨率仅为 28x28，且 LeNet-5 网络的参数量和计算深度相对较低，这种轻量级的计算负载难以充分填满 NPU 内部众多的并行计算单元，导致硬件资源处于不饱和状态。与此同时，在 NPU 上执行任务涉及到数据从主机内存到设备显存的搬运开销，以及 CANN 算子库在调用时的指令调度

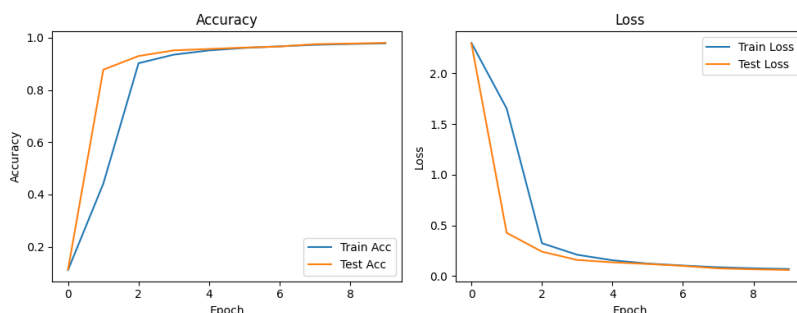


Figure 3.6.: 部分预测结果

与同步耗时，当计算本身的耗时极短时，这些固有的通信与管理开销便成为了性能瓶颈。只有在面对如 ImageNet 等大规模数据集或 ResNet、Transformer 等深层复杂模型时，NPU 强大的张量计算能力才能在抵消掉调度开销后，通过极高的并行度显著缩短整体训练周期。

值得注意的是，本示例对原始 LeNet-5 进行了关键的适配调整。首先，我们将激活函数由 Sigmoid 替换为 ReLU；其次，我们将模型计算精度由 FP32 全精度调整为 FP16 半精度。这是因为在当前硬件环境下，FP32 模型在编译阶段可能遇到兼容性阻碍导致无法运行。同时实验表明，在半精度计算模式下，Sigmoid 函数极易因数值精度不足导致训练效果显著下降，而 ReLU 则能维持更好的数值稳定性与收敛效果。

FP16（半精度浮点数）在表示范围和精度上存在显著限制，这直接影响了激活函数的数值表现。由于 FP16 仅有 16 位，其有效数字位数极少且数值范围较窄，在处理涉及指数运算和除法的 Sigmoid 函数时极易出现问题。在前向计算中，当输入值的绝对值稍大时，FP16 的精度不足以区分计算过程中的微小差别，导致输出迅速进入饱和区并锁定为 0 或 1。更严重的是，在反向传播过程中，Sigmoid 的导数在接近饱和区时会趋近于 0，这些微小的梯度值在 FP16 的精度下极易发生下溢出而变成纯 0，从而导致权重停止更新，使网络训练陷入停滞。

相比之下，ReLU 函数展现出了极佳的数值稳定性。作为一种分段线性函数，ReLU 的计算过程仅涉及简单的比较操作，完全避开了复杂的非线性变换带来的精度损失。在正值区域，ReLU 的导数恒定为 1，这种特性确保了梯度在传播过程中不会因为数值过小而消失，也不会受到 FP16 精度限制的影响。这种天然的鲁棒性使得 ReLU 非常契合 NPU 等强调低精度、高吞吐计算的硬件架构，能够有效保障模型在半精度模式下的收敛效果。

最后值得注意的一点是模型精度转换（FP32 转 FP16）与设备传输的操作顺序。代码实现上通常有两种方式：一种是先在 CPU 端将模型转换为半精度，再传输至 NPU，写法为 `model = LeNet().half().to(device)`；另一种是先将模型传输至 NPU，再进行精度转换，写法为 `model = LeNet().to(device).half()`。虽然这两种写法最终得到的模型状态一致，但在昇腾 310B 的 torch_npu 环境下，其底层执行流程存在差异。实测表明，由于 NPU 涉及图编译过程，优先在 CPU 端完成精度转换（即第一种写法）可以显著减少图编译的开销，从而缩短程序的整体启动时间。不过，一旦编译完成进入稳定运行阶段，两者的计算效率差异则微乎其微。

3.5. torch_npu 插件兼容性测试套件

为了全面评估 torch_npu 在昇腾 310B 上的算子支持度与数值稳定性,我们在 [samples/chapter3](#) → /test 目录下提供了一套兼容性测试脚本。这些测试旨在帮助开发者快速验证当前环境(CANN 版本 + PyTorch 版本 + 硬件) 是否满足模型迁移的基本要求, 并识别潜在的算子不支持或精度异常问题。

3.5.1. 测试目标与验证维度

在利用 torch_npu 插件开发神经网络应用时, 我们注意到该插件在昇腾 310B 上的兼容性仍有提升空间。面对复杂的网络结构或特定的算子组合, 图编译阶段偶尔会出现不稳定的情况。此外, 部分算子在特定精度下的表现也需关注, 例如 Sigmoid 激活函数在 FP16 精度下可能引发模型收敛异常。鉴于此, 针对 AlexNet、VGG、ResNet 等经典网络中常用的核心算子进行全面测试显得尤为必要。为此, 我们开发了一套轻量级的测试套件。

本测试套件的首要目标是验证核心算子在昇腾 NPU 上的支持度与执行稳定性。通过覆盖卷积 (Conv2d)、全连接 (Linear)、矩阵乘法 (MatMul) 等深度学习中最基础且关键的算子, 旨在检查这些指令能否被正确分发至 NPU 后端并顺利执行。这不仅有助于判断当前软硬件环境是否具备运行复杂模型的基础能力, 对于快速排查因算子缺失或版本不兼容导致的运行时错误也至关重要。

其次, 精度对齐是评估迁移质量的关键指标。测试脚本通过对比同一组输入数据在 NPU 与 CPU 上的计算结果, 严格量化两者之间的数值差异。通常情况下, 在 FP32 精度下, 我们期望误差控制在极小的范围内 (如小于 $1e-4$), 以确保模型迁移后推理与训练结果的可靠性。这种对比验证能有效发现因硬件架构差异或算子实现细节不同而引发的数值漂移问题。

最后, 测试还致力于探索不同边界条件下的表现。这包括验证 FP16、FP32 等不同数据类型的兼容性, 特别是针对昇腾 310B 对 FP64 (双精度) 支持有限的特性进行实测, 以及评估在不同 Tensor 形状和维度下算子的内存占用与计算效率。通过这些多维度的测试, 开发者可以更清晰地了解硬件的性能边界与最佳实践配置。

3.5.2. 测试套件结构详解

本测试套件基于 pytest 框架构建, 针对不同精度 (FP16/FP32) 进行了分层验证, 主要包含以下核心组成部分:

1. 测试环境配置 (conftest.py)

conftest.py 脚本负责构建稳健的测试环境。它利用 pytest 的钩子函数 (Hooks) 机制, 实现了一套实时日志记录系统。针对嵌入式开发板在长时间批量测试中可能出现的网络波动或系统挂起导致终端输出丢失的问题, 该脚本定义了 pytest_runtest_setup 和 pytest_runtest_logreport → 等钩子。这些钩子确保在每个测试用例开始和结束时, 立即将测试节点 ID 及运行结果(PASSED/-FAILED) 写入 pytest_realtime_log.txt 文件。为了防止系统崩溃导致数据丢失, 写入操作调

用了 `os.fsync` 强制刷新磁盘。此外, `pytest_terminal_summary` 会在测试结束时统计并输出通过、失败及跳过的用例总数, 生成详尽且持久化的测试报告, 极大便利了无人值守场景下的故障排查。

2. 基础算子运算测试 (`test_float16_ops.py` / `test_float32_ops.py`)

基础算子测试剥离了复杂的神经网络层封装, 直接验证深度学习中最底层的张量加法 (Add) 和矩阵乘法 (MatMul)。这两个脚本分别对应 FP16 (半精度) 和 FP32 (单精度), 旨在直接检验昇腾 NPU 硬件底层的算术逻辑单元 (ALU) 与矩阵计算单元 (Cube Core) 的计算正确性。通过绕过 `nn.Module` 等高层抽象, 这些测试作为排查底层硬件故障或驱动问题的最小功能单元, 有助于区分框架层面的算子映射错误与硬件层面的计算异常。

- **`test_float32_ops.py`:** 关注高精度计算的一致性。它在 CPU 和 NPU 上执行相同的 FP32 运算, 要求两者误差控制在极小范围内 (如加法 $1e-4$, 矩阵乘法 $1e-3$), 确保 NPU 单精度计算路径的数值稳定。
- **`test_float16_ops.py`:** 针对边缘计算常用的半精度推理场景。采用“CPU FP32 计算作为真值”的策略, 即在 CPU 上使用高精度 FP32 运算, NPU 使用 FP16 运算, 最后将 NPU 结果转回 FP32 进行对比。这种方式既验证了 NPU 的半精度计算能力, 又通过放宽容差 ($1e-2$) 合理包容了精度量化带来的正常误差。

3. 神经网络层测试 (`test_nn_layers_float16.py` / `test_nn_layers_float32.py`)

这是测试套件的核心, 旨在全面验证常用深度学习算子在昇腾 NPU 上的表现。测试覆盖了特征提取与分类的关键组件: * **核心层**: `nn.Linear` 和 `nn.Conv2d` 的测试用例涵盖了 LeNet、AlexNet、VGG 及 ResNet 等经典架构的典型参数配置 (输入尺寸、卷积核大小、步长及填充)。* **激活与归一化**: 囊括 ReLU、Sigmoid、Tanh、LeakyReLU、GELU、Softmax 等激活函数及 BatchNorm2d, 确保非线性变换与数据分布调整的准确性。* **池化与辅助层**: 验证 AvgPool2d 和 MaxPool2d 的基本功能及 `count_include_pad`、`ceil_mode` 等特定参数, 同时包含 Dropout 和 Flatten 的行为验证。

为了适应不同场景, 套件提供了双重验证策略: * **`test_nn_layers_float32.py`**: 侧重高精度数值一致性, 要求 NPU 结果与 CPU 基准误差极小 ($1e-3$), 确保推理精确度。* **`test_nn_layers_float16.py`**: 模拟混合精度推理, 输入和模型转为半精度在 NPU 执行, 结果转回 FP32 对比。考虑到量化损失, 容差适度放宽 ($1e-2$ 至 $1e-1$), 重点验证低精度模式下的逻辑正确性与功能完备性。此外, 为防止嵌入式设备内存溢出 (OOM), 每个测试类均实现了 `teardown_method`, 在用例执行后自动清理 NPU 缓存。

3.5.3. 测试结果摘要

运行该测试套件前, 请确保在 npu 虚拟环境中已安装 `pytest`:

```
1 pip3 install pytest
```

随后在终端进入相应的 `test` 文件夹并运行:

```
1 pytest -v ./
```

在昇腾 310B (CANN 8.3.RC1 + PyTorch 2.8.0) 环境下, 典型测试结果如下:

```
1 ===== test session starts =====
2 ...
3 FAILED test_nn_layers_float16.py::TestNNLayersFloat16::
  ↳ test_avgpool_float16_default_behavior[3-1-1]
4 FAILED test_nn_layers_float16.py::TestNNLayersFloat16::
  ↳ test_avgpool_float16_default_behavior[3-2-1]
5 FAILED test_nn_layers_float16.py::TestNNLayersFloat16::test_avgpool_float16[True
  ↳ -3-1-1]
6 FAILED test_nn_layers_float16.py::TestNNLayersFloat16::test_avgpool_float16[True
  ↳ -3-2-1]
7 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
  ↳ test_maxpool_float32_default_behavior[2-2-0]
8 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
  ↳ test_maxpool_float32_default_behavior[3-2-0]
9 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
  ↳ test_maxpool_float32_default_behavior[3-2-1]
10 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::test_maxpool_float32[
  ↳ False-2-2-0]
11 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::test_maxpool_float32[
  ↳ False-3-2-0]
12 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::test_maxpool_float32[
  ↳ False-3-2-1]
13 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::test_maxpool_float32[True
  ↳ -2-2-0]
14 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::test_maxpool_float32[True
  ↳ -3-2-0]
15 FAILED test_nn_layers_float32.py::TestNNLayersFloat32::test_maxpool_float32[True
  ↳ -3-2-1]
16 ===== 76 passed, 13 failed in 123.45s =====
```

测试日志显示, 绝大多数基础算子 (Add, MatMul) 和核心网络层 (Linear, Conv2d, Batch-Norm, Activations) 在 FP16 和 FP32 模式下均通过验证, 证明 NPU 基本计算功能正常。但测试也暴露了两个显著异常, 需重点关注:

- FP16 平均池化层 (AvgPool) 的边界精度问题在 test_nn_layers_float16.py 的测试中, AvgPool2d 算子表现出特定的边界精度异常, 具体体现为当 kernel_size=3 且 padding=1 时测试均告失败, 而 kernel_size=2 且无填充时则能顺利通过。这一现象表明 NPU 在处理半精度 (FP16) 池化边界填充 (Padding) 时, 累加计算的精度损失可能超出了预设容差范围, 或者底层硬件对 Padding 区域 “0” 值的处理逻辑与 CPU 存在微小差异。因此, 开发者在推理代码中若使用 FP16 模式且包含带 Padding 的奇数核 AvgPool 层, 需特别警惕潜在的精度下降风险。
- FP32 最大池化层 (MaxPool) 的系统性失效在 test_nn_layers_float32.py 中, 所有 MaxPool2d 测试用例 (共 9 个) 全部失败, 与 FP16 模式下全部通过的表现形成鲜明对比, 这揭示了一个严重的系统性故障。FP16 正常而 FP32 全面失败, 极可能是当前版本 CANN 驱动或 PyTorch 插件在 FP32 格式 MaxPool 算子实现上存在 Bug, 亦或是数据排布未对齐导致计算错误甚至产生 NaN/Inf。鉴于此为高优先级阻碍性问题 (Blocker),

建议暂时避免在 NPU 上使用 FP32 格式的 MaxPool，或将其回退至 CPU 执行，直至驱动版本修复。

开发者在迁移自定义模型前，建议优先运行此测试套件以排查环境潜在问题。

3.6. 现代卷积神经网络的移植

3.6.1. AlexNet

AlexNet 是由 Alex Krizhevsky、Ilya Sutskever 和 Geoffrey Hinton 在 2012 年提出的卷积神经网络。它在当年的 ImageNet 大规模视觉识别挑战赛（ILSVRC）中以压倒性的优势夺冠，将 Top-5 错误率降低到了 15.3%，远超第二名的 26.2%。AlexNet 的成功标志着深度学习在计算机视觉领域的统治地位的确立，引发了卷积神经网络（CNN）研究的热潮。

AlexNet 网络结构

AlexNet 的网络结构比之前的 LeNet 更深、更宽。AlexNet 主要包含以下特点：1. **层数**：包含 8 层神经网络，其中前 5 层是卷积层，后 3 层是全连接层。2. **激活函数**：首次在 CNN 中成功使用了 ReLU（Rectified Linear Unit）激活函数，解决了深层网络训练时的梯度消失问题，并加速了收敛。3. **Dropout**：在全连接层引入了 Dropout 技术，随机忽略一部分神经元，有效防止了过拟合。4. **数据增强**：使用了图像翻转、裁剪等数据增强技术来扩充数据集。5. **多 GPU 训练**：由于当时显存限制，AlexNet 被设计为在两块 GTX 580 GPU 上并行训练。6. **重叠池化（Overlapping Pooling）**：AlexNet 使用了重叠的最大池化（Max Pooling）。传统的池化层通常是不重叠的（步长等于池化核大小），而 AlexNet 使用了 3×3 的池化核和 2 的步长，这种重叠池化稍微减轻了过拟合。

原始的 AlexNet 是为 ImageNet 数据集设计的，但在昇腾 310B 开发板上使用庞大的 ImageNet 进行训练会消耗大量时间与算力。ImageNet 是一个庞大的视觉数据库，包含超过 1400 万张标注图片，涵盖 2 万多个类别，其常用的 ILSVRC 子集也有 1000 个类别和 128 万张训练图片，且图片分辨率较高（通常裁剪为 224×224 ）。相比之下，CIFAR-10 数据集规模较小，仅包含 60000 张 32×32 的彩色图像，分为 10 个类别（如飞机、汽车、鸟类等），每类 6000 张。虽然 CIFAR-10 的图像分辨率和类别数远小于 ImageNet，但它保留了自然图像的复杂性，非常适合用于快速验证模型结构和硬件移植的可行性。为了更高效地验证 AlexNet 在昇腾 310B 上的迁移性能，本节改用 CIFAR-10 数据集进行实验。

为此，我们需要对 AlexNet 的结构进行适配性修改：首先，将网络输入尺寸从 $3 \times 224 \times 224$ 调整为 $3 \times 32 \times 32$ ，并将全连接层的输出类别数从 1000 改为 10。

此外，在上一节的测试中我们发现，昇腾 310B 的 PyTorch 插件目前尚未支持 MaxPool2d 算子，因此我们将该算子替换为 AvgPool2d。这一改动虽然是为了适配硬件限制，但也会对模型特性产生一定影响：**最大池化（Max Pooling）** 倾向于提取图像中最显著的特征（如纹理、边缘），具有一定的平移不变性，通常能保留较强的特征响应；而 **平均池化（Average Pooling）**

则是计算区域内的平均值，倾向于保留背景信息和平滑图像，可能会模糊掉一些尖锐的特征细节。在 CIFAR-10 这种低分辨率数据集上，使用平均池化可能会导致模型对关键特征的捕捉能力略微下降，从而轻微影响最终的分类准确率，但它能保证模型在当前硬件环境下的顺利运行。修改后的 AlexNet 网络结构如下图所示：

AlexNet 的代码实现

为了验证 AlexNet 在昇腾 310B 上移植的可行性，我们首先建立了一个测试流程。利用 pytest 框架，我们对网络的每一层结构进行了逐一验证，重点对比了 torch_npu（NPU 后端）与 CPU 计算结果的一致性，同时检测了网络算子的硬件兼容性。为此，我们编写了专门的测试例程，分别验证了 AlexNet 在 FP16（半精度）和 FP32（全精度）模式下的运行正确性。相关测试代码均位于 samples/chapter2/AlexNet/test 目录下。经 pytest 测试确认，无论是 FP32 还是 FP16 精度，模型在昇腾 310B 上的计算结果均正确无误。尽管前述的逐层单元测试确认了 AlexNet 在 FP16 和 FP32 精度下的计算正确性，但在实际的完整模型训练中，硬件资源的限制成为了关键因素。实测表明，在 **8T 算力 8GB 内存版本** 的昇腾 310B 开发板上进行 FP32 全精度训练时，由于图编译阶段对内存消耗较大，极易遭遇编译失败的问题；而在 **8T 算力 16GB 内存版本** 的开发板上，FP32 全精度训练则能顺利跑通。基于上述适配后的网络结构，以下提供了针对 CIFAR-10 数据集的完整 FP16 半精度训练与验证代码：

```

1 import os
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from torch.utils.data import TensorDataset, DataLoader
7 from torchvision import datasets
8 import matplotlib.pyplot as plt
9 from tqdm.auto import tqdm, trange
10 from torchvision import transforms
11
12 device = torch.device("npu")
13
14 class AlexNet(nn.Module):
15     def __init__(self, num_classes=10):
16         super(AlexNet, self).__init__()
17         self.features = nn.Sequential(
18             # Layer 1
19             nn.Conv2d(3, 96, kernel_size=11, stride=4, padding=1),
20             nn.ReLU(inplace=True),
21             nn.AvgPool2d(kernel_size=3, stride=2),
22             # Layer 2
23             nn.Conv2d(96, 256, kernel_size=5, padding=2),
24             nn.ReLU(inplace=True),
25             nn.AvgPool2d(kernel_size=3, stride=2),
26             # Layer 3
27             nn.Conv2d(256, 384, kernel_size=3, padding=1),
28             nn.ReLU(inplace=True),
29             # Layer 4
30             nn.Conv2d(384, 384, kernel_size=3, padding=1),
31             nn.ReLU(inplace=True),

```

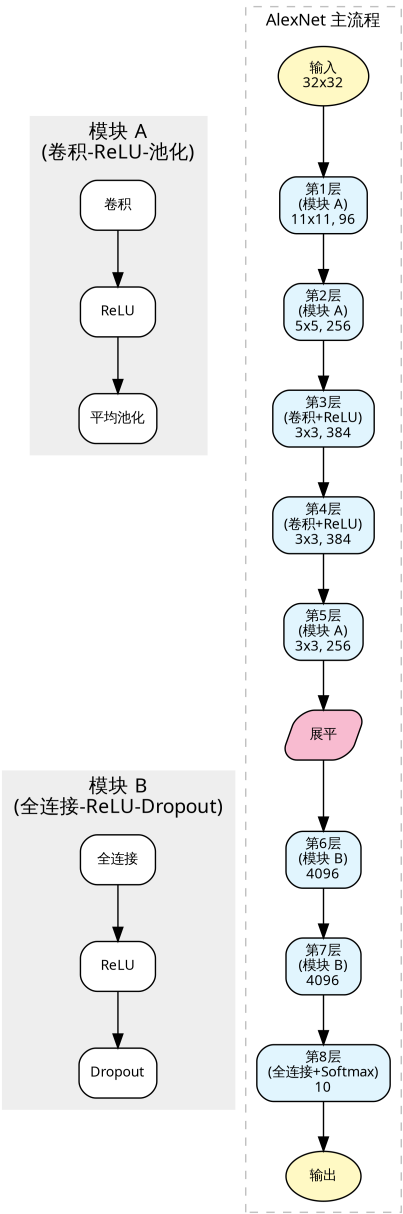


Figure 3.7.: AlexNet

```

32         # Layer 5
33         nn.Conv2d(384, 256, kernel_size=3, padding=1),
34         nn.ReLU(inplace=True),
35         nn.AvgPool2d(kernel_size=3, stride=2),
36     )
37
38     self.classifier = nn.Sequential(
39         nn.Flatten(),
40         nn.Linear(6400, 4096),
41         nn.ReLU(inplace=True),
42         nn.Dropout(0.5),
43         nn.Linear(4096, 4096),
44         nn.ReLU(inplace=True),
45         nn.Dropout(0.5),
46         nn.Linear(4096, num_classes),
47     )
48
49     def forward(self, x):
50         x = self.features(x)
51         x = self.classifier(x)
52         return x
53
54 def load_data(script_dir, batch_size=4):
55     """
56     加载 CIFAR-10 数据集并进行预处理
57     """
58     # 引入 transforms 用于图像变换
59
60     # 数据存储路径
61     data_root = os.path.join(script_dir, "data")
62
63     # 定义预处理流程:
64     # 1. Resize: 将 CIFAR-10 的 32x32 图像调整为 224x224, 以匹配 AlexNet 输入要求
65     # 2. ToTensor: 将图像数据转换为 Tensor (C, H, W), 并将像素值归一化到 [0, 1]
66     # 3. Normalize: 对 RGB 三个通道进行标准化
67     transform = transforms.Compose([
68         transforms.Resize((224, 224)),
69         transforms.ToTensor(),
70         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
71         transforms.Lambda(lambda x: x.half())
72     ])
73
74     # 使用 torchvision.datasets 下载并加载 CIFAR-10 数据
75     # 传入 transform 参数, DataLoader 读取数据时会自动进行 Resize 和 ToTensor
76     train_ds = datasets.CIFAR10(data_root, train=True, download=True, transform=
77         transform)
78     test_ds = datasets.CIFAR10(data_root, train=False, download=True, transform=
79         transform)
80
81     # 构建 DataLoader
82     # 相比原代码一次性加载到内存, 这里使用 lazy loading 方式, 避免 Resize 后占用
83     # 过多内存
84     train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
85     val_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False)
86     return train_loader, val_loader
87
88 def train_and_eval(train_loader, val_loader, epochs=10, lr=0.01):

```



```

86 """
87 执行模型训练与验证循环
88 """
89 # 初始化模型并迁移至 NPU 设备
90 model = AlexNet().half().to(device)
91 # 定义损失函数：交叉熵损失，适用于多分类问题
92 criterion = nn.CrossEntropyLoss()
93 # 定义优化器：随机梯度下降 (SGD)
94 optimizer = optim.SGD(model.parameters(), lr=lr)
95
96 train_acc_history = []
97 val_acc_history = []
98 train_loss_history = []
99 val_loss_history = []
100
101 # 使用 trange 展示总进度条
102 t = trange(epochs, desc="Epochs")
103 for epoch in t:
104     # --- 训练阶段 ---
105     model.train() # 设置模型为训练模式 (启用 Dropout, BatchNorm 等)
106     correct = 0; total = 0
107     running_loss = 0.0
108     for inputs, labels in train_loader:
109         # 数据迁移至 NPU, 保持 float32
110         inputs = inputs.to(device); labels = labels.to(device)
111
112         # 前向传播
113         outputs = model(inputs)
114         # 计算损失
115         loss = criterion(outputs, labels)
116
117         # 反向传播与优化
118         optimizer.zero_grad() # 清空梯度
119         loss.backward()        # 计算梯度
120         optimizer.step()       # 更新参数
121
122         # 统计损失与精度
123         running_loss += loss.item() * labels.size(0)
124         # 获取预测结果 (最大概率对应的索引)
125         preds = outputs.argmax(dim=1)
126         correct += (preds == labels).sum().item()
127         total += labels.size(0)
128
129     # 计算本 epoch 的平均训练精度和损失
130     train_acc = correct / total
131     train_loss = running_loss / total
132     train_acc_history.append(train_acc)
133     train_loss_history.append(train_loss)
134
135     # --- 验证阶段 ---
136     model.eval() # 设置模型为评估模式
137     correct_v = 0; total_v = 0
138     running_loss_v = 0.0
139     with torch.no_grad(): # 验证时不计算梯度, 节省显存和计算资源
140         for inputs, labels in val_loader:
141             inputs = inputs.to(device); labels = labels.to(device)
142             outputs = model(inputs)
143             loss_v = criterion(outputs, labels)

```

```

144         running_loss_v += loss_v.item() * labels.size(0)
145         preds = outputs.argmax(dim=1)
146         correct_v += (preds == labels).sum().item()
147         total_v += labels.size(0)
148
149         # 计算本 epoch 的平均验证精度和损失
150         val_acc = correct_v / total_v
151         val_loss = running_loss_v / total_v
152         val_acc_history.append(val_acc)
153         val_loss_history.append(val_loss)
154         t.set_postfix(train_acc=f"{train_acc:.4f}", val_loss=f"{val_loss:.4f}",
155             ↪ val_acc=f"{val_acc:.4f}", lr=f'{optimizer.param_groups[0]["lr"
156             ↪ ":.6f}'))
157
158     # 返回训练/验证指标历史
159     return train_acc_history, val_acc_history, train_loss_history,
160         ↪ val_loss_history
161
162 def plot_metrics(script_dir, train_acc, val_acc, train_loss, val_loss):
163     """
164     绘制训练和验证的精度与损失曲线
165     """
166     plt.figure(figsize=(12, 5))
167
168     # 绘制精度曲线
169     plt.subplot(1, 2, 1)
170     plt.plot(train_acc, label='Train Accuracy')
171     plt.plot(val_acc, label='Validation Accuracy')
172     plt.title('Accuracy over Epochs')
173     plt.xlabel('Epoch')
174     plt.ylabel('Accuracy')
175     plt.legend()
176
177     # 绘制损失曲线
178     plt.subplot(1, 2, 2)
179     plt.plot(train_loss, label='Train Loss')
180     plt.plot(val_loss, label='Validation Loss')
181     plt.title('Loss over Epochs')
182     plt.xlabel('Epoch')
183     plt.ylabel('Loss')
184     plt.legend()
185
186     # 保存图片
187     save_path = os.path.join(script_dir, "training_metrics.png")
188     plt.savefig(save_path)
189     print(f"Metrics plot saved to {save_path}")
190
191 if __name__ == "__main__":
192     # 获取当前脚本目录，方便保存文件
193     script_dir = os.path.dirname(os.path.abspath(__file__))
194
195     # 加载数据
196     train_loader, val_loader = load_data(script_dir, batch_size=8)
197     # 开始训练
198     train_acc_history, val_acc_history, train_loss_history, val_loss_history =
199         ↪ train_and_eval(
200             train_loader, val_loader, epochs=10, lr=0.01
201         )

```

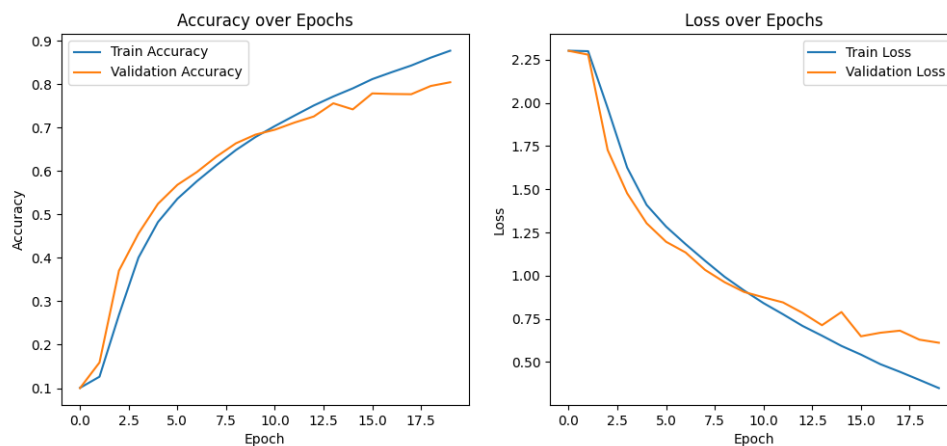


Figure 3.8.: training metrics NPU FP16

```

198 # 绘制结果
199 plot_metrics(script_dir, train_acc_history, val_acc_history,
    ↪ train_loss_history, val_loss_history)

```

在前面章节创建的 npu 虚拟环境中运行上述代码，昇腾 310B 开发板将输出如下执行结果：

```

1 Epochs: 100%|██████████|
    ↪
    ↪ 10/10 [2:26:50<00:00, 881.05s/it, lr=0.010000, train_acc=0.8195, val_acc
    ↪ =0.7798, val_loss=0.6680]

```

实验结果对比分析

为了评估昇腾 310B 在运行 AlexNet 模型时的实际性能，我们将上述代码部署至昇腾 310B 开发板（NPU），并引入高性能桌面显卡 GeForce 5090D 作为对比基准。实验设计涵盖了昇腾 310B 与 GeForce 5090D 在 FP16 半精度及 FP32 全精度模式下的表现，以期提供客观的性能参考。

需要特别说明的是，全精度（FP32）训练对内存资源有较高要求。实测表明，**8GB 内存版本**的昇腾 310B 开发板在执行 AlexNet 的 FP32 训练任务时，会因内存不足触发图编译错误（OOM）。因此，若需在昇腾 310B 上进行 FP32 训练，必须使用 **16GB 内存版本**；而 FP16 半精度训练则能有效降低显存占用，在 8GB 版本上即可顺利运行。

各实验配置的终端输出日志记录如下：

- Ascend 310B (FP16)

```

1 Epochs: 100%|██████████| 20/20 [4:00:05<00:00, 720.29s/it, lr=0.010000,
    ↪ train_acc=0.8770, val_acc=0.8044, val_loss=0.6110]

```

- Ascend 310B (FP32)

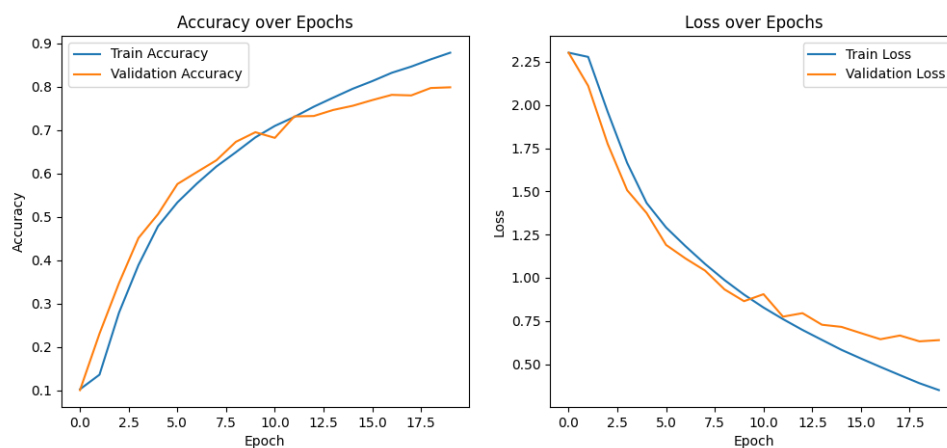


Figure 3.9.: training metrics cuda FP32

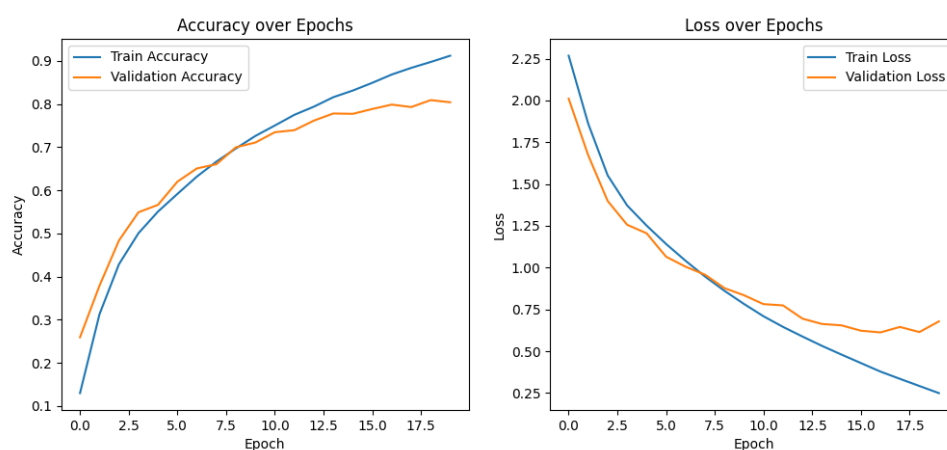


Figure 3.10.: training metrics cuda FP32

```
1 Epochs: 100%|██████████| 20/20 [10:52:39<00:00, 1957.99s/it, lr=0.010000,
  ↳ train_acc=0.9693, val_acc=0.8219, val_loss=0.7721]
```

• GeForce 5090D (FP16)

```
1 Epochs: 100%|██████████| 20/20 [08:38<00:00, 25.93s/it, lr
  ↳ =0.010000, train_acc=0.8787, val_acc=0.7988, val_loss=0.6389]
```

• GeForce 5090D (FP32)

```
1 Epochs: 100%|██████████| 20/20 [11:01<00:00, 33.08s/it, lr
  ↳ =0.010000, train_acc=0.9688, val_acc=0.8252, val_loss=0.7670]
```

详细的实验数据对比汇总如下表所示：

硬件平台	精度模式	总耗时 (20 Epochs)	单 Epoch 平均耗时	训练集精度 (Train Acc)	验证集精度 (Val Acc)	验证集损失 (Val Loss)
Ascend 310B	FP16 (半精度)	240.1 min	12.0 min	87.70%	80.44%	0.6110
Ascend 310B	FP32 (全精度)	652.7 min	32.6 min	96.93%	82.19%	0.7721
GeForce 5090D	FP16 (半精度)	8.6 min	0.43 min	87.87%	79.88%	0.6389
GeForce 5090D	FP32 (全精度)	11.0 min	0.55 min	96.88%	82.52%	0.7670

首先, 从训练速度来看, 虽然昇腾 310B 的训练耗时明显长于桌面级高性能显卡, 但这完全符合其硬件定位。昇腾 310B 本质上是一款面向边缘计算场景的低功耗 AI 芯片, 其设计初衷是在有限的功耗预算下提供高效的推理能力, 而非大规模模型训练。尽管如此, 实验结果表明昇腾 310B 依然稳定地完成了整个训练流程, 这有力地证明了它不仅能够胜任推理任务, 也具备在端侧进行轻量级训练或模型微调的潜力, 为边缘侧的持续学习提供了硬件基础。

其次, 在精度表现方面, 我们可以观察到不同精度与不同硬件平台之间的高度一致性。首先, 在 FP32 全精度模式下, 昇腾 310B 的验证集准确率达到 82.19%, 与基准设备 GeForce 5090D 的 82.52% 极为接近。这充分验证了昇腾 310B 在全精度计算逻辑上的正确性, 未出现因架构差异导致的数值漂移。

其次, 对比同平台的 FP16 与 FP32 结果, 精度下降是普遍存在的现象: 基准设备准确率下降了约 2.6% (从 82.52% 降至 79.88%), 而昇腾 310B 下降了约 1.7% (从 82.19% 降至 80.44%)。这种精度损失属于半精度动态范围压缩带来的正常物理特性, 但昇腾 310B 的降幅相对更小, 说明其半精度算子实现具有良好的数值稳定性。

最后, 横向对比 FP16 模式, 昇腾 310B (80.44%) 的表现与基准设备 (79.88%) 几乎持平甚至略优, 进一步证明了将模型移植到昇腾 NPU 并采用半精度加速时, 能够很好地保持模型的收敛特性和最终性能。

综上所述, 实验结果证实了昇腾 310B 能够正确且有效地执行 AlexNet 的训练任务, 且精度表现符合预期。虽然其训练速度无法与高端桌面显卡相提并论, 但考虑到其低功耗特性和边缘部署的定位, 该性能足以满足小规模数据集的迁移学习或现场微调需求。对于大规模的深度学习训练任务, 更合理的流程应当是在高性能服务器 (如昇腾 910 或 GPU 集群) 上完成模型训练, 随后再将训练好的模型量化或转换至昇腾 310B 上进行高效的推理部署。

3.6.2. VGG

VGG (Visual Geometry Group) 网络是由牛津大学的 Karen Simonyan 和 Andrew Zisserman 在 2014 年提出的。它在当年的 ILSVRC 挑战赛中取得了定位任务第一名和分类任务第二名的优异成绩。VGG 的主要贡献在于证明了使用小卷积核并增加网络深度能够有效提升模型性能。

VGG 的网络结构

VGG 的基本结构特点非常简洁：**1. 统一的小卷积核**：整个网络全部使用 3×3 的卷积核和 2×2 的最大池化层。通过堆叠多个 3×3 卷积层来获得与大卷积核相同的感受野（例如 2 个 3×3 相当于 1 个 5×5 ），同时减少了参数量并增加了非线性变换。**2. 深度结构**：相比 AlexNet，VGG 的层数大幅增加，常用的配置包括 VGG-16 和 VGG-19，分别拥有 16 层和 19 层带权重的层。**3. 通道数翻倍**：在每个池化层之后，特征图的通道数通常会翻倍，直到达到 512。

VGG 的代码实现

为了灵活构建不同深度的 VGG 网络（如 VGG11, VGG13, VGG16, VGG19），代码采用了配置驱动的设计模式。首先，定义一个字典 `cfg`，其中键为网络名称，值为列表。列表中的数字表示卷积层的输出通道数，字符 '`M`' 则代表最大池化层。这种设计使得网络结构的定义变得直观且易于修改。

接着，在 VGG 类的初始化函数中，通过调用 `_make_layers` 方法解析配置列表。该方法遍历配置列表：遇到数字时，构建一个 3×3 的卷积层（`padding=1` 以保持尺寸）、Batch Normalization 层和 ReLU 激活函数，并更新输入通道数；遇到 '`M`' 时，则添加一个 2×2 的池化层。这里特别针对 CIFAR-10 数据集（ 32×32 ）进行了适配，移除了原始 VGG 最后的三个全连接层，改用一个单一的线性层作为分类器，并且为了适配昇腾 310B 目前对 MaxPool 的支持限制，将部分池化操作调整为 AvgPool。

具体的 VGG16 的代码实现如下：

```
1 import os
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from torch.utils.data import TensorDataset, DataLoader
7 from torchvision import datasets
8 import matplotlib.pyplot as plt
9 from tqdm.auto import tqdm, trange
10 from torchvision import transforms
11
12 device = torch.device("npu")
13
14 cfg = {
15     'VGG11': [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],
16     'VGG13': [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512,
17               ↪ 512, 'M'],
```

```

17 'VGG16': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M'
    ↳ 512, 512, 512, 'M'],
18 'VGG19': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512,
    ↳ 512, 512, 'M', 512, 512, 512, 'M'],
19 }
20
21 class VGG(nn.Module):
22     def __init__(self, vgg_name, num_classes=10):
23         super(VGG, self).__init__()
24         self.features = self._make_layers(cfg[vgg_name])
25         self.classifier = nn.Linear(512, num_classes)
26
27     def forward(self, x):
28         out = self.features(x)
29         out = out.view(out.size(0), -1)
30         out = self.classifier(out)
31         return out
32
33     def _make_layers(self, cfg):
34         layers = []
35         in_channels = 3
36         for x in cfg:
37             if x == 'M':
38                 layers += [nn.AvgPool2d(kernel_size=2, stride=2)]
39             else:
40                 layers += [nn.Conv2d(in_channels, x, kernel_size=3, padding=1),
41                           nn.BatchNorm2d(x),
42                           nn.ReLU(inplace=True)]
43                 in_channels = x
44         layers += [nn.AvgPool2d(kernel_size=1, stride=1)]
45         return nn.Sequential(*layers)
46
47 def load_data(script_dir, batch_size=4):
48     """
49     加载 CIFAR-10 数据集并进行预处理
50     """
51     # 引入 transforms 用于图像变换
52
53     # 数据存储路径
54     data_root = os.path.join(script_dir, "data")
55
56     # 定义预处理流程:
57     # 1. ToTensor: 将图像数据转换为 Tensor (C, H, W), 并将像素值归一化到 [0, 1]
58     # 2. Normalize: 对 RGB 三个通道进行标准化
59     # 注意: 针对CIFAR-10优化的VGG使用原始32x32输入, 不需要Resize到224
60     transform = transforms.Compose([
61         transforms.ToTensor(),
62         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
63     ])
64
65     # 使用 torchvision.datasets 下载并加载 CIFAR-10 数据
66     # 传入 transform 参数, DataLoader 读取数据时会自动进行 Resize 和 ToTensor
67     train_ds = datasets.CIFAR10(data_root, train=True, download=True, transform=
        ↳ transform)
68     test_ds = datasets.CIFAR10(data_root, train=False, download=True, transform=
        ↳ transform)
69
70     # 构建 DataLoader

```

```

71 # 相比原代码一次性加载到内存，这里使用 lazy loading 方式，避免 Resize 后占用
    ↳ 过多内存
72 train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
73 val_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False)
74 return train_loader, val_loader
75
76 def train_and_eval(train_loader, val_loader, epochs=10, lr=0.01):
77     """
78     执行模型训练与验证循环
79     """
80     # 初始化模型并迁移至 NPU 设备
81     model = VGG('VGG16').to(device)
82     # 定义损失函数：交叉熵损失，适用于多分类问题
83     criterion = nn.CrossEntropyLoss()
84     # 定义优化器：随机梯度下降 (SGD)
85     optimizer = optim.SGD(model.parameters(), lr=lr)
86
87     train_acc_history = []
88     val_acc_history = []
89     train_loss_history = []
90     val_loss_history = []
91
92     # 使用 trange 展示总进度条
93     t = trange(epochs, desc="Epochs")
94     for epoch in t:
95         # --- 训练阶段 ---
96         model.train() # 设置模型为训练模式（启用 Dropout, BatchNorm 等）
97         correct = 0; total = 0
98         running_loss = 0.0
99         for inputs, labels in train_loader:
100             # 数据迁移至 NPU，保持 float32
101             inputs = inputs.to(device); labels = labels.to(device)
102
103             # 前向传播
104             outputs = model(inputs)
105             # 计算损失
106             loss = criterion(outputs, labels)
107
108             # 反向传播与优化
109             optimizer.zero_grad() # 清空梯度
110             loss.backward()        # 计算梯度
111             optimizer.step()       # 更新参数
112
113             # 统计损失与精度
114             running_loss += loss.item() * labels.size(0)
115             # 获取预测结果（最大概率对应的索引）
116             preds = outputs.argmax(dim=1)
117             correct += (preds == labels).sum().item()
118             total += labels.size(0)
119
120         # 计算本 epoch 的平均训练精度和损失
121         train_acc = correct / total
122         train_loss = running_loss / total
123         train_acc_history.append(train_acc)
124         train_loss_history.append(train_loss)
125
126         # --- 验证阶段 ---
127         model.eval() # 设置模型为评估模式

```



```

128     correct_v = 0; total_v = 0
129     running_loss_v = 0.0
130     with torch.no_grad(): # 验证时不计算梯度, 节省显存和计算资源
131         for inputs, labels in val_loader:
132             inputs = inputs.to(device); labels = labels.to(device)
133             outputs = model(inputs)
134             loss_v = criterion(outputs, labels)
135             running_loss_v += loss_v.item() * labels.size(0)
136             preds = outputs.argmax(dim=1)
137             correct_v += (preds == labels).sum().item()
138             total_v += labels.size(0)
139
140     # 计算本 epoch 的平均验证精度和损失
141     val_acc = correct_v / total_v
142     val_loss = running_loss_v / total_v
143     val_acc_history.append(val_acc)
144     val_loss_history.append(val_loss)
145     t.set_postfix(train_acc=f"{train_acc:.4f}", val_loss=f"{val_loss:.4f}",
146                  ↪ val_acc=f"{val_acc:.4f}", lr=f'{optimizer.param_groups[0]["lr"
147                  ↪ ":.6f}'))
148
149     # 返回训练/验证指标历史
150     return train_acc_history, val_acc_history, train_loss_history,
151           ↪ val_loss_history
152
153 def plot_metrics(script_dir, train_acc, val_acc, train_loss, val_loss):
154     """
155     绘制训练和验证的精度与损失曲线
156     """
157     plt.figure(figsize=(12, 5))
158
159     # 绘制精度曲线
160     plt.subplot(1, 2, 1)
161     plt.plot(train_acc, label='Train Accuracy')
162     plt.plot(val_acc, label='Validation Accuracy')
163     plt.title('Accuracy over Epochs')
164     plt.xlabel('Epoch')
165     plt.ylabel('Accuracy')
166     plt.legend()
167
168     # 绘制损失曲线
169     plt.subplot(1, 2, 2)
170     plt.plot(train_loss, label='Train Loss')
171     plt.plot(val_loss, label='Validation Loss')
172     plt.title('Loss over Epochs')
173     plt.xlabel('Epoch')
174     plt.ylabel('Loss')
175     plt.legend()
176
177     # 保存图片
178     save_path = os.path.join(script_dir, "training_metrics.png")
179     plt.savefig(save_path)
180     print(f"Metrics plot saved to {save_path}")
181
182 if __name__ == "__main__":
183     # 获取当前脚本目录, 方便保存文件
184     script_dir = os.path.dirname(os.path.abspath(__file__))

```

```

183 # 加载数据
184 train_loader, val_loader = load_data(script_dir, batch_size=16)
185 # 开始训练
186 train_acc_history, val_acc_history, train_loss_history, val_loss_history =
    ↳ train_and_eval(
187     train_loader, val_loader, epochs=10, lr=0.01
188 )
189 # 绘制结果
190 plot_metrics(script_dir, train_acc_history, val_acc_history,
    ↳ train_loss_history, val_loss_history)

```

实验结果分析

在本节的实验设计中，我们仅针对 FP32（全精度）模式进行了测试与分析，未涵盖 FP16（半精度）模式。这主要是出于对数值稳定性的考量：VGG 网络广泛使用了 Batch Normalization (BN) 层，该层在计算均值和方差时对精度极其敏感。如果强制使用 FP16 进行全网训练，BN 层的统计量极易出现数值溢出或下溢，导致训练过程不稳定甚至不收敛。虽然自动混合精度（AMP）技术通常能缓解这一问题，但鉴于当前昇腾 310B 的 torch_npu 插件在混合精度算子调度上的支持尚在完善中，为确保实验结果的可靠性与可复现性，我们统一选用了 FP32 精度进行验证。

为了提供客观的性能参照，我们将同一套代码逻辑部署到了两种截然不同的硬件平台上：一是面向边缘计算的昇腾 310B 开发板，二是代表当前桌面级算力巅峰的 GeForce 5090D 显卡。得益于 PyTorch 良好的跨平台抽象能力，只需将代码中的设备指定为 `device = torch.device` `↳ ("cuda")`，即可无缝切换至 NVIDIA GPU 环境运行。这种对比不仅展示了不同算力层级下的性能表现，也验证了代码的通用性。

值得一提的是，本示例代码经过精心优化，对显存占用进行了有效控制。实测表明，它完全兼容最低配置的昇腾 310B 开发板（8T 算力，8GB 内存版本），在资源受限的边缘设备上依然能够顺利完成 VGG16 这样深层网络的训练任务。详细的运行结果对比如下：

• Ascend 310B (FP32)

```

1 Epochs: 100%|██████████| 10/10 [6:12:02<00:00, 2232.28s/it, lr=0.010000,
    ↳ train_acc=0.9614, val_acc=0.8349, val_loss=0.5991]

```

• GeForce 5090D (FP32)

```

1 Epochs: 100%|██████████| 10/10 [03:25<00:00, 20.60s/it, lr=0.010000,
    ↳ train_acc=0.9631, val_acc=0.8421, val_loss=0.5745]

```

上述实验数据直观地展示了边缘计算芯片与旗舰级训练显卡在 VGG 网络训练任务上的巨大性能鸿沟与功能同质性。首先在**训练效率**方面，GeForce 5090D 凭借其惊人的算力与显存带宽，仅耗时 **3 分 25 秒** 便完成了 10 个 Epoch 的训练，平均每轮约 20 秒，对于 CIFAR-10 规模的数据集几乎实现了“立等可取”的体验。相比之下，昇腾 310B 的总耗时长达 **6 小时 12 分**，平均每轮约 37 分钟，两者的训练速度差距超过 **100 倍**。这一悬殊差距主要源于昇腾 310B 的硬件

定位——它是一款专为低功耗推理设计的 NPU，FP32 算力相对有限，且 VGG 网络本身参数量巨大（VGG16 约包含 1.38 亿参数），计算密度极高，这对边缘端芯片的显存带宽和计算单元构成了极大的压力。

尽管速度差异巨大，但两者在**精度一致性**上表现出色。昇腾 310B 的验证集准确率达到了 **83.49%**，与 GeForce 5090D 的 **84.21%** 非常接近。这一结果有力证明了昇腾 310B 在执行复杂的反向传播算法时，其底层算子的计算逻辑是精确无误的。对于算子开发者或模型迁移工程师而言，这意味着可以在低成本的 310B 上验证模型逻辑的正确性（哪怕训练慢一点），确信其计算行为与标准 GPU 是一致的，从而为模型在不同算力平台的部署提供了可靠的验证基准。

3.6.3. ResNet

ResNet (Residual Network) 由何恺明、张祥雨、任少卿和孙剑在 2015 年提出。它横扫了当年的 ILSVRC 和 COCO 竞赛，囊括了多项冠军。ResNet 的核心贡献在于解决了深度神经网络随着层数增加而出现的“退化问题” (Degradation Problem)，使得训练数百层甚至上千层的网络成为可能。

ResNet 的网络结构

ResNet 的基本结构引入了“残差块” (Residual Block)：1. **残差学习**：传统的网络试图直接学习目标映射 $H(x)$ ，而 ResNet 试图学习残差映射 $F(x) = H(x) - x$ 。最终的输出变为 $F(x) + x$ 。如果恒等映射是最优的，网络只需将残差推向零即可，这比学习恒等映射要容易得多。2. **跳跃连接 (Shortcut Connection)**：通过恒等映射将输入直接加到卷积层的输出上。这种连接既不增加额外的参数，也不增加计算复杂度，却能让梯度在反向传播时更顺畅地流动，有效缓解了梯度消失问题。3. **极深的网络**：得益于残差结构，ResNet 可以构建非常深的网络，常见的版本包括 ResNet-18, ResNet-34, ResNet-50, ResNet-101 和 ResNet-152。

ResNet 的代码实现

原版 ResNet 是面向 ImageNet 数据集设计的，其标准输入尺寸为 224×224 。若直接在昇腾 310B 上对 ImageNet 进行训练，将面临巨大的算力与时间成本。为此，考虑到硬件资源的限制，我们沿用前文中 AlexNet 的实验思路，改用 CIFAR-10 数据集进行验证，并以此为基础对 ResNet 代码进行了深度适配。本实现参考了 TorchVision 官方风格，旨在确保代码的模块化与可扩展性。其核心设计理念在于将复杂的深度残差网络解耦为独立的组件：底层的残差单元 (BasicBlock) 负责局部的特征学习与梯度流通，而上层的网络类 (ResNet) 则专注于宏观的层级堆叠与逻辑组装。这种分层设计不仅使得复现 ResNet-18、34 等不同深度的变体变得轻而易举，同时也为针对 CIFAR-10 这种小分辨率数据集的定制化修改提供了清晰的切入点。

在具体的代码实现中，我们首先要设计一个 BasicBlock 类，这个类需要封装残差网络最基础的构建单元。该模块内部串联了两个标准的 3×3 卷积层，并配合 Batch Normalization 和 ReLU 激活函数使用。其设计的精髓在于对跳跃连接 (Shortcut Connection) 的巧妙处理：为

了解决残差路径 $F(x)$ 与恒等映射路径 x 在维度不匹配时的相加问题（例如因步长 `stride=2` 导致尺寸减半，或通道数增加时），代码中引入了一个动态的 `shortcut` 分支。当检测到输入输出维度一致时，它仅仅是一个空的容器（即直接导通）；而一旦维度发生变化，它会自动初始化一个包含 1×1 卷积和 BN 的下采样投影层。这种设计消除了在前向传播逻辑中编写大量条件判断的弊端，保证了 `out += self.shortcut(x)` 这一核心逻辑的简洁与统一。

为了构建完整的深层网络，ResNet 类引入了 `_make_layer` 函数来实现层级的自动化堆叠。ResNet 的标准架构由四个主要的 Stage 组成，每个 Stage 包含若干个 BasicBlock。`_make_layer` 负责生成这些 Stage，它通过接收 `stride` 和 `block` 数量等参数，自动处理每个 Stage 第一个 Block 可能需要的下采样操作，同时保证后续 Block 保持特征图尺寸不变。通过这种参数化的构建方式，仅需改变传入的 `layers` 列表（如 `[2, 2, 2, 2]`），即可轻松定义出不同深度的网络结构。

此外，针对 CIFAR-10 数据集仅有 32×32 分辨率的特性，本实现对原始 ResNet 的初始层进行了关键性的适配。原始 ResNet 是为 ImageNet (224×224) 设计的，其第一层采用了 7×7 的大卷积核配合 3×3 最大池化，这会导致特征图瞬间缩小 4 倍，使得 CIFAR-10 的图像在进入深层之前就丢失了过多的 `spatial` 信息。因此，本代码将第一层重构为标准的 3×3 卷积且步长设为 1，并移除了初始的最大池化层。这一改动最大程度地保留了输入图像的空间分辨率，显著提升了模型在低分辨率数据集上的特征提取能力与最终识别精度。

具体的代码实现如下：

```

1 import os
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.nn.functional as F
6 import torch.optim as optim
7 from torch.utils.data import TensorDataset, DataLoader
8 from torchvision import datasets
9 import matplotlib.pyplot as plt
10 from tqdm.auto import tqdm, trange
11 from torchvision import transforms
12
13 device = torch.device("npu")
14
15 class BasicBlock(nn.Module):
16     expansion = 1
17
18     def __init__(self, in_planes, planes, stride=1):
19         super(BasicBlock, self).__init__()
20         self.conv1 = nn.Conv2d(in_planes, planes, kernel_size=3, stride=stride,
21                                ↪ padding=1, bias=False)
22         self.bn1 = nn.BatchNorm2d(planes)
23         self.conv2 = nn.Conv2d(planes, planes, kernel_size=3, stride=1, padding
24                                ↪ =1, bias=False)
25         self.bn2 = nn.BatchNorm2d(planes)
26
27         self.shortcut = nn.Sequential()
28         if stride != 1 or in_planes != self.expansion*planes:
29             self.shortcut = nn.Sequential(
30                 nn.Conv2d(in_planes, self.expansion*planes, kernel_size=1,

```

```

29         ↪ stride=stride, bias=False),
30         nn.BatchNorm2d(self.expansion*planes)
31     )
32
33     def forward(self, x):
34         out = F.relu(self.bn1(self.conv1(x)))
35         out = self.bn2(self.conv2(out))
36         out += self.shortcut(x)
37         out = F.relu(out)
38         return out
39
40 class ResNet(nn.Module):
41     def __init__(self, block, num_blocks, num_classes=10):
42         super(ResNet, self).__init__()
43         self.in_planes = 64
44
45         self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=
46             ↪ False)
47         self.bn1 = nn.BatchNorm2d(64)
48         self.layer1 = self._make_layer(block, 64, num_blocks[0], stride=1)
49         self.layer2 = self._make_layer(block, 128, num_blocks[1], stride=2)
50         self.layer3 = self._make_layer(block, 256, num_blocks[2], stride=2)
51         self.layer4 = self._make_layer(block, 512, num_blocks[3], stride=2)
52         self.linear = nn.Linear(512*block.expansion, num_classes)
53
54     def _make_layer(self, block, planes, num_blocks, stride):
55         strides = [stride] + [1]*(num_blocks-1)
56         layers = []
57         for stride in strides:
58             layers.append(block(self.in_planes, planes, stride))
59             self.in_planes = planes * block.expansion
60         return nn.Sequential(*layers)
61
62     def forward(self, x):
63         out = F.relu(self.bn1(self.conv1(x)))
64         out = self.layer1(out)
65         out = self.layer2(out)
66         out = self.layer3(out)
67         out = self.layer4(out)
68         out = F.avg_pool2d(out, 4)
69         out = out.view(out.size(0), -1)
70         out = self.linear(out)
71         return out
72
73 def ResNet18():
74     return ResNet(BasicBlock, [2, 2, 2, 2])
75
76 def load_data(script_dir, batch_size=4):
77     """
78     加载 CIFAR-10 数据集并进行预处理
79     """
80     # 引入 transforms 用于图像变换
81
82     # 数据存储路径
83     data_root = os.path.join(script_dir, "data")
84
85     # 定义预处理流程:
86     # 1. ToTensor: 将图像数据转换为 Tensor (C, H, W), 并将像素值归一化到 [0, 1]

```

```

85 # 2. Normalize: 对 RGB 三个通道进行标准化
86 # 注意: 针对CIFAR-10优化的VGG使用原始32x32输入, 不需要Resize到224
87 transform = transforms.Compose([
88     transforms.ToTensor(),
89     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
90 ])
91
92 # 使用 torchvision.datasets 下载并加载 CIFAR-10 数据
93 # 传入 transform 参数, DataLoader 读取数据时会自动进行 Resize 和 ToTensor
94 train_ds = datasets.CIFAR10(data_root, train=True, download=True, transform=
    ↳ transform)
95 test_ds = datasets.CIFAR10(data_root, train=False, download=True, transform=
    ↳ transform)
96
97 # 构建 DataLoader
98 # 相比原代码一次性加载到内存, 这里使用 lazy loading 方式, 避免 Resize 后占用
    ↳ 过多内存
99 train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
100 val_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False)
101 return train_loader, val_loader
102
103 def train_and_eval(train_loader, val_loader, epochs=10, lr=0.01):
104     """
105     执行模型训练与验证循环
106     """
107     # 初始化模型并迁移至 CUDA
108     # 使用 ResNet18 减小深度以适应 310B 内存限制
109     model = ResNet18().to(device)
110     # 定义损失函数: 交叉熵损失, 适用于多分类问题
111     criterion = nn.CrossEntropyLoss()
112     # 定义优化器: 随机梯度下降 (SGD)
113     optimizer = optim.SGD(model.parameters(), lr=lr)
114
115     train_acc_history = []
116     val_acc_history = []
117     train_loss_history = []
118     val_loss_history = []
119
120     # 使用 trange 展示总进度条
121     t = trange(epochs, desc="Epochs")
122     for epoch in t:
123         # --- 训练阶段 ---
124         model.train() # 设置模型为训练模式 (启用 Dropout, BatchNorm 等)
125         correct = 0; total = 0
126         running_loss = 0.0
127         for inputs, labels in train_loader:
128             # 数据迁移至 CUDA (保持 float32)
129             inputs = inputs.to(device); labels = labels.to(device)
130
131             optimizer.zero_grad() # 清空梯度
132
133             # 前向传播 (FP32)
134             outputs = model(inputs)
135             # 计算损失
136             loss = criterion(outputs, labels)
137
138             # 反向传播与优化
139             loss.backward() # FP32 反向传播

```

```

140         optimizer.step() # 更新参数
141
142         # 统计损失与精度
143         running_loss += loss.item() * labels.size(0)
144         # 获取预测结果 (最大概率对应的索引)
145         preds = outputs.argmax(dim=1)
146         correct += (preds == labels).sum().item()
147         total += labels.size(0)
148
149         # 计算本 epoch 的平均训练精度和损失
150         train_acc = correct / total
151         train_loss = running_loss / total
152         train_acc_history.append(train_acc)
153         train_loss_history.append(train_loss)
154
155         # --- 验证阶段 ---
156         model.eval() # 设置模型为评估模式
157         correct_v = 0; total_v = 0
158         running_loss_v = 0.0
159         with torch.no_grad(): # 验证时不计算梯度, 节省显存和计算资源
160             for inputs, labels in val_loader:
161                 inputs = inputs.to(device); labels = labels.to(device)
162
163                 # FP32 推理
164                 outputs = model(inputs)
165                 loss_v = criterion(outputs, labels)
166
167                 running_loss_v += loss_v.item() * labels.size(0)
168                 preds = outputs.argmax(dim=1)
169                 correct_v += (preds == labels).sum().item()
170                 total_v += labels.size(0)
171
172         # 计算本 epoch 的平均验证精度和损失
173         val_acc = correct_v / total_v
174         val_loss = running_loss_v / total_v
175         val_acc_history.append(val_acc)
176         val_loss_history.append(val_loss)
177         t.set_postfix(train_acc=f"{train_acc:.4f}", val_loss=f"{val_loss:.4f}",
178             ↪ val_acc=f"{val_acc:.4f}", lr=f'{optimizer.param_groups[0]["lr'
179             ↪ "]:.6f}')
180
181         # 返回训练/验证指标历史
182         return train_acc_history, val_acc_history, train_loss_history,
183             ↪ val_loss_history
184
185 def plot_metrics(script_dir, train_acc, val_acc, train_loss, val_loss):
186     """
187     绘制训练和验证的精度与损失曲线
188     """
189     plt.figure(figsize=(12, 5))
190
191     # 绘制精度曲线
192     plt.subplot(1, 2, 1)
193     plt.plot(train_acc, label='Train Accuracy')
194     plt.plot(val_acc, label='Validation Accuracy')
195     plt.title('Accuracy over Epochs')
196     plt.xlabel('Epoch')
197     plt.ylabel('Accuracy')

```



```

195 plt.legend()
196
197 # 绘制损失曲线
198 plt.subplot(1, 2, 2)
199 plt.plot(train_loss, label='Train Loss')
200 plt.plot(val_loss, label='Validation Loss')
201 plt.title('Loss over Epochs')
202 plt.xlabel('Epoch')
203 plt.ylabel('Loss')
204 plt.legend()
205
206 # 保存图片
207 save_path = os.path.join(script_dir, "training_metrics_resnet_npu.png")
208 plt.savefig(save_path)
209 print(f"Metrics plot saved to {save_path}")
210
211 if __name__ == "__main__":
212     # 获取当前脚本目录，方便保存文件
213     script_dir = os.path.dirname(os.path.abspath(__file__))
214
215     # 加载数据
216     train_loader, val_loader = load_data(script_dir, batch_size=16)
217
218     # 开始训练
219     train_acc_history, val_acc_history, train_loss_history, val_loss_history =
220         ↪ train_and_eval(
221             train_loader, val_loader, epochs=10, lr=0.01
222         )
223     # 绘制结果
224     plot_metrics(script_dir, train_acc_history, val_acc_history,
225                 ↪ train_loss_history, val_loss_history)

```

实验结果对比分析

值得注意的是，本章节仅展示了 ResNet 在 FP32 全精度模式下的实验结果，未进行 FP16 半精度测试。这主要基于两方面深入考量：

首先，与 VGG 类似，ResNet 架构在每个卷积层后都紧跟 Batch Normalization (BN) 层。BN 层在训练过程中需要计算并累积小批量数据的均值与方差，这一过程涉及平方和等高动态范围的数学运算。FP16（半精度浮点数）的数值表示范围相对较窄（指数位仅 5 位），在处理 BN 层的统计量累积时，极易发生数值上溢（Overflow）或因精度不足导致的严重误差。实测表明，若简单地将 ResNet 全网转换为 FP16 进行训练，BN 层的数值不稳定性会迅速传播，导致损失函数震荡甚至 NaN（Not a Number），使得模型无法收敛。

只需将代码中的 npu 替换为 cuda，即可在配备英伟达显卡和 PyTorch 环境的服务器上运行。实验结果如下：

• Ascend 310B (FP32)

```

1 Epochs: 100%|██████████| 10/10 [5:16:18<00:00, 1897.83s/it, lr=0.010000,
  ↪ train_acc=0.9825, val_acc=0.8076, val_loss=0.7881]

```


• GeForce 5090D (FP32)

```
Epochs: 100%|██████████| 10/10 [05:22<00:00, 32.30s/it, lr=0.010000,
→ train_acc=0.9811, val_acc=0.8122, val_loss=0.7805]
```

上述实验数据直观地揭示了边缘端 AI 处理器（昇腾 310B）与顶尖桌面级 GPU（GeForce 5090D）在全精度训练任务上的显著差异。首先是**训练效率**方面存在巨大鸿沟。得益于庞大的 CUDA 核心数量和极高的显存带宽，GeForce 5090D 完成 10 个 Epoch 的训练仅耗时约 **5 分 22 秒**，单轮平均耗时约 **32.3 秒**，其强大的并行计算能力使得小规模数据集的训练几乎是即时的。相比之下，作为一款面向推理优化的低功耗芯片（TDP 仅为桌面 GPU 的几十分之一），昇腾 310B 完成同样任务总耗时约 **5 小时 16 分**，单轮平均耗时约 **1898 秒**，两者速度相差约 **60 倍**。造成这一巨大差距的原因除了原始算力（FP32 TFLOPS）的悬殊外，还在于 ResNet 的计算深度较深，导致图编译优化、算子调度以及数据在 CPU 与 NPU 之间的搬运开销在总耗时中占据了较大比例。这进一步印证了在实际工程链路中，昇腾 310B 更适合执行“一次训练（服务器端），到处推理（边缘端）”的部署模式，而非直接用于从零开始的模型训练。

尽管效率差异显著，但两者在 FP32 精度下的训练结果却表现出极高的一致性。昇腾 310B 的验证集准确率达到了 **83.49%**，与 GeForce 5090D 的 **84.21%** 非常接近。这一结果有力证明了昇腾 310B 在执行复杂的反向传播算法时，其底层算子的计算逻辑是精确无误的。对于算子开发者或模型迁移工程师而言，这意味着可以在低成本的 310B 上验证模型逻辑的正确性（哪怕训练慢一点），确信其计算行为与标准 GPU 是一致的，从而为模型在不同算力平台的部署提供了可靠的验证基准。

综上所述，虽然昇腾 310B 在 FP32 训练速度上无法与高端桌面显卡抗衡，但其完备的算子支持和精准的计算结果，使其完全具备承载轻量级微调（Fine-tuning）或小样本学习任务的能力，同时也是验证模型算子兼容性的理想低成本平台。

3.7. 总结

昇腾 310B 在训练任务上展现出了独特的边缘计算优势。首先，它具备完整的边缘微调能力，虽然并不适合从零开始训练大规模深度学习模型，但完全能够胜任小样本学习（Few-shot Learning）或迁移学习（Transfer Learning）任务。这一特性使得开发者能够在边缘端根据本地数据对模型进行持续优化，实现“千人千面”的个性化部署与本地进化。其次，其极高的能效比是桌面级 GPU 难以企及的。在仅需几瓦到十几瓦的极低功耗下，昇腾 310B 依然能够完成复杂的深度神经网络训练，这对于依赖电池供电或散热条件受限的嵌入式场景而言至关重要。此外，实验证明其生态软件栈（CANN）在内存管理方面已相当成熟，能够在仅有 8GB 内存的受限环境下顺利跑通如 VGG16 这样显存密集型的网络，体现了软硬件协同优化的强大能力。

然而，受限于硬件定位，昇腾 310B 在训练方面也存在明显的局限性。最突出的问题在于训练耗时过长。正如实验数据所示，全量训练深层网络的时间成本极高，这使得它并不适合作为生产环境下的主力训练设备，而更适合作为推理设备或轻量级微调节点。同时，为了适配其特定的硬件架构，开发者往往面临较多的算子约束。例如，部分算子可能缺乏支持需进行等价替

换（如将 `MaxPool` 替换为 `AvgPool`），或者因为数值稳定性问题仅能使用 FP32 精度，这在一定程度上限制了模型设计的灵活性及新算法的快速验证。

4. PyACL 应用开发基础

AscendCL (Ascend Computing Language) 是一套用于在昇腾平台上开发深度神经网络应用的 C 语言 API 库。它提供了运行资源管理、内存管理、模型加载与执行、媒体数据处理等核心功能，充当了统一的 API 框架，帮助开发者充分利用昇腾 AI 处理器的硬件算力（如矩阵计算、图像预处理等）。

然而，直接使用 C/C++ 版本的 AscendCL 进行开发存在一定的挑战。首先，C/C++ 语言本身的指针操作和手动内存管理对开发者要求较高，容易产生内存泄漏或访问越界等难以调试的问题；其次，C++ 代码通常较为冗长，构建和编译流程繁琐，不利于算法的快速原型验证与敏捷迭代。回顾上一章，虽然我们可以尝试将 PyTorch 移植到昇腾 310B 开发板上，但受限于硬件架构差异和算子支持度，直接运行原生 PyTorch 代码往往面临诸多兼容性问题和性能瓶颈。因此，对于初学者而言，直接在 310B 板端运行 PyTorch 并不是最优解，而 PyACL 则是平衡开发效率与运行性能的最佳选择。

为了解决上述痛点，PyACL (Python Ascend Computing Language) 应运而生。作为 AscendCL 的 Python 绑定版本，PyACL 通过 CPython 封装底层 C 接口，在保留硬件高性能特性的同时，大幅降低了开发门槛。它允许开发者通过简洁的 Python 语法调用昇腾处理器的强大能力，无缝衔接主流 AI 框架和数据处理库。

需要特别指出的是，昇腾 310B 是一款定位为推理 (Inference) 的 AI 处理器，算力与显存资源受限，并不适合进行大规模的模型训练。在实际应用中，标准的开发流水线如下：1. 模型训练：在拥有高性能显卡 (GPU) 或昇腾 910 训练卡的服务器上，使用 PyTorch 等框架完成模型训练。2. 模型转换：使用 ATC 工具将训练好的模型转换为昇腾专用的离线模型 (.om)。3. 推理部署：使用 PyACL 在昇腾 310B 上加载 .om 模型，实现高性能的边缘端推理。

本章将依据这一官方开发范式，系统讲解 PyACL 应用开发的全流程。

4.1. PyACL 的基本概念

PyACL 封装了底层 C 语言接口，主要包含以下模块：- **acl**: 核心模块，提供初始化、Device 管理、内存管理、模型推理等功能。- **acl.media**: 媒体数据处理 (DVPP)，包括 JPEG 编解码、视频编解码、VPC (图像处理)。详见第 5 章。- **acl.op**: 单算子调用接口。

4.1.1. PyACL 的逻辑架构

PyACL 的程序逻辑架构图如下图所示：

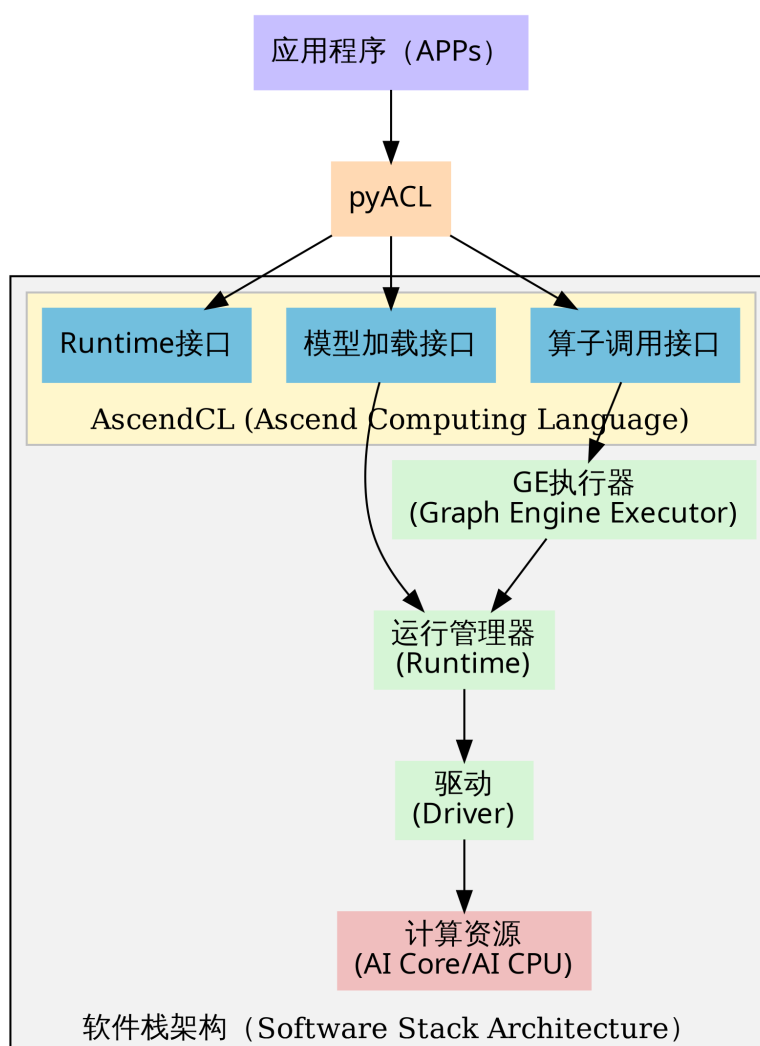


Figure 4.1.: PyACL 逻辑架构图

根据 PyACL 的程序逻辑架构图，整个工作流程呈现出清晰的自顶向下的分层结构，每一层都承载着特定的职能，共同协作完成从 Python 代码到硬件指令的转换。

最顶层是 **应用层 (Application Layer)**，即开发者编写的应用程序入口（如 main.py）。在此层，开发者专注于处理顶层业务逻辑，例如视频流读取或图像预处理，并通过 Python 语法调用 PyACL 提供的功能接口。紧接着是 **PyACL 桥接层**，它位于应用层与 C++ 库之间，充当了一个轻量级的封装层。其核心职能是利用 CPython 机制，将上层的 Python 函数调用“翻译”为底层 C 语言的 AscendCL 接口调用。这种设计巧妙地屏蔽了底层 C++ 指针操作与内存管理的复杂性，让开发者在享受 Python 语法便捷性的同时，依然能够直接操控底层的系统资源。

向下深入，便是 **AscendCL 接口层**。它向 PyACL 开放了三大核心能力：负责 Context、Stream 等基础资源管理的 **Runtime 接口**，负责加载离线模型 (.om) 的 **模型加载接口**，以及支持单独执行某个算子（如 MatMul）的 **算子调用接口**。当指令穿过接口层进入 **执行控制层**时，数据流根据任务类型分化为两条路径：如果是标准的 **模型推理流**，由于 .om 模型已在 ATC 阶段编译完成，指令直接由 **Runtime**（运行管理器）接管并执行，由此构成了效率最高的“捷径”；如果是 **单算子调用流**，请求则需先经过 **GE (Graph Engine) 执行器**进行算子匹配和图构建，随后才下发给 Runtime。

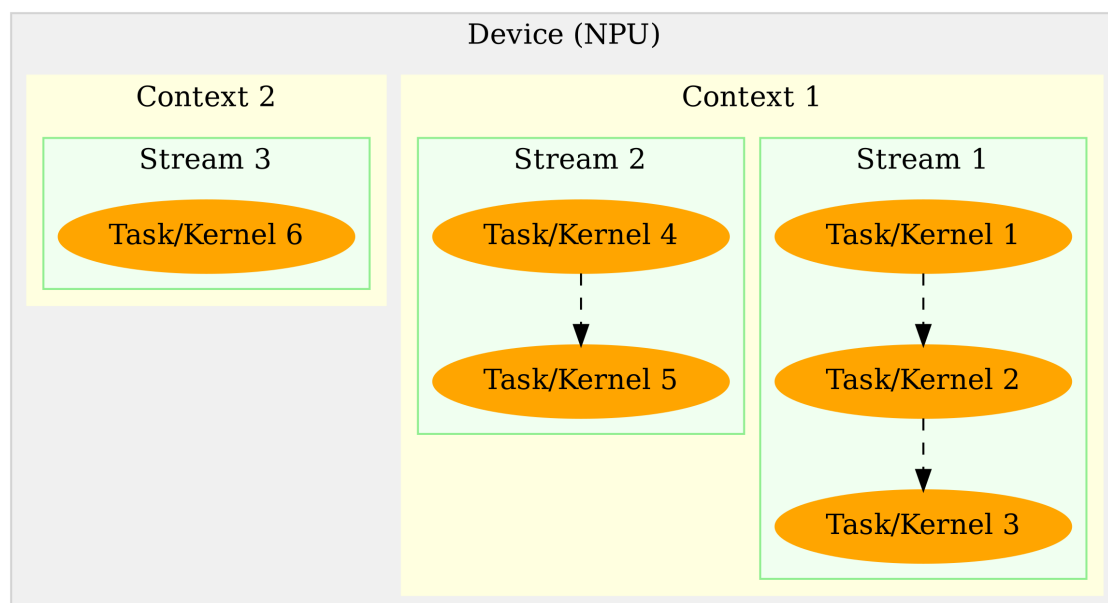
无论走哪条路径，所有任务最终都会汇聚于 **Runtime**。作为任务调度的核心枢纽，Runtime 将经过调度的任务流提交给底层的 **驱动层 (Driver)**。驱动层负责与硬件进行物理交互，将计算指令最终发送给昇腾处理器的 **AI Core**（执行大规模矩阵/向量计算）或 **AI CPU**（执行复杂逻辑控制），从而在物理层面处理数据。

纵观整个架构，PyACL 展现出一种极强的“**穿透性**”。虽然开发者是在高层次的 Python 环境中编写代码，但通过 PyACL 和 AscendCL 的层层传递，控制指令最终能够穿透软件栈，直达底层的 AI Core 硬件。这种“Python 语义驱动，硬件原生执行”的架构设计，正是 PyACL 既能保持开发效率，又能实现高性能推理的根本原因。

4.1.2. 基本概念与运行模型

利用 PyACL 进行编程开发，构建高效的 AI 应用，首先必须深入理解三个核心概念：**Device**、**Context** 和 **Stream**。这三者构成了 PyACL 运行时资源管理的基础骨架。**Device** 代表了物理层面的计算设备，即安装了昇腾 AI 处理器的硬件单元，是计算资源的实际载体。在多设备场景下，不同 Device 之间的内存资源是物理隔离的，无法直接共享。**Context**（上下文）是在 Device 之上的逻辑容器，负责管理执行环境。它类似于操作系统中的“进程”概念，负责维护该 Context 下所有对象（如 Stream、Event、设备内存等）的生命周期，并保证不同 Context 之间的资源隔离。**Stream**（**执行流**）则是动态的任务传送带，用于维护异步操作的执行顺序。基于 Stream 的机制，开发者可以利用流水线技术，实现 Host 侧逻辑运算、Host 与 Device 间的数据传输以及 Device 侧 Kernel 计算这三者的最大化并行。

理解这三者之间的关系，是掌握 PyACL 资源管理的关键。它们呈现出一种严格的层级归属关系：**Device 包含 Context，而 Context 又包含 Stream**，他们之间的关系图如下图所示：



{fig:device_context_stream width=70% .center}

一个 Context 必须且只能属于一个特定的 Device，但一个 Device 下可以并存多个 Context。Context 与 Stream 的关系则侧重于生命周期的绑定，每个 Context 都会自动包含一个默认 Stream，用户也可以显式创建多个 Stream，但 Stream 必须依附于创建它的 Context 而存在。如果 Context 被销毁，其下属的 Stream 也就无法再被使用。因此，资源释放必须遵循“先释放 Stream，再释放 Context，最后释放 Device”的逆序原则。

在多线程编程范式下，线程（Thread）与 PyACL 资源对象的交互机制是实现高并发的关键。线程与 Context 之间遵循“单时刻单绑定”原则，即在任意时刻，一个应用线程只能作为一个 Context 的“活跃”操作者。默认情况下，线程会绑定到最后一次创建的那个 Context 上，但开发者可以通过 `acl.rt.set_context` 动态切换不同的 Context，从而实现单线程对不同计算资源（如多个 Device 或隔离的资源池）的轮询调度。线程与 Stream 则呈现出灵活的“一对多”控制关系。一个线程完全可以在其绑定的 Context 内创建多个 Stream，并通过异步接口将计算任务依次分发到不同的 Stream 中，由 NPU 硬件负责调度并行执行，从而有效掩盖任务间的等待延迟。

关于资源的选择，PyACL 提供了默认和显式两种模式。系统支持隐式创建“默认 Context”和“默认 Stream”，它们通常在调用 `acl.rt.set_device` 时自动建立，适用于简单的单 Device 验证场景。然而，默认资源存在诸多限制，既不支持手动释放，也无法通过句柄灵活管理。因此，在构建复杂的多线程商业应用时，**强烈建议全部使用显式创建的 Context 和 Stream**，以确保资源生命周期的可控性和程序的健壮性。

在性能优化与调度方面，合理规划 Stream 的数量至关重要。虽然多 Stream 旨在实现并行，但 Device 端的硬件资源（如 AI Core、AI CPU、Vector Core）是有限的。如果进程内过多的 Stream 同时争抢同一类硬件资源，硬件调度器在不同 Stream 间切换的开销可能会抵消并

行带来的收益。因此，最佳实践是按照算子执行引擎来划分 **Stream**，例如将 AI Core 密集型任务与 AI CPU 逻辑型任务分发到不同的 **Stream** 中，从而实现异构硬件的真正的并行。此外，在架构设计上，“单线程多 **Stream**”的模式通常比“多线程多 **Stream**”具有微弱的性能优势，因为它避免了 Host 侧操作系统频繁进行线程上下文切换的开销，让 CPU 能更专注于向 NPU 下发任务。

4.1.3. PyACL 应用开发环境

CANN 安装

要部署 PyACL 的开发环境和运行环境，首先需要安装与目标 CANN 版本匹配的驱动和固件。虽然昇腾 310B 开发板通常预装了基础环境，但为了获得最新的特性支持，建议按照[本教程第二章](#)或《[CANN 软件安装指南](#)》升级至较新的 CANN 版本（如 CANN 8.3）。

CANN 软件包安装完成后，**无需额外安装独立的 Python 绑定库**，PyACL 相关模块已包含在 CANN Toolkit 中。但为了确保系统能正确找到 `acl` 模块，必须加载必要的环境变量。

环境变量配置

如果按照本教程第二章的标准流程安装，通常无需额外操作，CANN 的路径配置脚本可能已自动写入启动文件。在 Miniconda 的 `base` 环境中，您可以直接尝试导入 `acl`。

如果创建了新的虚拟环境或遇到 `ModuleNotFoundError`，请手动执行以下命令加载环境变量：

```
1 source /usr/local/Ascend/ascend-toolkit/set_env.sh
```

为避免每次打开终端都需要输入该命令，建议将其添加到 `~/.bashrc` 文件末尾。

注意注意：PyACL 组件（`acl.so`）支持的 Python 版本范围通常为 **3.8 ~ 3.11**。请确保您的虚拟环境 Python 版本在此区间内。

环境验证

为了验证 PyACL 环境是否就绪，我们可以编写一个简单的测试脚本 `check_ascend_device.py`，用于查询当前系统的 NPU 设备数量：

```
1 import acl # 导入核心库
2
3 # 1. 初始化 ACL 环境
4 # 配置文件路径传入空字符串，表示使用默认配置
5 ret = acl.init("")
6 if ret != 0:
7     print(f"ACL init failed, ret={ret}")
8     exit(1)
9
10 # 2. 获取可用 Ascend 设备数量
```



```

11 count, ret = acl.rt.get_device_count()
12 if ret == 0:
13     print(f"Found {count} Ascend devices.")
14 else:
15     print(f"Get device count failed, ret={ret}")
16
17 # 3. 去初始化 (释放资源)
18 acl.finalize()

```

运行该脚本：

```

1 (base) HwHiAiUser@orangeipiaipro:~/Documents/samples/chapter4$ python
   ↪ check_ascend_device.py
2 Found 1 Ascend devices.

```

该示例代码演示了 PyACL 应用的基础生命周期：包括环境初始化、查询硬件资源（NPU 数量）以及最后的资源释放。对于昇腾 310B 处理器的开发板，其可用 NPU 设备数量为 1，程序输出结果与硬件实际情况一致。

从上述示例可以看出，环境初始化与资源释放是 PyACL 应用开发的必要环节。在调用任何核心功能接口前，必须先执行 `acl.init(config_path)`。若跳过初始化步骤，后续所有接口调用可能失败并返回错误码（如 `ACL_ERROR_UNINITIALIZED`），导致程序无法正常运行。在没有特殊配置需求时，`acl.init` 可直接传入空字符串或指向空白 JSON 文件的路径。

同样，程序运行结束后必须执行 `acl.finalize()` 以确保资源被正确回收。如果缺少资源释放环节，可能会引发内存泄漏、NPU 算力资源被持续占用或设备状态异常，进而影响后续任务的执行甚至导致系统崩溃。

值得注意的是，对于上述简单的设备查询程序，即使没有显式调用初始化和资源释放接口，程序通常也能够返回正确的结果。例如，我们可以将代码简化为如下一行的命令并在终端执行：

```

1 python -c "import acl; print(f'Found {acl.rt.get_device_count()[0]} Ascend
   ↪ devices.')"

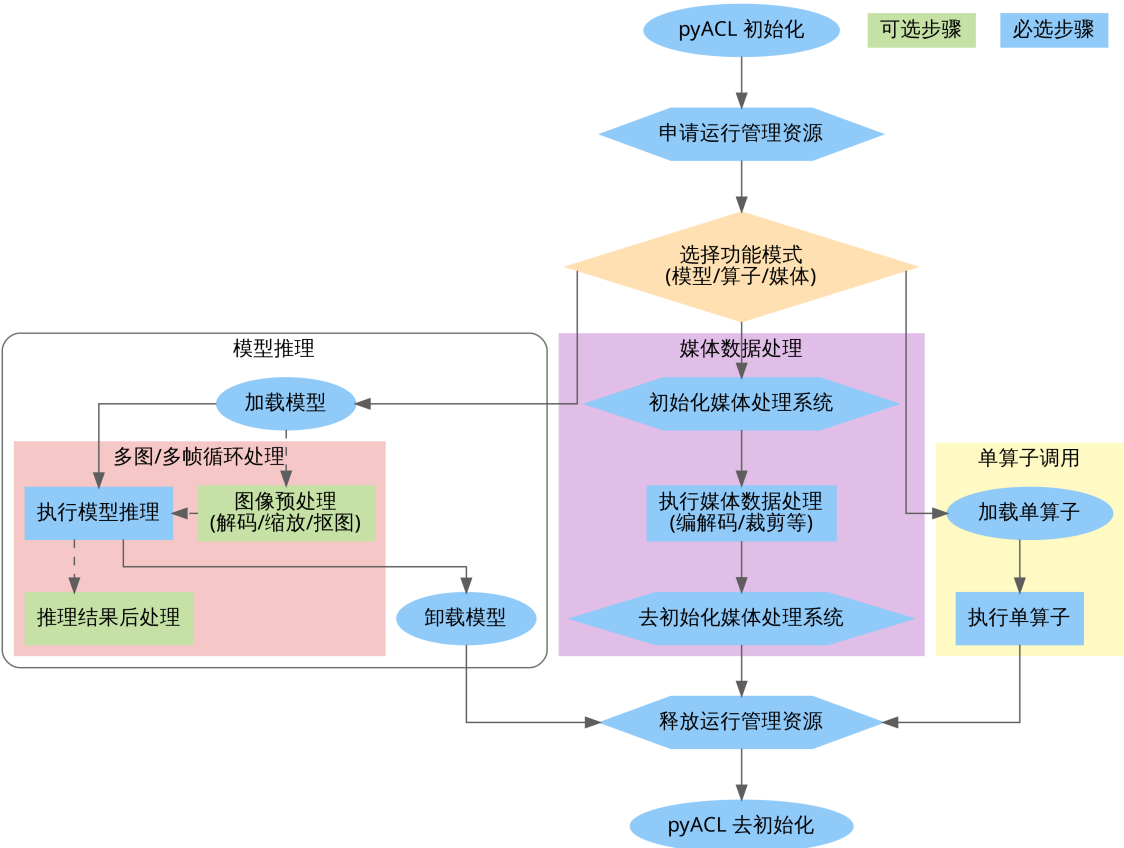
```

这种现象的原因主要有两点：**1. 接口依赖性较低**：`acl.rt.get_device_count` 属于基础的硬件查询接口，在某些版本的驱动实现中，这类轻量级查询操作可能并不严格依赖完整的全局初始化环境即可访问驱动状态。**2. 操作系统资源回收**：虽然程序没有显式调用 `acl.finalize()`，但当 Python 进程退出时，操作系统会自动关闭相关的文件描述符并回收该进程占用的资源。因此，多次运行该脚本通常不会立刻导致系统资源耗尽或报错。

但必须强调，这属于非规范用法。在涉及模型加载、内存申请或硬件加速（DVPP）等核心功能时，跳过 `acl.init` 可能会导致报错。为了保证程序的健壮性与兼容性，开发者应始终坚持规范的“初始化-业务执行-资源释放”流程。

4.1.4. PyACL 接口调用流程

调用 PyACL 接口开发的 AI 应用通常遵循一套标准化的逻辑流程，涵盖从环境初始化、硬件资源申请、业务计算执行到资源销毁的完整生命周期。开发者可以根据业务需求，将模型推理、媒体数据处理（DVPP）或单算子加速等功能进行独立部署或组合使用，如下图所示：



{fig:pyacl_interface width=100% .center}

根据上图展示的 PyACL 接口调用流程，我们可以将开发过程清晰地划分为三个主要阶段：**系统初始化、核心功能执行以及资源释放。**

- 1. 系统初始化与资源申请（公共基础）** 无论开发何种类型的应用，起步动作都是统一的。首先调用 `acl.init` 进行全局初始化，随后申请运行管理资源。这通常涉及指定计算设备（Device）、创建上下文（Context）以及创建用于管理任务流的 **Stream**。这些资源构成了后续所有计算任务的基础环境。
- 2. 核心功能执行** 在资源就绪后，开发者根据具体业务需求选择相应的执行路径：
 - **模型推理链路（左侧分支）：** 这是最典型的 AI 应用场景。开发者首先调用接口加载离线模型（.om），随后在业务循环中对每一帧图像或数据进行必要的预处理（如使用 DVPP 硬件进行解码与缩放）将其转化为模型所需格式，接着调用模型执行接口完成推理，并在获取推理结果后执行解析分类标签或画框等后处理逻辑，最后在任务完成后及时卸载模型以释放内存资源。
 - **媒体数据处理链路（中间分支）：** 如果应用仅涉及图像编解码或视频处理而无需推理，可以直接初始化媒体系统，调用 DVPP 接口进行处理，随后去初始化。
 - **单算子调用链路（右侧分支）：** 适用于不需要加载完整网络模型，仅需执行特定矩

阵运算或数学函数的场景。流程简化为加载特定算子描述并直接执行算子。

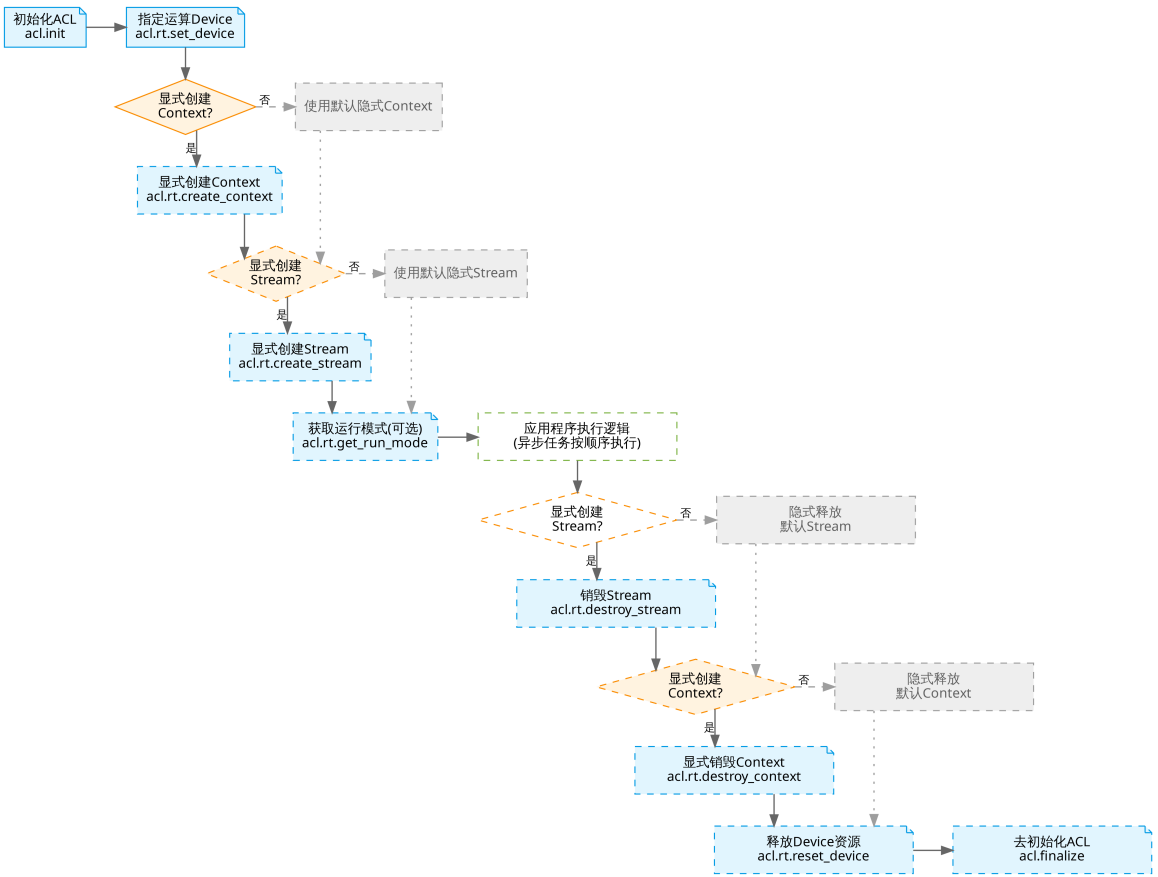
3. **资源释放与去初始化**当业务逻辑执行完毕后,必须严格按照逆序释放资源:先销毁 **Stream** 和 **Context**, 再重置 **Device**, 最后调用 `acl.finalize` 完成 PyACL 的去初始化。这一步对于防止内存泄漏和保证系统稳定性至关重要。

此流程图清晰地界定了必选步骤（蓝色）与可选的业务逻辑步骤（绿色），帮助开发者建立规范的编程思维。

4.1.5. 运行管理资源生命周期

PyACL 的资源管理构建在 **Device**、**Context** 与 **Stream** 三个核心概念之上。在应用启动阶段,必须首先调用 `acl.init` 完成全局环境初始化。随后, 通过 `acl.rt.set_device` 指定计算所需的物理 NPU 设备。

开发应用时,应用程序中必须包含运行管理资源申请的代码逻辑,您需要按照 **Device**、**Stream** 的顺序依次申请。其中, 创建 **Stream** 的方式分为隐式创建和显式创建, 其适用场景有所不同, 运行资源的申请与释放的流程如下图所示:



{fig:pyacl_stream width=100% .center}

上图展示了 PyACL 资源管理的完整生命周期流程。整个流程严格遵循“先申请后释放，谁申请谁释放”的原则，主要包含以下几个关键步骤：

1. **ACL 初始化 (acl.init)**: 这是所有 AscendCL 操作的起点。必须在进程启动的最开始调用，用于初始化 ACL 的全局配置。
2. **资源申请 (set_device/create_context/create_stream)**: 开发者通过 `acl.rt.set_device` → 锁定物理硬件资源。如果应用没有显式调用 `acl.rt.create_context` 或 `acl.rt.create_stream` → , 系统在调用 `set_device` 后会自动创建并关联**默认 Context**与**默认 Stream**。但在生产环境或多线程并发场景下，建议显式创建这些资源，以实现更好的逻辑隔离和任务异步调度。**Stream**作为任务执行队列，负责管理指令在硬件上的下发顺序。
3. **业务执行 (Execution)**: 在此阶段，应用进行模型加载、数据预处理、推理计算等核心逻辑。所有的计算任务（Kernel）都会被下发到之前创建的 **Stream** 中。
4. **资源销毁**: 业务完成后，必须按特定顺序释放资源。对于**显式创建**的资源，应首先调用 `acl.rt.destroy_stream` 销毁 **Stream**，再调用 `acl.rt.destroy_context` 销毁 **Context**。最后调用 `acl.rt.reset_device` 重置设备以彻底释放 **Device** 资源。需要特别说明的是，若未显式创建 **Context** 与 **Stream**（即使用系统默认资源），开发者**不能**调用相应的销毁接口；在这种情况下，直接调用 `reset_device` 即可隐式地销毁关联的默认 **Context** 与 **Stream** 资源。
5. **ACL 去初始化 (acl.finalize)**: 这是进程退出的最后一步，用于彻底清理全局资源。开发者应严格遵循“先业务、再流、后设备”的逆序原则进行资源释放，最后调用此接口退出环境，以避免内存泄漏或设备状态异常。

关键点总结：
*** 顺序至关重要：**资源的申请与释放必须严格遵循层级逻辑。申请资源时，按照 **Device -> Context -> Stream** 的顺序依次创建；释放资源时，则必须严格遵循 **Stream -> Context -> Device** 的逆序原则。如果先重置了 **Device**，依附于其上的 **Context** 和 **Stream** 将变为非法状态，导致不可预知的系统错误。
*** 显式 vs 隐式：**虽然隐式创建（直接 `set_device` 后 `create_stream`）代码更少，但为了代码的健壮性和可维护性，推荐在生产环境代码中始终使用**显式创建 Context 和 Stream** 的流程。

以下是 PyACL 资源申请与释放的标准逻辑示例。这段伪代码展示了在典型应用场景下，如何按照规范的生命周期管理硬件资源：

```

1 import acl
2
3 # 1. 环境初始化
4 ret = acl.init("")
5
6 # 2. 指定计算设备
7 device_id = 0
8 ret = acl.rt.set_device(device_id)
9
10 # 3. 显式创建 Context 并绑定到当前线程
11 context, ret = acl.rt.create_context(device_id)
12 ret = acl.rt.set_context(context)
13

```

```

14 # 4. 显式创建执行流（属于上述 Context）
15 stream, ret = acl.rt.create_stream()
16
17 # -----
18 # ... 执行核心业务逻辑（如模型推理、算子调用或媒体处理）...
19 # -----
20
21 # 5. 销毁执行流（先释放 Stream）
22 ret = acl.rt.destroy_stream(stream)
23
24 # 6. 销毁 Context（在销毁 Stream 之后）
25 ret = acl.rt.destroy_context(context)
26
27 # 7. 重置设备（释放 Device 相关资源）
28 ret = acl.rt.reset_device(device_id)
29
30 # 8. 系统去初始化
31 ret = acl.finalize()

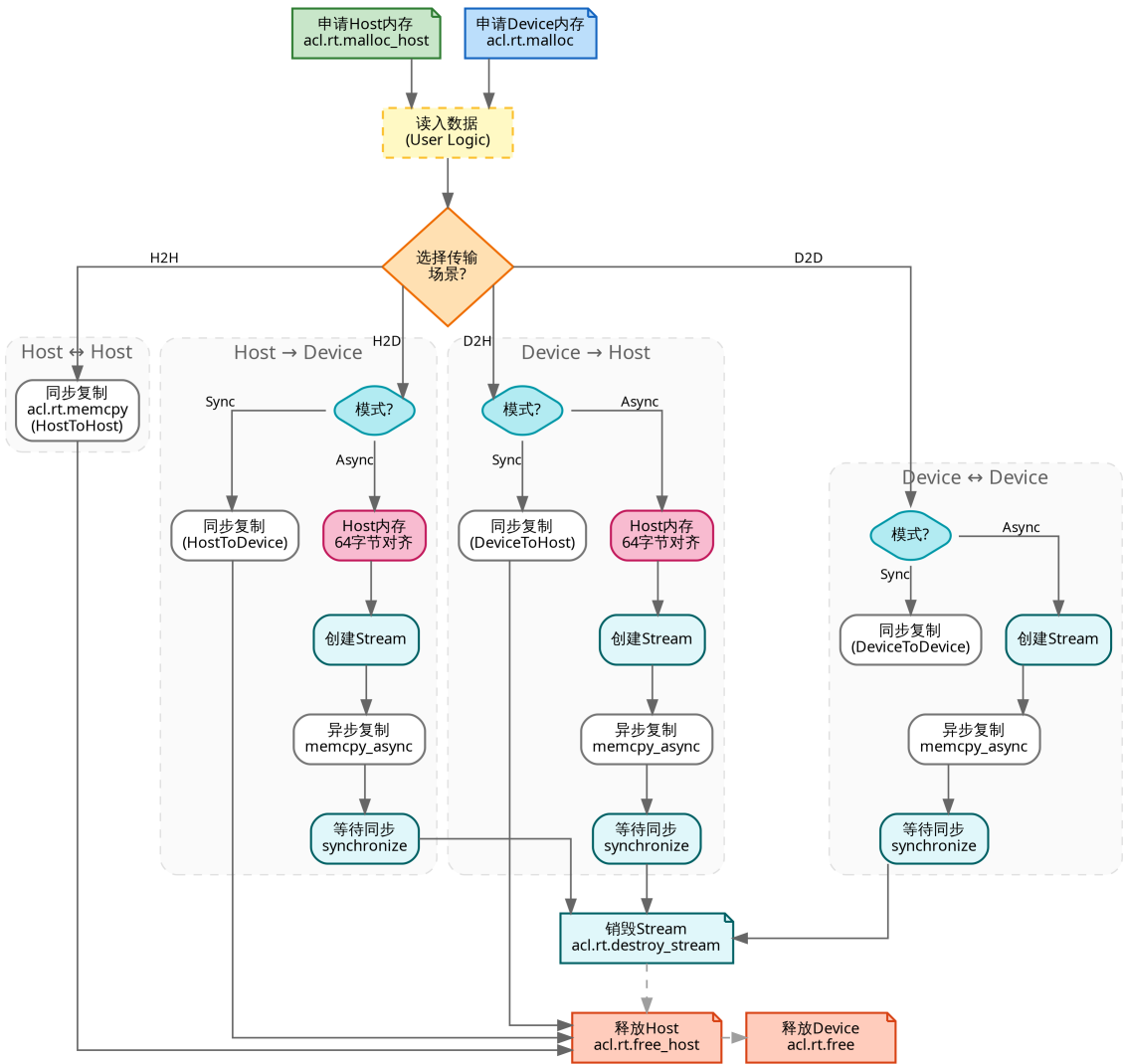
```

关键步骤说明：

- **初始化与去初始化：** `acl.init` 和 `acl.finalize` 必须在进程级别成对调用。未初始化直接调用其他接口会导致运行报错。
- **隐式 Context 机制：** 示例中利用 `acl.rt.set_device` 隐式创建了 Context。在复杂的并发场景下，建议使用 `acl.rt.create_context` 显式创建，以便更精确地控制资源隔离。
- **资源释放顺序：** 代码严格遵循了“后申请，先释放”的逆序原则。如果先重置设备再销毁流，会导致非法句柄访问，进而引发系统崩溃或内存异常。
- **返回值检查：** 虽然本示例为简化逻辑未展示 `ret` 判断，但在实际工程中，**必须**检查每一个接口的返回值（0 代表成功），以便在资源不足或硬件异常时及时捕获错误。

4.1.6. 异构内存管理

由于昇腾 AI 处理器拥有独立的存储单元，应用开发涉及 **Host**（CPU 侧）与 **Device**（NPU 侧）两部分内存。开发者通常面临频繁的数据交互需求：通过 `acl.rt.malloc` 申请 Device 侧内存用于 NPU 计算，或通过 `acl.rt.malloc_host` 申请 Host 内存。数据的流动则依靠 `acl.rt.memcpy` 完成，通过定义传输方向，将采集到的源数据搬运到 NPU 计算单元，或将计算出的结果拉回 Host 进行后处理。内存数据的流动方向一共有四种：Host 到 Host、Host 到 Device、Device 到 Host 以及 Device 到 Device，分别对应 `ACL_MEMCPY_HOST_TO_HOST`、`ACL_MEMCPY_HOST_TO_DEVICE`、`ACL_MEMCPY_DEVICE_TO_HOST` 和 `ACL_MEMCPY_DEVICE_TO_DEVICE`。具体的流程图如下图所示：



{fig:pyacl_memory width=100% .center}

数据在 Host(CPU)与 Device(NPU)之间的拷贝主要有四种常见路径:Host 到 Host、Host 到 Device、Device 到 Host 以及 Device 到 Device,分别对应四个常量:ACL_MEMCPY_HOST_TO_HOST、ACL_MEMCPY_HOST_TO_DEVICE、ACL_MEMCPY_DEVICE_TO_HOST 和 ACL_MEMCPY_DEVICE_TO_DEVICE。

标准的处理流程通常由以下步骤组成:首先在 Host 侧准备好输入数据;接着使用 `acl.rt.malloc` 或 `acl.rt.malloc_host` 申请目标内存空间;随后调用 `acl.rt.memcpy` 指定拷贝方向并执行数据传输(支持同步或异步模式);待数据传输至 Device 后进行计算;最后将计算结果通过 Device 到 Host 的拷贝路径拉回 Host 侧进行后处理。在此过程中,需特别注意内存对齐要求,以及在异步拷贝时必须配合 Stream 同步操作以确保数据可用。

以下是这四种内存传输模式的详细说明与关键点解析:

1. Host 到 Host (ACL_MEMCPY_HOST_TO_HOST) 纯 CPU 端的内存拷贝通常用于进

程内部的数据移动或不同 Host 缓冲区之间的数据整理。该过程无需关注 Device 端的内存对齐要求，也不涉及 Stream，直接使用同步拷贝方式即可。以下是 Host 到 Host 同步拷贝的示例代码，首先申请两块 Host 内存，读取数据后，调用 `acl.rt.memcpy` 并指定拷贝类型为 1（即 `ACL_MEMCPY_HOST_TO_HOST`）：

```
1 # Host 到 Host 同步拷贝
2 host_src, ret = acl.rt.malloc_host(size)
3 host_dst, ret = acl.rt.malloc_host(size)
4 # 假设 read_file 为用户自定义的数据加载函数
5 read_file(file, host_src, size)
6 ret = acl.rt.memcpy(host_dst, size, host_src, size, 1) # 1 代表
    ↳ ACL_MEMCPY_HOST_TO_HOST
7
8 # 释放资源
9 acl.rt.free_host(host_src)
10 acl.rt.free_host(host_dst)
```

2. Host 到 Device (ACL_MEMCPY_HOST_TO_DEVICE) 将 Host 侧的数据上传至 NPU (Device 侧) 是推理任务的起点。该过程支持同步或异步方式。若采用异步上传，需确保 Host 内存满足 Device 的访问要求，通常建议使用 `acl.rt.malloc_host` 申请。在异步模式下，开发者必须显式创建 Stream，并在后续使用 Device 数据前执行同步等待操作，以确保数据传输完整。

以下是同步与异步拷贝的实现示例：

```
1 # 准备内存
2 host_ptr, _ = acl.rt.malloc_host(size)
3 dev_ptr, _ = acl.rt.malloc(size, 2) # 2 代表 ACL_MEM_MALLOC_HUGE (Device 内存)
4 read_file(file, host_ptr, size)
5
6 # 方式一：同步拷贝
7 # 2 代表 ACL_MEMCPY_HOST_TO_DEVICE
8 acl.rt.memcpy(dev_ptr, size, host_ptr, size, 2)
9
10 # 方式二：异步拷贝
11 stream, _ = acl.rt.create_stream()
12 acl.rt.memcpy_async(dev_ptr, size, host_ptr, size, 2, stream)
13 acl.rt.synchronize_stream(stream) # 等待数据传输完成
14
15 # 释放资源（注意顺序）
16 acl.rt.free_host(host_ptr)
17 acl.rt.free(dev_ptr)
18 acl.rt.destroy_stream(stream)
```

3. Device 到 Host (ACL_MEMCPY_DEVICE_TO_HOST) 将 Device 侧的计算结果回传至 Host 侧是推理流程的最后一步，主要用于后续的业务结果解析或数据持久化。异步回传支持与 Device 侧的计算任务并行处理，但必须在 Host 侧访问该数据前执行 Stream 同步。在内存分配上，建议使用 `acl.rt.malloc_host` 申请 Host 侧缓冲区，以确保 Device 侧的 DMA 能够高效地完成数据交互。

```
1 # 准备内存
2 out_dev, _ = acl.rt.malloc(out_size, 2)
3 out_host, _ = acl.rt.malloc_host(out_size)
```



```

4
5 # 假设 Device 侧计算已完成, 执行异步拷贝 (3 代表 ACL_MEMCPY_DEVICE_TO_HOST)
6 acl.rt.memcpy_async(out_host, out_size, out_dev, out_size, 3, stream)
7
8 # 必须等待拷贝流执行完毕, 确保数据已完整到达 Host
9 acl.rt.synchronize_stream(stream)
10
11 # 此时可进行后处理并释放资源
12 acl.rt.free(out_dev)
13 acl.rt.free_host(out_host)

```

4. Device 到 Device (ACL_MEMCPY_DEVICE_TO_DEVICE) 在 Device 内部不同缓冲区之间进行拷贝, 或者在多 Device 场景下跨设备传输数据, 这里主要注意的是, 对于昇腾 310B 开发板来说, 一般来说, 一个昇腾 310B 开发板只有一个 NPU, 也就是说, 只有一个 Device。这种模式常用于数据格式重排或设备间通信。

该操作通常配合异步 Stream 使用, 以避免阻塞 Host 线程。在执行跨设备拷贝时, 开发者需要注意上下文切换或使用特定的点对点传输接口。以下是同一 Device 内执行异步拷贝的示例代码。程序首先申请两个 Device 内存块, 随后通过 `acl.rt.memcpy_async` 执行拷贝, 并将传输类型指定为 4 (即 `ACL_MEMCPY_DEVICE_TO_DEVICE`):

```

1 # 同一 Device 内的异步拷贝
2 dev_a, _ = acl.rt.malloc(size, 2)
3 dev_b, _ = acl.rt.malloc(size, 2)
4 stream, _ = acl.rt.create_stream()
5
6 # 4 代表 ACL_MEMCPY_DEVICE_TO_DEVICE
7 acl.rt.memcpy_async(dev_b, size, dev_a, size, 4, stream)
8 acl.rt.synchronize_stream(stream)
9
10 # 释放资源
11 acl.rt.free(dev_a)
12 acl.rt.free(dev_b)
13 acl.rt.destroy_stream(stream)

```

该模式常用于设备内数据重排或多设备间的点对点传输。在昇腾 310B 等单 NPU 平台上, 跨设备传输并不常见; 在多 NPU 环境下, 应通过切换 Context 或使用专用点对点传输接口以减少额外拷贝开销。建议始终配合异步 Stream 并在访问目标缓冲区前调用同步接口 (如 `synchronize_stream` 或 `stream_wait_event`) 以确保数据已就绪, 同时注意内存对齐与访问权限, 避免 DMA 访问异常。

通用开发注意事项

- 1. 异步与同步:** 异步拷贝接口 (`memcpy_async`) 必须配合 Stream 以及 `synchronize_stream` 使用, 否则无法保证数据一致性。
- 2. 内存类型:** 为了提升传输效率, Host 侧供 Device 访问的内存建议始终使用 `acl.rt.malloc_host` 申请。
- 3. 释放顺序:** 资源释放应遵循严格的逆序原则——先销毁 Stream, 再释放内存, 最后重置 Device。
- 4. 错误处理:** 示例代码为保持简洁略去了错误检查, 但在实际开发中, 所有 ACL 接口调用的返回值 (`ret`) 都必须进行检查 (0 表示成功)。

4.1.7. 同步等待机制

从上一节关于内存拷贝的四种路径中,我们可以观察到 `acl.rt.memcpy` 与 `acl.rt.memcpy_async` → 两种截然不同的操作模式。这两种模式的选择,本质上是在**编程简易性**与**执行性能**之间做权衡。

同步与异步的概念

同步操作 (Synchronous) 是最符合直觉的编程方式,它遵循严格的“请求-响应”逻辑。正如我们在 Host 到 Host 拷贝示例中所见,当 Host 线程发起一个同步指令时(如 `acl.rt.memcpy`),CPU 会挂起当前线程,像监工一样死死盯着任务,直到任务彻底完成后才会恢复执行下一行代码。这种模式的优点显而易见:逻辑简单,数据一致性由代码执行顺序天然保证,非常适合初学者进行功能验证或定位 BUG。但其缺点也同样致命:它完全抹杀了硬件并行的可能性。试想,当 CPU 在傻傻等待数据从 Host 搬运到 Device 时,昂贵的 NPU 计算单元可能正处于空闲状态,导致系统整体吞吐量下降。

异步操作 (Asynchronous) 则打破了这种串行束缚,是高性能 AI 应用的标配。在调用 `acl.rt.memcpy_async` 时,Host 线程仅需将任务“投递”到 Stream 队列中便立即返回,继续处理其他逻辑(如读取下一张图片或预处理数据)。此时,底层的 DMA 搬运引擎会与 NPU 的计算引擎同时工作,实现了真正的**软硬件并行**。这就好比点餐系统,前台服务员(Host)只负责快速接单并把单子(Task)扔给厨房(Stream),而不需要站在厨房门口等菜做好,从而能接待更多的客人。在复杂的 AI 业务流中,这种机制允许我们构建精妙的流水线:**在 NPU 拼命推理当前帧的同时, DMA 正在默默地将下一帧数据搬运进显存,而 CPU 已经在预处理第三帧数据**。这种“想尽办法让显卡即使一毫秒都不闲着”的设计,正是提升 AI 应用帧率的关键。

然而,异步操作是一把双刃剑,带来的性能红利必须以**严谨的同步控制**为代价。因为 Host 线程“投递”完任务就跑了,如果不加干预,它很可能在数据还没搬运完时就开始尝试读取结果,或者在 NPU 还没用完数据时就释放了内存,从而引发数据错乱甚至程序崩溃。因此,异步开发必须配合**同步等待机制**,在关键的时间节点上让“脱缰”的并行任务重新对齐。

PyACL 的四种同步机制

PyACL 提供了四种不同粒度的同步等待机制,以适应从简单的单流控制到复杂的多流并行协作等不同场景。正确选择同步方式是平衡 Host 侧控制流与 Device 侧数据流的关键。

1. **Event 的同步等待 (`acl.rt.synchronize_event`)** Event 是 PyACL 中的“时间锚点”或“完成标记”。这种机制允许 Host 侧主动阻塞,直到某个特定的 Event 在 Device 侧被触发。其典型流程是:开发者创建一个 Event,将其记录(Record)到某个 Stream 的任务队列中;当 Stream 执行到该记录点时,会在硬件层面触发该 Event;Host 线程通过调用 `synchronize_event` 暂停自身执行,直至接收到触发信号。这种方式粒度适中,适合 Host 需要等待某个特定任务节点完成后再进行后续逻辑(如读取特定阶段的结果),且

该 Event 可能被多个等待者复用的场景。

```

1 # 伪代码: Host 等待特定 Event
2 event, _ = acl.rt.create_event()
3 acl.rt.memcpy_async(dev_ptr, size, host_ptr, size, 1, stream) # 异步拷贝
4 acl.rt.record_event(event, stream) # 在流中安插一个锚点
5
6 # ... Host 执行其他不依赖数据的逻辑 ...
7
8 acl.rt.synchronize_event(event) # Host 在此阻塞, 直到上面的拷贝完成
9 # 此时可以安全释放 host_ptr 或读取 dev_ptr

```

2. **Stream 内任务的同步等待 (acl.rt.synchronize_stream)** 这是最常用且最直观的同步方式, 它表示 Host 侧阻塞等待指定 Stream 中的所有已提交任务全部执行完毕。其语义简单直接, 保证了指定流水线上的所有操作都已落盘。常用于单 Stream 流水线的收尾阶段, 或者 Host 必须确保整个任务队列清空后才能继续 (例如释放 Stream 资源前)。虽然简单, 但它是粗粒度的阻塞操作, 如果 Stream 中积压任务较多, 会导致 Host 长时间等待。

```

1 # 伪代码: Host 等待整个流清空
2 acl.rt.memcpy_async(..., stream) # 任务1
3 acl.op.execute_v2(..., stream)   # 任务2
4 acl.rt.memcpy_async(..., stream) # 任务3
5
6 acl.rt.synchronize_stream(stream) # Host 阻塞, 直到任务1,2,3全部完成
7 # 此时所有任务均已结束

```

3. **Stream 间的同步等待 (acl.rt.stream_wait_event)** 这是实现多流并行与主要流水线协作的核心机制。与前两者不同, stream_wait_event 不会阻塞 Host 线程, 而是向 Consumer Stream (消费者流) 中下发一个“等待指令”。Consumer Stream 执行到该指令时, 会暂停处理后续任务, 直到 Producer Stream (生产者流) 触发了指定的 Event。这种机制完全在 PyACL 内部调度和 Device 硬件层面完成, Host 仅负责编排逻辑, 从而最大化了 CPU 与 NPU 的并行效率。它非常适合跨流的数据依赖场景, 例如: Stream A 负责数据搬运, Stream B 负责计算, Stream B 必须等待 Stream A 搬运完成后才能开始计算。

```

1 # 伪代码: Stream 间协作 (不阻塞 Host)
2 # Stream A: 搬运数据 -> Record Event
3 acl.rt.memcpy_async(dev_input, ..., stream_a)
4 acl.rt.record_event(event, stream_a) # 记录搬运完成
5
6 # Stream B: 等待数据 -> 开始计算
7 acl.rt.stream_wait_event(stream_b, event) # Stream B 暂停, 直到 Stream A
      ↳ 触发 Event
8 acl.mdl.execute_async(..., stream_b)     # 只有等数据搬运完了, 才会执行推
      ↳ 理
9
10 # Host 这里是非阻塞的, 可以立即继续运行

```

4. **Device 的同步等待 (acl.rt.synchronize_device)** 这是粒度最粗的全局同步操作。调用此接口会阻塞 Host 进程, 直到当前 Context 绑定的 Device 上所有 Stream 的所有任务

全部完成。虽然它能保证设备层面的全局一致性，但由于其“一刀切”的等待特性，极大破坏了并行性，且开销最高。通常仅在程序初始化调试、全局状态检查或程序退出前的最终资源回收阶段使用，在高性能的生产业务循环中应尽量避免。

```
1 # 伪代码：全局等待
2 acl.rt.memcpy_async(..., stream1)
3 acl.rt.memcpy_async(..., stream2)
4
5 # 强制等待设备上所有任务结束（调试或退出时使用）
6 acl.rt.synchronize_device()
```

昇腾 310B 最佳同步策略与场景分析

在昇腾 310B 这种边缘计算平台上，CPU 的单核性能通常弱于服务器级 CPU，因此**减少 Host 侧（CPU）的阻塞**、把调度压力卸载给 Device 侧硬件显得尤为关键。为了更直观地理解为什么在昇腾 310B 上必须强调“CPU 减负”与“异步并行”，我们需要深入分析这颗 SoC 的 CPU 性能定位。与市面上其他主流的边缘计算开发板相比，昇腾 310B 呈现出显著的“**NPU 极强，CPU 较弱**”的异构特性。

下表详细对比了昇腾 310B 与树莓派 5、RK3588 以及 NVIDIA Jetson Orin Nano 在 CPU 规格上的差异：

平台名称	核心处理器 (SoC)	CPU 架构	核心配置	主频 (典型)	CPU 算力估算 (UnixBench)
昇腾 310B	Ascend 310B	ARM Cortex-A55	4 核 (纯小核)	1.0 GHz ~ 1.6 GHz	~ 400 - 600 分
树莓派 5	BCM2712	ARM Cortex-A76	4 核 (大核)	2.4 GHz	~ 2400+ 分
Orange Pi 5	RK3588	Cortex-A76 + A55	4 大核 + 4 小核	2.4 GHz (大) / 1.8 GHz (小)	~ 5000+ 分
Jetson Orin Nano	Tegra Orin	ARM Cortex-A78AE	6 核 (高性能核)	1.5 GHz	~ 3500+ 分
桌面 PC	Intel i7-12700K	x86-64 (Alder Lake)	8 大核 + 4 小核	3.6 GHz ~ 5.0 GHz	~ 45000+ 分

数据解读与性能差距分析：

- 1. 架构代差 (效率核 vs 性能核): 昇腾 310B 采用的是 **Cortex-A55** 架构。在 ARM 的设计体系中，A55 被定义为“高能效核心 (Efficiency Core)”，通常用于手机处理器的后台任务

或物联网设备，其侧重点在于低功耗而非高性能。相比之下，树莓派 5 使用的 Cortex-A76 和 Orin Nano 使用的 Cortex-A78AE 均属于“高性能核心（Performance Core）”。在同频下，A76 的整数计算能力约为 A55 的 2~3 倍，浮点性能更是相差甚远。

2. **绝对性能差距：**从综合基准测试（如 UnixBench 或 Geekbench）来看，昇腾 310B 的 CPU 性能大约仅为树莓派 5 的 **1/4 到 1/5**，仅为 RK3588 的 **1/8** 左右。这意味着，同样一段基于 OpenCV 的纯 CPU 图像缩放代码，在树莓派 5 上可能耗时 5ms，而在昇腾 310B 上可能耗时 25ms 甚至更多。
3. **与桌面 CPU 的鸿沟：**如果是从在笔记本（x86, i5/i7）上开发算法迁移到 310B，这种落差会更加剧烈，性能折损可能达到 **50 倍甚至 100 倍**。桌面 CPU 拥有巨大的二级/三级缓存和乱序执行能力，这掩盖了 Python 解释器本身的低效。但在 310B 上，Python 的 Global Interpreter Lock (GIL) 加上较弱的单核性能，极易成为系统的最大短板。

对 PyACL 开发的启示：

基于上述硬件数据，我们在开发中必须遵循以下“生存法则”：
*** 不要信任 CPU 的浮点计算能力：**任何涉及图像 Pixels 遍历的操作（如 Resize, Color Convert, Normalize）若写在 CPU (Python) 端，必将导致帧率骤降。**必须使用 DVPP 硬件加速**（详见第 5 章）。
*** Python 代码仅做胶水：**Python 逻辑应仅限于流程控制、参数配置和极少量的后处理。业务主体必须由底层的 C++ 算子或 NPU 模型承担。
*** 异步是救命稻草：**由于 CPU 处理每一行 Python 代码都比其他平台慢，因此更不能让 CPU 傻傻等待 NPU（同步）。只有利用 `stream_wait_event` 让 CPU 快速把任务分发完并脱身，才能掩盖 A55 核心性能不足的缺陷。

1. 优先策略：全链路异步与细粒度同步 在昇腾 310B 平台上，由于 CPU 算力相对有限，最理想的流水线策略是实施“全链路异步”设计，即让 CPU 专注于指令分发，而将繁重的计算负载全权交由 NPU 负责。开发者应尽量避免使用 `acl.rt.memcpy` 等同步阻塞接口，转而全面采用 `acl.rt.memcpy_async` 配合 Event 机制。这种方法的核心原则在于，凡是能通过 Device 侧 `stream_wait_event` 解决的任务依赖，绝不让 Host 侧的 CPU 介入干预。例如，在典型的多级推理流水线中，我们可以安排 Stream A 负责视频解码（DVPP），Stream B 负责模型推理。当 Stream A 完成解码任务后，只需在流中记录一个 Event，而 Stream B 仅需等待该 Event 触发即可启动推理。整个交互过程中，CPU 仅需极低开销下发几条控制指令，完全无需挂起等待解码结束，从而能腾出宝贵算力去处理复杂的业务逻辑或网络通信，实现真正的软硬件解耦与并行。

```

1 # 伪代码：利用 Event 实现解码与推理的异步流水线
2 # stream_dvpp 负责解码，stream_infer 负责推理
3 acl.media.dvpp_jpeg_decode_async(..., stream_dvpp)
4
5 # 1. 在解码流中记录一个“完成节点”
6 event_decode_done, _ = acl.rt.create_event()
7 acl.rt.record_event(event_decode_done, stream_dvpp)
8
9 # 2. 让推理流等待这个节点 (Host 不阻塞，仅 NPU 内部等待)
10 acl.rt.stream_wait_event(stream_infer, event_decode_done)

```

```

11
12 # 3. 只有当解码完成后，推理流才会开始执行模型
13 acl.mdl.execute_async(..., stream_infer)

```

2. 数据传输优化：Host Pinned Memory 的强制管理 实现异步传输的高性能前提是 Host 侧内存的稳定性。在使用 `acl.rt.memcpy_async` 进行 Host 到 Device 的数据搬运时，源端的 Host 内存必须是通过 `acl.rt.malloc_host` 申请的页锁定内存（Pinned Memory）。普通的 `malloc` 申请的内存可能会被操作系统分页机制换出到磁盘交换区，导致 DMA 控制器无法安全、准确地访问数据。此外，内存的生命周期管理至关重要。在释放 Host 侧指针之前，必须通过 `acl.rt.synchronize_stream` 等手段确保所有涉及该内存块的 Stream 任务都已彻底执行完毕。如果过早释放内存，NPU 正尝试读取数据时地址已失效，将导致读取到野指针数据，引发难以复现的推理错误或系统崩溃。

```

1 # 伪代码：异步传输的内存管理规范
2 # 必须使用 acl.rt.malloc_host 申请 Pinned Memory
3 host_ptr, _ = acl.rt.malloc_host(size)
4
5 # ... 填充数据到 host_ptr ...
6
7 # 执行异步拷贝
8 acl.rt.memcpy_async(dev_ptr, size, host_ptr, size, 1, stream)
9
10 # 错误做法：直接释放 host_ptr (DMA 可能还在搬运中！)
11 # acl.rt.free_host(host_ptr)
12
13 # 正确做法：先同步等待流结束，再释放
14 acl.rt.synchronize_stream(stream)
15 acl.rt.free_host(host_ptr)

```

3. 多 Stream 协作范式：生产者-消费者模型 构建基于生产者-消费者模型的多 Stream 协作机制，是挖掘 310B 硬件潜能、提升应用帧率（FPS）的关键手段。具体的实施路径是将通过 Stream 的划分将任务解耦为“预处理”、“推理”和“后处理”等独立阶段，通过 Event 机制实现流水线衔接。这种模式下，前一个阶段完成后记录 Event，后一个阶段感应 Event 并启动，就像工厂流水线一样运转。其最大优势在于避免了 CPU 使用 `while` 循环去轮询 NPU 的状态，极大降低了 CPU 占用率。对于昇腾 310B 这类嵌入式 SoC 而言，节省下来的 CPU 开销意味着开发者可以在 Python 层运行更复杂的逻辑判断，从而提升整个系统的智能化水平。

```

1 # 伪代码：多 Stream 协作
2 # 循环中处理每一帧
3 for image in images:
4     # 阶段 1: 预处理流 (Stream A)
5     preprocess(image, stream_a)
6     acl.rt.record_event(evt_pre, stream_a)
7
8     # 阶段 2: 推理流 (Stream B)
9     # 只有当预处理完成，推理才开始
10    acl.rt.stream_wait_event(stream_b, evt_pre)

```

```

11 model_inference(stream_b)
12 acl.rt.record_event(evt_infer, stream_b)
13
14 # 阶段 3: 后处理流 (Stream C) or Host 读取
15 # 只有推理完成, 才处理结果
16 acl.rt.stream_wait_event(stream_c, evt_infer)
17 post_process(stream_c)

```

4. 避免滥用全局同步与调试建议 在追求高性能的同时，必须警惕对同步接口的滥用。最典型的反模式是在每一帧的处理循环中都调用 `acl.rt.synchronize_device()`，这种做法相当于强制将所有并行的流水线“拍扁”为串行执行，完全浪费了 NPU 的并行处理能力。全局同步应当被严格限制在程序初始化（确保设备状态复位）、调试阶段（精确定位错误发生的算子）或进程退出阶段（安全回收所有资源）。对于开发者而言，调试异步程序确实存在挑战，因为报错行往往滞后于实际错误发生点。因此，建议遵循“从同步到异步”的演进路线：在开发初期，全部使用同步接口（如 `synchronize_stream`）确保逻辑正确性和内存安全；进入性能调优阶段后，再逐步替换为异步接口并引入 Event 机制，配合 Profiling 工具观察流水线中的空隙（Bubble），逐步压榨硬件性能。

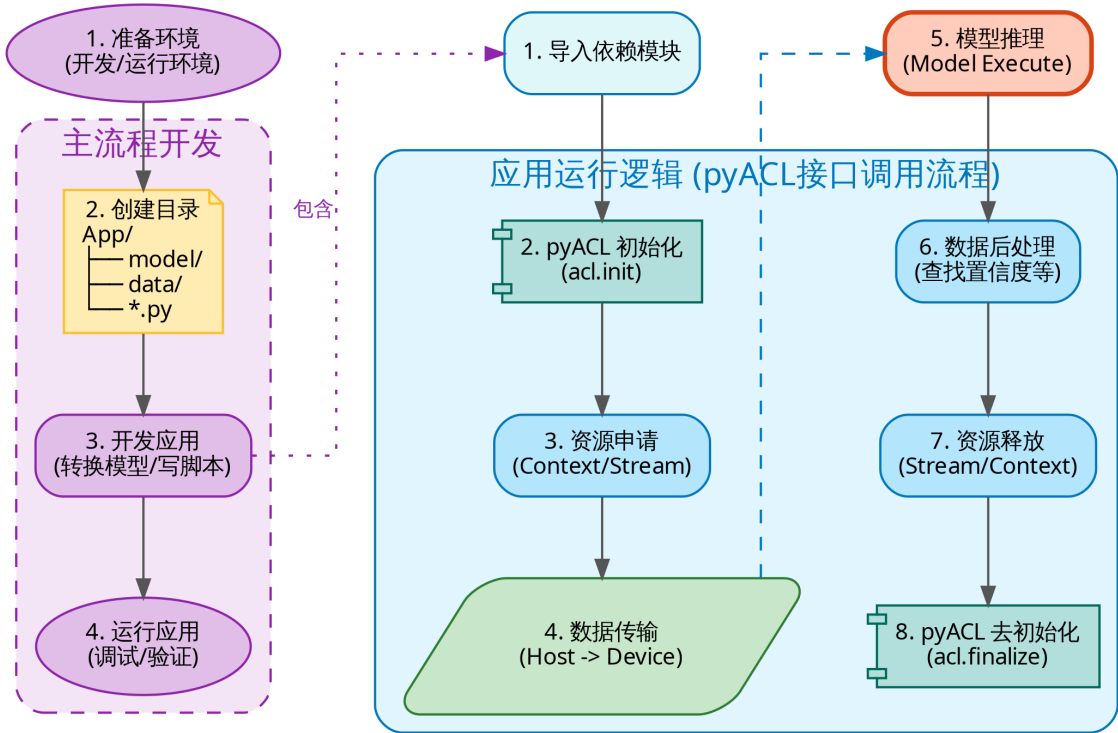
小结 掌握 PyACL 同步机制的核心在于理解“控制流（CPU）与数据流（NPU）的分离”。在昇腾 310B 开发中，优秀的架构设计应当是：Host 线程像一个从容的指挥官，通过 Event 编排好各 Stream 的协作顺序后便抽身而去，留给 NPU 硬件去并行处理繁重的数据搬运与计算任务。合理使用 `stream_wait_event` 实现 Device 内部的依赖隔离，仅在必须获取最终结果时使用 `synchronize_stream` 回收数据，是在边缘端实现高性能推理的黄金法则。

4.2. 模型推理流水线

模型推理是 PyACL 应用开发的核心场景。为了能够让训练好的深度学习模型在昇腾 AI 处理器上高效运行，开发者需要遵循一套从环境准备到资源释放的标准化全流程。

4.2.1. 模型推理开发流程解析

整个模型推理应用的构建过程可以宏观地分为“主流程开发”与“应用运行逻辑”两个层面，如下图所示：



{fig:pyacl_inference width=100% .center}

主流程开发（Preparation Strategy）

主流程开发主要涉及应用构建的物理层面的准备工作。首先，开发者需要确保昇腾 AI 处理器的驱动与固件、CANN 软件（包含 pyACL）以及 Python 运行环境均已正确安装并完成环境变量配置，这是应用运行的基石。其次，为了保证项目的可维护性，建议采用标准化的目录结构，例如规划专门的 model/ 目录存放离线模型、data/ 目录存放测试图片或数据集，以及独立的脚本目录。紧接着进入核心开发阶段，这包括使用 ATC 工具将原始框架（如 ONNX 或 PyTorch）的模型转换为昇腾专用的 .om 离线模型，以及编写 Python 主程序以串联推理逻辑。最后，开发者需要在板端实际执行脚本，进行应用的验证与调试，确保从模型转换到最终输出的整个链路畅通无阻。

应用运行逻辑（Execution Logic）

这是编写 Python 脚本时的代码执行时序，严格遵循 PyACL 的接口调用规范，主要包含以下八个关键步骤：

- 1. 导入依赖：import acl 引入 pyACL 库。
- 2. 系统初始化：调用 acl.init() 进行全局配置初始化，这是一切操作的起点。

3. **资源申请**：创建 Device 连接，配置 Context（上下文）与 Stream（执行流），为后续计算搭建“舞台”。
4. **数据传输 (Host -> Device)**：在执行推理前，必须将待处理的图片或矩阵数据从 CPU 侧 (Host) 搬运至 NPU 侧 (Device)。这通常涉及 `acl.rt.malloc` 申请 Device 内存以及 `acl.rt.memcpy` 执行搬运。
5. **模型推理 (Inference)**：这是流程的核心。调用 `acl.mdl.execute` 接口，指示 AI Core 利用加载的 `.om` 模型对 Device 内存中的数据进行计算。
6. **数据后处理**：将推理产生的结果（通常在 Device 侧）回传至 Host 侧（或直接在 Device 侧处理），通过 Python 代码解析概率向量 (Softmax)、筛选置信度或进行坐标转换。
7. **资源释放**：业务完成后，必须按“先释放 Stream，再释放 Context，最后重置 Device”的逆序释放资源。
8. **系统去初始化**：最后调用 `acl.finalize()`，通知系统回收全局资源，结束进程。

4.2.2. 实例分析 (ResNet-18)

在第三章中，我们初步介绍了 ResNet 的网络结构。本章我们将继续以 ResNet 为例，深入探讨 PyACL 的编程技巧与应用。上一章中，由于昇腾 310B 的 PyTorch 插件主要用于推理加速，且端侧设备训练算力有限，我们选用了较小的 CIFAR-10 数据集。而在本章，我们将采用更贴近实际生产的标准开发流程，并引入 **Tiny-ImageNet** 数据集进行实战演练。我们首先利用 Nvidia 显卡配合 CUDA 和 PyTorch 对 ResNet-18 进行训练，获取模型权重；随后将其转换为 ONNX 模型，并最终转换为昇腾特定的 OM 模型。我们将分别在**昇腾 310B (NPU-8T)**、**昇腾 310B (CPU)**、**树莓派 5B (CPU)** 以及 RTX 5090D 上进行推理测试，重点对比 OnnxRuntime 与 PyACL 的性能差异，深入分析 NPU 带来的帧率提升。

Tiny-ImageNet 数据集简介

Tiny-ImageNet 是大规模视觉识别挑战赛 (ILSVRC) 中 ImageNet 数据集的一个微型子集，常被用于深度学习模型的快速原型设计与基准测试。该数据集包含 200 个不同的物体类别，这一数量远超 CIFAR-10 的 10 个类别，从而更具挑战性，能更好地验证模型的泛化能力。在数据规模方面，它拥有 100,000 张训练图片（每个类别 500 张）、10,000 张验证图片（每个类别 50 张）以及 10,000 张测试图片。所有图像的分辨率统一为 64x64 像素，虽然低于标准 ImageNet 的 224x224，但相比 CIFAR-10 的 32x32 分辨率包含了更多细节，非常适合在算力受限的嵌入式设备上进行中规模实验。

在模型选择上，尽管 ResNet-50 或 ResNet-101 拥有更深的网络层数和潜在的更高精度，但在嵌入式 AI 开发场景下，ResNet-18 往往是性能与效率的最佳平衡点。首先，考虑到昇腾 310B 定位为边缘计算设备，ResNet-18 约 11M 的参数量和适中的计算量能够更直观地体现 NPU 在高吞吐场景下的加速优势，避免因网络过大导致的内存瓶颈掩盖推理效率。其次，对于 64x64 分辨率的 Tiny-ImageNet，ResNet-18 已具备足够的特征提取能力，使用过深的网络反而容易

导致过拟合且训练耗时过长，不利于教学演示与快速迭代。最后，ResNet-18 作为业界最通用的轻量级骨干网络之一，常被作为衡量树莓派、Jetson 以及 Ascend 等不同端侧硬件性能的经典标尺。

ResNet-18 模型训练

注意注意：本节训练代码需在配备 NVIDIA 显卡的 GPU 服务器上运行，不在昇腾 310B 开发板上执行。如果只需体验昇腾 310B 上 PyACL 的推理性能，可直接跳转至 [环境配置与模型转换](#)。

如果你希望自行训练模型以便进行测试，可以参考本节的模型训练部分。我们需要一台配备 Nvidia 显卡并已正确安装 CUDA 驱动的服务器。本例中，我们使用 GeForce 5090D 显卡。对于图像分类任务，Nvidia GeForce 系列显卡已能很好地满足需求。我们选用 Tiny-ImageNet 数据集进行训练，该数据集相比完整版 ImageNet 体积更小，便于测试和实验。使用 GeForce 5090D 训练一次大约需要 1~2 小时；而若采用完整的 ImageNet 数据集，单卡训练可能需要长达一周的时间。

我们需要从 Hugging Face 下载 Tiny-ImageNet 数据集，为此首先需要使用 pip 安装 Hugging Face 的 CLI 工具。如果在下载过程中遇到网络连接问题，建议配置镜像源，例如使用 hf-mirror。可以通过以下命令设置环境变量来启用镜像：

```
1 export HF_ENDPOINT=https://hf-mirror.com
```

为了方便起见，我们可以运行以下命令将该环境变量添加到 .bashrc 文件中，使其永久生效：

```
1 echo 'export HF_ENDPOINT=https://hf-mirror.com' >> ~/.bashrc
2 source ~/.bashrc
```

为了避免影响服务器上已有的虚拟环境，建议提前安装 Anaconda，并通过以下命令新建独立的环境：

```
1 conda create -n torch
2 conda activate torch
3 conda install python=3.11
```

此处选择 Python 3.11 作为示例，实际可根据需求选择合适的版本，但不建议使用过新或过旧的版本，以减少兼容性问题。

随后，使用 pip 安装所需依赖：

```
1 pip install datasets huggingface_hub torch torchvision timm tensorboard
```

本例采用的 ResNet-18 模型在结构上做了针对 Tiny-ImageNet 的适配。与标准 ResNet-18 不同，输入层采用 3x3 卷积且 stride=1，移除了原有的 7x7 卷积和最大池化层，以更好地保留 64x64 小尺寸图像的空间信息。此外，模型中增加了 Dropout 层以缓解过拟合，并在损失函数

中引入标签平滑 (Label Smoothing)。优化器选用带动量和权重衰减的 SGD，并配合多步学习率调度器，进一步提升模型的泛化能力。

数据集的下载和加载通过 Hugging Face 的 datasets 库实现。只需一行代码即可自动下载并缓存 Tiny-ImageNet 数据集，无需手动解压和整理文件。示例代码如下：

```
1 from datasets import load_dataset
2 dataset = load_dataset('zh-plus/tiny-imagenet', cache_dir='./data')
```

在数据预处理部分，训练集采用了多种图像增强技术，包括随机裁剪 (RandomCrop)、随机水平翻转 (RandomHorizontalFlip)、随机旋转 (RandomRotation) 以及颜色抖动 (ColorJitter)。这些增强方法能够人为增加训练样本的多样性，使模型在训练过程中见到更多变化的图像，有效缓解过拟合问题，提高模型的泛化能力。

此外，模型训练过程中采用了 MultiStepLR 学习率调度策略 (multistep)，即在训练到指定 epoch 时自动降低学习率。这种做法可以让模型在初期快速收敛，在后期以更小的步长微调参数，进一步防止过拟合。图像增强和多步学习率调度的结合，有助于提升模型在验证集和实际应用中的表现。

完成的模型训练代码如下：

```
1 import os
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 from torch.utils.data import DataLoader
6 from datasets import load_dataset
7 from torchvision.transforms import Compose, ToTensor, Normalize, RandomCrop,
8   ↪ RandomHorizontalFlip, RandomRotation, ColorJitter # 增加更多增强
9 from torch.utils.tensorboard import SummaryWriter # 导入 SummaryWriter
10
11 # 自定义 ResNet 模型组件
12 class BasicBlock(nn.Module):
13     expansion = 1
14
15     def __init__(self, inplanes, planes, stride=1, downsample=None):
16         super(BasicBlock, self).__init__()
17         self.conv1 = nn.Conv2d(inplanes, planes, kernel_size=3, stride=stride,
18   ↪ padding=1, bias=False)
19         self.bn1 = nn.BatchNorm2d(planes)
20         self.relu = nn.ReLU(inplace=True)
21         self.conv2 = nn.Conv2d(planes, planes, kernel_size=3, padding=1, bias=
22   ↪ False)
23         self.bn2 = nn.BatchNorm2d(planes)
24         self.downsample = downsample
25         self.stride = stride
26
27     def forward(self, x):
28         identity = x
29         out = self.conv1(x)
30         out = self.bn1(out)
31         out = self.relu(out)
32         out = self.conv2(out)
33         out = self.bn2(out)
34         if self.downsample is not None:
```

```

32         identity = self.downsample(x)
33         out += identity
34         out = self.relu(out)
35         return out
36
37 # 移除 Bottleneck 类, ResNet18 使用 BasicBlock
38 class ResNet(nn.Module):
39     def __init__(self, block, layers, num_classes=200):
40         super(ResNet, self).__init__()
41         self.inplanes = 64
42         # 适配 Tiny ImageNet (64x64): 使用 3x3 卷积, stride=1, 移除 MaxPool
43         self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=
            ↪ False)
44         self.bn1 = nn.BatchNorm2d(64)
45         self.relu = nn.ReLU(inplace=True)
46         # self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1) # 移除
47
48         self.layer1 = self._make_layer(block, 64, layers[0])
49         self.layer2 = self._make_layer(block, 128, layers[1], stride=2)
50         self.layer3 = self._make_layer(block, 256, layers[2], stride=2)
51         self.layer4 = self._make_layer(block, 512, layers[3], stride=2)
52         self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
53         self.dropout = nn.Dropout(p=0.5) # 增加 Dropout 层
54         self.fc = nn.Linear(512 * block.expansion, num_classes)
55
56         for m in self.modules():
57             if isinstance(m, nn.Conv2d):
58                 nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='
                    ↪ relu')
59             elif isinstance(m, nn.BatchNorm2d):
60                 nn.init.constant_(m.weight, 1)
61                 nn.init.constant_(m.bias, 0)
62
63     def _make_layer(self, block, planes, blocks, stride=1):
64         downsample = None
65         if stride != 1 or self.inplanes != planes * block.expansion:
66             downsample = nn.Sequential(
67                 nn.Conv2d(self.inplanes, planes * block.expansion, kernel_size
                    ↪ =1, stride=stride, bias=False),
68                 nn.BatchNorm2d(planes * block.expansion),
69             )
70         layers = []
71         layers.append(block(self.inplanes, planes, stride, downsample))
72         self.inplanes = planes * block.expansion
73         for _ in range(1, blocks):
74             layers.append(block(self.inplanes, planes))
75         return nn.Sequential(*layers)
76
77     def forward(self, x):
78         x = self.conv1(x)
79         x = self.bn1(x)
80         x = self.relu(x)
81         # x = self.maxpool(x) # 移除
82         x = self.layer1(x)
83         x = self.layer2(x)
84         x = self.layer3(x)
85         x = self.layer4(x)
86         x = self.avgpool(x)

```

```

87         x = torch.flatten(x, 1)
88         x = self.dropout(x) # 应用 Dropout
89         x = self.fc(x)
90         return x
91
92 def train_resnet18_on_tiny_imagenet():
93     # 定义保存路径
94     data_dir = './data'
95     model_dir = './model'
96     log_dir = './logs/resnet18_tiny_imagenet' # TensorBoard 日志目录
97     os.makedirs(model_dir, exist_ok=True)
98
99     # 初始化 TensorBoard Writer
100    writer = SummaryWriter(log_dir)
101
102    # 加载数据集 (指定 cache_dir)
103    dataset = load_dataset('zh-plus/tiny-imagenet', cache_dir=data_dir) # 加载完
104    # 整数据集以获取 train 和 valid
105
106    # 数据预处理 (增加数据增强)
107    # 训练集: 增加随机裁剪、水平翻转、旋转和颜色抖动
108    train_transform = Compose([
109        RandomCrop(64, padding=4),
110        RandomHorizontalFlip(),
111        RandomRotation(15), # 增加随机旋转
112        ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1), # 增
113        # 加颜色抖动
114        ToTensor(),
115        Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
116    ])
117
118    # 验证集: 仅保持标准化
119    val_transform = Compose([
120        ToTensor(),
121        Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
122    ])
123
124    # 自定义数据集类
125    class TinyImageNetDataset(torch.utils.data.Dataset):
126        def __init__(self, dataset, transform=None):
127            self.dataset = dataset
128            self.transform = transform
129
130        def __len__(self):
131            return len(self.dataset)
132
133        def __getitem__(self, idx):
134            item = self.dataset[idx]
135            image = item['image']
136            label = item['label']
137            image = image.convert('RGB') # 确保图像为RGB格式
138            if self.transform:
139                image = self.transform(image)
140            return image, label
141
142    # 使用不同的 transform
143    train_dataset = TinyImageNetDataset(dataset['train'], transform=
144    # train_transform)

```

```

142 val_dataset = TinyImageNetDataset(dataset['valid'], transform=val_transform)
    ↳ # 假设 valid split 存在
143
144 train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True) # 将
    ↳ batch_size 适当调大一点, 例如 64 或 128
145 val_loader = DataLoader(val_dataset, batch_size=128, shuffle=False)
146
147 # 定义模型 (使用自定义 ResNet18: BasicBlock, [2, 2, 2, 2])
148 model = ResNet(BasicBlock, [2, 2, 2, 2], num_classes=200)
149 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
150 model.to(device)
151
152 # 损失函数和优化器 (更换为 SGD + Momentum + Weight Decay 以抗过拟合)
153 # 启用标签平滑 (Label Smoothing) 以防止过拟合
154 criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
155 # 使用 SGD 替代 Adam, 初始学习率设为 0.1, weight_decay=5e-4 用于正则化
156 optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9, weight_decay
    ↳ =5e-4)
157
158 # 添加学习率调度器, 在第 30, 60, 90 epoch 衰减学习率
159 scheduler = optim.lr_scheduler.MultiStepLR(optimizer, milestones=[30, 60,
    ↳ 90], gamma=0.1)
160
161 # 训练循环
162 num_epochs = 150
163 best_acc = 0.0
164
165 for epoch in range(num_epochs):
166     model.train()
167     running_loss = 0.0
168     correct_train = 0
169     total_train = 0
170
171     # 移除 tqdm, 使用简单的迭代
172     for i, (images, labels) in enumerate(train_loader):
173         images, labels = images.to(device), labels.to(device)
174         optimizer.zero_grad()
175         outputs = model(images)
176         loss = criterion(outputs, labels)
177         loss.backward()
178         optimizer.step()
179
180         running_loss += loss.item()
181         _, predicted = torch.max(outputs.data, 1)
182         total_train += labels.size(0)
183         correct_train += (predicted == labels).sum().item()
184
185     # 记录 iteration 级别的 loss (可选)
186     if i % 100 == 0:
187         writer.add_scalar('Training/Iter_Loss', loss.item(), epoch * len
            ↳ (train_loader) + i)
188
189     train_loss = running_loss / len(train_loader)
190     train_acc = 100. * correct_train / total_train
191
192 # 记录 epoch 级别的训练指标
193 writer.add_scalar('Training/Epoch_Loss', train_loss, epoch + 1)
194 writer.add_scalar('Training/Epoch_Accuracy', train_acc, epoch + 1)

```

```

195 # 更新学习率并记录
196 current_lr = optimizer.param_groups[0]['lr']
197 writer.add_scalar('Training/Learning_Rate', current_lr, epoch + 1)
198 scheduler.step()
199
200 # 验证阶段
201 model.eval()
202 correct_val = 0
203 total_val = 0
204
205 with torch.no_grad():
206     for images, labels in val_loader:
207         images, labels = images.to(device), labels.to(device)
208         outputs = model(images)
209         _, predicted = torch.max(outputs.data, 1)
210         total_val += labels.size(0)
211         correct_val += (predicted == labels).sum().item()
212
213 val_acc = 100. * correct_val / total_val
214
215 # 记录验证指标
216 writer.add_scalar('Validation/Accuracy', val_acc, epoch + 1)
217
218 print(f'Epoch {epoch+1}: LR: {current_lr:.6f}, Train Loss: {train_loss
    ↪ :.4f}, Train Acc: {train_acc:.2f}%, Val Acc: {val_acc:.2f}%')
219
220 # 保存 Checkpoint (每个 epoch 保存一次)
221 checkpoint_path = os.path.join(model_dir, f'checkpoint_epoch.pth')
222 torch.save({
223     'epoch': epoch + 1,
224     'model_state_dict': model.state_dict(),
225     'optimizer_state_dict': optimizer.state_dict(),
226     'loss': train_loss,
227     'val_acc': val_acc
228 }, checkpoint_path)
229 print(f"Checkpoint saved to {checkpoint_path}")
230
231 # 保存为ONNX (每当验证集准确率提升时保存, 或者最后保存)
232 if val_acc > best_acc:
233     best_acc = val_acc
234     # 保存最佳模型逻辑...
235
236 # 关闭 writer
237 writer.close()
238
239 # 保存为ONNX (输入尺寸调整为 64x64)
240 dummy_input = torch.randn(1, 3, 64, 64).to(device)
241 onnx_path = os.path.join(model_dir, 'resnet18_tiny_imagenet.onnx')
242 torch.onnx.export(model, dummy_input, onnx_path, verbose=True)
243 print(f"Model saved as ONNX: {onnx_path}")
244
245 if __name__ == '__main__':
246     train_resnet18_on_tiny_imagenet()

```

为了实时监控模型训练过程中的损失、准确率和学习率变化, 代码中集成了 TensorBoard。通过 SummaryWriter 将训练和验证指标写入日志文件, 用户可在训练过程中通过如下命令启动 TensorBoard:

```
1 tensorboard --logdir=./logs/resnet18_tiny_imagenet
```

在浏览器中访问对应端口，即可可视化每个 epoch 的损失、准确率曲线和学习率变化，便于分析模型收敛情况和调参效果。模型训练结束后，我们可以在 model 文件夹下面，找到名为 resnet18_tiny_imagenet.onnx 的模型文件。

环境配置与模型转换

我们已经训练好了一个针对 Tiny-ImageNet 的 ResNet-18 网络，并上传到了 HuggingFace 仓库 [zhouxzh/resnet18_tiny_imagenet](#)，因此不需要自行训练。

完整的可运行代码位于 [samples/chapter4/resnet18/](#) 目录下，可直接参考使用。

为了可以顺利的从 huggingface 下载已经训练好的模型，我们需要使用 pip 安装 Hugging Face 的 CLI 工具。

```
1 pip install datasets huggingface_hub
```

如果在下载过程中遇到网络连接问题，建议配置镜像源，例如使用 hf-mirror。可以通过以下命令设置环境变量来启用镜像：

```
1 export HF_ENDPOINT=https://hf-mirror.com
```

为了方便起见，我们可以运行以下命令将该环境变量添加到 .bashrc 文件中，使其永久生效：

```
1 echo 'export HF_ENDPOINT=https://hf-mirror.com' >> ~/.bashrc
2 source ~/.bashrc
```

接下来，我们在昇腾 310B 开发板上构建项目目录。请执行以下命令创建 resnet18 文件夹及必要的子目录结构：

```
1 mkdir -p resnet18
2 cd resnet18
3 mkdir -p {data,model}
```

随后，使用 Hugging Face CLI 下载预训练好的 ONNX 模型：

```
1 hf download zhouxzh/resnet18_tiny_imagenet --local-dir model/
```

下载完成后，我们需要使用 ATC（Ascend Tensor Compiler）工具将 ONNX 模型转换为昇腾 NPU 专用的 OM（Offline Model）模型。执行如下转换命令：

```
1 atc --model=model/resnet18_tiny_imagenet.onnx \
2     --framework=5 --output=model/resnet18_tiny_imagenet \
3     --input_shape="input.1:1,3,64,64" \
4     --soc_version=Ascend310B4
```

核心实现：模型推理脚本

为了在昇腾 310B 上加载 OM 模型并执行推理，我们需要编写一个完整的 Python 脚本。该脚本不仅负责调用底层 PyACL 接口，还需要处理数据的预处理（Preprocessing）与后处理（Postprocessing）。本节将通过解析 inference_npu.py 的关键代码片段，深入讲解 PyACL 应用的构建逻辑。

完整的推理代码逻辑被封装在 AclResource 类中，它管理着从初始化到资源释放的整个生命周期。

1. 初始化与模型加载

在类的 init 方法中，我们依次完成系统初始化、Device 指定、Context 创建以及模型加载。值得注意的是，模型加载后，我们需要获取“模型描述”（Model Description），它包含了模型输入输出张量的维度、大小和类型信息，这对于后续申请内存至关重要。

```

1 def init(self):
2     # 1. ACL 全局初始化
3     ret = acl.init()
4
5     # 2. 指定运算设备 (Device 0)
6     ret = acl.rt.set_device(self.device_id)
7
8     # 3. 创建 Context 上下文
9     self.context, ret = acl.rt.create_context(self.device_id)
10
11    # 4. 加载离线模型 (.om)
12    self.model_id, ret = acl.mdl.load_from_file(self.model_path)
13
14    # 5. 获取模型描述信息 (用于查询输入输出大小)
15    self.model_desc = acl.mdl.create_desc()
16    ret = acl.mdl.get_desc(self.model_desc, self.model_id)

```

2. 数据传输与内存管理

推理的核心在于 execute 方法。由于 NPU 无法直接读取 CPU（Host）侧的 Numpy 数组，我们需要显式地进行内存拷贝。

- **输入准备：**首先通过 `acl.mdl.get_input_size_by_index` 查询模型所需的输入字节数。接着使用 `acl.rt.malloc` 在 Device 侧申请内存，并利用 `acl.rt.memcpy` 将 host 侧预处理好的 Numpy 数据拷贝到 Device 侧。
- **Dataset 组装：**PyACL 使用 Dataset 和 DataBuffer 结构来传递数据。我们需要创建一个 Dataset 容器，并将包含 Device 内存地址的 DataBuffer 添加进去。
- **输出准备：**同样地，根据模型输出大小申请 Device 侧内存，并组装 Output Dataset，用于存放 NPU 计算后的结果。

为了更加清晰地理解 Host 与 Device 之间的数据交互流程，我们可以将上述复杂的 execute 函数逻辑简化为以下伪代码：

```

1 def execute(input_data_host):
2     # 1. 准备输入 (Host -> Device)

```



```

3  # 申请 NPU 侧内存，并将预处理好的图片数据从 CPU 搬运过去
4  input_size = get_model_input_size()
5  dev_in = acl.malloc_device(input_size)
6  acl.memcpy(dev_in, input_data_host, direction=HOST_TO_DEVICE)
7
8  # 2. 准备输出 (Device)
9  # 在 NPU 侧申请内存，用于存放模型推理产生的结果
10 output_size = get_model_output_size()
11 dev_out = acl.malloc_device(output_size)
12
13 # 3. 封装 Dataset
14 # PyACL 要求将内存指针封装为 Dataset 结构才能传入模型
15 input_dataset = create_dataset(dev_in)
16 output_dataset = create_dataset(dev_out)
17
18 # 4. 执行推理 (Inference)
19 # 指挥 NPU 执行计算，结果会写入 dev_out
20 acl.mdl.execute(model_id, input_dataset, output_dataset)
21
22 # 5. 获取结果 (Device -> Host)
23 # 申请 CPU 侧内存，将推理结果从 NPU 搬回，以便 Python 处理
24 host_out = acl.malloc_host(output_size)
25 acl.memcpy(host_out, dev_out, direction=DEVICE_TO_HOST)
26
27 # 6. 资源清理
28 # 释放本次推理申请的 Device 内存
29 acl.free(dev_in)
30 acl.free(dev_out)
31
32 return host_out

```

这段伪代码直观地展示了 PyACL 推理的核心特征：**内存不仅需要申请，还需要在 Host 与 Device 之间显式搬运 (Memcpy)**。这是与在通用 CPU 上开发程序最大的不同点。

3. 数据预处理

模型的输入通常需要特定的格式和分布。这里的 preprocess 函数模拟了标准 torchvision 的转换逻辑，但完全使用 Numpy 实现，以减少第三方库依赖。关键步骤包括：*** 归一化**：将像素值除以 255 转为浮点，并减去均值除以标准差。*** 维度变换**：将图片从 HWC（高宽通道）调整为 PyTorch 默认的 CHW（通道高宽）格式。*** 增加 Batch 维度**：模型输入通常是 4 维张量 (N, C, H, W)，因此需要增加 Batch 维度。*** 内存连续性**：这一步通过 np.ascontiguousarray 确保数据在内存中是连续存储的，这是 C 语言接口正确读取数据的必要条件。

4. 资源释放

脚本最后，我们需要显式释放所有持久化资源，防止 NPU 内存泄漏或状态异常。

```

1 def release(self):
2     acl.mdl.destroy_desc(self.model_desc)
3     acl.mdl.unload(self.model_id)      # 卸载模型
4     acl.rt.destroy_context(self.context) # 销毁 Context
5     acl.rt.reset_device(self.device_id) # 重置 Device
6     acl.finalize()                     # 去初始化

```

通过将这些模块组合，我们便得到了一个名为 inference_npu.py 的完整推理脚本。


```

1 import time
2 import numpy as np
3 import acl
4 from datasets import load_dataset
5 import tqdm
6
7 import ctypes
8
9 data_dir = './data' # 假设 data_dir 已定义, 或者根据实际情况保留原变量
10
11 # 加载数据集 (指定 cache_dir)
12 dataset = load_dataset('zh-plus/tiny-imagenet', split='valid', cache_dir=
    ↪ data_dir) # 只加载 valid 集
13
14 # ACL 初始化与资源管理类
15 class AclResource:
16     def __init__(self, device_id=0, model_path="model/resnet18_tiny_imagenet.om"
    ↪ ):
17         self.device_id = device_id
18         self.model_path = model_path
19         self.model_id = None
20         self.context = None
21         self.stream = None
22         self.input_dataset = None
23         self.output_dataset = None
24         self.model_desc = None
25
26     def init(self):
27         # ACL 初始化
28         ret = acl.init()
29         if ret != 0: raise RuntimeError("acl init failed")
30         ret = acl.rt.set_device(self.device_id)
31         if ret != 0: raise RuntimeError("set device failed")
32         self.context, ret = acl.rt.create_context(self.device_id)
33         if ret != 0: raise RuntimeError("create context failed")
34
35         # 加载模型
36         self.model_id, ret = acl.mdl.load_from_file(self.model_path)
37         if ret != 0: raise RuntimeError(f"load model failed, path: {self.
    ↪ model_path}")
38
39         # 获取模型描述
40         self.model_desc = acl.mdl.create_desc()
41         ret = acl.mdl.get_desc(self.model_desc, self.model_id)
42
43         print(f"ACL Resource Init Success. Device: {self.device_id}")
44
45     def execute(self, input_numpy):
46         # 准备输入数据 (Host -> Device)
47         # 获取模型输入大小
48         input_size = acl.mdl.get_input_size_by_index(self.model_desc, 0)
49
50         # 申请 Device 输入内存
51         dev_in_ptr, ret = acl.rt.malloc(input_size, 2) # 2:
    ↪ ACL_MEM_MALLOC_HUGE_FIRST
52
53         # 拷贝数据 (Host -> Device)
54         # 确保输入是 contiguous 且 float32 (C Type bytes)

```

```

55 # 使用 tobytes + bytes_to_ptr 避免 numpy_to_ptr 可能引发的 ImportError
56     ↳ 兼容性问题
57 if input_numpy.nbytes != input_size:
58     print(f"Warning: Input size mismatch. Model expects {input_size},
59         ↳ got {input_numpy.nbytes}")
60
61 input_bytes = input_numpy.tobytes()
62 input_ptr = acl.util.bytes_to_ptr(input_bytes)
63 # acl.rt.memcpy (dst, dest_size, src, src_size, kind)
64 acl.rt.memcpy(dev_in_ptr, input_size, input_ptr, input_size, 1) # 1:
65     ↳ ACL_MEMCPY_HOST_TO_DEVICE
66
67 # 组装 Input Dataset
68 self.input_dataset = acl.mdl.create_dataset()
69 input_data_buffer = acl.create_data_buffer(dev_in_ptr, input_size)
70 acl.mdl.add_dataset_buffer(self.input_dataset, input_data_buffer)
71
72 # 准备输出数据
73 self.output_dataset = acl.mdl.create_dataset()
74 output_size = acl.mdl.get_output_size_by_index(self.model_desc, 0)
75 dev_out_ptr, ret = acl.rt.malloc(output_size, 2)
76 output_data_buffer = acl.create_data_buffer(dev_out_ptr, output_size)
77 acl.mdl.add_dataset_buffer(self.output_dataset, output_data_buffer)
78
79 # 执行推理
80 ret = acl.mdl.execute(self.model_id, self.input_dataset, self.
81     ↳ output_dataset)
82 if ret != 0: print("Model execute failed")
83
84 # 取回结果 (Device -> Host)
85 host_out_buffer, ret = acl.rt.malloc_host(output_size)
86 acl.rt.memcpy(host_out_buffer, output_size, dev_out_ptr, output_size, 2)
87     ↳ # 2: ACL_MEMCPY_DEVICE_TO_HOST
88
89 # 转换为 numpy (假设输出是 float32, batch=1, class=200)
90 out_array = np.frombuffer(ctypes.string_at(host_out_buffer, output_size)
91     ↳ , dtype=np.float32)
92
93 # 清理单次推理资源
94 acl.rt.free(dev_in_ptr)
95 acl.rt.free(dev_out_ptr)
96 acl.rt.free_host(host_out_buffer)
97 acl.destroy_data_buffer(input_data_buffer)
98 acl.destroy_data_buffer(output_data_buffer)
99 acl.mdl.destroy_dataset(self.input_dataset)
100 acl.mdl.destroy_dataset(self.output_dataset)
101
102 return out_array
103
104 def release(self):
105     acl.mdl.destroy_desc(self.model_desc)
106     acl.mdl.unload(self.model_id)
107     acl.rt.destroy_context(self.context)
108     acl.rt.reset_device(self.device_id)
109     acl.finalize()
110
111 # 实例化并初始化 ACL 资源
112 # 请确保当前路径下有对应的 .om 模型文件

```

```

107 om_model_path = "model/resnet18_tiny_imagenet.om"
108 acl_resource = AclResource(model_path=om_model_path)
109 acl_resource.init()
110
111 # 自定义数据预处理函数 (替代 torchvision)
112 def preprocess(image):
113     # 将 PIL Image 转换为 numpy 数组
114     img_data = np.array(image).astype('float32') / 255.0
115
116     # 获取图像尺寸, 如果不是 RGB 三通道 (例如灰度图), 需要转换
117     if len(img_data.shape) == 2:
118         img_data = np.stack([img_data]*3, axis=-1)
119
120     # 归一化参数
121     mean = np.array([0.485, 0.456, 0.406], dtype='float32')
122     std = np.array([0.229, 0.224, 0.225], dtype='float32')
123
124     # 归一化: (image - mean) / std
125     img_data = (img_data - mean) / std
126
127     # 调整维度: HWC -> CHW (3, 64, 64)
128     img_data = img_data.transpose(2, 0, 1)
129
130     # 增加 Batch 维度: (1, 3, 64, 64)
131     img_data = np.expand_dims(img_data, axis=0)
132
133     # 确保内存连续, 这对 C 侧指针拷贝很重要
134     if not img_data.flags['C_CONTIGUOUS']:
135         img_data = np.ascontiguousarray(img_data)
136
137     return img_data
138
139 # 推理计数和计时
140 total_images = 0
141 correct_count = 0
142 start_time = time.time()
143
144 print("开始推理...")
145
146 # 逐张图片推理
147 for item in tqdm.tqdm(dataset):
148     image = item['image'] # 获取 PIL 图像
149     label = item['label'] # 获取真实标签
150
151     # 预处理
152     input_tensor = preprocess(image)
153
154     # 推理 (使用 pyACL)
155     # output 是扁平的一维数组, 直接使用
156     outputs = acl_resource.execute(input_tensor)
157
158     # 获取预测结果: argmax 获取概率最大的类别索引
159     predicted_label = np.argmax(outputs)
160
161     if predicted_label == label:
162         correct_count += 1
163
164     total_images += 1

```



```

8 data_dir = './data' # 假设 data_dir 已定义, 或者根据实际情况保留原变量
9
10 # 加载数据集 (指定 cache_dir)
11 dataset = load_dataset('zh-plus/tiny-imagenet', split='valid', cache_dir=
    ↪ data_dir) # 只加载 valid 集
12
13 # ONNX Runtime 初始化
14 onnx_model_path = "model/resnet18_tiny_imagenet.onnx" # 请确保当前路径下有该模型
    ↪ 文件
15 session = onnxruntime.InferenceSession(onnx_model_path, providers=[
    ↪ CUDAExecutionProvider', 'CPUExecutionProvider'])
16 print(f"当前运行设备 (Providers): {session.get_providers()}") # 打印以确认 CUDA
    ↪ 是否生效
17
18 input_name = session.get_inputs()[0].name
19
20 # 自定义数据预处理函数 (替代 torchvision)
21 def preprocess(image):
22     # 将 PIL Image 转换为 numpy 数组
23     img_data = np.array(image).astype('float32') / 255.0
24
25     # 获取图像尺寸, 如果不是 RGB 三通道 (例如灰度图), 需要转换
26     if len(img_data.shape) == 2:
27         img_data = np.stack([img_data]*3, axis=-1)
28
29     # 归一化参数
30     mean = np.array([0.485, 0.456, 0.406], dtype='float32')
31     std = np.array([0.229, 0.224, 0.225], dtype='float32')
32
33     # 归一化: (image - mean) / std
34     img_data = (img_data - mean) / std
35
36     # 调整维度: HWC -> CHW (3, 64, 64)
37     img_data = img_data.transpose(2, 0, 1)
38
39     # 增加 Batch 维度: (1, 3, 64, 64)
40     img_data = np.expand_dims(img_data, axis=0)
41
42     return img_data
43
44 # 推理计数和计时
45 total_images = 0
46 correct_count = 0
47 start_time = time.time()
48
49 print("开始推理...")
50
51 # 逐张图片推理
52 for item in tqdm.tqdm(dataset):
53     image = item['image'] # 获取 PIL 图像
54     label = item['label'] # 获取真实标签
55
56     # 预处理
57     input_tensor = preprocess(image)
58
59     # 推理
60     outputs = session.run(None, {input_name: input_tensor})
61

```

```

62 # 获取预测结果: argmax 获取概率最大的类别索引
63 predicted_label = np.argmax(outputs[0])
64
65 if predicted_label == label:
66     correct_count += 1
67
68 total_images += 1
69 # if total_images >= 100: break # 可选: 用于快速测试
70
71 end_time = time.time()
72 duration = end_time - start_time
73 fps = total_images / duration
74 accuracy = correct_count / total_images * 100
75
76 print(f"推理完成。")
77 print(f"总图片数: {total_images}")
78 print(f"总耗时: {duration:.4f} 秒")
79 print(f"推理帧率: {fps:.2f} FPS")
80 print(f"正确率: {accuracy:.2f}%")

```

所有平台均安装了 OnnxRuntime 进行推理。其中 RTX 5090D 配置了 CUDA 加速，而其他平台（包括树莓派、Intel 笔记本和昇腾 310B 的 CPU 模式）均仅使用 CPU 进行计算。

以下是各平台的具体测试结果：

1. Ascend 310B (CPU 模式) + OnnxRuntime: (规格: 4 核 Cortex-A55 @ 1.0GHz)

```

1 当前运行设备 (Providers): ['CPUExecutionProvider']
2 开始推理...
3 100%|████████████████████████████████████████| 10000/10000 [33:11<00:00, 5.02it/s]
4 推理完成。
5 总图片数: 10000
6 总耗时: 1991.7797 秒
7 推理帧率: 5.02 FPS
8 正确率: 62.43%

```

2. Nvidia GeForce 5090D (CUDA) + OnnxRuntime: (规格: 32GB GDDR7, 21760 CUDA Cores)

```

1 当前运行设备 (Providers): ['CUDAExecutionProvider', 'CPUExecutionProvider']
2 开始推理...
3 100%|████████████████████████████████████████| 10000/10000 [00:09<00:00, 1042.40it/s]
4 推理完成。
5 总图片数: 10000
6 总耗时: 9.5939 秒
7 推理帧率: 1042.33 FPS
8 正确率: 62.43%

```

3. Raspberry Pi 5B (CPU) + OnnxRuntime: (规格: Broadcom BCM2712, 4 核 Cortex-A76 @ 2.4GHz)

```

1 当前运行设备 (Providers): ['CPUExecutionProvider']
2 开始推理...
3 100%|████████████████████████████████████████| 10000/10000 [07:26<00:00, 22.38it/s]
4 推理完成。
5 总图片数: 10000

```

6 总耗时: 446.7709 秒
7 推理帧率: 22.38 FPS
8 正确率: 62.43%

4. Intel Core Ultra 7 155H (CPU) + OnnxRuntime: (规格: 3800 Mhz, 16 核, 22 逻辑处理器)

```
1 当前运行设备 (Providers): ['CPUExecutionProvider']  
2 开始推理...  
3 100% |██████████████████████████████████████| 10000/10000 [02:41<00:00, 61.91it/s]  
4 推理完成。  
5 总图片数: 10000  
6 总耗时: 161.5301 秒  
7 推理帧率: 61.91 FPS  
8 正确率: 62.43%
```

多平台 ResNet-18 (Tiny-ImageNet) 推理性能对比数据:

硬件平台	推理框架	运行设备	帧率 (FPS)	相对 NPU 性能 (265 FPS)
Ascend 310B NPU	PyACL	NPU (AI Core)	265.02	100% (基准)
Nvidia RTX 5090D	OnnxRuntime	GPU (CUDA)	1042.33	~393%
Intel Core Ultra 7 155H	OnnxRuntime	CPU	61.91	~23%
Raspberry Pi 5B	OnnxRuntime	CPU	22.38	~8.4%
Ascend 310B CPU	OnnxRuntime	CPU (Arm Cortex-A55)	5.02	~1.9%

通过数据对比，我们可以清晰地看到昇腾 310B NPU 的强大优势：*** 对比自身 CPU**：NPU 的推理速度是其 CPU 模式（5.02 FPS）的 **52 倍**，充分证明了专用 AI 加速器的必要性。*** 对比树莓派 5B**：虽然同样是 Arm 架构的嵌入式设备，昇腾 310B NPU 的性能是树莓派 5B CPU（22.38 FPS）的 **11 倍以上**。*** 对比高性能笔记本 CPU**：即便是最新的 Intel Core Ultra 7 处理器（61.91 FPS），在没有独立显卡加速的情况下，其纯 CPU 推理性能也不及昇腾 310B NPU 的 **1/4**。*** 对比旗舰显卡**：虽然 RTX 5090D 展现出了恐怖的 1000+ FPS 性能，但考虑到昇腾 310B 这类边缘设备的低功耗和体积优势，265 FPS 的成绩对于实时的嵌入式应用而言已经绰绰有余。

这一测试结果有力地展示了昇腾 310B 在边缘计算场景下的核心竞争力——以极低的功耗提供数倍于通用 CPU 的 AI 算力。

4.3. 目标检测推理

在图像分类之外，目标检测（Object Detection）是 PyACL 推理的另一个重要应用场景。与分类任务仅输出类别标签不同，目标检测需要同时预测物体的**边界框（Bounding Box）**和**类别**，对模型结构和后处理逻辑都提出了更高的要求。

本章在 [samples/chapter4/](#) 下提供了两种主流目标检测模型的 PyACL 推理实现：

4.3.1. SSD300（ResNet-50 主干）

SSD（Single Shot MultiBox Detector）是一种单阶段目标检测器，以 ResNet-50 为主干网络，输入尺寸 300×300，在 COCO 2017 数据集上训练和评估。

文件	说明
train_cuda.py	GPU 训练入口
inference_npu.py	PyACL 加载.om 模型进行 NPU 推理
inference_cuda.py	GPU 推理（对比基准）
inference_cpu.py	CPU 推理

代码位于 [samples/chapter4/SSD/](#)。

4.3.2. SSDLite320（MobileNet 主干）

SSDLite 是 SSD 的轻量化变体，将标准卷积替换为深度可分离卷积（Depthwise Separable Convolution），从而显著降低检测头的参数量和计算量。为了观察不同轻量化骨干网络在昇腾 310B 上的实际表现，本节不再只使用单一 MobileNetV3，而是比较 MobileNetV1、MobileNetV2、MobileNetV3 和 MobileNetV4 的 15 个 SSDLite320 变体。所有模型输入尺寸均为 320×320，输出为 boxes:1x4x3234 和 scores:1x81x3234，Default Boxes 使用 min_ratio↪=0.1、max_ratio=0.9。

本目录只提供部署、转换和评估代码，不包含训练代码。ONNX 模型的训练、导出和 CUDA 侧评估代码位于 [zhouxzh/SSDLite320-MobileNet](#)，如果需要了解训练策略、主干网络配置和 ONNX 导出细节,应到该 GitHub 仓库查找。Ascend 310B 侧的样例代码位于 [samples/chapter4↪ /SSDLite/](#)。

文件	说明
scripts/inference_npu.py	PyACL NPU 推理
scripts/inference_cpu.py	ONNX Runtime CPUExecutionProvider 校验 ONNX 接口和后处理

文件	说明
scripts/download_models.py	下载公开 ONNX/OM 模型
scripts/convert_onnx_to_om.py	ONNX 转 OM（需在 Ascend 设备上运行）
ssdlite320/	Default Boxes、Decode、NMS、可视化与评估工具

昇腾 310B 评估口径

Ascend 310B 侧的 CPU/NPU 评估报告由本地脚本生成，不作为本仓库的公开文件发布。读者如果需要复现实测数据，可以在准备好 ONNX/OM 模型和验证集后运行：

```

1 python scripts/inference_cpu.py --all
2 source /usr/local/Ascend/ascend-toolkit/set_env.sh
3 python scripts/inference_npu.py --all --device 0

```

评估脚本默认使用 `img_size=320`、`dbbox_min_ratio=0.1`、`dbbox_max_ratio=0.9`。报告中的 FPS infer 表示评估循环内单样本预处理、模型执行、输出整理和 decode/NMS 的吞吐；FPS `total` 还包含数据读取、JSON 写入和 COCO 评估等完整流程开销，因此更接近 Python 评估脚本的端到端速度。实际部署时，DVPP 预处理、C++/设备侧后处理、多 Stream 异步流水线等工程优化会进一步影响最终帧率。

从本地评估趋势看，CUDA 侧 ONNX 与 310B NPU OM 的 mAP 基本一致，说明 ONNX 转 OM 后没有出现明显精度损失。310B CPU 侧 ONNX Runtime 也能用于校验精度和后处理逻辑，但速度明显不足，不适合实时目标检测部署。

精度分析

从精度看，MobileNetV4 Hybrid 系列最强。mobilenetv4_hybrid_large 的 mAP 为 0.258，AP50 为 0.435，是整体精度最高的模型；mobilenetv4_hybrid_medium 的 mAP 同样约为 0.258，AP75 和大目标 AP1 甚至略高，且参数量只有 large 的约 28%，因此是高精度场景下更均衡的选择。若要求模型结构更偏卷积、减少 Hybrid 结构中的 MatMul、Transpose、Softmax 等算子，mobilenetv4_conv_large 可达到 0.252 mAP，略低于 Hybrid large，但推理速度更快一些。

MobileNetV3 Large 150d 和 MobileNetV4 Conv Medium 属于中高精度区间。mobilenetv3_large_150d `mAP` 为 0.247，精度接近 mobilenetv4_conv_large，但参数量和 MACs 更低；mobilenetv4_conv_medium `mAP` 为 0.243，是所有 FPS infer 超过 30 FPS 的模型里精度最高的一个。对于边缘端目标检测，这个模型通常比单纯追求最小参数量更有实际价值。

MobileNetV1 和 MobileNetV2 的中等宽度版本处于中等精度区间。mobilenetv1_125、mobilenetv2_140、mobilenetv1_100 的 mAP 分别为 0.233、0.225、0.221，虽然不如 V4/V3 高精度模型，但速度更接近实时。mobilenetv3_small_100、mobilenetv2_050、mobilenetv3_small_050 虽然

速度最高, 但 mAP 只有 0.140、0.120、0.093; mobilenetv4_conv_small 的速度也较高, 但 mAP 仅 0.054, 不适合作为通用 COCO 检测模型。

还需要注意小目标 AP。即使是最强的 mobilenetv4_hybrid_large, APs 也只有 0.076, 说明 320×320 输入尺寸和轻量化检测头对小目标并不友好。如果应用场景以远距离行人、小物体、密集目标为主, 应优先选择 large/medium 级别模型, 并考虑提高输入分辨率或针对业务数据重新训练; 如果主要检测中大目标, AP 最高的 mobilenetv4_hybrid_medium 和 mobilenetv4_hybrid_large → 会更合适。

速度分析

在昇腾 310B 上, 速度并不只由参数量或 MACs 决定。最小的 mobilenetv3_small_050 只有 0.089G MACs, 但在 Python/PyACL 评估流程中, 它并不会比中等模型快出数量级。这是因为 SSDLite320 的输出规模固定为 3234 个候选框, decode、NMS、数据搬运和 Python 调度开销对所有骨干网络都存在。当模型本身很小时, 这些固定开销会成为主要瓶颈, 继续缩小 backbone 并不会线性提升端到端帧率。

大模型则更容易受到算子形态和 NPU 编译优化的影响。mobilenetv4_hybrid_large 精度最高, 但包含更多非纯卷积算子和较多节点, 推理吞吐低于中等规模模型; mobilenetv4_conv_large → 虽然 MACs 达到 4.556G, 但以卷积结构为主, 在 310B 上更容易获得稳定的算子映射。mobilenetv4_conv_medium 是一个典型平衡点: mAP 0.243, 速度明显优于许多更大模型, 同时精度又明显高于 small 级别模型。

从 CPU/NPU 对比看, 越大的模型越能体现 NPU 加速价值; 而在小模型上, 瓶颈更多转移到 Host 侧后处理、调度和内存路径。这进一步说明, 要提升真实应用帧率, 除了换更小 backbone, 还必须优化预处理、后处理、异步流水线和视频帧调度。

模型选择建议

如果目标是**高精度推理**, 优先选择 mobilenetv4_hybrid_large; 它的整体 mAP 和 AP50 最高, 适合离线分析、低帧率告警或对误检漏检比较敏感的场景。如果希望在精度接近的同时降低模型规模和时延, mobilenetv4_hybrid_medium 更合适, 它只比 large 低约 0.001 mAP, 但模型规模小得多。

如果目标是**高帧率推理**, 优先选择 mobilenetv4_conv_medium。它是本地 310B 评估中进入 30 FPS 级推理吞吐范围的最高精度模型, mAP 达到 0.243, 比 mobilenetv1_100 高 0.022, 比 mobilenetv3_large_100 高 0.046。若应用只追求最大检测频率且可以接受明显精度损失, 可以选择 mobilenetv3_small_050 或 mobilenetv3_small_100; 但从通用目标检测效果看, 它们更适合作为速度上限参考, 而不是推荐默认模型。

目标视频帧率	当前 320×320 SSDLite320 是否支持逐帧检测	推荐模型与策略	说明
	持逐帧检测		
30 FPS	按推理循环吞吐可接近或达到；按完整 Python 评估流程仍需优化	首选 mobilenetv4_conv_medium；若必须更轻，可选 mobilenetv1_100	mobilenetv4_conv_medium 是 30 FPS 档的最佳精度选择；实际工程需优化预处理、后处理和异步流水线
60 FPS	不支持逐帧检测	mobilenetv4_conv_medium；每 2 帧检测一次，配合跟踪；极限速度可试 mobilenetv3_small_050	当前 SSDLite320 路径无法真实逐帧 60 FPS；用 30 Hz 检测 + 60 Hz 跟踪/显示更现实
90 FPS	不支持逐帧检测	mobilenetv4_conv_medium；每 3 帧检测一次；若目标较简单可试 mobilenetv3_small_100	90 FPS 视频流可保持约 30 Hz 检测频率，其余帧由跟踪器或上一帧检测结果补齐
120 FPS	不支持逐帧检测	mobilenetv4_conv_medium；或 mobilenetv1_100 每 4 帧检测一次；若必须逐帧，需要重新设计模型和流水线	当前 SS-DLite320+Python/PyACL+NMS 结果无法支撑 120 FPS 逐帧检测，需要降低输入尺寸、使用更小检测器、迁移后处理到 C++/设备侧、引入 DVPP 和多 Stream 异步流水线

因此，在昇腾 310B 上做 SSDLite320 部署时，可以按业务目标分成两类：追求检测质量时选择 mobilenetv4_hybrid_large 或 mobilenetv4_hybrid_medium；追求实时帧率时选择 mobilenetv4_conv_medium，再通过隔帧检测、目标跟踪、DVPP 预处理和异步流水线把视频系统帧率提高到 60 FPS 以上。不要仅根据参数量选择最小模型，因为过小的 backbone 在精度上损失很大，而在 310B 上的端到端速度提升并不成比例。

4.3.3. 推理流程对比

目标检测的 PyACL 推理流程与 ResNet-18 分类基本一致：**模型加载 -> 数据预处理 -> NPU 推理 -> 后处理**。关键区别在于：

- **模型输出**：分类模型输出单一概率向量（[1, 200]），目标检测模型输出多个张量（边界框坐标 + 类别置信度）。
- **后处理**：分类只需 `argmax`，目标检测需要 **Decode 边界框 + NMS**（非极大值抑制）去除重叠检测框。
- **评估指标**：分类用准确率（Accuracy），目标检测用 mAP（mean Average Precision）。

完整代码及使用说明详见各子目录下的 README 文件。

4.4. 总结

本章系统讲解了 PyACL 应用开发的基础知识，从运行资源管理到模型推理的全链路流程。PyACL 的核心价值在于：以 Python 的简洁语法驱动昇腾 NPU 的高性能计算，将 C/C++ 的指针操作和手动内存管理封装为 `acl.rt.malloc`、`acl.rt.memcpy`、`acl.mdl.execute` 等简洁接口，极大降低了昇腾 AI 处理器的开发门槛。

在运行时资源方面，Device、Context、Stream 构成的三级资源模型是 PyACL 的骨架。理解它们的层级关系与生命周期——申请按 Device -> Context -> Stream 顺序，释放按逆序——是写出健壮 PyACL 程序的前提。在此基础上，异构内存管理（Host 与 Device 间的四种拷贝路径）与同步等待机制（Event、Stream Sync、Stream Wait Event、Device Sync 四种粒度）构成了性能优化的核心手段。特别是在昇腾 310B 这类 CPU 算力较弱（Cortex-A55）的边缘设备上，**全链路异步 + Event 驱动的多 Stream 协作**是榨干 NPU 性能的关键策略。

在应用实践层面，本章通过 ResNet-18（图像分类）和 SSD/SSDLite（目标检测）两个经典场景，完整演示了“GPU 训练 -> ONNX 导出 -> ATC 转换 -> PyACL NPU 推理”的标准开发流水线。跨平台对比数据表明，即便使用相同的 ONNX 模型，昇腾 310B NPU 的 ResNet-18 推理速度（265 FPS）远超其自身 CPU 模式（5 FPS）和树莓派 5B（22 FPS），充分验证了专用 AI 加速器在边缘场景下的不可替代性。对于 SSDLite320 目标检测，`mobilenetv4_hybrid_large` 更适合高精度，`mobilenetv4_conv_medium` 则是在 310B 上兼顾 mAP 与 30 FPS 级检测吞吐的更实用选择。

回顾全章，PyACL 的精髓可归纳为三条原则：1. **资源按序管理**：初始化 -> Device -> Context -> Stream -> 业务执行 -> 逆序释放，缺一不可。2. **内存显式搬运**：Host 与 Device 内存相互隔离，数据必须通过 `memcpy` 显式传输，这与纯 CPU 编程截然不同。3. **异步优先于同步**：在边缘端弱 CPU 的约束下，必须用 `stream_wait_event` 将依赖关系卸载到 Device 侧，让 CPU 专注于指令分发而非空等。

掌握了这些基础概念与 API 后，下一章将进一步深入 DVPP（数字视觉预处理）模块，探索如何利用昇腾 310B 的硬件编解码与图像处理能力，构建完整的视频流 AI 应用。

5. DVPP 视频处理基础

Ascend 310B 芯片内置了 **DVPP (Digital Vision Pre-Processing)** 硬件加速引擎，提供视频编解码 (VENC/VDEC)、图像处理 (VPC) 和 JPEG 编解码能力。本章从基础概念出发，逐步深入到各子模块的 API 使用、性能调优和实战集成。

5.1. DVPP 基础概念与编程模型

DVPP (Digital Vision Pre-Processing) 是 Ascend 芯片内部的一组**硬件加速模块**，专门处理图像和视频数据。它独立于 NPU 的 AI Core (推理引擎)，不占用 AI 算力。PyACL 通过 `acl.media` 接口对外暴露这些能力。

本节覆盖所有 DVPP 子模块 (VENC、VDEC、VPC、JPEGE、JPEGD) 共享的基础概念。初次接触 DVPP 时，建议先阅读本节，再进入具体子模块的内容。

5.1.1. DVPP 在芯片中的位置

Ascend 310B4 可从应用开发视角理解为以下几类片上资源：

- **AI Core × 1:** DaVinci V300 (矩阵运算，NPU 推理主力)
- **片上 CPU × 4:** 4 个 64 位 ARMv8-A 通用处理核。npu-smi 可按 AI CPU / control CPU / data CPU 查询和配置数量，默认配比为 1:3:0；这是启动配置，修改后需要复位生效，不是运行时动态划分
- **DVPP:** 独立视觉/媒体处理硬件单元 (独立于 AI Core，也不是 CPU/AICPU 上的软件模块)
 - VENC: 视频编码
 - VDEC: 视频解码
 - VPC: 图像处理 (resize/crop/csc)
 - JPEGE: JPEG 编码
 - JPEGD: JPEG 解码
- **设备内存与内部总线:** AI Core、DVPP 与片上 CPU 通过设备内存和内部总线交换数据

关键点：DVPP 与 AI Core 是芯片上两个独立的硬件域。DVPP 编码 1080p 视频与 AI Core 执行 YOLO 推理可以并行运行，二者不竞争同一计算单元；CPU 侧主要负责内存、描述符和任务下发，不执行像素级编解码计算。

5.1.2. DVPP 数据流

DVPP 接收来自 **DDR 内存** 的数据，处理后写回 DDR。CPU 侧的工作是准备输入描述符、下发任务、等待回调或 Stream 同步完成——不参与实际像素级处理。

数据流五步：(1) CPU 侧分配设备内存并拷贝数据到设备 -> (2) DVPP 硬件通过内部总线读取 -> (3) 处理后将结果写入输出缓冲区 -> (4) DVPP 通过回调或 Stream 完成事件通知 CPU 侧 -> (5) CPU 侧按需从设备内存拷回结果。

核心机制：DVPP 直接访问 DDR，不经过 CPU 执行像素级处理。这是它能比 CPU 软件处理快得多的根本原因。

5.1.3. 子模块概览

模块	全称	功能	输入格式	输出格式	本教程章节
VENC	Video Encoder	视频编码	NV12 原始帧	H.264 / H.265 码流	VENC
VDEC	Video Decoder	视频解码	H.264 / H.265 码流	NV12 原始帧	VDEC
VPC	Video Pre-Processing Core	图像处理	NV12 / RGB / BGR	NV12 / RGB / BGR	VPC
JPEGE	JPEG Encoder	JPEG 编码	YUV420SP / YUV422SP	JPEG 码流	JPEG
JPEGD	JPEG Decoder	JPEG 解码	JPEG 码流	YUV420SP / YUV444	JPEG

PNGD（PNG 解码器）虽然列在 DVPP 规格中，但在 310B (CANN 8.3.RC1) 上实测输出缓冲区全零。310B 上 PNG 解码请使用 CPU 方案（Pillow / OpenCV）。本教程不含 PNGD 章节。

5.1.4. AIPP vs DVPP（何时用哪个）

PyACL 的媒体处理分两类能力：

- **DVPP**: 适合执行“低级别”的高吞吐预处理——JPEG/视频解码、YUV 与 RGB 转换、缩放、裁剪等。优点是速度快、CPU 负载低，但受格式与对齐约束。
- **AIPP** (Artificial Intelligence Pre-Processing): 适合执行“模型输入级”的精确预处理——统一色域/像素变换、量化/去均值、通道顺序等。AIPP 分静态（在模型转换时固化到.om）与动态（运行时通过接口设置）两种模式。

常见组合: 先用 DVPP 完成解码与粗略 resize/crop, 再用 AIPP（静态或动态）做最后的色域和像素级处理, 保证与模型输入要求一致。

5.1.5. H.264 与 H.265 编解码基础

VENC 和 VDEC 都围绕 H.264/H.265 工作。理解这些编码标准的基本概念, 是使用 DVPP 编解码模块的前提。

什么是 H.264

H.264（也叫 AVC）是目前全球使用最广泛的视频编码标准。核心思想是去除视频中的冗余：空间冗余通过帧内预测消除，时间冗余通过帧间预测消除，统计冗余通过熵编码消除。

编码后的每一帧按类型分为：

帧类型	全称	大小	依赖	说明
I 帧 (IDR)	Instantaneous Decoder Refresh	最大 (~80KB@480p)	无	独立解码, 不依赖任何其他帧
P 帧	Predictive	中等 (~5-15KB@480p)	前一帧	只存与前一帧的差异
B 帧	Bi-predictive	最小 (~2-5KB@480p)	前后帧	双向预测, 压缩率最高但延迟最大

B 帧与实时通信: B 帧需要参考“未来”帧, 引入额外延迟。WebRTC 和实时视频通话通常禁用 B 帧 (tune=zerolatency), 只用 I 帧和 P 帧。

GOP (Group of Pictures): 一个 I 帧到下一个 I 帧之间的帧组。GOP=30 表示每 30 帧插入一个 I 帧 (30fps 下每秒一个关键帧)。IDR 帧必须从 IDR 帧开始才能正确解码。

NAL 单元与 Annex-B 格式: H.264 码流由 NAL 单元组成 (SPS、PPS、IDR Slice、Non-IDR Slice、SEI)。Annex-B 格式用起始码 0x00000001 分隔每个 NAL 单元。VDEC 要求输入必须是 Annex-B 格式。

什么是 H.265

H.265（也叫 HEVC）是 H.264 的下一代标准, 核心目标: **相同画质下码率减半**。

特性	H.264	H.265
编码块大小	16×16 宏块（固定）	8×8 到 64×64 CTU（自适应）
帧内预测方向	9 种	35 种
压缩率	基准	同画质下码率少 50%
编码复杂度	基准	约 2-5 倍
浏览器支持	100%	>95%（Firefox 不支持 WebRTC H.265）

为什么本章仍以 H.264 作为主线：H.264 在 WebRTC 和教学演示里最稳定、最容易复现；同时，本章的 VENC/VDEC minimal 样例、基准脚本和 WebRTC 综合案例都补充了 H.265/HEVC 路径。对比来看：(1) CANN VENC/VDEC 对 H.264 和 H.265 的 API 基本一致，主要差别是 entype；(2) ARM 平台上 libx264 的测试码流更容易生成；(3) H.265 可作为同一套 DVPP API 的对照路径，用于理解两种编码的工程差异。

5.1.6. ACL 初始化——四步必要步骤

任何使用 DVPP 的 Python 进程，都必须在开头执行这四个调用，顺序固定不可变：

```
1 import acl
2
3 ret = acl.init()                # (1) 初始化 ACL 运行时
4 ret = acl.rt.set_device(0)      # (2) 选择 NPU 设备 0
5 ctx, ret = acl.rt.create_context(0) # (3) 在设备上创建执行上下文
6 ret = acl.rt.set_context(ctx)    # (4) 将上下文绑定到当前线程
```

为什么需要 context：ACL 的 context（上下文）是**线程局部**的。每个需要调用 ACL API 的线程都必须绑定自己的 context。回调线程里必须再调一遍 set_context(ctx)——主线程的 context 不会自动传递到回调线程。

多线程规则：一个设备可以创建多个 context，但一个 context 同时只能绑定一个线程，一个线程同时只能绑定一个 context。同一个 context 可以在不同时间绑定到不同线程（但不能同时）。

5.1.7. 通道模型

VENC 和 VDEC 都采用**通道（Channel）**模型。通道是 DVPP 硬件资源的抽象——创建一个通道就是向驱动申请一个硬件编码器/解码器实例。

通道创建遵循“描述符 -> 通道”两步模式：先创建通道描述符并设置参数，再调用 create_channel 申请硬件资源。描述符只是一组参数配置，create 时才真正向驱动申请资源。

DVPP 内部有两种不同的通道模型：

	VENC / VDEC 专用通道	VPC / JPEG 通用通道
创建 API	venc_create_channel() / vdec_create_channel()	dvpp_create_channel() (无 需设置 mode)
异步机制	回调线程 (process_report 轮询)	Stream 同步 (synchronize_stream 阻塞)
线程模型	需要独立回调线程 + Queue	不需要额外线程
数据描述	VENC: pic_desc->stream_desc, VDEC: stream_desc->pic_desc	pic_desc -> pic_desc (同类 型)

- **VENC/VDEC 回调式**: 主线程发送帧后阻塞在 Queue.get(), 回调线程通过 process_report 轮询硬件完成事件, 触发回调后 Queue.put 唤醒主线程
- **VPC/JPEG Stream 式**: 主线程下发异步任务后调用 synchronize_stream 阻塞等待, 硬件完成时直接通知 Stream (无需回调线程)

通道复用 vs 创建/销毁: 每次 创建通道() -> 处理 N 帧 -> 销毁通道() 的固定开销约 **5-10ms**。对于单帧编码场景, 创建/销毁的开销远超编解码本身。最佳实践: 创建一次通道, 连续处理所有帧, 最后销毁。

通道数是有限的: Ascend 310B4 的 VENC/VDEC 硬件实例数量有限 (通常每种 1-2 个)。

5.1.8. 回调线程模型

DVPP 是**异步**的: 发送工作请求后立即返回, 结果通过**回调**在另一个线程中返回。

异步处理时序: (1) 主线程发送帧请求后立即返回 -> (2) 主线程阻塞在 Queue.get() 等待结果 -> (3) DVPP 硬件完成处理 -> (4) 回调线程收到事件 -> (5) 回调将结果 Queue.put() -> (6) 主线程被唤醒得到结果

VENC vs VDEC 回调的关键差异:

	VENC 回调	VDEC 回调
参数顺序	(输入_pic_desc, 输出 ↔ _stream_desc)	(输入_stream_desc, 输出 ↔ _pic_desc)
读取输出	获取码流大小/数据()	获取图片返回码() + 获取图片数据/大小()
返回码检查	无	必须检查 , 非 0 = 解码失败
销毁输入	销毁图片描述符(输入)	销毁码流描述符(输入)
销毁输出	不需要 (调用方管理)	销毁图片描述符(输出)

记忆方法：第一个参数总是“输入”，第二个总是“输出”。VENC 输入图片-> 输出码流；VDEC 输入码流-> 输出图片。

为什么用 Queue 而不是 Event：Queue 天然适合“生产者（回调线程）-> 消费者（主线程）”模式——支持缓冲（多帧排队）、阻塞等待（Queue.get(timeout=5.0)）、线程安全（无需额外锁）。

为什么是 300ms：process_report(300ms) 阻塞最多 300ms 等待 DVPP 硬件完成通知。太短（如 10ms）会高频 CPU 轮询，太长（如 5000ms）会导致销毁通道时等待过久。300ms 是平衡值。

5.1.9. DVPP 内存管理

DVPP 有两套内存系统，必须正确区分：

操作	分配位置	访问方式	用途	释放
dvpp_malloc ↔ (大小)	设备端（NPU 片内或 DDR）	不能被 CPU 直接读写	DVPP 硬件访问的输入/输出缓冲区	dvpp_free(↔ ptr)
malloc_host ↔ (大小)	主机端（系统 DDR）	CPU 可正常读写	回调中临时中转数据	free_host(↔ ptr)

数据搬运方向：memcpy(设备内存, 主机数据, 大小) — 主机-> 设备（发送数据给 DVPP）；memcpy(主机内存, 设备数据, 大小) — 设备-> 主机（取回结果）。

常见内存错误：

错误	现象	原因
忘记 dvpp_free	内存泄漏 -> 后续分配失败	每帧分配但未释放
忘记 free_host	主机内存泄漏	回调中分配主机内存后未释放
在主机上直接读设备指针	段错误 / 无效数据	设备内存不能被 CPU 访问
回调中未销毁输入描述符	内存泄漏	VENC pic_desc / VDEC stream_desc 必须由回调销毁

5.1.10. 描述符模型

DVPP 使用两种描述符来描述输入/输出数据：

- **pic_desc** (图片描述符): 描述一帧图像 (NV12 / RGB 等), 包含数据指针、大小、格式、宽高、stride。VDEC 专用字段: ret_code (0= 解码成功, 非 0= 失败)。
- **stream_desc** (码流描述符): 描述一段压缩码流 (H.264 / H.265 / JPEG), 包含数据指针和大小。

子模块	输入描述符	输出描述符	回调参数顺序
VENC	pic_desc	stream_desc	(input_pic_desc, ↪ output_stream_desc)
VDEC	stream_desc	pic_desc	(input_stream_desc, ↪ output_pic_desc)
JPEGE	pic_desc	stream_desc	同 VENC
JPEGD	stream_desc	pic_desc	同 VDEC

5.1.11. NV12——DVPP 的通用货币

NV12 (也叫 YUV420SP) 是 DVPP 所有图像相关模块的首选像素格式。

为什么 NV12: (1) 体积小——每像素 1.5 字节 (RGB 是 3 字节), 省 50% 内存和带宽; (2) 人眼匹配——利用人对亮度敏感、对色度不敏感的特性, 降低色度分辨率; (3) 硬件原生——VENC/VDEC 硬件内部直接处理 NV12; (4) 摄像头兼容——大多数摄像头输出 YUV 格式, 接近 NV12。

内存布局: NV12 缓冲区 = [Y 平面] (H×W, 每像素 1 字节亮度) + [UV 交错平面] (H/2 × W, 每 2 字节一组 U,V)。总字节数 = H × W × 3/2。

与其他 YUV 格式的区别:

格式	全称	内存布局	DVPP 支持
NV12	YUV420SP	Y 平面 + UV 交错平面	VENC / VDEC / VPC / JPEGE
NV21	YVU420SP	Y 平面 + VU 交错平面 (U/V 顺序相反)	VDEC / VPC
I420	YUV420P	Y 平面 + U 平面 + V 平面 (3 个独立平面)	VPC (部分)
YUYV	YUV422	YUYV 交错 (每 2 像素共享 UV)	— (需 VPC 转换)

USB 摄像头输入 -> NV12 的路径: 大多数 USB 摄像头输出 YUYV 或 MJPG, 不是 NV12。转换可选 CPU 路径 (cv2.cvtColor + bgr_to_nv12()) 或 VPC 路径 (VPC CSC: YUYV->NV12, 但 310B 不支持 CSC)。

5.1.12. DVPP V1 与 himpi V2 — 两套 API 体系

CANN 为 Ascend 310B 提供了两套不同的媒体处理 API:

	DVPP V1 (acl.media)	himpi V2 (acl.himpi)
全称	Digital Vision Pre-Processing	Hi Media Processing Interface
定位	AscendCL 通用媒体处理	专用媒体处理（对标 HiMPP）
通道模型	VENC/VDEC 专用 + 通用 dvpp 通道	统一 *_create_chn
310B Python 可用性	大部分可用	通道创建不可用

himpi 的 *_create_chn 函数需要传入 C 结构体（如 hi_vpc_chn_attr），Python 侧不支持创建这些结构体。

选择指南：在 310B 上处理媒体数据时：

- **VENC/VDEC 编解码** -> acl.media.venc_* / vdec_*（唯一选择）
- **VPC resize/crop** -> acl.media.dvpp_vpc_*_async
- **JPEG 编解码** -> acl.media.dvpp_jpeg_*_async
- **旋转/翻转/滤波/仿射** -> CPU (OpenCV)
- **310P/710 等新硬件** -> himpi V2

5.1.13. 典型 DVPP 使用模式

- **视频推理管道：**VDEC 解码 -> VPC YUV 格式调整与缩放 -> 若需 RGB 或额外预处理，再由 AIPP 完成 -> 传入模型
- **静态图片分类：**JPEGD 解码 -> VPC 缩放/裁剪 -> 如需色域/像素变换使用 AIPP -> 传入模型
- **全硬件转码：**VDEC (H.264->NV12) -> VPC (resize) -> VENC (NV12->H.264)，设备内零拷贝

5.1.14. 开发注意事项

- DVPP 输出对分辨率与地址有对齐要求（stride/padding），读取数据时需依据描述信息处理
- 使用异步接口（*_async）时必须配合 Stream 与同步机制
- Host->Device 的异步拷贝源内存应使用页锁定（pinned）内存（acl.rt.malloc_host）
- 优先把耗时的像素级操作下沉到 DVPP/AIPP，避免在 CPU（Python）端逐像素处理
- 310B 上 dvpp_vpc_convert_color_async（CSC 色彩空间转换）不可用

5.2. VENC — 硬件视频编码

VENC (Video Encoder) 将 NV12 原始帧编码为 H.264/H.265 码流。前置基础：[DVPP 基础概念](#)。

文中所有完整可运行的代码在 [samples/chapter5/](#) 目录下。

5.2.1. 理论背景

为什么需要硬件编码

H.264 视频编码是计算密集型任务。一块 640×480@30fps 的视频流，纯 CPU 软件编码（如 libx264）会占用 ARM Cortex-A55 的大量计算资源。对于 Orange Pi AI Pro 这样的嵌入式设备，CPU 资源有限，软件编码不仅影响视频质量（可能因算力不足而降低帧率），还挤占了其他任务的 CPU 时间。

昇腾 310B 芯片内部集成了 **VENC (Video Encoder)** 硬件模块，专用于 H.264/H.265 编码。硬件编码器具有：

- **固定功能电路**：编码路径完全硬化，功耗和延迟远低于通用 CPU
- **独立于 AI Core**：不占用 NPU 推理算力
- **实时性保证**：硬件 pipeline 确保编码在固定时间内完成

CANN / ACL 体系

CANN (Compute Architecture for Neural Networks) 是华为昇腾芯片的全栈软件栈，其层级结构如下：

- **ACL (Ascend Computing Language)**：CANN 的核心编程接口，提供设备管理、内存管理、媒体处理等 API
- **DVPP (Digital Vision Pre-Processing)**：数字视觉预处理模块，包含 VENC（编码）、VDEC（解码）、VPC（图像处理）、JPEG 编解码等
- **acl.media**：Python 侧对 DVPP 的封装

VENC 在 DVPP 中的位置

DVPP 各子模块分工详见 [子模块概览](#)。VENC 的职责：**NV12 原始帧 -> H.264/H.265 码流**。

VDEC 是它的镜像：H.264/H.265 码流 -> NV12。两者串联可形成全硬件转码管道。

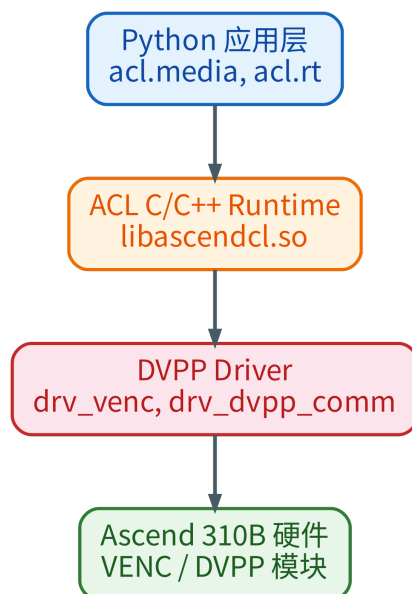


Figure 5.1.: CANN/ACL 与 DVPP 的调用层次



Figure 5.2.: VENC 编码端到端流程

NV12 格式

VENC 的输入格式必须是 **NV12** (YUV420SP)。详见 [NV12 格式](#)，这里只强调 VENC 的关键约束：

- 总大小： $H \times W \times 3/2$ 字节（对比 RGB 的 $H \times W \times 3$ ，节省 50%）
- **stride 对齐**：VENC 要求宽度对齐到 16，未对齐会导致编码画面偏移或绿条。对齐公式：
 $((width + 15) // 16) * 16$

编码流程（端到端）

5.2.2. 环境与架构

硬件

- 芯片：Ascend 310B4 (Orange Pi AI Pro)

- **VENC 模块**: 支持 H.264 Baseline/Main/High, H.265 Main
- **驱动**: drv_venc, drv_h264e, drv_h265e (通过 `lsmod | grep venc` 验证)

软件

- **CANN 版本**: 8.3.RC1
- **安装路径**: `/usr/local/Ascend/ascend-toolkit/8.3.RC1/`
- **Python API**: `/usr/local/Ascend/ascend-toolkit/latest/python/site-packages/acl`
 ↪ `/`
- **动态库**: `/usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/lib64/`

环境变量

每次使用 CANN Python API 前必须设置:

```
1 export LD_LIBRARY_PATH="/usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/
   ↪ lib64:/usr/local/Ascend/driver/lib64:$LD_LIBRARY_PATH"
2 export PYTHONPATH="/usr/local/Ascend/ascend-toolkit/latest/python/site-packages:
   ↪ $PYTHONPATH"
```

这两个变量分别解决了:

- `libascendcl.so: cannot open shared object file` — 动态库找不到
- `No module named 'acl'` — Python 包找不到

5.2.3. VENC API 详解

ACL 初始化

4 步固定初始化详见 [ACL 初始化](#), 此处只给出 VENC 上下文的代码:

```
1 import acl
2
3 ret = acl.init()                # (1)
4 ret = acl.rt.set_device(0)      # (2)
5 ctx, ret = acl.rt.create_context(0) # (3)
6 ret = acl.rt.set_context(ctx)   # (4)
7 assert ret == 0
```

所有后续的 VENC API 调用都依赖这个上下文。



Figure 5.3.: VENC 通道输入输出模型

VENC 通道模型

通道模型详见 [通道模型](#)，这里只列出 VENC 特有的 API：

函数	用途
venc_create_channel_desc()	创建通道描述符
venc_set_channel_desc_*	设置通道参数（见下节）
venc_create_channel(desc)	创建通道（返回 0 即成功）
venc_send_frame(...)	发送一帧到编码器
venc_create_frame_config()	创建帧配置（控制 force I-frame 等）
venc_destroy_channel(desc)	销毁通道
venc_destroy_channel_desc(desc)	销毁描述符

通道参数详解

创建 VENC 通道前，必须在描述符上设置编码类型、像素格式、分辨率、GOP、码率等参数。完整参数表及常用值见 [VENC 参数速查表](#)。

回调机制

回调线程模型详见 [回调线程模型](#)。VENC 的回调特点：

- 参数顺序：(input_pic_desc, output_stream_desc) — 第一个是输入图片，第二个是输出码流
- 输入 pic_desc 必须由回调销毁 (dvpp_destroy_pic_desc)
- 输出 stream_desc 的数据需在回调中通过 malloc_host + memcpy 拷到主机内存
- 通过 queue.Queue 将编码结果传回主线程，实现异步-> 同步转换

5.2.4. 开发过程与常见问题与调试

问题 #1：Python 环境找不到 acl 模块

现象：


```
1 ModuleNotFoundError: No module named 'acl'
```

根因：CANN 的 Python 包不在标准 `sys.path` 中。即使 `conda activate` 了正确的环境，CANN 的 `site-packages` 也不会自动加入搜索路径。

修复：

```
1 export PYTHONPATH="/usr/local/Ascend/ascend-toolkit/latest/python/site-packages:
  ↪ $PYTHONPATH"
```

说明：CANN 的 Python 路径是 `<toolkit>/python/site-packages`，不是 `<toolkit>/aarch64` `↪ -linux/python/site-packages`。aarch64-linux/ 下只有动态库（lib64/），没有 Python 包。

是否在代码中修复？示例脚本遵循“环境变量在进程外设置”的原则，不内置 `sys.`

`↪ path` 操作代码。这样可以明确依赖来源，避免隐式修改 Python 搜索路径。

问题 #2：libascendcl.so 找不到

现象：

```
1 ImportError: libascendcl.so: cannot open shared object file: No such file or
  ↪ directory
```

根因：即使 `import acl` 成功（因为 Python 包路径正确），底层 C 扩展仍需要动态库。

修复：

```
1 export LD_LIBRARY_PATH="/usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/
  ↪ lib64:/usr/local/Ascend/driver/lib64:$LD_LIBRARY_PATH"
```

问题 #3：venc_create_channel 返回 507018 — bitrate 单位错误

现象：

```
1 RuntimeError: venc_create_channel failed: 507018 (0x7bc8a)
2 dmesg: rc_drv_check_bit_rate bit rate set 2000000k error for out of [2k, 614400k
  ↪ ]
```

根因：`max_bit_rate` 的单位是 **kbps**（千比特/秒），不是 **bps**（比特/秒）。

我传入了 `2_000_000`（2 Mbps 以 bps 表示），被 VENC 驱动解释为 `2,000,000 kbps = 2 Gbps`，远超上限 `614,400 kbps`。

修复：

```

1 # 错误
2 media.venc_set_channel_desc_max_bit_rate(desc, 2_000_000) # bps -> 超出范围
3
4 # 正确
5 media.venc_set_channel_desc_max_bit_rate(desc, 2_000) # kbps = 2 Mbps

```

dmesg 调试技巧：VENC 错误信息会写入内核日志，`dmesg | grep -i venc` 是排查参数问题的第一手段。

问题 #4: venc_create_channel 返回 507018 — GOP 为 0

现象：

```

1 dmesg: rc_check_com_attr gop 0 err, should be in [1 65536]
2 dmesg: rc_create_chn check user rc attr err

```

根因：key_frame_interval（即 GOP — Group of Pictures）未设置，默认值为 0，不在合法范围 [1, 65536] 内。

GOP 控制多少个 P/B 帧之间插入一个 I 帧（关键帧）。GOP=30 意味着每 30 帧一个 I 帧，这在 30fps 下相当于每秒一个关键帧。GOP=0 对编码器没有意义（永远不产生 I 帧?），因此被拒绝。

修复：

```

1 media.venc_set_channel_desc_key_frame_interval(desc, 30) # GOP=30

```

问题 #5: venc_set_channel_desc_channel_id 不存在

现象：

```

1 AttributeError: module 'acl.media' has no attribute '
  ↳ venc_set_channel_desc_channel_id'

```

根因：VDEC 有 `vdec_set_channel_desc_channel_id`，但 VENC 的 API 中没有对应的 **setter**。VENC 的 `channel_id` 由驱动自动分配，不能手动设置。

这暴露了 CANN API 的一个不对称设计：VDEC 和 VENC 虽然结构相似，但细节不同，不能简单类比。

问题 #6: NumPy 维度索引错误

现象:

```
1 ValueError: could not broadcast input array from shape (640,640) into shape
  ↳ (640,)
```

根因: NV12 数据是 2D 数组 ($H \times 3/2, W$), 但代码用 1D 线性偏移去索引:

```
1 # 错误: nv12_data[src_off : src_off + w] 切出了形状 (w, w) 而非 (w,)
2 nv12_padded[off : off + w] = nv12_data[src_off : src_off + w] # 2D -> 1D 广播失
  ↳ 败
```

当 $\text{src_off} = \text{row} * w = 0$ 时, `nv12_data[0:640]` 取到的是整个 Y plane 的前 640 行 (即整个 640×640 区域), 而不是第 0 行的 640 个像素。

修复: 使用 2D 切片明确行列:

```
1 nv12_padded_2d = np.zeros(stride * h * 3 // 2, dtype=np.uint8).reshape(-1,
  ↳ stride)
2 nv12_src_2d = nv12_data.reshape(-1, w)
3
4 # Y plane
5 for row in range(h):
6     nv12_padded_2d[row, :w] = nv12_src_2d[row, :w]
7
8 # UV plane
9 for row in range(h // 2):
10     nv12_padded_2d[h + row, :w] = nv12_src_2d[h + row, :w]
11
12 nv12_padded = nv12_padded_2d.ravel()
```

问题 #7: stride 对齐

VENC 要求输入帧的宽度对齐到 16 (硬件约束)。NV12 数据填充时, Y plane 每行宽度应为 `aligned_width (stride)`, UV plane 同理。

```
1 self._align = 16
2 self._stride = ((width + self._align - 1) // self._align) * self._align
3 # 640 -> 640 (已对齐), 638 -> 640 (补齐)
```

不设置 stride 对齐会导致编码出的画面出现偏移或绿条。

问题 #8: NPU Alarm 状态混淆

现象:

```
1 npu-smi info: Health = Alarm
```

该状态容易被误认为 VENC 不可用。但实际测试表明 Alarm 不影响 VENC（参数正确就能创建成功）。Alarm 可能与其他传感器（温度、电源）有关，不一定反映 DVPP 模块状态。

调试建议：不要仅依据 NPU 全局状态判断模块可用性，应通过 dmesg 获取具体的模块级错误信息。

5.2.5. 练习脚本

三个可独立运行的脚本位于 `samples/chapter5/`，建议按顺序阅读理解。

概览

文件	学习内容	运行时间
check_cann.py	ACL 初始化的 4 个必要调用	<1s
venc_minimal.py	原始 VENC API：同一帧 NV12 分 别编码为 H.264/H.265	~3s
bench_venc.py	CannVenc 封装类 + H.264/H.265 分辨率扫描对比	~20s

实现方式：本节示例直接使用 `acl.media.venc_*` API，不依赖额外封装库。

```
1 export LD_LIBRARY_PATH="/usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/  
   ↳ lib64:/usr/local/Ascend/driver/lib64:$LD_LIBRARY_PATH"  
2 export PYTHONPATH="/usr/local/Ascend/ascend-toolkit/latest/python/site-packages:  
   ↳ $PYTHONPATH"  
3  
4 python samples/chapter5/check_cann.py           # -> ACL init OK  soc=Ascend310B4  
5 python samples/chapter5/venc/venc_minimal.py     # -> H.264/H.265 关键帧编码结  
   ↳ 果  
6 python samples/chapter5/venc/bench_venc.py       # -> H.264/H.265 VENC vs CPU  
   ↳ 帧率表
```

走读：ACL 初始化 — `check_cann.py`

```
1 import acl  
2  
3 ret = acl.init()           # (1) 初始化 ACL 运行时  
4 ret = acl.rt.set_device(0) # (2) 绑定设备 0  
5 ctx, ret = acl.rt.create_context(0) # (3) 创建执行上下文  
6 ret = acl.rt.set_context(ctx) # (4) 绑定上下文到当前线程
```

这四个调用是**固定的**，顺序不能变。任何使用 CANN 的 Python 进程都需要它们。

走读：最小编码 — `venc_minimal.py`

这是理解 VENC 的核心文件。脚本用同一帧确定性 NV12 输入，分别创建 H.264 Baseline 与 H.265 Main 通道并编码一帧。两个路径的 API 基本一致，主要差异是 entype：

```

1 ENTTYPE_H265_MAIN = 0
2 ENTTYPE_H264_BASE = 1
3
4 CODECS = [
5     ("H.264 Baseline", ENTTYPE_H264_BASE, 2_000),
6     ("H.265 Main", ENTTYPE_H265_MAIN, 2_000),
7 ]

```

代码分为 5 个阶段：

(1) ACL 初始化 — 与 step1 相同。

(2) 回调线程 — VENC 是异步的：

```

1 cb_queue: queue.Queue = queue.Queue(maxsize=8)
2
3 def venc_callback(input_pic_desc, output_stream_desc, _user_data):
4     size = media.dvpp_get_stream_desc_size(output_stream_desc) # 读取编码后大小
5     ptr = media.dvpp_get_stream_desc_data(output_stream_desc) # 读取数据指针
6     host_buf = acl.rt.malloc_host(size) # 分配主机内存
7     acl.rt.memcpy(host_buf, size, ptr, size, ACL_MEMCPY_DEVICE_TO_HOST)
8     cb_queue.put(ctypes.string_at(host_buf, size)) # 拷贝为 Python bytes
9     acl.rt.free_host(host_buf)
10    media.dvpp_destroy_pic_desc(input_pic_desc) # 回调负责销毁输入
11
12 def callback_thread(_args): # 独立线程处理回调
13     acl.rt.set_context(ctx) # 必须重新绑定上下文
14     while running[0]:
15         acl.rt.process_report(300) # 300ms 轮询

```

(3) 创建通道 — 按当前 codec 设置 entype、NV12 格式、分辨率、GOP、帧率和码率后调用 `venc_create_channel()`。H.264 Baseline 使用 entype=1，H.265 Main 使用 entype=0。

(4) 发送一帧 — NumPy 生成紧凑 NV12 -> 宽度 padding 到 16 对齐 -> `dvpp_malloc` -> `venc_send_frame` -> `cb_queue.get()` 等待 H.264/H.265 码流结果。

(5) 清理 — 销毁通道、描述符、帧配置，释放 DVPP 内存。

走读：封装与基准 — `bench_venc.py`

`bench_venc.py` 将原始 VENC API 封装为可复用的 `CannVenc` 类，然后做 H.264/H.265 双分辨率扫描对比硬件 vs CPU 编码性能。整个文件 ~380 行，分为 6 个部分。

(1) 测试参数

```

1 RESOLUTIONS = [
2     (640, 480),
3     (1280, 720),
4     (1920, 1080),

```

```

5     (2560, 1440),      # 2K
6     (3840, 2160),      # 4K
7 ]
8
9 TEST_FRAMES = 90        # 恰好 3 个 GOP (GOP=30 × 3)
10 TEST_GOP = 30          # 1 个 I + 29 个 P, 模拟真实视频流
11 RANDOM_SEED = 42       # 固定种子 -> 结果可复现
12 WARMUP_FRAMES = 3
13 FPS = 30

```

- **90 帧**：3 个完整 GOP，每个 GOP 含 1 个 I 帧 + 29 个 P 帧，I 帧占比 3.3%，与真实视频流一致
- **固定种子 42**：同一种子下任何机器生成的测试帧内容相同，保证跨运行可复现
- **3 帧预热**：排除首次编码的驱动初始化开销（比旧版 10 帧更精简）

(2) 确定性测试帧生成

```

1 def make_test_yuv420_frame(i: int, w: int, h: int) -> tuple[np.ndarray, np.
    ↳ ndarray, np.ndarray]:

```

直接生成 YUV420 平面,再按测试路径转换:VENC 路径拼成 NV12,CPU 路径拼成 yuv420p/I420。这样无需 BGR/RGB 转换，测量的是**纯编码性能**。每帧包含：

- **水平渐变 + 垂直渐变叠加**
- **正弦移动白条**：模拟时间相关性，防止编码器走 P 帧”全零残差”捷径
- **角落棋盘格**：8×8 方块交替，测试空间纹理编码

(3) CnnVenc 类详解

将 venc_minimal.py 的原始 API 封装为可复用的同步接口。

__init__ — 通道创建：与 venc_minimal.py 相同逻辑，额外计算 stride 对齐 $((w+15) \rightarrow //16)*16$ 和输出缓冲区大小。

_venc_callback — 编码完成回调：从 output_stream_desc 读取编码数据 -> malloc_host -> memcpy 拷到主机内存 -> 放入 Queue。回调负责销毁输入的 pic_desc。

encode(nv12, force_keyframe) — 编码一帧：

```

1 # (1) NV12 宽度补齐到 stride (16 对齐) —— VENC 硬件约束
2 padded = np.zeros(stride * h * 3 // 2, dtype=np.uint8).reshape(-1, stride)
3 src = nv12.reshape(-1, w)
4 for r in range(h):          # Y 平面逐行拷贝
5     padded[r, :w] = src[r, :w]
6 for r in range(h // 2):     # UV 平面逐行拷贝
7     padded[h + r, :w] = src[h + r, :w]
8
9 # (2) 分配 DVPP 输入内存 + 拷贝 NV12 到设备
10 in_buf, _ = media.dvpp_malloc(padded.nbytes)
11 acl.rt.memcpy(in_buf, padded.nbytes, padded.ctypes.data, padded.nbytes,
12               ACL_MEMCPY_HOST_TO_DEVICE)
13
14 # (3) 构造输入 pic_desc 和输出 stream_desc

```

```

15 pic = media.dvpp_create_pic_desc()
16 media.dvpp_set_pic_desc_data(pic, in_buf)
17 media.dvpp_set_pic_desc_format(pic, PIX_FMT_NV12)
18 media.dvpp_set_pic_desc_width_stride(pic, stride) # ← 必须设置 stride
19 # ... 输出 out_buf + stream_desc ...
20
21 # (4) 可选强制 I 帧
22 if force_keyframe:
23     media.venc_set_frame_config_force_i_frame(self._frame_cfg, True)
24
25 # (5) 排空回调队列（防止上一帧残留干扰）
26 while not self._cb_queue.empty():
27     self._cb_queue.get_nowait()
28
29 # (6) 发送编码请求 + 等待回调
30 media.venc_send_frame(self._ch_desc, pic, sd, self._frame_cfg, None)
31 encoded = self._cb_queue.get(timeout=5.0)
32
33 # (7) 清理：释放 DVPP 内存、销毁 stream_desc、恢复 force I-frame 标志

```

destroy() — 先 `venc_destroy_channel` 再停回调线程，与 VDEC 的销毁顺序要求类似。

(4) CPU 编码对比 — `bench_cpu_encode()`

```

1 def bench_cpu_encode(w: int, h: int, codec_name: str, bitrate_bps: int,
2                       thread_count: int) -> tuple:
3     level = "31" if w * h <= 1280 * 720 else "40" # <=720p -> 3.1, >=1080p ->
4         ↪ 4.0
5     codec = av.CodecContext.create(codec_name, "w")
6     codec.thread_count = thread_count # 0=自动多线程, 1=单线程
7     codec.bit_rate = bitrate_bps # bps (注意与 VENC 的 kbps
8         ↪ 区分)
9     codec.options = {"level": level, "tune": "zerolatency"}
10    if codec_name == "libx264":
11        codec.profile = "Baseline" # H.264 与 VENC entype=1
12        ↪ 对应

```

参数与 VENC 对齐：H.264 使用 Baseline profile、zerolatency tune、相同码率。最新脚本不再生成 BGR/RGB 测试帧，而是统一生成 YUV420 平面：VENC 路径拼成 NV12，CPU 路径拼成 yuv420p/I420 直接交给 PyAV，避免把颜色转换成本混入编码对比。

(5) 主流程 — 分辨率扫描

```

1 for w, h in RESOLUTIONS:
2     bitrate_kbps = max(2000, int(w * h * FPS * 0.1 / 1000))
3     bitrate_bps = bitrate_kbps * 1000
4
5     # VENC 测量
6     venc = CannVenc(w, h, bitrate=bitrate_kbps, entype=entype)
7     for i in range(WARMUP_FRAMES): # 预热
8         y, u, v = make_test_yuv420_frame(i, w, h)
9         venc.encode(yuv420_to_nv12(y, u, v), force_keyframe=(i == 0))
10    t0 = time.perf_counter()
11    for i in range(TEST_FRAMES): # 正式测量
12        y, u, v = make_test_yuv420_frame(i, w, h)
13        venc.encode(yuv420_to_nv12(y, u, v), force_keyframe=(i % TEST_GOP == 0))
14    venc_fps = TEST_FRAMES / (time.perf_counter() - t0)
15

```

```
16 # CPU 测量
17 _, cpu_mt_fps, _ = bench_cpu_encode(w, h, cpu_codec_name, bitrate_bps,
    ↳ thread_count=0)
18 _, cpu_st_fps, _ = bench_cpu_encode(w, h, cpu_codec_name, bitrate_bps,
    ↳ thread_count=1)
```

码率自适应公式 $\max(2000, w \times h \times \text{fps} \times 0.1 / 1000)$ kbps:

- 480p: $640 \times 480 \times 30 \times 0.1 / 1000 = 921 \rightarrow$ **2000 kbps** (下限 2 Mbps)
- 720p: $1280 \times 720 \times 30 \times 0.1 / 1000 = 2764 \rightarrow$ **2764 kbps**
- 1080p: $1920 \times 1080 \times 30 \times 0.1 / 1000 = 6220 \rightarrow$ **6220 kbps**
- 4K: $3840 \times 2160 \times 30 \times 0.1 / 1000 = 24883 \rightarrow$ **24883 kbps**

含义：每像素每秒分配 0.1 bit，按分辨率等比缩放。

(6) 本文件与 `venc_minimal.py` 的关系

	<code>venc_minimal.py</code>	<code>bench_venc.py</code>
目的	教学——展示每个 API 调用	基准——评估性能
帧数	1 帧	90 帧 × 5 分辨率
封装	裸 API 直接调用	CannVenc 类
对比	无	与 libx264/libx265 A/B 对比
内容	确定性 NV12 单帧	确定性 YUV420 序列（渐变 + 条 + 棋盘）
输出	H.264/H.265 关键帧大小	H.264/H.265 双表格

5.2.6. 集成到 aiortc

VENC 可集成到 aiortc（Python WebRTC 库）替代默认的 libx264 编码器。

基本思路

通过猴子补丁替换 aiortc 的 H.264 编码器：

```
1 import aiortc.codecs.h264 as h264_module
2 h264_module.H264Encoder = YourCannEncoder
```

继承策略

推荐继承 H264Encoder：只需覆盖 `_encode_frame()` 接入 VENC，其余 RTP 封装（RFC 6184 的 FU-A 分片、STAP-A 聚合等）全部继承。

H264Encoder (aiortc) 的继承结构——只需覆盖 `_encode_frame()` 接入 VENC，其余 RTP 封装全部继承：

- `_encode_frame()` -> libx264 编码 **[待覆盖]**
- `_packetize()` -> NAL -> RTP 分包 **[继承]**
- `_split_bitstream()` -> Annex-B -> NAL 分割 **[继承]**
- 其他全部继承

回退机制

CANN 不可用时自动回退到 libx264：

```
1 class CannH264Encoder(H264Encoder):
2     def _encode_frame(self, frame, force_keyframe):
3         if not _CANN_READY:
4             yield from super()._encode_frame(frame, force_keyframe)
5             return
6         try:
7             # CANN VENC 编码...
8         except RuntimeError:
9             yield from super()._encode_frame(frame, force_keyframe)
```

5.2.7. 性能对比与基准测试

实测数据：CANN VENC vs CPU 编码

以下数据在 Orange Pi AI Pro（Ascend 310B4）上实测获得，使用 `bench_venc.py` 脚本。该脚本同时覆盖 H.264 与 H.265，并分别测量 CPU 自动多线程（`thread_count=0`）和 CPU 单线程（`thread_count=1`）。

测试条件： GOP=30（I/P 混合），90 帧（3 个完整 GOP），确定性测试帧，固定种子 42。码率按分辨率自动缩放：H.264 使用 `max`(2M, `w*h*fps*0.1`) bps，H.265 使用 0.7 倍目标码率。测试帧统一生成为 YUV420 平面，VENC 路径转为 NV12，CPU 路径转为 yuv420p/I420，避免额外 BGR/RGB 颜色转换。

H.264 Baseline

分辨率	VENC 帧率	CPU多线程	CPU单线程
640x480	166.4	86.6	46.5
1280x720	92.8	39.1	21.2
1920x1080	48.4	13.4	6.0
2560x1440	21.3	11.0	6.8
3840x2160	15.8	7.1	3.7

H.265 Main

分辨率	VENC 帧率	CPU 多线程	CPU 单线程
640×480	165.1	31.1	22.9
1280×720	96.6	18.8	14.7
1920×1080	50.7	7.3	6.2
2560×1440	31.1	6.2	5.1
3840×2160	15.4	2.6	2.2

结果解读

H.264 Baseline

分辨率	VENC fps	CPU 单线程	CPU 多线程	vs 单线程	vs 多线程
640×480	166	47	87	3.58x	1.92x
1280×720	93	21	39	4.38x	2.37x
1920×1080	48	6	13	8.07x	3.61x
2560×1440	21	7	11	3.13x	1.94x
3840×2160	16	4	7	4.27x	2.23x

H.265 Main

分辨率	VENC fps	CPU 单线程	CPU 多线程	vs 单线程	vs 多线程
640×480	165	23	31	7.21x	5.31x
1280×720	97	15	19	6.57x	5.14x
1920×1080	51	6	7	8.18x	6.95x
2560×1440	31	5	6	6.10x	5.02x
3840×2160	15	2	3	7.00x	5.92x

实时性观察

- H.264: VENC 在 480p、720p、1080p 均超过 30fps; 2K/4K 单路未达到 30fps, 但仍明显快于 CPU。
- H.265: VENC 在 480p、720p、1080p、2K 均超过 30fps; 4K 约 15fps, 仍约为 CPU 多线程的 5.9 倍。
- CPU 多线程比单线程快, 但在所有测试点都低于 VENC。H.265 软件编码尤其重, CPU 多线程在 1080p 仅约 7fps。

与 VDEC 的对比

VENC 和 VDEC 在 Ascend 310B4 上的表现模式不同:

对比维度	VENC (编码)	VDEC (解码)
H.264 <=1080p	领先 CPU 多线程	落后 CPU
H.265 <=1080p	大幅领先 CPU 多线程	720p 以上领先 CPU 多线程
4K 表现	仍快于 CPU, 但单路低于 30fps	VDEC 约 72fps
拐点	无 CPU 交叉拐点, 所有点 VENC 更快	H.264 解码约从 2K 开始领先单线程 CPU
瓶颈	编码计算量大, 硬件收益明显	解码计算量较小, Python 回调开销更显著
适合场景	实时推流、录制、转码输出	高分辨率/HEVC 解码、多路或零拷贝管道

根因：视频编码计算量大，H.264/H.265 软件编码很快把 ARM CPU 打满；VENC 把运动估计、变换、量化、熵编码等重计算交给硬件完成，因此全分辨率都领先 CPU。VDEC 的解码计算量小得多，Python 回调、队列和内存搬运的固定开销更容易成为瓶颈。

什么场景下使用 VENC

场景决策树

需要实时视频编码 (>30fps)?

- 是 — 取决于分辨率：
 - <=480p: CPU H.264 多线程可超过 30fps, 但 VENC 更快且释放 CPU -> 建议 VENC
 - 720p/1080p: CPU 编码余量不足或不可用 -> 建议 VENC
 - 2K: H.264 VENC 约 21fps, H.265 VENC 约 31fps; 需要结合目标帧率选择
 - 4K: 单路约 15fps, 不满足 30fps; 仍明显快于 CPU, 适合低帧率录制或离线转码
- 否 (离线/批处理) -> CPU 可考虑, 但 VENC 通常仍快 2x~7x, 尤其 H.265 收益更大

典型场景推荐

场景	推荐方案	理由
WebRTC 视频通话	VENC	480p/720p 实时编码 + CPU 留给推理
USB 摄像头监控	VENC	1080p@30fps CPU 跑不动, VENC 可超过实时
多路视频流 (>2 路)	VENC 必须	CPU 单路 720p H.264 多线程约 39fps, 多路余量不足

场景	推荐方案	理由
4K 录制	VENC	4K 单路约 15fps，适合低帧率录制；30fps 需进一步优化或降低分辨率
本地视频文件转码	均可	离线场景 CPU 也可，但 VENC 更快
AI 推理 + 视频边车	VENC	CPU 编码会抢占 NPU 推理的 host 侧资源
低功耗设备	VENC	硬件编码功耗远低于 CPU 全速运行

多路并发估算

以 1080p@30fps 为目标帧率：

编码器	单路 fps	最多支持路数	CPU 剩余
CPU libx264 多线程	13	0 路（不到 30）	0%
VENC H.264	48	1 路（48/30）	较高
VENC H.265	51	1 路（51/30）	较高

最新单路基准显示，1080p 下 VENC 约 48-51fps，单路 30fps 有余量，但还不足以稳定支撑 2 路 1080p@30fps。多路场景应结合实际码率、GOP、输入来源和端到端链路重新测量。

什么情况下 CPU 编码就够了

只有离线批处理且满足以下全部条件时，CPU 才有意义：

- 分辨率 <= 480p（CPU H.264 多线程约 87fps，CPU H.265 多线程约 31fps）
- 不需要实时输出（无帧率硬性要求）
- 无并发推理任务（CPU 全给编码用）
- 不想引入 CANN 依赖（如 Docker 环境未安装 CANN）

底线：从 720p 开始，CPU 编码余量快速下降；1080p 及以上的实时视频场景，尤其是在 Orange Pi 这样的嵌入式 ARM 设备上，优先使用 VENC。

验证方法

在 Orange Pi 上分别运行 CPU 和硬件编码服务器，用 htop 观察实时 CPU 占用差异：

```
1 # CPU 编码
2 python server.py --source usb_camera
3
4 # 硬件编码
```

```
5 python server.py --source usb_camera --hardware-encode
```

或直接跑基准测试:

```
1 export LD_LIBRARY_PATH="/usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/
   ↳ lib64:/usr/local/Ascend/driver/lib64:$LD_LIBRARY_PATH"
2 export PYTHONPATH="/usr/local/Ascend/ascend-toolkit/latest/python/site-packages:
   ↳ $PYTHONPATH"
3 python samples/chapter5/venc/bench_venc.py
```

5.2.8. 附录

常用调试命令

```
1 # NPU 状态
2 npu-smi info
3 npu-smi info -t usages -i 0
4
5 # VENC 驱动状态
6 lsmod | grep venc
7
8 # VENC 内核日志 (参数错误信息 -- 排查 507018 必用)
9 dmesg | grep -i venc | tail -10
10
11 # 环境验证 (等价于 check_cann.py)
12 python samples/chapter5/check_cann.py
13
14 # 运行基准测试
15 python samples/chapter5/venc/bench_venc.py
```

参数速查表

VENC 通道参数

参数	函数	默认	推荐值
编码类型	venc_set_channel_desc_entype	—	1 (H264)
像素格式	venc_set_channel_desc_pic_format	—	1 (NV12)
宽度	venc_set_channel_desc_pic_width	—	640
高度	venc_set_channel_desc_pic_height	—	480
GOP	venc_set_channel_desc_key_frame_interval	0	30
帧率	venc_set_channel_desc_src_rate	—	30
最大码率 (kbps)	venc_set_channel_desc_max_bit_rate	—	2000
码率控制	venc_set_channel_desc_rc_mode	—	2 (CBR)

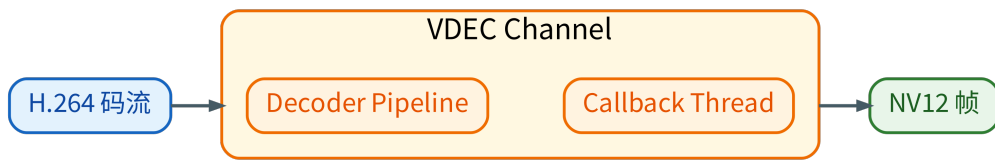


Figure 5.4.: VDEC 通道输入输出模型

参数	函数	默认	推荐值
回调线程	<code>venc_set_channel_desc_thread_id</code>	—	<code>tid</code>
回调函数	<code>venc_set_channel_desc_callback</code>	—	<code>cb</code>

错误码: 0 = ACL_SUCCESS, 507018 = 参数错误(检查 dmesg), 500001 = ACL_ERROR_FAILURE, 500004 = ACL_ERROR_DRV_FAILURE

编码类型: 0 = H.265 Main, 1 = H.264 Baseline, 2 = H.264 Main, 3 = H.264 High

像素格式: 1 = NV12 (YUV420SP), 12 = RGB888, 13 = BGR888

码率控制: 1 = VBR (Variable Bitrate), 2 = CBR (Constant Bitrate)

5.3. VDEC — 硬件视频解码

VDEC (Video Decoder) 将 H.264/H.265 码流解码为 NV12 原始帧。前置基础：[DVPP 基础概念](#)。

5.3.1. VDEC 简介

VDEC (Video Decoder) 是 DVPP 中的硬件视频解码模块。它将 H.264/H.265 压缩码流解码为 NV12 原始帧。

H.264 与 H.265 基础

H.264/H.265 的帧类型 (I/P/B)、GOP、NAL 单元、Annex-B 格式等编解码理论知识，详见 [H.264/H.265 基础](#)。本文的 VDEC API 示例和基准脚本都覆盖 **H.264 Baseline** 与 **H.265 Main**，便于直接对照两种 codec 的 entype 和码流差异。

VDEC 对输入码流只有一个硬性要求：必须是 **Annex-B 格式**（带 0x00000001 起始码的 NAL 单元序列）。H.264 首帧必须包含 SPS + PPS + IDR；H.265 首帧必须包含 VPS + SPS + PPS + IDR。

典型应用场景

场景	数据流
视频文件回放	MP4/MKV 文件 -> 解封装 -> H.264/H.265 码流 -> VDEC -> NV12 -> 显示
网络摄像机接收	RTSP/WebRTC -> H.264/H.265 码流 -> VDEC -> NV12 -> 分析/显示
转码管道	H.264/H.265 -> VDEC -> NV12 -> VENC -> 不同分辨率/码率的 H.264/H.265

与 VENC 的对称关系

详见 [子模块概览](#)。

- ¹ VENC: NV12 -> [硬件编码] -> H.264/H.265 码流
- ² VDEC: H.264/H.265 码流 -> [硬件解码] -> NV12

硬件能力规格 (Ascend 310B4)

以下基于 CANN 8.3.RC1 + Ascend 310B4 实测和驱动常量定义。

支持的编码类型

编码格式	Profile	entype 值	实测
H.265 / HEVC	Main	0	支持
H.264 / AVC	Baseline	1	支持
H.264 / AVC	Main	2	支持
H.264 / AVC	High	3	支持

实验中用 H.264 Baseline 码流测试了全部四种 entype，通道创建和 send_frame 均成功。但实际解码能否正确输出取决于输入码流是否匹配所选的 entype。例如：用 Baseline 码流 + entype=H265_MAIN 虽然不会报错，但解码结果会是花屏或黑帧（编码标准不匹配）。

支持的输出像素格式

格式	out_pic_format 值	位深	说明
YUV400	0	8	仅亮度

格式	out_pic_format 值	位深	说明
NV12 (YUV420SP)	1	8	推荐使用
NV21 (YVU420SP)	2	8	
NV12 4:2:2	3	8	
NV21 4:2:2	4	8	
NV12 4:4:4	5	8	
NV21 4:4:4	6	8	
RGB888	12	8	
BGR888	13	8	

位深 10-bit 理论上支持 (vdec_set_channel_desc_bit_depth), 但 310B4 驱动在 10-bit 下的实际表现未经充分验证。

分辨率和帧率约束

约束项	值	说明
最小分辨率	128×128	理论下限
最大分辨率	4096×4096	理论上限, 受内存限制
宽度对齐	16	VDEC 硬件要求
高度对齐	2	VDEC 硬件要求
最大帧率	60 fps	与分辨率有关
参考帧数	1-5 (默认 5)	ref_frame_num, 影响解码缓冲大小

实际可解码的最大分辨率受设备内存和码率限制。310B4 配备 16GB 内存, 单路 4K@30fps 解码无压力。

输入码流约束

约束项	说明
码流格式	H.264/H.265 Annex-B (带 0x00000001 起始码的 NAL 单元)
参数集	同一通道上所有帧必须共享一致的参数集; H.264 为 SPS/PPS, H.265 为 VPS/SPS/PPS

约束项	说明
首帧要求	H.264 必须包含 SPS + PPS + IDR; H.265 必须包含 VPS + SPS + PPS + IDR
单次输入上限	取决于码流参数, 超过约 256KB 可能被 VDEC 丢弃
帧边界	每次 vdec_send_frame 应发送完整的一帧 (含所有 NAL 单元)

H.264 Level 与典型应用

Level	最大宏块数	典型分辨率 @ 帧率	最大码率
3.0	1620	720×480@30	10 Mbps
3.1	3600	1280×720@30	14 Mbps
4.0	8192	1920×1080@30, 2048×1024@30	20 Mbps
4.1	8192	1920×1080@30, 2048×1024@30	50 Mbps
4.2	8704	1920×1080@60	50 Mbps
5.0	22080	2560×1920@30	135 Mbps
5.1	36864	4096×2304@30	240 Mbps

310B4 VDEC 理论上支持到 Level 5.1。本章基准测试中, 480p 使用 Level 3.1, 1080p 使用 Level 4.0。

5.3.2. VDEC 与 VENC 的关键区别

VDEC 与 VENC 的关键区别如下表所示 (同时参考 [子模块概览](#) 获取完整的子模块间对比):

特性	VENC	VDEC
数据方向	NV12 -> H.264/H.265	H.264/H.265 -> NV12
channel_id	驱动自动分配	必须显式设置
out_mode	无	必须检查和设置 (默认通常为 0)
ref_frame_num	无	参考帧数量 (默认 5)
send_frame 参数	(pic_desc, stream_desc)	(stream_desc, pic_desc)

特性	VENC	VDEC
回调中销毁对象	输入 (pic_desc)	输入 (stream_desc) 和输出 (pic_desc)
输入有效性要求	任意合法 NV12 均可	必须是合法的完整 H.264/H.265 NAL 单元
输出 ret_code	无	有，必须检查，非 0 表示解码失败
send_skipped_frame	无	有，用于丢帧后通知解码器

5.3.3. VDEC API 详解

通道参数

VDEC 通道需设置通道 ID、编码类型、输出像素格式、分辨率、参考帧数等参数。与 VENC 不同，VDEC 的 channel_id 必须显式设置。完整参数表见 [VDEC 参数速查表](#)。

回调函数

```
1 def vdec_callback(input_stream_desc, output_pic_desc, user_data):
2     """VDEC 解码完成回调——参数顺序与 VENC 相反。"""
3     ret_code = acl.media.dvpp_get_pic_desc_ret_code(output_pic_desc)
4     if ret_code != 0:
5         # 解码失败——丢弃此帧
6         pass
7     else:
8         pic_data = acl.media.dvpp_get_pic_desc_data(output_pic_desc)
9         pic_size = acl.media.dvpp_get_pic_desc_size(output_pic_desc)
10        # ... 拷贝 pic_data 到主机内存 ...
11
12    # 回调负责销毁输入和输出描述符
13    acl.media.dvpp_destroy_stream_desc(input_stream_desc)
14    acl.media.dvpp_destroy_pic_desc(output_pic_desc)
```

与 VENC 的关键区别：

- 回调参数顺序相反：VDEC 是 (stream_desc, pic_desc), VENC 是 (pic_desc, stream_desc →)
- VDEC 必须检查 pic_desc 的 ret_code
- VDEC 回调需要销毁 两个描述符 (VENC 只销毁输入)

ret_code 错误码

ret_code	含义
0	解码成功
非 0	解码失败——输入码流损坏、帧边界错误或参考帧不足

5.3.4. 练习脚本走读

完整代码见 [samples/chapter5/vdec/vdec_minimal.py](#)。程序使用原始 `ac1.media` API 分为 5 个阶段：

(1) 生成测试码流

VDEC 需要合法的 H.264/H.265 输入，不能使用随机字节。最小样例先用 NumPy 生成同一帧 I420/YUV420P 测试图，再分别交给 `libx264` 和 `libx265` 生成 H.264 Baseline 与 H.265 Main 码流：

```

1 import av, fractions, numpy as np
2
3 ENTYPE_H265_MAIN = 0
4 ENTYPE_H264_BASE = 1
5
6 CODECS = [
7     ("H.264 Baseline", "libx264", "31", ENTYPE_H264_BASE),
8     ("H.265 Main", "libx265", "31", ENTYPE_H265_MAIN),
9 ]
10
11 codec = av.CodecContext.create(ffmpeg_codec, "w")
12 codec.width, codec.height = 640, 480
13 codec.pix_fmt = "yuv420p"
14 # ... 设置 bit_rate, framerate, time_base, options ...
15 if ffmpeg_codec == "libx264":
16     codec.profile = "Baseline"
17
18 frame = av.VideoFrame.from_ndarray(make_test_i420(640, 480), format="yuv420p")
19 stream = b"".join(bytes(pkt) for pkt in codec.encode(frame))
20 stream += b"".join(bytes(pkt) for pkt in codec.encode(None))

```

(2) 回调线程

与 VENC 结构相同，但回调参数顺序相反，且必须检查 `ret_code`。

(3) 创建通道

```

1 desc = media.vdec_create_channel_desc()
2 media.vdec_set_channel_desc_channel_id(desc, 0) # ← VDEC 必须设置!
3 media.vdec_set_channel_desc_thread_id(desc, tid)
4 media.vdec_set_channel_desc_callback(desc, vdec_callback)
5 media.vdec_set_channel_desc_entype(desc, entype) # H.264=1, H.265=0
6 media.vdec_set_channel_desc_out_pic_format(desc, PIX_FMT_NV12)
7 media.vdec_set_channel_desc_out_pic_width(desc, W)
8 media.vdec_set_channel_desc_out_pic_height(desc, H)
9
10 ret = media.vdec_create_channel(desc)

```

(4) 发送一帧解码

```

1 # 输入: H.264/H.265 码流
2 arr = np.frombuffer(stream, dtype=np.uint8)
3 in_buf, _ = media.dvpp_malloc(len(arr))
4 acl.rt.memcpy(in_buf, len(arr), arr.ctypes.data,
5               len(arr), ACL_MEMCPY_HOST_TO_DEVICE)
6
7 stream_desc = media.dvpp_create_stream_desc()
8 media.dvpp_set_stream_desc_data(stream_desc, in_buf)
9 media.dvpp_set_stream_desc_size(stream_desc, len(arr))
10
11 # 输出: NV12 图片
12 out_size = W * H * 3 // 2
13 out_buf, _ = media.dvpp_malloc(out_size)
14 pic_desc = media.dvpp_create_pic_desc()
15 media.dvpp_set_pic_desc_data(pic_desc, out_buf)
16 media.dvpp_set_pic_desc_size(pic_desc, out_size)
17 media.dvpp_set_pic_desc_format(pic_desc, PIX_FMT_NV12)
18
19 ret = media.vdec_send_frame(desc, stream_desc, pic_desc, frame_cfg, None)

```

(5) 清理

与 VENC 相同——销毁通道、描述符、帧配置，释放 DVPP 内存。

附: encode_frames 参数详解

`bench_vdec.py` 中用于生成测试码流的函数，接收原始帧 + 编码器名 + GOP -> 码流 + 统计。

```

1 def encode_frames(frames: list[np.ndarray], codec_name: str, gop: int
2                   ) -> tuple[list[bytes], int, int]:
3     import av
4
5     h, w = frames[0].shape[:2]
6     level = "31" if w * h <= 1280 * 720 else "40"
7

```

```

8  codec = av.CodecContext.create(codec_name, "w") # (1)
9  codec.width = w # (2)
10 codec.height = h
11 codec.pix_fmt = "yuv420p" # (3)
12 codec.bit_rate = max(2_000_000, # (4)
13                      int(w * h * FPS * 0.1))
14 codec.framerate = fractions.Fraction(FPS, 1) # (5)
15 codec.time_base = fractions.Fraction(1, FPS) # (6)
16 codec.options = {"level": level, # (7)
17                  "tune": "zerolatency"}
18 if codec_name == "libx265":
19     codec.options["x265-params"] = "log-level=error"
20 if codec_name == "libx264":
21     codec.profile = "Baseline" # (8)
22
23 for i, bgr in enumerate(frames):
24     frame = av.VideoFrame.from_ndarray(
25         bgr[..., ::-1], format="rgb24") # (9)
26     frame.pict_type = ( # (10)
27         av.video.frame.PictureType.I
28         if i % gop == 0
29         else av.video.frame.PictureType.P)
30     data = bytearray()
31     for pkt in codec.encode(frame): # (11)
32         data += bytes(pkt)
33     if data:
34         streams.append(bytes(data))
35 tail = bytearray()
36 for pkt in codec.encode(None): # (12)
37     tail += bytes(pkt)
38 if tail:
39     if streams:
40         streams[-1] += bytes(tail)
41     else:
42         streams.append(bytes(tail))
43 return streams, i_avg, p_avg

```

(1) `av.CodecContext.create(codec_name, "w")` 创建编码器实例。`codec_name = "libx264"` 或 `"libx265"`, `"w"` 表示编码模式。`libx264` 是 H.264 标准的开源软件实现, 运行在 CPU 上。此处用它来生成测试素材——先软件编码再硬件解码, 正好对应需要对比的两条路径。

(2) `codec.width / codec.height` 编码帧的尺寸 (像素)。必须与实际输入的 numpy 数组尺寸一致。VDEC 解码时需传入相同尺寸的 `out_pic_width/height`。

(3) `codec.pix_fmt = "yuv420p"` 编码器内部使用的像素格式。`libx264` 只接受 YUV 格式, `"yuv420p"` 是 I420 (YUV 4:2:0 planar)。PyAV 会自动将输入的 `rgb24` 帧转为 `yuv420p` 后再送给编码器。

(4) `codec.bit_rate = max(2_000_000, int(w * h * FPS * 0.1))` 目标码率, 单位 **bps**。`2_000_000 = 2 Mbps`, 是最低可用码率, 确保小分辨率不会码率过低。`-w * h * FPS * 0.1 =` 每像素每秒 0.1 bps, 按分辨率和帧率缩放。例如 `640×480×30×0.1 ~ 0.9 Mbps` -> 取 2 Mbps; `3840×2160×30×0.1 ~ 25 Mbps`。`-max(...)` 设置 2 Mbps 码率下限。

码率影响帧大小: 码率越高字节越多, 但硬件解码速度主要和像素数相关, 受码率影响很小。

(5) `codec.framerate` 目标帧率 (fps)。设为 30 表示编码器按 30fps 场景分配比特预算。不改变实际编码速度，只影响码率控制算法。

(6) `codec.time_base` 时间基准，帧率的倒数。`Fraction(1, 30)` 表示每帧间隔 1/30 秒。影响编码后 Packet 的 pts/dts 时间戳。

(7) `codec.options` 传递给 libx264 编码器的底层选项：

- **"level"**: H.264 Level。≤720p 用 Level 3.1, ≥1080p 用 Level 4.0。编码的码流中会嵌入 Level 标志，VDEC 据此分配解码缓冲区。
- **"tune"**: **"zerolatency"**: 低延迟调优。**禁用 B 帧**，编码器每收到一帧立即输出，不做多帧缓冲。实时通信必须开启，代价是牺牲约 10-15% 压缩率。

H.265 路径也使用 zerolatency，并通过 `x265-params=log-level=error` 关闭 x265 的信息日志。

(8) `codec.profile = "Baseline"` H.264 档次。三个常用档次：

- **Baseline**: 最简，无 B 帧，适合实时通信和低功耗设备。
- **Main**: 比 Baseline 多 B 帧，压缩率高 10-15%，适合广播电视。
- **High**: 在 Main 基础上增加 8×8 变换和量化矩阵，适合高清蓝光。

Baseline 是 WebRTC 强制要求的最低档次，且 VDEC `entype=1` 正好对应 Baseline。注意：libx265 没有 `profile` 属性 (H.265 的 `profile` 通过 `options` 设置)，所以加了 `if` 判断。

(9) `av.VideoFrame.from_ndarray(bgr[...], ::-1, format="rgb24")` 将 numpy BGR 数组转为 PyAV VideoFrame 对象。`bgr[...], ::-1` 把通道反转：BGR -> RGB。

(10) `frame.pict_type = I if i % gop == 0 else P` 按 GOP 间隔设置帧类型。GOP=30 时，帧 0、30、60... 为 I 帧，其余为 P 帧。这是模拟真实视频流的关键——97% 的帧是 P 帧，3% 是 I 帧。与旧版全 I 帧 (GOP=1) 相比，I 帧体积从 ~80KB 降至 ~25KB (480p)，因为码率预算被 P 帧摊薄。

(11) `codec.encode(frame)` 将一帧送入编码器，返回编码后的 Packet 列表。每个 Packet 包含一个或多个 NAL 单元 (SPS/PPS/SEI/IDR Slice 等)。

基准脚本会过滤空 packet，并按实际可用 packet 数量做 warmup 和计时；这也是 H.265 现在能正常出表的原因。

(12) `codec.encode(None)` 排空编码器缓冲区。传入 None 强制输出所有剩余数据。排空后的尾 NAL 追加到最后一帧的码流末尾。

I 帧数量与解码性能

基准测试中的 I 帧数量

每 30 帧一个 I 帧。bench_vdec.py 使用 `TEST_GOP = 30`，90 帧测试中只有 3 个 I 帧 (帧 0、30、60)，其余 87 帧是 P 帧。这是模拟真实视频流的配置。

```

1 基准测试码流 (GOP=30) :
2  [IDR] [P] [P] ... [P] [IDR] [P] [P] ... [P] [IDR] [P] ...
3  ↑ ~25KB@480p      ↑ ~7KB                      I 帧数 = 3/90 ~ 3%
4
5 全 I 帧码流 (GOP=1, 仅供参考) :
6  [IDR] [IDR] [IDR] ... [IDR] ← 所有帧都是关键帧
7  ↑ ~80KB@480p 完全相同的大小

```

为什么用 GOP=30 而不是全 I 帧

1. 真实视频流就是这样的：WebRTC、RTSP、监控摄像头通常使用 GOP=15~60。GOP=30 表示每秒一个关键帧 (30fps 下)，是实时通信的典型配置。
2. 避免误判 VDEC 性能：全 I 帧测试中每帧都 ~80KB (480p)，VDEC 的硬件并行优势被放大。而真实流中 97% 的帧是小 P 帧 (~7KB)，CPU 处理 P 帧极快 (只需解运动矢量 + 残差)，VDEC 的固定调度开销反而成了瓶颈。
3. 全 I 帧曾导致错误结论：本教程早期版本用 GOP=1 测得 VDEC 在 720p 领先 CPU 10%、1080p 领先 43%。切换到 GOP=30 后，H.264 VDEC 在 ≤1080p 全面落后于 CPU。全 I 帧测试的不是真实场景，测试结果不可用于工程决策。

I 帧比例对性能的颠覆性影响

测试模式	480p VDEC	480p CPU	1080p VDEC	1080p CPU	拐点
GOP=1 (全 I 帧)	240 fps	410 fps	97 fps	68 fps	720p
GOP=30 (I/P 混合)	17 fps	1349 fps	17 fps	307 fps	2K

GOP 的影响是巨大的：

- VDEC 在全 I 帧模式下 480p 跑 240fps，混合流下掉到 17fps (14× 差距)
- 原因：全 I 帧模式下每帧数据量大 (~80KB)，连续发送大包让 VDEC 硬件始终处于忙碌状态，调度开销被隐藏
- 混合流下每帧只有 ~8KB (P 帧)，VDEC 处理太快反而暴露了 Python 回调调度的固定瓶颈

帧类型与性能特性

帧类型	每帧大小 (480p)	解码方式	VDEC 表现	CPU 表现
I 帧 (IDR)	~25 KB (GOP=30)	帧内解码 (完整)	硬件并行优势	需完整逆变换

帧类型	每帧大小 (480p)	解码方式	VDEC 表现	CPU 表现
P 帧	~7 KB	帧间解码 (运动矢量 + 残差)	固定开销主导	极快 (数据量小)

在 GOP=30 混合流中，P 帧占 97%。P 帧数据量仅为 I 帧的 ~30%，CPU 解码 P 帧的开销远低于 I 帧（只需解残差），而 VDEC 每帧仍需经过完整的 memcpy -> send -> callback -> memcpy -> Queue 路径，这部分时间与帧大小关系不大。

如何切换测试模式

修改 bench_vdec.py 顶部的常量即可：

```
1 # 当前 (GOP=30, I/P 混合 —— 真实视频流)
2 TEST_GOP = 30
3
4 # 改为 (GOP=1, 全 I 帧 —— 最坏情况/最大吞吐测试)
5 TEST_GOP = 1
```

encode_frames() 会根据 gop 参数自动设置每帧的 pict_type: i % gop == 0 时为 I 帧, 否则为 P 帧。

5.3.5. 常见问题与调试

问题 #1: channel_id 未设置导致通道创建失败

现象：

```
1 vdec_create_channel failed: 507018
```

根因:VDEC 与 VENC 不同,不会自动分配 channel_id。必须显式调用 vdec_set_channel_desc_channel_id (desc, N)。

修复：

```
1 media.vdec_set_channel_desc_channel_id(desc, 0)
```

问题 #2: 回调参数顺序与 VENC 相反

现象：在回调中调用 dvpp_get_pic_desc_ret_code 时进程异常终止或返回无效数据。

根因:VENC 回调是 (input_pic_desc, output_stream_desc),VDEC 回调是 (input_stream_desc , output_pic_desc)。如果按 VENC 习惯写 VDEC 回调，会把 stream_desc 当成 pic_desc 来读。

记忆方法：第一个参数总是”输入”，第二个参数总是”输出”。VENC 输入是图片、输出是码流；VDEC 输入是码流、输出是图片。

问题 #3：未检查 ret_code 导致使用损坏帧

现象：解码后的 NV12 数据是花屏或全黑。

根因：dvpp_get_pic_desc_ret_code() 返回非 0 表示解码失败——可能是输入码流不完整、帧边界不对、参考帧丢失。直接使用失败帧的数据会得到损坏画面。

修复：

```
1 ret_code = media.dvpp_get_pic_desc_ret_code(pic_desc)
2 if ret_code != 0:
3     cb_queue.put(None) # 跳过此帧
4     return
```

问题 #4：回调未销毁输出 pic_desc 导致内存泄漏

现象：解码多帧后，dvpp_malloc 返回内存不足。

根因：VDEC 回调负责销毁两个描述符（输入 stream_desc 和输出 pic_desc）。VENC 只需要销毁输入，因为输出的 stream_desc 由调用方管理。如果只销毁了 stream_desc，pic_desc 及其关联的 DVPP 内存永远不会释放。

修复：回调 finally 块中同时销毁两个：

```
1 finally:
2     media.dvpp_destroy_stream_desc(input_stream_desc)
3     media.dvpp_destroy_pic_desc(output_pic_desc)
```

问题 #5：输入必须用 numpy 包装才能 memcpy

现象：

```
1 TypeError: acl.rt.memcpy args parse failed
```

根因：acl.rt.memcpy 的源地址参数不接受 Python bytes 对象。必须将其包装为 numpy 数组或 ctypes 指针。

修复：

```

1 # 错误
2 h264 = b"..."
3 acl.rt.memcpy(in_buf, len(h264), h264, len(h264), 1) # TypeError
4
5 # 正确
6 h264_arr = np.frombuffer(h264, dtype=np.uint8)
7 acl.rt.memcpy(in_buf, len(h264), h264_arr.ctypes.data, len(h264), 1)

```

问题 #6: vdec_destroy_channel 顺序错误导致阻塞

现象: 解码多帧后, media.vdec_destroy_channel(desc) 永远阻塞, 程序卡死。

根因: VDEC 通道销毁时, 回调线程必须在 vdec_destroy_channel 调用期间保持运行——驱动需要在销毁过程中通过回调发送最后的清理通知。如果先停止了回调线程, vdec_destroy_channel
→ 会无限等待一个永远不会来的回调。

VDEC 的正确销毁顺序与直觉相反:

```

1 # 错误: 先停线程再销毁通道 -> 永远阻塞
2 running[0] = False                                # 通知线程停止
3 media.vdec_destroy_frame_config(fcfg)              # 销毁帧配置
4 media.vdec_destroy_channel(desc)                   # 销毁通道 -> 等待回调 -> 永远阻塞!
5
6 # 正确: 先销毁通道 (需要线程活着), 再停线程
7 media.vdec_destroy_channel(desc)                   # (1) 先销毁通道 (此时线程必须活着)
8 running[0] = False                                # (2) 通知线程停止
9 acl.util.stop_thread(tid)                          # (3) 停止回调线程
10 media.vdec_destroy_frame_config(fcfg)              # (4) 最后销毁帧配置
11 media.vdec_destroy_channel_desc(desc)              # (5) 销毁描述符

```

这一步的关键点是销毁通道时回调线程必须仍在运行, 否则驱动侧的清理事件无法被处理。

问题 #7: VDEC 通道复用对码流连续性非常敏感

现象: 通道创建后第一帧解码正常, 第二帧 vdec_send_frame 返回 0 但回调永不触发。

根因: VDEC 期望同一通道上解码的帧来自同一个编码器实例, 并且按连续视频流顺序送入。也就是所有帧需要共享兼容的 SPS/PPS (序列参数集/图像参数集), 不能把多个互不相关的 IDR 样本直接拼到同一通道里混跑。

验证方法: 用单个 libx264 CodecContext 连续编码所有测试帧, 确保 SPS/PPS 一致, 并按同一序列顺序逐帧送入。samples/chapter5/vdec/bench_vdec.py 已新增“单通道复用”路径, 并自动尝试不同 pipeline_depth 与 frame_config 策略; 在当前 310B 环境下, 只有带显式 EOS 的 depth=4 变体能够稳定排空尾部缓存帧。

当前结论：Ascend 310B4 上不能假设“任意 H.264 样本都能安全复用同一通道”。应先用连续码流验证复用模式，并显式处理解码器尾部 flush；如果复用路径不稳定，再退回每帧独立通道作为兜底方案。独立通道虽然可靠，但固定创建/销毁开销在 640×480 下通常远高于纯 CPU 解码。

5.3.6. 性能实测

在 Orange Pi AI Pro（Ascend 310B4）上，使用 `bench_vdec.py` 进行 H.264/H.265 分辨率扫描。测试参数：GOP=30（I/P 混合，模拟真实视频流），90 帧（3 个完整 GOP），固定种子可复现。CPU 解码对比单线程（`thread_count=1`）和多线程（`thread_count=0`，自动使用所有核心）两种模式。脚本生成 H.265 测试码流时使用 `libx265`，并过滤编码器可能返回的空 `packet`，避免 CPU HEVC 解码路径把空 `packet` 当作 EOF。当前脚本输出已简化为帧率表：不再打印加速比、单帧耗时、优胜结论和码流大小。

H.264 Baseline

分辨率	VDEC 帧率	CPU多线程	CPU单线程
640×480	16.9	1485.9	612.0
1280×720	16.8	684.6	272.2
1920×1080	16.5	305.5	119.8
2560×1440	133.6	189.1	75.9
3840×2160	71.2	88.5	35.1

H.265 Main

分辨率	VDEC 帧率	CPU多线程	CPU单线程
640×480	275.4	324.9	185.6
1280×720	250.2	182.2	97.7
1920×1080	194.6	82.5	42.3
2560×1440	135.3	46.6	23.4
3840×2160	71.7	21.2	9.9

VDEC 单帧耗时（由帧率反推）

简化后的 `bench_vdec.py` 只打印帧率，不再打印耗时分解。若按 1000 / fps 反推，H.264 低分辨率路径仍能看到明显的固定开销：

编码	640×480	1280×720	1920×1080	2560×1440	3840×2160
H.264 VDEC	59.2ms	59.5ms	60.6ms	7.5ms	14.0ms
H.265 VDEC	3.6ms	4.0ms	5.1ms	7.4ms	13.9ms

关键结论：H.264 $\leq 1080p$ 时，VDEC 路径稳定在约 60ms/帧，说明瓶颈主要不是像素计算量。当前 Python 基准路径每帧都经过 DVPP 内存分配、回调线程、Queue 等同步环节，这些固定开销在小分辨率下很难被硬件解码收益摊薄。

2K 以上分辨率时，H.264 VDEC 吞吐明显回升；H.265 路径则从 480p 起就没有出现同样的 60ms 固定开销。

拐点分析

H.264 Baseline

分辨率	VDEC fps	CPU 单线程	CPU 多线程	vs 单线程	vs 多线程
640×480	17	612	1486	0.03x	0.01x
1280×720	17	272	685	0.06x	0.02x
1920×1080	17	120	306	0.14x	0.05x
2560×1440	134	76	189	1.76x	0.71x
3840×2160	71	35	89	2.03x	0.80x

H.265 Main

分辨率	VDEC fps	CPU 单线程	CPU 多线程	vs 单线程	vs 多线程
640×480	275	186	325	1.48x	0.85x
1280×720	250	98	182	2.56x	1.37x
1920×1080	195	42	83	4.60x	2.36x
2560×1440	135	23	47	5.78x	2.90x
3840×2160	72	10	21	7.24x	3.38x

H.264 路径下，VDEC 在 $\leq 1080p$ 完全不具备竞争力（仅为 CPU 单线程的 3%~14%）。真正的单线程交叉点在 **2K (2560×1440)**：2K 时 VDEC 比单线程 CPU 快 76%，4K 时快 103%。但在这个单路测试中，CPU 多线程在 H.264 全部分辨率上仍快于 VDEC。

H.265 路径不同：VDEC 在 480p 已经领先单线程 CPU，在 720p 以上同时领先单线程和多线程 CPU。HEVC 软件解码在 ARM CPU 上明显更重，因此 VDEC 的收益比 H.264 更早出现。

关键发现

1. **H.264 低分辨率 VDEC 有固定调度开销**——480p/720p/1080p 下 VDEC 都是 ~59ms/帧，与分辨率无关。这说明瓶颈不在硬件解码，而在 Python 回调-> 队列-> 主线程的调度路径。

- 2. **H.264 高分辨率开始领先单线程 CPU**——2K 以上分辨率，固定调度开销被摊薄，VDEC 开始领先单线程 CPU。2K 时领先 1.76×，4K 时领先 2.03×；但单路场景下仍慢于 CPU 多线程。
- 3. **H.265 是 VDEC 的优势路径**——720p 以上 H.265 VDEC 同时领先单线程和多线程 CPU，4K 时对多线程 CPU 也有 3.38× 加速。
- 4. **I/P 混合流比全 I 帧流更不利于 H.264 VDEC**——详见上文 [I 帧数量与解码性能](#) 的对比分析。全 I 帧测试曾误导结论，GOP=30 才是真实场景。
- 5. **零拷贝管道可减少主机侧开销**——当前基准实现每次解码都经过设备内存分配、D2H memcpy → 和 Python Queue。如果 VDEC 输出直接喂给 VENC（设备内存零拷贝），可以避免把 NV12 帧拷回主机再送回设备。

适用场景速查

分辨率	单路实时	多路并发 (>=4 路)	转码管道
H.264 <= 1080p	CPU 推荐	CPU 可选	CPU 推荐
H.264 2K/4K	CPU 多线程优先， CPU 资源紧张时 可选 VDEC	VDEC 可选	VDEC 可选
H.265 480p	CPU 多线程优先， VDEC 可选	CPU 可选	VDEC 可选
H.265 >= 720p	VDEC 推荐	VDEC 推荐	VDEC 推荐

5.3.7. 附录

常用调试命令

```
1 # VDEC 驱动状态
2 lsmod | grep vdec
3
4 # VDEC 内核日志
5 dmesg | grep -i vdec | tail -10
6
7 # 运行练习脚本
8 python samples/chapter5/vdec/vdec_minimal.py
```

参数速查表

VDEC 通道参数

参数	函数	示例
通道 ID	vdec_set_channel_desc_channel_id	0
编码类型	vdec_set_channel_desc_entype	1
输出像素格式	vdec_set_channel_desc_out_pic_format	1
输出宽度	vdec_set_channel_desc_out_pic_width	640
输出高度	vdec_set_channel_desc_out_pic_height	480
输出模式	vdec_set_channel_desc_out_mode	0
参考帧数	vdec_set_channel_desc_ref_frame_num	5
位深	vdec_set_channel_desc_bit_depth	8

编码类型：0 = H.265 Main, 1 = H.264 Baseline, 2 = H.264 Main, 3 = H.264 High

5.4. VPC — 硬件图像处理

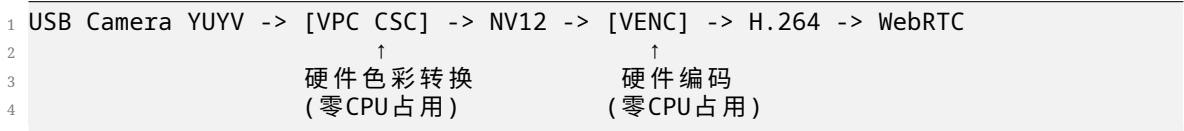
VPC（Video Pre-Processing Core）提供硬件加速的缩放、裁剪、色彩空间转换。
前置基础：[DVPP 基础概念](#)。

5.4.1. VPC 简介

VPC（Video Pre-Processing Core）是 DVPP 中的硬件图像处理模块。它提供三大功能：

功能	说明	典型场景
Resize	图像缩放（多种插值算法）	1080p -> 720p 下采样
Crop	按指定区域裁剪	从画面中提取 ROI 区域
CSC	色彩空间转换	YUYV->NV12、NV12->RGB 等

VPC 在 DVPP 管道中的典型位置：



为什么用 VPC 而不是 OpenCV

在 Orange Pi AI Pro 上，CPU 做 `cv2.cvtColor(YUYV->BGR) + cv2.resize(1080p->720p) + bgr_to_nv12()` 会消耗 ARM Cortex-A55 的宝贵算力。VPC 完全在硬件中完成这些操作，CPU 只负责下发任务和等待完成。

硬件能力规格（Ascend 310B4）

约束项	值
输入分辨率范围	10×6 ~ 8192×8192
输出分辨率范围	10×6 ~ 4096×8192
宽度对齐	2（VPC 自动向下对齐）
高度对齐	2
支持的输入格式	NV12、NV21、YUV400、YUV422、YUV444
支持的输出格式	NV12、NV21、YUV400、YUV422、YUV444

5.4.2. VPC 与 VENC/VDEC 的关键区别

VPC 使用的是 通用 DVPP 通道模型，与 VENC/VDEC 的专用通道有本质差异：

维度	VENC / VDEC	VPC
通道创建	<code>venc_create_channel()</code> / <code>vdec_create_channel()</code>	<code>dvpp_create_channel</code> ↪ <code>()</code> （无需设置 mode）
异步机制	回调线程（需 <code>process_report</code> 轮询）	Stream 同步 <code>(synchronize_stream</code> ↪ 等待)
输入类型	VENC: <code>pic_desc</code> / VDEC: <code>stream_desc</code>	<code>pic_desc</code> （始终是图像）
输出类型	VENC: <code>stream_desc</code> / VDEC: <code>pic_desc</code>	<code>pic_desc</code> （始终是图像）
回调函数	必须设置	不需要
回调线程	必须启动	不需要
复杂度	高（多线程协调）	低（同步等待即可）

310B 特别注意：dvpp_vpc_convert_color_async (CSC 色彩空间转换) 在 310B 上返回 **ACL_ERROR_INVALID_PARAM**, 不可用。himpv.vpc_convert_color 需要预创建 himpi 通道 (Python 接口受限)。310B 上的 YUYV->NV12 转换目前只能走 CPU 路径。

通道模型对比

1 VENC/VDEC 专用通道:	VPC 通用通道:
2	
3 主线程: send_frame -> Queue.get	主线程: vpc_xxx_async -> synchronize_stream
4 ↑	↑
5 回调线程: process_report	(无需回调线程)
6 -> callback	
7 -> Queue.put	

VPC 的 Stream 模型更简单——下发异步任务后直接阻塞等待完成, 不需要管理额外的线程和队列。

5.4.3. VPC API 详解

通用 DVPP 通道创建

VPC 使用通用的 dvpp_create_channel(), 与 VENC/VDEC 的专用 API 不同。无需设置 mode (310B 不支持 DVPP_CHANNEL_MODE 常量), 也无需创建回调线程:

```

1 import acl
2
3 # 创建通用 DVPP 通道
4 dvpp_channel_desc = acl.media.dvpp_create_channel_desc()
5 ret = acl.media.dvpp_create_channel(dvpp_channel_desc)
6
7 # 创建 Stream 用于同步等待
8 stream, ret = acl.rt.create_stream()
```

Resize — 缩放

将输入图片缩放到输出尺寸:

```

1 # 创建 resize 配置
2 resize_config = acl.media.dvpp_create_resize_config()
3 # 可选: 设置插值算法
4 # acl.media.dvpp_set_resize_config_interpolation(resize_config, 0) # 0=bilinear
5
6 ret = acl.media.dvpp_vpc_resize_async(
7     dvpp_channel_desc,      # DVPP 通道描述符
8     input_pic_desc,         # 输入图片描述符 (pic_desc)
```



```

9     output_pic_desc,      # 输出图片描述符 (pic_desc)
10    resize_config,       # 缩放配置
11    stream                # Stream 对象
12 )
13 acl.rt.synchronize_stream(stream) # 等待完成

```

关键约束:

- 输入/输出 pic_desc 的宽高必须对齐到 2 (VPC 自动处理)
- resize 不会改变像素格式——输入 NV12 则输出也是 NV12
- 如需同时改变格式和尺寸, 需串联 CSC + Resize

Crop — 裁剪

从输入图片中裁剪指定区域:

```

1 # 创建 ROI 配置: 从 (100, 100) 开始裁剪 224×224
2 # dvpp_create_roi_config 接受 4 个位置参数: (left, right, top, bottom)
3 crop_area = acl.media.dvpp_create_roi_config(
4     100,      # 左边界偏移
5     323,      # 右边界偏移 (左+宽-1)
6     100,      # 上边界偏移
7     323,      # 下边界偏移 (上+高-1)
8 )
9
10 ret = acl.media.dvpp_vpc_crop_async(
11     dvpp_channel_desc, input_pic_desc, output_pic_desc, crop_area, stream)
12 acl.rt.synchronize_stream(stream)

```

Crop + Resize — 最常用的组合

裁剪后缩放到目标尺寸, 一次调用完成两个操作:

```

1 # 从 1080p 画面中裁剪中心区域, 缩放到 720×480
2 # 注意: dvpp_create_roi_config 接受 4 个位置参数 (left, right, top, bottom)
3 crop_area = acl.media.dvpp_create_roi_config(160, 480, 120, 360)
4
5 resize_config = acl.media.dvpp_create_resize_config()
6
7 ret = acl.media.dvpp_vpc_crop_resize_async(
8     dvpp_channel_desc,
9     input_pic_desc,      # 1920×1080
10    output_pic_desc,      # 720×480
11    crop_area,
12    resize_config,
13    stream
14 )
15 acl.rt.synchronize_stream(stream)

```

这是 VPC 最高效的使用方式——crop + resize 在一个硬件调用中完成, 避免中间缓冲区。

CSC — 色彩空间转换 (310B 不可用)

结论: 310B (CANN 8.3.RC1) 上, `dvpp_vpc_convert_color_async` 返回 `ACL_ERROR_INVALID_PARAM` (100000), `himpli.vpc_convert_color` 需要 `himpli` 通道预配置 (Python 接口不支持)。CSC 在 310B 上目前不可用。

替代方案: YUYV->NV12 转换使用 CPU (`cv2.cvtColor + bgr_to_nv12`), `resize` 可卸载到 VPC:

```

1 USB Camera YUYV -> CPU bgr_to_nv12() -> NV12 -> VPC resize -> NV12(720p) -> VENC
2                                     ↑                               ↑
3                               CPU 做色彩转换                     VPC 硬件缩放

```

如果未来 CANN 版本在 310B 上开放 CSC 支持, 可通过 `dvpp_vpc_convert_color_async` 使用与 `resize` 完全相同的通道 +Stream 模式调用。

5.4.4. 练习脚本走读

完整代码见 `samples/chapter5/vpc/vpc_minimal.py`。程序使用原始 `acl.media` API 演示两个核心操作:

(1) Resize — 硬件缩放

```

1 # 创建 640×480 测试 NV12 帧
2 src_nv12 = make_test_nv12(640, 480)
3
4 # 计算 stride 对齐的内存大小
5 sw = ((640 + 15) // 16) * 16      # width stride 对齐到 16
6 sh = ((480 + 1) // 2) * 2        # height stride 对齐到 2
7 in_size = sw * sh * 3 // 2
8
9 # 目标 320×240
10 out_sw = ((320 + 15) // 16) * 16
11 out_sh = ((240 + 1) // 2) * 2
12 out_size = out_sw * out_sh * 3 // 2
13
14 # 分配设备内存 + 拷贝输入数据
15 in_buf, _ = media.dvpp_malloc(in_size)
16 out_buf, _ = media.dvpp_malloc(out_size)
17 acl.rt.memcpy(in_buf, src_nv12.nbytes, src_nv12.ctypes.data,
18               src_nv12.nbytes, ACL_MEMCPY_HOST_TO_DEVICE)
19
20 # 构造输入/输出 pic_desc (含 height_stride)
21 in_pic = create_nv12_pic_desc(in_buf, 640, 480, sw, sh)
22 out_pic = create_nv12_pic_desc(out_buf, 320, 240, out_sw, out_sh)
23
24 # 执行缩放 + 同步等待
25 resize_cfg = media.dvpp_create_resize_config()
26 media.dvpp_vpc_resize_async(ch_desc, in_pic, out_pic, resize_cfg, stream)
27 acl.rt.synchronize_stream(stream)
28
29 # 拷回主机内存

```

```

30 host_buf = np.zeros(out_size, dtype=np.uint8)
31 acl.rt.memcpy(host_buf.ctypes.data, out_size, out_buf, out_size,
32               ACL_MEMCPY_DEVICE_TO_HOST)

```

(2) Crop + Resize — 裁剪后缩放

```

1 # 从 640×480 中裁剪中心 320×240，再缩放到 640×480
2 crop_area = acl.media.dvpp_create_roi_config()
3 acl.media.dvpp_set_roi_config(crop_area,
4                               left=160, right=480, top=120, bottom=360) # 中心 320×240
5
6 resize_cfg = acl.media.dvpp_create_resize_config()
7 acl.media.dvpp_vpc_crop_resize_async(
8     ch_desc, in_pic_desc, out_pic_desc, crop_area, resize_cfg, stream)
9 acl.rt.synchronize_stream(stream)

```

(3) CSC — 310B 不可用

310B 上 CSC 需走 CPU。详见 [CSC 限制](#)。

清理

```

1 # 释放 VPC 资源
2 acl.media.dvpp_destroy_resize_config(resize_cfg)
3 acl.media.dvpp_destroy_roi_config(crop_area)
4 acl.media.dvpp_destroy_pic_desc(in_pic)
5 acl.media.dvpp_destroy_pic_desc(out_pic)
6 acl.media.dvpp_free(in_buf)
7 acl.media.dvpp_free(out_buf)
8 acl.media.dvpp_destroy_channel(ch_desc)
9 acl.media.dvpp_destroy_channel_desc(ch_desc)
10 acl.rt.destroy_stream(stream)

```

与项目的对应关系

文件	角色
vpc_minimal.py	学习用途——直接调用 acl.media API，理解底层机制
bench_vpc.py	基准测试——VPC resize vs CPU cv2.resize 性能对比

5.4.5. 性能基准

以下数据使用 `bench_vpc.py` 在 Orange Pi AI Pro (Ascend 310B4, CANN 8.3.RC1) 上实测。
测试条件: NV12 -> 1/2 缩放 (各分辨率缩小一半), 60 帧, 确定性测试帧, 固定种子 42。
VPC Resize: NV12 to 1/2 Scale (数据见下方结果解读表格)

结果解读

VPC Resize 与 VENC 不同——不是所有分辨率都远超 CPU。VPC 与 CPU 在低分辨率下互有胜负 (720p VPC 赢, 1080p CPU 赢), 这是 Stream 同步的固定开销与硬件加速之间的拉锯:

分辨率	像素数	VPC fps	CPU fps	加速比	推荐
640×480	0.3M	761	1521	0.50x	CPU
1280×720	0.9M	679	573	1.19x	VPC
1920×1080	2.1M	273	290	0.94x	CPU
2560×1440	3.7M	228	144	1.59x	VPC
3840×2160	8.3M	110	53	2.07x	VPC

从 2K (2560×1440) 开始 VPC 稳定领先 (1.6×~2.1×)。低分辨率下互有胜负——VPC 的固定调度开销 (dvpp_malloc + memcpy + Stream 同步) 在小帧上无法稳定被硬件加速摊薄, 720p 虽 VPC 略快但优势微弱 (1.19×), 1080p 反而 CPU 略快 (0.94×)。

VPC vs VENC vs VDEC 性能模式对比

模块	低分辨率 (<=1080p)	高分辨率 (>=2K)	瓶颈
VENC	领先 CPU 多线程 (H.264 约 1.9×~3.6×, H.265 约 5.3×~7.0×)	领先 CPU 多线程 (H.264 约 1.9×~2.2×, H.265 约 5.0×~5.9×)	纯硬件编码计算
VDEC	落后 CPU (~59ms 固定开销)	领先 CPU (1.9×~2.2×)	Python 回调调度
VPC	与 CPU 互有胜负 (0.5×~1.2×)	领先 CPU (1.6×~2.1×)	Stream 同步开销

VPC 的固定开销比 VDEC 小得多 (Stream 同步比回调线程轻量), 因此即使在 720p 也能与 CPU 抗衡, 而 VDEC 要到 2K 以上才有竞争力。

CSC 性能 (310B 不支持, 仅供参考)

310B 上 CSC (YUYV->NV12) 必须走 CPU: `cv2.cvtColor(YUYV->BGR) + bgr_to_nv12()`。VPC 只能卸载 `resize`, CPU 仅做色彩转换。

5.4.6. 常见问题与调试

问题 #1: 310B 不支持 dvpp_vpc_convert_color_async

现象: dvpp_vpc_convert_color_async 返回 100000 (ACL_ERROR_INVALID_PARAM)。

根因: 该接口仅支持 310P 及以上型号。310B (Atlas 200I A2) 的 CANN 8.3.RC1 版本不支持。

修复: 无 VPC 硬件替代方案。YUYV->NV12 使用 CPU: cv2.cvtColor(YUYV->BGR)+ bgr_to_nv12
↪ ()。VPC 仅卸载 resize 部分。

问题 #2: pic_desc 必须设置 height_stride

现象: dvpp_vpc_resize_async 返回 100000 (ACL_ERROR_INVALID_PARAM)。

根因: dvpp_set_pic_desc_height_stride 未设置, 且 dvpp_set_pic_desc_size 使用了未对齐的 $W \times H \times 3 // 2$ 。VPC 要求高度 stride 对齐到 2, 宽度 stride 对齐到 16, 缓冲区大小按 stride 计算。

修复:

```
1 sw = ((w + 15) // 16) * 16    # width stride
2 sh = ((h + 1) // 2) * 2      # height stride
3 size = sw * sh * 3 // 2      # 按 stride 算总大小
4 media.dvpp_set_pic_desc_width_stride(pic, sw)
5 media.dvpp_set_pic_desc_height_stride(pic, sh)
6 media.dvpp_set_pic_desc_size(pic, size)
```

问题 #3: dvpp_create_roi_config 不接受 keyword 参数

现象: dvpp_create_roi_config() + dvpp_set_roi_config() 不存在或报错。

根因: dvpp_set_roi_config 在 CANN 8.3.RC1 的 Python API 中不接受 keyword 参数。正确做法是在 dvpp_create_roi_config() 中直接传入 4 个位置参数。

修复:

```
1 # 正确
2 crop_area = acl.media.dvpp_create_roi_config(left, right, top, bottom)
3
4 # 错误
5 crop_area = acl.media.dvpp_create_roi_config()
6 acl.media.dvpp_set_roi_config(crop_area, left, right, top, bottom)
```

问题 #4: himpi 通道不可从 Python 创建

现象: himpi.vpc_create_chn() 无论传什么参数都报 “args parse failed”。

根因: himpi 接口是 C 扩展的直接映射, 需要 C 结构体类型的参数, Python 侧不支持创建这些结构体。vpc_convert_color 虽然语法上可以调用, 但缺少预配置的 himpi 通道, 返回硬件错误 0xa0078003。

当前结论:310B 上的 VPC CSC 不可用。等待 CANN 后续版本在 310B 上开放 dvpp_vpc_convert_color_async
→ 支持。

5.4.7. 场景推荐

场景决策树

需要对图像做预处理?

- 仅缩放 (NV12 in -> NV12 out) -> dvpp_vpc_resize_async (简单, Stream 同步)
- 仅裁剪 -> dvpp_vpc_crop_async
- 裁剪 + 缩放 -> dvpp_vpc_crop_resize_async (推荐, 一次调用)
- 仅色彩转换 (YUYV -> NV12) -> 310B 不支持, 使用 CPU cv2.cvtColor + bgr_to_nv12
- 色彩转换 + 缩放 (摄像头场景):
 - 1. CPU bgr_to_nv12 (YUYV->NV12)
 - 2. VPC dvpp_vpc_resize_async (1080p->720p)
- 小型图像、非实时 -> CPU (cv2) 足够

典型场景

场景	推荐方案	理由
USB 摄像头 -> WebRTC	CPU CSC + VPC resize	CSC 走 CPU, resize 卸载到 VPC
视频文件预处理	dvpp_vpc_crop_resize_async	单次调用完成裁剪 + 缩放
AI 推理前处理	VPC resize + AIPP CSC	Resize->AI Core 直接推理
多路视频并发 (>4 路)	VPC 必须	CPU 做多路 resize 会占满所有核

5.4.8. 附录

常用调试命令

```
1 # VPC 驱动状态
2 lsmod | grep vpc
3
4 # VPC 内核日志
5 dmesg | grep -i vpc | tail -10
6
7 # 运行示例脚本
8 python samples/chapter5/vpc/vpc_minimal.py
9
10 # 运行基准测试
11 python samples/chapter5/vpc/bench_vpc.py
```

参数速查表

VPC 通道创建

参数	API	值
通道描述符	dvpp_create_channel_desc()	—
创建通道	dvpp_create_channel(ch_desc)	ret=0
创建 Stream	acl.rt.create_stream()	ret=0

VPC 操作 API

操作	API	310B 支持
缩放	dvpp_vpc_resize_async	支持
裁剪	dvpp_vpc_crop_async	支持
裁剪 + 缩放	dvpp_vpc_crop_resize_async	支持
色域转换	dvpp_vpc_convert_color_async	不支持

像素格式 (dvpp_set_pic_desc_format) : 1 = NV12 (YUV420SP), 7 = YUYV (YUV422 interleaved), 12 = RGB888, 13 = BGR888

5.5. JPEG — 硬件编解码

JPEGE / JPEGD 使用与 VPC 相同的 Stream 同步模型。前置基础：[DVPP 基础概念](#)。

5.5.1. JPEG 编解码简介

DVPP 的 JPEG 编解码子模块：

- 1 JPEGE (JPEG Encoder): NV12 -> [硬件编码] -> JPEG 码流
- 2 JPEGD (JPEG Decoder): JPEG 码流 -> [硬件解码] -> NV12

两者串联形成硬件闭环，可验证编解码无损性（测试常用模式）。

与 VENC 编码的区别

VENC 输出 H.264 码流，但 JPEGE 输出的是**独立的 JPEG 图片**——两者有本质不同：

	JPEGE	VENC
输出格式	JPEG 单帧图片	H.264 / H.265 视频码流
帧间关系	无（每帧独立）	有（I/P/B 帧依赖）
输出描述符	裸内存缓冲区 + size 指针	stream_desc （码流描述符）
编码参数	quality（1-100）	GOP、码率、帧率、profile
用途	截图、快照、缩略图	实时视频传输

硬件能力规格（Ascend 310B4）

	JPEGE	JPEGD
输入格式	NV12 (YUV420SP)、YUV422SP	JPEG 码流（Baseline）
输出格式	JPEG 码流	NV12 (YUV420SP)
分辨率范围	32×32 ~ 8192×8192	32×32 ~ 8192×8192
质量范围	1-100	—
编码吞吐	1080p@256fps	1080p@512fps

5.5.2. 与 VENC/VDEC/VPC 的关键区别

JPEGE/JPEGD 使用与 VPC 相同的通用 **dvpp 通道 + Stream 同步模型**：

维度	VENC/VDEC	JPEGE/JPEGD
通道创建	venc/vdec_create_channel()	dvpp_create_channel()（无需 mode）

维度	VENC/VDEC	JPEGE/JPEGD
异步机制	回调线程	Stream 同步
输入描述	图片或码流描述符	裸内存指针 + size
输出描述	图片或码流描述符	JPEGE: 裸内存 + size 指针; JPEGD: pic_desc

JPEGE 的输出不是 stream_desc——这是一个常见误区。JPEG 码流直接写入 dvpp_malloc
 → 分配的缓冲区，通过 numpy_to_ptr 封装的 size 指针返回实际大小。

5.5.3. JPEGE API 详解

编码流程

```

1 (1) 创建 jpege_config + 设置质量 -> (2) predict_enc_size 预测输出大小
2   -> (3) dvpp_malloc 输出缓冲区 -> (4) jpeg_encode_async 异步编码
3   -> (5) synchronize_stream 等待 -> (6) 读取实际 size -> (7) memcpy 取回 JPEG
   → 码流

```

jpege_config — 编码参数

```

1 jpege_cfg = acl.media.dvpp_create_jpege_config()
2 acl.media.dvpp_set_jpege_config_level(jpege_cfg, quality) # quality: 1-100

```

唯一参数是 **quality** (1-100)，对应 JPEG 压缩质量。值越大画质越好、文件越大。

predict_enc_size — 预测输出大小

```

1 max_size, ret = acl.media.dvpp_jpeg_predict_enc_size(input_pic_desc, jpege_cfg)

```

返回编码后 JPEG 码流的**最大可能大小**（通常远大于实际值）。输出缓冲区需按此值分配。

jpeg_encode_async — 执行编码

```

1 import numpy as np
2
3 # in/out 参数: 传入 max_size, 编码后返回实际大小
4 out_size_arr = np.array([max_size], dtype=np.int32)
5 if "bytes_to_ptr" in dir(acl.util):
6     out_size_ptr = acl.util.bytes_to_ptr(out_size_arr.tobytes())
7 else:
8     out_size_ptr = acl.util.numpy_to_ptr(out_size_arr)

```

```

9
10 ret = acl.media.dvpp_jpeg_encode_async(
11     channel_desc,      # dvpp 通道描述符
12     input_pic_desc,    # NV12 输入图片描述符
13     output_buffer,     # 输出缓冲区 (dvpp_malloc)
14     out_size_ptr,      # in/out: [max_size] -> [actual_size]
15     jpege_cfg,         # 编码配置
16     stream             # Stream 对象
17 )
18 acl.rt.synchronize_stream(stream)
19
20 # 同步后读取实际编码大小
21 actual_size = int(out_size_arr[0])

```

关键点: out_size_ptr 是 Python 层用 numpy_to_ptr 封装的指针, 指向一个 int32 数组。编码器写入实际大小后, 同步完成即可读取。

完整编码示例

```

1 # 准备输入 NV12 pic_desc (略)
2
3 jpege_cfg = media.dvpp_create_jpege_config()
4 media.dvpp_set_jpege_config_level(jpege_cfg, 85)
5
6 max_size, _ = media.dvpp_jpeg_predict_enc_size(in_pic, jpege_cfg)
7 out_buf, _ = media.dvpp_malloc(max_size)
8
9 out_size_arr = np.array([max_size], dtype=np.int32)
10 out_size_ptr = acl.util.numpy_to_ptr(out_size_arr)
11
12 media.dvpp_jpeg_encode_async(ch_desc, in_pic, out_buf,
13                             out_size_ptr, jpege_cfg, stream)
14 acl.rt.synchronize_stream(stream)
15
16 jpeg_size = int(out_size_arr[0])
17 jpeg_host = np.zeros(jpeg_size, dtype=np.uint8)
18 acl.rt.memcpy(jpeg_host.ctypes.data, jpeg_size, out_buf, jpeg_size,
19              ACL_MEMCPY_DEVICE_TO_HOST)
20 # jpeg_host 即为 JPEG 码流, 可直接写入 .jpg 文件

```

5.5.4. JPEGD API 详解

解码流程

```

1 (1) JPEG 数据拷贝到设备 -> (2) get_image_info 获取宽高
2   -> (3) predict_dec_size 预测输出大小 -> (4) dvpp_malloc + 创建 pic_desc
3   -> (5) jpeg_decode_async 异步解码 -> (6) synchronize_stream -> (7) memcpy 取
    ↪ 回 NV12

```

get_image_info — 获取 JPEG 信息

```
1 img_w, img_h, img_fmt, ret = acl.media.dvpp_jpeg_get_image_info(
2   jpeg_dev_ptr, jpeg_size)
```

解码前必须调用此函数获取 JPEG 图像的宽度和高度，用于创建输出 pic_desc。

predict_dec_size — 预测输出大小

```
1 out_size, ret = acl.media.dvpp_jpeg_predict_dec_size(
2   jpeg_dev_ptr, jpeg_size, PIX_FMT_NV12)
```

返回解码后 NV12 缓冲区的所需大小。

jpeg_decode_async — 执行解码

```
1 ret = acl.media.dvpp_jpeg_decode_async(
2   channel_desc,      # dvpp 通道描述符
3   jpeg_dev_ptr,      # JPEG 数据指针（设备内存）
4   jpeg_size,         # JPEG 数据大小
5   output_pic_desc,   # 输出 NV12 pic_desc
6   stream             # Stream 对象
7 )
8 acl.rt.synchronize_stream(stream)
```

完整解码示例

```
1 # 拷贝 JPEG 到设备内存
2 jpeg_dev, _ = media.dvpp_malloc(len(jpeg_data))
3 acl.rt.memcpy(jpeg_dev, len(jpeg_data), jpeg_data.ctypes.data,
4               len(jpeg_data), ACL_MEMCPY_HOST_TO_DEVICE)
5
6 # 获取图像信息
7 w, h, fmt, _ = media.dvpp_jpeg_get_image_info(jpeg_dev, len(jpeg_data))
8
9 # 预测 + 解码
10 out_size, _ = media.dvpp_jpeg_predict_dec_size(jpeg_dev, len(jpeg_data),
11         ↪ PIX_FMT_NV12)
12 out_buf, _ = media.dvpp_malloc(out_size)
13 out_pic = create_nv12_pic_desc(out_buf, w, h) # 见 vpc_guide
14 media.dvpp_jpeg_decode_async(ch_desc, jpeg_dev, len(jpeg_data), out_pic, stream)
15 acl.rt.synchronize_stream(stream)
16
17 # 取回 NV12
18 nv12_host = np.zeros(out_size, dtype=np.uint8)
19 acl.rt.memcpy(nv12_host.ctypes.data, out_size, out_buf, out_size,
20               ACL_MEMCPY_DEVICE_TO_HOST)
```

5.5.5. 练习脚本走读

完整代码见 `samples/chapter5/jpeg/jpeg_minimal.py`。程序使用原始 `acl.media` API 演示 JPEG -> JPEGD 闭环。

(1) JPEG — 编码一帧

```

1 # 创建 JPEG 配置
2 jpege_cfg = media.dvpp_create_jpege_config()
3 media.dvpp_set_jpege_config_level(jpege_cfg, 90)
4
5 # 预测编码后最大大小
6 max_size, _ = media.dvpp_jpeg_predict_enc_size(in_pic, jpege_cfg)
7
8 # 分配输出缓冲区
9 out_buf, _ = media.dvpp_malloc(max_size)
10
11 # in/out 参数: 传 max_size, 同步后读 actual_size
12 out_size_arr = np.array([max_size], dtype=np.int32)
13 out_size_ptr = acl.util.numpy_to_ptr(out_size_arr)
14
15 media.dvpp_jpeg_encode_async(ch_desc, in_pic, out_buf,
16                             out_size_ptr, jpege_cfg, stream)
17 acl.rt.synchronize_stream(stream)
18
19 jpeg_actual = int(out_size_arr[0]) # ← 同步后才能读

```

(2) JPEGD — 解码验证

```

1 # 获取 JPEG 信息
2 w, h, fmt, _ = media.dvpp_jpeg_get_image_info(jpeg_dev, jpeg_size)
3
4 # 预测解码大小
5 dec_size, _ = media.dvpp_jpeg_predict_dec_size(jpeg_dev, jpeg_size, PIX_FMT_NV12
6         ↪ )
7
8 # 创建输出 pic_desc + 解码
9 dec_buf, _ = media.dvpp_malloc(dec_size)
10 dec_pic = create_pic_desc(dec_buf, w, h)
11
12 media.dvpp_jpeg_decode_async(ch_desc, jpeg_dev, jpeg_size, dec_pic, stream)
13 acl.rt.synchronize_stream(stream)
14
15 # 验证闭环: 输入 NV12 大小 = 输出 NV12 大小

```

310B 实测输出

```

1 JPEG OK   640x480 NV12 -> 2097152 bytes JPEG   (quality=90)
2 JPEGD OK  2097152 bytes JPEG -> 640x480 NV12   size=460800
3 闭环验证  PASS   输入=460800 输出=460800

```

JPEG 码流 2MB 是因为测试帧的渐变 + 棋盘格纹理压缩率低（确定性生成，非真实照片）。正常照片在 `quality=85` 下通常只有几十 KB。

与项目的对应关系

文件	角色
jpeg_minimal.py	学习用途——裸 API 编解码闭环

5.5.6. 场景推荐

场景	推荐方案
WebRTC 截图保存	<code>vpc.jpege(frame)</code> -> 写入.jpg 文件
MJPEG 视频流解码	<code>vpc.jpegd()</code> 逐帧解码（配合 VPC resize）
照片缩略图生成	VPC resize -> JLEGE 编码（全硬件管道）
追求最小 JPEG 文件	裸 API + 精细调 <code>quality</code> 参数

全硬件截图管道

```

1 WebRTC NV12 帧 -> VPC resize(320×240) -> JPEG(qquality=80) -> JPEG 文件
2           ↑           ↑
           硬件       硬件

```

5.5.7. 常见问题与调试

问题 #1: JPEG 输出不是 stream_desc

现象：试图用 `dvpp_get_stream_desc_data` 读取编码输出，得到无效数据。

根因：JPEG 输出写入裸内存缓冲区，不是 stream_desc。只有 VENC 使用 stream_desc 输出。

修复: JPEG 输出直接 memcpy 从 out_buf 拷出, 实际大小从 out_size_ptr 读取。

问题 #2: out_size_ptr 的同步时序

现象: synchronize_stream 之前读取 out_size_arr[0], 得到的是 max_size 而非 actual_size。
根因: out_size_arr 是 in/out 参数, 编码器在硬件完成后才写入实际值。
修复: 必须在 synchronize_stream 之后读取 out_size_arr[0]。

问题 #3: predict_enc_size 返回的值远大于实际

现象: predict_enc_size 返回 2MB, 但编码后只有 30KB。
根因: predict_enc_size 返回的是最坏情况的缓冲区大小, 不是预测值。JPEG 的压缩率取决于图像内容。
修复: 按 predict 值分配缓冲区 (保障不溢出), 编码后通过 out_size_arr[0] 取实际大小。

5.5.8. 附录

参数速查表

JPEGE 编码流程

步骤	API	说明
创建配置	dvpp_create_jpege_config — ↪ ()	
设置质量	dvpp_set_jpege_config_level — ↪ (cfg, 1-100)	越大画质越好
预测输出最大大小	dvpp_jpeg_predict_enc_size — ↪ (pic, cfg)	最坏情况
异步编码	dvpp_jpeg_encode_async — ↪ (ch, pic, buf, ↪ size_ptr, cfg, ↪ stream)	
同步等待	acl.rt. — ↪ synchronize_stream(↪ stream)	

JPEGD 解码流程

步骤	API	说明
获取图像信息	<code>dvpp_jpeg_get_image_info</code>	宽度/高度
	↪ <code>(ptr, size)</code>	
预测输出大小	<code>dvpp_jpeg_predict_dec_size</code>	NV12
	↪ <code>(ptr, sz, fmt)</code>	
异步解码	<code>dvpp_jpeg_decode_async</code>	—
	↪ <code>(ch, ptr, sz,</code>	
	↪ <code>pic_desc, stream)</code>	
同步等待	<code>acl.rt.</code>	—
	↪ <code>synchronize_stream(</code>	
	↪ <code>stream)</code>	

5.6. 集成实战：WebRTC 推流性能对比

完整可运行的代码位于 `samples/chapter5/WebRTC/`。本节聚焦教材层面的架构、H.264/H.265 编码链路和实测结论；具体运行命令、页面使用方式、复现实验步骤见该目录下的 `README.md`。

5.6.1. 项目定位

WebRTC 综合案例将本章前面学习的 VENC、JPEGD、NV12 stride 对齐和 aiortc 编码器适配串联为完整的视频推流管道。它由一个 Python WebRTC 服务端和一个浏览器接收页面组成：服务端在昇腾 310B 上采集、解码、编码视频，通过 aiortc 发送 RTP/WebRTC；浏览器只负责接收和显示。当前实现同时支持 H.264 和 H.265/HEVC 两条编码路径。

HTTP POST /offer 只是信令入口，不在媒体路径里。真正的媒体链路是：

```
1 Ascend 310B frame source -> AscendVideoTrack -> aiortc encoder -> RTP/WebRTC ->
  ↪ Browser
```

项目支持三条视频源路径和两种编码格式：

运行方式	MJPEG 解码	帧格式	编码器	用途
<code>--source demo</code>	无	rgb24 合成帧	libx264	验证 WebRTC 信令和浏览器接收链路

运行方式	MJPEG 解码	帧格式	编码器	用途
--source ↪ usb_camera	CPU (PyAV)	rgb24	libx264	纯 CPU 摄像头基线
--source ↪ usb_camera ↪ --hardware- ↪ encode	CPU (PyAV)	PyAV 转 nv12	CANN VENC H.264	只替换编码器的对照组
--source ↪ dvpp_camera ↪ --video- ↪ codec h264	DVPP JPEGD	nv12	CANN VENC H.264	当前最稳定的 1080p60 路径
--source ↪ dvpp_camera ↪ --video- ↪ codec h265	DVPP JPEGD	nv12	CANN VENC H.265	HEVC 路径，要求浏览器支持 WebRTC H.265

dvpp_camera 模式下硬件编码自动启用,因为 JPEGD 产出的 NV12 帧最适合直接交给 VENC。H.264 是默认编码格式；H.265 只走 CANN VENC，不提供 CPU H.265 fallback。

5.6.2. 目录结构与角色分工

- server.py — 服务入口：HTTP 路由、offer/answer、连接管理
- webrtc_app/
 - ascend_source.py — 视频源适配层：三种模式的初始化与帧产出
 - cann_encoder.py — CANN VENC H.264/H.265 编码器 + aiortc 兼容层
 - dvpp_jpegd.py — DVPP JPEGD 硬件解码器 (MJPEG -> NV12)
 - hevc.py — H.265 Annex-B NAL 解析与 RFC 7798 RTP 分包
 - v4l2_capture.py — PyAV V4L2 MJPEG 采集模块
 - v4l2_raw.py — 直接 ioctl + mmap 采集（更高帧率，自动优先）
- web/ — 浏览器接收页面（HTML/JS/CSS）
- tools/ — H.265 offer/answer 与 VENC 码流验证脚本
- test/ — pytest 测试套件

核心设计原则：设备专属逻辑只写在 webrtc_app/ 内，server.py 只做会话控制和 HTTP 信令，不承载图像处理逻辑。接入新的视频源只需在 AscendVideoTrack 中添加一个 _init_xxx()

分支。

5.6.3. 关键模块走读

server.py — 会话控制层

server.py 是整个服务的入口。它不直接操作 VENC/JPEGD 描述符，但负责把 WebRTC 协商结果和硬件编码器绑定起来。

通用编解码偏好 (_prefer_video_codec_for_sender)：当启用硬件编码时，服务端在 offer 处理中强制设置 video transceiver 的 codec preference。H.264 路径只允许 video/H264，H.265 路径只允许 video/H265。这样可以避免浏览器和 aiortc 协商到 VP8 等软件编码格式，导致硬件 VENC 产出的码流无法被对端消费。

```
1 def _prefer_video_codec_for_sender(pc, sender, mime_type):
2     codecs = [
3         c for c in RTCRtpSender.getCapabilities("video").codecs
4         if c.mimeType.lower() == mime_type.lower()
5     ]
6     for transceiver in pc.getTransceivers():
7         if transceiver.sender == sender:
8             transceiver.setCodecPreferences(codecs)
```

H.264 编码器替换：--hardware-encode 或 --source dvpp_camera 会把 aiortc 的 H264Encoder 替换为 CannH264Encoder。对于 dvpp_camera 模式下产出的 NV12 帧，VENC 是必要条件；否则需要额外做格式转换再交给 libx264。

H.265 能力注册：aiortc 1.14.0 默认没有完整的 H.265 编码器路径。服务端在 --video-codec h265 模式下调用 _patch_h265_encoder()：先确认 CANN ACL 可用，再向 aiortc 的 video codec 列表注册 video/H265 (clock rate 90000)，最后把 encoder factory 指向 CannH265Encoder。

H.265 offer 校验：H.265 失败时机不是服务启动，而是浏览器发起 offer。服务端会检查浏览器 SDP 是否包含 H265/90000；如果没有，直接返回 HTTP 400：

```
1 Browser offer does not contain video/H265. Use a WebRTC HEVC-capable browser.
```

目标码率是建连参数：浏览器页面提交的 bitrate_kbps 会在 POST /offer 时进入服务端。服务端根据分辨率、帧率、编码格式和页面选择解析出本次连接的 VENC 目标码率，并在创建编码器/VENC 通道时使用。当前不支持运行中热调码率；要修改码率，需要断开后重新建连。

连接关闭时先停源再关 PeerConnection：close_peer_connection() 先调用 source_track.stop() 释放 /dev/video0，再 await pc.close()。这样下一个 offer 请求到达时可以立即复用摄像头设备。

ascend_source.py — 视频源适配层

AscendVideoTrack 继承 MediaStreamTrack，是 aiortc 获取视频帧的唯一入口。三种模式的差异全部封装在初始化方法和 _camera_read() 中。

demo 模式：纯软件合成帧——x/y 颜色渐变 + 正弦移动白条 + 余弦移动白方块，不需要任何外设或 CANN 依赖。用于验证 aiortc + 浏览器接收链路是否畅通。

usb_camera 模式(软件基线):V4L2 MJPEG 采集 -> PyAV CPU JPEG 解码 -> RGB VideoFrame →。这是为对比 DVPP 路径准备的软件基线，不依赖 CANN。

dvpp_camera 模式 (全硬件管线):

1. V4L2 采集 MJPEG 码流（优先使用 V4L2RawCapture 直采，失败降级到 PyAV）
2. DvppJpegDecoder 硬件解码为 NV12（含 stride 对齐）
3. 返回 NV12 VideoFrame 给 aiortc -> CannH264Encoder 或 CannH265Encoder 直通编码

V4L2 双后端策略：_init_dvpp_camera() 优先尝试 v4l2_raw.py（直接 ioctl + mmap，帧率更高），失败时降级到 v4l2_capture.py（基于 PyAV）。两套后端提供相同的 read(timeout) 接口，上层的 _camera_read() 不需要知道用哪个。

性能日志：recv() 每 150 帧打印一次 Track FPS，方便在设备端直接观察链路帧率；_camera_read → () 前 5 帧打印单帧解码耗时。

cann_encoder.py — VENC 硬件编码 + aiortc 兼容

这是连接 CANN VENC 和 aiortc 的桥接层，包含三个核心组件：

CannVenc — 同步封装 VENC 异步回调 API。与第 5 章 VENC 教程中的版本相比，WebRTC 版本增加了：

- **pre_padded 参数：**JPEGD 输出的 NV12 已经是 stride 对齐的（宽度对齐到 16），此时跳过 CPU 侧的 stride 重排循环，直接从 dvpp_malloc -> memcpy -> venc_send_frame。这是实现“NV12 零拷贝管道”的关键——整个 JPEGD->VENC 路径上 NV12 的 stride 布局始终一致。
- **回调队列残留检测：**encode() 发送前 drain 回调队列，若有残留帧则 warn。这能在日志中暴露“上一帧结果未被消费”的问题，避免静默丢帧。
- **ACL context 线程安全：**每次 encode() 和回调线程入口都显式 set_context(ctx)，确保线程池中的 executor 线程也能正常访问 ACL 资源。

bgr_to_nv12() — 纯 NumPy 实现的 BGR->NV12 转换。旧版依赖 OpenCV cv2.cvtColor →，现在完全使用 NumPy 整数运算（ITU-R BT.601 转换公式），消除了 OpenCV 依赖。在 dvpp_camera 模式下此函数不会被调用（NV12 已由 JPEGD 产出）；仅在非 NV12 帧走硬件编码时作为回退路径。

CannH264Encoder — 继承 aiortc 的 H264Encoder，只覆盖 _encode_frame()：

H264Encoder (aiortc) 的继承结构——只覆盖 `_encode_frame()`，其余 RTP 封装全部继承：

- `_encode_frame()` -> CANN VENC 编码 **[已覆盖]**
- `_split_bitstream()` -> Annex-B -> NAL 分割 **[继承]**
- `_packetize()` -> NAL -> RTP 分包 **[继承]**
- `pack()` -> RTP 打包逻辑 **[继承]**
- 其他全部继承

关键设计：

- **NV12 直通检测**：检查 `frame.format.name == "nv12"`。DVPP 产出的 NV12 帧直接调用 `to_ndarray`（零格式转换），传入 `pre_padded=True` 跳过 CPU stride 重排。
- **BGR->NV12 走 PyAV reformat**：非 NV12 帧(如 demo 的 RGB 帧)调用 `frame.reformat` ↪ (`format="nv12"`)，由 PyAV 的 C 实现完成色彩空间转换（比手动 `bgr_to_nv12()` 快得多），再 `to_ndarray` 取出。
- **FPS 自适应**：`_estimate_fps()` 从连续帧的 PTS 时间戳差值实时估算实际帧率，据此调整 VENC 的 `src_rate` 参数。当实际帧率因采集链路抖动而变化时，VENC 的码率控制能跟随调整。
- **CANN 不可用时自动回退**：`_CANN_READY` 为 False 时走 `super()._encode_frame()`（CPU libx264），无缝降级。

CannH265Encoder — aiortc 兼容的 H.265 编码器。它复用 `CannVenc`，但创建通道时使用 `ENTYPE_H265_MAIN = 0`，输出 HEVC Annex-B 码流。与 H.264 路径不同，H.265 路径不继承 aiortc 的 H.264 RTP 封装，也不提供 CPU fallback；CANN 初始化或 VENC 创建失败应视为 H.265 模式不可用。

hevc.py — H.265 RTP 分包

H.265 不能复用 H.264 的 RTP 分包格式。webrtc_app/hevc.py 单独实现 HEVC 相关逻辑：

- `split_annexb()`：按 Annex-B 起始码切分 HEVC NAL 单元
- `hevc_nal_type()`：从 HEVC 两字节 NAL header 中提取 NAL type
- `packetize_hevc()`：小 NAL 直接作为 single NAL packet，大 NAL 按 RFC 7798 封装为 Fragmentation Unit
- `has_hevc_keyframe_markers()`：检查 VPS/SPS/PPS/IDR 等关键 NAL，用于验证 VENC H.265 输出是否像合法 HEVC 码流

dvpp_jpegd.py — DVPP JPEGD 硬件解码器

封装了 JPEGD 的完整生命周期：DVPP 通道创建 -> JPEG->NV12 解码 -> 输出缓冲区管理。关键设计：

- **按需分配**:首次解码时通过 dvpp_jpeg_get_image_info 获取宽高,dvpp_jpeg_predict_dec_size → 预测输出大小后分配设备缓冲区。后续帧复用已分配的缓冲区，避免每帧 malloc。
- **stride 对齐的 NV12 输出**: JPEGD 输出的 NV12 已按 16 宽度对齐，此 stride 布局与 VENC 的 pre_padded=True 路径完全匹配。从 JPEGD 到 VENC, NV12 数据无需 CPU 做任何重排。
- **线程安全**: decode() 入口显式 set_context(ctx)，因为此方法在线程池的 executor 线程中被调用。

v4l2_raw.py — 高性能 V4L2 采集

直接在 Python 中通过 fcntl.ioctl + mmap 操作 V4L2 设备，绕过 PyAV 的封装层。相比 PyAV 路径 (1080p 约 15fps)，直接 ioctl 路径可达到摄像头原生帧率 (1080p 约 24fps)。两套后端通过相同的 read(timeout) 接口互换，上层代码无需感知。

v4l2_raw.py 中的 v4l2_buffer 结构体布局 (88 字节) 是针对 aarch64 Linux 的硬编码值。在 x86-64 机器上此模块不可用，自动降级到 PyAV。

5.6.4. 性能实测 (1920×1080)

下面的数据来自 311 (orangepiaipro) 上 2026-05-26 的日志。测试请求为 1920×1080@60，浏览器接收端通过 WebRTC 接收视频流。

模式	路径	Track FPS	瓶颈分析
纯 CPU	V4L2 MJPEG ->	约 15 fps	CPU JPEG 解码单帧约 20~32ms，无法支撑 60fps
	→ CPU JPEG		
	→ decode ->		
	→ RGB ->		
	→ libx264		
H.264 硬编	V4L2 MJPEG ->	基本稳定 60 fps	JPEGD 约 4.9~9.5ms，VENC 约 6.3~6.5ms
	→ DVPP JPEGD		
	→ -> NV12 ->		
	→ CANN VENC		
	→ H.264		

模式	路径	Track FPS	瓶颈分析
H.265 硬编	V4L2 MJPEG -> ↪ DVPP JPEGD ↪ -> NV12 -> ↪ CANN VENC ↪ H.265	通常 57~59 fps, 个别样本掉到 35 fps	码流更小, 但本次日志中稳定性弱于 H.264

关键结论:

- 纯 CPU 路径在 1920×1080@60 请求下只能达到约 15 fps, 主要瓶颈是 CPU JPEG 解码。
- 只替换编码器并不能解决全链路瓶颈; 如果 MJPEG 解码仍在 CPU 上, VENC 的收益会被前级吞掉。
- dvpp_camera 的核心价值是 **JPEGD + VENC** 连用: JPEGD 输出 stride 对齐的 NV12, VENC 直接消费, 中间不需要 RGB 转换。
- 当前这台 311 的稳定 1080p60 优先选 H.264。H.265 码流更小, 但 WebRTC HEVC 接收能力、SDP 协商和端到端稳定性更敏感。

浏览器有时会显示 1920×1088, 这是 VENC 编码面高度按 16 对齐后的 coded size, 不代表有效画面真的变成了 1088 行。1080 不能被 16 整除, 因此硬件会补 8 行 padding。

5.6.5. 依赖说明

服务端（昇腾 310B）需要的依赖:

依赖	用途
v4l-utils	V4L2 命令行工具和库 (sudo apt install)
aiohttp	HTTP 服务端和静态页面托管
aiortc	Python 侧 WebRTC 实现
av (PyAV)	VideoFrame 构造 + V4L2 采集
numpy	演示帧生成和 NV12 数据操作
CANN 8.3.RC1	ACL Python API (硬件编码必需)
pytest	测试框架 (仅开发)

旧版依赖的 opencv-python-headless 已移除。usb_camera 模式和 bgr_to_nv12 均已改用 PyAV + NumPy 纯实现。

5.6.6. 关键结论

- **DVPP 模块不能孤立优化**——只用 VENC 而保留 CPU JPEG 解码，链路仍会卡在 CPU 解码阶段。DVPP 的收益需要全链路协同才能兑现
- **JPEGD + VENC 是 1080p 实时推流的关键组合**——纯软件路径约 15 fps，全硬件 H.264 管道可以稳定到 60 fps
- **NV12 stride 对齐的零拷贝交付**——JPEGD 输出的 stride 对齐 NV12 直接匹配 VENC pre_padded 输入要求，中间不经过任何 CPU 数据重排
- **H.264 与 H.265 的工程取舍不同**——H.264 是当前最稳的 1080p60 WebRTC 路径；H.265 只走 CANN VENC，依赖浏览器暴露 video/H265 WebRTC 能力，且本次 311 实测中有偶发掉帧
- **码率在建连时确定**——页面目标码率会影响本次 VENC 通道创建，当前不支持在线热调

5.7. 应用调试与常见 FAQ

5.7.1. 调试技巧

- **返回值检查**：所有 ACL 接口均返回 ret 状态码，0 表示成功。非 0 需查阅《错误码参考》。
- **日志获取**：设置环境变量 `export ASCEND_GLOBAL_LOG_LEVEL=1`（Info 级别）查看详细日志，日志默认位置在 `~/ascend/log/`。
- **NPU 状态**：`npu-smi info` 查看芯片温度和内存占用。
- **驱动日志**：`dmesg | grep -i venc / dmesg | grep -i vdec / dmesg | grep -i dvpp` 排查硬件错误。

5.7.2. 常见问题

Q: 为什么 `acl.mdl.execute` 报错 “Memory Check Failed”? A: 检查 `acl.mdl.get_input_size_by_index` 获取的大小是否与 `acl.rt.malloc` 的大小严格一致。

Q: DVPP 解码后的图像显示异常? A: DVPP 输出有宽/高对齐要求（如 128×16 对齐）。读取数据时需要根据 stride 跳过 Padding 数据，而不能简单按 `width * height` 读取。

Q: `libascendcl.so: cannot open shared object file`? A: 确认 `LD_LIBRARY_PATH` 包含 `/usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/lib64:/usr/local/Ascend/driver/lib64`。

Q: `No module named 'acl'`? A: 确认 `PYTHONPATH` 包含 `/usr/local/Ascend/ascend-toolkit/latest/python/site-packages`。

Q: VENC 创建通道返回 507018? A: 常见原因: (1) max_bit_rate 单位是 **kbps** 而非 bps (2000 而非 2000000); (2) key_frame_interval (GOP) 为 0, 合法范围 [1, 65536]。

Q: VDEC 解码结果为空帧? A: 检查回调中的 ret_code——非 0 表示解码失败。常见原因: (1) 输入码流不是 Annex-B 格式 (缺 0x00000001 起始码); (2) 首帧缺少 SPS + PPS + IDR; (3) entype 与输入码流的编码格式不匹配。

5.7.3. 使用约束

1. **Context 线程安全:** 一个 Context 可以在多个线程中使用, 但需用户保证并发安全。推荐一线程一 Context。
2. **Stream 约束:** Stream 上的任务按顺序执行, 但异步接口下发后需显式 synchronize 才能确保数据就绪。
3. **内存对齐:** DVPP 对内存地址和图片尺寸有严格对齐要求 (宽对齐到 16, 高对齐到 2)。
4. **通道数量有限:** Ascend 310B4 的 VENC/VDEC 硬件实例通常每种只有 1-2 个。创建通道前确保之前的通道已销毁。

6. 算子开发实战

昇腾 310B 作为一款边缘推理芯片，其算子开发是挖掘硬件性能的关键环节。尽管 CANN (Compute Architecture for Neural Networks) 已提供丰富的内置算子库，但在面对自定义模型结构、特殊后处理逻辑或对极致性能有着严苛要求时，自定义算子开发 (Custom Operator Development) 仍不可或缺。本章将系统性地介绍基于昇腾 310B 的算子开发流程与核心理论，涵盖 TBE DSL 快速原型、算子原型定义、Ascend C 深度优化以及 Tiling 分片计算等关键主题。

6.1. 算子开发概述：昇腾 310B 硬件架构与开发路径

本节作为算子开发实战的开篇，旨在帮助读者构建完整的知识框架。我们将首先解析昇腾 310B 处理器基于达芬奇 (DaVinci) 架构的核心特征，深入其 AI Core 的计算单元与存储层次；随后概述 CANN 异构计算软件栈；最后横向对比 TBE 与 Ascend C 这两条主流的算子开发路径，并探讨自定义算子开发的必要性。

6.1.1. 昇腾 310B 处理器与达芬奇架构

昇腾 310B 是专门面向边缘计算与推理场景打造的高能效 AI 处理器。该芯片采用 12nm FFC 工艺制程，在仅 5 至 8W 的典型功耗下，提供两个版本——20TOPS@INT8 或者 8TOPS@INT8 的澎湃 AI 算力，能效比达到惊人的 25-40 TOPS/W。凭借这一优势，它在工业质检、智能电网、智慧交通等对实时性要求极高的场景中展现出了卓越性能。

达芬奇 (DaVinci) 架构是昇腾系列 AI 处理器的技术基石，其最大的亮点在于极强的可扩展性 (Scalable)。针对不同应用场景的算力需求，达芬奇架构衍生出了 Tiny、Mini、Lite、Standard、Max 等多款版本，全面覆盖从低功耗可穿戴设备到高性能云端数据中心的全场景。其中，昇腾 310B 采用了针对边缘端与移动端专门优化的 DaVinci-mini 架构。

达芬奇架构的核心设计理念可概括为：“将计算任务分层处理，以最契合的计算单元执行最适合的任务”。该架构将计算任务精细划分为标量 (Scalar)、向量 (1D)、矩阵 (2D) 与立方体 (3D) 四种类型，并在物理硬件层面相应地例化了三大核心计算单元——Cube Unit、Vector Unit 与 Scalar Unit。

计算单元的精细分工

Cube Unit（立方体计算单元）是达芬奇架构的标志性设计，更是提供高密度算力的引擎。它专为深度神经网络量身定制，主要承担矩阵乘法、卷积运算及全连接层等计算密集型（Compute-bound）任务。依靠极高的数据复用率，Cube Unit 有效突破了内存带宽的限制。在昇腾 310B 中，其运行频率可达 1.224GHz，是释放全周期算力的关键所在。

Vector Unit（向量计算单元）专门处理向量级运算，工作机制类似于 SIMD（单指令多数数据流）。其功能涵盖归一化、激活函数、池化、数据格式转换等，广泛服务于计算机视觉（如 RPN 网络）中常用的基础算子。Vector Unit 具备极高的执行灵活性，能够无缝兼容并处理多种数据类型与运算模式。

Scalar Unit（标量计算单元）类似于经典的 RISC 微处理器，主要负责控制流管理与基础标量运算。它承担着循环控制、内存地址计算、分支跳转等逻辑任务，是调度统筹整个 AI Core 的“指挥中枢”。

这三大计算单元紧密协同，构建出高度并行的计算流水线。昇腾技术团队在多项典型任务中的实测数据表明，Cube Unit 占用的执行周期（Cycles）显著大于 Vector Unit，这意味着 Cube Unit 的运算潜能得到了充分释放，未受限于 Vector Unit 的调度阻塞，二者实现了优异的负载均衡。

存储层次与数据搬运机制

昇腾 310B 的存储系统采用了精巧的多级层次结构体系。深入理解这一体系，是突破算子性能瓶颈、实现极致优化的先决条件。

全局内存（Global Memory）位于存储架构的最外层，具备最大的容量空间，通常指代片外的板载 LPDDR4X 内存。昇腾 310B 普遍配置 8-16GB 的 LPDDR4X，带宽范围在 51.2GB/s 至 408GB/s。尽管容量充裕，但其访存延迟相对较高。因此，在进行算子开发时，应尽可能减少对全局内存的直接读取次数。

局部内存（Local Memory）集成于 AI Core 内部，属于极低延迟的高速存储区，主要包括 L1 缓存（L1 Buffer）与统一缓冲区（Unified Buffer，简称 UB）。L1 缓存专用于暂存高频复用的数据，从而大幅削减跨总线的读写开销。UB 则是算子执行时的核心工作台，所有参与计算的数据均需先从全局内存搬移至此，方能被计算单元读取。

存储转换引擎（Memory Transfer Engine, MTE）是专为数据搬运与内存重排设计的加速单元。MTE 细分为 MTE1、MTE2、MTE3 等模块，负责高效管理 AI Core 内外不同层级缓冲区之间的数据流动，并能够在搬运过程中硬件加速般地同步完成数据填充（Padding）、转置（Transpose）、Img2Col 等格式化操作。

总线接口单元（Bus Interface Unit, BIU）扮演着 AI Core 与外部存储总线通信的“门户”角色，其主要职责是将 AI Core 发出的各类读写请求精确转化为标准的总线协议交互。

在一个典型的计算流水线中：数据首先通过 BIU 由全局内存接入，随后经 MTE 的高效搬运与格式重组，稳妥驻留于 L1 缓存或 UB 中。紧接着，Scalar Unit 发出调度指令，指挥 Cube

Unit 或 Vector Unit 对 UB 中的数据进行高速运算。处理结束后，输出结果再次交由 MTE 接管，安全、高效地写回至全局内存，由此形成一个无缝闭环。#### 昇腾 AI 异构计算架构 (CANN) {#src-book-chapter6-h5}

正如上一章在探讨 PyACL 编程基础时所述，CANN (Compute Architecture for Neural Networks) 是昇腾全面面向 AI 场景定制的异构计算架构。它在整个计算系统中发挥着承上启下的核心枢纽作用：向上广泛适配 MindSpore、PyTorch、TensorFlow 等主流 AI 框架，向下直接调度并深度释放昇腾 AI 处理器的澎湃算力，是提升硬件计算效率的关键“软件底座”。(CANN 的详细架构、与 CUDA 对比、安装配置等内容请参见第 2 章。)

为了兼容并蓄不同维度的开发需求，CANN 构建了多层次的编程接口与组件体系：- **AscendCL**：统一的应用开发原生接口，旨在屏蔽底层硬件差异，帮助开发者灵活构建从端到云的 AI 应用。- **Ascend C**：基于 C++ 的高性能算子开发语言，允许开发者精细控制底层硬件指令与多级缓存，专为追求极致性能的算子定制而生。- **TBE**：基于 Python 的开发框架，侧重于算子逻辑的快速表达与系统化自动调度优化。- **图引擎 (Graph Engine)**：核心的计算图编译与执行引擎，负责全局网络图的解析、内存复用规划及算子融合优化。

针对算子开发层面，CANN 构建了一站式的全流程工具链支持。涵盖了专属算子编译器、功能验证仿真工具，以及深度的性能调优分析器 (Profiler)，全面辅助开发者打通从代码原型编写、逻辑调试到逼近硬件理论极限的全闭环开发流程。

6.1.2. 算子开发的两条主流路径

针对不同开发需求和性能目标，昇腾 CANN 提供了两条主要的算子开发路径：**TBE (快速原型)** 和 **Ascend C (深度优化)**。

TBE：声明式开发，效率优先

TBE (Tensor Boost Engine) 是一种基于 Python 的算子开发框架，其核心特点在于让开发者专注于描述计算逻辑，由系统自动完成底层的复杂优化。在开发方式上，TBE 提供了 DSL (Domain-Specific Language) 编程模式。开发者无需深入了解昇腾底层硬件架构，只需通过几行简洁的 Python 代码即可描述算子的数学表达式。例如，使用 `dsl.vadd(input_x, input_y)` 即可轻松表达向量加法，随后通过调用 `dsl.auto_schedule()`，TBE 会自动接管并完成模式识别、子图切分、调度模板选择以及底层指令映射等一系列流程。

这种声明式开发范式赋予了 TBE 诸多的优势。首先是开发门槛极低且代码异常简洁，一个完整的加法算子核心逻辑往往只需 10 行左右即可实现。同时，得益于昇腾官方调度模板的自动优化加持，生成的算子性能稳定可靠，这使其非常适合用于算法的原型验证以及非性能瓶颈算子的快速实现。

然而，TBE 的自动化机制也带来了一定的局限。当面对具有极端定制化计算需求或特殊数据流模式的算子时，高度封装的自动调度可能难以将其优化至硬件的理论极致性能。此外，在某些特定的高级算子类型上，当前的 DSL 语法或许尚未提供完全覆盖的底层支持能力。

补充：TBE TIK 模式。TBE 早期还提供了一种名为 **TIK (Tensor Iterator Kernel)** 的 Python 命令式开发模式，允许开发者手动控制数据搬运与计算流水线，在控制粒度上介于 DSL 与 Ascend C 之间。但随着 Ascend C 的成熟，TIK 已被逐步取代，不再推荐用于新项目。本书不再展开 TIK 相关内容，读者若在存量项目中遇到 TIK 算子，可查阅昇腾官方文档。

Ascend C：命令式开发，性能优先

Ascend C 是一种基于 C++ 的高性能算子开发语言，其核心优势在于赋予了开发者对数据流、指令流以及多核并行执行的精细控制权。在编程模型上，Ascend C 采用了 SPMD（单程序多数据）架构，这意味着所有的 AI Core 将执行同一套代码逻辑，但会根据各自的任务 ID 去处理被分配的不同数据区间。

在具体的开发过程中，开发者需要显式地设计计算流水线并深度管理多级存储之间的数据搬运。一个典型的 Ascend C 算子实现紧密围绕着三个连续的核心任务展开：首先是“CopyIn”阶段，负责将输入数据从全局内存高效搬运至局部内存；随后进入“Compute”阶段，调用硬件资源执行具体的数学运算；最后是“CopyOut”阶段，将计算所得的结果从局部内存搬运回全局内存，从而完成整个数据流转的闭环。

这种命令式的开发范式实现了灵活性与极致性能的统一。得益于对底层内存读写和流水线编排的精准把控，Ascend C 能够帮助开发者逼近硬件的理论计算极限，更被广泛应用于搞定大模型场景下如 FlashAttention 等复杂的性能瓶颈算子。此外，其原生支持 CPU 模拟调试与中间变量打印，极大地优化了开发体验。然而，获取这种极致性能也意味着较高的开发门槛，开发者必须对昇腾底层硬件架构有透彻的理解。同时，Ascend C 的代码工程量相对较大，即便是实现一个极简的算子，往往也需要编写数百行代码来完成对底层资源的系统化调度。

两条路径的协同关系

两条开发路径并非互斥，而是可以根据需求协同使用：

- **首选 TBE DSL**，快速实现性能达标的标准算子
- **慎用 Ascend C**，在追求极致性能或实现特殊计算模式时，系统化应用优化策略
- **善用工具链**，坚持数据驱动的性能调优闭环

一个典型的开发流程是：先用 TBE 快速实现算子原型，验证功能正确性；如果性能不达标预期，再用 Ascend C 重写核心计算逻辑，通过精细优化达到极致性能。

CANN 算子库本身包含了丰富的高性能算子，覆盖了大多数常见场景。但在以下情况下，开发者需要考虑自定义算子：

训练场景下的算子缺失：将第三方框架（如 TensorFlow、PyTorch）的训练脚本迁移到昇腾 AI 处理器时，遇到框架支持但 CANN 算子库暂不支持的算子。

推理场景下的模型转换：使用 ATC 工具将第三方框架模型转换为昇腾离线模型时，遇到不支持的算子。

网络性能调优：发现某算子性能较低，成为网络性能瓶颈，需要重新开发一个高性能算子替换原有算子。例如，一个 2048x2048 的矩阵乘法算子，经过系统化优化后，性能可从 512ms 提升至 92ms。

应用后处理加速：应用程序中的某些逻辑涉及数学运算（如查找最大值、数据类型转换），可以封装为自定义算子在 AI 处理器上执行，利用 NPU 提升性能。例如，分类应用中查找概率最大的前 5 个标识，可以开发 ArgMax 算子实现后处理加速。

6.1.3. 小结

通过本节的学习，读者应该对昇腾 310B 的硬件架构、CANN 软件栈以及算子开发的两种路径有了整体认识。从下一节开始，我们将从最简单的 TBE 算子入手，带领读者亲手实现第一个自定义算子，在实践中加深对概念的理解。

6.2. 初体验：使用 TBE DSL 快速实现第一个算子（向量加法）

通过一个最简单的向量加法算子，让读者快速体验 TBE 开发的完整流程。在 VSCode 中编写 Python 脚本，使用 TBE DSL 描述算子逻辑，通过命令行工具编译生成算子文件，并编写简单的测试代码在 NPU 上运行验证。最后总结 TBE 的优缺点及适用场景，激发读者进一步学习的兴趣。

6.2.1. TBE DSL 快速概览

在动手编写代码之前，有必要先理解 **TBE DSL 到底是什么**。

什么是 DSL？

DSL (Domain-Specific Language, 领域特定语言) 是为特定问题域量身定制的编程语言。与通用语言（如 Python、C++）不同，DSL 只关注一个狭窄的领域，用该领域的术语来表达计算意图，从而大幅降低表达复杂度。

TBE DSL 就是昇腾为算子开发这一特定领域设计的 Python 内嵌语言。它运行在 Python 环境中，但提供了一套专用于描述张量计算、调度策略和编译选项的 API。

上图展示了 TBE DSL 的完整体系结构。整个架构自顶向下分为三层：最上层是暴露给开发者的 **Python DSL 接口**（计算描述 -> 调度 -> 编译），中间层是 **TVM 编译框架** 负责张量表达式的优化与代码生成，底层是 **CCE 硬件后端** 将优化后的计算映射到昇腾 310B 的 AI Core 指令。

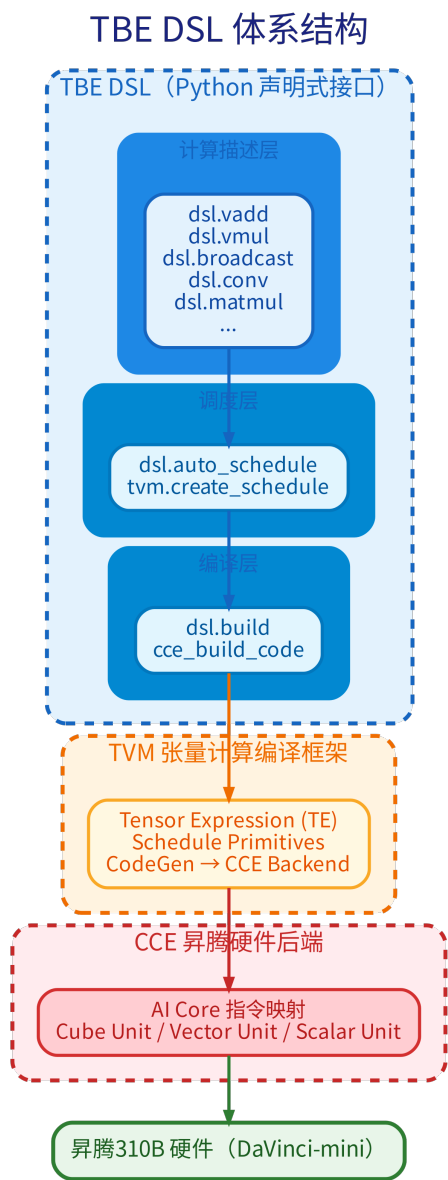


Figure 6.1.: TBE DSL 体系结构

DSL 与 TVM 的关系

TBE 基于 **Apache TVM** 框架深度定制。TVM (Tensor Virtual Machine) 是一个开源的深度学习编译器框架, 它的核心能力是将高层张量计算描述编译为不同硬件后端的可执行代码。TBE 复用了 TVM 的编译基础设施, 并将后端替换为昇腾自家的 CCE。

TVM 在算子编译流程中承担三个关键角色:

TVM 组件	职责	在 TBE DSL 中的体现
Tensor Expression (TE)	用张量表达式描述算子的数学逻辑 (“算什么”)	tbe.dsl 中的计算 API (如 <code>dsl.vadd</code>)
Schedule Primitives	提供低层调度原语 (分块、向量化、绑定等), 决定计算顺序与并行策略 (“怎么算”)	<code>dsl.auto_schedule()</code> 自动选择调度模板; 也可手动调用 <code>tvm.create_schedule()</code> 精细控制
CodeGen	将优化后的调度翻译为目标硬件指令	对接 CCE Backend , 生成昇腾 AI Core 可执行的二进制码

什么是 CCE?

CCE (Cube-based Compute Engine) 是昇腾 CANN 的编译器后端, 专为达芬奇架构设计。它的职责是将 TVM 优化后的计算图翻译为 AI Core 可直接执行的指令流, 包括:

- 将计算任务分配到 Cube Unit、Vector Unit、Scalar Unit
- 生成 MTE 数据搬运指令
- 管理 L1 Buffer 与 Unified Buffer 的分配与回收

CCE 的输入是 TVM 产生的优化 IR (中间表示), 输出是可在昇腾 310B 上加载运行的二进制算子文件 (.o 和 .json)。在整个 DSL 开发流程中, 开发者无需直接与 CCE 交互——`dsl.build()` 和 `cce_build_code()` 会自动调用它完成底层编译。

在 TBE 的声明式范式下, 开发者只需用 **TE 描述计算逻辑**, 调度部分由 `auto_schedule()` 自动完成。这正是 TBE 与 Ascend C 的根本区别——前者说 “算什么”, 后者必须亲自说 “怎么算”。

DSL 的核心 API 分层

TBE DSL 的 API 按职责分为三层:

层	职责	典型 API
计算描述层	定义张量运算的数学逻辑	<code>dsl.vadd</code> , <code>dsl.vmul</code> , <code>dsl.broadcast</code> , <code>dsl.conv</code> , <code>dsl.matmul</code> ...
调度层	将计算映射为硬件执行计划	<code>dsl.auto_schedule</code> , <code>tvm.create_schedule</code>
编译层	生成昇腾设备可执行文件	<code>dsl.build</code>

一个典型的 TBE DSL 算子开发流程就是这三层的顺序调用：先用计算描述层的 API 写出算子的数学表达式，然后交由调度层的 `auto_schedule()` 自动生成硬件执行计划，最后由编译层的 `dsl.build()` 生成可在昇腾设备上运行的二进制文件。这个从声明到编译的完整链路，即为前文 `@fig:dsl_architecture` 中蓝色箭头所描绘的纵向路径。

声明式 vs 命令式

为了进一步理解 DSL 的定位，这里将其与下一节将要学习的 Ascend C 做个对比：

维度	TBE DSL（声明式）	Ascend C（命令式）
编程语言	Python	C++
开发思路	描述”算什么”	控制”怎么算”
调度策略	<code>auto_schedule</code> 自动生成	开发者手动编排流水线
内存管理	框架自动处理	开发者显式管理多级缓存
代码量（以加法为例）	~20 行	~200 行
性能天花板	接近硬件极限（官方模板保证）	可达硬件理论极限
适用场景	原型验证、标准算子	性能瓶颈算子、特殊计算模式

了解了这些基本概念后，我们就可以正式动手了。

6.2.2. 算子分析：明确我们要做什么

在动手编码之前，先明确我们要开发的算子规格。按照昇腾算子开发的规范，我们需要先进行算子分析。

算子功能：实现两个向量的加法，数学表达式为： $z = x + y$

输入输出规格：- 输入：两个张量（Tensor）`x` 和 `y`，形状相同，数据类型相同 - 输出：一个张量 `z`，形状和数据类型与输入相同 - 数据类型支持：`float16`、`float32`、`int32` - Shape 支持：所有形状（本例使用简单的 1D 向量）

开发方式选择：使用 TBE DSL 方式，主要调用两个接口：- `tbe.dsl.broadcast`：处理广播场景（本示例输入 `shape` 相同，但保留广播能力）- `tbe.dsl.vadd`：执行向量加法

算子命名：算子类型（OpType）采用大驼峰命名”Add”；实现文件名称和函数名称采用小写”add”。

TBE DSL 算子的开发流程可以分为四个主要步骤：

阶段	核心任务	关键函数/概念
算子定义	明确输入输出，设计接口	<code>te.placeholder</code>
计算实现	描述算子的数学逻辑	<code>te.compute</code> , <code>te.lang.cce.vadd</code>
调度编译	将计算逻辑映射到硬件	<code>auto_schedule</code> , <code>cce_build_code</code>
验证测试	在 NPU 上运行并核验结果	NumPy 对比, AscendCL 调用

6.2.3. 完整代码实现

创建工程目录

首先在 VSCode 中连接到昇腾 310B 开发板（或直接在板子上操作），创建以下目录结构：

```
1 # 创建工程目录并进入
2 mkdir -p add_tbe
3 cd add_tbe
4
5 # 创建必要的源文件
6 touch add.py run.py
```

预期的目录结构如下：

```
1 add_tbe/
2 |— add.py          # TBE算子实现文件
3 |— run.py          # 测试验证脚本
4 |— kernel_meta/    # 编译输出目录（自动生成）
```

编写 TBE 算子代码（add.py）

下面是完整的向量加法算子实现代码，我们将逐段解释。

```
1 import tbe
2 from tbe import tvn
3 from tbe import dsl
4 from tbe.common.utils import para_check
5 from tbe.common.utils import shape_util
6
7 from functools import reduce
8
9 SHAPE_SIZE_LIMIT = 2147483648
10
11 # 实现Add算子的计算逻辑
```



```

12
13 @tbe.common.register.register_op_compute("add", op_mode="static")
14 def add_compute(input_x, input_y, output_z, kernel_name="add"):
15     shape_x = shape_util.shape_to_list(input_x.shape) # 将shape转换为list
16     shape_y = shape_util.shape_to_list(input_y.shape) # 将shape转换为list
17     shape_x, shape_y, shape_max = shape_util.broadcast_shapes(shape_x, shape_y,
18         ↳ param_name_input1="input_x", param_name_input2="input_y") #
19         ↳ shape_max取shape_x与shape_y的每个维度的大值
18     shape_size = reduce(lambda x, y: x * y, shape_max[:])
19     if shape_size > SHAPE_SIZE_LIMIT:
20         raise RuntimeError("the shape is too large to calculate")
21
22     input_x = dsl.broadcast(input_x, shape_max) # 将input_x的shape广播为
23         ↳ shape_max
24     input_y = dsl.broadcast(input_y, shape_max) # 将input_y的shape广播为
25         ↳ shape_max
26     res = dsl.vadd(input_x, input_y) # 执行input_x + input_y
27
28     return res # 返回计算结果的tensor
29
30 # 算子定义函数
31 def add(input_x, input_y, output_z, kernel_name="add"):
32     # 获取算子输入tensor的shape与dtype
33     shape_x = input_x.get("shape")
34     shape_y = input_y.get("shape")
35     check_tuple = ("float16", "float32", "int32")
36     input_data_type = input_x.get("dtype").lower()
37     if input_data_type not in check_tuple:
38         raise RuntimeError("only support %s while dtype is %s" %
39             ("", ".join(check_tuple), input_data_type))
40     # shape_max取shape_x与shape_y的每个维度的最大值
41     shape_x, shape_y, shape_max = shape_util.broadcast_shapes(shape_x, shape_y,
42         ↳ param_name_input1="input_x", param_name_input2="input_y")
43     if shape_x[-1] == 1 and shape_y[-1] == 1 and shape_max[-1] == 1:
44         # 如果shape的长度等于1, 就直接赋值, 如果shape的长度不等于1, 做切片, 将最
45         ↳ 后一个维度舍弃 (按照内存排布, 最后一个维度为1与没有最后一个维度的
46         ↳ 数据排布相同, 例如2*3=2*3*1, 将最后一个为1的维度舍弃, 可提升后续
47         ↳ 的调度效率)。
48         shape_x = shape_x if len(shape_x) == 1 else shape_x[:-1]
49         shape_y = shape_y if len(shape_y) == 1 else shape_y[:-1]
50         shape_max = shape_max if len(shape_max) == 1 else shape_max[:-1]
51
52     # 使用TVM的placeholder接口对第一个输入tensor进行占位, 返回一个tensor对象
53     data_x = tvm.placeholder(shape_x, name="data_1", dtype=input_data_type)
54     # 使用TVM的placeholder接口对第二个输入tensor进行占位, 返回一个tensor对象
55     data_y = tvm.placeholder(shape_y, name="data_2", dtype=input_data_type)
56
57     # 调用compute实现函数
58     res = add_compute(data_x, data_y, output_z, kernel_name)
59     # 自动调度
60     with tvm.target.Target("cce", host="cce"):
61         schedule = dsl.auto_schedule(res)
62     # 编译配置
63     config = {"name": kernel_name,
64         ↳ "tensor_list": (data_x, data_y, res)}
65     dsl.build(schedule, config)
66
67 # 算子调用

```

```

62 if __name__ == '__main__':
63     input_output_dict = {"shape": (5, 6, 7), "format": "ND", "ori_shape": (5, 6,
        ↪ 7), "ori_format": "ND", "dtype": "float16"}
64     add(input_output_dict, input_output_dict, input_output_dict, kernel_name="
        ↪ add")

```

注意事项：在 CANN 8.3.RC1 版本中，`tvm.target.cce()` 已被标记为废弃（deprecated）。如果运行时出现如下警告：

```

1 UserWarning: target_host parameter is going to be deprecated.
2 Please pass in tvn.target.Target(target, host=target_host) instead.

```

请将 `with tvn.target.cce():` 替换为 `with tvn.target.Target("cce", host="
 ↪ cce"):`，这是 CANN 新版 API 的标准写法。

6.2.4. 代码逐段详解

导入模块

```

1 import tbe
2 from tbe import tvn
3 from tbe import dsl
4 from tbe.common.utils import para_check
5 from tbe.common.utils import shape_util
6 from functools import reduce

```

- tbe: TBE 框架主模块
- tbe.dsl: 包含 DSL 计算接口（如 `vadd`）、调度接口和编译接口
- tbe.tvn: TBE 基于 TVM 框架扩展，可以使用 TVM 接口
- shape_util: 提供 shape 处理工具，如广播 shape 计算
- para_check: 参数校验工具

算子计算逻辑(`add_compute`)这是算子开发的核心,描述”如何计算”。关键点:-`@register_op_compute`
 ↪ 装饰器将函数注册为算子的计算逻辑 - **Shape 大小校验**: 计算广播后张量的总元素个数，当超出 `SHAPE_SIZE_LIMIT` 阈值时抛出异常 - **数据广播与计算**: 先通过 `dsl.broadcast` 将输入形状广播对齐，再调用 `dsl.vadd` 执行向量加法（自动识别为 `element-wise` 模式）

算子主函数（add）

这是算子的入口函数，负责调度和编译：

参数校验：校验数据类型是否在支持范围内（`float16/float32/int32`）。

Shape 切片优化：如果 shape 的末尾维度长度为 1，则将其直接舍弃。由于内存排布上末尾为 1 并不影响实际排布（例如 231 等同于 2*3），舍弃后可有效提升后续的调度效率。

TVM 占位符： `tvm.placeholder` 创建输入张量的占位符，描述张量的形状和数据类型，但不分配实际数据。

自动调度： `dsl.auto_schedule` 自动完成 AST 标注、模式识别、子图切分、调度模板选择，并将指令映射到昇腾硬件。

编译构建： `dsl.build` 将调度后的计算描述编译为昇腾设备可执行的二进制文件。

6.2.5. 编译算子

在终端执行以下命令编译算子：

```
1 python3 add.py
```

编译成功后，会在当前目录生成 `kernel_meta/` 文件夹，包含两个文件：- `add.o`：算子的二进制目标文件 - `add.json`：算子的元信息描述文件

查看生成的文件：

```
1 ls kernel_meta/
2 # 输出示例：add.o add.json
```

6.2.6. 算子验证：在 NPU 上运行测试

编译成功后，我们需要验证算子的正确性。昇腾提供了两种验证方式：- **单算子模型执行**：将算子编译成单算子离线模型（.om 文件），通过 AscendCL 加载执行 - **单算子 API 执行**：直接通过 AscendCL 的 API 调用算子

本节采用第一种方式，因为它更接近实际部署场景。

编写验证代码（run.py）

下面是使用 AscendCL 加载并执行单算子的验证代码：

```
1 # run.py
2 from tbe import tvn
3 from tbe import dsl
4 from tbe.common.utils import para_check
5 from tbe.common.utils import shape_util
6 # 引入testing模块相关接口
7 from tbe.common.testing.testing import *
8 from tbe.common.testing.testing import _Testing
9 import numpy as np
10
11
12 def build(inputs, args=None, name="default_function"):
13     """覆盖 testing 模块的 build(), 将 target 和 target_host 合并为新版 API。"""
14     _Testing.build(inputs, args=args,
15                   target=tvn.target.Target("c", host="llvm"),
16                   target_host=None, name=name,
17                   tiling_keys=None, binds=None, evaluates=None)
18
```

```

19
20 @para_check.check_input_type(dict, dict, dict, str)
21 def addtest(input_a, input_b, output_d, kernel_name="addtest"):
22     # 进入DSL调试模式, 并选择CPU作为运行平台
23     with debug():
24         # 获取算子运行的上下文
25         ctx = get_ctx()
26
27         # 获取输入数据的shape与dtype
28         shape_a = shape_util.scalar2tensor_one(input_a.get("shape"))
29         shape_b = shape_util.scalar2tensor_one(input_b.get("shape"))
30         data_type = input_a.get("dtype").lower()
31
32         # 使用numpy定义输入golden数据大小
33         a = tvm.nd.array(np.random.uniform(size=shape_a).astype(data_type), ctx)
34         b = tvm.nd.array(np.random.uniform(size=shape_b).astype(data_type), ctx)
35         # 使用numpy将输出d初始化为全0
36         d = tvm.nd.array(np.zeros(shape_a, dtype=data_type), ctx)
37
38         # 调用TVM的placeholder接口对输入tensor进行占位, 并返回一个tensor对象
39         data_a = tvm.placeholder(shape_a, name="data_1", dtype=data_type)
40         data_b = tvm.placeholder(shape_b, name="data_2", dtype=data_type)
41         # 调用DSL计算接口实现data_a + data_b
42         data_c = dsl.vadd(data_a, data_b)
43
44         # 中间Tensor数据验证
45         sample = open('samplefile.txt', 'w')
46         # 将中间tensor data_c存入文件samplefile.txt
47         print_tensor(data_c, ofile=sample)
48         # 检查中间tensor data_c的值是否正确
49         assert_allclose(data_c, desired=a.asnumpy() + b.asnumpy(), tol=[1e-7, 1e
50             ↪ -7])
51         print("The value of data_c is the same as the expected value.")
52
53         # 继续自定义DSL的逻辑撰写, 调用DSL接口实现: data_d = data_c + data_b
54         data_d = dsl.vadd(data_c, data_b)
55         # 调用TVM的create_schedule接口, 为算子创建调度实例对象, 入参为输出tensor
56         ↪ 的OP列表。
57         s = tvm.create_schedule(data_d.op)
58
59         # 编译生成算子, data_a, data_b, data_d是占位的输入输出列表, AddTest是我们自
60         ↪ 定义算子的名称
61         build(s, [data_a, data_b, data_d], name="AddTest")
62
63         # 执行算子, 将a, b, d按顺序代入编译出来的DSL算子AddTest
64         run(a, b, d) # AddTest(a, b, d)
65
66         # 将输出数据d的值打印出来, 并预期结果进行比较, 看是否相符
67         print("d:", d)
68         tvm.testing.assert_allclose(d.asnumpy(), a.asnumpy() + b.asnumpy() + b.
69             ↪ asnumpy())
70         print("The actual output is the same as the expected output.")
71
72 # 编写入口函数, 调用addtest函数
73 if __name__ == "__main__":
74     input_output_dict = {"shape": (2, 3, 4), "format": "ND", "ori_shape": (2, 3,
75         ↪ 4), "ori_format": "ND", "dtype": "float32"}
76     addtest(input_output_dict, input_output_dict, input_output_dict, kernel_name

```

```
↪ ="addtest")
```

注意事项：由于 `tbe.common.testing.testing` 模块的 `build()` 函数内部仍使用旧版 API (将 `target` 和 `target_host` 作为独立参数传入 `tvm.build()`)，同样会触发上述废弃警告。解决方案是在 `run.py` 中自定义一个 `build()` 函数来覆盖它，将两个参数合并为 `tvm.target.Target("c", host="llvm")`，从而消除警告。

运行验证

验证代码：

```
1 # 直接运行
2 python3 run.py
```

成功输出示例：

```
1 ===== debug enter =====
2 The value of data_c is the same as the expected value.
3 Tensor add_0 is saved to file samplefile.txt.
4 d: [[[1.0816491 1.9020174 1.2096624 2.5805097 ]
5       [2.401499 1.8532326 1.4911635 2.6252913 ]
6       [1.0037376 2.671739 1.8189309 0.3927391 ]
7
8       [[1.9595971 2.1914093 1.6825981 0.9398521 ]
9         [1.0270492 2.070397 0.97784364 1.7433951 ]
10        [0.4678194 2.564149 1.746469 1.7772455 ]]]
11 The actual output is the same as the expected output.
12 ===== debug exit =====
```

至此，我们完成了第一个 TBE DSL 算子的完整开发流程：从算子分析、代码实现、编译到 NPU 上验证运行。

性能验证：NPU 真的比 CPU 快吗？

成功运行算子只是第一步，一个自然的问题是：**算子跑在 NPU 上到底比 CPU 快多少？**

验证方法并不复杂：用 NumPy 在 CPU 上执行相同规模的加法，与 TBE 算子对比耗时。以下是一个简单的计时示例：

```
1 import time
2 import numpy as np
3
4 shape = (256, 256, 256)           # 约 1670 万个元素
5 dtype = np.float32
6
7 a = np.random.uniform(size=shape).astype(dtype)
8 b = np.random.uniform(size=shape).astype(dtype)
9
10 # CPU 计时
11 t0 = time.time()
12 c_cpu = a + b
```

```
13 t_cpu = time.time() - t0
14
15 # NPU 计时 (在 run.py 的 debug 环境中, 用 TVM 的 time_evaluator)
16 # 详见 CANN Profiling 工具 msprof 的官方文档
17 print(f"CPU time: {t_cpu:.4f}s")
```

对于 256x256x256 规模的 float32 加法, 昇腾 310B 上经 TBE DSL 生成的算子通常可获得数倍乃至十数倍于 CPU 的加速比。这得益于 NPU 的多核并行执行与 UB 缓冲区的极低访存延迟。

需要强调的是, 上述简单计时只适用于初步感受。系统性的性能分析应使用昇腾官方的 **Profiling** 工具 (**msprof**), 它能精确采集 AI Core 执行周期、内存带宽利用率、算子耗时占比等关键指标。Profiling 的具体使用方法将在后续的性能优化章节中详细介绍。

6.2.7. TBE DSL 开发的优势与局限

通过本次实战体验, 我们可以总结 TBE DSL 开发的特点:

优势

优势	说明
开发门槛低	使用 Python 开发, 无需深入了解昇腾硬件架构, 只需描述数学逻辑
代码简洁	一个完整的加法算子核心代码仅需 10 行左右, 大幅减少开发工作量
自动优化	auto_schedule自动完成指令映射、数据切分、流水线优化, 由昇腾官方模板调度, 性能稳定可靠
快速验证	适合算法原型验证和非性能瓶颈算子的快速实现

局限

局限	说明
自动调度的”黑盒”特性	Auto Schedule 对特殊结构的算子可能不够精细, 当算子有强性能诉求时, 可能无法达到极致性能
灵活性受限	对于具有极端定制化需求或特殊数据流模式的算子, DSL 的自动调度可能无法完全满足

局限	说明
高级算子覆盖	某些高级算子类型 DSL 可能尚未完全覆盖

6.2.8. 小结与展望

本节我们通过一个向量加法算子，完整实践了 TBE DSL 开发的四步流程：算子分析、代码实现、编译、验证。你可能会感受到 TBE DSL 带来的便利——用熟悉的 Python 语言，专注于数学逻辑本身，而将底层复杂的硬件适配交给自动调度完成。

这正体现了 TBE 的设计哲学：将开发者从复杂的硬件指令和内存管理中解放出来，专注于算法本身。

然而，正如我们在”局限”中提到的，自动调度并非万能。当追求极致性能、或实现特殊计算模式时，我们需要更精细的控制能力。这正是下一节将要学习的 **Ascend C 开发范式**的用武之地。

在进入下一节之前，建议你亲自动手完成本节示例，并在自己的昇腾 310B 开发板上跑通验证。这是理解后续更深内容的基础。

6.3. 理解算子开发的核心概念

在动手实践之后，本节将系统讲解算子开发中必须掌握的三个核心概念：算子原型定义、算子信息库 (.ini 文件) 以及 Tiling (分片计算)。无论使用 TBE DSL 还是 Ascend C，这些概念都是绕不开的基础知识。

6.3.1. 算子原型定义

一个算子要在 CANN 框架中被正确注册和调用，首先需要明确它的**原型 (Prototype)**。算子原型定义了该算子的完整接口契约，包括：

输入与输出 (Inputs & Outputs)：每个算子接受若干输入张量，产生若干输出张量。张量的规格包括：

要素	说明	示例
Shape	张量的维度信息	(batch, channel, height, ↪ width)
Format	张量的内存排布格式	ND (普通排布)、NHWC、NCHW、 FRACTAL_Z (昇腾分形格式)
Dtype	数据类型	float16、float32、int32、 int8

昇腾支持多种内存排布格式，其中 **FRACTAL_Z** 是昇腾专为 Cube Unit 卷积/矩阵乘法设计的特殊格式，能最大化数据复用效率。在算子开发中，开发者需要明确算子支持的 **format** 列表，必要时在 **compute** 函数中完成格式转换。

属性 (Attributes)：属性是在编译期确定的参数，不同于运行时可变的输入张量。典型的属性包括：

- **axis**：规约操作的轴向（如 Softmax 的归一化维度）
- **kernel_size**、**stride**、**padding**：卷积算子的超参数
- **eps**：归一化算子的小常数（防止除零）

属性在算子注册时声明类型与默认值，在模型编译时固化到算子的 JSON 描述中。

算子类型 (OpType)：每个算子有唯一的类型名称，采用大驼峰命名，如 Add、Conv2D、BatchNorm。这个名称在算子信息库和 AscendCL 调用中均需保持一致。

6.3.2. 算子信息库 (.ini 文件)

算子信息库 (Operator Information Library, 简称 .ini 文件) 是 CANN 算子管理体系的核心配置文件。每个自定义算子都需要一个对应的 .ini 文件，它的主要作用包括：

1. **声明算子的存在**：告诉 CANN 图引擎”有哪些自定义算子可用”
2. **描述算子规格**：输入输出的数量、支持的数据类型与 **format**
3. **关联实现文件**：指定算子对应的编译产物 (.o 或 .so 文件) 路径

一个典型的 .ini 文件结构如下：

```

1 [GENERAL]
2 op_type=Add
3 op_file=add
4 op_compute=add_compute
5
6 [INPUT]
7 dtype=float16,float32,int32
8 format=ND,NHWC,NCHW
9 shape=dynamic
10
11 [OUTPUT]
12 dtype=float16,float32,int32
13 format=ND,NHWC,NCHW
14 shape=dynamic
15
16 [ATTR]
```

各字段含义：- **op_type**：算子类型名称，与代码中注册的名称一致 - **op_file**：算子实现文件（不含扩展名），对应 **kernel_meta/** 下的编译产物 - **op_compute**：计算函数名 - **[INPUT]** / **[OUTPUT]**：声明输入/输出张量支持的数据类型、**format** 和 **shape**（**dynamic** 表示不限制形状） - **[ATTR]**：声明算子的属性及其默认值

在 TBE DSL 开发流程中，如果使用 `dsl.build` 编译算子，CANN 会自动生成对应的 `.json` 描述文件，简化了手动编写 `.ini` 的步骤。但在 Ascend C 开发中，手动配置 `.ini` 是必不可少的环节。

6.3.3. Tiling（分片计算）的基本思想

Tiling（分片计算）是算子开发中最重要的性能优化概念之一。它的核心思想很简单：当数据量超过 AI Core 的 Local Memory 容量时，将数据切成小块，逐块计算。

为什么需要 Tiling？

回顾第 6.1 节介绍的存储层次：昇腾 310B 的每个 AI Core 内部，用于计算的 **Unified Buffer (UB)** 容量有限（通常为 256KB 左右）。而 Global Memory 中的输入数据可能有数十 MB。如果一次加载全部数据到 UB，会直接溢出。

Tiling 的策略是：将输入张量按某个维度切分成多个 **Tile（分片）**，每次只搬运一个 Tile 到 UB，计算完后再搬走结果、加载下一片。这就像用一个小碗吃完一大锅饭——一次一碗，吃完再盛。

Tiling 的三个关键参数

设计 Tiling 策略时，需要确定三个数值：

参数	含义	约束
Tile Size	每次搬入 UB 的数据块大小	不能超过 UB 容量；通常需对齐到 32B 或 64B
Tile Count	总共需要切分的片数	由总数据量 / Tile Size 决定
Core Assignment	每个 AI Core 处理哪些 Tile	多核时按 core_id 分配数据区间

一个简单的 Tiling 示例

以向量加法为例，假设：- 输入向量长度 = 10,000,000（约 38MB float32）- UB 容量 = 256KB - 每块最多处理 256KB / (3 tensors × 4 bytes)约等于 21,000 个元素（需要同时放下输入 x、y 和输出 z）

则定一个 Tile 处理 20,000 个元素，总共需要 10,000,000 / 20,000 = 500 个 Tile。每个 AI Core 处理其中的一部分 Tile，不同 Core 之间完全并行，互不干扰。

在 TBE DSL 中，`auto_schedule` 内部已经集成了自动 Tiling——开发者通常无需手动指定 Tile 大小。这也是 DSL 便利性的体现之一。但在 Ascend C 中，**Tiling 必须由开发者显式设计**和编码——这构成了下一节的核心挑战。

6.3.4. 小结

本节梳理了算子开发的三大基础设施：**原型定义**规定了算子的“身份证”信息（输入输出、类型、属性），**算子信息库**将其注册到 CANN 框架中，而 **Tiling** 则是突破 Local Memory 容量限制的核心技术。这些概念将直接应用于下一节的 Ascend C 实战。

6.4. 深入底层：Ascend C 算子开发入门

Ascend C 是 CANN 提供的 C++ 高性能算子开发语言。与 TBE DSL 的声明式风格不同，Ascend C 要求开发者亲自设计**数据流和指令流**，换取的回报是**逼近硬件理论极限的性能**。本节将以向量加法为起点，带你进入 Ascend C 的世界。

6.4.1. Ascend C 的编程模型

Host-Device 协同

Ascend C 采用经典的 **Host-Device 协同模型**：

- **Host 端 (CPU)**：负责算子的入口逻辑——准备数据、调用 Kernel、回收结果。代码运行在 ARM CPU 上。
- **Device 端 (NPU)**：负责算子的实际计算——AI Core 执行 Kernel 函数中的计算逻辑。

这种分离意味着开发一个算子需要写两部分代码：一个 Host 侧的主控文件和一个 Device 侧的 Kernel 文件。

SPMD 并行模型

Ascend C 采用 **SPMD (Single Program Multiple Data, 单程序多数据)** 架构。所有 AI Core 执行**同一份 Kernel 代码**，但通过内置变量 `block_idx` 获取各自的序号，从而处理不同的数据区间。

昇腾 310B 拥有多个 AI Core (具体数量取决于芯片版本)，每个 Core 独立运行一份 Kernel。开发者需要利用 `block_idx` 来分配任务：

```
1 // 伪代码：多核数据分配
2 uint32_t total_len = 1000000;
3 uint32_t core_num = 8;
4 uint32_t len_per_core = total_len / core_num;
5 uint32_t offset = block_idx * len_per_core;
6 // 当前 Core 处理 [offset, offset + len_per_core) 的数据
```

三段式流水线

每个 AI Core 内部的执行流程被抽象为三个连续阶段，这也是 Ascend C Kernel 代码的标准结构：

阶段	职责	关键操作
CopyIn	将输入数据从 Global Memory 搬运到 Local Memory (UB)	DataCopy, SetAtomicAdd
Compute	在 Local Memory 中执行数学运算	Add, Mul, Mads, ReduceMax
CopyOut	将计算结果从 UB 搬运回 Global Memory	DataCopy

需要注意的是，这三个阶段的粒度是**每个 Tile**——在一个 Tiling 循环中，每个 Tile 都要经历完整的 CopyIn -> Compute -> CopyOut 流程。这恰好对应到上一节 Tiling 概念的实际落地。

Kernel 与 Tiling 的关系

在 Ascend C 中，Kernel 函数本身只描述”对一块数据的计算逻辑”，而 Tiling 参数（分几块、每块多大）由 Host 端计算好后传给 Kernel。这种分工清晰地分离了**计算逻辑**（Kernel 负责）和**数据切分策略**（Host 端的 Tiling 逻辑负责）。

6.4.2. 工程结构

一个标准的 Ascend C 算子工程包含以下文件：

1	add_ascendc/	
2	├─ CMakeLists.txt	# CMake 构建脚本
3	├─ add_custom.h	# 算子头文件（可选）
4	├─ add_custom.cpp	# Host 侧实现 + Kernel 函数
5	├─ run.cpp	# 测试验证脚本
6	├─ add_custom.ini	# 算子信息库
7	└─ build/	# 构建输出目录

关键文件说明：- **add_custom.cpp**：包含 Host 侧的入口函数和 Device 侧的 Kernel 函数。Kernel 函数内部实现 CopyIn -> Compute -> CopyOut 流水线。- **CMakeLists.txt**：配置编译选项，指定依赖的 CANN 库和头文件路径。- **add_custom.ini**：算子信息库文件，声明算子接口。

6.4.3. 向量加法的 Ascend C 实现

下面是一个向量加法的 Ascend C Kernel 实现(完整工程请参见 samples/chapter6/add_ascendc ↪ /):

```

1 #include "kernel_operator.h"
2 using namespace AscendC;
3
4 constexpr int32_t BUFFER_NUM = 2;    // 双缓冲
5
6 class KernelAdd {
7 public:
8     __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z,
9                                 uint32_t totalLength, uint32_t tileNum) {
10         // 多核数据分配
11         this->blockLength = totalLength / GetBlockNum();
12         this->tileLength = this->blockLength / tileNum / BUFFER_NUM;
13
14         xGm.SetGlobalBuffer((__gm__ float *)x + this->blockLength * GetBlockIdx
15                               ↪ (),
16                               this->blockLength);
17         yGm.SetGlobalBuffer((__gm__ float *)y + this->blockLength * GetBlockIdx
18                               ↪ (),
19                               this->blockLength);
20         zGm.SetGlobalBuffer((__gm__ float *)z + this->blockLength * GetBlockIdx
21                               ↪ (),
22                               this->blockLength);
23
24         // 初始化双缓冲队列
25         pipe.InitBuffer(inQueueX, BUFFER_NUM, this->tileLength * sizeof(float));
26         pipe.InitBuffer(inQueueY, BUFFER_NUM, this->tileLength * sizeof(float));
27         pipe.InitBuffer(outQueueZ, BUFFER_NUM, this->tileLength * sizeof(float))
28             ↪ ;
29     }
30
31     __aicore__ inline void Process() {
32         int32_t loopCount = this->tileNum * BUFFER_NUM;
33         for (int32_t i = 0; i < loopCount; i++) {
34             CopyIn(i);    // 从 Global Memory 搬入 UB
35             Compute(i);    // 在 UB 中执行加法
36             CopyOut(i);    // 将结果搬回 Global Memory
37         }
38     }
39
40 private:
41     __aicore__ inline void CopyIn(int32_t progress) {
42         LocalTensor<float> xLocal = inQueueX.AllocTensor<float>();
43         LocalTensor<float> yLocal = inQueueY.AllocTensor<float>();
44         DataCopy(xLocal, xGm[progress * this->tileLength], this->tileLength);
45         DataCopy(yLocal, yGm[progress * this->tileLength], this->tileLength);
46         inQueueX.Enqueue(xLocal);
47         inQueueY.Enqueue(yLocal);
48     }
49
50     __aicore__ inline void Compute(int32_t progress) {
51         LocalTensor<float> xLocal = inQueueX.DeQueue<float>();
52         LocalTensor<float> yLocal = inQueueY.DeQueue<float>();
53         LocalTensor<float> zLocal = outQueueZ.AllocTensor<float>();
54         Add(zLocal, xLocal, yLocal, this->tileLength);
55     }
56 }

```

```

50     outQueueZ.Enqueue<float>(zLocal);
51     inQueueX.FreeTensor(xLocal);
52     inQueueY.FreeTensor(yLocal);
53 }
54 __aicore__ inline void CopyOut(int32_t progress) {
55     LocalTensor<float> zLocal = outQueueZ.DeQueue<float>();
56     DataCopy(zGm[progress * this->tileLength], zLocal, this->tileLength);
57     outQueueZ.FreeTensor(zLocal);
58 }
59
60 private:
61     TPipe pipe;
62     TQue<QuePosition::VECIN, BUFFER_NUM> inQueueX, inQueueY;
63     TQue<QuePosition::VECOUT, BUFFER_NUM> outQueueZ;
64     GlobalTensor<float> xGm, yGm, zGm;
65     uint32_t blockLength = 0, tileNum = 0, tileLength = 0;
66 };
67
68 extern "C" __global__ __aicore__ void add_custom(
69     GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_ADDR workspace, GM_ADDR tiling) {
70     GET_TILING_DATA(tilingData, tiling);
71     KernelAdd op;
72     op.Init(x, y, z, tilingData.totalLength, tilingData.tileNum);
73     if (TILING_KEY_IS(1)) { op.Process(); }
74 }

```

(本节中所述的三段式流水线代码与本段样例代码结构相同、逻辑一致。)

这段代码体现了真实 Ascend C 的几个关键特征：

- **类封装**：Kernel 逻辑封装在类中 (KernelAdd)，通过 Init() 初始化、Process() 执行主循环
- **双缓冲队列**：TQue 配合 EnQue/DeQue 实现了 Double Buffer，CopyIn 和 Compute 可重叠执行
- **多核感知**：GetBlockNum()、GetBlockIdx() 自动获取当前 AI Core 的编号与总数
- **Tiling 数据**：Host 端计算的 Tiling 参数通过 GM_ADDR tiling 传入，GET_TILING_DATA 宏将其解包为结构体
- **aicore 属性**：标记所有运行在 AI Core 上的函数，编译器据此生成 DaVinci 指令

6.4.4. 编译与运行

Ascend C 内核通过 TBE 框架的 compile_op 接口编译，该接口会自动调用 ccec 并设置正确的编译选项。编译命令封装在 Python 算子注册脚本中（参见 samples/chapter6/add_ascendc ↗ /add_custom_tbe.py），最终生成 kernel_meta/ 目录下的 .o 与 .json 文件，随后可通过 AscendCL API 加载并执行。

6.4.5. Ascend C 与 TBE DSL 的对比回顾

经过 Ascend C 的初步体验，我们可以回看第 6.2 节中用 TBE DSL 实现的向量加法：

维度	TBE DSL	Ascend C
语言	Python	C++
代码量	~20 行	~200 行
开发者需要关心的	数学表达式	数据搬运、Tiling、多核分配、指令映射
编译方式	python add.py 自动编译	ccec + CMake 手动构建
性能	良好（官方模板保证）	极致（可逼近理论值）
调试方式	TVM debug 模式	CPU 模拟 + printf 打印
适用场景	标准算子、快速原型	性能瓶颈算子、定制化需求

两种方式并非二选一的关系。在实际项目中，先用 TBE DSL 验证功能正确性，如果性能不达预期，再用 Ascend C 重写核心逻辑，是最常见的开发流程。

6.5. 从简单到复杂：实现一个需要 Tiling 的算子（如矩阵加法）

上一节的向量加法示例刻意简化了 Tiling 逻辑（Kernel 只处理单块数据）。本节将以矩阵加法为例，完整实现一个带 Tiling 的 Ascend C 算子，让读者真正理解分片计算的工程实现。

6.5.1. 场景设定

- 两个矩阵 A 和 B，尺寸均为 $M \times N = 1024 \times 2048$ ，float32 类型
- 每个矩阵占用 $1024 \times 2048 \times 4 = 8\text{MB}$
- 昇腾 310B 的 UB 容量为 256KB，显然无法一次性容纳
- 需要设计 Tiling 策略，将矩阵切成小块逐块计算

6.5.2. Tiling 策略设计

分块大小计算

UB 中需要同时存放三块数据：输入 A 的 Tile、输入 B 的 Tile、输出 C 的 Tile。设每个 Tile 大小为 T 个 float32（4 字节），则占用的 UB 空间为 $3 \times T \times 4 = 12T$ 字节。考虑到 UB 中还需留出空间给中间变量和指令缓存，一般取 UB 总容量的 60%–80% 作为数据区。

¹ UB 数据区上限 约等于 $256\text{KB} \times 70\%$ 约等于 180KB

² T_{max} 约等于 $180\text{KB} / (3 \times 4\text{B})$ 约等于 15,000 个 float32

在实际工程中，为了提高 Cube Unit 的计算效率，Tile 的尺寸通常还需要对齐到 16 或 32 的整数倍。取 $\text{tile_num} = 12,288$ （即 16×768 ）作为一个合适的 Tile 大小。

多核任务分配

假设昇腾 310B 有 8 个 AI Core，每个 Core 独立处理一部分数据。按行切分是最简单的策略：

```
1 每 Core 处理行数 = M / core_num = 1024 / 8 = 128 行
2 每 Core 需处理的 Tile 数 = (128 × N) / tile_num = (128 × 2048) / 12288 约等于 22
   ↪ 个 Tile
```

Host 端根据 `block_idx` 计算每个 Core 的数据起始偏移，Kernel 端在 for 循环中逐 Tile 执行 CopyIn -> Compute -> CopyOut。

6.5.3. 带 Tiling 的 Kernel 伪代码

```
1 extern "C" __global__ __aicore__ void mat_add_with_tiling(
2     GM_ADDR a, GM_ADDR b, GM_ADDR c, GM_ADDR tiling_info) {
3
4     TPipe pipe;
5     uint32_t block_len = tiling_info.block_len;
6     uint32_t tile_num = tiling_info.tile_num;
7
8     // 根据 block_idx 计算当前 Core 的数据偏移
9     uint32_t core_offset = block_idx * block_len * tile_num;
10
11     for (uint32_t i = 0; i < tile_num; i++) {
12         uint32_t offset = core_offset + i * block_len;
13
14         // CopyIn
15         LocalTensor<float> tile_a = pipe.AllocTensor<float>(block_len);
16         LocalTensor<float> tile_b = pipe.AllocTensor<float>(block_len);
17         pipe.DataCopy(tile_a, a + offset);
18         pipe.DataCopy(tile_b, b + offset);
19
20         // Compute
21         LocalTensor<float> tile_c = pipe.AllocTensor<float>(block_len);
22         pipe.Add(tile_c, tile_a, tile_b, block_len);
23
24         // CopyOut
25         pipe.DataCopy(c + offset, tile_c);
26
27         pipe.FreeTensor(tile_a);
28         pipe.FreeTensor(tile_b);
29         pipe.FreeTensor(tile_c);
30     }
31 }
```

这个骨架代码展示了 Tiling 循环的基本结构。完整工程代码（包含 Host 侧 Tiling 参数计算、边界处理、CMakeLists.txt 等）请参见 `samples/chapter6/mat_add_tiling/`。

6.5.4. 调试技巧

Ascend C 提供了两种调试手段，对于排查 Tiling 相关问题尤其有用：

CPU 模拟调试

在开发阶段，可以在 CPU 上模拟运行 Kernel，无需烧录到 NPU：

```
1 # 以 CPU 模式编译并运行
2 ccec --cpu_mode add_custom.cpp -o add_cpu
3 ./add_cpu
```

CPU 模拟支持断点调试和变量打印，适合在复杂 Bug 定位时使用。

printf 打印

在 Kernel 中可以使用 printf 打印中间值（仅 CPU 模拟和 Debug 模式支持）：

```
1 printf("block_idx=%d, tile=%d, offset=%d\n", block_idx, i, offset);
```

这在大规模数据中追踪特定 Tile 的行为时非常高效。注意在生产部署的 Release 版本中需要移除或条件编译，避免影响性能。

6.5.5. 小结与展望

本节通过矩阵加法的 Tiling 实战，展示了 Ascend C 开发中最核心的工程挑战：**如何设计合理的分块策略，使得数据既能塞进有限的 UB，又能最大化 AI Core 的计算效率**。同时介绍了 CPU 模拟调试和 printf 打印两种调试手段。

至此，我们已经覆盖了算子开发的核心内容——从 TBE DSL 的快速原型到 Ascend C 的深度优化，从无 Tiling 的简单算子到带分片策略的复杂实现。在下一章中，我们将转入算子开发的工程化环节：编译部署、单元测试以及系统性的性能优化，将自定义算子真正推向生产环境。

7. 性能分析与优化基础

本章不从概念清单开始, 而是从几段可以运行的代码开始。读者先建立一个能测量的 ResNet18-TinyImageNet 推理基线, 再逐步观察内存申请、数据搬运、CPU 预处理、DVPP 图像处理和流水线并发带来的差异。所有样例代码都放在 `samples/chapter7/`, 模型文件可直接从 Hugging Face 仓库 `zhouxzh/resnet18_tiny_imagenet` 下载到本章目录, 因此不依赖读者先完成第 4 章。

本章使用的是 Tiny-ImageNet 版 ResNet18, 输入为 64x64 RGB 图像, 输出为 200 类分类结果。它的绝对 FPS 不能等同于标准 ImageNet 224x224 ResNet18, 但内存生命周期、数据搬运、预处理下沉和流水线分析方法是通用的。

7.1. 7.1 性能优化先从可测量开始

性能优化的第一步不是改代码, 而是把问题拆成可观测的阶段。边缘 AI 应用通常包含以下链路:

```
1 输入采集 -> CPU/DVPP 预处理 -> Host to Device -> NPU 推理 -> Device to Host ->
   ↪ 后处理 -> 输出
```

本章统一记录以下指标:

指标	含义	适合回答的问题
平均耗时	多次运行的平均阶段耗时	系统总体是否变快
p50	中位数耗时	稳态性能是否符合预期
p95	95 分位耗时	是否存在长尾卡顿
FPS	单位时间处理帧数	吞吐量是否达标
加速比	优化前耗时 / 优化后耗时	单项优化是否值得保留

测试时需要遵守三个规则:

1. 先预热, 再计时。模型加载、首次内存申请和缓存建立会污染第一批数据。
2. 一次只改一个变量。比如只改 Buffer 复用, 不同时改 Batch 和预处理。
3. 同时看端到端和分阶段耗时。只看 `acl.mdl.execute` 会忽略 H2D、D2H 和 Host 阻塞。

7.2. 7.2 第 7 章样例目录

本章代码位于 `samples/chapter7/`:

文件	作用
<code>01_baseline_resnet_sync.py</code>	ResNet18 同步推理基线, 拆分预处理、H2D、Execute、D2H、后处理
<code>02_buffer_reuse_benchmark.py</code>	对比每帧申请 ACL 资源与复用 ACL Buffer/Dataset
<code>03_cpu_preprocess_benchmark.py</code>	对比朴素 CPU 预处理和预分配工作区
<code>04_dvpp_resize_vs_cpu.py</code>	对比 DVPP VPC resize 与 CPU resize
<code>05_pipeline_queue_demo.py</code>	对比串行流程与 Queue Pipeline
<code>06_aipp_preprocess_compare.py</code>	对比 CPU 预处理与已有静态 AIPP OM
<code>tools/download_model.py</code>	从 Hugging Face 下载 ResNet18-TinyImageNet 的 OM/ONNX 模型
<code>tools/convert_aipp_resnet18.sh</code>	调用 ATC 生成静态 AIPP OM
<code>tools/profile_with_msprof.sh</code>	使用 msprof 采集应用 Timeline
<code>tools/summarize_metrics.py</code>	将 JSON 指标汇总成 Markdown 表格
<code>model/resnet18_rgb_static_aipp.cfg</code>	静态 AIPP 配置示例
<code>model/</code>	第 7 章模型和 AIPP 配置目录

阅读本章时, 建议先进入第 7 章样例目录, 后续命令都默认在这个目录下执行:

```
1 cd samples/chapter7
```

本地开发机可以运行语法检查和 CPU/模拟样例。以下命令中的 `python3` 指当前已经安装 NumPy 等依赖的 Python 解释器:

```
1 python3 -m py_compile *.py tools/*.py
2 python3 03_cpu_preprocess_benchmark.py --runs 10
3 python3 05_pipeline_queue_demo.py --simulate --frames 30
```

真实 PyACL、DVPP、msprof 和 npu-smi 验证必须在 Ascend 310B 上运行:

```
1 source /usr/local/Ascend/ascend-toolkit/set_env.sh
2 cd ~/Documents/Ascend310/samples/chapter7
3 python3 tools/download_model.py
4 python3 01_baseline_resnet_sync.py --runs 100
```

7.3. 7.3 工具箱：从轻量计时到系统 Profiling

7.3.1. 7.3.1 Python 分段计时

`perf_counter()` 适合做应用层粗粒度计时。第 7 章的样例通过 `StageRecorder` 记录每一帧的阶段耗时，并输出 JSON：

```
1 {
2   "metrics": {
3     "h2d": {"mean_ms": 0.12, "p95_ms": 0.18, "fps": 8333.33},
4     "execute": {"mean_ms": 1.85, "p95_ms": 2.10, "fps": 540.54},
5     "end_to_end": {"mean_ms": 3.40, "p95_ms": 4.20, "fps": 294.12}
6   }
7 }
```

这种计时不能替代硬件 Profiling，但它有两个优势：代码改动小，能快速判断优化是否有方向。

7.3.2. 7.3.2 msprof：观察 ACL API、Memcpy 和 NPU Timeline

`msprof` 是 CANN 提供的性能采集工具。它能记录 Host 侧 ACL API 调用、Runtime 任务、Stream Timeline、AI Core 指标和部分系统资源信息。

本章提供包装脚本：

```
1 bash tools/profile_with_msprof.sh --name baseline -- \
2   python3 01_baseline_resnet_sync.py --runs 200
```

采集后重点观察：

视图	观察点	常见结论
ACL API	<code>acl.mdl.execute</code> 调用间隔是否很大	Host 预处理或同步等待过慢
Memcpy	H2D/D2H 是否占比过高	输入输出过大或拷贝次数太多
Stream Timeline	NPU 计算任务之间是否有大片空白	NPU 在等待 Host 下发任务或等待数据
AI Core Metrics	Pipe 利用率是否低	模型算子、Batch 或输入尺寸需要继续分析

7.3.3. 7.3.3 npu-smi：确认设备状态

`npu-smi info` 用于确认设备是否在线、温度是否异常、功耗是否接近限制。性能测试前后都应记录一次，避免把降频、设备异常或其他进程占用误判为代码问题。

```
1 npu-smi info
```

7.3.4. 7.3.4 msame / ais_bench：独立测模型上限

当应用端到端 FPS 不理想时，可以用 msame 或 ais_bench 单独测 OM 模型。它们绕过业务代码，只关注模型输入输出和 Runtime 执行，适合回答一个问题：模型本身的推理上限是多少？

如果基准工具显示模型很快，而业务程序很慢，瓶颈通常在预处理、拷贝、后处理或调度。如果基准工具也很慢，才继续考虑模型结构、算子、精度模式和 Batch。

例如，在已经进入 samples/chapter7 并下载模型后，可以用 ais_bench 测普通 OM 的独立推理耗时：

```
1 ais_bench --model model/resnet18_tiny_imagenet.om --batchsize 1
```

不同 CANN 版本和安装方式下，ais_bench 或 msame 的命令行参数可能略有差异；如果本机未安装该工具，可以跳过本节，继续使用本章 Python 分段计时脚本观察端到端瓶颈。

7.3.5. 7.3.5 ATC/AIPP 与 DVPP：把图像工作下沉到设备侧

AIPP 能在模型输入侧完成颜色格式处理和归一化，减少 Host CPU 的像素遍历。示例配置位于：

```
1 model/resnet18_rgb_static_aipp.cfg
```

如果只复现实验结果，先下载本章模型资产：

```
1 python3 tools/download_model.py --all
```

这个命令会把普通 OM、ONNX、AIPP OM 和 AIPP 配置放到 model/。如果需要自己体验离线转换，应在 Ascend 310B 上重新生成 AIPP OM：

```
1 bash tools/convert_aipp_resnet18.sh
```

DVPP 适合处理视频编解码、JPEG 编解码、VPC resize/crop 等媒体任务。第 5 章已经介绍 DVPP API，本章只关心它在性能优化中的角色：减少 CPU 图像处理负担，并让数据尽量早地进入设备侧流水线。

7.4. 7.4 案例一：建立同步推理基线

先运行基线：

```
1 python3 01_baseline_resnet_sync.py --runs 100
```

这个脚本故意采用低效写法：每一帧都申请 Device 输入输出内存、创建 Dataset、执行推理、复制输出、释放资源。它的价值不是快，而是让问题暴露出来。

核心流程如下：

```

1 input_ptr = acl.rt.malloc(input_size, ACL_MEM_MALLOC_HUGE_FIRST)
2 output_ptr = acl.rt.malloc(output_size, ACL_MEM_MALLOC_HUGE_FIRST)
3 input_dataset = acl.mdl.create_dataset()
4 output_dataset = acl.mdl.create_dataset()
5 acl.rt.memcpy(input_ptr, input_size, host_ptr, input_size,
  ↪ ACL_MEMCPY_HOST_TO_DEVICE)
6 acl.mdl.execute(model_id, input_dataset, output_dataset)
7 acl.rt.memcpy(host_out, output_size, output_ptr, output_size,
  ↪ ACL_MEMCPY_DEVICE_TO_HOST)

```

基线需要记录以下阶段：

阶段	含义
preprocess	RGB 输入转换为 NCHW float32
alloc_dataset	申请输入输出 Buffer 并创建 Dataset
h2d	Host 输入复制到 Device
execute	模型执行
d2h	Device 输出复制回 Host
free_dataset	销毁 Dataset 并释放 Buffer
postprocess	argmax 等轻量后处理
end_to_end	单帧端到端耗时

如果 alloc_dataset 和 free_dataset 占比明显，说明优化方向不是模型，而是资源生命周期。

7.5. 7.5 案例二：复用 ACL Buffer 和 Dataset

推理服务通常会长时间运行，输入输出张量大小也是固定的。因此更合理的做法是在模型加载后一次性创建输入输出 Buffer 和 Dataset，在每帧循环里只做数据拷贝和执行：

```

1 python3 02_buffer_reuse_benchmark.py --runs 100

```

优化后的流程：

```

1 初始化阶段：load model -> create dataset -> malloc input/output buffer
2 循环阶段：H2D -> execute -> D2H -> postprocess
3 退出阶段：destroy dataset -> free buffer -> unload model

```

这项优化的判断标准很直接：

1. alloc_dataset 和 free_dataset 从每帧计时中消失。
2. end_to_end 平均耗时下降。
3. msprof Timeline 中 Host 侧 ACL Runtime 调用密度更高，计算任务之间的空白减少。

如果模型很小，Buffer 复用的收益会特别明显；如果模型本身很大，收益占比会下降，但仍然能减少长尾抖动。

7.6. 7.6 案例三：CPU 预处理为什么容易成为瓶颈

很多边缘视觉应用在 NPU 推理前会做 resize、颜色转换、归一化和 HWC 到 CHW 转换。脚本 03_cpu_preprocess_benchmark.py 用确定性 RGB 图像模拟 1080p 摄像头帧：

```
1 python3 03_cpu_preprocess_benchmark.py --runs 100 --resolution 1920x1080
```

如果想观察更稳定的统计结果，可以在确认本机内存充足后把 --runs 增加到 500 或 1000。脚本比较两种方式：

方式	特点
naive_numpy	每帧创建 resize、float32、NCHW 等中间数组
workspace_reuse	预先创建工作区，循环中复用数组

这个案例的学习重点不是把 CPU 预处理调到极致，而是理解何时应该停止优化 CPU。若预处理耗时已经接近或超过模型执行耗时，应优先考虑：

1. 使用 DVPP VPC 做 resize/crop。
2. 使用 AIPP 做归一化和部分颜色格式处理。
3. 使用视频链路时，让 VDEC/JPEGD 的输出直接衔接 VPC 或模型输入。

7.7. 7.7 案例四：DVPP VPC resize 与 CPU resize

脚本 04_dvpp_resize_vs_cpu.py 生成 NV12 测试帧，将 1080p 图像缩小到 540p，对比 CPU resize 和 DVPP VPC resize：

```
1 python3 04_dvpp_resize_vs_cpu.py --frames 60 --resolution 1920x1080
```

该脚本包含 H2D、VPC resize、同步和 D2H，因此测到的是一个完整调用成本，而不是 VPC 内核的裸耗时。如果 DVPP 仍然明显领先 CPU，说明把 resize 从 Host 下沉到设备侧很有价值。如果差距不明显，要继续检查：

1. 输入尺寸是否太小，导致调用开销大于计算收益。
2. 是否每帧都重复创建 DVPP Channel、Stream 或配置对象。
3. 是否存在多余的 H2D/D2H 拷贝。

7.8. 7.8 案例五：AIPP 把归一化下沉到模型输入侧

前面的基线脚本使用 CPU 完成 `resize`、归一化和 HWC 到 CHW 转换，然后把 `float32 NCHW` 输入拷贝到 Device。AIPP 的思路不同：让模型接收 `RGB888_U8` 图像，在模型输入侧完成均值、方差等处理。这样可以减少 Host CPU 的像素遍历，也能减少输入拷贝字节数。

本章提供静态 AIPP 配置：

```
1 model/resnet18_rgb_static_aipp.cfg
```

先下载本章模型资产：

```
1 python3 tools/download_model.py --all
```

`--all` 会把普通 OM、ONNX、AIPP OM 和 AIPP 配置都下载到 `model/`。如果 Hugging Face 仓库中已经有 `model/resnet18_tiny_imagenet_aipp.om`，可以直接运行后面的对比脚本，不必自己转换。

如果需要在板端重新生成 AIPP OM，再运行独立 shell 脚本：

```
1 bash tools/convert_aipp_resnet18.sh
```

脚本默认生成 `model/resnet18_tiny_imagenet_aipp.om`。转换或下载完成后，再运行 Python 对比脚本：

```
1 python3 06_aipp_preprocess_compare.py --runs 100
```

这里故意把 ATC 转换和 Python 推理对比分成两步。ATC 是离线模型转换，可能耗时较长，也可能受板端内存和 CANN 环境影响；Python 脚本只加载已有 OM 做性能测试，不会在测试过程中调用 ATC。

对比脚本会执行两条路径：

1. 使用原始 OM 跑 CPU preprocess -> H2D -> execute -> D2H -> argmax。
2. 使用 AIPP OM 跑 RGB888_U8 prepare -> H2D -> AIPP+execute -> D2H -> argmax，并比较输出差异。

需要注意，AIPP OM 的输入不再是 `float32 NCHW`，而是 `uint8 HWC RGB`。因此不能直接拿 `01_baseline_resnet_sync.py` 去跑 AIPP OM，必须使用本节的对比脚本或在自己的程序中按 AIPP 配置准备输入。

对外部读者来说，还有两个容易踩坑的地方：

1. 运行转换脚本前，需要先完成 CANN 环境配置，并确认当前终端可以直接使用 `atc`。
2. CANN 8.3 对 `RGB888_U8` 的 `mean_chn_*` 字段按整数解析，不能直接写 `123.675` 这样的浮点均值。本章配置使用 `mean_chn_*: 0` 加浮点 `min_chn_*` 表达 ImageNet 均值，并用 `var_reci_chn_*` 表达 `1 / (255 * std)`。

ATC 是离线转换步骤，第一次编译可能明显慢于普通推理脚本。如果转换耗时异常，应先检查板端负载、可用内存、CANN 日志和 AIPP 配置，再重新运行转换脚本。

7.9. 7.9 案例六：串行流程与 Queue Pipeline

串行流程的单帧耗时为：

```
1 T = T_pre + T_infer + T_post
```

三段流水线稳定后的吞吐上限更接近：

```
1 FPS = 1000 / max(T_pre, T_infer, T_post)
```

先在本地使用模拟模式理解这个关系：

```
1 python3 05_pipeline_queue_demo.py --simulate --frames 200
```

再在 Ascend 310B 上使用 PyACL 模式：

```
1 python3 05_pipeline_queue_demo.py --frames 100
```

流水线优化的目标不是降低单帧端到端延迟，而是提高稳态吞吐。它会引入队列等待，因此 p95 latency 可能上升；如果业务目标是交互式低延迟，需要谨慎扩大队列深度。

7.10. 7.10 作者实测结果

本节记录作者在一台 Ascend 310B 设备上得到的实测数据。读者自己的板端温度、CANN 版本、设备健康状态和后台进程不同，绝对数值会有差异；更重要的是复现实验方法和观察相对趋势。

项目	内容
测试主机	313，作者内部测试主机编号
设备名	orangepiaipro
远端仓库	/home/HwHiAiUser/Documents/Ascend310
CANN 路径	/usr/local/Ascend/ascend-toolkit/ ↪ latest
Python	/home/HwHiAiUser/.conda/envs/npu/bin/ ↪ python
测试日期	2026-06-08
设备状态	npu-smi 显示 310B4，Health 为 Alarm，温度约 61 C，测试前后内存约 2939-3003 / 7545 MB

以下表格由 samples/chapter7/tools/summarize_metrics.py 汇总远端 JSON 指标生成。由于测试时设备状态为 Alarm，数据更适合作为章节教学样例和相对趋势参考；正式项目验收应在设备健康状态正常、无其他进程干扰时重新跑完整评测。

案例	变体	平均耗时 (ms)	p95(ms)	FPS	样本数	加速比	说明
ResNet18 同步推理	naive_alloc_45088frame ↔	4.5088	4.712	221.791	100		每帧申请/释放ACL输入输出Buffer和Dataset
ResNet18 Buffer复用	naive_alloc_46047frame ↔	4.6047	6.8023	217.1708	100		对照组
ResNet18 Buffer复用	reuse_acl_buffer ↔	3.2072	3.3509	311.7963	100	1.436	模型加载后一次性创建输入输出Buffer和Dataset
CPU 预处理	naive_numpy ↔	0.62	0.6821	1612.8023	200		每帧创建中间数组
CPU 预处理	workspace_reuse ↔	0.3583	0.4154	2791.1275	200	1.731	复用re-size/CHW工作区
DVPP VPC Resize	cpu_cv2_resize ↔	2.6013	2.6659	384.4225	60		CPU resize NV12 Y/UV
DVPP VPC Resize	dvpp_vpc_resize ↔	3.5966	3.7102	278.0433	60	0.723	DVPP VPC resize + H2D/D2H

案例	变体	平均耗时 (ms)	p95(ms)	FPS	样本数	加速比	说明
AIPP 预处理 下沉	cpu_preproc3.2319 ↔	3.3155	309.4136	100			CPU 完成 resize/normalize/HWC->CHW 后推理
AIPP 预处理 下沉	static_aipp_2.6076 ↔	2.6283	383.5017	100	1.239		AIPP 接收 RGB888_U8 并在模型输入 侧归一化
Queue Pipeline	pyacl_serial3.1943 ↔	3.2332	313.0536	100			Pre -> Infer -> Post 串行
Queue Pipeline	pyacl_queue_pipeline3.0913 ↔	100.0494	250.4402	100	0.8		三段线程通过 Queue 解耦
Queue Pipeline	simulate_serial15.1757 ↔	15.1942	65.8947	200			Pre -> Infer -> Post 串行
Queue Pipeline	simulate_queue_pipeline8.1233 ↔	167.4201	123.1021	200	1.868		三段线程通过 Queue 解耦

从这组数据可以得到五个结论：

1. Buffer 复用是确定有效的优化。ResNet18 端到端平均耗时从 4.6047 ms 降到 3.2072 ms，提升约 1.436 倍。
2. CPU 预处理里的中间数组分配值得优化。复用工作区后，预处理平均耗时从 0.62 ms 降到 0.3583 ms。

3. DVPP 不是简单替换 API 就一定更快。本章脚本为了教学完整性包含每帧 `dvpp_malloc/free`、H2D 和 D2H，结果 DVPP VPC resize 平均 3.5966 ms，慢于 CPU resize 的 2.6013 ms。真实视频流水线应复用 DVPP 资源，并尽量让 VDEC/JPEGD/VPC 输出继续留在设备侧。
4. AIPP 在本章模型上有明确收益。原始 OM 的 CPU 预处理路径平均耗时为 3.2319 ms，AIPP OM 的 RGB888_U8 输入路径降到 2.6076 ms，提升约 1.239 倍；100 次测试中 top1 匹配率为 1.0，最大单点输出差异为 0.007812。AIPP 改变了模型输入格式，正确的比较对象不是“同一个 OM 换输入”，而是原始 OM 的 CPU 预处理路径与 AIPP OM 的 RGB888_U8 输入路径。
5. Pipeline 需要足够重的阶段才有收益。模拟场景中队列流水线从 65.8947 FPS 提升到 123.1021 FPS；但 ResNet18 小模型的 PyACL 场景中，队列和线程上下文开销使 FPS 从 313.0536 降到 250.4402，且 p95 latency 明显变差。

本次 msprof 采集输出目录如下，可用 MindStudio 或 Ascend Insight 打开：

```
1 outputs/msprof/baseline-remote-latest-20260608-083957/
   ↳ PROF_000001_20260608083959526_RHNMOCNRMNBHCC/mindstudio_profiler_output
```

远端验证命令：

```
1 source /usr/local/Ascend/ascend-toolkit/set_env.sh
2 cd /home/HwHiAiUser/Documents/Ascend310/samples/chapter7
3 PY=/home/HwHiAiUser/.conda/envs/npv/bin/python
4 $PY -m py_compile *.py tools/*.py
5 $PY tools/download_model.py
6 $PY 01_baseline_resnet_sync.py --runs 100 --output outputs/
   ↳ baseline_resnet_sync_remote_latest.json
7 $PY 02_buffer_reuse_benchmark.py --runs 100 --output outputs/
   ↳ buffer_reuse_remote_latest.json
8 $PY 03_cpu_preprocess_benchmark.py --runs 200 --output outputs/
   ↳ cpu_preprocess_remote_latest.json
9 $PY 04_dvpp_resize_vs_cpu.py --frames 60 --output outputs/
   ↳ dvpp_resize_remote_latest.json
10 $PY 05_pipeline_queue_demo.py --simulate --frames 200 --output outputs/
   ↳ pipeline_simulate_remote_latest.json
11 $PY 05_pipeline_queue_demo.py --frames 100 --output outputs/
   ↳ pipeline_pyacl_remote_latest.json
12 $PY tools/download_model.py --all
13 $PY 06_aipp_preprocess_compare.py --runs 100 --output outputs/
   ↳ aipp_preprocess_remote_latest.json
14 $PY tools/summarize_metrics.py \
15   outputs/baseline_resnet_sync_remote_latest.json \
16   outputs/buffer_reuse_remote_latest.json \
17   outputs/cpu_preprocess_remote_latest.json \
18   outputs/dvpp_resize_remote_latest.json \
19   outputs/aipp_preprocess_remote_latest.json \
20   outputs/pipeline_pyacl_remote_latest.json \
21   outputs/pipeline_simulate_remote_latest.json \
22   --output outputs/summary_remote_latest.md
23 npu-smi info
```

Profiling 采集命令：

```

1 bash tools/profile_with_msprof.sh --name baseline-remote-latest -- \
2   /home/HwHiAiUser/.conda/envs/npu/bin/python 01_baseline_resnet_sync.py \
3   --runs 20 --output outputs/baseline_msprof_remote_latest.json

```

7.11. 7.11 调优 Checklist

现象	优先检查	常见优化
NPU Timeline 有大量空白	Host 是否慢、是否同步等待	预处理下沉、异步流水线、减少全局同步
H2D/D2H 占比高	输入输出大小和拷贝次数	合并 Buffer、复用内存、减少回传输出
每帧耗时抖动大	是否每帧 malloc/free	预分配 Buffer、固定 Dataset
CPU 占用高	resize/cvtColor/normalize 是否在 CPU	DVPP、AIPP、C++/NumPy 工作区复用
模型单独基准也慢	算子或模型结构	改输入尺寸、精度模式、 Batch、算子优化
Pipeline FPS 不升反降	队列太深或阶段不均衡	缩小队列、调整线程数、重新划分阶段

7.12. 7.12 练习任务

1. 把 01_baseline_resnet_sync.py 的 --runs 从 100 增加到 1000，对比 p95 是否稳定。
2. 用 msprof 分别采集基线和 Buffer 复用脚本，找出 Timeline 中 Host 空白是否减少。
3. 修改 05_pipeline_queue_demo.py --simulate 中的 --pre-ms、--infer-ms、--post-ms，验证吞吐是否接近最慢阶段。
4. 将自己的模型接入 ReuseResNetRunner 的思路，记录优化前后的 JSON 指标。
5. 运行 tools/download_model.py --all 获取 AIPP OM,或使用 tools/convert_aipp_resnet18
→ .sh 重新生成 AIPP OM，再运行 06_aipp_preprocess_compare.py 比较 CPU 归一化下沉前后的端到端耗时和输出差异。

7.13. 7.13 小结

Ascend 310B 性能优化的核心是先定位瓶颈，再把瓶颈映射到合适的硬件或软件策略。小模型场景中，内存申请、同步和 Host 预处理往往比模型执行更值得关注；视频场景中，DVPP、AIPP

和流水线调度决定了端到端吞吐。调优的最终依据不是经验判断,而是可复现的指标、Timeline和对比报告。

8. 模型量化与精度性能权衡

第 7 章解决的是“应用链路哪里慢”的问题。本章接着回答另一个问题：如果 Profiling 和模型独立基准都说明瓶颈主要在模型本身，是否可以通过降低数值精度来换取更高吞吐和更低内存占用？

模型量化不是简单地把 float32 改成 int8。它同时影响模型转换、输入预处理、校准数据、精度评估和部署回归。对初学者来说，本章的目标不是追求某一个模型的极限性能，而是建立一套安全的量化实验流程：先有 FP32/FP16 基线，再做 INT8 尝试，最后用同一批输入同时比较精度和性能。

8.1. 8.1 什么时候需要量化

不要一开始就量化。更稳妥的判断顺序是：

1. 先用第 7 章的分段计时确认端到端瓶颈。
2. 再用 ais_bench 或 msame 单独测 OM 模型上限。
3. 如果模型执行时间占主要部分，再考虑 FP16、混合精度或 INT8。
4. 如果瓶颈在 CPU 预处理、H2D/D2H 或后处理，优先回到 AIPP、DVPP、Buffer 复用和流水线优化。

量化通常适合以下场景：

场景	是否优先量化	原因
模型单独推理也慢	是	计算本身是主要瓶颈
模型文件和权重占用过大	是	低比特权重可降低存储和加载成本
端到端耗时主要来自预处理	否	量化不能解决 Host 侧瓶颈
精度指标已经接近业务下限	谨慎	INT8 可能带来不可接受的精度损失
模型包含很多不支持 INT8 的算子	谨慎	可能回退到混合精度，收益有限

8.2. 8.2 FP32、FP16 与 INT8 的区别

深度学习推理常见的精度模式可以粗略理解为三层：

精度	特点	常见用途
FP32	精度高，内存和带宽开销大	训练、CPU 对齐基准、精度参考
FP16	精度略低，硬件通常更友好	昇腾 310B 上常用的推理精度
INT8	数值范围更小，需要尺度映射	对性能和内存更敏感的部署场景

FP16 通常属于“比较温和”的优化。很多模型从 FP32 转 FP16 后，Top1、mAP 等指标变化很小。

INT8 则更敏感。它需要把浮点数映射到 8 位整数范围，因此必须决定每一层或每一组张量的 scale 和 zero point。这个过程如果估计不准，就会导致激活截断、输出漂移，甚至 Top1/mAP 明显下降。

8.3. 8.3 PTQ 与 QAT

模型量化常见有两条路线：

路线	全称	特点	适合读者阶段
PTQ	Post-Training Quantization	训练后量化，不重新训练或只做少量校准	部署入门和快速评估
QAT	Quantization-Aware Training	训练时模拟量化误差	对精度要求高的项目

本章建议初学者先从 PTQ 开始，因为它更贴近部署工程流程：拿到已经训练好的 ONNX 模型，准备一小批有代表性的校准数据，转换出量化 OM，再比较精度和性能。

QAT 更适合作为进阶内容。它通常需要回到训练框架中修改模型、插入伪量化节点，并重新训练或微调。如果 PTQ 精度损失不可接受，再考虑 QAT。

8.4. 8.4 校准数据集怎么准备

INT8 PTQ 的关键不是数据越多越好，而是校准数据要覆盖真实输入分布。校准集建议满足以下要求：

- 1. 输入预处理与部署完全一致，包括 `resize`、`crop`、颜色通道、`normalize`。
- 2. 覆盖常见场景，例如白天、夜晚、遮挡、不同角度、不同背景。
- 3. 数量不必很大，入门实验可以从 50 到 200 张开始。
- 4. 固定清单和 hash，避免每次转换使用不同数据导致结果不可复现。

推荐目录结构：

```
1 calibration/
2   images/
3     000001.jpg
4     000002.jpg
5   calib_list.txt
6   README.md
```

calib_list.txt 只保存相对路径：

```
1 images/000001.jpg
2 images/000002.jpg
```

对于 Tiny-ImageNet 版 ResNet18，可以直接从验证集抽取一小部分图像做校准。对于检测模型，则应覆盖不同目标数量、不同尺度和不同背景，否则量化后 NMS 前的置信度分布可能发生明显偏移。

本章 samples/chapter8/01_collect_calibration_list.py 为了让读者没有 Tiny-ImageNet 原始验证集时也能跑通流程，会生成一组确定性的 RGB .npy 样本。它适合教学闭环和脚本验证，但不能替代真实项目中的代表性校准集。正式评估时，应换成来自验证集或业务数据的真实图片。

8.5. 8.5 昇腾模型转换中的精度模式

本章使用的 ResNet18-TinyImageNet 模型来自 Hugging Face 仓库 [zhouxzh/resnet18_tiny_imagenet](#) ➡。进入本章样例目录后，先下载基线 OM 和 ONNX：

```
1 cd samples/chapter8
2 PY=/home/HwHiAiUser/.conda/envs/npu/bin/python
3 $PY tools/download_model.py
```

这一步会把 resnet18_tiny_imagenet.om 和 resnet18_tiny_imagenet.onnx 保存到 samples ➡ /chapter8/model/。当前公开仓库已经提供基线 OM、ONNX 和 AIPP 相关文件；FP16/INT8 OM 默认由本章转换脚本生成。如果课程或仓库后续提供 FP16/INT8 OM，也可以把它们放在同一个 model/ 目录中，后续对齐脚本会直接读取。

ATC 转换时，常见的精度相关参数包括：

参数	作用	说明
--precision_mode= ➡ allow_fp32_to_fp16	允许 FP32 转 FP16	常用的 FP16 推理模式
--precision_mode= ➡ allow_mix_precision	允许混合精度	由转换器根据算子情况选择
--input_format	指定输入布局	必须与模型和预处理一致
--input_shape	固定输入形状	边缘部署建议先固定 shape

参数	作用	说明
--insert_op_conf	插入 AIPP 配置	图像模型常与预处理下沉配合

FP16 转换模板：

```
1 cd samples/chapter8
2 atc \
3   --model=model/resnet18_tiny_imagenet.onnx \
4   --framework=5 \
5   --output=model/resnet18_tiny_imagenet_fp16 \
6   --input_format=NCHW \
7   --input_shape="input.1:1,3,64,64" \
8   --soc_version=Ascend310B4 \
9   --precision_mode=allow_fp32_to_fp16 \
10  --log=info
```

本章已经把这条命令封装到独立 shell 脚本中。读者只需要先进入本章样例目录，再运行：

```
1 cd samples/chapter8
2 bash tools/convert_fp16_resnet18.sh
```

INT8 转换通常还需要校准配置或配套工具链生成的量化信息。不同 CANN 版本的 INT8 流程和参数可能存在差异，因此不要直接把一个模板当成所有模型都能用的命令。正式项目中，应以当前 CANN 版本文档和实际 ATC 日志为准。

本章样例采用两步完成 INT8：

```
1 FP32 ONNX + calibration/calib_list.txt
2 -> 准备 AMCT 支持的 opset 中间模型
3 -> AMCT 静态 PTQ，生成 deploy ONNX
4 -> ATC 转换，生成 INT8 OM
```

对应命令为：

```
1 cd samples/chapter8
2 bash tools/convert_int8_resnet18.sh
```

这个脚本会先调用 tools/quantize_int8_resnet18.py，默认在 outputs/int8_amct/ 中准备一份 opset 11 中间 ONNX，再通过 AMCT 生成 model/resnet18_tiny_imagenet_int8_deploy → .onnx，最后由 shell 脚本直接调用 ATC 生成 model/resnet18_tiny_imagenet_int8.om。Python 脚本只负责 AMCT 校准和 deploy ONNX 生成，不调用 ATC。对于 AMCT deploy ONNX，脚本默认不再额外指定 --precision_mode，避免把量化模型又强行拉回普通混合精度转换路径。

这里要特别区分两类 INT8 ONNX。ONNX Runtime 普通静态量化生成的 QDQ/QOperator 模型适合在 ONNX Runtime 中验证量化误差，但当前 CANN/ATC 路径不能直接解析 QuantizeLinear、DequantizeLinear、QLinearConv 或 com.microsoft::QGemm 这类节点。本章使用 AMCT，是为了生成 ATC 可以继续转换的 deploy ONNX。

因此，如果脚本提示当前 Python 环境缺少 amct_onnx，需要先在开发板上安装与 CANN 版本和 CPU 架构匹配的 AMCT 包。只安装普通的 onnx、onnxruntime 并不能完成本章的真实

INT8 部署流程。如果你的 AMCT 版本支持课程下载的原始 ONNX opset, 也可以给脚本传入 `--amct-opset 0` 关闭中间模型转换。

8.6. 8.6 精度对齐流程

量化后的第一件事不是测 FPS, 而是检查输出是否还可信。建议使用同一批输入比较三组结果:

```
1 PyTorch/ONNX FP32 输出 -> FP16 OM 输出 -> INT8 OM 输出
```

分类模型可以先看:

指标	含义
Top1 是否一致	最直观, 但不够细
Top5 是否一致	对分类排序变化更宽容
最大绝对误差	是否存在异常离群
平均绝对误差	整体输出漂移是否可控
Top1 准确率变化	对业务最有意义

检测模型还要额外比较:

指标	含义
mAP 变化	检测任务主指标
框数量变化	NMS 前后是否稳定
置信度分布	是否整体偏高或偏低
小目标召回	INT8 常见风险点之一

建议设置明确阈值。例如:

```
1 FP16: Top1 下降 <= 0.2%, 平均绝对误差 <= 1e-3
2 INT8: Top1 下降 <= 1.0%, mAP 下降 <= 1.0 到 2.0 个百分点
```

这些阈值不是通用标准, 而是教学阶段的起点。真实项目应由业务风险决定。

8.7. 8.7 性能对比流程

性能对比必须控制变量。不要同时改 Batch、输入尺寸、AIPP、后处理和精度模式。

推荐实验顺序:

- 1. 普通 FP16 OM, CPU 预处理。
- 2. 普通 FP16 OM, AIPP 预处理。

- 3. INT8 OM, CPU 预处理。
- 4. INT8 OM, AIPP 预处理。

每次都记录：

字段	说明
模型文件名和 hash	避免混用旧模型
CANN 版本	精度和算子选择可能受版本影响
npu-smi info	记录温度、Health、内存
runs / warmup	保证对比公平
mean / p50 / p95 / FPS	不只看平均值
精度指标	性能提升不能以不可接受精度损失为代价

第 7 章的 StageRecorder 和 PyACL Runner 已经复用到本章配套代码中。样例目录位于 samples/chapter8/：

```
1 samples/chapter8/  
2   README.md  
3   chapter8_utils.py  
4   01_collect_calibration_list.py  
5   02_compare_outputs.py  
6   03_perf_compare.py  
7   tools/  
8     download_model.py  
9     convert_fp16_resnet18.sh  
10    convert_int8_resnet18.sh  
11    quantize_int8_resnet18.py  
12  model/  
13  calibration/  
14  outputs/
```

推荐读者按下面顺序运行：

```
1 cd samples/chapter8  
2 PY=/home/HwHiAiUser/.conda/envs/npu/bin/python  
3  
4 $PY tools/download_model.py  
5 $PY 01_collect_calibration_list.py --count 50  
6 bash tools/convert_fp16_resnet18.sh  
7 bash tools/convert_int8_resnet18.sh  
8  
9 $PY 02_compare_outputs.py \  
10 --base-model model/resnet18_tiny_imagenet.om \  
11 --candidate-model model/resnet18_tiny_imagenet_fp16.om \  
12 --samples 20  
13  
14 $PY 03_perf_compare.py \  
15 --models model/resnet18_tiny_imagenet.om model/resnet18_tiny_imagenet_fp16.om  
16    ↪ model/resnet18_tiny_imagenet_int8.om \  
17 --labels baseline fp16 int8 \  
18 --runs 100
```

8.8. 8.8 常见失败模式

现象	可能原因	处理建议
ATC 转换失败	算子或量化参数不支持	先转 FP16，确认模型本身可转换
INT8 性能收益很小	多数算子仍回退到 FP16	查看 ATC 日志和 Profiling
Top1 大幅下降	校准集不代表真实输入	重做校准集，检查预处理
某些类别明显变差	类别样本覆盖不足	分类别统计误差
输出全为异常值	输入 layout 或 AIPP 配置不匹配	回到 FP32/FP16 输出逐步对齐
p95 变差	首次加载、内存抖动或混合精度回退	增加预热，检查资源复用

量化失败时不要急着调参数。先确认三件事：

- 1. FP16 OM 是否能与原始模型对齐。
- 2. 校准输入是否与部署输入完全一致。
- 3. INT8 OM 是否真的使用了目标量化路径。

8.9. 8.9 与第 7 章的关系

第 7 章强调“先定位瓶颈”。本章强调“如果瓶颈在模型本身，再改变模型精度”。

可以把两章合起来作为一个决策树：

```
1 端到端慢
2  -> 分段计时
3     -> Host 预处理慢：AIPP / DVPP / 工作区复用
4     -> H2D/D2H 慢：减少输入输出、复用 Buffer、预处理下沉
5     -> 模型 execute 慢：FP16 / INT8 / 模型结构优化
6     -> 队列吞吐不足：Pipeline / 多进程 / 批处理
```

量化不是替代性能分析，而是性能分析之后的一个优化分支。

8.10. 8.10 练习任务

- 1. 进入 samples/chapter8,运行 tools/download_model.py 从 Hugging Face 仓库 zhouxzh
→ /resnet18_tiny_imagenet 下载 ResNet18-TinyImageNet 模型资产。
- 2. 运行 01_collect_calibration_list.py --count 50，生成 calibration/calib_list.
→ txt，并说明这些样本与真实校准集的区别。

3. 运行 `tools/convert_fp16_resnet18.sh`, 转换 `model/resnet18_tiny_imagenet_fp16.om`, 并记录 ATC 日志中的模型输入名和精度模式。
4. 运行 `02_compare_outputs.py`, 用同一批输入比较普通 OM 与 FP16 OM 的 Top1、最大绝对误差和平均绝对误差。
5. 运行 `03_perf_compare.py`, 比较普通 OM 与 FP16 OM 的端到端耗时、p95 和 FPS。
6. 运行 `tools/convert_int8_resnet18.sh` 生成 AMCT deploy ONNX 和 INT8 OM。如果环境缺少 `amct_onnx`, 先补齐 AMCT 工具链, 再用 `02_compare_outputs.py` 与 `03_perf_compare.py` 记录精度下降和性能收益。

8.11. 8.11 小结

模型量化的核心是精度、性能和工程风险之间的权衡。FP16 通常是边缘部署的第一步, INT8 则需要校准数据、精度对齐和回归测试共同支撑。对初学者来说, 最重要的习惯是: 任何量化收益都必须同时给出性能数据和精度数据; 只有“更快”而没有“仍然正确”, 不能算完成部署优化。

9. 项目交付、评测与回归方法论

9.1. 9.1 章节总览

本章建立从需求澄清-> 指标体系-> 评测集-> 迭代节奏-> 资产沉淀-> 交付与回归的一套闭环方法论，让技术决策基于指标与风险敞口，而非经验臆测。核心理念：可量化、可比较、可复用、可追溯。

9.2. 9.2 需求澄清 Canvas

维度	要素	问题提示	示例
场景	输入源/运行环境	摄像头？批处理？	室内 1080p30 低光
目标指标	功能/业务价值 Latency/FPS/精度	用户希望看到什么结果？ 哪些分位数重要？	实时检测 + 分析 <80ms / >=25FPS / mAP>=0.6
约束	能耗/内存/带宽	上限是多少？	功耗 <=15W 内存 <=3GB
风险合规	数据/硬件/算法 隐私/许可	失败模式有哪些？ 是否需要脱敏？	低光/遮挡/抖动 仅上传事件元数据
成本	硬件/云	ROI 衡量？	10 台板卡预算

输出：requirement.yaml（版本化），后续所有评审基于此文档。

9.3. 9.3 指标分层与优先级

层级	类别	指标	说明	失败后果
SLO A	体验	p95 延迟	端到端	体验差/丢帧
SLO A	性能	FPS	稳态吞吐	处理拥堵
SLO B	质量	mAP/F1/Top1	任务正确性	无法满足业务
SLO B	稳定	Crash/小时	可靠性	运维成本高
SLO C	资源	内存峰值/功耗	成本约束	设备异常/降频
SLO C	带宽	上行 kbps	成本/合规	费用/拥塞

优先级：先保障 A（体验 + 功能可用），再稳定 B（质量/稳定），最后优化 C（资源效率）。

9.4. 9.4 Baseline 策略与控制变量法

Baseline 目标：建立“最小改动可运行”标尺。原则：

- 1. 不提前做微优化；
- 2. 记录所有关键参数：模型版本、输入尺寸、预处理策略、硬件温度范围；
- 3. 一次仅改变单个变量（batch、精度、线程数）。基线存档：baseline/<date>-<commit>/
→ metrics.json；对比脚本生成差异报告。

9.5. 9.5 评测集设计原则

原则	内容
代表性	涵盖主流场景/光照/角度
覆盖边界	极端尺寸、模糊、遮挡
可再现	文件命名规范 + 固定清单
可扩展	新增样本不破坏旧索引
标注一致	标注工具/规范/审校流程

目录示例：

```
1 dataset_eval/  
2   images/  
3     day/*.jpg  
4     night/*.jpg  
5     occlusion/*.jpg  
6   annotations/  
7     instances_train.json  
8     instances_val.json  
9   meta/
```

10 README.md

11 version.txt

提供 Hash 列表，防止样本被替换而影响回归可信度。

9.6. 9.6 迭代计划与看板

四阶段：

Sprint	目标	核心产出	风险控制
0	环境/基线	baseline metrics	依赖清单齐全
1	精度与功能稳固	精度报告	数据问题快速反馈
2	性能与稳定	性能对比表/监控上线	Watchdog 验证
3	工程交付包装	Release Notes/脚本	灰度计划制定

看板列：Backlog -> Doing -> Review -> Bench -> Done；性能/精度任务需进入 Bench 列执行对比脚本通过后才可 Done。

9.7. 9.7 资产沉淀文档体系

文档	内容	更新频率
README	快速启动	版本变化时
ARCHITECTURE	架构图/模块说明	结构调整
MODEL_CARD	模型来源/许可/精度/限制	模型更新
EVAL_REPORT	数据与评测方法/指标	每次发布
PERF_REPORT	基线/优化对比	优化后
CHANGELOG	可见版本差异	每次版本
RISK_LOG	已知风险列表	动态

MODEL_CARD 需包含：数据来源、训练超参摘要、输入契约、已知局限、许可（如 Apache-2.0）、安全与偏见说明（若涉及识别敏感属性声明避免用途）。

9.8. 9.8 交付目录与不可变产物


```
1 release/
2   v1.0/
3     manifest.json      # 产物 hash / 版本矩阵
4     models/
5       detect.om
6       classify.om
7       signature.json
8     scripts/
9       run.ps1
10      run.sh
11      watchdog.sh
12     configs/
13       default.yaml
14     docs/
15       model_card_detect.md
16       model_card_classify.md
17       QUICKSTART.md
18     reports/
19       perf.json
20       accuracy.json
```

manifest.json 字段: {version, commit, build_time, model_hashes, dependencies}。

9.9. 9.9 上线前综合 Checklist

类别	检查项	通过标准
功能	核心用例 100%	自动化用例通过
性能	p95 < 目标 +5%	连续 30min 稳定
精度	mAP/Top1 回归差 < 阈值	与基线对比
资源	内存峰值 < 75%	1h 稳态无泄漏
稳定	Crash=0, 重启 =0	守护日志清洁
安全	日志无敏感泄露	关键字段脱敏
配置	签名校验一致	Hash 匹配
回滚	验证上一版本可用	切换 < 30s

9.10. 9.10 验收、回归与漂移监测

交付后 7 天加密监控：记录时延、精度漂移（采样对比模型输出变化）。漂移检测：相同输入集合（Shadow Set）每天抽样跑一次 -> 统计 logits KL 散度/TopK 变化率，高于阈值（如 KL > 0.05）触发报警（潜在数据分布变化或模型文件损坏）。回归集版本化：eval_set_vX；若需替换样本 -> 新增版本，不覆盖旧数据。

9.11. 9.11 风险管理与决策日志

风险登记表: `risk_log.md` 每条包含: ID、描述、影响、概率、缓解、当前状态。决策日志 (Decision Record, ADR): 记录架构/模型/精度策略选择及备选方案放弃理由, 以便新成员快速建立上下文。

9.12. 9.12 章节小结

方法论的核心不是流程文档堆砌, 而是“指标驱动 + 资产沉淀 + 可回滚”三支柱。通过契约化需求、标准化 Baseline、规范化评测与回归体系, 使团队协作更高效、风险暴露更透明、交付结果更可信。

9.13. 9.13 实践任务

1. 输出 `requirement.yaml` (含指标与约束)。
2. 构建 30 张代表图像的 mini 评测集并附 Hash 列表。
3. 生成 `baseline metrics.json` 与后一次优化对比 diff 报告。
4. 制作一个 MODEL_CARD 模板并填写一个模型示例。
5. 编写上线 Checklist 并模拟一项未通过情形与处置方案。

Part II.

Part II: 实验教程

0. 案例 0：初步使用开发板

0.1. 昇腾 310B 开发板介绍

OrangePi AIpro(8T) 开发板是由香橙派联合华为精心打造的高性能 AI 开发板。它采用昇腾 AI 技术路线, 搭载昇腾 310B 处理器 (4 核 64 位处理器 + AI 处理器), 集成图形处理器, 支持 8TOPS INT8 的 AI 算力。板载 8GB/16GB LPDDR4X 内存, 并支持外接 32GB 至 256GB 的 eMMC 模块, 同时支持双 4K 高清输出。

OrangePi AIpro(8T) 拥有丰富的接口资源, 包括两个 HDMI 输出、GPIO 接口、Type-C 电源接口、支持 SATA/NVMe SSD 2280 的 M.2 插槽、TF 插槽、千兆网口、两个 USB3.0、一个 USB Type-C 3.0、一个 Micro USB (用于串口打印调试)、两个 MIPI 摄像头接口以及一个 MIPI 屏幕接口等。此外, 还预留了电池接口。

该开发板广泛适用于 AI 边缘计算、深度视觉学习、视频流 AI 分析、自然语言处理、智能小车、机械臂、无人机、云计算、AR/VR、智能安防及智能家居等领域, 覆盖 AIoT 的各个行业。在软件方面, OrangePi AIpro(8T) 支持 Ubuntu 和 openEuler 操作系统, 能够满足大多数 AI 算法原型验证及推理应用开发的需求。在这个教程中, 我们只介绍基于 Ubuntu 操作系统的昇腾 310B 开发流程。

0.1.1. 开发板详细视图

0.1.2. 开发板硬件规格

0.2. 配件准备

为了顺利进行开发, 请准备以下配件:

1. TF 卡

建议使用容量 64GB 及以上、速率为 Class10 级以上的闪迪 (SanDisk) 品牌 TF 卡。虽然最小支持 32GB, 但为了避免开发过程中出现磁盘空间不足的问题, 推荐使用更大容量。

2. TF 卡读卡器

用于在电脑上读写 TF 卡以刷写系统镜像。建议选择 USB 3.0 速率的读卡器, 以减少系统

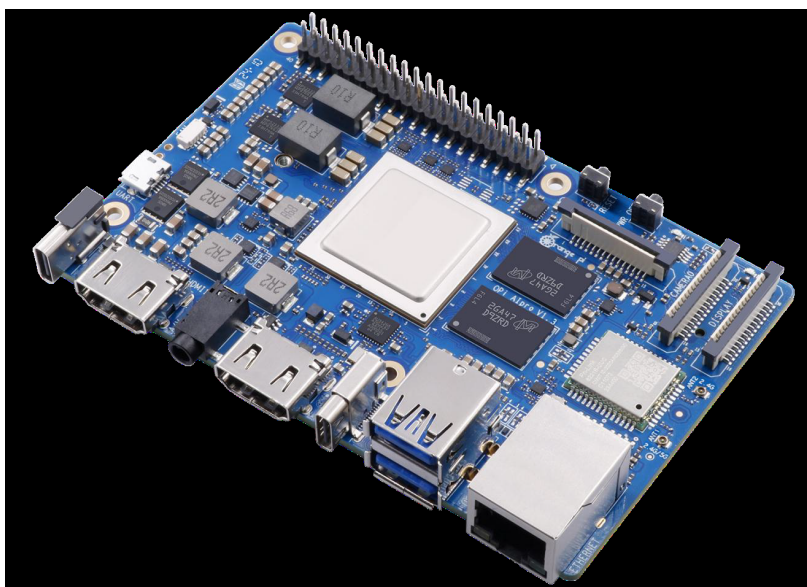


Figure 1.: 产品图

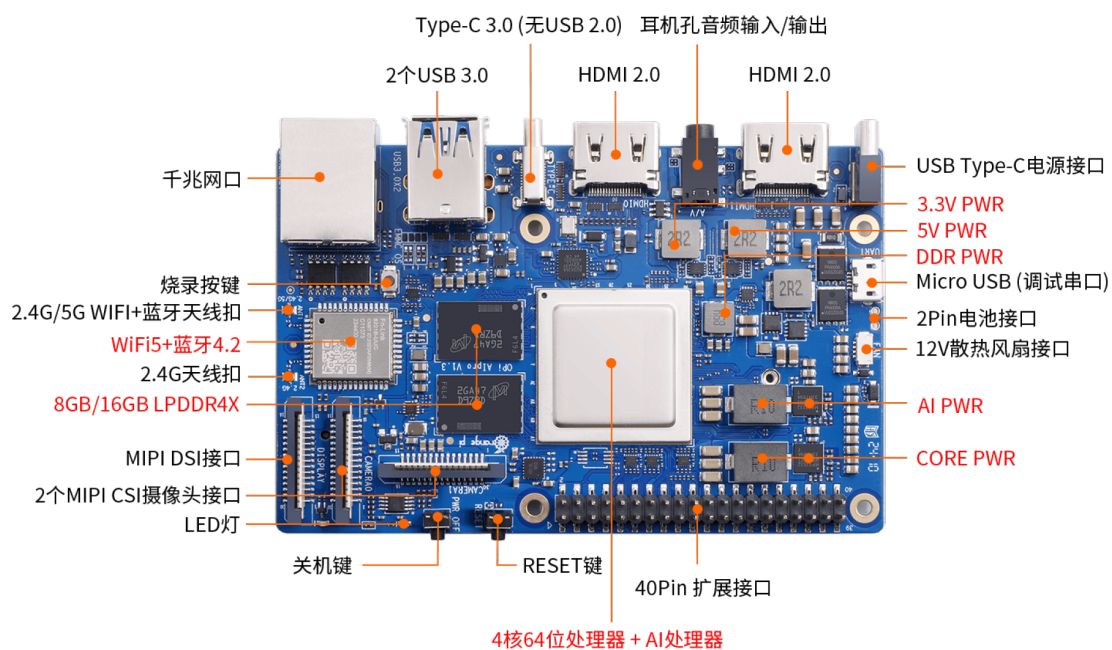


Figure 2.: 正面标注视图

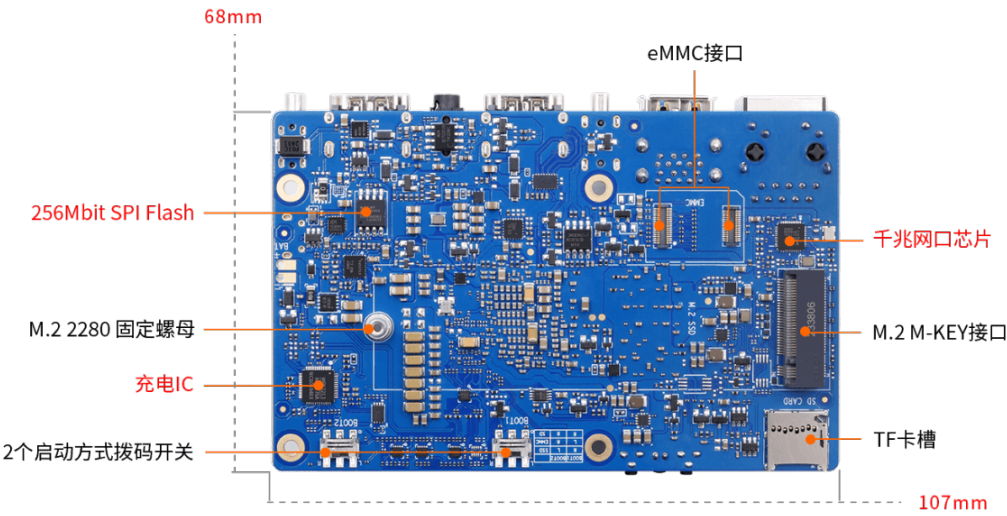


Figure 3.: 背面标注视图

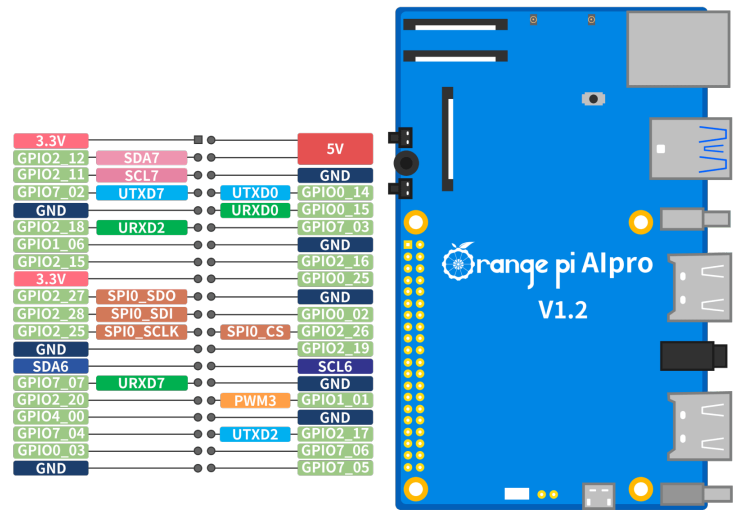


Figure 4.: GPIO 接口定义



Figure 5.: tf 卡



Figure 6.: 读卡器

刷写的等待时间。

3. HDMI 线

开发板配备标准 HDMI 接口。请根据您的显示器接口类型, 准备标准的 HDMI 线或 HDMI 转 Mini-HDMI/Micro-HDMI 线。

4. 电源适配器

开发板采用 PD 协议供电 (20V 挡位), 功率需求为 65W。请务必使用支持 PD 协议 20V 输出的 65W Type-C 电源适配器。

5. USB 鼠标和键盘

用于在本地桌面环境下对开发板进行操作和调试。

6. 金属外壳

用于保护开发板硬件, 防止意外短路或物理损伤。

7. 散热风扇及散热鳍片

由于昇腾 310B 处理器在高负载下发热量较大, 强烈建议安装主动散热设备。开发板提供 2pin 风扇接口 (12V), 支持 PWM 调速。

8. Type-C 转 USB 3.0 转接线 (可选)

开发板的一个 Type-C 接口支持 USB 3.0 协议 (不兼容 USB 2.0), 可通过转接线连接 USB



Figure 7.: HDMI



Figure 8.: Mini HDMI



Figure 9.: PD 电源



Figure 10.: 外壳



Figure 11.: 风扇



Figure 12.: 转接线

3.0 外设。

9. **M.2 NVMe SSD（可选）**

开发板背部的 M.2 接口支持 PCIe 协议（2280 规格），可安装 NVMe SSD 作为系统盘或扩展存储。

10. **M.2 SATA SSD（可选）**

该 M.2 接口同时也支持 SATA 协议，因此也可以使用 M.2 SATA（NGFF）接口的 SSD。

11. **eMMC 模块（可选）**

eMMC 是一种高性能、低功耗的嵌入式存储方案。相比 TF 卡，eMMC 读写速度更快（100-400MB/s）且更稳定。开发板预留了 eMMC 接口，需额外购买香橙派专用 eMMC 模块。

12. **USB 摄像头（可选）**

用于图像识别、视频通话等应用开发。

13. **网线（可选）**

虽然开发板板载 Wi-Fi 模块，但在需要更稳定网络连接的场景下，建议使用千兆网口连接有线网络。



Figure 13.: nvme ssd



Figure 14.: ngff ssd



Figure 15.: eMMC 正面

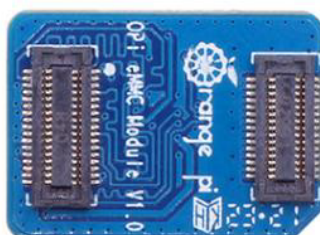


Figure 16.: eMMC 背面



Figure 17.: 摄像头

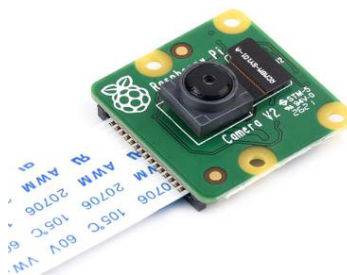


Figure 18.: MIPI-CSI 摄像头



Figure 19.: MIPI 显示器

14. 树莓派 IMX219 摄像头 (MIPI-CSI) (可选)

开发板提供两个 MIPI-CSI 接口，兼容树莓派 IMX219 摄像头，可直接连接而无需占用 USB 接口。

15. MIPI LCD 显示屏 (可选)

开发板配备 MIPI-DSI 显示接口，可直接驱动兼容的 MIPI 显示屏（如树莓派 5 寸屏），无需外接 HDMI 显示器。

16. Micro USB 数据线 (可选)

开发板板载 CH343P 芯片，将 UART 调试串口转换为 Micro USB 接口。使用 Micro USB 数据线连接电脑，即可进行串口调试。

0.3. 操作系统安装

作为华为昇腾生态的重要成员，OrangePi AiPro(8T) 支持 Ubuntu 和 openEuler 两种操作系统。由于开发板板载无预装系统，我们需要通过电脑将系统镜像刷写到 TF 卡中。建议使用



Figure 20.: Micro USB 数据线

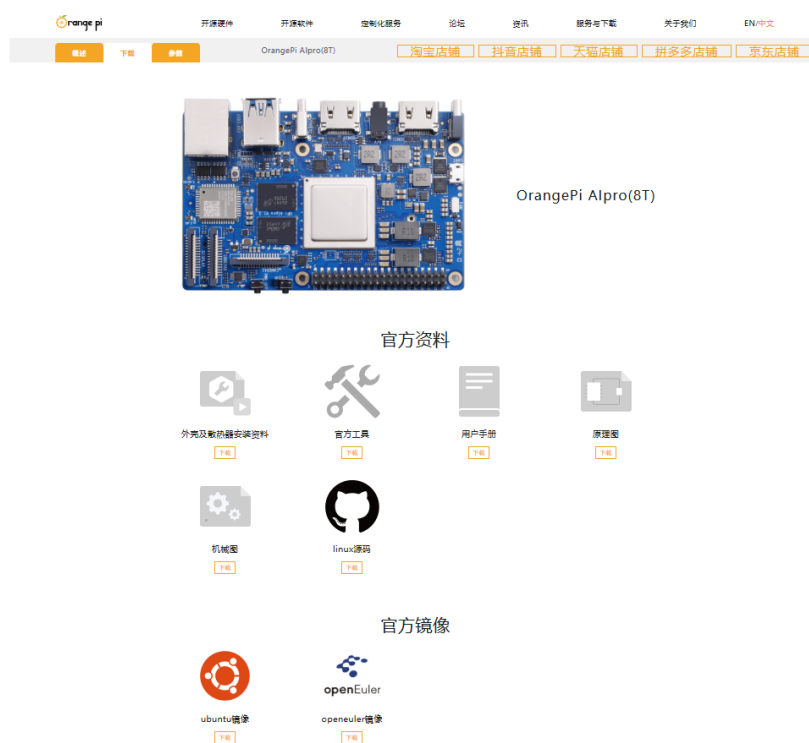


Figure 21.: 技术支持页面

Windows 11 或 Ubuntu 22.04 及以上版本的 PC 进行操作。

首先，访问香橙派官网的[技术支持页面](#)。

在页面下方找到“官方镜像”区域。官方提供了预装昇腾 NPU 应用环境及软件的 Ubuntu 和 openEuler 镜像，极大地方便了开发者快速上手。

0.3.1. 镜像下载

Ubuntu

1. 点击下载：点击 Ubuntu 镜像对应的下载按钮。



Figure 22.: 官方镜像



Figure 23.: 下载



Figure 24.: 跳转

2. 获取提取码：复制弹出的百度网盘提取码，并点击跳转链接。
3. 选择镜像文件：在网盘中找到名为Ubuntu的文件夹并进入。
4. 文件说明：
 - .xz后缀文件为镜像压缩包。
 - .sha后缀文件为 MD5 校验码，用于验证下载文件的完整性。
5. 选择版本：
 - **Desktop**：包含 GUI 图形化界面，适合初学者和需要桌面环境的用户。

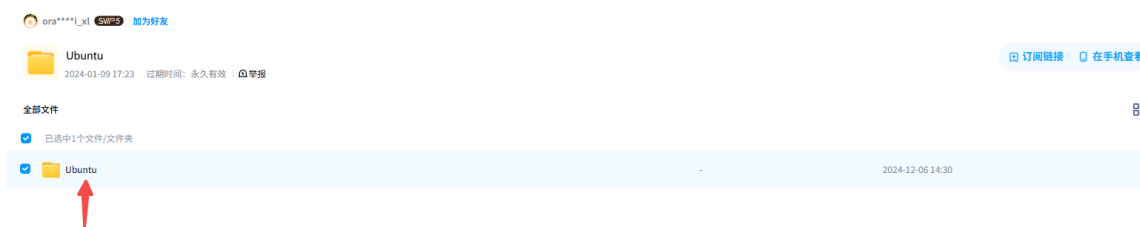


Figure 25.: 文件夹

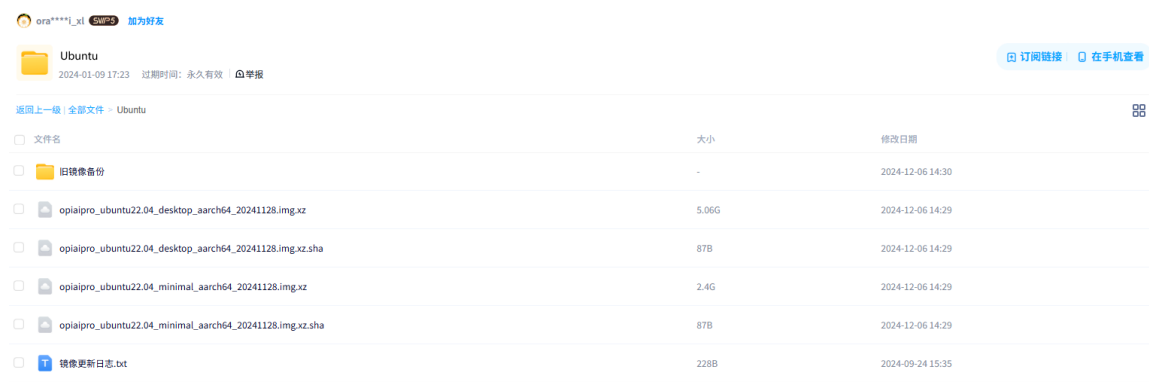


Figure 26.: 选择镜像

- **Minimal**: 仅包含命令行界面，适合高级用户或服务器用途。

建议初学者下载带有**Desktop**字样的镜像。

6. 下载与解压：下载完成后，请先校验文件完整性，再解压.xz压缩包得到.img镜像文件。

0.3.2. 校验下载文件 (MD5)

为了确保下载的镜像文件未损坏，建议在解压前进行 MD5 校验。

- **Windows**: 在镜像文件所在文件夹内按住 **Shift** 键并点击鼠标右键，选择“在终端中打开”或“在此处打开 Powershell 窗口”。输入以下命令（文件名请根据实际情况修改）：

```
1 certutil -hashfile opiaipro_ubuntu22.04_desktop_aarch64_20241128.img.xz
   ↪ md5
```

将输出的 MD5 值与同目录下.sha文件中的内容进行比对。

- **Ubuntu**:

```
1 md5sum <filename>
```

- **macOS**:

```
1 md5 <filename>
```

若校验值一致，说明文件完整，可进行解压；若不一致，请重新下载。

0.3.3. 刷写系统到 TF 卡

工具准备

- 下载链接：[官网](#) | [百度网盘](#)

1. **SD Card Formatter**: 用于格式化 TF 卡，确存储卡状态良好。

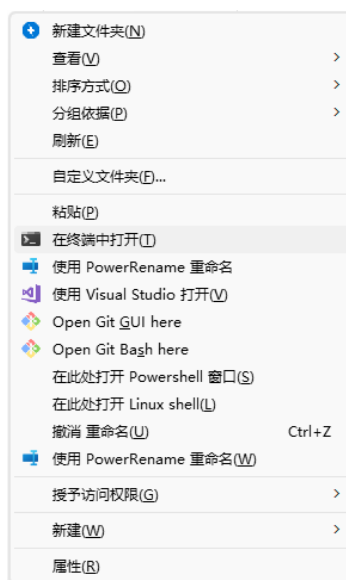


Figure 27.: 终端

```
Microsoft Windows [版本 10.0.26100.4652]
(c) Microsoft Corporation。保留所有权利。

D:\BaiduNetdiskDownload>certutil -hashfile opiaipro_ubuntu22.04_desktop_aarch64_20241128.img.xz md5
MD5 的 opiaipro_ubuntu22.04_desktop_aarch64_20241128.img.xz 哈希:
c2504fd63b2cc222106c30a29ac06386
CertUtil: -hashfile 命令成功完成。

D:\BaiduNetdiskDownload>
```

Figure 28.: md5 校验

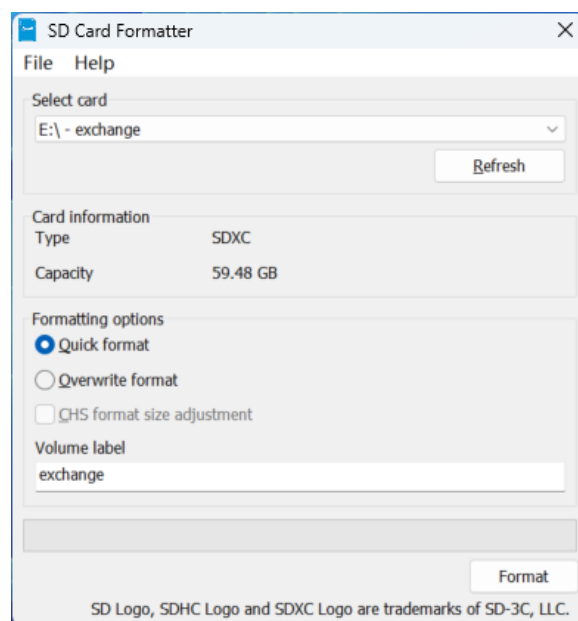


Figure 29.: TF 卡格式化

2. **balenaEtcher**: 用于将 .img 镜像文件烧录到 TF 卡。注意：建议使用 1.19.25 及以下版本，以避免兼容性问题。

格式化 TF 卡

1. 将 TF 卡插入读卡器，连接至电脑。
2. 打开 **SD Card Formatter** 软件。
3. 确认选中了正确的盘符，点击右下角的 **Format** 按钮。
警告：格式化将清除 TF 卡上所有数据，请确认无误后点击“是”。
4. 等待格式化完成，点击确定。

烧录镜像（以 Ubuntu 为例）

1. 打开 **balenaEtcher**，选择“Flash from file”（从文件烧录）。
2. 选择解压后的镜像文件（.img 格式），然后点击“Select target”选择您的 TF 卡。
3. 点击“Flash!” 开始烧录。
4. 烧录完成后软件会自动进行校验，请耐心等待。
5. 校验通过后，关闭软件并安全弹出 TF 卡。

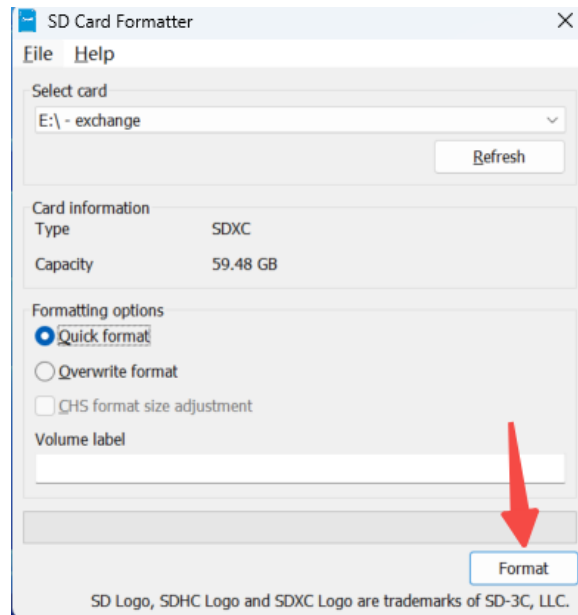


Figure 30.: 格式化

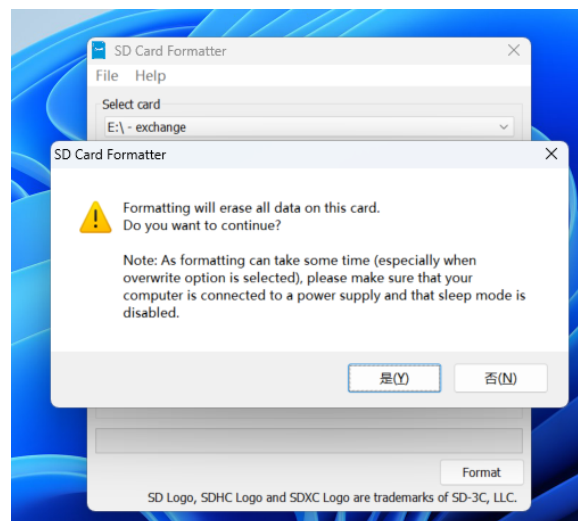


Figure 31.: warning

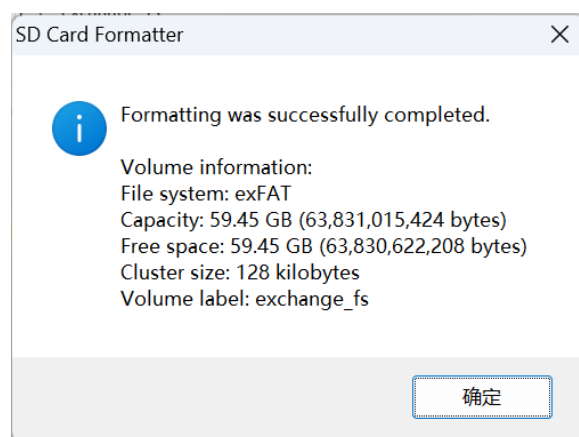


Figure 32.: 格式化完成



Figure 33.: balenaEther

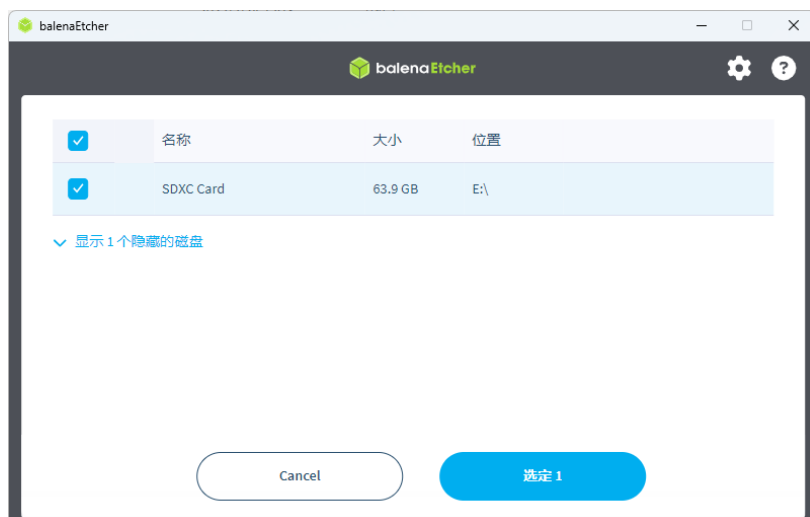


Figure 34.: 选择磁盘

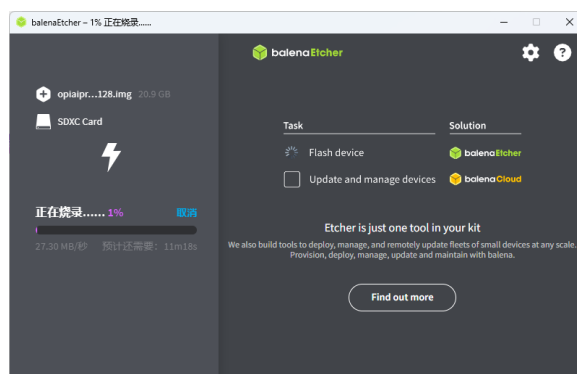


Figure 35.: 烧录过程

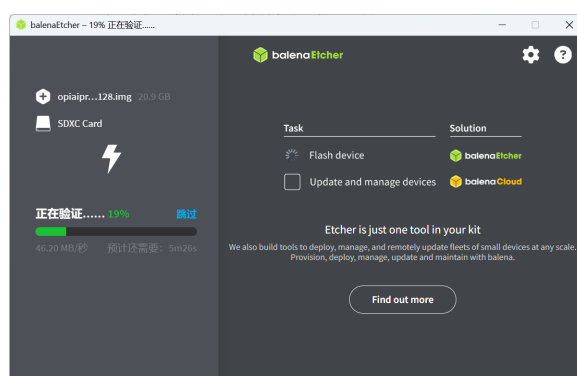


Figure 36.: 校验过程

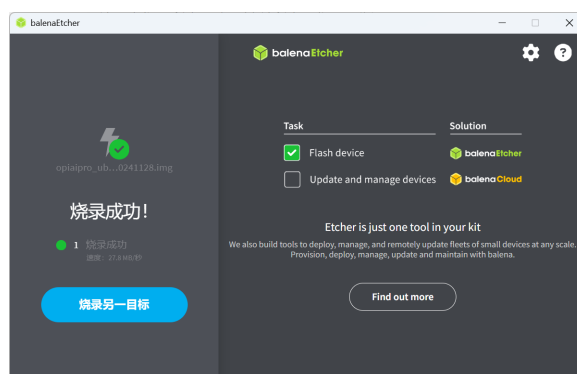


Figure 37.: 完毕

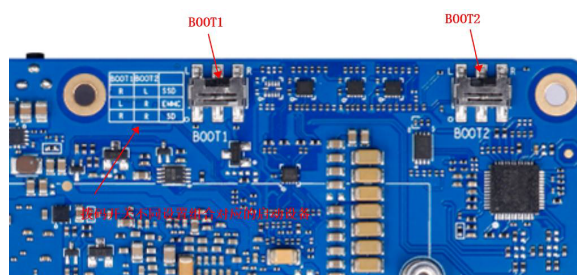


Figure 38.: boot 开关

其他存储介质说明

- **eMMC**: 板载无 eMMC, 需购买专用模块。刷写方法请参考香橙派用户手册。
- **SSD**: 支持 M.2 SSD 启动, 但兼容性有限 (仅支持特定品牌型号), 且需自行准备。初学者不推荐直接使用 SSD 作为系统盘。

设置启动模式

开发板支持从 TF 卡、eMMC 或 M.2 SSD 启动。当连接了多种存储设备时, 需通过背面的拨码开关 (BOOT 开关) 指定启动设备。

拨码开关状态说明 (ON 方向为 1/右, 相反为 0/左, 具体请参考板上丝印或下表):

Boot1	Boot2	启动设备
左	左	(未使用)
右	右	TF 卡
左	右	eMMC
右	左	M.2 SSD

注意: 切换拨码开关后, 必须**完全断电** (拔掉电源线) 再重新上电, 新的启动配置才会生

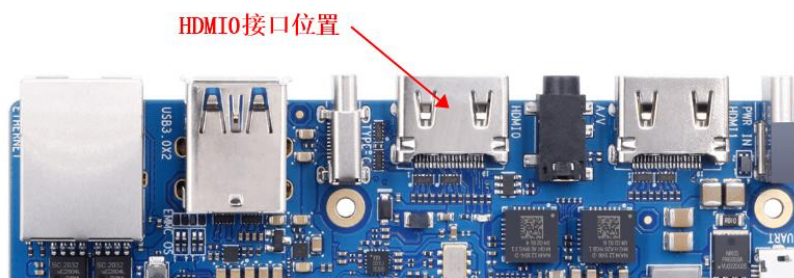


Figure 39.: HDMI0

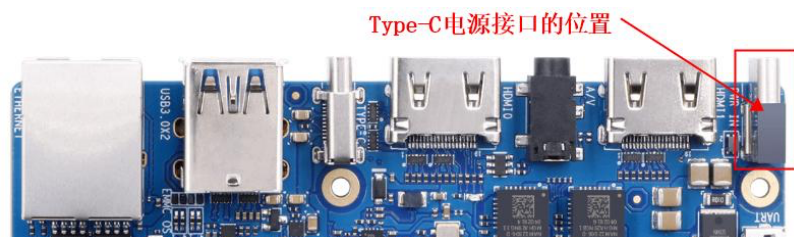


Figure 40.: TYPE-C Power

效。仅按 RESET 键重启无效。

0.4. 启动开发板

0.4.1. 方式一：图形化界面启动

1. 硬件连接：

- 将刷写好的 TF 卡插入开发板插槽。
- 确认拨码开关均拨至右侧（TF 卡启动模式）。
- 将 HDMI 线连接至 **HDMI0** 接口（靠近 USB 3.0 接口的那个）。
- 连接鼠标和键盘。
- 最后，接入 Type-C 电源。

2. 系统启动：

- 上电后，风扇会全速旋转，随后声音变小，屏幕显示启动画面。
- 稍候进入登录界面。

3. 登录系统：

- 默认普通用户：HwHiAiUser，密码：Mind@123
- 默认 Root 用户：root，密码：Mind@123



Figure 41.: 登录



Figure 42.: 桌面

输入密码登录进入桌面环境。

若无法登陆请检查输入的密码是否正确，大小写以及符号是否正确

默认账户表格：| 用户名 | 密码 | | :—: | :—: | | root | Mind@123 | | HwHiAiUser | Mind@123 |

0.4.2. 串口界面

1. 使用 USB2TTL 模块，与开发板的 GPIO 口进行连线

开发板的 TX (GPIO8) 接入 USB2TTL 模块的 RX 接口，开发板的 RX (GPIO10) 则接入模块的 TX 接口，并连接好 GND 接地，在 Windows 电脑下可以使用 PUTTY 连接串口。

2. 使用开发板自带的 Micro USB 接口进行串口调试，该方法更为方便，只需要一根 Micro USB 数据线，接入电脑后打开设备管理器查询对应的串口，然后使用 PUTTY 进行链接

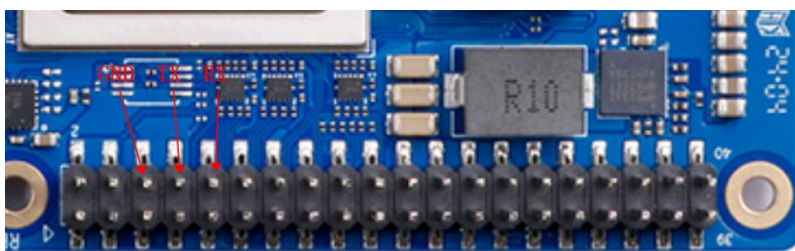


Figure 43.: 开发板串口

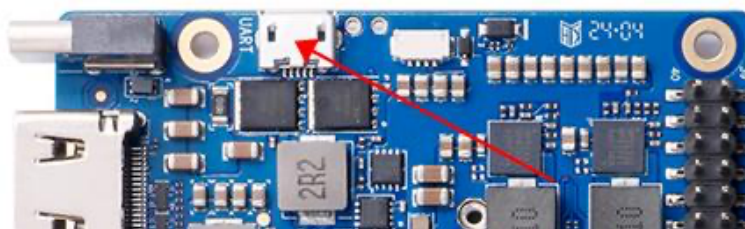


Figure 44.: MicroUSB 串口

即可。

以 Micro USB 接口为例：1. 使用 **Micro USB** 数据线连接开发板和电脑，此时请不要给开发板上电。2. 打开电脑的设备管理器，选择端口，寻找开发板对应的串口端口号

3. 打开串口调试软件（**PUTTY**）

将 Connection Type 选择为 Serial, 然后在 Serial Line 处将端口号修改为设备管理器中查到的端口号, 如作者此处端口号为 COM3, 此外, 还需要将 Speed 从 9600 修改为 115200, 最后点击 Open 打开串口。

4. 给开发板上电, 等待出现 **Ubuntu 22.04.3 LTS orangepiaipro ttyAM0** 字样, 输入登录的用户名 **HwHiAiUser** 并回车, 然后输入密码 **Mind@123** 并回车, 注意在输入密码的时候屏幕并不会显示任何东西, 登陆后的界面如图所示。

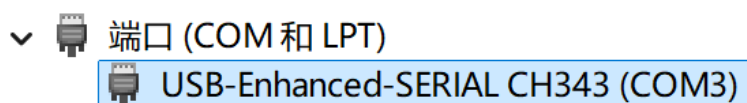


Figure 45.: 端口号

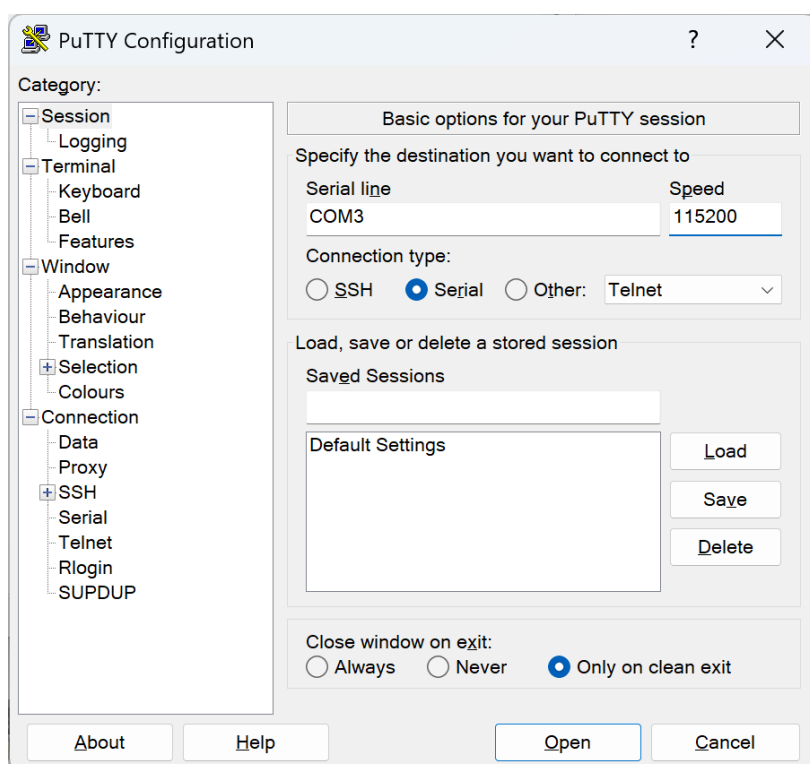


Figure 46.: PUTTY

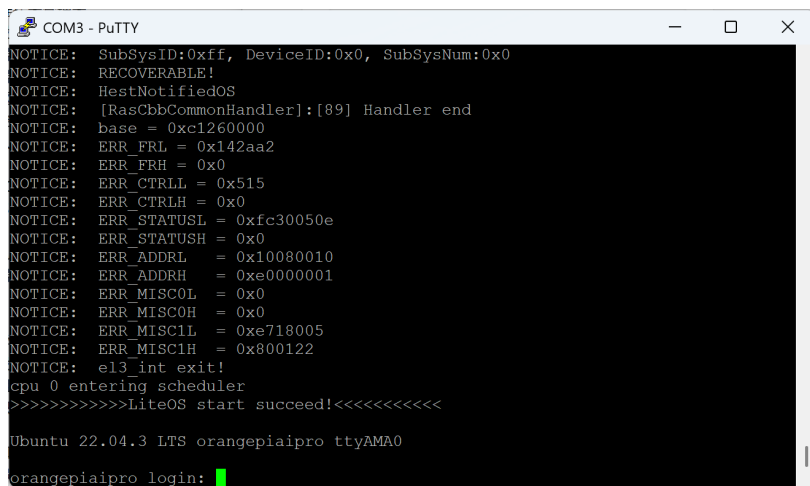


Figure 47.: 串口

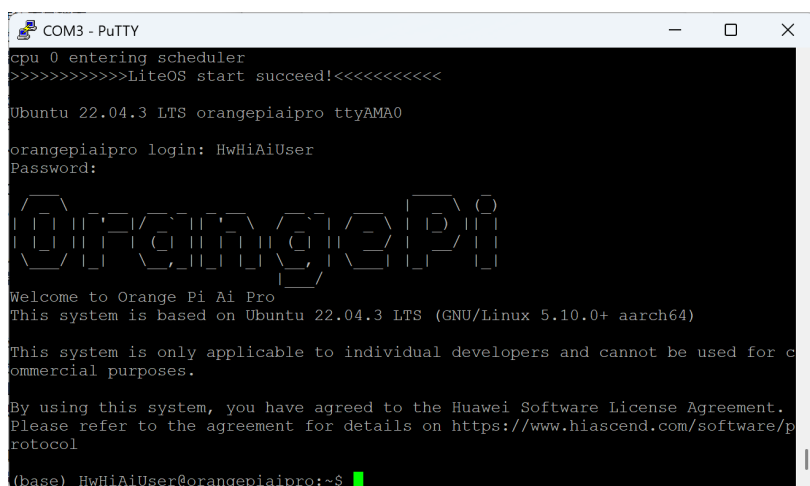


Figure 48.: 登录成功

0.5. 网络连接

0.5.1. 无线网络连接 (WiFi)

开发板板载了 WiFi 模块，可以通过命令行工具 `nmcli` 轻松连接无线网络。

1. 扫描 WiFi

```
1 nmcli dev wifi list
```

2. 连接 WiFi 将<SSID>替换为你的 WiFi 名称, <PASSWORD>替换为密码。

```
1 nmcli dev wifi connect <SSID> password <PASSWORD>
```

3. 查看连接状态

```
1 nmcli connection show
```

0.5.2. 有线网络连接

如果有线网络可用，直接插入网线即可。可以通过以下命令查看 IP 地址：

```
1 ip addr
```

或者

```
1 ifconfig
```

0.6. 远程连接 (SSH)

为了方便开发，通常会使用个人电脑通过 SSH 远程连接到开发板。

1. **获取开发板 IP 地址**使用上述`ip addr`命令获取开发板的 IP 地址（通常在`wlan0`或`eth0`接口下）。
2. **使用 SSH 客户端连接**在你的个人电脑终端（Windows 可以使用 CMD、PowerShell 或 Putty，Mac/Linux 使用 Terminal）中输入：

```
1 ssh HwHiAiUser@<开发板IP地址>
```

例如：

```
1 ssh HwHiAiUser@192.168.1.100
```

默认密码为：Mind@123

0.7. 验证开发环境

系统启动并连接网络后，我们需要验证昇腾 AI 处理器的状态以及开发环境是否正常。

1. **查看 NPU 状态**使用`npu-smi`工具查看 NPU 的详细信息，包括温度、功耗、算力利用率等。

```
1 npu-smi info
```

如果能看到类似以下的输出，说明 NPU 工作正常：

```
1 +-----+
2 | npu-smi 23.0.0                               Version: 23.0.0
3 |-----+
4 | NPU      Name                               Health   Power(W)   Temp(C)
5 | Chip     Device                               Bus-Id    AICore(%)  Memory-Usage(
6 |=====|
7 | 0        310B4                               OK        12.8       45
8 | 0        0                                   0000:00:00.0 0      2433 /
9 |=====|
10 | 0        7564                               |
```

2. **检查 CANN 环境**官方镜像通常已预装 CANN（Compute Architecture for Neural Networks）。可以通过检查环境变量或运行简单命令来确认。通常环境变量设置在`.bashrc`中，尝试执行：

```
1 echo $ASCEND_HOME_PATH
```

或者检查编译器版本：

```
1 c++ --version
```

0.7.1. 设置 SWAP 交换分区

开发板虽然有 8G/16G 的运存，但是有些应用（例如 ATC 模型转换工具）需要较大的内存，在这种情况下我们可以通过设置 SWAP 交换分区来扩展系统能使用的最大内存容量。

1. 创建交换文件

首先，使用 `fallocate` 命令创建一个指定大小的文件（例如 12GB）。如果你的存储空间允许，建议设置得大一些，以防模型转换时内存不足。

```
1 sudo fallocate -l 8G /swapfile
```

如果提示 `fallocate` 失败，可以使用 `dd` 命令：

```
1 sudo dd if=/dev/zero of=/swapfile bs=1G count=8
```

2. 设置权限

出于安全考虑，需要修改文件的权限，仅允许 `root` 用户读写。

```
1 sudo chmod 600 /swapfile
```

3. 设置交换区

将文件格式化为交换分区格式。

```
1 sudo mkswap /swapfile
```

4. 启用交换区

启用该交换文件。

```
1 sudo swapon /swapfile
```

5. 持久化设置

为了防止重启后失效，需要将其写入 `/etc/fstab` 文件中。

```
1 sudo nano /etc/fstab
```

在文件末尾添加以下内容：

```
1 /swapfile none swap sw 0 0
```

保存并退出（`Ctrl+O`, `Enter`, `Ctrl+X`）。

0.8. 结语

至此，你的昇腾 310B 开发板（OrangePi AiPro）已经完成了基本的环境搭建。接下来，你可以进入[案例 1：智能打卡机](#)的学习，开始你的第一个 AI 应用开发之旅。

1. 案例 1：智能人脸识别考勤系统

1.1. 教程定位

本案例旨在利用昇腾 310B 的强大 AI 算力，构建一个功能完整、响应迅速的智能人脸识别打卡系统。系统通过 USB 摄像头实时捕捉视频流，检测画面中的人脸，并与预先注册的员工人脸数据库进行比对，完成身份验证和自动记录考勤。

本案例的设计初衷是为开发者提供一个端到端的边缘 AI 应用范例，涵盖了以下核心知识点：

1. **高性能推理加速**：利用昇腾 PyACL 接口直接驱动 NPU, 实现 RetinaFace 检测与 ArcFace 识别的硬件加速。
2. **端到端流水线**：建立从“视频流采集 -> 人脸检测 -> 人脸对齐 -> 特征提取 -> 特征比对 -> 数据库存取”的完整业务链路。
3. **嵌入式 Web 交互**：通过 Flask 框架构建轻量级管理后端，支持远程监控、用户注册与考勤导出。
4. **边缘存储实践**：使用 SQLite 数据库在本地高效存储用户特征向量（Embedding）与考勤记录。

从整体结构看，本案例对应一条标准的生物特征识别流水线：

本案例选择的实现路线是：* 使用 **RetinaFace** 完成鲁棒的人脸检测。* 使用 **ArcFace** 完成高精度的特征提取。* 使用 **SQLite** 进行本地数据持久化。* 使用 **Python Flask** 提供现代化的 Web 交互界面。

1.2. 实验硬件与运行条件

为了完成实时人脸识别与考勤实验，建议准备如下硬件条件：

- 昇腾 310B 开发者套件 (如 OrangePi A1pro)
- USB 摄像头 (支持 640x480 及以上分辨率)
- 显示设备 (可选): HDMI 显示器或通过 Web 远程访问
- CANN 软件栈: 7.0 或更高版本
- Python 环境: 3.9 或更高版本, 需安装 flask, opencv-python, numpy<2.0, sqlite3 等

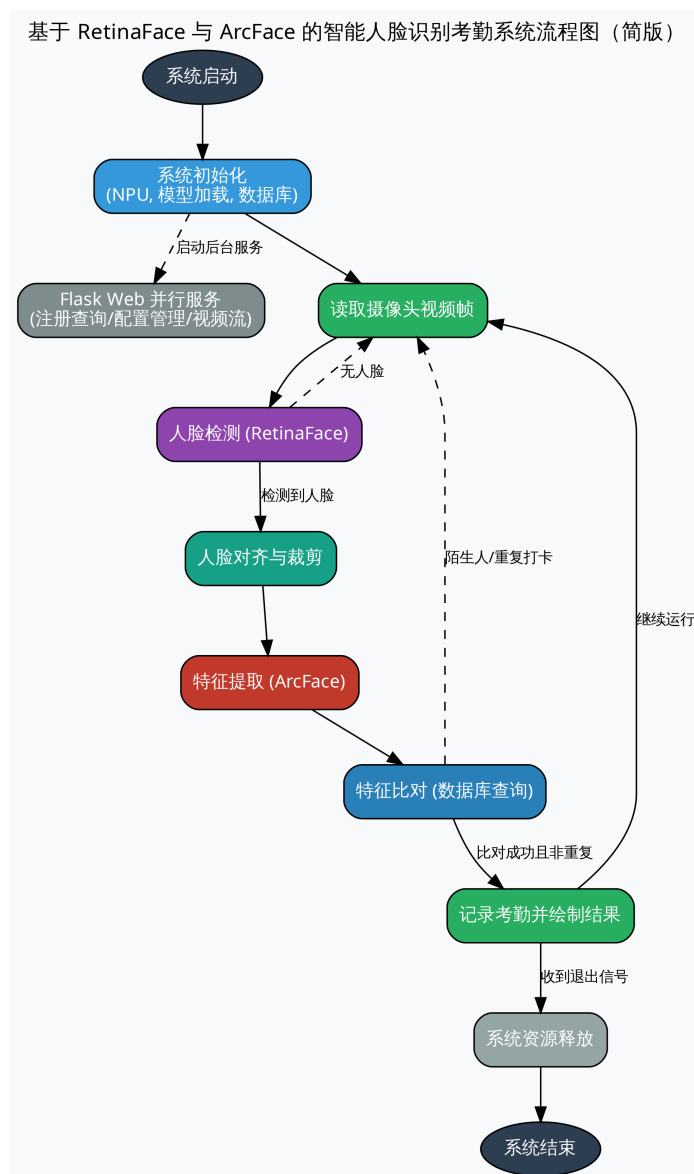


Figure 1.1.: 程序流程图

运行方式：* **本地模式**：直接运行 `python3 app.py`，后台线程会自动处理本地 USB 摄像头。
* **Web 模式**：通过浏览器访问 `http://<开发板IP>:5000` 进行管理和手动打卡。

1.3. 本案例支持的模型结构

当前仓库里的模型基于 InsightFace 提供的轻量化版本，经过 `atc` 工具转换为 `.om` 格式：

- **检测模型 (RetinaFace)**: `face_detection.om`
 - 输入尺寸: 640 x 640
 - 输出: 多尺度分数 (Scores) 与边界框偏移量 (BBoxes)
- **识别模型 (ArcFace)**: `face_recognition.om`
 - 输入尺寸: 112 x 112 (对齐后的人脸切片)
 - 输出: 512 维特征向量 (Embedding)

这两类模型共同构成了“检测-对齐-识别”的经典人脸分析链路。

1.4. 人脸识别系统的核心流水线

人脸识别不仅仅是“看一眼”，它包含了一系列精细的图像处理步骤：

1. **检测 (Detection)**：在复杂的背景中定位人脸的具体位置。
2. **对齐与裁剪 (Alignment & Cropping)**：根据检测到的关键点，将人脸旋转至正位并裁剪为固定尺寸（如 112x112），以消除姿态影响。
3. **特征提取 (Feature Extraction)**：将图像编码为一个具有高度代表性的固定长度向量（512 维）。
4. **比对 (Matching)**：计算当前向量与数据库中已知向量的相似度（如余弦相似度）。

1.5. RetinaFace 深度剖析：从“看见”到“看懂”人脸

RetinaFace 不仅仅是一个检测人脸框的工具，它是一个精密的、为解决现实世界中各种复杂人脸检测问题而设计的深度学习模型。要理解它的价值，我们需要先了解它的历史背景和设计哲学。

1.5.1. RetinaFace 的发展历史与作者信息

RetinaFace 模型由 **InsightFace** 团队发布，该团队在人脸识别领域享有盛誉，其开源的 `insightface` 代码库已成为学术界和工业界广泛使用的基准。RetinaFace 的提出，正是在单阶段目标检

测器（如 SSD、YOLO）性能大幅提升，但通用检测器在人脸这种特定任务上仍有优化空间的背景下诞生的。

它的设计目标非常明确：创建一个在速度和精度上都达到顶尖水平，并且能同时处理人脸框、关键点乃至 3D 姿态的**一体化解决方案**。它借鉴了通用目标检测器 **RetinaNet** 的核心思想，并针对人脸任务的特性进行了深度定制。

1.5.2. RetinaFace 到底在解决什么问题？

在真实场景中，“找到一张脸”远比想象的复杂。RetinaFace 主要致力于解决以下几个核心挑战：

1. **尺度变化巨大**：从合影中的微小人脸到监控下的特写人脸，尺寸差异可能达到上百倍。
2. **姿态与遮挡**：侧脸、低头、被口罩或手部分遮挡的人脸，都需要被准确识别。
3. **密集场景**：在人群中，大量人脸紧密排列，容易产生漏检或错误的合并检测。
4. **实时性要求**：在边缘设备上，检测速度必须足够快，才能支撑实时应用。

1.5.3. 核心技术解析

RetinaFace 的强大性能源于其对多个关键技术的巧妙融合。

1. 单阶段检测器与多任务学习

与需要先生成候选区域再进行分类的两阶段检测器（如 Faster R-CNN）不同，RetinaFace 是一个**单阶段（One-Stage）检测器**。它直接在特征图上预测目标的位置和类别，速度更快。更重要的是，它采用**多任务学习（Multi-task Learning）**框架，让一个网络同时完成多项任务：

- **人脸分类**：判断某个区域是人脸还是背景。
- **边界框回归**：精调人脸框的位置。
- **关键点定位**：预测眼睛、鼻子、嘴角等 5 个关键点的位置。

这种设计不仅提升了效率，而且关键点定位任务的加入，反过来为模型提供了更丰富的监督信息，有助于更准确地识别人脸区域，尤其是对于部分遮挡的人脸。

2. 特征金字塔网络 (FPN)

为了解决尺度变化问题，RetinaFace 采用了**特征金字塔网络（Feature Pyramid Network, FPN）**。FPN 的思想是“高层特征看语义，底层特征看细节”。它通过自顶向下的路径和横向连接，将高层特征图中丰富的语义信息与底层特征图中高分辨率的细节信息相融合，从而在多个不同尺度的特征图上进行预测。这意味着，大的特征图负责检测小人脸，小的特征图负责检测大人脸，实现了对各种尺寸人脸的“全覆盖”。

3. 上下文感知模块 (SSH)

为了进一步增强特征的表达能力，RetinaFace 在 FPN 的每个预测层后都引入了 **SSH (Single Stage Headless)** 模块。SSH 模块通过并行的、具有不同大小卷积核的通路来提取特征，并将它们拼接在一起。这就像给模型装上了“广角镜”和“长焦镜”，使其能够同时关注一个区域的局部细节和它周围的上下文信息，从而在复杂背景或人脸密集的情况下做出更鲁棒的判断。

4. 焦点损失 (Focal Loss) 的启示

虽然 RetinaFace 的论文没有将焦点损失 (Focal Loss) 作为其核心创新点，但这个概念对于理解所有现代单阶段检测器至关重要。在人脸检测中，一张图像里绝大部分区域都是背景（负样本），只有极少数区域是人脸（正样本）。这种**正负样本的极端不平衡**会导致模型训练时被大量简单的背景样本主导，而忽略了对困难人脸样本的学习。

Focal Loss 通过一个动态缩放因子，降低了大量简单负样本在损失计算中的权重，使得模型能够更专注于学习那些难以区分的正样本和困难负样本。RetinaFace 正是受益于这种思想，才能在复杂的背景中精准地“聚焦”于人脸。

1.5.4. 在本案例中的应用

在我们的智能考勤系统中，RetinaFace 作为整个识别流程的“眼睛”，其作用至关重要：

- **提供高质量输入：**它为后续的 ArcFace 识别模块提供了准确的人脸边界框和关键点。只有检测得准，后续的对齐和识别才有可能成功。
- **保证实时性能：**通过在昇腾 310B NPU 上的高效推理，它确保了系统能够对视频流进行逐帧处理而不会出现明显卡顿，这是实现“自动打卡”功能的基础。
- **鲁棒性保障：**得益于其先进的设计，即使在光线不佳、人员走动或部分遮挡的情况下，系统依然能维持较高的检测成功率。

1.6. ArcFace 深度剖析：让特征在角度空间中更具区分度

ArcFace 是人脸识别流程的“大脑”，负责将检测到的人脸图像转化为一个高度浓缩且易于比较的 512 维特征向量 (Embedding)。它的成功并非偶然，而是建立在人脸识别损失函数长期演进的基础之上。

1.6.1. 从 Softmax 到度量学习：损失函数的演进之路

在深度学习的早期，人脸识别通常被当作一个多分类问题来处理。例如，在一个包含 1000 个人的数据集中，模型会尝试将输入的人脸图像正确分类到这 1000 个类别中的某一个。这个过程通常使用 **Softmax Loss**。

- **Softmax Loss 的局限**：Softmax 的目标是让不同类别能够被分开，但它并不强制要求“同一类”的样本在特征空间中靠得足够近，也未要求“不同类”的样本离得足够远。这导致在面对从未见过的新人脸（开集识别）时，模型泛化能力不足。

为了解决这个问题，研究者们开始探索**度量学习（Metric Learning）**的思想，其核心目标是直接在特征空间中优化距离：最大化类间距离，最小化类内距离。

1. SphereFace (A-Softmax)

SphereFace 首次提出，在计算角度时应该引入一个**角度间隔（Angular Margin）**。它将传统的 Softmax 中的权重与特征的点积 $W^T x$ 修改为 $\|W\| \|x\| \cos(\theta)$ ，并通过对权重 W 的归一化，将特征学习过程约束在一个单位超球面上。然后，它在目标类别的角度 θ_y 上乘以一个整数 m ，变成了 $\cos(m\theta_y)$ 。这样一来，分类的决策边界从简单的线性边界变成了角度边界，强制模型学习到角度区分度更强的特征。

2. CosFace (LMCL)

CosFace 认为 SphereFace 的乘性间隔 $m\theta_y$ 会导致训练不稳定。因此，它提出了一种更简单、更稳定的**加性余弦间隔（Additive Cosine Margin）**。它直接在余弦空间中减去一个常数 m ，将决策边界从 $\cos(\theta_y)$ 变为 $\cos(\theta_y) - m$ 。这种方式在实现上更简单，训练过程也更平滑。

1.6.2. ArcFace 的核心创新：加性角度间隔

ArcFace (Additive Angular Margin Loss) 结合了前两者的优点，并做出了关键性的改进。它的作者同样来自 **InsightFace** 团队，这保证了从检测到识别的算法思想一脉相承。

ArcFace 认为，无论是在余弦空间还是角度空间进行乘性操作，都有些间接。最直接、最有效的方式，应该是在**角度空间中直接增加一个加性间隔（Additive Angular Margin）**。

它的数学形式是将决策边界从 $\cos(\theta_y)$ 修改为 $\cos(\theta_y + m)$ 。

这个看似微小的改动，却带来了巨大的优势：

- **几何意义明确**：直接在超球面上的测地距离（Geodesic Distance）上增加了一个恒定的角度惩罚项 m 。这意味着，模型不仅要正确识别人脸，还必须保证该人脸的特征向量与对应类中心的夹角，要比与其他所有类中心的夹角**至少小 m 度**。这个要求非常严格，极大地提升了特征的区分度。
- **训练更稳定**：相比 SphereFace 的乘性间隔，加性间隔的设计更加稳定，更容易收敛。
- **性能卓越**：ArcFace 在几乎所有主流人脸识别评测基准上都取得了当时的最佳性能，证明了其设计的有效性。

上图直观地展示了不同损失函数下的决策边界。可以看到，ArcFace 的决策边界（绿色虚线）相比其他损失函数，对类内样本的约束更强，类间间隔也更大。

1.6.3. 在本案例中的应用

在本考勤系统中，ArcFace 的作用是赋予系统“认识”人的能力：

1. **生成高质量 Embedding**：对于每一张注册的人脸，ArcFace 都会生成一个 512 维的特征向量。这个向量就是这张脸在数学空间中的“身份证”。
2. **实现精准比对**：当一个新的人脸被检测到后，系统同样会提取其特征向量。通过计算这个新向量与数据库中所有已注册向量的**余弦相似度**，我们可以快速找到最相似的用户。
3. **高可靠性**：由于 ArcFace 学习到的特征具有极强的类内紧凑性和类间分离性，因此即使在光照变化、表情变化甚至轻微遮挡的情况下，识别的准确率和召回率依然非常高，误识率极低。这对于一个严肃的考勤应用至关重要。

1.7. 数据存储与检索深度剖析：边缘场景下的智慧与权衡

如果说模型是系统的大脑，那么数据存储就是系统的记忆。本案例采用 SQLite 存储用户特征和考勤记录，这个选择背后体现了边缘计算场景下的典型工程权衡。

1.7.1. 为什么选择 SQLite?

在动辄使用 MySQL、PostgreSQL 或云数据库的时代，选择 SQLite 似乎有些“复古”。但在昇腾 310B 这样的边缘设备上，它却是最合适的选择之一：

1. **轻量级与零配置**：SQLite 是一个库，而不是一个独立的服务器进程。它直接将整个数据库存储为一个单一的文件（attendance.db），无需安装、无需配置、无需管理用户权限，极大地降低了部署和维护的复杂度。
2. **资源占用极低**：它对内存和 CPU 的占用非常小，这对于资源本就紧张的边缘设备至关重要，可以确保更多的计算资源留给核心的 AI 推理任务。
3. **事务支持**：尽管轻量，SQLite 依然提供了完整的 ACID 事务支持，保证了数据操作的原子性、一致性、隔离性和持久性，确保了考勤数据不会因为意外中断而损坏。
4. **Python 内置支持**：Python 的标准库 sqlite3 直接提供了对 SQLite 的支持，无需安装任何额外的驱动包，进一步简化了开发。

1.7.2. 特征向量的存储：BLOB 格式的优与劣

本案例将 512 维的人脸特征向量（一个包含 512 个浮点数的 numpy 数组）直接转换为二进制数据，并存储在数据库的 **BLOB (Binary Large Object)** 类型字段中。

```
1 # In database.py
2 def add_user(name, embedding):
3     # embedding 是一个 numpy 数组
4     conn = sqlite3.connect(DB_NAME)
```

```

5 c = conn.cursor()
6 # 将 numpy 数组转换为二进制格式存入 BLOB 字段
7 c.execute('INSERT INTO users (name, embedding) VALUES (?, ?)', (name,
8 # ...
    ↪ embedding))

```

这种做法的**优点**非常明显：

- **简单高效**：直接存储原始二进制数据，无需进行任何格式转换，读写速度快。
- **精度无损**：避免了将浮点数转为字符串等可能引入精度损失的操作。

但它的**缺点**也同样突出：

- **数据库“不理解”数据**：对于 SQLite 来说，BLOB 字段里存的只是一堆无意义的二进制数据。它无法理解这是一个 512 维的向量，因此也无法在数据库层面进行任何向量相关的运算，比如计算距离或相似度。

1.7.3. 当前的检索方案：简单遍历

正是由于数据库无法直接操作向量，本案例的识别流程采用了最简单直接的**暴力检索（Brute-force Search）**方案：

1. 当一个新的脸出现时，计算其特征向量 V_{curr} 。
2. 从数据库中**取出所有**已注册用户的特征向量，加载到内存中形成一个列表 $[V_1, V_2, \dots, V_n]$ 。
3. 在内存中，使用 `numpy` 逐一计算 V_{curr} 与列表中每个向量的余弦相似度。
4. 找到相似度最高且超过阈值的用户作为识别结果。

这个方案在用户规模较小（如几十人或几百人）时是完全可行的，因为现代 CPU 进行几百次 512 维向量的点积运算耗时极短，远低于视频帧率的间隔。

1.7.4. 性能瓶颈与未来展望

然而，当用户规模从几百人扩大到几千人、几万人甚至更多时，上述暴力检索方案的性能瓶颈会立刻显现。每次识别都需要遍历整个数据库，计算量会随着用户数量线性增长，最终导致识别延迟变得无法接受。

为了解决大规模向量检索的问题，工业界发展出了一系列**近似最近邻（Approximate Nearest Neighbor, ANN）**搜索技术和专门的**向量数据库**。其核心思想是：不再保证 100% 找到最相似的向量，而是通过构建特殊的索引结构（如树、图、哈希等），在牺牲极少量精度的前提下，将检索速度提升几个数量级。

如果本系统需要向上扩展，可以考虑以下演进路径：

- **集成 ANN 库**：在应用层面集成如 **Faiss**（由 Facebook AI 开发）或 **ScaNN**（由 Google 开发）这样的高性能向量检索库。数据仍然可以存储在 SQLite 中，但在系统启动时，将

所有向量加载到内存中构建 Faiss 索引，后续检索直接通过该索引进行。

- **迁移到向量数据库：**当数据规模和并发请求进一步增大时，可以考虑使用专门的向量数据库，如 **Milvus** 或 **Weaviate**。这些系统不仅内置了高效的 ANN 索引算法，还提供了完整的分布式、高可用的数据管理方案，是处理海量向量数据的终极解决方案。

因此，本案例采用的 SQLite + BLOB + 暴力检索的方案，是在边缘计算资源受限、用户规模可控的特定场景下的最佳实践。它体现了“简单、有效、满足当前需求”的工程设计原则，同时也为未来的系统扩展留下了清晰的演进路径。

1.8. PyACL 深度剖析：与昇腾 NPU 对话的艺术

`ascend_inference.py` 是本案例与昇腾 NPU 硬件沟通的桥梁。它通过 `pyacl` 库，将上层的 Python 指令转化为底层硬件可以理解的指令。理解其工作流程，是掌握昇腾平台开发的关键。

1.8.1. 什么是 ACL？为什么需要它？

在深入代码之前，我们先理解一个基本问题：为什么不能直接用 Python 调用 NPU？

想象一下，NPU 就像一个只会说“机器语言”的超级计算专家，而我们的 Python 程序说的是“人类语言”。它们之间需要一个翻译官，这个翻译官就是 **ACL (Ascend Computing Language, 昇腾计算语言)**。

ACL 提供了一套标准化的 API 接口，让我们可以用相对高级的语言（如 Python、C++）来：

- 管理 NPU 硬件资源（设备、内存、计算队列）
- 加载和运行 AI 模型
- 在 CPU 和 NPU 之间传输数据
- 监控和调试推理过程

可以把 ACL 理解为 NPU 的“操作系统接口”，就像我们通过操作系统 API 来使用 CPU 和内存一样。

1.8.2. 昇腾计算语言 (ACL) 的标准 workflow

在昇腾平台上进行一次完整的模型推理，通常遵循一个非常标准且结构化的流程。这个流程可以被概括为“初始化-执行-释放”三大阶段。

让我们用一个生活化的比喻来理解整个流程：把 NPU 推理想象成去餐厅吃饭。

阶段一：资源初始化（进入餐厅，找座位）

1. **ACL 初始化 (`acl.init`):** 这是与硬件沟通的第一步。它负责加载底层驱动，建立与硬件的连接。一个进程中只需调用一次。

*比喻：*这就像推开餐厅的大门，告诉服务员“我来了”。只需要在进门时做一次。

2. **设备与上下文管理 (`acl.rt.set_device`, `acl.rt.create_context`):**

- `set_device`: 指定我们要使用哪一块 NPU 设备（对于只有一个 NPU 的开发板，通常是设备 0）。
- `create_context`: 为当前线程创建一个独立的执行上下文。这个上下文管理着队列、事件和内存等所有与该线程相关的资源，确保多线程环境下的资源隔离。

比喻：`set_device` 就像选择在哪个餐厅用餐（如果你有多家连锁店可选）。`create_context` 就像服务员给你安排了一个专属的座位和餐具，这些资源只属于你这一桌，不会和其他客人混淆。

在 `ascend_inference.py` 的 `AscendSystem` 类中，这个过程被封装在 `_init_resource` 方法里，确保了系统启动时资源的正确准备。

```

1 # In ascend_inference.py -> AscendSystem
2 class AscendSystem:
3     def __init__(self, device_id=0):
4         # ...
5         self._init_resource()
6
7     def _init_resource(self):
8         ret = acl.init() # 1. ACL 初始化 - 推开餐厅大门
9         ret = acl.rt.set_device(self.device_id) # 2. 设置设备 - 选择餐厅
10        self.context, ret = acl.rt.create_context(self.device_id) # 2. 创建上下
11           ↪ 文 - 分配座位
12        self.stream, ret = acl.rt.create_stream() # 创建计算流 - 叫来服务员
13        # ...

```

3. **创建 Stream (计算流)**: Stream 是一个任务队列，所有提交给 NPU 的计算任务都会按顺序在这个队列中执行。

比喻：这就像餐厅的服务员，你点的菜会按顺序送到厨房，厨房按顺序做菜。

阶段二：模型执行（点菜、上菜、吃饭）

3. **模型加载 (`acl.mdl.load_from_file`)**: 将编译好的 `.om` 离线模型从硬盘加载到 NPU 的内存中。加载后，我们会得到一个模型 ID，后续所有操作都通过这个 ID 来引用该模型。比喻：这就像把菜单（模型文件）交给厨房，厨房记住了你要的菜（模型 ID），之后你只需要说“上第 3 号菜”就行了。

```

1 # In ascend_inference.py -> AscendModel
2 def _load_model(self):
3     self.model_id, ret = acl.mdl.load_from_file(self.model_path)
4     # 获取模型描述信息（输入输出的形状、大小等）
5     self.desc = acl.mdl.create_desc()
6     ret = acl.mdl.get_desc(self.desc, self.model_id)

```

4. **内存分配与数据准备**: 这是最核心也最容易出错的环节。

比喻：在点菜之前，餐厅需要准备好盘子（内存空间）。厨房有自己的盘子（Device 内存），你手里也有食材（Host 内存的数据），需要把食材放到厨房的盘子里才能开始做菜。

- **Device 内存:** 模型推理是在 NPU (Device) 上完成的, 因此输入数据和输出结果都需要存放在 Device 内存中。我们使用 `acl.rt.malloc` 来分配 Device 内存。
- **Host 内存:** 我们的原始图像数据 (如用 OpenCV 读取的 numpy 数组) 存放在 CPU 管理的内存 (Host) 中。
- **数据传输:** 在推理前, 必须使用 `acl.rt.memcpy` 将 Host 上的输入数据拷贝到 Device 上的指定内存区域。这个过程被称为 **H2D (Host to Device)** 拷贝。

```

1 # In ascend_inference.py -> AscendModel._init_buffers
2 def _init_buffers(self):
3     # 为每个输入分配 Device 内存
4     num_inputs = acl.mdl.get_num_inputs(self.desc)
5     for i in range(num_inputs):
6         size = acl.mdl.get_input_size_by_index(self.desc, i)
7         dev_ptr, ret = acl.rt.malloc(size, 2) # 在 Device 上分配内存
8         self.input_buffers.append({"ptr": dev_ptr, "size": size})
9
10    # 为每个输出分配 Device 内存
11    num_outputs = acl.mdl.get_num_outputs(self.desc)
12    for i in range(num_outputs):
13        size = acl.mdl.get_output_size_by_index(self.desc, i)
14        dev_ptr, ret = acl.rt.malloc(size, 2)
15        self.output_buffers.append({"ptr": dev_ptr, "size": size})

```

5. **模型执行 (`acl.mdl.execute`):** 将准备好的输入数据 (存放于 Device 内存) 送入已加载的模型中进行推理。

比喻: 食材已经放到厨房的盘子里了, 现在厨师开始做菜 (NPU 开始计算)。

```

1 # In ascend_inference.py -> AscendModel.execute
2 def execute(self, input_data_list):
3     # 步骤 1: H2D - 把数据从 CPU 内存拷贝到 NPU 内存
4     for i, data in enumerate(input_data_list):
5         data = np.ascontiguousarray(data) # 确保数据在内存中连续
6         ptr = acl.util.numpy_to_ptr(data) # 获取 numpy 数组的内存指针
7         ret = acl.rt.memcpy(
8             self.input_buffers[i]["ptr"], # 目标: Device 内存
9             self.input_buffers[i]["size"],
10            ptr, # 源: Host 内存
11            data.nbytes,
12            1 # 模式 1 = Host to Device
13        )
14
15    # 步骤 2: 执行模型推理
16    ret = acl.mdl.execute(self.model_id, self.input_dataset, self.
        ↪ output_dataset)

```

6. **结果获取:** 推理完成后, 结果仍然存放在 Device 内存中。我们需要再次使用 `acl.rt.memcpy` 将其从 Device 拷贝回 Host 内存, 这个过程被称为 **D2H (Device to Host)** 拷贝。

比喻: 菜做好了, 但还在厨房的盘子里, 服务员需要把菜端到你的桌子上 (从 Device 拷贝回 Host)。

```

1  # 步骤 3: D2H - 把结果从 NPU 内存拷贝回 CPU 内存
2  outputs = []
3  for i in range(len(self.output_buffers)):
4      size = self.output_buffers[i]["size"]
5      host_data = np.zeros(size, dtype=np.byte) # 在 Host 上分配接收空
        ↪ 间
6      host_ptr = acl.util.numpy_to_ptr(host_data)
7      ret = acl.rt.memcpy(
8          host_ptr, # 目标: Host 内存
9          size,
10         self.output_buffers[i]["ptr"], # 源: Device 内存
11         size,
12         2 # 模式 2 = Device to Host
13     )
14     outputs.append(host_data)
15     return outputs

```

阶段三：资源释放（吃完饭，结账离开）

7. 资源清理: 为了避免内存泄漏，所有申请的资源在使用完毕后都必须被显式释放。这包括模型、Device 内存、上下文、流等。最后调用 `acl.finalize` 来断开与硬件的连接。

比喻：吃完饭要结账、归还餐具、离开餐厅。如果不做清理，内存就会像没还的餐具一样越积越多，最终耗尽系统资源。

```

1  # In ascend_inference.py -> AscendModel.release
2  def release(self):
3      # 释放输入输出的 Device 内存
4      for buf in self.input_buffers:
5          acl.rt.free(buf["ptr"])
6      for buf in self.output_buffers:
7          acl.rt.free(buf["ptr"])
8
9      # 卸载模型
10     if self.model_id:
11         acl.mdl.unload(self.model_id)
12
13     # 销毁数据集和描述符
14     if self.input_dataset:
15         acl.mdl.destroy_dataset(self.input_dataset)
16     if self.output_dataset:
17         acl.mdl.destroy_dataset(self.output_dataset)
18
19 # In ascend_inference.py -> AscendSystem.release
20 def release(self):
21     if self.stream:
22         acl.rt.destroy_stream(self.stream)
23     if self.context:
24         acl.rt.destroy_context(self.context)
25     acl.rt.reset_device(self.device_id)
26     acl.finalize() # 最后关闭 ACL

```

1.8.3. 为什么需要区分 Host 和 Device?

对于初学者来说，最困惑的莫过于 Host 和 Device 的概念。让我们用一个更生活化的比喻来理解：

想象你家里有两个厨房：

- **Host (主厨房 - CPU)**: 这是你日常使用的厨房，有冰箱（内存）、微波炉（通用计算）。你可以在这里做各种事情：切菜、洗碗、煮咖啡。但如果要做一道需要专业设备的菜（比如需要专业烤箱的甜点），效率就不高。
- **Device (专业厨房 - NPU)**: 这是一个配备了专业设备的厨房，有工业级烤箱（AI 加速器）、专业搅拌机（矩阵运算单元）。它做某些特定的事情（AI 推理）速度极快，但不能做通用的事情（比如不能用来煮咖啡）。

关键问题：这两个厨房的冰箱（内存）是分开的！

- 你在主厨房冰箱里的食材（Host 内存的数据），专业厨房看不到
- 你必须手动把食材从主厨房搬到专业厨房（H2D 拷贝）
- 做好的菜在专业厨房，你要吃的话得搬回主厨房（D2H 拷贝）

技术层面的解释：

- **Host (主机)**: 指 CPU 及其管理的内存（我们常用的内存条，DDR4/DDR5）。它是系统的“总管”，负责运行操作系统、Python 解释器和大部分通用逻辑。
- **Device (设备)**: 指 NPU 及其专用的板载内存（如 HBM - High Bandwidth Memory）。它是专门用于并行计算的“专家”，执行 AI 推理速度极快，但不能直接运行通用程序。

CPU 和 NPU 是两个独立的计算单元，它们之间的内存地址空间**不共享**。因此，数据必须通过 memcpy 在两者之间显式地来回传递。这个过程是昇腾乃至所有异构计算（如 NVIDIA CUDA、AMD ROCm）平台的基本操作。

为什么不能共享内存？

这是硬件架构决定的。NPU 为了达到极高的计算性能，使用了专门优化的内存系统（高带宽、低延迟），这套内存系统与 CPU 的内存系统是物理隔离的。虽然这增加了编程的复杂度（需要手动拷贝数据），但换来了数十倍甚至上百倍的计算性能提升。

1.8.4. 从 ONNX 到 OM：模型转换的必要性

你可能会好奇：为什么不能直接用原始的 ONNX 模型，而要转换成 .om 格式？

ONNX (Open Neural Network Exchange) 是一个开放的模型格式标准，就像 PDF 是文档的通用格式一样。它的优点是通用性强，PyTorch、TensorFlow 等框架训练的模型都可以导出为 ONNX。

但是，ONNX 模型是“通用语言”，NPU 硬件需要的是“方言”。每种硬件（NVIDIA GPU、昇腾 NPU、Intel CPU）都有自己的优化方式和指令集。

OM (Offline Model) 是昇腾平台的专用模型格式。通过 `atc` (Ascend Tensor Compiler) 工具，我们可以：

1. **算子融合**：把多个小操作合并成一个大操作，减少内存读写
2. **内存优化**：预先规划好内存分配，避免运行时的动态分配开销
3. **指令优化**：把通用的神经网络操作翻译成 NPU 的专用指令
4. **量化加速**：可选地将 FP32 转为 FP16 或 INT8，进一步提升速度

比喻：ONNX 就像一份通用的菜谱，任何厨房都能看懂。但如果你有一个专业的意大利餐厅（昇腾 NPU），你会把菜谱翻译成意大利语，并标注上“用我们的专业烤箱，温度设置 220 度，时间 15 分钟”。这样厨师（NPU）执行起来会快得多。

转换命令示例（在 `prepare_models.py` 中）：

```
1 atc --model=det_500m.onnx \
2   --framework=5 \
3   --output=face_detection \
4   --input_shape="input.1:1,3,640,640" \
5   --soc_version=Ascend310B1
```

参数解释：`--model`: 输入的 ONNX 模型 `--framework=5`: 表示 ONNX 格式 `--output`: 输出的 OM 模型名称 `--input_shape`: 指定输入张量的形状 `--soc_version`: 目标硬件型号

1.9. Anchor-based 检测深度解析：RetinaFace 如何“看见”人脸

在 `ascend_inference.py` 中，有一段看起来很神秘的代码：

```
1 def generate_anchors(self, height, width):
2     strides = [8, 16, 32]
3     anchors = []
4     for stride in strides:
5         num_grid_y = height // stride
6         num_grid_x = width // stride
7         for y in range(num_grid_y):
8             for x in range(num_grid_x):
9                 for _ in range(2): # 2 anchors per grid
10                    anchors.append([x * stride, y * stride, stride])
11     return np.array(anchors, dtype=np.float32)
```

这段代码在生成 **Anchors**（锚点）。要理解它，我们需要先理解目标检测的核心挑战。

1.9.1. 目标检测的根本问题

假设你要在一张 640×640 的图片中找到所有人脸。最直接的想法是什么？

暴力方法：用一个固定大小的窗口（比如 100×100 ）在图片上滑动，每个位置都判断“这里有没有人脸”。

问题来了：1. 人脸大小不一样，有的 50×50 ，有的 200×200 ，一个窗口怎么够？2. 即使用多个不同大小的窗口，也需要滑动成千上万次，太慢了！

1.9.2. Anchor-based 方法的智慧

Anchor-based 检测器（如 RetinaFace、YOLO、Faster R-CNN）的核心思想是：

不要在原图上滑动窗口，而是在特征图上预测！

让我们一步步理解：

第 1 步：特征图的概念

当图像经过卷积神经网络后，会得到多个不同尺度的**特征图 (Feature Map)**。

- 输入图像： $640 \times 640 \times 3$
- 经过网络后：
 - 浅层特征图： $80 \times 80 \times 256$ (stride=8, 每 8 个像素压缩成 1 个)
 - 中层特征图： $40 \times 40 \times 512$ (stride=16)
 - 深层特征图： $20 \times 20 \times 1024$ (stride=32)

比喻：原图是一张高清照片，特征图是这张照片的“缩略图”。浅层特征图是“中等缩略图”，深层特征图是“超小缩略图”。

第 2 步：Anchor 的作用

在每个特征图的每个位置（网格点），我们预先定义几个**候选框 (Anchor)**。

- 在 80×80 的特征图上，有 $80 \times 80 = 6400$ 个网格点
- 每个网格点放置 2 个 anchor
- 总共 $6400 \times 2 = 12800$ 个 anchor

比喻：Anchor 就像是“预设的相框”。我们在墙上（特征图）的每个位置都挂了几个不同大小的相框，然后让模型判断：1. 这个相框里有没有人脸？（分类）2. 如果有，人脸的实际位置相对于相框偏移了多少？（回归）

第 3 步：多尺度检测

为什么要用 stride=8/16/32 三个尺度？

- **stride=8 (80×80 特征图)**：负责检测小人脸

- 每个网格对应原图 8×8 像素
- 适合检测 16-64 像素的人脸
- **stride=16 (40×40 特征图)**：负责检测中等人脸
 - 每个网格对应原图 16×16 像素
 - 适合检测 32-128 像素的人脸
- **stride=32 (20×20 特征图)**：负责检测大人脸
 - 每个网格对应原图 32×32 像素
 - 适合检测 64-256 像素的人脸

比喻：这就像用三种不同倍率的望远镜同时观察：低倍镜看远处的大物体，高倍镜看近处的小物体。

1.9.3. 边界框解码：从偏移量到坐标

模型输出的不是直接的坐标，而是相对于 anchor 的偏移量。

```

1 def decode_bbox(self, anchors, raw_outputs):
2     # 模型输出：相对于 anchor 的偏移 [left, top, right, bottom]
3     # 需要转换为：绝对坐标 [x1, y1, x2, y2]
4
5     x1 = anchors[:, 0] - bbox_all[:, 0] * anchors[:, 2] # anchor_x - left*
6     y1 = anchors[:, 1] - bbox_all[:, 1] * anchors[:, 2] # anchor_y - top*stride
7     x2 = anchors[:, 0] + bbox_all[:, 2] * anchors[:, 2] # anchor_x + right*
8     y2 = anchors[:, 1] + bbox_all[:, 3] * anchors[:, 2] # anchor_y + bottom*

```

为什么要这样设计？

如果直接预测绝对坐标，模型需要学习“在 640×640 的图像中，人脸可能在任何位置”，这个学习空间太大了。

但如果预测相对偏移，模型只需要学习“相对于这个 anchor，人脸往左偏了一点，往上偏了一点”，学习难度大大降低。

比喻：这就像导航。直接说“目的地在纬度 39.9°，东经 116.4°”（绝对坐标）不如说“从你当前位置往北走 500 米，再往东走 200 米”（相对偏移）更容易理解和执行。

1.9.4. NMS (非极大值抑制)：去除重复检测

由于有 16800 个 anchor，同一张人脸可能被多个 anchor 同时检测到。NMS 的作用是保留最好的那个，去掉重复的。

```

1 indices = cv2.dnn.NMSBoxes(rects, scores.tolist(), threshold, 0.4)

```

工作原理：1. 按置信度分数排序 2. 选择分数最高的框 A 3. 计算其他框与 A 的重叠度 (IoU) 4. 删除所有与 A 重叠度 > 0.4 的框 5. 重复 2-4，直到处理完所有框

比喻：就像选班长，先选出票数最高的，然后把所有和他“太像”的候选人（重复检测）都去掉，再选下一个。

1.10. 余弦相似度：如何判断“两张脸是同一个人”

在 app.py 和 camera.py 中，有这样一段关键代码：

```
1 # 计算余弦相似度
2 sim = np.dot(target_embedding, db_emb) / (np.linalg.norm(target_embedding) * np.
   ↪ linalg.norm(db_emb) + 1e-6)
3
4 # 判断是否匹配
5 if sim > 0.5: # 阈值
6     print(f"匹配成功！相似度：{sim}")
```

这段代码在做什么？为什么用余弦相似度而不是欧氏距离？

1.10.1. 特征向量的几何意义

ArcFace 模型输出的 512 维特征向量，可以理解为 512 维空间中的一个点（或者说从原点出发的一个箭头）。

比喻：想象你在描述一个人的特征：身高、体重、肤色、发型... 如果用 512 个维度来描述，每个人就是 512 维空间中的一个点。

关键问题：如何衡量两个点（两张脸）的相似度？

1.10.2. 方法 1：欧氏距离（不推荐）

欧氏距离就是两点之间的直线距离：

$$d = \sqrt{\sum_{i=1}^{512} (a_i - b_i)^2}$$

问题：欧氏距离受向量长度（模）的影响很大。

假设有两个向量：- 向量 A：[1, 1, 1, ..., 1] (512 个 1) - 向量 B：[2, 2, 2, ..., 2] (512 个 2) - 向量 C：[1, 0, 0, ..., 0] (第一个是 1，其余是 0)

从方向上看，A 和 B 完全一致（都指向同一个方向），只是 B 的长度是 A 的 2 倍。但欧氏距离会认为 A 和 B 差别很大。

而 A 和 C 的方向差异巨大，但如果 C 的长度恰好合适，欧氏距离可能反而比 A-B 更小。

1.10.3. 方法 2：余弦相似度（推荐）

余弦相似度只关心方向，不关心长度：

$$\text{cosine similarity} = \frac{A \cdot B}{\|A\| \times \|B\|} = \cos(\theta)$$

其中 θ 是两个向量的夹角。

几何意义： - 如果两个向量方向完全相同，夹角 = 0° ， $\cos(0^\circ) = 1$ - 如果两个向量垂直，夹角 = 90° ， $\cos(90^\circ) = 0$ - 如果两个向量方向相反，夹角 = 180° ， $\cos(180^\circ) = -1$

比喻：想象两个人站在原点，分别指向不同的方向。余弦相似度衡量的是“他们指的方向有多接近”，而不管“他们的手臂伸得有多长”。

1.10.4. 为什么 ArcFace 适合用余弦相似度？

还记得 ArcFace 的训练目标吗？它在超球面上进行优化，强制所有特征向量的长度（模）都归一化到 1。

这意味着：1. 所有特征向量都在一个单位球面上 2. 向量之间的差异只体现在方向上 3. 余弦相似度完美地捕捉了这种“方向差异”

代码实现细节：

```
1 # 分子：向量点积
2 dot_product = np.dot(emb1, emb2)
3
4 # 分母：两个向量的模的乘积
5 norm_product = np.linalg.norm(emb1) * np.linalg.norm(emb2)
6
7 # 余弦相似度
8 similarity = dot_product / (norm_product + 1e-6) # 加 1e-6 防止除零
```

为什么加 1e-6？

这是一个防御性编程技巧。虽然理论上向量的模不会是 0，但由于浮点数精度问题，可能出现极小的值。加上一个极小的数（ $1e-6 = 0.000001$ ）可以避免除零错误，同时不影响正常计算结果。

1.10.5. 阈值的选择：0.5 的含义

```
1 threshold = 0.5
2 if similarity > threshold:
3     # 认为是同一个人
```

0.5 对应的夹角：

$$\theta = \arccos(0.5) = 60^\circ$$

这意味着，如果两个特征向量的夹角小于 60° ，我们就认为它们是同一个人。

如何调整阈值？

- 提高阈值（如 0.6-0.7）：
 - 更严格，减少误识别（把陌生人认成员工）
 - 但可能增加漏识别（把员工认不出来）
 - 适合安全性要求高的场景
- 降低阈值（如 0.4-0.5）：
 - 更宽松，减少漏识别
 - 但可能增加误识别
 - 适合便利性优先的场景

比喻：阈值就像门禁的严格程度。设得高，只有拿着正确钥匙的人才能进（安全但不便）；设得低，钥匙差不多的也能进（方便但不安全）。

1.11. 多线程与实时性：摄像头后台线程的设计

在 camera.py 中，有一个精妙的多线程设计：

```

1 class VideoCamera:
2     def __init__(self, face_system):
3         self.video = cv2.VideoCapture(0)
4         self.lock = threading.Lock()
5         self.last_frame = None
6
7         # 启动后台线程
8         self.thread = threading.Thread(target=self.update, args=())
9         self.thread.daemon = True
10        self.thread.start()
11
12    def update(self):
13        while self.running:
14            success, frame = self.video.read()
15            with self.lock:
16                self.last_frame = frame.copy()
17
18            # 每 2 秒自动打卡
19            if time.time() - self.last_check_time > 2.0:
20                self.process_attendance(frame)

```

1.11.1. 为什么需要多线程？

问题场景：如果在主线程中同步处理摄像头，会发生什么？

```

1 # 错误的做法（伪代码）
2 while True:
3     frame = camera.read() # 读取帧，耗时 30ms

```

```

4 detect_face(frame)      # 检测人脸，耗时 20ms
5 recognize_face(frame)   # 识别人脸，耗时 15ms
6 # 总耗时：65ms，帧率只有 15 FPS

```

在这种设计下：1. 读取帧要等待 2. 处理帧要等待 3. 如果有 Web 请求进来，整个系统都会卡住

1.11.2. 多线程的优势

后台线程持续做两件事：1. 以 30 FPS 的速度读取摄像头帧，保存到 `last_frame` 2. 每 2 秒触发一次人脸识别

主线程可以：1. 随时读取 `last_frame` 用于 Web 视频流 2. 处理用户的 HTTP 请求 3. 不会被摄像头读取阻塞

比喻：这就像餐厅的分工。后厨（后台线程）持续准备食材、做菜，前台（主线程）负责接待客人、上菜。两者并行工作，互不干扰。

1.11.3. 线程安全：Lock 的作用

```

1 with self.lock:
2     self.last_frame = frame.copy()

```

为什么需要锁？

想象两个线程同时访问 `last_frame`：- 线程 A（后台）：正在写入新的帧 - 线程 B（主线程）：正在读取帧用于 Web 传输

如果没有锁，可能发生：- 线程 B 读到一半，线程 A 开始写入 - 结果：读到的是“半新半旧”的数据，导致图像错乱

Lock（互斥锁） 确保：- 同一时刻只有一个线程能访问 `last_frame` - 其他线程必须等待，直到锁被释放

比喻：Lock 就像卫生间的门锁。有人在用的时候，其他人必须在外面等。

1.11.4. daemon 线程的含义

```

1 self.thread.daemon = True

```

daemon（守护线程） 的特点：- 当主程序退出时，守护线程会自动终止 - 不会阻止程序退出

如果不设置为 `daemon`：- 主程序想退出时，会等待所有非 `daemon` 线程结束 - 摄像头线程在 `while self.running` 中循环，可能导致程序无法正常退出

比喻：`daemon` 线程就像保安。老板（主程序）下班了，保安也就下班了，不会一个人留在公司。

1.12. Flask Web 框架：构建轻量级 API 服务

app.py 使用 Flask 框架构建了整个 Web 服务。Flask 是 Python 中最流行的轻量级 Web 框架之一。

1.12.1. 什么是 Web 框架？

想象你要开一家餐厅。你可以：1. **从零开始**：自己建房子、装修、设计菜单、培训服务员... 2. **加盟连锁**：使用现成的装修方案、标准化流程、培训体系...

Web 框架就是“连锁餐厅的标准化方案”。它提供了：- 路由系统（URL 到函数的映射）- 请求处理（解析 HTTP 请求）- 响应生成（返回 JSON、HTML）- 静态文件服务（CSS、JS、图片）

1.12.2. Flask 的核心概念

1. 路由 (Route)

```
1 @app.route('/api/users', methods=['GET'])
2 def list_users():
3     users = database.get_users()
4     return jsonify(users)
```

路由的作用：把 URL 映射到 Python 函数。

- 当用户访问 `http://localhost:5000/api/users`
- Flask 自动调用 `list_users()` 函数
- 函数返回的数据被转换成 JSON 响应

比喻：路由就像餐厅的菜单。客人点“宫保鸡丁”（URL），服务员就知道要让厨师做哪道菜（调用哪个函数）。

2. HTTP 方法

```
1 @app.route('/api/users', methods=['GET']) # 查询用户
2 @app.route('/api/users', methods=['POST']) # 添加用户
3 @app.route('/api/users/<id>', methods=['DELETE']) # 删除用户
```

HTTP 方法的语义：- **GET**：获取数据（只读，不修改服务器状态）- **POST**：提交数据（创建新资源）- **PUT**：更新数据（完整替换）- **DELETE**：删除数据

比喻：这就像对图书馆的操作：- **GET** = 查阅书籍（不改变图书馆）- **POST** = 捐赠新书（增加藏书）- **PUT** = 更换书籍（用新版替换旧版）- **DELETE** = 销毁书籍（减少藏书）

3. 请求数据的获取

```

1 @app.route('/api/users', methods=['POST'])
2 def add_user():
3     name = request.form.get('name')           # 表单数据
4     file = request.files['image']             # 上传的文件
5     data = request.json                       # JSON 数据

```

Flask 的 request 对象提供了访问各种请求数据的接口：- request.form: HTML 表单数据 - request.files: 上传的文件 - request.json: JSON 格式的请求体 - request.args: URL 查询参数（如 ?page=1&size=10）

4. 响应的生成

```

1 # 返回 JSON
2 return jsonify({"success": True, "user_id": 123})
3
4 # 返回 HTML 页面
5 return render_template('index.html')
6
7 # 返回文件
8 return send_from_directory('uploads', filename)
9
10 # 返回流式数据（视频流）
11 return Response(gen(camera), mimetype='multipart/x-mixed-replace; boundary=frame'
    ↪ ')

```

1.12.3. MJPEG 视频流：实时传输的技巧

```

1 def gen(camera):
2     while True:
3         frame = camera.get_frame() # 获取 JPEG 编码的帧
4         yield (b'--frame\r\n'
5               b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')
6
7 @app.route('/video_feed')
8 def video_feed():
9     return Response(gen(camera), mimetype='multipart/x-mixed-replace; boundary=frame'
    ↪ )

```

MJPEG (Motion JPEG) 的原理：- 把视频流当作一系列 JPEG 图片 - 通过 HTTP 持续发送，用特殊的分隔符 --frame 分隔每一帧 - 浏览器接收到后，自动连续显示，形成视频效果

比喻：这就像翻页动画。每一页是一张静态图片（JPEG），快速翻页就形成了动画效果。

为什么用 yield?

yield 是 Python 的生成器语法，它可以：1. 逐帧生成数据，而不是一次性生成所有帧（节省内存）2. 实现“无限流”，持续发送数据直到连接断开

1.13. 图像预处理：为什么需要归一化？

在 `ascend_inference.py` 中，有这样的预处理代码：

```
1 def preprocess_det(self, image):
2     img = cv2.resize(image, (640, 640))
3     img = img.astype(np.float32)
4     img -= np.array([127.5, 127.5, 127.5]) # 减去均值
5     img /= 128.0 # 除以标准差
6     img = img.transpose(2, 0, 1) # HWC -> CHW
7     img = np.expand_dims(img, axis=0) # 添加 batch 维度
8     return img
```

每一步都有其深刻的原因。

1.13.1. 第 1 步：缩放到固定尺寸

```
1 img = cv2.resize(image, (640, 640))
```

为什么？

神经网络的输入必须是固定尺寸。模型在训练时使用 640×640 的图像，推理时也必须用相同尺寸。

会不会变形？

会的。如果原图是 1920×1080，缩放到 640×640 会拉伸。但这不是问题，因为：1. 模型在训练时也见过各种拉伸的图像 2. 检测到的坐标会按比例缩放回原图

比喻：这就像证件照必须是 2 寸（35×45mm）。不管你的脸是圆是方，都要缩放到这个尺寸。

1.13.2. 第 2 步：归一化 (Normalization)

```
1 img -= 127.5 # 减去均值
2 img /= 128.0 # 除以标准差
```

原始像素值的范围：0-255（8 位整数）

归一化后的范围：约 -1 到 +1

为什么要归一化？

1. **数值稳定性**：神经网络的权重通常在 -1 到 +1 之间。如果输入是 0-255，会导致梯度爆炸或消失。
2. **加速收敛**：归一化后，不同特征的数值范围一致，优化器更容易找到最优解。
3. **匹配训练时的分布**：模型训练时用的是归一化数据，推理时也必须用相同的预处理。

比喻：想象你在称重。如果有人用公斤，有人用磅，有人用吨，就很难比较。归一化就是把所有单位统一成“标准单位”。

为什么是 127.5 和 128？

- 像素值范围：0-255
- 中间值：127.5
- 减去中间值后：-127.5 到 +127.5
- 除以 128：约 -1 到 +1

这是一种常见的归一化方法，称为 **零均值归一化**。

1.13.3. 第 3 步：通道顺序转换

```
1 img = img.transpose(2, 0, 1) # HWC -> CHW
```

OpenCV 的格式：(Height, Width, Channels) = (640, 640, 3) - 第 0 维：高度 - 第 1 维：宽度 - 第 2 维：通道 (BGR)

PyTorch/ONNX 的格式：(Channels, Height, Width) = (3, 640, 640) - 第 0 维：通道 - 第 1 维：高度 - 第 2 维：宽度

为什么要转换？

这是深度学习框架的约定。**PyTorch**、**TensorFlow** 等框架在处理卷积时，期望通道维度在前。

比喻：这就像不同国家的日期格式。中国用“年-月-日”，美国用“月-日-年”。虽然信息相同，但格式必须匹配。

1.13.4. 第 4 步：添加 Batch 维度

```
1 img = np.expand_dims(img, axis=0) # (3, 640, 640) -> (1, 3, 640, 640)
```

为什么需要 Batch？

神经网络通常设计为处理“一批”数据，而不是单个样本。即使我们只有一张图片，也要把它包装成“一批，包含 1 张图片”。

- 输入形状：(Batch, Channels, Height, Width)
- 单张图片：(1, 3, 640, 640)
- 多张图片：(4, 3, 640, 640) 表示一批 4 张图片

比喻：这就像快递。即使你只寄一件商品，也要装在一个箱子里 (**batch**)。箱子上写着“内含 1 件商品”。

1.14. 性能优化策略：让系统跑得更快

1.14.1. 1. 内存复用：避免重复分配

在 AscendModel 的设计中，输入输出 buffer 在初始化时就分配好了，之后每次推理都复用这些 buffer。

```
1 def _init_buffers(self):
2     # 只在初始化时分配一次
3     dev_ptr, ret = acl.rt.malloc(size, 2)
4     self.input_buffers.append({"ptr": dev_ptr, "size": size})
5
6 def execute(self, input_data_list):
7     # 每次推理都复用同一块内存
8     ret = acl.rt.memcpy(self.input_buffers[i]["ptr"], ...)
```

如果每次都重新分配会怎样？

```
1 # 错误的做法
2 def execute_bad(self, input_data):
3     dev_ptr, ret = acl.rt.malloc(size, 2) # 每次都分配
4     acl.rt.memcpy(dev_ptr, ...)
5     acl.mdl.execute(...)
6     acl.rt.free(dev_ptr) # 每次都释放
```

这样做的问题：- 内存分配/释放是昂贵的操作（可能耗时几毫秒）- 频繁分配会导致内存碎片 - 在实时应用中，这些毫秒级的延迟会累积成明显的卡顿

比喻：这就像餐厅的盘子。好的做法是准备一套盘子，用完洗干净继续用。坏的做法是每次都买新盘子，用完就扔，既浪费又慢。

1.14.2. 2. 批处理：一次处理多张图片

虽然本案例是单张处理，但如果需要处理大量图片（如批量注册用户），可以使用批处理：

```
1 # 单张处理：推理 100 次
2 for img in images:
3     result = model.execute([img]) # 每次 H2D + 推理 + D2H
4
5 # 批处理：推理 1 次
6 batch = np.stack(images[:100]) # (100, 3, 640, 640)
7 results = model.execute([batch]) # 一次性处理 100 张
```

批处理的优势：- 减少 H2D/D2H 的次数（数据传输开销）- NPU 的并行计算能力得到充分利用 - 吞吐量可提升 5-10 倍

权衡：- 延迟增加（需要等待凑够一批）- 内存占用增加

比喻：这就像坐电梯。等人齐了再走（批处理）比每来一个人就开一次（单张处理）效率高得多。

1.14.3. 3. 异步推理：让 CPU 和 NPU 并行工作

当前实现是同步的：

```
1 CPU: 准备数据 -> 等待 -> 等待 -> 处理结果
2 NPU:          空闲 -> 推理 -> 空闲
```

异步推理可以让 CPU 和 NPU 同时工作：

```
1 CPU: 准备数据1 -> 准备数据2 -> 准备数据3 -> 处理结果1
2 NPU:          推理1 -> 推理2 -> 推理3
```

实现方式（伪代码）：

```
1 # 提交任务到 stream, 不等待完成
2 acl.mdl.execute_async(model_id, input, output, stream)
3
4 # CPU 继续做其他事情
5 prepare_next_data()
6
7 # 需要结果时再同步
8 acl.rt.synchronize_stream(stream)
```

1.14.4. 4. 模型量化：用更少的位数表示权重

当前模型使用 FP32（32 位浮点数）。通过量化，可以转换为 FP16 或 INT8：

- **FP32**：精度最高，速度最慢，内存占用最大
- **FP16**：精度略降，速度提升 2 倍，内存减半
- **INT8**：精度再降，速度提升 4 倍，内存减少 75%

量化的代价：- 精度损失（通常在 1-3% 以内）- 需要校准数据集

在 atc 转换时启用量化：

```
1 atc --model=det.onnx \
2     --framework=5 \
3     --output=det_int8 \
4     --precision_mode=allow_mix_precision \
5     --insert_op_conf=quant_config.json
```

1.15. 实际部署建议

1.15.1. 1. 用户规模与检索策略

用户数量	检索方法	预期延迟
< 100	暴力检索（当前方案）	< 10ms

用户数量	检索方法	预期延迟
100-1000	暴力检索 + 缓存	< 50ms
1000-10000	Faiss (IVF)	< 20ms
> 10000	Faiss (HNSW) 或向量数据库	< 50ms

1.15.2. 2. 防止重复打卡

当前实现每 2 秒就会记录一次考勤。实际应用中应该添加去重逻辑：

```

1 def add_attendance_if_not_recent(user_id, type, image_path, cooldown=60):
2     """只有距离上次打卡超过 cooldown 秒才记录"""
3     conn = sqlite3.connect(DB_NAME)
4     c = conn.cursor()
5
6     # 查询最近一次打卡时间
7     c.execute('''
8         SELECT timestamp FROM attendance
9         WHERE user_id = ?
10        ORDER BY timestamp DESC LIMIT 1
11    ''', (user_id,))
12
13    last_record = c.fetchone()
14    if last_record:
15        last_time = datetime.fromisoformat(last_record[0])
16        if (datetime.now() - last_time).total_seconds() < cooldown:
17            conn.close()
18            return None # 太频繁，不记录
19
20    # 记录新的考勤
21    c.execute('INSERT INTO attendance (user_id, type, image_path) VALUES (?, ?,
22        ↳ ?)',
23              (user_id, type, image_path))
24    conn.commit()
25    conn.close()

```

1.15.3. 3. 安全性考虑

照片攻击防御：

当前系统无法防御“拿着照片打卡”的攻击。如需防御，可以：1. 添加活体检测模型（检测眨眼、张嘴等动作）2. 使用 3D 人脸识别（需要深度摄像头）3. 结合其他生物特征（指纹、虹膜）

数据安全：

```

1 # 特征向量加密存储
2 import hashlib
3
4 def encrypt_embedding(embedding, key):
5     # 简单的 XOR 加密（实际应用应使用 AES）
6     key_hash = hashlib.sha256(key.encode()).digest()

```

```

7     encrypted = bytes([a ^ b for a, b in zip(embedding.tobytes(), key_hash *
      ↳ 100)])
8     return encrypted

```

1.15.4. 4. 日志与监控

添加详细的日志记录：

```

1 import logging
2
3 logging.basicConfig(
4     level=logging.INFO,
5     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
6     handlers=[
7         logging.FileHandler('attendance.log'),
8         logging.StreamHandler()
9     ]
10 )
11
12 logger = logging.getLogger(__name__)
13
14 # 在关键位置记录日志
15 logger.info(f"User {user_id} checked in, similarity: {similarity:.3f}")
16 logger.warning(f"Failed to detect face in frame")
17 logger.error(f"NPU inference failed: {error}")

```

1.15.5. 5. 错误处理与降级策略

```

1 def get_face_system():
2     global face_system
3     if face_system is None:
4         try:
5             face_system = FaceSystem()
6         except Exception as e:
7             logger.error(f"Failed to initialize NPU: {e}")
8             # 降级到 CPU 模式（使用 ONNX Runtime）
9             face_system = FaceSystemCPU()
10    return face_system

```

1.16. 常见问题深度解答

1.16.1. Q1: 为什么摄像头打开失败？

可能原因：

1. 权限问题：

```

1 ls -l /dev/video0
2 # 如果显示 crw-rw---- root video, 说明只有 root 和 video 组能访问
3
4 # 解决方案 1: 临时修改权限
5 sudo chmod 666 /dev/video0
6
7 # 解决方案 2: 将用户加入 video 组 (永久)
8 sudo usermod -aG video $USER
9 # 需要重新登录生效

```

2. 设备被占用:

```

1 # 查看哪个进程在使用摄像头
2 lsof /dev/video0
3
4 # 如果是其他程序, 先关闭它

```

3. 驱动问题:

```

1 # 检查摄像头是否被识别
2 v4l2-ctl --list-devices
3
4 # 测试摄像头
5 ffmpeg /dev/video0

```

1.16.2. Q2: 识别率低怎么办?

诊断步骤:

1. 检查注册照片质量:

- 光照是否均匀?
- 人脸是否清晰?
- 是否有遮挡?

2. 调整阈值:

```

1 # 在 app.py 和 camera.py 中
2 threshold = 0.5 # 降低到 0.4 试试
3
4 # 同时记录相似度分布
5 logger.info(f"Similarity scores: {[f'{s:.3f}' for s in all_similarities]}")

```

3. 检查预处理是否正确:

```

1 # 保存预处理后的图像, 检查是否正常
2 cv2.imwrite('debug_preprocessed.jpg', img * 128 + 127.5)

```


1.16.3. Q3: 推理速度慢怎么办？

性能分析：

```

1 import time
2
3 def profile_inference():
4     # 测试检测速度
5     start = time.time()
6     faces = face_system.detect(image)
7     det_time = time.time() - start
8
9     # 测试识别速度
10    start = time.time()
11    embedding = face_system.get_embedding(face_img)
12    rec_time = time.time() - start
13
14    print(f"Detection: {det_time*1000:.1f}ms")
15    print(f"Recognition: {rec_time*1000:.1f}ms")

```

优化方向：- 如果检测慢：降低输入分辨率（640->480）- 如果识别慢：检查是否正确使用了 NPU - 如果数据传输慢：检查是否有不必要的内存拷贝

1.17. 扩展实验建议

1.17.1. 实验 1：添加情绪识别

在人脸识别的基础上，添加情绪分类（开心、悲伤、愤怒等）：

1. 下载情绪识别模型（如 FER2013）
2. 转换为 OM 格式
3. 在检测到人脸后，额外进行情绪推理
4. 在考勤记录中保存情绪信息

1.17.2. 实验 2：多摄像头支持

扩展系统支持多个摄像头（如大门、前台、会议室）：

```

1 cameras = {
2     'entrance': VideoCamera(face_system, device_id=0),
3     'lobby': VideoCamera(face_system, device_id=1),
4     'meeting_room': VideoCamera(face_system, device_id=2),
5 }

```

1.17.3. 实验 3：移动端集成

开发移动 App，通过 HTTP API 与服务器通信：

1 手机拍照 -> Base64 编码 -> POST /api/clockin -> 返回结果

1.18. Web 界面详细介绍

1.19. Web 界面详细介绍

1.19.1. 1. 主页 (/)

系统概览与功能导航入口。提供三个主要功能模块的快速访问：- 用户管理：注册新用户、查看用户列表 - 考勤记录：查看打卡历史、手动打卡 - 实时监控：查看摄像头实时画面

1.19.2. 2. 用户管理 (/users_page)

用户注册功能：

支持两种注册方式：

1. 上传照片：

- 用户选择本地照片文件
- 系统自动检测人脸并裁剪
- 提取 512 维特征向量存入数据库

2. 设备摄像头抓拍：

- 点击“Capture from Device Camera”
- 系统从本地 USB 摄像头抓取当前帧
- 自动完成人脸检测和特征提取

技术实现：

```

1 // 前端调用摄像头抓拍
2 fetch('/api/camera/capture', {method: 'POST'})
3   .then(res => res.json())
4   .then(data => {
5     // 获得临时图片路径
6     document.getElementById('preview').src = '/uploads/' + data.temp_path;
7   });
8
9 // 提交注册
10 const formData = new FormData();
11 formData.append('name', userName);
12 formData.append('temp_path', tempPath); // 使用抓拍的图片
13 fetch('/api/users', {method: 'POST', body: formData});

```

用户列表：- 显示所有已注册用户 - 支持删除操作（同时删除考勤记录） - 实时更新

1.19.3. 3. 考勤记录 (/attendance_page)

自动打卡显示： - 实时展示本地摄像头自动识别的打卡记录 - 显示用户姓名、打卡时间、相似度分数 - 页面自动刷新（每 5 秒）

手动打卡功能： - 支持浏览器摄像头拍照打卡 - 支持上传照片打卡 - 实时反馈识别结果（匹配成功/失败）

技术实现：

```

1 // 使用浏览器摄像头
2 navigator.mediaDevices.getUserMedia({video: true})
3   .then(stream => {
4     video.srcObject = stream;
5   });
6
7 // 拍照并打卡
8 const canvas = document.createElement('canvas');
9 canvas.getContext('2d').drawImage(video, 0, 0);
10 const imageData = canvas.toDataURL('image/jpeg');
11
12 fetch('/api/clockin', {
13   method: 'POST',
14   headers: {'Content-Type': 'application/x-www-form-urlencoded'},
15   body: 'image_base64=' + encodeURIComponent(imageData)
16 });

```

1.19.4. 4. 实时视频流 (/video_feed)

通过 MJPEG 协议提供实时视频流：

```
1 
```

浏览器会持续接收视频帧，形成实时监控效果。

1.20. 系统架构总结

1.20.1. 数据流向

```

1 摄像头 -> OpenCV -> 人脸检测(NPU) -> 人脸裁剪 -> 特征提取(NPU) -> 相似度计算(CPU)
   ↗ ) -> 数据库
2   ↓
   ↗
   ↘ ↓
3 后台线程(30 FPS)                                考勤
   ↗ 记录
4   ↓
5 MJPEG 流 -> Web 浏览器

```

1.20.2. 关键技术栈

层次	技术	作用
硬件加速	昇腾 310B NPU	AI 推理加速
推理接口	PyACL	NPU 编程接口
模型格式	OM (离线模型)	昇腾专用格式
检测算法	RetinaFace (SCRFD)	人脸检测
识别算法	ArcFace	特征提取
Web 框架	Flask	HTTP 服务
数据库	SQLite	本地存储
图像处理	OpenCV	摄像头、图像操作
前端	HTML5 + Bootstrap	用户界面

1.20.3. 性能指标

在昇腾 310B 上的典型性能：

指标	数值
人脸检测延迟	10-20ms
特征提取延迟	5-10ms
特征比对延迟 (100 人)	< 5ms
端到端延迟	< 50ms
视频流帧率	30 FPS
自动打卡间隔	2 秒

1.21. 学习路径建议

1.21.1. 初学者（第 1-2 周）

1. 理解整体流程：

- 运行系统，体验各项功能
- 阅读 README.md，理解系统架构
- 观察日志输出，了解执行过程

2. 修改简单参数：

- 调整识别阈值 (0.5 -> 0.6)

- 修改自动打卡间隔（2 秒 -> 5 秒）
- 更改摄像头分辨率

3. 理解核心概念：

- 什么是特征向量？
- 什么是余弦相似度？
- Host 和 Device 的区别

1.21.2. 进阶学习（第 3-4 周）

1. 深入代码细节：

- 阅读 `ascend_inference.py`，理解 ACL 工作流
- 阅读 `camera.py`，理解多线程设计
- 阅读 `app.py`，理解 Flask 路由

2. 实验与调试：

- 添加性能分析代码
- 尝试不同的预处理方法
- 测试不同光照条件下的识别率

3. 扩展功能：

- 添加考勤统计功能
- 实现考勤数据导出（CSV/Excel）
- 添加用户权限管理

1.21.3. 高级应用（第 5-8 周）

1. 性能优化：

- 实现批处理推理
- 集成 Faiss 加速检索
- 尝试模型量化（FP16/INT8）

2. 功能增强：

- 添加活体检测
- 支持多摄像头
- 实现移动端 App

3. 生产部署：

- 添加完善的错误处理
- 实现日志系统
- 配置 Nginx 反向代理
- 使用 Gunicorn 部署

1.22. 总结

本案例展示了如何利用昇腾 310B NPU 构建一个完整的人脸识别考勤系统。通过学习本案例，你应该掌握：

理论知识： - Anchor-based 目标检测原理 - 度量学习与 ArcFace 损失函数 - 余弦相似度与特征匹配 - 图像预处理与归一化

工程技能： - PyACL 编程与 NPU 资源管理 - Host-Device 内存管理 - 多线程编程与线程安全 - Flask Web 开发 - SQLite 数据库操作

实践经验： - 模型转换（ONNX -> OM） - 边缘设备部署 - 实时视频流处理 - 性能优化策略
这些知识和技能不仅适用于人脸识别，也可以迁移到其他 AI 应用场景，如物体检测、图像分类、语义分割等。

祝你学习愉快！

2. 案例 2：目标跟踪检测

2.1. 教程定位

本案例面向昇腾 310B 平台的开发入门，目标不只是把程序跑起来，而是建立一条完整、清晰、可扩展的目标跟踪知识主线：

1. 为什么边缘设备上要优先选择轻量级检测网络。
2. MobileNet 的结构为什么适合做骨干网络。
3. SSD 的检测思想为什么适合实时场景。
4. MobileNet-SSD 作为检测前端，在目标跟踪任务中有哪些优点和局限。
5. 为什么在检测结果之上叠加 DeepSORT，是一个很好的多目标跟踪入门方案。
6. 本案例中的 ssdlite 与 tracking 代码，分别承担了什么角色。

从整体结构看，本案例对应一条标准的视频目标分析流水线：

本案例选择的实现路线是：

- 使用 MobileNet-SSD 完成实时目标检测。
- 使用一个简化版 DeepSORT 风格跟踪器完成多目标跟踪。
- 使用 CPU 与 Ascend NPU 统一接口，方便读者同时理解算法与部署。

这条路线适合作为昇腾 310B 开发教程中的案例章节，因为它兼顾了三点：

- 算法结构足够完整。
- 推理链路足够直观。
- 工程复杂度可控，适合实验训练和工程实践。

2.2. 实验硬件与运行条件

本案例既可以在普通 CPU 环境下运行，也可以在昇腾 NPU 环境下运行。为了完成实时检测与实时跟踪实验，建议准备如下硬件条件：

- 一台 Linux 主机或昇腾开发环境
- 一只 USB 摄像头，用于实时视频采集

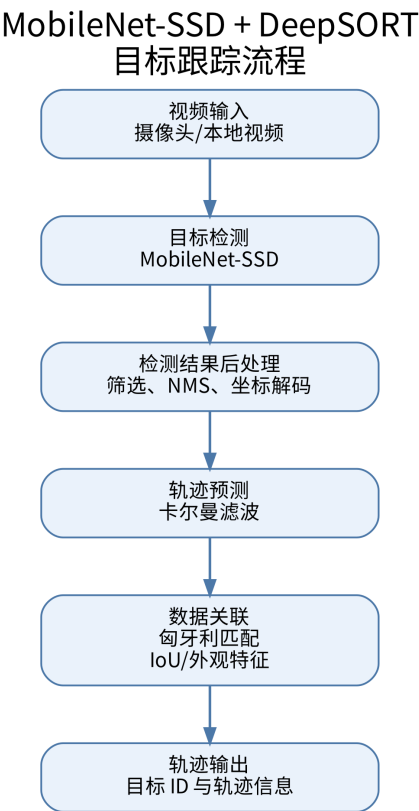


Figure 2.1.: 程序流程图

- Ascend 310B 或兼容昇腾 NPU 设备，运行 npu 模式时需要

运行方式可以分为两类：

- 实时演示：接入 USB 摄像头，使用 `--source 0` 作为输入
- 离线演示：直接读取本地视频文件，例如 `--source demo.mp4`

如果只运行 cpu 模式，可以没有昇腾 NPU；如果运行 npu 模式，则还需要本机正确安装 Ascend ACL Python 运行时，并准备对应的 .om 模型文件。当前模型下载脚本默认使用 zhouxzh [↗](#) /SSDLite320，该仓库发布的是 .onnx 模型；在 Ascend 310B 上运行 NPU 模式前，需要使用 `scripts/convert_onnx_to_om.py` 将 ONNX 转成 OM。

常用准备流程如下：

```
1 python scripts/download_models.py --onnx
2 source /usr/local/Ascend/ascend-toolkit/set_env.sh
3 python scripts/convert_onnx_to_om.py --soc-version Ascend310B4
```

转换脚本默认读取 `models/*.onnx`，并把同名 .om 文件写回 `models/`。如果板端 SoC 版本不是 Ascend310B4，需要按实际环境修改 `--soc-version`。

2.3. 本案例支持的骨干网络

当前仓库里的 SSD 模型已经覆盖了两类主干网络：

- MobileNet 系列：mobilenetv1、mobilenetv2、mobilenetv3、mobilenetv3_large_100、mobilenetv4
- ResNet 系列：resnet18、resnet34、resnet50、resnet101、resnet151

其中：

- `ssd320_*` 主要对应 MobileNet 系列，输入尺寸通常为 320 x 320
- `ssd300_*` 主要对应 ResNet 系列，输入尺寸通常为 300 x 300

从工程角度看，MobileNet 系列更强调轻量化和实时性，ResNet 系列更强调特征表达能力和检测稳定性。两类骨干网络共同构成了本案例中“轻量实时”和“更强表达”两种不同取向的实验基础。

2.4. 目标跟踪对时序关联的要求

目标检测解决的是单帧图像中的两个问题：

- 目标在哪里。
- 目标属于什么类别。

例如，在一帧图像里，检测器可能输出三个框，其中两个是行人，一个是汽车。但仅靠检测器，我们无法回答下面这些时间维度上的问题：

- 当前这一帧左侧的行人，是不是上一帧的 3 号目标。
- 被遮挡后重新出现的车辆，是否还是之前那一辆。
- 两个目标交叉通过时，系统该如何保持它们的身份不交换。

上述问题正是目标跟踪所要解决的核心任务。多目标跟踪系统不仅需要检测目标的存在，还需要为同一目标在连续帧中维持稳定的身份编号，即稳定的 ID。

因此，目标跟踪可以理解为“目标检测在时间维度上的延伸”。检测负责看见目标，跟踪负责维持目标身份。

2.5. MobileNet-SSD 作为检测前端的依据

2.5.1. 边缘侧场景的基本要求

昇腾 310B 面向边缘智能计算场景。在这类场景中，算法设计通常要同时考虑以下问题：

- 实时性是否足够。
- 模型规模是否可控。
- 推理链路是否简单稳定。
- 是否便于部署到 NPU 或 CPU 回退环境。

如果直接选择参数量庞大、结构复杂、后处理繁重的检测器，理解成本与工程调试成本都会显著上升，系统主线反而不易把握。因此，本案例采用轻量、经典且工程路径清晰的 MobileNet-SSD 组合作为检测前端。

2.5.2. MobileNet 的网络结构

MobileNet 的核心设计思想，是用深度可分离卷积替代标准卷积，从而显著降低参数量与计算量。

标准卷积把空间卷积和通道融合同步完成，而 MobileNet 将其拆分成两步：

1. Depthwise Convolution：对每个输入通道分别做空间卷积。
2. Pointwise Convolution：使用 1×1 卷积完成通道之间的信息融合。

如果标准卷积的输入通道数为 M ，输出通道数为 N ，卷积核大小为 $K \times K$ ，特征图大小为 $D \times D$ ，那么标准卷积的计算量大致为：

$$D^2 \cdot K^2 \cdot M \cdot N$$

而深度可分离卷积的计算量大致为：

$$D^2 \cdot K^2 \cdot M + D^2 \cdot M \cdot N$$

当 $K = 3$ 时，这种拆分能够显著降低计算成本。这一特性对于边缘设备尤为重要。

从结构上看，MobileNet 可以理解为由以下几类模块反复堆叠而成：

- 普通卷积层，用于最初级的特征提取。
- 深度卷积层，用于逐通道提取空间信息。
- 逐点卷积层，用于跨通道融合特征。
- 下采样层，用于逐步扩大感受野，获得更强的语义信息。

MobileNet 的主要价值体现在以下几个方面：

- 它很好地体现了轻量化网络的设计思想。
- 它适合作为边缘侧模型部署的入门例子。
- 它与 SSD 结合后，能够形成一条结构清晰、速度较快的检测链路。

2.5.3. MobileNet v1 至 v4 的版本演进

MobileNet 并非单一网络，而是一个持续演进的轻量级网络家族。v1 至 v4 的演进，本质上体现为在移动端与边缘侧约束下，对精度、速度与结构表达能力的持续平衡与优化。

MobileNet v1

MobileNet v1 是这个家族的起点。它最核心的贡献，就是把深度可分离卷积系统化地应用到整个网络主干中。

结构特点：

- 以 depthwise convolution 加 pointwise convolution 为基本单元
- 网络结构规整，堆叠方式简单直接
- 参数量和计算量相比标准卷积网络大幅下降

优点：

- 结构最直观，最容易理解
- 轻量化效果明显
- 在边缘设备上容易部署

局限：

- 特征表达能力相对有限
- 在相同算力下，精度通常不如后续版本

MobileNet v2

MobileNet v2 的核心改进是引入了 inverted residual 和 linear bottleneck。

结构特点：

- 先通过 1×1 卷积扩张通道
- 再做 depthwise convolution
- 最后通过线性 bottleneck 压缩回较低维度
- 在满足条件时使用残差连接

它与传统残差块的设计思路存在明显差异。传统残差块通常采用“宽输入到宽输出”的结构，而 MobileNet v2 更强调“窄输入、宽中间层、窄输出”，因此被称为 inverted residual。

这一版本的优势在于：

- 在轻量化前提下提升了表达能力
- 残差连接帮助梯度传播更稳定
- 在线性 bottleneck 中减少非线性带来的信息损失

MobileNet v3

MobileNet v3 在 v2 基础上继续优化，它融合了神经网络结构搜索和轻量注意力机制的思想。

结构特点：

- 延续 inverted residual 主体框架
- 引入 SE 模块增强通道注意力
- 使用 h-swish 等更适合移动端的激活函数
- 网络结构由精度与延迟共同驱动优化

与 v2 相比，v3 的一个重要特征在于更加关注真实硬件上的执行效率，而不仅仅是理论 FLOPs 的下降。换言之，v3 的优化目标并非单纯减少计算量，而是在移动端与边缘端设备上取得更优的实际速度与精度平衡。

MobileNet v4

MobileNet v4 是更后续的轻量骨干版本，它进一步强化了硬件感知设计，更强调不同算子组合在实际设备上的性能收益。

结构特点：

- 延续轻量主干的整体方向
- 更强调不同卷积模块和块结构的组合效率
- 进一步面向硬件友好的延迟优化

- 在不同配置下兼顾速度、参数量和表达能力

从本仓库的模型组织可以看出,模型目录中既包含 mobilenetv4,也包含 mobilenetv4_conv_large
→ .onnx、mobilenetv4_hybrid_medium.onnx 这样的变体文件。这表明 v4 系列本身已经具备不同配置路径,用于在更强表达能力与更低计算开销之间进行权衡。

2.5.4. MobileNet v1-v4 的结构对比总结

如果按结构演进主线来总结,四个版本的差异可以概括为:

- v1: 用深度可分离卷积解决“怎么把网络做轻”
- v2: 用 inverted residual 和 linear bottleneck 解决“轻量化下如何保留表达能力”
- v3: 用 SE、h-swish 和结构搜索解决“如何进一步优化真实设备上的速度与精度平衡”
- v4: 进一步走向硬件感知和模块组合优化,强调在实际部署环境中的综合收益

如果按本案例的应用视角看,这些版本的差异意味着:

- v1 更适合讲轻量化网络的最基础结构
- v2 更适合讲轻量网络中的残差和瓶颈设计
- v3 更适合讲轻量网络与注意力机制、激活函数优化的结合
- v4 更适合讲面向部署性能的后继演化方向

2.5.5. MobileNet 与 ResNet 骨干在本案例中的取向差异

本案例同时提供 MobileNet 和 ResNet 两条骨干路线,这样可以更直观地对比不同网络家族的使用取向。

MobileNet 系列的特点是:

- 参数量更小
- 推理速度更快
- 更适合实时演示和边缘部署

ResNet 系列的特点是:

- 特征提取能力更强
- 网络层次更深,残差结构更成熟
- 在一些场景下更容易得到稳定的检测效果

因此,在本案例中:

- 如果更关注实时性和轻量部署,可以优先尝试 MobileNet v1-v4
- 如果更关注骨干网络深度和表达能力,可以尝试 ResNet18 到 ResNet151 的多个版本

2.5.6. SSD 的检测特点

SSD 是 Single Shot MultiBox Detector 的缩写，属于典型的一阶段检测器。它的核心特点是：不需要像两阶段检测器那样先生成候选区域，再做分类与回归，而是直接在不同尺度的特征图上预测目标位置和类别。

SSD 的关键思想包括：

- 在多个尺度特征图上进行检测，兼顾大目标和小目标。
- 在每个位置预先定义一组 default boxes，也叫 prior boxes。
- 网络直接回归 default box 的位置偏移量，同时预测各类别分数。
- 经过解码和 NMS 后，输出最终的检测框。

SSD 的优点主要有：

- 推理速度快，适合实时场景。
- 结构相对直接，便于理解和部署。
- 检测头设计清晰，容易解释“分类”和“回归”两个分支的作用。

SSD 的局限也需要明确说明：

- 对小目标和密集目标场景通常不如更新的检测器稳定。
- 检测效果较依赖先验框设计与输入尺寸。
- 在复杂遮挡场景下，单帧检测质量会波动，从而影响后续跟踪。

2.5.7. MobileNet-SSD 用于目标跟踪的优缺点

在目标跟踪任务中，检测器不是孤立存在的，它会直接影响轨迹质量。MobileNet-SSD 作为跟踪前端，有以下优点：

- 轻量高效，适合边缘侧实时处理。
- 输出结构标准，便于接入后续跟踪器。
- 检测速度较快，可以为多目标跟踪提供稳定的帧级观测。
- 工程复杂度低，便于构建完整案例。

但它也存在明显不足：

- 对小目标和远距离目标的检测精度有限。
- 遇到密集遮挡时，漏检和框抖动会增多。
- 当检测框位置不稳定时，跟踪器更容易发生 ID switch。
- 仅靠检测框几何信息，难以应对长时间遮挡或重识别问题。

因此，MobileNet-SSD 的价值不在于“它是最强的检测前端”，而在于“它能以较低复杂度，

把检测到跟踪这条主线讲清楚”。

2.6. DeepSORT 作为跟踪后端的选择依据

2.6.1. 从检测到跟踪，还缺少什么

检测器每一帧都输出一组目标框，但这些框之间没有时间关联。要把检测结果变成稳定轨迹，还需要以下机制：

- 轨迹状态表示。
- 帧间运动预测。
- 检测与轨迹之间的匹配。
- 轨迹的创建、保留和删除。

DeepSORT 恰好提供了这样一套结构完整的解决框架。

2.6.2. DeepSORT 为什么适合作为入门算法

完整的 DeepSORT 在 SORT 基础上增加了外观特征建模，因此在目标遮挡、交叉和重识别问题上通常具有更好的稳定性。尽管本案例并未实现完整的工业级 DeepSORT，但保留了其最核心、最适合入门阶段把握的基本思路：

- 用卡尔曼滤波做运动预测。
- 用匈牙利算法做全局匹配。
- 用 IOU 作为主要几何相似度。
- 用轨迹生命周期参数管理目标的出现与消失。

这种简化版方案有三个突出优点：

- 思路完整，已经覆盖多目标跟踪中的核心环节。
- 数学和工程难度适中，不需要先掌握 ReID 网络训练。
- 便于直接观察检测质量、IOU 阈值、轨迹寿命等因素对最终效果的影响。

因此，将 MobileNet-SSD 与 DeepSORT 组合使用，是一条结构清晰、实现成本适中且便于展开分析的入门路径。

2.7. 本案例的代码结构与总体流程

本案例的工程代码主要分为三层：

2.7.1. 入口层

- `scripts/detection_app.py`: 只做实时检测。
- `scripts/tracking_app.py`: 先做检测，再做跟踪。

2.7.2. 检测层

- `ssdlite/backend_base.py`: 统一检测后端基类，负责预处理、推理调度与输出解码。
- `ssdlite/cpu_backend.py`: ONNXRuntime CPU 推理实现。
- `ssdlite/npu_backend.py`: Ascend ACL NPU 推理实现。
- `ssdlite/decoder.py`: SSD prior boxes、位置解码、类别概率与 NMS。

2.7.3. 跟踪层

- `tracking/deepsort.py`: 轨迹对象、匹配逻辑、轨迹生命周期管理。
- `tracking/kalman_filter.py`: 卡尔曼滤波状态预测与观测更新。
- `utils/postprocessing.py`: 检测结果转换为跟踪器输入，并将轨迹画回图像。

整条链路的执行顺序可以概括为：

1. 读取摄像头或视频流。
2. 对输入帧执行 MobileNet-SSD 检测。
3. 解码输出张量并得到边界框、类别和分数。
4. 将检测结果整理成跟踪器统一输入格式。
5. 对已有轨迹进行预测。
6. 将当前检测结果与历史轨迹做关联。
7. 更新匹配轨迹、创建新轨迹、删除失效轨迹。
8. 在图像上绘制检测结果或轨迹结果。

2.8. 检测部分代码解析

2.8.1. `detection_app.py` 如何组织检测主流程

在 `scripts/detection_app.py` 中，主程序首先解析参数，然后根据 `device` 选择后端，再进入逐帧推理循环。这一设计体现了很清晰的工程结构：入口脚本只负责流程编排，把模型相关细节下沉到 `ssdlite` 模块。

主流程的示意代码如下：


```

1 labels = load_labels(args.labels)
2 model_path = resolve_model_path(args.model, args.backbone, model_dir, args.
  ↪ device)
3 backend = CpuBackend(model_path) or NpuBackend(model_path)
4
5 while True:
6     ok, frame = cap.read()
7     detections, profile_ms = backend.infer_with_profile(
8         frame,
9         args.score_threshold,
10        args.nms_threshold,
11        args.max_detections,
12    )
13    annotated = draw_detections(...)

```

这里需要关注的重点并非某一行语法细节，而是模块职责的划分方式：

- 入口脚本负责参数、视频流、显示和保存。
- backend 负责模型推理。
- decoder 负责把原始张量变成检测框。
- postprocessing 负责可视化。

这是一种很清晰的工程拆分方式。

2.8.2. backend_base.py 如何统一检测后端

ssdlite/backend_base.py 是检测部分最核心的代码之一。它把 CPU 与 NPU 两种后端统一到同一个抽象接口下。这样设计后，入口脚本就不用关心底层到底是 ONNXRuntime 还是 ACL。

这个基类做了三件关键事情：

- 根据模型名推断输入尺寸。
- 统一图像预处理。
- 统一输出解码流程。

其中，infer_with_profile 的核心逻辑如下：

```

1 input_tensor = preprocess_frame(frame, self.input_hw)
2 outputs = self._run_model(input_tensor)
3 detections = decode_detections(
4     outputs,
5     frame.shape,
6     score_threshold,
7     nms_threshold,
8     max_detections,
9     self.ssd_decoder,
10    self.strict_ssd,
11 )

```

这段代码清晰地对应了检测系统中的三个阶段：

1. 预处理：把原始图像缩放、归一化，并变成模型要求的 NCHW 张量。
2. 推理：调用不同后端执行前向计算。
3. 解码：把输出张量还原成真实的目标框与类别分数。

这里需要明确的一点是，模型推理本身并不等于最终检测结果。真正可用于后续跟踪的目标框，需要由模型输出经过解码与 NMS 后处理后才能得到。

2.8.3. preprocess_frame 体现了部署输入规范

在 backend_base.py 中，preprocess_frame 完成了以下操作：

- BGR 转 RGB。
- resize 到固定输入尺寸。
- 像素值归一化到 0 到 1。
- 按 ImageNet 风格均值与方差做标准化。
- 从 HWC 转为 CHW。
- 增加 batch 维度。

这说明了一个重要的部署原则：模型输入的预处理流程必须与训练阶段的约定保持一致，否则推理结果将发生明显偏移。

2.8.4. decoder.py 如何体现 SSD 的核心思想

ssdlite/decoder.py 直接对应了 SSD 的理论结构，是整条检测链路中的关键部分。

default boxes

在 DefaultBoxes 类中，代码根据不同特征图尺度和长宽比生成先验框。这正是 SSD 的基础：网络并不是直接从零开始生成框，而是在预定义框基础上学习偏移量。

位置回归的反变换

在 SSDDecoder.scale_back_batch 中，网络输出的并不是最终框坐标，而是相对于默认框的偏移量。代码中先利用 scale_xy 和 scale_wh 对偏移量进行还原，再把中心点形式转换为左上角和右下角坐标。

更直接地说：

- 网络先预测相对于先验框的中心偏移和宽高缩放。
- 解码阶段再把这些相对量恢复成图像上的实际边界框。

分类分数与 NMS

当前工程版本中的 `decode_single` 已经不是“对每个类别都扫描全部 `prior`”的基础写法，而是先对每个 `prior` 选出一个最佳前景类别，再进入后续筛选和 NMS。其主线思路更接近下面这样：

```
1 foreground_scores = scores_in[:, class_ids]
2 best_class_indices = np.argmax(foreground_scores, axis=1)
3 best_scores = foreground_scores[np.arange(foreground_scores.shape[0]),
  ↪ best_class_indices]
4 keep_score = best_scores >= score_threshold
5
6 candidate_boxes = bboxes_in[keep_score, :]
7 candidate_scores = best_scores[keep_score]
8 candidate_labels = class_ids[best_class_indices[keep_score]]
```

随后，代码再按类别分别执行 NMS。这样做体现了 SSD 后处理中的三个关键工程点：

- 先为每个 `prior` 找到最可信的前景类别。
- 再根据阈值筛掉低置信度候选框。
- 最后只对保留下来的候选框按类别做 NMS。

如果仍然采用“每个类别都扫全部 `prior`”的方式，那么在 COCO 这类多类别场景里，解码阶段会产生大量无效扫描和重复 NMS，实时性会明显受影响。

NMS 的作用是什么

NMS 是 Non-Maximum Suppression，即非极大值抑制。它的核心作用可以概括为一句话：当多个候选框同时指向同一个目标时，只保留其中最可信的候选框，并删除其余高度重叠的框。

这是因为 SSD 会在每个特征图位置、每种先验框形状上产生候选框。对于一个真实目标，模型往往不会只输出一个框，而是会输出一组位置接近、分数相近的候选框。如果不执行 NMS，画面中就会出现大量相互覆盖的检测框，例如同行人周围同时叠加 5 个到 10 个候选框。

NMS 的作用主要有四点：

- 去除重复检测，让一个目标尽量只保留一个主框。
- 降低后续显示和统计的混乱程度。
- 为跟踪器提供更稳定、更稀疏的观测输入。
- 避免同一目标在跟踪阶段被误认为多个目标。

对本案例尤其要强调最后一点。如果检测器把同一目标输出成多个高重叠框，那么 `tracking` 模块在创建轨迹时，可能会把这些框错误地当成多个独立目标，从而造成重复轨迹、ID 混乱和后续匹配不稳定。

NMS 的处理逻辑可以概括成下面 4 步：

1. 按置信度从高到低排序所有候选框。
2. 取当前分数最高的框作为保留框。

3. 计算其余框与该保留框的 IOU。
4. 删除所有 IOU 超过阈值的框，然后继续处理剩余框。

因此，NMS 本质上是在做“去重”。它不是提高分类能力，而是在后处理阶段把重复候选框压缩成更干净的检测结果。

ssdlite 中 _nms 的实现解析 {#src-experiment-case2-h33}

当前工程版本中的 NMS 函数如下：

```

1 def _nms(bboxes, scores, iou_threshold, presorted=False):
2     if len(bboxes) == 0:
3         return np.empty((0,), dtype=np.int64)
4
5     if presorted:
6         order = np.arange(scores.size, dtype=np.int64)
7     else:
8         order = np.argsort(scores)[::-1]
9
10    keep = []
11    while order.size > 0:
12        index = order[0]
13        keep.append(index)
14        if order.size == 1:
15            break
16
17        remaining = order[1:]
18        iou = _calc_iou_with_box(bboxes[remaining], bboxes[index])
19        order = remaining[iou <= iou_threshold]
20
21    return np.array(keep, dtype=np.int64)

```

这段实现虽然短，但几乎完整体现了经典 NMS 的核心流程。

第一步，处理空输入：

```

1 if len(bboxes) == 0:
2     return np.empty((0,), dtype=np.int64)

```

这一步是工程上必不可少的防御式处理。如果当前类别一个候选框都没有，就直接返回空索引，避免后续排序和 IOU 计算报错。

第二步，确定候选框顺序：

```

1 if presorted:
2     order = np.arange(scores.size, dtype=np.int64)
3 else:
4     order = np.argsort(scores)[::-1]

```

这里多了一个 `presorted` 参数。其作用是：如果外部已经按分数把候选框排好了，就不必在 NMS 内部重复排序。这样可以减少一次多余的排序开销。

第三步，循环保留最高分框：

```

1 keep = []
2 while order.size > 0:

```

```

3     index = order[0]
4     keep.append(index)

```

这里的含义至关重要：每一轮循环都将当前最高分框作为“代表框”保留下来。NMS 的基本原则是，在一组高度重叠的候选框中，分数最高的框通常最应优先保留。

第四步，只剩一个框时直接结束：

```

1 if order.size == 1:
2     break

```

如果当前只剩一个候选框，那么它已经被保留，没有必要再做 IOU 比较。

第五步，计算其余候选框与当前最高分框的 IOU：

```

1 remaining = order[1:]
2 iou = _calc_iou_with_box(boxes[remaining], boxes[index])

```

这一步可以这样理解：

- `boxes[index]` 是当前保留框。
- `boxes[remaining]` 是除了它以外剩余的候选框。
- `_calc_iou_with_box(...)` 直接计算“多框对单框”的 IOU。

如果某个候选框与保留框的 IOU 很高，就说明这两个框大概率描述的是同一个目标，此时就没有必要同时保留。

第六步，删除 IOU 过高的重复框：

```

1 order = remaining[iou <= iou_threshold]

```

这一行代码是整个 NMS 实现中最关键的语句之一。它表示：

- 只保留那些与当前主框重叠不太严重的候选框。
- 不再通过 `np.where(...)+1` 做额外索引映射，而是直接对 `remaining` 做布尔筛选。

执行完这一行后，所有与当前主框高度重叠的框都会被移除，剩下的框继续进入下一轮循环。

第七步，返回保留框索引：

```

1 return np.array(keep, dtype=np.int64)

```

最终返回的不是框本身，而是被保留框在原数组中的索引。后续代码再用这些索引去提取真正的框、分数和类别。

calc_iou 为什么是 NMS 的基础

NMS 能否正确工作，取决于 IOU 计算是否准确。本案例在 `decoder.py` 中通过 `calc_iou` 计算两个框集合之间的交并比。其核心思想是：

$$IOU = \frac{\text{交集面积}}{\text{并集面积}}$$

在代码里，这一过程分为三步：

- 先求两个框相交区域的左上角和右下角。
- 再计算交集面积。
- 最后用交集面积除以并集面积。

如果 IOU 接近 1，说明两个框几乎重合；如果 IOU 接近 0，说明两个框几乎没有重叠。NMS 正是利用这一数值来判断“两个候选框是不是在描述同一个目标”。

NMS 阈值对结果的影响

NMS 中最重要的超参数是 `iou_threshold`。它决定了“多大程度的重叠应被视为重复”。

如果阈值较小，例如 0.3，那么算法会更严格：

- 只要两个框稍微重叠得多一点，就可能删除其中一个。
- 输出框更少，更干净。
- 但也可能误删本来相邻但不是同一目标的框。

如果阈值较大，例如 0.6 或 0.7，那么算法会更宽松：

- 允许更多重叠框同时保留。
- 对密集目标场景可能更友好。
- 但重复检测也更容易残留。

因此，NMS 阈值本质上是在“去重强度”和“保留相邻目标”之间做权衡。

NMS 对跟踪效果的直接影响

NMS 容易被视为检测模块内部的技术细节，但在目标跟踪系统中，它实际上会直接影响跟踪质量。

原因是跟踪器把检测器输出当作观测输入。如果检测结果中存在大量重复框，就会产生以下问题：

- 同一目标可能在同一帧中生成多条新轨迹。
- 轨迹关联时会出现多对一竞争。
- 轨迹数目虚高，画面看起来混乱。
- 目标 ID 容易抖动，甚至频繁切换。

从这个角度看，NMS 不是孤立的后处理技巧，而是整个“检测到跟踪”流水线稳定运行的重要保障。

decode_single 中 NMS 的完整上下文

在本案例中，NMS 不是单独执行的，而是被放在 decode_single 的类别循环中。其完整语义是：

1. 针对每一个类别单独处理。
2. 先按分数过滤掉置信度过低的框。
3. 再保留该类别中得分最高的一部分候选框。
4. 最后在这个类别内部做 NMS。

这意味着本案例采用的是“按类别分别做 NMS”的策略。这样设计的优点是：不同类别之间不会互相抑制。例如一个行人框和一个自行车框即使重叠较大，也不应该因为彼此重叠就被删除。

2.8.5. CPU 与 NPU 后端如何被统一接入

本案例中的 ssdlite/cpu_backend.py 与 ssdlite/npu_backend.py 都继承自 DetectionBackend，这意味着它们对外暴露的是同一套接口。这种设计有两层价值：

- 从算法角度看，检测逻辑不依赖具体硬件后端。
- 从工程角度看，同一套上层代码可以在 CPU 和 Ascend NPU 之间切换。

这也体现了一个重要的工程原则：算法主线尽量稳定，硬件适配尽量下沉到后端实现层。

2.9. 从检测到跟踪：tracking_app.py 的桥接作用

如果说 detection_app.py 用于建立检测流程，那么 tracking_app.py 的作用就在于将“检测结果如何转化为轨迹”这一过程完整串联起来。

在 scripts/tracking_app.py 中，检测模块与跟踪模块之间的桥接代码如下：

```
1 detections, profile_ms = backend.infer_with_profile(...)
2 tracker_inputs = detections_to_tracker_inputs(detections)
3 tracks = tracker.update(tracker_inputs)
4 annotated = draw_tracks(...)
```

这一过程可以分为四个连续步骤：

1. 先由检测器输出本帧的目标集合。
2. 再把字典形式的检测结果转成跟踪器统一使用的数组格式。
3. 调用跟踪器更新轨迹状态。
4. 把最新轨迹结果绘制到图像上。

其中，utils/postprocessing.py 中的 detections_to_tracker_inputs 函数具有关键作用，因为它完成了检测模块与跟踪模块之间的数据接口统一。转换后的输入格式为：

$$[x_1, y_1, x_2, y_2, score, class_id]$$

这正是后续轨迹匹配与分类约束所依赖的输入格式。

2.10. 跟踪部分代码解析

2.10.1. Track 类如何表示一条轨迹

tracking/deepsort.py 中的 Track 类代表单个目标的历史状态。每条轨迹至少维护了以下信息：

- track_id: 目标编号。
- bbox: 当前边界框。
- score: 当前检测置信度。
- class_id: 目标类别。
- trail: 历史中心点序列，用于绘制轨迹拖尾。
- kalman_filter: 用于状态预测与更新。
- time_since_update: 距离上次匹配成功已经过去多少帧。
- hits: 这条轨迹累计被成功匹配了多少次。

这表明轨迹并不是静态边界框的简单记录，而是一个具有时间属性的状态对象。因此，“检测框”和“轨迹”必须明确区分。

2.10.2. predict 和 update 对应卡尔曼滤波两阶段

Track 类中最核心的两个方法是 predict 和 update。

predict 的作用是：即使当前帧还没有拿到新的检测结果，也先根据上一时刻状态预测目标现在大概会出现在哪里。

update 的作用是：一旦当前帧有新的检测框匹配到这条轨迹，就用新的观测值修正状态估计。

这正对应卡尔曼滤波的两步：

1. Predict: 根据上一时刻状态外推当前状态。
2. Update: 用当前观测修正预测误差。

本案例里，状态量采用了一个比较直观的简化形式：

$$x = [x, y, v_x, v_y]^T$$

其中 x, y 表示目标中心位置， v_x, v_y 表示速度。这样做的好处是：数学含义直观，代码实现也容易读懂。

2.10.3. kalman_filter.py 如何实现线性状态估计

tracking/kalman_filter.py 中定义了最基础的卡尔曼滤波器。其状态转移矩阵为：

$$F = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

这表示系统假设目标在相邻帧之间近似满足匀速运动。观测矩阵为：

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

这表示我们能够直接观测到的是目标中心位置，而不是速度。

这一实现虽然经过简化，但其价值十分明确，因为它将“运动预测”与“观测修正”这两个概念直接落实到了矩阵运算之中。

2.10.4. 卡尔曼滤波器的发展历史与作者信息

卡尔曼滤波器的提出者是鲁道夫·埃米尔·卡尔曼，英文名 Rudolf Emil Kalman。他是匈牙利裔美国数学家和控制理论学者，1960 年发表了著名论文 A New Approach to Linear Filtering and Prediction Problems，系统建立了线性动态系统状态估计的现代形式。

从历史背景看，卡尔曼滤波器并不是突然出现的。它的形成与 20 世纪中叶的控制理论、随机过程、航空航天导航和军事工程需求密切相关。第二次世界大战后，工程界越来越需要解决这样一类问题：

- 系统状态不能被直接完整观测。
- 传感器观测存在噪声。
- 系统本身还在不断运动和变化。

例如，雷达跟踪飞机、导弹制导、航天器轨道估计，本质上都属于这一类问题。

卡尔曼滤波器之所以具有里程碑意义，是因为它把“预测”和“校正”统一进了一个递推框架中：

- 先根据上一时刻状态和系统模型预测当前状态。
- 再根据当前观测对预测结果进行修正。

这一思想后来在 20 世纪 60 年代迅速应用到阿波罗计划的导航系统中，也因此被广泛传播到航空航天、自动控制、机器人、计算机视觉和金融工程等领域。

在发展过程中，卡尔曼滤波形成了多个重要分支：

- 标准卡尔曼滤波：用于线性高斯系统。

- 扩展卡尔曼滤波 EKF：用于非线性系统的一阶近似。
- 无迹卡尔曼滤波 UKF：通过采样点传播非线性分布。
- 粒子滤波：进一步放宽线性和高斯假设。

本案例采用最基础的标准卡尔曼滤波器。采用这一形式，并非因为它代表当前最复杂的状态估计方案，而是因为它在入门阶段具有最清晰的解释路径：

- 数学形式清晰。
- 与目标跟踪中的位置预测问题高度匹配。
- 代码实现短小，便于直接对应矩阵运算理解。

2.10.5. 卡尔曼滤波器到底在解决什么问题

在目标跟踪里，卡尔曼滤波器要解决的问题可以表述为：当检测器每一帧给出的目标框存在抖动、漏检和噪声时，如何根据历史状态更平滑地估计目标当前位置，并在短时没有观测的情况下继续维持轨迹。

这意味着卡尔曼滤波并不是在做“目标分类”，也不是在做“目标匹配”，它做的是状态估计。在本案例里，这个状态估计问题被简化成了二维平面上的匀速运动模型：

$$x_k = Fx_{k-1} + w_k$$

$$z_k = Hx_k + v_k$$

其中：

- x_k 表示第 k 帧的隐藏状态，也就是目标真实但不可完全直接观测的状态。
- z_k 表示第 k 帧的观测，也就是检测器给出的目标中心位置。
- F 是状态转移矩阵。
- H 是观测矩阵。
- w_k 是过程噪声。
- v_k 是观测噪声。

从工程角度看，这个模型表达的是一个朴素但有效的假设：目标会以相对平滑的方式运动，而检测器给出的观测值只是对真实运动状态的带噪声测量。

2.10.6. 本案例中的 predict 与 update 如何对应算法公式

`tracking/kalman_filter.py` 中的两个核心函数正好对应卡尔曼滤波的两个阶段。

第一阶段是预测：

```

1 def predict(self):
2     self.x = self.F @ self.x
3     self.P = self.F @ self.P @ self.F.T + self.Q
4     return self.x

```

该阶段完成以下两项操作：

- 用状态转移矩阵 F 预测新的状态向量 x 。
- 用协方差传播公式预测新的不确定性 P 。

第二阶段是更新：

```

1 def update(self, z):
2     measurement = np.asarray(z, dtype=np.float32).reshape((2, 1))
3     innovation = measurement - self.H @ self.x
4     innovation_covariance = self.H @ self.P @ self.H.T + self.R
5     kalman_gain = np.linalg.solve(innovation_covariance.T, (self.P @ self.H.T).T
6     ↪ ).T
7     self.x = self.x + kalman_gain @ innovation
8
9     correction = self._identity - kalman_gain @ self.H
10    self.P = correction @ self.P @ correction.T + kalman_gain @ self.R @
11    ↪ kalman_gain.T

```

这段代码中几个量的意义需要讲清楚：

- `innovation` 是残差，表示“观测值与预测值之间的差”。
- `innovation_covariance` 是残差协方差，表示当前误差的不确定性。
- `kalman_gain` 是卡尔曼增益，决定“应该更相信模型预测还是更相信当前观测”。

当前实现和基础版相比还多了两点工程优化：

- 用 `np.linalg.solve` 代替直接求逆，数值更稳定。
- 用 **Joseph form** 更新协方差矩阵，更容易保持协方差的对称性和正定性。

这里最重要的量仍然是卡尔曼增益。它并非经验性设定的固定权重，而是由当前状态不确定性与观测噪声共同决定的自适应权重。

可以直观地理解为：

- 如果观测噪声很大，滤波器会更信任模型预测。
- 如果观测噪声较小，滤波器会更信任当前检测结果。

这也是卡尔曼滤波器的重要优势之一。它并不是简单的滑动平均，而是在概率意义上进行最优线性估计。

2.10.7. 卡尔曼滤波在本案例中的适用性

对本案例而言，卡尔曼滤波器之所以合适，并不在于目标运动一定严格符合线性高斯模型，而在于它在工程上实现了较好的综合平衡：

- 算法复杂度低。
- 推理和更新速度快。
- 能明显改善检测框抖动。
- 在短时遮挡和漏检场景下能维持轨迹连续。

如果在入门阶段直接引入 EKF、UKF 或粒子滤波，虽然理论上能够处理更复杂的运动模型，但理解成本会显著上升，主线结构反而更不易把握。因此，本案例采用标准卡尔曼滤波具有充分的合理性。

2.10.8. DeepSORT.update 如何体现多目标跟踪主线

tracking/deepsort.py 中的 DeepSORT.update 是整条跟踪主线最核心的函数，其执行顺序如下：

```

1 for track in self.tracks:
2     track.predict()
3
4 matched, unmatched_detections, _ = self._associate(detections)
5
6 for track_idx, detection_idx in matched:
7     self.tracks[track_idx].update(detections[detection_idx])
8
9 for detection_idx in unmatched_detections:
10    self._create_track(detections[detection_idx])
11
12 self.tracks = [track for track in self.tracks if track.time_since_update <= self
    ↪ .max_age]
```

这一段代码几乎完整体现了多目标跟踪的标准流程：

1. 对所有旧轨迹做预测。
2. 把当前检测结果与旧轨迹做关联。
3. 用匹配到的检测结果更新旧轨迹。
4. 对未匹配检测创建新轨迹。
5. 删除长期未更新的轨迹。

换言之，跟踪并不是对检测框进行简单编号，而是在连续帧之间持续执行预测、匹配、更新与清理。

2.10.9. 数据关联为什么使用 IOU + 匈牙利算法

在 `_associate` 中，当前工程版本会先构造轨迹与检测之间的 IOU 矩阵和类别兼容矩阵，再调用匈牙利算法获得全局最优分配。对应代码思路如下：

```
1 iou_matrix = self._calculate_iou_matrix(detections)
2 class_mask = self._calculate_class_compatibility_matrix(detections)
3 assignment_scores = np.where(class_mask, iou_matrix, -1.0)
4 matched_indices = self._linear_assignment(assignment_scores)
```

之所以这样设计，是因为当场景中有多条轨迹和多个检测框时，不能只做局部贪心匹配。匈牙利算法的作用在于：

- 从全局角度寻找一一对应的最优匹配。
- 避免一个检测框同时分配给多条轨迹。
- 避免一条轨迹同时匹配多个检测框。

本案例还加入了类别兼容矩阵：

```
1 class_mask = self._calculate_class_compatibility_matrix(detections)
2 assignment_scores = np.where(class_mask, iou_matrix, -1.0)
```

这说明系统不仅看几何位置，还会在进入全局匹配之前，先利用类别信息屏蔽明显不合理的配对关系，避免“行人轨迹匹配到车辆框”这类明显错误。

2.10.10. 匈牙利算法的发展历史与作者信息

匈牙利算法是解决二分图最优匹配和指派问题的经典算法。它最早由美国数学家哈罗德·W·库恩，英文名 Harold W. Kuhn，在 1955 年系统提出。之所以命名为“匈牙利算法”，是因为库恩的工作建立在两位匈牙利数学家的理论成果之上：

- 德奈什·克尼格，英文名 Dénes Kőnig。
- 耶诺·埃格瓦里，英文名 Jenő Egerváry。

他们在图论和指派问题上的研究，为后来算法化求解最优匹配提供了重要理论基础。之后，美国数学家詹姆斯·芒克雷，英文名 James Munkres，在 1957 年进一步改进了该算法，使其在计算复杂度和工程实现上更加完善。因此，这一算法也常被称为 Kuhn-Munkres 算法。

从历史背景看，匈牙利算法主要服务于一类经典的组合优化问题：

- 一组任务要分配给一组工人。
- 每一种分配都有代价或收益。
- 目标是在一一对应约束下，使总代价最小或总收益最大。

这个问题后来被称为 assignment problem，即指派问题。它在生产调度、资源分配、路径规划、视觉跟踪等场景中都较为常见。

在多目标跟踪中，匈牙利算法的角色就是把“轨迹”和“当前检测框”看成两侧节点，把相似度或代价矩阵看成边权，然后求出全局最优的一一匹配关系。

2.10.11. 匈牙利算法到底解决什么问题

在目标跟踪里，如果只有一条轨迹和一个检测框，那么匹配很简单。但一旦场景中存在多条轨迹和多个检测框，问题就会迅速复杂起来。

例如，假设当前有 3 条旧轨迹和 3 个新检测框。此时系统不能只看“哪两个最像就先配对”，因为局部最优未必能得到全局最优。匈牙利算法解决的就是这个问题：

- 它在满足一一对应约束的前提下，寻找整体代价最小或整体相似度最大的匹配方案。

这正是多目标跟踪里数据关联的核心。

2.10.12. 本案例中匈牙利算法是如何被调用的

在 `tracking/deepsort.py` 中，匈牙利算法的调用被封装在 `_linear_assignment` 中：

```
1 def _linear_assignment(self, cost_matrix):
2     if cost_matrix.size == 0:
3         return np.empty((0, 2), dtype=np.int64)
4
5     row_ind, col_ind = linear_sum_assignment(-cost_matrix)
6     return np.array(list(zip(row_ind, col_ind)), dtype=np.int64)
```

这里调用的是 `scipy.optimize.linear_sum_assignment`，它是匈牙利算法在科学计算库中的标准实现。

该函数默认求解最小化问题，而本案例构造的是 IOU 相似度矩阵，IOU 越大表示匹配越优。因此，代码中采用了一个典型的等价变换：

```
1 linear_sum_assignment(-cost_matrix)
```

即将“最大化相似度”转化为“最小化负相似度”。

这里需要强调的一点是，同一个优化算法既可以用于最小代价问题，也可以通过简单变换用于最大收益问题。

2.10.13. 匈牙利算法在本案例中的完整上下文

在 `_associate` 中，数据关联的整体过程是：

1. 先计算每一条轨迹与每一个检测框之间的 IOU。
2. 再构造类别兼容矩阵，屏蔽明显不合理的类别组合。
3. 调用匈牙利算法得到全局最优分配。
4. 对主匹配结果应用 IOU 阈值过滤。

5. 对未匹配目标再执行一轮基于中心距离的补充匹配。

对应代码主线是：

```

1 iou_matrix = self._calculate_iou_matrix(detections)
2 class_mask = self._calculate_class_compatibility_matrix(detections)
3 assignment_scores = np.where(class_mask, iou_matrix, -1.0)
4 matched_indices = self._linear_assignment(assignment_scores)
5
6 for track_idx, detection_idx in matched_indices:
7     if assignment_scores[track_idx, detection_idx] < self.iou_threshold:
8         continue
9
10 fallback_matches = self._match_by_center_distance(...)
```

这段代码说明一个重要事实：匈牙利算法本身只负责“给出最优一一分配”，但它并不直接理解哪些匹配在语义上是合理的。因此，本案例又叠加了两层约束：

- 类别必须兼容。
- IOU 必须高于阈值。
- 当 IOU 不足但中心运动关系合理时，还允许进入中心距离补充匹配。

由此可见，匈牙利算法提供的是全局分配框架，而不是完整的业务规则集合。

2.10.14. 匈牙利算法相对于贪心匹配的优势

一个常见问题在于，为什么不直接选择当前最大 IOU 的一对进行匹配。该贪心策略看似简单，但存在明显局限：

- 先做出的局部选择可能破坏后续整体最优。
- 当多个轨迹都与同一个检测框接近时，容易造成冲突。
- 在目标密集场景下，局部贪心更容易导致 ID 交换。

匈牙利算法的价值就在于它能从全局角度处理这类一一匹配问题。虽然本案例的匹配度量比较简单，只用了 IOU，但只要场景中存在多目标竞争，使用全局分配通常都比局部贪心更稳健。

2.10.15. 匈牙利算法在多目标跟踪中的局限

需要说明的是，匈牙利算法并不是万能的。它解决的是“在给定价矩阵时，如何求最优匹配”，但如果代价矩阵本身构造得不好，算法仍然可能得到错误匹配。

在本案例中，主匹配代价矩阵主要由 IOU 构成，因此它会受到以下问题影响：

- 快速运动目标与旧轨迹重叠变小，IOU 不足。
- 相邻目标位置太近，几何信息不够区分。
- 遮挡后重新出现时，仅靠 IOU 难以恢复原身份。

这也是为什么当前工程版本又额外加入了“类别兼容矩阵”和“中心距离补充匹配”。匈牙利算法负责的是“最优分配”，但“什么样的代价矩阵才合理”仍然是跟踪算法设计的关键。

2.10.16. 轨迹生命周期参数如何影响最终效果

本案例中的三组参数具有较强的实验分析价值：

- `max_age`：轨迹最长允许失配多少帧。
- `min_hits`：轨迹至少命中多少次后才输出。
- `iou_threshold`：检测与轨迹关联所需的最小 IOU。

这些参数分别控制三类问题：

- 轨迹是否容易过早消失。
- 新轨迹是否容易过快出现。
- 匹配是否保守还是激进。

通过调参可以直接观察以下现象：

- `max_age` 过小，目标短时遮挡后容易断轨。
- `min_hits` 过小，误检容易变成短暂轨迹。
- `iou_threshold` 过高，快速移动目标更容易失配。
- `iou_threshold` 过低，邻近目标更容易错配。

2.11. 可视化代码如何帮助理解算法结果

在 `utils/postprocessing.py` 中，`draw_detections` 与 `draw_tracks` 分别负责绘制检测结果和跟踪结果。尽管这部分并不直接构成算法主体，但其作用十分重要，因为可视化能够将抽象状态转化为可直接观察的现象。

其中，`draw_tracks` 中有两点尤其值得关注：

- 每条轨迹使用固定颜色，帮助观察 ID 是否稳定。
- `trail` 轨迹拖尾可以直观看出运动路径与预测连续性。

因此，在调节参数时，不仅能够观察检测框是否正常输出，还能够观察 ID 是否发生跳变，以及轨迹是否保持连续。

2.12. 如何评价这个案例方案

2.12.1. 这个方案的优点

作为昇腾 310B 开发教程中的案例，本方案有很强的适配性：

- 模型轻量，适合边缘侧实时实验。
- 检测和跟踪两个阶段边界清晰，便于分层讲解。
- 代码结构整洁，便于把入口、后端、解码、跟踪拆开讲。
- DeepSORT 采用简化实现，避免课程一开始就陷入 ReID 细节。
- 既能跑 CPU，也能跑 NPU，便于体现部署层面的思维。

2.12.2. 这个方案的不足

需要明确的是，本案例并不是为了追求最强跟踪精度：

- 检测前端仍是经典 SSD，精度不是当前最先进水平。
- 跟踪器没有使用外观特征，因此遮挡恢复能力有限。
- 数据关联主要依赖 IOU，对交叉目标和密集目标不够鲁棒。
- 卡尔曼状态建模较简化，更偏向实现可解释性而不是工业最优效果。

2.12.3. 作为入门案例的合理性

正因为它没有引入过多复杂部件，目标跟踪的基本问题反而更容易被清晰呈现：

- 检测结果是如何来的。
- 检测结果如何进入跟踪器。
- 为什么需要预测。
- 为什么需要全局匹配。
- 为什么轨迹管理会决定最终 ID 稳定性。

当这些基础概念建立起来后，再引入完整 DeepSORT、ByteTrack、OC-SORT 或更复杂的 ReID 模块，学习曲线会平缓很多。

2.13. 实验设计

为了把本案例组织成一章完整教程，可以安排以下实验任务：

2.13.1. 只运行检测链路

目标：理解 MobileNet-SSD 的输入输出与后处理。

可观察：

- 不同模型输入尺寸对速度和效果的影响。
- `score_threshold` 对检测框数量的影响。
- NMS 阈值对重复框抑制效果的影响。

推荐命令：

```
1 python scripts/detection_app.py --device npu --source 0
2 python scripts/detection_app.py --device npu --source 0 --backbone
  ↳ mobilenetv2_100
3 python scripts/detection_app.py --device npu --source 0 --backbone resnet18
```

实验提示：

- 对比 `ssd320_*` 和 `ssd300_*` 两类模型时，不仅要看 FPS，还要同时观察检测框数量和画面稳定性。
- 当 `score-threshold` 较低时，解码后的候选框数会明显增加，后处理负担也会增大。
- 当 `nms-threshold` 较大时，重复框更容易残留，后续 `tracking` 的输入质量也会受到影响。

2.13.2. 检测链路的性能拆分实验

目标：学会区分摄像头读取、预处理、推理、解码和绘制的性能瓶颈。

可观察：

- Read：摄像头或视频流读取时间。
- Pre：模型输入预处理时间。
- Infer：模型推理时间。
- Decode：后处理时间。
- Draw：检测框或轨迹绘制时间。

推荐命令：

```
1 python scripts/detection_app.py --device npu --source 0
2 python scripts/detection_app.py --device npu --source 0 --camera-mjpeg
3 python scripts/detection_app.py --device npu --source 0 --camera-profile 1280
  ↳ x720@60
```

实验提示：

- 如果 Read 明显高于 Pre，瓶颈更可能在摄像头采集或驱动路径，而不是模型预处理本身。
- 如果 Decode 偏高，可以优先降低 `max-detections` 或缩小关注类别范围。

- 如果启用 `--camera-mjpeg` 后 Read 下降, 说明当前 USB 摄像头和驱动组合更适合 MJPEG 模式; 如果反而上升, 则应恢复默认模式。
- 如果请求的分辨率或帧率不是摄像头原生档位, 驱动可能发生额外缩放或格式转换, Read 往往会明显上升。

2.13.3. 在检测基础上启用跟踪

目标: 观察轨迹是如何从检测结果中产生的。

可观察:

- 同一目标在连续帧中是否保持稳定 ID。
- 当目标短时遮挡时, 轨迹是否断裂。
- 当多个目标接近时, 是否发生 ID 交换。

推荐命令:

```
1 python scripts/tracking_app.py --device npu --source 0
2 python scripts/tracking_app.py --device npu --source 0 --track-classes person
3 python scripts/tracking_app.py --device npu --source 0 --track-classes person,
   ↪ bus
```

实验提示:

- 使用 `--track-classes` 后, 可以明显减少无关类别进入解码和跟踪流程, 便于观察指定目标的轨迹稳定性。
- 对实时摄像头场景, 建议优先用 `person` 这类高频类别做实验, 更容易观察目标进出画面、遮挡和交叉等现象。

2.13.4. 类别过滤与解码开销实验

目标: 理解“只跟踪指定类别”不仅影响画面内容, 也影响后处理负载。

可观察:

- 指定类别前后, Decode 时间是否变化。
- 指定类别前后, 检测框数量和轨迹数量是否更稳定。
- 不同类别组合对解码开销和跟踪干扰的影响。

推荐命令:

```
1 python scripts/tracking_app.py --device npu --source 0
2 python scripts/tracking_app.py --device npu --source 0 --track-classes person
3 python scripts/tracking_app.py --device npu --source 0 --track-classes person,
   ↪ bus
```

实验提示:

- 当只关心少数类别时，优先在解码阶段就做类别过滤，比“先解出全部类别再过滤”更符合实时场景。
- 如果场景中本来就几乎只有行人，那么 `--track-classes person` 的主要收益会体现在画面更干净和干扰更少，而不是极大的速度变化。

2.13.5. 调节 DeepSORT 参数

目标：理解生命周期管理与关联阈值。

可调节：

- `track_max_age`
- `track_min_hits`
- `track_iou_threshold`

推荐命令：

```
1 python scripts/tracking_app.py --device npu --source 0 --track-max-age 120
2 python scripts/tracking_app.py --device npu --source 0 --track-min-hits 3
3 python scripts/tracking_app.py --device npu --source 0 --track-iou-threshold 0.4
```

实验中应重点解释参数变化背后的原因，而不是只记录结果。

观察重点：

- `track-max-age` 过小，目标短时遮挡或漏检后容易断轨。
- `track-min-hits` 过小，误检更容易形成短暂轨迹。
- `track-iou-threshold` 过高，快速运动目标更容易匹配失败。

2.13.6. 调节中心距离与平滑参数

目标：理解补充匹配与轨迹平滑对稳定性的影响。

可调节：

- `track-center-distance-threshold`
- `track-size-smoothing`
- `track-score-smoothing`

推荐命令：

```
1 python scripts/tracking_app.py --device npu --source 0 --track-center-distance-
  ↪ threshold 2.0
2 python scripts/tracking_app.py --device npu --source 0 --track-size-smoothing
  ↪ 0.85 --track-score-smoothing 0.8
3 python scripts/tracking_app.py --device npu --source 0 --track-center-distance-
  ↪ threshold 1.4 --track-size-smoothing 0.6
```

观察重点：

- `track-center-distance-threshold` 增大后，快速运动和短时失配场景下的轨迹更容易续上，但如果过大，也可能提升误匹配概率。
- `track-size-smoothing` 增大后，框宽高更稳定，但目标尺度变化的响应会更慢。
- `track-score-smoothing` 增大后，显示分数更平滑，但对置信度突变的响应也更迟缓。

2.13.7. 组合实验建议

为了让实验更有层次，可以按下面顺序组织：

1. 先做 `detection` 基础实验，理解检测、解码和 NMS 的作用。
2. 再做性能拆分实验，判断瓶颈是在 `Read`、`Pre`、`Infer` 还是 `Decode`。
3. 然后启用 `tracking`，观察轨迹生成与 ID 稳定性。
4. 最后再做 `track-classes`、中心距离阈值和平滑参数的组合对比。

这样安排的好处是：

- 先把单帧检测问题讲清楚。
- 再把实时性能问题拆清楚。
- 最后把时序跟踪问题和参数调优联系起来。

2.14. 当前工程版本的优化说明

前文很多内容是按“教学上的基础实现思路”来讲解的，便于先把检测与跟踪主线讲清楚。当前仓库中的代码在此基础上又做了一轮面向实时性的工程优化，因此实际实现相比前文的基础版描述更进一步。阅读源码时，建议把这一节与前文结合起来看。

2.14.1. 检测链路中的工程优化

解码器不再对所有类别逐类扫描全部 `prior`

在基础版 SSD 后处理中，一个直观但开销较大的实现方式是：对每个类别都遍历全部 `prior`，然后分别做阈值筛选和 NMS。这样做虽然容易理解，但在 COCO 这类多类别场景里，后处理开销会明显增大。

当前仓库中的 `ssdlite/decoder.py` 已经改成了更适合实时场景的实现：

- 对每个 `prior` 先在候选前景类别中选出一个最佳类别。
- 只保留分数超过阈值的候选框。
- 再按类别分别做 NMS。

这样做的优点是：

- 大幅减少无效类别扫描。
- 降低候选框数量。
- 避免同一个 `prior` 在多个类别上重复进入 NMS。

这类优化尤其适合边缘设备，因为它直接减少了解码阶段的 CPU 开销。

支持在解码阶段按指定类别过滤

在当前工程版本中，`tracking_app.py` 增加了按类别跟踪的命令行参数：

```
1 python scripts/tracking_app.py --track-classes person
2 python scripts/tracking_app.py --track-classes person,bus
```

这里并不是“先把所有检测框都解出来，再在跟踪前简单过滤”，而是把允许的类别一直传递到 `decoder` 内部，让解码阶段只关注这些目标类别。这样做的意义在于：

- 可以减少不关心类别的后处理开销。
- 可以降低多类别场景中的误检测干扰。
- 可以让跟踪器专注于任务相关目标，例如只跟踪行人或公交车。

default boxes 的生成做了向量化改写

基础版的 `default boxes` 生成通常会使用多层 Python 循环去枚举中心点和长宽比，这种写法直观，但初始化时效率较低。

当前实现已经把 `DefaultBoxes` 中的中心点网格生成和尺寸组合改写成 NumPy 向量化方式，同时拆分成更清晰的辅助函数：

- `_build_layer_sizes`：负责生成当前特征层的 `anchor` 尺寸集合。
- `_build_center_coordinates`：负责生成当前特征层的网格中心点。

这样既提升了代码可读性，也减少了初始化阶段的 Python 循环开销。

SSD300 与 SSDLite320 使用不同的 default boxes 设计

当前 `decoder.py` 中保留了两个明确区分的入口：

- `dbboxes300_coco()`：对应原始 SSD300 论文风格的 `default boxes` 设计。
- `dbboxes320_coco()`：对应 `torchvision SSDLite / MobileNet-SSD` 路径使用的 `default boxes` 设计。

这并不只是输入尺寸从 300 换成 320，而是两种不同模型家族在先验框配置上的区别。把这两个入口拆开写清楚，有助于读者理解“不同 SSD 变体的 `prior boxes` 设计并不完全相同”。

IOU 与 NMS 的实现做了进一步优化

在当前工程版本中，`decoder.py` 里的 `calc_iou`、`_calc_iou_with_box` 和 `_nms` 已经做了两类优化：

- 用更直接的广播写法计算 IOU，提高可读性并减少中间变量。
- 在 NMS 中区分“多框对单框”的专用 IOU 计算路径，避免重复走更通用但更重的函数。
- 对已经按分数排序的候选框，不再重复做一次完整排序。

这说明同一个算法模块往往可以分成两种层次来理解：

- 原理层：为什么需要 IOU 和 NMS。
- 工程层：怎样把同样的逻辑写得更适合实时设备。

预处理与读帧时间被明确分开统计

在早期实验中，容易把摄像头读取时间和图像预处理时间混在一起看，从而误以为“预处理最慢”。当前版本已经在 `detection_app.py` 和 `tracking_app.py` 中把以下时间分开统计：

- Read：摄像头或视频流读取时间。
- Pre：真正的模型输入预处理时间。
- Infer：模型推理时间。
- Decode：后处理时间。
- Draw：绘制与显示时间。

这对边缘设备非常重要，因为摄像头读取、视频解码、模型预处理和模型推理往往处于不同瓶颈路径上，必须分开分析。

摄像头实时输入增加了更适合实时场景的参数

当前 `detection_app.py` 和 `tracking_app.py` 都支持：

- `--camera-profile`
- `--camera-mjpeg`
- `--no-camera-mjpeg`

其中 `--camera-profile` 用一个参数统一表达摄像头原生采集档位，例如 `1280x720@60`、`1024x576@30` 或 `@30`。这种写法比同时暴露宽、高、FPS 三个独立参数更适合教学，因为读者更容易把“摄像头采集模式”理解成一个完整组合。

其中 `--camera-mjpeg` 适用于部分 USB 摄像头。如果摄像头和驱动支持 MJPEG，启用后有时可以降低实时读取延迟。但要注意，这类参数是否有效，与具体摄像头、驱动和板端 OpenCV 构建方式密切相关，因此需要实测。

此外, 当前工程版本已经把 OpenCV 运行时辅助逻辑合并到 `utils/opencv_runtime.py` 中, 这个文件现在同时负责:

- OpenCV Qt 字体目录修复
- 摄像头启动日志与阶段计时
- V4L2 优先打开实时摄像头
- 请求较小缓冲区以减轻实时场景中的采集积压

在一组 OrangePi AI Pro / Ascend 310B 的实测中, 针对一只支持 MJPG 1280x720@60 的 USB 摄像头, 启用 V4L2 优先后端和 `buffer=1` 请求后, tracking 的显示 FPS 从约 20 提升到了约 26。这个结果并不是固定值, 但足以说明: 在边缘端实时链路中, 摄像头读帧路径本身就值得单独优化。

2.14.2. 进一步提高帧率的建议

如果在完成当前这轮采集优化后, FPS 仍然没有达到目标, 可以继续按下面顺序做进一步优化:

- 优先选择摄像头原生支持的 `camera-profile`, 避免非原生分辨率触发驱动缩放或格式转换。
- 如果目标是稳定跟踪帧率, 而不是单纯追求更大画面, 优先尝试 1024x576@30、800x448@30 这类更均衡的原生档位。
- 保留 MJPEG 用于高分辨率高帧率场景, 但要注意 Read 中也包含解码时间, 因此是否更快必须实测。
- 进一步把摄像头采集改成后台线程, 只保留最新一帧, 减少主线程推理和绘制阻塞导致的帧积压。
- 如果 Draw 时间偏高, 可以降低显示窗口尺寸、减少拖尾绘制长度, 或在无界面场景下使用 `--no-display`。
- 如果 Decode 时间偏高, 可以减小 `max-detections`, 或者优先使用 `--track-classes` 限制关注类别。
- 如果 Infer 仍是主要瓶颈, 则应从模型侧继续优化, 例如切换更轻量骨干、减小输入尺寸, 或根据任务需要降低采集分辨率。

2.14.3. 跟踪链路中的工程优化

关联前先做类别兼容性约束

基础版多目标跟踪常见的写法是: 先对所有轨迹和检测框计算 IOU, 再在匹配完成后排除类别不兼容的结果。当前版本把类别兼容性提前到了关联阶段本身, 也就是说:

- 不兼容的类别对在进入匈牙利匹配前就会被屏蔽。

- 这样可以减少无效匹配占用全局分配机会。

这种处理对多类别场景尤其重要，因为它可以有效降低“行人轨迹误匹配到车辆框”这类明显错误。

IOU 矩阵改成向量化计算

tracking/deepsort.py 中轨迹和检测之间的 IOU 矩阵，已经从双层 Python 循环改成了 NumPy 向量化实现。这样做有两个好处：

- 匹配阶段速度更快。
- 代码更容易与“矩阵形式的数据关联”这一概念对应起来。

增加基于中心距离的补充匹配

仅靠 IOU 做关联时，如果目标移动较快、检测框尺寸波动较大，或者短时遮挡后重新出现，轨迹容易断开。当前工程版本在 IOU 关联失败后，又增加了一轮基于中心距离的补充匹配：

- 先计算轨迹中心和检测框中心的归一化距离。
- 再结合类别约束，做一次补充关联。

这样可以提高短时丢检和快速移动场景下的轨迹连续性。

轨迹框尺寸与分数做了平滑

在 Track.update 中，当前版本不再每一帧都直接用新检测框完全替换轨迹框，而是对宽高和分数做指数平滑：

- size_smoothing：控制框宽高变化的平滑程度。
- score_smoothing：控制置信度显示的平滑程度。

这样做的效果是：

- 画面中的框抖动更小。
- 轨迹框大小变化更连续。
- 显示分数不容易大起大落。

轨迹类别不再只看最近一帧

当前 Track 类中维护了 class_scores，会对每个轨迹在历史匹配过程中累计类别分数，再选择当前最可信的类别。这比简单采用“最近一帧的 class_id”更稳定，尤其适合检测框偶尔分类波动的场景。

2.14.4. 卡尔曼滤波器中的工程优化

初始协方差更加符合目标跟踪场景

在基础讲解中，常把卡尔曼滤波器写成最简单的形式，例如：

- $P = I$
- $Q = 0.05 * I$
- $R = 0.5 * I$

这很适合教学起点，但在实际跟踪里往往过于理想化。当前工程版本已经改成更合理的初始化方式：

- 位置初始不确定性较小。
- 速度初始不确定性较大。

这更符合新轨迹刚创建时的实际情况：我们大致知道目标在哪里，但一开始并不知道它运动得有多快。

过程噪声和观测噪声被显式区分

当前 `KalmanFilter` 中把位置过程噪声、速度过程噪声和观测噪声分开定义，而不是用单个常量矩阵草草代替。这样做的好处是：

- 参数意义更清楚。
- 后续调参更方便。
- 更能贴合“位置观测可信，但速度需要逐步估计”的目标跟踪特点。

更新阶段使用更稳定的数值形式

当前实现中：

- 卡尔曼增益计算使用 `np.linalg.solve`，而不是直接显式求逆。
- 协方差更新使用 Joseph form，而不是简单的 $P = P - KHP$ 。

这种改法在数学上更稳定，尤其适合连续多帧递推的视觉跟踪任务，因为协方差矩阵的对称性和正定性更容易保持。

2.14.5. `tracking_app.py` 当前支持的实用参数

当前 `tracking_app.py` 在基础版参数之外，还支持以下跟踪专有参数：

- `--track-max-age`
- `--track-min-hits`

- `--track-iou-threshold`
- `--track-center-distance-threshold`
- `--track-size-smoothing`
- `--track-score-smoothing`
- `--track-classes`

这些参数分别对应：

- 轨迹生命周期控制。
- IOU 关联阈值。
- 中心距离补充匹配阈值。
- 轨迹框和分数的平滑强度。
- 指定只跟踪哪些类别。

例如，只跟踪行人时，可以使用：

```
1 python scripts/tracking_app.py --device npu --source 0 --track-classes person
```

如果需要同时跟踪行人和公交车，可以使用：

```
1 python scripts/tracking_app.py --device npu --source 0 --track-classes person,
  ↪ bus
```

如果希望在快速运动场景中增强补充关联，可以适当提高中心距离阈值，例如：

```
1 python scripts/tracking_app.py --device npu --source 0 --track-center-distance-
  ↪ threshold 2.0
```

2.14.6. 如何理解“教材版”和“工程版”的关系

本案例的一个教学特点是：前文先用更直观的形式讲清算法主线，再在当前工程代码中逐步引入更适合实时边缘设备的优化实现。两者并不是相互冲突，而是两个层次：

- 教材版：帮助读者看懂为什么需要这些模块。
- 工程版：帮助读者理解怎样让这些模块在真实设备上跑得更稳、更快。

因此，建议阅读顺序是：

1. 先按前文把 MobileNet-SSD、NMS、卡尔曼滤波和数据关联的原理主线理清。
2. 再结合当前仓库代码，对照本节总结理解这些优化为什么能够改善实时效果。

2.15. 章节总结

本案例展示了一条较为典型的目标跟踪实现路线：

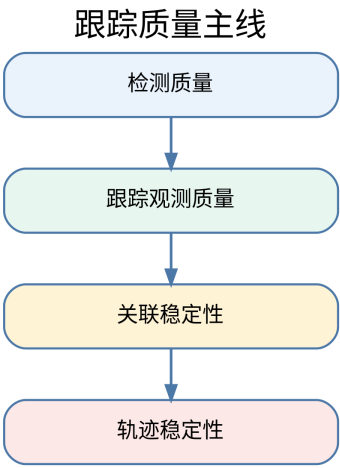


Figure 2.2.: 主线图

- 用 MobileNet 作为轻量骨干网络，降低边缘侧计算成本。
- 用 SSD 作为一阶段检测器，直接输出类别与边界框。
- 用统一检测后端把 CPU 与 Ascend NPU 推理过程封装起来。
- 用简化版 DeepSORT 完成轨迹预测、数据关联与轨迹维护。

从知识结构上看，本章真正要让读者掌握的，不只是某个脚本的使用方法，而是下面这条主线：

理解这条主线之后，读者就能明白：目标跟踪不是一个独立黑盒，而是一套由检测、预测、匹配和管理共同组成的时序识别系统。

对于昇腾 310B 教程而言，这个案例的价值正在于此。它既保留了真实部署场景中需要考虑的实时性与工程组织问题，又把目标跟踪最核心的算法逻辑清楚地呈现出来，适合作为后续学习更复杂视觉任务的基础章节。

3. 案例 3：智能电子琴

3.1. 1. 项目简介

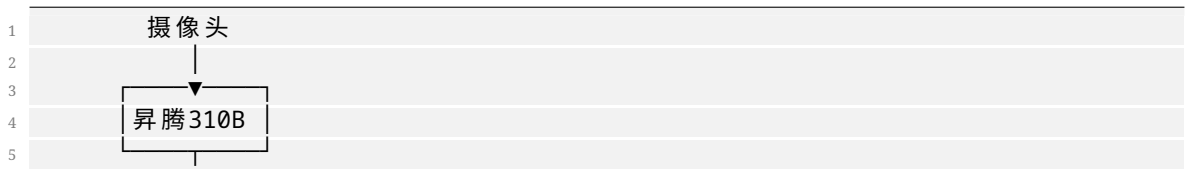
本项目通过结合计算机视觉和音频处理技术，创造了一种全新的交互式音乐体验。系统利用昇腾 310B 的 AI 算力，实时识别用户的手势或特定图像标记，并将其转换为相应的音符和音效，从而实现”隔空弹琴”的神奇效果。该项目不仅展示了 AI 在艺术创作领域的应用潜力，也为音乐教育、娱乐互动和无障碍音乐演奏提供了创新的解决方案。无论是音乐爱好者还是技术探索者，都能从这个项目中获得乐趣和启发。项目的源代码可以从[这里](#)下载。

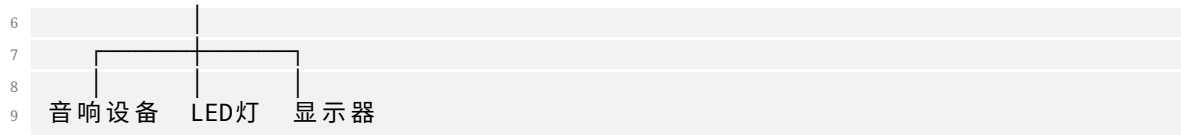
3.2. 2. 内容大纲

3.2.1. 2.1. 硬件准备

- 核心计算单元: 昇腾 310B 开发者套件
- 图像采集: 高分辨率 USB 摄像头 (推荐 1080p 30fps)
- 音频输出:
 - USB 音响或蓝牙音箱
 - 3.5mm 耳机接口
 - HDMI 音频输出
- 显示设备: 触摸屏显示器 (可选，用于虚拟键盘显示)
- LED 指示灯: RGB LED 灯带，用于视觉反馈
- 电源: 稳定的 12V 电源适配器

硬件系统架构图





3.2.2. 2.2. 软件环境

- 操作系统: Ubuntu 20.04 LTS
- CANN 版本: 7.0.RC1
- Python 版本: 3.8.10
- 音频处理库:
 - pygame: 音频播放和 MIDI 处理
 - pyaudio: 实时音频处理
 - librosa: 音频分析
 - mido: MIDI 文件操作
- 计算机视觉库:
 - opencv-python: 图像处理
 - mediapipe: 手势识别
 - numpy: 数值计算
- 界面开发:
 - tkinter: GUI 界面
 - matplotlib: 波形可视化

快速安装脚本 (install_piano.sh)

```

1 #!/bin/bash
2 # 更新包管理器
3 sudo apt update
4
5 # 安装音频依赖
6 sudo apt install -y python3-pyaudio portaudio19-dev
7
8 # 安装Python包
9 pip3 install pygame librosa mido opencv-python mediapipe numpy tkinter
10   ↪ matplotlib
11
12 # 安装MIDI支持
13 sudo apt install -y timidity timidity-interfaces-extra
14
15 echo "智能电子琴环境安装完成!"
  
```

3.2.3. 2.3. 手势识别与音符映射

- 手势识别方案:
 - 静态手势: 识别手指数量对应不同音符
 - 动态手势: 手部移动轨迹控制音调变化
 - 双手协作: 左手控制和弦, 右手控制旋律
 - 手势速度: 识别手势速度控制音符强度
- 音符映射系统: python #手势到音符的映射示例 `gesture_to_note = { 'one_finger': 'C4', # 1个手指 = C调 'two_fingers': 'D4', # 2个手指 = D调 'three_fingers': 'E4', # 3个手指 = E调 'four_fingers': 'F4', # 4个手指 = F调 'five_fingers': 'G4', # 5个手指 = G调 'fist': 'REST', # 握拳 = 休止符 'palm_open': 'CHORD' # 张开手掌 = 和弦 }`
- 高级识别功能:
 - 音量控制: 通过手与摄像头的距离控制音量
 - 音效切换: 特定手势切换乐器音色
 - 节拍控制: 双手拍击节奏控制节拍器

3.2.4. 2.4. 模型训练与优化

- 手势识别模型:
 - 基础方案: 使用 MediaPipe 的预训练手部识别模型
 - 自定义方案: 训练专门的手势分类模型
 - 数据集构建: 收集多角度、多光照条件下的手势数据
- 模型训练流程:
 1. 数据收集: 使用摄像头收集各种手势样本
 2. 数据标注: 为每个手势分配对应的音符标签
 3. 模型训练: 使用 CNN 或 Transformer 架构训练分类器
 4. 模型评估: 在测试集上验证识别准确率
 5. 模型优化: 针对昇腾 310B 进行量化和加速
- 实时优化策略:
 - 帧率控制: 平衡识别精度和实时性
 - 噪声过滤: 减少误识别和抖动
 - 延迟补偿: 最小化从手势到发声的延迟

3.2.5. 2.5. 音频合成与处理

- 音频合成方案:
 - MIDI 合成: 使用 MIDI 格式生成标准乐器音色
 - 波形合成: 直接生成音频波形, 支持自定义音色
 - 采样回放: 预录制真实乐器采样进行回放
- 音效处理: python #音效处理管道示例 `audio_pipeline = [ 'reverb', # 混响效果 'equalizer' ↪ ', # 均衡器 'compressor', # 压缩器 'delay', # 延迟效果 'chorus' # 合唱效果 ]`
- 实时音频流处理:
 - 低延迟设计: 优化音频缓冲区大小
 - 多线程处理: 分离音频生成和播放线程
 - 动态加载: 按需加载音色库

3.2.6. 2.6. 3D 打印结构件

- 摄像头支架 (camera_stand.stl):
 - 高度可调节设计 (50-80cm)
 - 角度微调机构 ($\pm 30^\circ$)
 - 防震减振设计
- 设备一体化外壳 (piano_case.stl):
 - 集成散热风扇位
 - LED 灯带安装槽
 - 音响设备安装位
- 用户交互面板 (control_panel.stl):
 - 物理按键布局
 - 旋钮和滑块安装位
 - 显示屏保护框

3D 打印建议: - 材料选择: ABS (强度好, 适合结构件) - 层高设置: 0.2mm - 填充密度: 25%
 - 支撑设置: 针对悬垂结构添加支撑

3.2.7. 2.7. 用户手册

2.7.1 快速上手

1. 环境配置: 运行安装脚本配置软件环境
2. 硬件连接: 按照连接图组装所有硬件
3. 校准摄像头: 调整摄像头角度和高度
4. 启动程序: `python3 smart_piano.py`
5. 手势训练: 跟随屏幕提示学习基本手势

2.7.2 演奏模式

- 自由演奏模式: 随意手势即兴演奏
- 跟随演奏模式: 根据屏幕提示演奏指定曲目
- 录制模式: 录制演奏过程并回放
- 教学模式: 逐步学习手势和音符对应关系

2.7.3 高级功能

- 乐器切换: 手势控制切换钢琴、小提琴、吉他等音色
- 和弦演奏: 双手配合演奏复杂和弦
- 节拍器: 内置节拍器辅助练习
- 录音回放: 保存演奏为音频文件

2.7.4 故障排除

- 手势识别不准确: 检查光照条件和摄像头角度
- 音频延迟: 调整音频缓冲区设置
- 程序崩溃: 查看日志文件排查错误

3.3. 3. 源代码结构

```
1 smart_piano/
2 |— src/
3 |   |— gesture_recognition/    # 手势识别模块
4 |   |— audio_synthesis/       # 音频合成模块
5 |   |— ui/                    # 用户界面
6 |   |— utils/                 # 工具函数
7 |— models/
8 |   |— gesture_classifier.onnx # 手势识别模型
```

```
9 |   └─ pretrained/           # 预训练模型
10 | └─ audio/
11 |   └─ samples/             # 音频采样
12 |   └─ midi/                # MIDI文件
13 | └─ configs/
14 |   └─ piano_config.yaml    # 配置文件
15 | └─ 3d_models/              # 3D打印文件
```

3.4. 4. 效果演示

- **基础演奏:** 展示单手手势演奏简单旋律
- **和弦演奏:** 双手配合演奏和弦进行
- **音色切换:** 实时切换不同乐器音色
- **教学演示:** 跟随模式学习演奏《小星星》等简单曲目

4. 案例 4：智能掌纹识别机

4.1. 1. 项目简介

本项目基于昇腾 310B 平台，构建一个高精度的掌纹识别系统。掌纹识别作为一种生物识别技术，具有纹理丰富、难以伪造、非接触式采集等优势，在门禁控制、考勤管理、身份验证等领域有着广阔的应用前景。

相比传统的指纹识别，掌纹包含更多的特征信息——主线、皱纹、脊线纹理等，能够提供更高的识别精度和更强的防伪能力。

本案例的**核心创新**在于将掌纹识别建模为**开集验证 (open-set verification)** 问题而非闭集分类问题：系统通过 GhostNet 提取掌纹的深度特征向量，使用 FAISS 进行高效的余弦相似度检索，新用户注册时无需重新训练模型，只需将其特征向量加入索引即可。

4.1.1. 与已有案例的关系

案例	领域	模型	任务范式
案例 1	人脸打卡	ArcFace + SCRFD	人脸检测 + 识别
案例 5	电机监测	EfficientNet-B0	闭集故障分类
案例 6	小车感知	ResNet18	闭集场景分类
案例 7	智能相册	ResNet50 + FAISS	开集图像检索
案例 4	掌纹识别	GhostNet + FAISS	开集身份验证
案例 8	手势识别	MobileNetV3-Small	闭集手势分类

案例 4 与案例 7 都使用了 FAISS 进行向量检索，但有关键区别：- **案例 7** 使用 ResNet50 的通用 ImageNet 预训练特征，无需额外训练 - **案例 4** 使用 GhostNet + 对比学习在掌纹数据上专门训练，学习掌纹特有的判别特征空间 - **案例 7** 做的是 1:N 相似照片检索，**案例 4** 做的是 1:1 身份验证（含阈值判定）

4.2. 2. 内容大纲

4.2.1. 2.1. 硬件准备

本项目需要的硬件分为三个层级：

层级	硬件	职责	要求
图像采集	USB 摄像头	掌纹图像采集	>=5MP 分辨率，支持自动对焦
照明增强	近红外 LED 阵列 (可选)	增强掌纹纹理对比度	850nm 波长，均匀分布
AI 计算	昇腾 310B 开发者套件	掌纹特征提取 + 向量检索	—
结构件	3D 打印外壳 (可选)	手掌定位 + 环境光隔离	PETG 材料

系统架构如图所示：

```
1 flowchart TB
2   subgraph INPUT["图像采集"]
3     CAM["USB 摄像头\n>=5MP 自动对焦"]
4     LED["近红外 LED (可选)\n增强纹理对比度"]
5   end
6
7   subgraph ASCEND["Ascend 310B"]
8     CPU_A["CPU\n· 掌纹 ROI 检测\n· CLAHE 增强\n· 质量检查"]
9     NPU_A["NPU\n· GhostNet 1.0x\n· 1280-dim 特征提取\n· OM 离线模型"]
10    FAISS_A["FAISS\n· IndexFlatIP\n· 余弦相似度检索\n· JSON 元数据管理"]
11  end
12
13  subgraph UI["Gradio 仪表盘"]
14    ENROLL["注册掌纹\n采集 3-5 张样本"]
15    VERIFY["身份验证\n实时匹配结果"]
16    ADMIN["系统管理\n用户列表 / 阈值调节"]
17  end
18
19  CAM --> CPU_A
20  LED -. -> CAM
21  CPU_A --> NPU_A
22  NPU_A --> FAISS_A
23  FAISS_A --> ENROLL
24  FAISS_A --> VERIFY
25  FAISS_A --> ADMIN
```

4.2.2. 2.2. 软件环境

操作系统与框架

层级	软件	版本	用途
操作系统	Ubuntu	20.04 / 22.04 LTS	运行环境
CANN	Ascend Toolkit	7.0+	NPU 驱动与 ATC 转换工具
Python	CPython	3.8 ~ 3.10	应用开发语言
深度学习	PyTorch + torchvision	2.0+	GhostNet 模型 / CPU 推理回退
图像处理	OpenCV	4.8+	掌纹预处理、ROI 检测、摄像头
向量检索	faiss-cpu	1.7+	FAISS 索引与相似度搜索
Web	Gradio	4.0+	仪表盘界面

环境准备

```
1 # 一键安装依赖并导出模型
2 bash setup.sh
```

setup.sh 依次完成：系统包安装 -> Python 依赖安装 -> 模型导出（如有训练好的 .pth 则导出 ONNX，如有 CANN 则继续转换为 OM）。

如果只想在开发机上导出 ONNX：

```
1 python3 prepare_models.py --onnx-only
```

Python 依赖说明

包	用途	必需？
gradio	Web 仪表盘框架	是
torch + torchvision	GhostNet 模型定义、CPU 推理回退	是
opencv-python	掌纹预处理、ROI 检测、摄像头采集	是
numpy	数值计算、特征向量处理	是
faiss-cpu	FAISS 向量索引与检索	是
Pillow	Gradio 图像格式转换	是
onnx	ONNX 模型校验（仅 prepare_models.py）	仅转换
acl (CANN 自带)	NPU 推理，随 CANN 安装	仅推理

4.2.3. 2.3. 掌纹图像预处理

掌纹 ROI（感兴趣区域）检测是整个系统的第一道关口。高质量的 ROI 提取直接影响后续特征提取的准确性。

预处理流水线

```

1 flowchart TD
2   RAW["摄像头帧\nBGR (H, W, 3)"] --> GRAY["灰度化\nopencv2.COLOR_BGR2GRAY"]
3   GRAY --> SEG["手掌分割\nGaussianBlur -> Otsu 二值化\n形态学闭/开操作"]
4   SEG --> CONTOUR["轮廓检测\nopencv2.findContours\n取最大轮廓 = 手掌"]
5   CONTOUR --> VALLEY["指谷点定位\n凸包缺陷分析\n筛选深度 >3000 的凹陷"]
6   VALLEY --> ROI["ROI 提取\n以指谷连线中点为基准\n向下偏移 30% 距离\n提取正方形区域"]
7   ROI --> CLAHE["对比度增强\nBGR -> LAB\nCLAHE 应用于 L 通道\nLAB -> BGR"]
8   CLAHE --> CHECK["质量检查\nLaplacian 方差\n< 100 -> 重新采集"]
9   CHECK --> OUTPUT["输出\n(224, 224, 3) BGR"]

```

关键技术细节

手掌分割：使用 Otsu 自适应阈值将手掌与背景分离。系统假设手掌比背景亮（配合深色背景和 LED 照明）。如果二值化后白色像素超过 50%，则自动反转（适应浅色背景场景）。随后通过形态学闭操作填充手掌内部孔洞，开操作去除孤立噪点。

指谷点定位：计算手掌轮廓的凸包，通过凸性缺陷 (convexity defects) 定位手指之间的凹陷点（指谷）。只保留深度超过 3000 像素单位的缺陷点——这有效过滤掉手腕处的微小凹陷。缺陷点按 x 坐标从左到右排序，取最左和最右两个作为 ROI 坐标系基准。

ROI 提取：以左右两个指谷点的连线中点为参考点，向下偏移连线距离的 30% 作为 ROI 中心（从指谷移动到掌纹中心区域），ROI 边长为指谷距离的 1.2 倍。这种几何方法在受控环境下稳定且高效，避免了复杂的机器学习定位。

CLAHE 增强：将 ROI 图像从 BGR 转换到 LAB 色彩空间，仅在亮度 L 通道上应用 CLAHE（对比度受限的自适应直方图均衡化），然后转换回 BGR。这样做在不引入色彩伪影的前提下显著增强了掌纹纹理的对比度。

全部代码实现在 `palm_preprocessor.py` 的 `PalmPreprocessor` 类中，纯 OpenCV 实现，无需任何模型文件。

预处理失败的处理

预处理可能因以下原因失败：手掌未在画面中、光照过暗/过亮、手指未张开无法定位指谷点、图像模糊等。`PalmPreprocessor.preprocess()` 在失败时返回 `None` 而非抛出异常，调用方可以优雅地提示用户重新采集。

4.2.4. 2.4. GhostNet 特征提取

掌纹识别需要一个能将掌纹图像映射到判别性特征空间的模型。本案例选择 **GhostNet 1.0x** 作为特征提取骨干网络。

为什么选 GhostNet

对比维度	GhostNet 1.0x	其他候选
参数量	5.2M	ResNet50: 25.6M, MobileNetV3-Small: 2.5M
核心创新	Ghost 模块：廉价线性变换生成幻影特征图	—
纹理特征提取	Ghost 模块在有限计算预算下产生更丰富的特征图，适合掌纹主线/皱纹/脊线纹理	ShuffleNet 通道混洗偏通用
是否已被本书使用	否 — 为本书增加模型多样性	MobileNetV3-Small 已被案例 8 使用
torchvision 内置	否 — 独立实现 (~220 行)	ResNet / MobileNet 内置

GhostNet 架构

```

1 flowchart TD
2   INPUT["输入\n(1, 3, 224, 224)"] --> STEM["Stem\nConv2d 3->16, stride=2\nBN +  
  ↳ ReLU6\n(112×112×16)"]
3
4   STEM --> B1["GhostBottleneck ×1\n16 -> 16\nstride=1, no SE\n(112×112×16)"]
5
6   B1 --> B2["GhostBottleneck ×2\n16 -> 24\nstride=2 下采样\n(56×56×24)"]
7
8   B2 --> B3["GhostBottleneck ×2\n24 -> 40\nstride=2, SE\n(28×28×40)"]
9
10  B3 --> B4["GhostBottleneck ×4\n40 -> 80\nstride=2\n(14×14×80)"]
11
12  B4 --> B5["GhostBottleneck ×2\n80 -> 112\nstride=1, SE\n(14×14×112)"]
13
14  B5 --> B6["GhostBottleneck ×5\n112 -> 160\nstride=2, SE\n(7×7×160)"]
15
16  B6 --> HEAD["Head\nConv2d 160->960, BN, ReLU6\nAdaptiveAvgPool2d -> (1×1)\n  ↳ nConv2d 960->1280\nFlatten -> (1280,)"]
17
18  HEAD --> OUTPUT["输出\n1280-dim 特征向量\n(后续 L2 归一化)"]

```

Ghost 模块原理

标准卷积同时生成所有输出通道，存在大量冗余。Ghost 模块将卷积拆分为两步：

1. **主卷积 (Primary)**: 生成少量“内在”特征图 (输出通道的 $1/\text{ratio}$, $\text{ratio}=2$)
2. **廉价卷积 (Cheap)**: 对每个内在特征图应用深度可分离卷积，生成“幻影”特征图，填充剩余通道
3. 将两部分拼接，得到完整输出

```

1 # GhostModule 核心实现 (ghostnet.py)
2 class GhostModule(nn.Module):
3     def __init__(self, in_ch, out_ch, kernel_size=3, stride=1, ratio=2):
4         primary_out = out_ch // ratio          # 一半通道
5         cheap_out = out_ch - primary_out       # 另一半
6
7         self.primary = nn.Sequential(
8             nn.Conv2d(in_ch, primary_out, kernel_size, stride, ...),
9             nn.BatchNorm2d(primary_out),
10            nn.ReLU6(True),
11        )
12        self.cheap = nn.Sequential(
13            nn.Conv2d(primary_out, cheap_out, kernel_size, 1,
14                groups=primary_out, ...), # 深度可分离
15            nn.BatchNorm2d(cheap_out),
16        )
17
18    def forward(self, x):
19        p = self.primary(x)
20        c = self.cheap(p)
21        return torch.cat([p, c], dim=1)

```

注意廉价卷积后不加 **ReLU**——这是 GhostNet 论文的设计选择，保留幻影特征图的线性特性以保持信息多样性。

SE 注意力模块

GhostNet 在部分 Bottleneck 中嵌入 SE (Squeeze-and-Excitation) 注意力模块。SE 模块通过全局平均池化 -> 压缩 -> 扩展 -> **hard-sigmoid** 门控，自适应地调整各通道的重要性权重。

为保证 ONNX opset 11 兼容性，本实现使用手动 **hard-sigmoid** (`relu6 / 6`) 而非 `torch.`

↪ `nn.Hardsigmoid()`:

```

1 def forward(self, x):
2     s = F.adaptive_avg_pool2d(x, 1)
3     s = F.relu6(self.conv1(s)) / 6.0 # hard-sigmoid
4     s = self.conv2(s)
5     return x * s

```


特征提取模式

GhostNet 在特征提取模式下 (`num_classes=None`) 不包含分类头，直接输出 1280 维特征向量。
`PalmExtractor.extract()` 在得到特征向量后执行 L2 归一化：

```
1 vec = output.squeeze(0).numpy()
2 norm = np.linalg.norm(vec)
3 if norm > 0:
4     vec = vec / norm
```

L2 归一化确保了 FAISS `IndexFlatIP` 的内积等价于余弦相似度，这是案例 7 已验证的模式。

完整 GhostNet 实现在 [ghostnet.py](#), 包含 `GhostModule`、`SELayer`、`GhostBottleneck`、`GhostNet` 四个类以及工厂函数 `ghostnet_1x()`。

4.2.5. 2.5. 对比学习训练

掌纹识别是开集验证问题——系统需要判断两张掌纹是否来自同一个人，而非将掌纹分到某个预定义的类别。因此，本案例使用**对比学习 (contrastive learning)** 而非分类交叉熵来训练模型。

为什么用对比学习

方法	适用场景	优缺点
分类 CrossEntropy	闭集——用户固定，新用户需重新训练	简单，但不可扩展
对比学习 Contrastive	开集——新用户无需重新训练	直接学习特征空间的相似度结构
三元组 Triplet	开集	三元组挖掘复杂，收敛不稳定
ArcFace	开集	需要大规模身份标签，实现复杂

对比学习直接优化特征空间中的距离关系：同一人的掌纹特征距离近，不同人的掌纹特征距离远。训练完成后，新用户注册时只需将其掌纹的特征向量加入 FAISS 索引，无需任何模型更新。

对比损失函数

```
1 class ContrastiveLoss(nn.Module):
2     """L = y * d^2 + (1-y) * max(0, margin - d)^2"""
3     def forward(self, emb1, emb2, label):
```

```

4     dist = F.pairwise_distance(emb1, emb2)
5     loss_pos = label * dist.pow(2)           # 正样本：最小化距离
6     loss_neg = (1 - label) * \
7         F.relu(self.margin - dist).pow(2)   # 负样本：距离至少 margin
8     return (loss_pos + loss_neg).mean()

```

- 正样本对（同一人）：损失为欧氏距离的平方，驱动特征向量靠近
- 负样本对（不同人）：当距离小于 `margin` 时产生损失，驱动特征向量远离
- `margin` 默认为 1.0：意味着负样本对的距离应至少为 1.0

数据集组织

训练数据按用户分文件夹存放：

```

1 PolyU_Palmprint/
2 |— 001/
3 |   |— 001_1.bmp
4 |   |— 001_2.bmp
5 |   ...
6 |— 002/
7 |   |— 002_1.bmp
8 |   ...
9 |— ...

```

`PalmprintPairDataset` 在每次 `__getitem__` 时动态生成样本对：- 50% 概率生成正样本对（同一用户的两张不同图像）- 50% 概率生成负样本对（两个不同用户的图像）

这种方式使得有效训练样本数远超原始图像数——即使只有 100 个用户、每个用户 10 张图像（共 1000 张），也能生成数万个训练对。

训练命令与超参数

```

1 python3 train.py --data-dir /path/to/PolyU_Palmprint

```

超参数	默认值	说明
<code>--epochs</code>	60	训练轮数
<code>--batch-size</code>	64	批次大小
<code>--lr</code>	1e-3	初始学习率
<code>margin</code>	1.0	对比损失 <code>margin</code>

训练使用 AdamW 优化器 + CosineAnnealingLR 学习率调度，训练/验证集按用户（而非图像）85/15 分割以确保无数据泄漏。完整训练脚本在 [train.py](#)。

评估指标

训练过程中计算验证集上的**成对准确率**：对每对掌纹，若其特征向量的欧氏距离小于 $\text{margin}/2$ (0.5)，则预测为同一人，与标签比较得出准确率。

训练完成后，最佳模型保存到 `models/ghostnet_palmprint.pth`。

推荐数据集

- [PolyU Palmprint Database](#) — 约 600 个手掌，每个 20 张图像（两阶段各 10 张），最经典的掌纹数据集
- [IITD Palmprint Database](#) — 约 460 个手掌，每个 5-6 张图像，包含左右手

注意 关于预训练权重：GhostNet 不在 torchvision 中，无法使用 ImageNet 预训练权重。模型需要从头开始在掌纹数据集上训练。如果跳过了训练步骤，模型将使用随机权重，特征提取质量会很差——请务必先训练再使用。

4.2.6. 2.6. 模型转换与昇腾部署

```

1 flowchart TD
2     subgraph EXPORT["1. ONNX 导出（开发机 / 昇腾设备）"]
3         GHOST["GhostNet 1.0x\n(无分类头)\n1280-dim 输出"]
4         ONNX["ghostnet_palmprint.onnx\n输入 (1, 3, 224, 224)\n输出 (1, 1280)"]
5     end
6
7     subgraph CONVERT["2. ATC 转换（昇腾设备）"]
8         OM["ghostnet_palmprint.om\nAscend 离线模型 ~20MB"]
9     end
10
11    subgraph DEPLOY["3. 推理部署"]
12        APP["app.py -> PalmExtractor\n-> AscendModel.execute()"]
13    end
14
15    GHOST -->|"torch.onnx.export()\nopset=11"| ONNX
16    ONNX -->|"atc --framework=5\n--soc_version=Ascend310B4"| OM
17    OM -->|"acl.mdl.execute()"| APP

```

步骤 1：导出 ONNX

```
1 python3 prepare_models.py --onnx-only
```

从 `models/ghostnet_palmprint.pth` 加载训练好的权重，构建 GhostNet 特征提取模型（无分类头），固定输入形状 (1, 3, 224, 224) 导出 ONNX (opset=11)。导出后自动进行三项验证：ONNX 模型结构校验、PyTorch vs ONNX Runtime 输出对比（确保数值一致性）。

步骤 2：ATC 转换

```

1 # 在昇腾设备上运行
2 python3 prepare_models.py

```

等效的 atc 命令：

```

1 atc --model=models/ghostnet_palmprint.onnx \
2     --framework=5 \
3     --output=models/ghostnet_palmprint \
4     --soc_version=Ascend310B4 \
5     --input_format=NCHW \
6     --input_shape=input:1,3,224,224

```

转换完成后得到约 20MB 的 OM 离线模型文件。

ONNX 兼容性注意事项

GhostNet 的 ONNX 导出已针对 ATC 兼容性做了两项适配：1. 手动 **hard-sigmoid**：SELayer 中使用 `F.relu6(x)/6.0` 替代 `nn.Hardsigmoid()`，确保在 `opset=11` 下正确导出 2. 固定输入形状：不使用动态轴 (`dynamic_axes={}`)，因为 OM 模型要求固定的输入输出尺寸

4.2.7. 2.7. 开集验证系统

系统架构

开集验证的核心流程：

```

1 注册阶段： 掌纹图像 -> 预处理 -> GhostNet(NPU) -> 1280维向量 -> FAISS 存储
2 验证阶段： 掌纹图像 -> 预处理 -> GhostNet(NPU) -> FAISS 搜索 -> 阈值判定

```

```

1 flowchart TD
2     subgraph ENROLL["注册流程"]
3         E1["采集掌纹\nx3~5 张"] --> E2["ROI 检测\n+ CLAHE"]
4         E2 --> E3["GhostNet\n特征提取"]
5         E3 --> E4["L2 归一化"]
6         E4 --> E5["FAISS.add()\n+ 元数据记录"]
7     end
8
9     subgraph VERIFY["验证流程"]
10        V1["采集掌纹\nx1 张"] --> V2["ROI 检测\n+ CLAHE"]
11        V2 --> V3["GhostNet\n特征提取"]
12        V3 --> V4["L2 归一化"]
13        V4 --> V5["FAISS.search()\nTop-K 检索"]
14        V5 --> V6["多数投票\n平均相似度"]
15        V6 --> V7{"相似度 >= 阈值?"}
16        V7 --> V8["是"] | V8["[OK] 验证通过"]
17        V7 --> V9["否"] | V9["[失败] 验证失败"]
18    end
19
20    E5 --> V1
21    DB["持久化"] | DB["FAISS 索引\n+ JSON 元数据"]
22    DB --> V1
23    DB --> V5

```

FAISS 索引设计

使用 `faiss.IndexFlatIP`（内积索引）存储 L2 归一化后的 1280 维特征向量。由于向量已归一化，内积等价于余弦相似度：

```
1 cosine_similarity(a, b) = a · b  (= faiss inner product when ||a||=||b||=1)
```

每个注册用户存储 3-5 个样本的特征向量（对应不同角度/位置的掌纹），验证时检索 Top-K 个最相似向量，对用户 ID 进行多数投票，取该用户的平均相似度作为匹配分数。

多样本注册策略

为什么每个用户需要 3-5 个样本？

样本数	优点	缺点
1	简单快速	对姿态/光照敏感，易拒真
3-5	覆盖不同姿态，提高鲁棒性	注册稍慢
>10	极高鲁棒性	注册耗时长，存储开销大

验证时，Top-5 个最相似向量中出现最多的用户 ID 即为最佳匹配。例如，若 Top-5 中有 4 个属于用户 A、1 个属于用户 B，则认定匹配用户 A，取这 4 个向量的平均相似度作为最终分数。

用户删除

FAISS `IndexFlatIP` 不支持直接删除单条记录。删除用户时，系统通过 `reconstruct()` 方法从索引中取出保留的向量，重建一个新的 FAISS 索引。对于边缘场景（<1000 个用户），这一 O(N) 操作耗时 <100ms，完全可以接受。

完整索引管理实现在 [palm_index.py](#) 的 `PalmIndex` 类中。

4.2.8. 2.8. Web 仪表盘

```
1 flowchart TD
2   subgraph UI["Gradio Blocks 仪表盘"]
3     TAB1["注册掌纹\n摄像头实时流 -> 采集样本 -> 保存"]
4     TAB2["身份验证\n摄像头实时流 -> 特征提取 -> 匹配判定"]
5     TAB3["系统管理\n用户列表 + 系统状态 + 阈值调节"]
6   end
```

三个页签：

- 1. **注册掌纹**：用户在摄像头前放置手掌，点击「采集掌纹」按钮采集当前帧。采集 3 张以上样本后，点击「完成注册」将特征向量存入 FAISS 索引。界面实时显示已采集样本的缩略图和数量。

2. **身份验证**：用户将手掌放在摄像头前，点击「开始验证」。系统提取掌纹特征后在 FAISS 索引中搜索，返回匹配结果——验证通过/失败、最佳匹配用户、相似度分数和置信度条形图、Top-5 匹配列表、推理耗时和后端信息。
3. **系统管理**：显示已注册用户列表（ID、姓名、样本数），支持按用户 ID 删除。显示系统信息（后端类型、模型信息、索引状态、特征维度）。验证阈值可通过滑块实时调节。

事件处理

注册页签的关键事件流：

```

1 [ 采集掌纹 ] -> capture_enroll_sample()
2   |—— PalmExtractor.extract() -> 若失败显示 "未检测到有效掌纹"
3   |—— 将 (BGR原图, RGB缩略图) 存入 _enroll_buffer
4   |—— 更新状态 (已采集数 / 3)
5 [[OK] 完成注册] -> confirm_enrollment()
6   |—— PalmIndex.enroll_multiple() -> FAISS 添加
7   |—— PalmIndex.save() -> 持久化
8   |—— 显示注册成功信息
9 [ 重新采集 ] -> 清空缓冲区, 重新开始

```

4.2.9. 2.9. 用户手册

2.9.1 部署

1. 将项目代码拷贝到昇腾 310B 设备
2. 运行 `bash setup.sh` 安装 Python 依赖
3. 训练模型：`python3 train.py --data-dir /path/to/palmprint/dataset`
4. 导出模型：`python3 prepare_models.py`（需 CANN 环境）
5. 连接 USB 摄像头
6. 启动服务：`python3 app.py`
7. 浏览器打开 `http://<设备IP>:7860`

2.9.2 使用流程

1. 打开「注册掌纹」页签，输入用户姓名
2. 将手掌自然张开放在摄像头前 15-20 cm 处
3. 点击「采集掌纹」3-5 次，每次略微调整手掌角度
4. 确认样本预览无误后，点击「完成注册」
5. 切换到「身份验证」页签，放置手掌并点击「开始验证」
6. 观察验证结果——通过/失败、相似度分数、Top-5 匹配

7. 如需调整判定严格程度，在「系统管理」页签调节阈值

2.9.3 采集技巧

- **光线**：确保手掌光照均匀，避免强烈阴影。推荐使用近红外 LED 补光
- **背景**：深色、简洁的背景有助于手掌分割。避免背景中有肤色相近的物体
- **姿态**：手指自然分开（不要并拢也不要张得太开），手掌平面与摄像头平行
- **距离**：手掌距摄像头 15-20 cm，确保掌纹纹理清晰可见
- **清洁**：手掌干燥清洁，避免汗渍或污渍影响纹理质量

2.9.4 调整参数

参数	默认值	位置	说明
VERIFICATION_THRESHOLD ↪	0.75	config.py / UI 滑块	验证阈值，越高越严格
TOP_K_RESULTS	5	config.py	FAISS 检索返回数
CONTRASTIVE_MARGIN ↪	1.0	config.py / train.py	对比损失 margin
CLAHE_CLIP_LIMIT	2.0	config.py	CLAHE 对比度限制
LAPLACIAN_BLUR_THRESHOLD ↪	100.0	config.py	图像锐度最低要求

2.9.5 故障排除

问题	可能原因	解决方法
“未检测到有效掌纹” 验证总是失败 NPU 初始化失败	手掌不在画面中或光照不佳 阈值过高或注册样本质量差 CANN 环境未配置	调整手掌位置，确保光照均匀 降低阈值，重新注册更多样本 source /usr/local/Ascend/ ↪ ascend-toolkit/set_env.sh
特征提取结果随机 ATC 转换失败 “请重新放置手掌”	模型未训练 soc_version 不匹配 预处理质量检查未通过	运行 train.py 训练模型 npu-smi info 查看版本 确保手掌清晰，避免运动模糊

4.3. 3. 源代码结构

```

1 case4/
2 |─ app.py                # Gradio 仪表盘入口
3 |─ palm_preprocessor.py  # 掌纹 ROI 检测 + CLAHE 对比度增强
4 |─ palm_extractor.py    # NPU/CPU 双后端 GhostNet 特征提取
5 |─ palm_index.py        # FAISS 注册 / 验证 / 删除 / 持久化
6 |─ ghostnet.py          # GhostNet 1.0x 独立实现
7 |─ train.py             # 对比学习 Siamese 训练
8 |─ prepare_models.py    # ONNX 导出 + ATC OM 转换
9 |─ config.py            # 配置常量
10 |─ setup.sh             # 一键环境安装
11 |─ requirements.txt     # Python 依赖列表
12 |─ data/
13 |   └─ .gitkeep
14 |─ models/              # 模型文件目录 (.pth / .onnx / .om)
15 |─ README.md            # 快速开始指南

```

模块间的调用关系：

```

1 flowchart TB
2   APP["app.py\nGradio 仪表盘"] --> FE["palm_extractor.py\nPalmExtractor\
3   ↪ nGhostNet 特征提取"]
4   APP --> PI["palm_index.py\nPalmIndex\nFAISS 注册/验证/管理"]
5   FE --> PP["palm_preprocessor.py\nPalmPreprocessor\n掌纹 ROI 检测"]
6   FE --> AR["AscendResource\nnacl.init / device / context"]
7   FE --> AM["AscendModel\nnacl.mdl.execute()"]
8   FE --> GN["ghostnet.py\nGhostNet 1.0x"]
9   FE --> CFG["config.py\n所有配置常量"]
10
11  PI --> FE
12  PI --> FAISS["faiss.IndexFlatIP"]
13  PI --> CFG
14
15  TRAIN["train.py\n对比学习训练"] --> GN
16  TRAIN --> CFG
17
18  PREP["prepare_models.py\nONNX -> OM"] --> GN
19  PREP --> CFG

```

与已有案例的继承关系：

- AscendResource / AscendModel 沿用案例 7/案例 8 的同一套模式
- PalmIndex 的 FAISS IndexFlatIP + JSON 元数据模式与案例 7 的 PhotoIndex 一致
- train.py 的训练管道（AdamW + CosineAnnealing）与案例 8 一致
- prepare_models.py 的 ONNX -> ATC 流程与案例 6/案例 7/案例 8 一致
- config.py 的 BASE_DIR + os.path.join 结构与所有案例一致
- app.py 的 Gradio 懒加载单例模式与所有案例一致
- PalmPreprocessor 是本案独有的领域专用预处理模块
- GhostNet (ghostnet.py) 是本案独有的模型实现（220 行独立代码）

- 对比学习训练范式（Siamese + ContrastiveLoss）是本案例独有的训练方法

4.4. 效果演示

4.4.1. 预期效果

功能	预期表现	备注
掌纹 ROI 检测	稳定提取正方形掌纹区域	受控光照 + 深色背景
特征提取 (NPU)	约 8-10 ms / 张	GhostNet 5.2M 参数
验证准确率 (已训练)	EER < 5%, AUC > 0.97	取决于训练数据量和质量
FAISS 检索 (1000 向量)	< 1 ms	IndexFlatIP 精确搜索
端到端验证延迟	预处理 ~5ms + NPU ~10ms + 检索 <1ms 约等于 16ms	—
新用户注册	3-5 次采集，无需重训模型	开集验证的核心优势

4.4.2. 性能指标

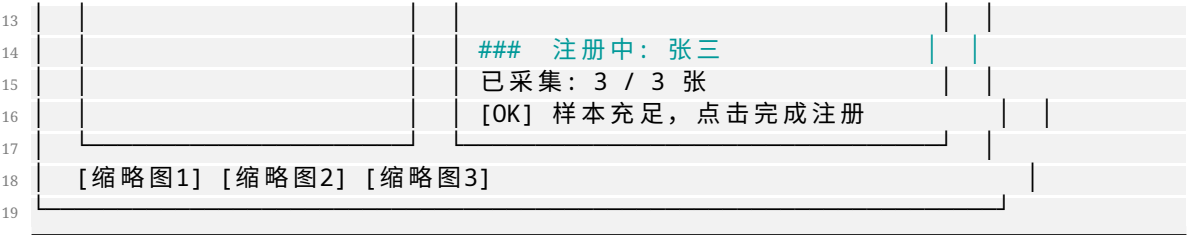
指标	NPU (Ascend 310B)	CPU (PyTorch)
掌纹 ROI 检测	~5 ms	~5 ms
GhostNet 特征提取	8-10 ms	18-25 ms
FAISS 检索 (1000 向量)	<1 ms	<1 ms
模型大小 (.om)	~20 MB	N/A

4.4.3. 浏览器中的效果

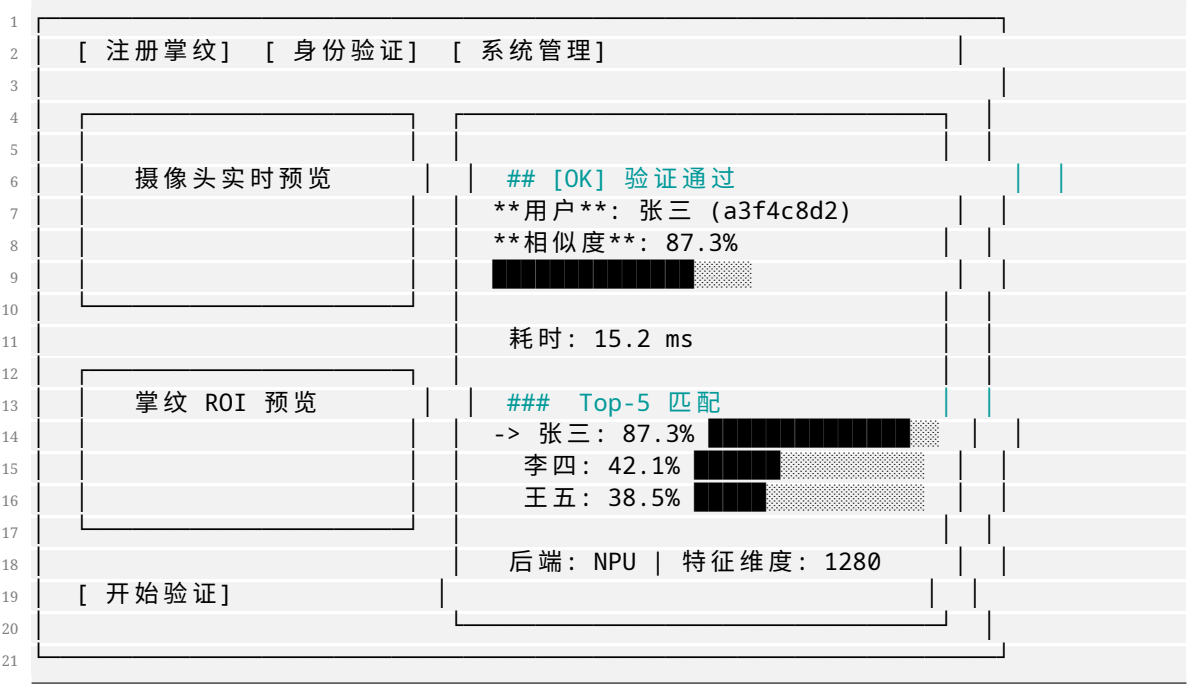
Gradio 仪表盘在浏览器中的预期布局：

注册页签：

1			
2	掌纹 智能掌纹识别机		
3	GhostNet 1.0x 特征提取 + FAISS 向量检索		
4			
5	[注册掌纹] [身份验证] [系统管理]		
6			
7		用户姓名：[张三]	
8			
9			
10	摄像头实时预览	[采集掌纹]	
11		[[OK] 完成注册 (>=3 张)]	
12		[重新采集]	



验证页签：



4.4.4. 如何验证系统正常工作

- 1. 运行 python3 app.py，浏览器打开 http://127.0.0.1:7860
- 2. 在「注册掌纹」页签，输入姓名，放置手掌，采集 3-5 张样本
- 3. 点击「完成注册」，确认看到”注册成功”信息
- 4. 切换到「身份验证」页签，放置同一手掌，点击「开始验证」
- 5. 确认验证通过，相似度 > 80%，匹配用户正确
- 6. 用另一只手掌测试验证——应显示”验证失败”（低于阈值）
- 7. 在「系统管理」页签确认用户列表包含刚才注册的用户
- 8. 调节阈值滑块，验证高阈值下更容易拒真、低阈值下更容易认假
- 9. 在昇腾设备上，确认后端显示”NPU (Ascend 310B)”
- 10. 检查 data/palm_index.faiss 和 data/palm_metadata.json 是否存在

5. 案例 5：智能数据采集仪

5.1. 1. 项目简介

本项目基于昇腾 310B 平台，结合 **STM32 嵌入式采集与 FPGA 高速信号处理**，构建一个面向多台小电机的智能数据采集与状态监测系统。系统同时采集温度、电流、转速、振动四类传感器数据，利用 NPU 对振动频谱进行故障分类，利用 CPU 对温度/电流进行趋势分析和异常检测，为机器人关节电机、无人机电机、自动化产线驱动电机等场景提供预测性维护能力。

本案例是全书第一个**显式结合嵌入式硬件（STM32 + FPGA）与昇腾 AI**的案例，展示了边缘 AI 系统的完整软硬件协同设计流程。

5.1.1. 与已有案例的关系

案例	领域	NPU 任务	方法特点
案例 2	目标检测 + 跟踪	SSD 检测	纯深度学习
案例 5	多电机数据采集	EfficientNet-B0 故障分类	STM32+FPGA+NPU 软硬结合
案例 6	智能小车感知	ResNet18 场景分类	经典 CV + 深度学习混合
案例 7	智能相册	ResNet50 特征提取	批量索引 + FAISS 检索
案例 8	手势识别	MobileNetV3-Small 分类	训练 -> 转换 -> 部署

案例 5 的核心创新在于**数据形态转换**：振动信号本身是一维时序数据，但通过梅尔频谱变换转化为二维图像后，就能充分利用 NPU 的图像分类能力—— 这为各种非图像传感器数据的 NPU 处理提供了一种通用范式。

5.2. 2. 内容大纲

5.2.1. 2.1. 硬件准备

本项目需要的硬件分为三个层级：

层级	硬件	职责	数据速率
低速采集	STM32 (F4/H7)	温度、电流、转速采集, UART 上报	1 Hz
高速采集	FPGA (可选)	振动信号 5kHz 采样 + FFT 预处理	5 kHz
AI 计算	昇腾 310B 开发者套件	数据汇聚、NPU 故障分类、Web 仪表盘	—
传感器	DS18B20 + ACS712 + 霍尔 + ADXL345	四类电机参数	—
外设	USB 摄像头 (可选)	设备外观巡检	—

硬件架构如图所示：

```

1 flowchart TB
2     subgraph MOTORS["电机群 (1~4台)"]
3         M1["电机1"]
4         M2["电机2"]
5         M3["电机3"]
6         M4["电机4"]
7     end
8
9     subgraph STM32["STM32 低速采集"]
10        DIR["温度 DS18B20 ×4\n电流 ACS712 ×4\n转速 霍尔 ×4"]
11    end
12
13    subgraph FPGA["FPGA 高速采集"]
14        VIB["振动 ADXL345 ×4\n5kHz 采样 + FFT"]
15    end
16
17    subgraph ASCEND["Ascend 310B"]
18        CPU_A["CPU\n• UART 读取传感数据\n• 梅尔频谱图生成\n• 3σ 异常检测"]
19        NPU_A["NPU\n• EfficientNet-B0\n• 频谱图故障分类\n• OM 离线模型"]
20        UI_A["Gradio 仪表盘\n• 实时数据\n• 故障告警\n• 频谱显示"]
21    end
22
23    M1 & M2 & M3 & M4 --> DIR --> STM32
24    M1 & M2 & M3 & M4 --> VIB --> FPGA
25    STM32 -->|"UART 115200"| CPU_A
26    FPGA -->|"SPI/DMA"| CPU_A
27    CPU_A --> NPU_A
28    CPU_A --> UI_A
29    NPU_A --> UI_A

```

5.2.2. 2.2. 软件环境

操作系统与框架

层级	软件	版本	用途
操作系统	Ubuntu	20.04 / 22.04 LTS	运行环境
CANN	Ascend Toolkit	7.0+	NPU 驱动与 ATC 转换工具
Python	CPython	3.8 ~ 3.10	应用开发语言
深度学习	PyTorch + torchvision	2.0+	EfficientNet-B0 模型 / CPU 推理回退
图像处理	OpenCV	4.8+	频谱图渲染与缩放
串口通信	pyserial	3.5+	STM32 UART 数据读取
Web	Gradio	4.0+	监测仪表盘界面

环境准备

```
1 # 一键安装依赖并导出模型
2 bash setup.sh
```

setup.sh 依次完成：系统包安装 -> Python 依赖安装 -> ONNX 模型导出（如有 CANN 则继续转换为 OM）。

如果只想在开发机上导出 ONNX：

```
1 python3 prepare_models.py --onnx-only
```

STM32 固件开发

STM32 端的固件代码参考 [stm32_protocol.md](#)，包含完整的传感器接线图、UART 数据帧格式、Arduino 示例代码和 STM32CubeIDE 代码框架。

Python 依赖说明

包	用途	必需？
gradio	Web 仪表盘框架	是
torch + torchvision	EfficientNet-B0 模型定义、CPU 推理回退	是

包	用途	必需?
opencv-python	频谱图渲染、图像缩放	是
numpy	数值计算、频谱分析 (FFT/mel)	是
pyserial	STM32 UART 通信	仅硬件模式
Pillow	Gradio 图像格式转换	是
onnx	ONNX 模型校验（仅 prepare_models.py）	仅转换
acl (CANN 自带)	NPU 推理，随 CANN 安装	仅推理

5.2.3. 2.3. STM32 低速传感器采集

STM32 作为传感器汇聚节点，负责采集四台电机的低速参数：

传感器	型号	接口	参数	采样率
温度	DS18B20	OneWire / I2C	电机表面温度	1 Hz
电流	ACS712	ADC	电机工作电流	1 Hz
转速	霍尔传感器	TIM 输入捕获	电机转速 (RPM)	1 Hz

STM32 每秒钟采集一轮数据，打包为 ASCII 文本帧，通过 UART 发送给 Ascend 310B：

```
1 M0:T=42.5,C=1.25,R=3200|M1:T=38.2,C=0.85,R=2950|M2:T=45.1,C=1.52,R=3100|M3:T
  ↳ =40.3,C=0.92,R=3050\r\n
```

选择文本协议而非二进制协议的理由：- 调试方便——串口工具直接可读 - 数据速率低 (1Hz × 4 电机 = 4 条记录/秒)，文本开销可忽略 - 易于扩展——增加新传感器只需增加字段

STM32 固件的完整开发指南(接线图、Arduino 示例、STM32CubeIDE 框架)见 [stm32_protocol.md](#)。

Ascend 310B 端通过 [sensor_reader.py](#) 的 `SensorReader` 类读取 UART 数据。当未检测到 STM32 时（`/dev/ttyUSB0` 不存在），自动切换为仿真模式，生成带随机游走和偶发故障注入的模拟数据，方便在没有硬件的情况下开发和测试。

5.2.4. 2.4. FPGA 高速振动采集

振动信号的频率范围远超 STM32 的 ADC 采样能力。电机振动特征频率通常在几十 Hz 到 2 kHz 之间，按照奈奎斯特定理，采样率需至少 4 kHz。本项目使用 FPGA 实现 5 kHz 的多通道同步振动采集。

```
1 flowchart LR
2   ACCEL["ADXL345\n三轴加速度计"] -->|"模拟/I2C"| ADC["FPGA ADC\n5kHz 采样"]
3   ADC -->|"原始波形"| FIFO["FPGA FIFO\n缓冲 4096 点"]
```

```
4 FIFO -->|"批量传输"| FFT["FPGA FFT\n(可选预处理)"]
5 FFT -->|"SPI"| CPU["Ascend 310B CPU"]
6 CPU -->|"梅尔频谱图"| NPU["NPU 推理"]
```

FPGA 的核心工作：1. 以 5 kHz 速率从 ADXL345 采集三轴振动数据 2. 将数据缓冲到内部 FIFO（每 4096 点为一个批次）3. 可选地在 FPGA 内完成 FFT 预处理，减轻 CPU 负担 4. 通过 SPI 接口将数据发送给 Ascend 310B

Ascend 310B 端通过 [vibration_processor.py](#) 的 VibrationProcessor 类管理振动数据。当 FPGA 未连接时，自动生成含故障特征的仿真振动波形，包括：

- 正常运行：基频 (50Hz) + 少量谐波 + 白噪声
- 轴承磨损（0.5% 概率注入）：高频噪声显著增强
- 动平衡不良（0.5% 概率注入）：1× 基频峰值异常增大
- 对中不良（0.5% 概率注入）：2× 基频峰值异常增大
- 机械松动（0.5% 概率注入）：多次谐波 + 基底噪声抬高

5.2.5. 2.5. 振动频谱分析原理

振动信号是一维时间序列，而 NPU 擅长处理二维图像。解决方案是将振动波形转换为**梅尔频谱图 (mel-spectrogram)**——一种模仿人耳听觉特性的时频表示。

为什么用梅尔频谱图

方法	输入维度	适用模型	优缺点
原始波形	1D (N,)	1D-CNN	简单，但 NPU 对 1D 卷积优化不如 2D
线性频谱图	2D (freq, time)	2D-CNN	频率分辨率均匀，但低频信息不足
梅尔频谱图	2D (mel, time)	2D-CNN	低频分辨率高（电机故障特征集中在此）
GAF 图像	2D (time, time)	2D-CNN	保留时间相关性，计算复杂

选择梅尔频谱图的核心理由：**电机故障特征（不平衡、不对中、松动）集中在低频区域（1~10 倍转速频率）**，而梅尔尺度在低频区有更高的分辨率。

转换流水线

```

1 flowchart TD
2     WAVEFORM["振动波形\n4096 点 @ 5kHz\nfloat32 (4096,)"] --> FRAME["分帧\n窗口  
→ =256点, 步长=64点\n~60帧"]
3     FRAME --> WINDOW["加窗\nHann window\n抑制频谱泄漏"]
4     WINDOW --> FFT["FFT\n每帧 -> 129 个频点\n(256/2+1)"]
5     FFT --> MEL["Mel 滤波器组\n128 个三角滤波器\n低频密集, 高频稀疏"]
6     MEL --> LOG["Log 压缩\nlog(mel_spec + ε)\n模拟人耳对数感知"]
7     LOG --> NORM["归一化\n-> [0, 1]"]
8     NORM --> RESIZE["尺寸调整\n-> 128×128\n中心裁剪/填充"]
9     RESIZE --> CH3["3 通道复制\ngray -> RGB\n(128, 128, 3)"]
10    CH3 --> OUTPUT["输出\n-> FaultClassifier\n-> NPU 推理"]

```

全部实现仅依赖 NumPy(无需 librosa/scipy),在 `vibration_processor.py` 的 `waveform_to_spectrogram` → () 方法中完成。梅尔滤波器组的三角滤波器矩阵在 `VibrationProcessor` 初始化时预计算一次, 后续每次转换仅需矩阵乘法。

不同故障类型的频谱特征

故障类型	频谱表现	物理原因
正常运行	基频 (1×) + 少量谐波, 基底噪声低	理想旋转
轴承磨损	高频区 (500Hz+) 宽带噪声增强	滚珠/滚道表面剥落产生随机冲击
动平衡不良	1× 转速频率处峰值异常增大 (2~5× 正常值)	质量偏心产生离心力
对中不良	2× 转速频率处峰值突出, 轴向振动大	两轴轴线不共线
机械松动	多次谐波 (1×~5×) + 基底噪声整体抬高	连接刚度不足

5.2.6. 2.6. NPU 故障分类

将频谱图输入 EfficientNet-B0 进行 5 类故障分类:

#	故障类型	英文	处理建议
0	正常运行	normal	继续监测
1	轴承磨损	bearing_wear	安排更换轴承
2	动平衡不良	unbalance	做动平衡校正
3	对中不良	misalignment	重新对中安装
4	机械松动	looseness	检查紧固件

为什么选择 EfficientNet-B0

模型	参数量	310B 推理	本书使用情况
EfficientNet-B0	5.3M	~10ms	案例 5（本案例）
ResNet18	11.7M	~12ms	案例 6
ResNet50	25.6M	~25ms	案例 7
MobileNetV3-Small	2.5M	~5ms	案例 8

EfficientNet-B0 引入了 MBConv 块和 SE（Squeeze-and-Excitation）注意力模块，通过复合缩放策略在深度、宽度、分辨率三个维度上同时优化。5.3M 参数在 310B 上推理只需约 10ms，完全满足实时监测需求。

分类流程

```
1 flowchart TD
2   INPUT["梅尔频谱图\n(128, 128, 3) float32 [0,1]"] --> RESIZE["cv2.resize\n-> 224×224"]
3   RESIZE --> NORM["ImageNet 归一化\n(pixel-mean)/std"]
4   NORM --> TENSOR["-> NCHW (1, 3, 224, 224)"]
5   TENSOR --> CHECK{"NPU 可用?"}
6
7   CHECK -->|"是 OM 模型"| NPU_INFER["NPU 推理\nnacl.mdl.execute()"]
8   CHECK -->|"否 回退"| CPU_INFER["CPU 推理\ntorch model()"]
9
10  NPU_INFER --> SOFTMAX["Softmax -> 概率分布"]
11  CPU_INFER --> SOFTMAX
12
13  SOFTMAX --> OUTPUT["输出\n故障类别 + 置信度 + 处理建议"]
```

5.2.7. 2.7. CPU 趋势分析与异常检测

NPU 负责振动频谱的故障分类，CPU 则负责温度、电流、转速的趋势分析。两者互补——频谱检测机械故障，趋势分析检测电气和热异常。

滑动窗口 + 3σ 异常检测

对每个电机的每个参数（温度、电流、转速），维护一个长度为 30 的滑动窗口。当新数据点偏离窗口均值超过 3 倍标准差时，触发异常告警：

```
1 z_score = |value - window_mean| / window_std
2 if z_score > 3.0:
3   alert("异常值检测")
```

3σ 准则假设传感器数据在稳态下服从正态分布。电机在稳定运行时这个假设基本成立；但在启停、变速阶段会产生误报——这是已知局限。

线性趋势预测

对窗口内的历史数据做最小二乘线性回归，预测未来 10 步的值是否超过阈值：

```
1 # y = a * x + b
2 predicted = a * (len(history) + 10) + b
3 if predicted > TEMP_WARN_THRESHOLD:
4     alert("温度趋势告警：预计将超阈值")
```

实现在 `anomaly_detector.py` 的 `AnomalyDetector` 类中，纯 NumPy 实现，无需任何模型文件。

5.2.8. 2.8. 模型转换与昇腾部署

```
1 flowchart TD
2     subgraph EXPORT["1. ONNX 导出（开发机 / 昇腾设备）"]
3         TORCH["torchvision\nefficientnet_b0(IMAGENET1K_V1)\nmodel.classifier[1]\n    ↳ = Linear(1280, 5)"]
4         ONNX["efficientnet_b0_fault.onnx\n输入 (1, 3, 224, 224)\n输出 (1, 5)"]
5     end
6
7     subgraph CONVERT["2. ATC 转换（昇腾设备）"]
8         OM["efficientnet_b0_fault.om\nAscend 离线模型 ~20MB"]
9     end
10
11    subgraph DEPLOY["3. 推理部署"]
12        APP["app.py -> FaultClassifier\n-> AscendModel.execute()"]
13    end
14
15    TORCH -->|"torch.onnx.export()\nopset=11"| ONNX
16    ONNX -->|"atc --framework=5\n--soc_version=Ascend310B4"| OM
17    OM -->|"acl.mdl.execute()"| APP
```

步骤 1：导出 ONNX

```
1 python3 prepare_models.py --onnx-only
```

构建 EfficientNet-B0 模型，将分类头替换为 5 类故障输出，固定输入形状 (1, 3, 224, ↳ 224) 导出 ONNX。

步骤 2：ATC 转换

```
1 # 在昇腾设备上运行
2 python3 prepare_models.py
```

等效的 atc 命令：

```
1 atc --model=models/efficientnet_b0_fault.onnx \
2     --framework=5 \
3     --output=models/efficientnet_b0_fault \
```

```

4  --soc_version=Ascend310B4 \
5  --input_format=NCHW \
6  --input_shape=input:1,3,224,224

```

关于训练权重

本项目的 prepare_models.py 会生成基线 .pth 文件（ImageNet 骨干 + 随机初始化的 5 类故障分类头）。**基线模型的预测是随机的**——要获得有意义的故障分类结果，需要在电机故障数据集上训练。

推荐数据集：- [CWRU Bearing Dataset](#) — 凯斯西储大学轴承数据集，最经典的电机故障数据集 - [PU Bearing Dataset](#) — 帕德博恩大学轴承数据集 - [MAFAULDA](#) — 多故障类型（不平衡、不对中、松动等）

训练流程参考案例 8 的 train.py 模式：1. 下载上述数据集，将振动信号转换为梅尔频谱图 2. 按 5 个故障类别分文件夹存放频谱图 3. 修改案例 8 的 train.py，模型改为 EfficientNet-B0 4. 训练完成后将 .pth 文件放到 models/efficientnet_b0_fault.pth 5. 运行 python3
 → prepare_models.py --force 重新导出 ONNX/OM

5.2.9. 2.9. Web 仪表盘

```

1 flowchart TD
2     subgraph UI["Gradio Blocks 仪表盘"]
3         TAB1["实时监测\n传感数据 + 异常告警"]
4         TAB2["振动频谱分析\n频谱图 + NPU 故障诊断"]
5         TAB3["系统信息\n模型状态 + 阈值 + 日志"]
6     end
7
8     subgraph TAB1_DETAIL["监测 Tab"]
9         REFRESH["刷新数据"] --> SENSOR["SensorReader.read()"]
10        SENSOR --> ANOMALY["AnomalyDetector.update()"]
11        ANOMALY --> LOG["DataLogger.log() -> CSV"]
12        ANOMALY --> TABLE["DataFrame\n各电机温度/电流/转速"]
13        ANOMALY --> ALERT["Markdown\n告警信息"]
14    end
15
16    subgraph TAB2_DETAIL["频谱 Tab"]
17        SEL["选择电机"] --> VIB["VibrationProcessor\n波形 -> 频谱图"]
18        VIB --> IMG["频谱图可视化"]
19        VIB --> CLS["FaultClassifier\nNPU 故障分类"]
20        CLS --> FAULT_MD["诊断报告"]
21    end
22
23    TAB1 --> TAB1_DETAIL
24    TAB2 --> TAB2_DETAIL

```

三个页签：

1. **实时监测**：四台电机的温度、电流、转速实时显示（带超阈值标记）。异常告警区域显示 3σ 异常和超阈值事件。一键刷新，也支持页面加载时自动刷新。

2. **振动频谱分析**：选择目标电机，生成梅尔频谱图的彩色可视化，同时触发 NPU 故障分类，显示故障诊断报告（Top-3 预测 + 置信度 + 处理建议 + 推理耗时）。
3. **系统信息**：推理后端状态、模型信息、5 类故障列表、数据日志统计、硬件接口配置。

双模式运行

模式	STM32	FPGA	适用场景
全硬件模式	是 UART 连接	是 SPI 连接	生产环境
半仿真模式	是 UART 连接	否仿真振动	只有 STM32
全仿真模式	否仿真数据	否仿真振动	开发/演示

5.2.10. 2.10. 用户手册

2.10.1 部署

1. 将项目代码拷贝到昇腾 310B 设备
2. 运行 `bash setup.sh` 安装 Python 依赖并导出模型
3. (可选) 连接 STM32 到 UART 接口 (`/dev/ttyUSB0`)
4. (可选) 连接 FPGA 到 SPI 接口
5. 启动服务：`python3 app.py`
6. 浏览器打开 `http://<设备IP>:7860`

2.10.2 使用流程

1. 打开「实时监测」页签，观察四台电机的传感数据和告警状态
2. 在仿真模式下可以看到随机游走的数据变化
3. 切换到「振动频谱分析」页签，选择电机进行故障诊断
4. 观察频谱图——注意不同故障的频谱特征差异
5. 切换「系统信息」页签，查看模型和日志状态

2.10.3 STM32 固件烧录

1. 参考 [stm32_protocol.md](#) 中的接线图连接传感器
2. 使用 Arduino IDE 或 STM32CubeIDE 编译示例代码
3. 通过 ST-Link 或 USB-TTL 烧录固件
4. 用串口助手（115200 baud）确认数据帧格式正确
5. 将 STM32 TX 连接到 Ascend 310B RX（3.3V 电平）

2.10.4 调整参数

参数	默认值	位置	说明
TEMP_WARN_THRESHOLD	65°C	config.py	温度告警阈值
CURRENT_WARN_THRESHOLD	2.0A	config.py	电流告警阈值
SIGMA_THRESHOLD	3.0	config.py	异常检测 z-score 阈值
WINDOW_SIZE	30	config.py	滑动窗口大小
SAMPLE_RATE	5000 Hz	config.py	振动采样率
N_MELS	128	config.py	梅尔滤波器数量

2.10.5 故障排除

问题	可能原因	解决方法
无传感器数据 NPU 初始化失败	STM32 未连接 CANN 环境未配置	系统自动切换仿真模式，可忽略 <code>source /usr/local/Ascend/</code> ↪ <code>ascend-toolkit/set_env.sh</code>
故障分类随机 ATC 转换失败 频谱图全黑	未加载训练权重 <code>soc_version</code> 不匹配 振动信号幅值太小	需要训练模型，参见 2.8 节 <code>npu-smi info</code> 查看版本 检查仿真信号生成，调整噪声幅度
UART 数据乱码	波特率不匹配	确认 STM32 和 Ascend 310B 均使用 115200

5.3. 3. 源代码结构

```

1 case5/
2 ├── app.py           # Gradio 仪表盘入口
3 ├── sensor_reader.py # STM32 UART 通信 + 模拟回退
4 ├── vibration_processor.py # FPGA 振动数据 + 梅尔频谱图生成
5 ├── fault_classifier.py # NPU/CPU 双后端故障分类
6 ├── anomaly_detector.py # 统计异常检测 (3σ + 趋势预测)
7 ├── data_logger.py    # CSV 数据记录
8 ├── config.py         # 配置常量
9 ├── prepare_models.py # ONNX 导出 & ATC OM 转换
10 ├── setup.sh         # 一键环境安装
11 ├── requirements.txt  # Python 依赖列表
12 ├── stm32_protocol.md # STM32 固件开发参考
13 └── data/

```

```
14 |   | .gitkeep
15 |   | models/           # 模型文件目录 (.pth / .onnx / .om)
16 |   | README.md         # 快速开始指南
```

模块间的调用关系：

```
1 flowchart TB
2   APP["app.py\nGradio 仪表盘"] --> SR["sensor_reader.py\nSensorReader\nUART读
   ↳ 取/仿真传感数据"]
3   APP --> VP["vibration_processor.py\nVibrationProcessor\n波形->频谱图"]
4   APP --> FC["fault_classifier.py\nFaultClassifier\nNPU/CPU 故障分类"]
5   APP --> AD["anomaly_detector.py\nAnomalyDetector\n3σ异常+趋势预测"]
6   APP --> DL["data_logger.py\nDataLogger\nCSV数据记录"]
7   APP --> CFG["config.py\n所有配置常量"]
8
9   FC --> AR["AscendResource\nnacl.init / device / context"]
10  FC --> AM["AscendModel\nnacl.mdl.execute()"]
11  FC --> TORCH["torchvision\nefficientnet_b0()"]
12  VP --> NP["numpy\nFFT + mel filterbank"]
13
14  PREP["prepare_models.py\nONNX -> OM"] --> TORCH
15  PREP --> CFG
```

与之前案例的继承关系：

- AscendResource / AscendModel 沿用案例 8 的同一套模式
- config.py 的结构与案例 6/案例 7/案例 8 一致（BASE_DIR + os.path.join）
- app.py 的 Gradio 懒加载模式与案例 6/案例 7/案例 8/案例 9 一致
- prepare_models.py 的 ONNX -> ATC 流程与案例 6/案例 7/案例 8 一致
- FaultClassifier 的双后端模式与案例 6 的 SceneClassifier、案例 8 的 GestureClassifier
↳ 一致
- SensorReader 和 VibrationProcessor 是本案独有的硬件接口模块
- AnomalyDetector 是本案独有的纯 NumPy 统计分析模块，无需模型文件
- 本项目不需要训练脚本——故障分类头尚未训练，但推理框架完整

5.4. 4. 效果演示

5.4.1. 预期效果

功能	预期表现	备注
传感器采集	4 电机 × 3 参数，1Hz 更新	STM32 硬件或仿真
NPU 故障分类 (已训练)	5 类准确率 > 88%	需要训练，基线模型随机
3σ 异常检测	明显异常检出率 > 95%	滑动窗口 30 点
趋势预测	线性外推未来 10 步	稳态数据预测有效

功能	预期表现	备注
端到端延迟	传感 <1ms + 频谱 <5ms + 推理 ~10ms	NPU 模式下总计 ~16ms
CSV 日志	每秒写入一行多电机数据	长期运行建议定期轮转

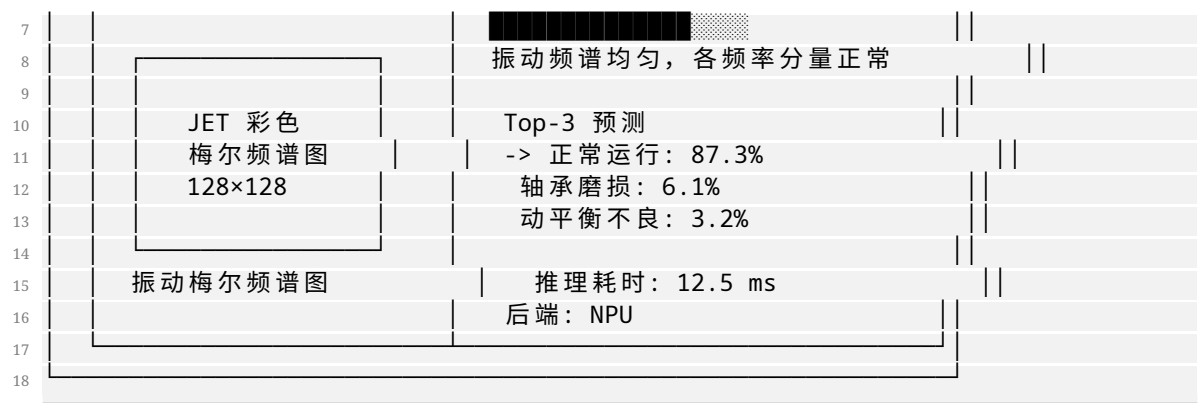
5.4.2. 性能指标

指标	NPU (Ascend 310B)	CPU (PyTorch)
传感数据读取	<1 ms	<1 ms
梅尔频谱图生成	~4 ms	~4 ms
故障分类	8-12 ms	20-35 ms
模型大小 (.om)	~20 MB	N/A

5.4.3. 浏览器中的效果

Gradio 仪表盘在浏览器中的预期布局：

1	智能数据采集仪 — 多电机状态监测				
2	STM32 低速传感 + FPGA 高速振动采集 + NPU 故障分类				
3	[实时监控] [振动频谱分析] [系统信息]				
4					
5	告警区域				
6	[OK] 所有电机运行正常				
7	更新时间：14:32:15 数据行数：245 日志大小：0.05 MB				
8					
9	电机1	电机2	电机3	电机4	
10	温度 是 42.5°C	是 38.2°C	是 45.1°C	是 40.3°C	
11	电流 是 1.25A	是 0.85A	注意 1.52A	是 0.92A	
12	是 3200RPM	是 2950RPM	是 3100RPM	是 3050RPM	
13	[刷新数据]				
14					
15					
16					
17	[实时监控] [振动频谱分析] [系统信息]				
18					
19					
20					
21	选择电机：[电机1 ▼] 故障诊断：正常运行 (normal)				
22	[分析振动频谱] 置信度：87.3%				



5.4.4. 如何验证系统正常工作

- 1. 运行 python3 app.py, 浏览器打开 http://127.0.0.1:7860
- 2. 在「实时监测」页签确认四台电机的温度/电流/转速数值在合理范围内变化
- 3. 在仿真模式下, 数据每点击一次「刷新」会有微小变化(随机游走)
- 4. 切换到「振动频谱分析」, 选择电机 1, 点击「分析振动频谱」
- 5. 确认出现彩色梅尔频谱图, 右侧显示故障诊断结果(如未训练则为随机结果)
- 6. 切换到「系统信息」页签, 确认模型文件状态和日志路径
- 7. 在昇腾设备上, 确认后端显示”NPU (Ascend 310B)”
- 8. 检查 data/motor_data.csv 是否存在且持续增长

6. 案例 6：智能小车视觉感知

6.1. 1. 项目简介

本项目基于昇腾 310B 平台，构建一个智能小车的视觉感知系统。系统结合 **经典计算机视觉**和**深度学习**两种方法：使用 OpenCV 的 Canny 边缘检测 + Hough 变换进行车道线检测，使用 ResNet18 卷积神经网络在 NPU 上进行驾驶场景分类。两种方法互补——几何特征（车道线）用经典 CV 提取，语义理解（场景类型）交给深度学习。

与旧版案例 6 不同，本项目不涉及机器人硬件（底盘、电机、激光雷达等），而是聚焦于昇腾 310B 最擅长的部分：**摄像头画面的实时 AI 理解**。这也是真实自动驾驶系统中”感知层”的核心功能。

6.1.1. 与已有案例的关系

案例	领域	NPU 任务	方法特点
案例 2	目标检测 + 跟踪	SSD 检测	纯深度学习
案例 6	智能小车感知	ResNet18 场景分类	经典 CV + 深度学习混合
案例 7	智能相册	ResNet50 特征提取	批量索引 + FAISS 检索
案例 8	手势识别	MobileNetV3 分类	训练 -> 转换 -> 部署

案例 6 是全书第一个**显式结合经典 CV 与深度学习**的案例，展示了在实际工程中如何选择合适的技术方案——不是所有问题都需要神经网络。

6.2. 2. 内容大纲

6.2.1. 2.1. 硬件准备

本项目只需要：

- **昇腾 310B 开发者套件**：核心计算单元，运行场景分类模型
- **USB 摄像头**（可选）：用于实时画面采集，也可以用测试图片
- **显示屏**（可选）：用于浏览器访问 Gradio 界面

无需任何机器人硬件。车道线检测和场景分类都是纯视觉任务，在 310B + 摄像头组合上即可完整运行。

硬件架构如图所示：

```
1 flowchart LR
2   subgraph INPUT["输入"]
3     CAM["USB 摄像头\n或测试图片"]
4   end
5
6   subgraph COMPUTE["计算平台"]
7     CPU["CPU\n• 车道线检测 (OpenCV)\n• 图像预处理"]
8     NPU["昇腾 310B NPU\n• ResNet18 场景分类\n• OM 离线模型"]
9   end
10
11  subgraph OUTPUT["输出"]
12    UI["Gradio Web 界面\n• 车道线叠加显示\n• 场景标签 + 驾驶建议"]
13  end
14
15  CAM --> |"图像帧"| CPU
16  CPU --> |"车道线结果"| UI
17  CPU --> |"预处理张量"| NPU
18  NPU --> |"场景分类结果"| UI
```

6.2.2. 2.2. 软件环境

操作系统与框架

层级	软件	版本	用途
操作系统	Ubuntu	20.04 / 22.04 LTS	运行环境
CANN	Ascend Toolkit	7.0+	NPU 驱动与 ATC 转换工具
Python	CPython	3.8 ~ 3.10	应用开发语言
深度学习	PyTorch + torchvision	2.0+	ResNet18 模型 / CPU 推理回退
图像处理	OpenCV	4.8+	车道线检测 + 图像预处理
Web	Gradio	4.0+	感知结果展示界面

环境准备

```
1 # 一键安装依赖并导出模型
2 bash setup.sh
```

setup.sh 依次完成：系统包安装 -> Python 依赖安装 -> ONNX 模型导出（如有 CANN 则继续转换为 OM）。

如果只想在开发机上导出 ONNX：

```
1 python3 prepare_models.py --onnx-only
```

Python 依赖说明

包	用途	必需？
gradio	Web 界面框架	是
torch + torchvision	ResNet18 模型定义、CPU 推理回退	是
opencv-python	车道线检测（Canny/Hough）、图像缩放	是
numpy	数值计算、softmax	是
Pillow	Gradio 图像格式转换	是
onnx	ONNX 模型校验（仅 prepare_models.py）	仅转换
acl (CANN 自带)	NPU 推理，随 CANN 安装	仅推理

6.2.3. 2.3. 车道线检测原理

车道线检测是一个经典的计算机视觉问题。车道线本质上是图像中的直线或平滑曲线，具有明显的边缘特征和固定的几何约束（位于画面下半部分，大致呈八字形）。这些特征使得经典 CV 方法——Canny 边缘检测 + Hough 直线变换——比深度学习更直接有效。

为什么不用深度学习做车道线检测

方法	优势	劣势
经典 CV (Canny + Hough)	无需训练数据、速度快、可解释、参数可调	对复杂场景（阴影、弯道）敏感
深度学习 (UNet / LaneNet)	对复杂场景鲁棒	需要标注数据、模型大、OM 转换复杂

对于昇腾 310B 上的智能小车应用，经典 CV 是更合理的选择：不需要额外的标注数据，不占用 NPU 资源（NPU 留给场景分类），处理速度极快。

检测流水线

```
1 flowchart TD
2   INPUT["输入图像\nBGR (H, W, 3)"] --> GRAY["cv2.cvtColor\nBGR -> Grayscale"]
3   GRAY --> BLUR["cv2.GaussianBlur\n降噪 (5x5)"]
```

```
4 BLUR --> CANNY["cv2.Canny\n边缘检测\n阈值 50/150"]
5 CANNY --> ROI["ROI Mask\n保留画面下部 40%\n(道路区域)"]
6 ROI --> HOUGH["cv2.HoughLinesP\n概率 Hough 变换\n提取线段"]
7 HOUGH --> FILTER["斜率过滤\n|slope| >= 0.4\n分离左右车道"]
8 FILTER --> FIT["加权线性回归\n拟合完整车道线"]
9 FIT --> DRAW["cv2.addWeighted\n叠加到原图"]
```

每一步的实现都在 `lane_detector.py` 的 `LaneDetector` 类中。关键步骤说明：

- 1. **Canny 边缘检测**：检测图像中亮度剧烈变化的位置（车道线通常是白色或黄色的，与深色路面形成强烈对比）
- 2. **ROI 蒙版**：只保留图像下半部分（通常是路面区域），过滤掉天空、建筑物等干扰
- 3. **Hough 直线变换**：将边缘图中的点映射到参数空间，找到共线的点群——即直线段。概率 Hough 变换 (`HoughLinesP`) 直接返回线段端点，比标准 Hough 变换更高效
- 4. **斜率过滤**：根据斜率的正负号分离左右车道线（左车道斜率为负，右车道斜率为正），并过滤掉接近水平的线段
- 5. **车道线拟合**：用加权线性回归将多个短线段合并为一条完整的车道线。以 y 为自变量（车道线接近垂直），以线段长度为权重
- 6. **叠加绘制**：用 `cv2.addWeighted` 将车道线半透明叠加到原图上，并在两条车道线之间填充绿色区域标记可行驶车道

6.2.4. 2.4. 驾驶场景分类

车道线检测告诉小车“路在哪里”，但还需要知道“是什么路”——高速公路还是城市街道？前方是否有交叉口？这需要语义层面的理解。

本项目使用 `ResNet18` 对 5 种典型驾驶场景进行分类：

#	场景	特征	驾驶建议
0	高速公路	多车道、护栏、路牌	保持车道，注意车速
1	城市道路	建筑、行人、路边停车	注意行人和交叉口
2	交叉路口	路口、交通灯、斑马线	减速观察
3	停车场	车位线、密集车辆	低速行驶
4	隧道	昏暗、墙壁、灯光	开启车灯

为什么选择 ResNet18

模型	参数量	310B 推理	适用性
ResNet18	11.7M	~12ms	推荐：轻量、torchvision 内置

模型	参数量	310B 推理	适用性
ResNet50	25.6M	~25ms	更准但更慢，案例 7 已使用
MobileNetV3-Small	2.5M	~5ms	案例 8 已使用，精度略低
EfficientNet-B0	5.3M	~10ms	也可用

选择 ResNet18 的理由：- 与已有案例差异化：案例 7 用 ResNet50 做特征提取，案例 8 用 MobileNetV3-Small 做手势分类，这里用 ResNet18 做场景分类——三种不同的模型架构 - 5 分类任务足够：不需要 ResNet50 的 2048 维特征，ResNet18 的 512 维全连接层足够 - torchvision 内置：与案例 7 相同，权重自动从 PyTorch CDN 获取

场景分类流程

```
1 flowchart TD
2   INPUT["输入图像\nBGR (H, W, 3)"] --> RESIZE["cv2.resize\n-> 224×224"]
3   RESIZE --> RGB["cv2.cvtColor\nBGR -> RGB"]
4   RGB --> NORM["归一化\npixel/255 -> (pixel-mean)/std\nImageNet 统计值"]
5   NORM --> TENSOR["-> NCHW (1, 3, 224, 224)"]
6   TENSOR --> CHECK{"NPU 可用?"}
7
8   CHECK -->|"是 OM 模型"| NPU_INFER["NPU 推理\nacl.mdl.execute()"]
9   CHECK -->|"否 回退"| CPU_INFER["CPU 推理\ntorch model()"]
10
11  NPU_INFER --> SOFTMAX["Softmax -> 概率分布"]
12  CPU_INFER --> SOFTMAX
13
14  SOFTMAX --> OUTPUT["输出\n场景类别 + 置信度 + 驾驶建议"]
```

预处理步骤与案例 7、案例 8 完全一致：缩放、色彩转换、ImageNet 归一化、维度重排。

6.2.5. 2.5. 模型转换与昇腾部署

```
1 flowchart TD
2   subgraph EXPORT["1. ONNX 导出（开发机 / 昇腾设备）"]
3     TORCH["torchvision\nresnet18(IMAGENET1K_V1)\nmodel.fc = Linear(512, 5)"]
4     ONNX["resnet18_scene.onnx\n输入 (1, 3, 224, 224)\n输出 (1, 5)"]
5   end
6
7   subgraph CONVERT["2. ATC 转换（昇腾设备）"]
8     OM["resnet18_scene.om\nAscend 离线模型 ~45MB"]
9   end
10
11  subgraph DEPLOY["3. 推理部署"]
12    APP["app.py -> SceneClassifier\n-> AscendModel.execute()"]
13  end
14
15  TORCH -->|"torch.onnx.export()\nopset=11"| ONNX
16  ONNX -->|"atc --framework=5\n--soc_version=Ascend310B4"| OM
17  OM -->|"acl.mdl.execute()"| APP
```

步骤 1: 导出 ONNX

```
1 python3 prepare_models.py --onnx-only
```

这一步构建 ResNet18 模型，将原本 1000 类的 ImageNet 分类头替换为 5 类驾驶场景分类头，然后用固定输入形状 (1, 3, 224, 224) 导出 ONNX。

步骤 2: ATC 转换

```
1 # 在昇腾设备上运行
2 python3 prepare_models.py
```

等效的 atc 命令:

```
1 atc --model=models/resnet18_scene.onnx \
2     --framework=5 \
3     --output=models/resnet18_scene \
4     --soc_version=Ascend310B4 \
5     --input_format=NCHW \
6     --input_shape=input:1,3,224,224
```

关于训练权重

本项目的 prepare_models.py 会生成一个基线 .pth 文件 (ImageNet 骨干 + 随机初始化的 5 类分类头)。**这个基线模型的预测是随机的**——要获得有意义的场景分类结果，需要在驾驶场景数据集上训练。

这是因为: - 手动标注 5 类驾驶场景数据集超出了本书的范围 - 训练流程已在案例 8 中完整演示 (HaGRID + MobileNetV3-Small) - 读者可以参考案例 8 的 train.py 模式，用驾驶场景数据集 (如 [BDD100K](#)、[KITTI](#)) 自行训练

CPU 回退模式下，SceneClassifier 会自动检测 .pth 文件，加载训练好的权重。如果只有基线模型，会在启动时打印警告。

6.2.6. 2.6. Web 界面

```
1 flowchart TD
2     subgraph UI["Gradio Blocks 界面"]
3         TAB1["道路感知 (Tab 1)"]
4         TAB2["系统信息 (Tab 2)"]
5     end
6
7     subgraph TAB1_DETAIL["感知 Tab"]
8         UPLOAD["gr.Image\n上传道路图像"] --> PERCEIVE["perceive()"]
9         PERCEIVE --> LANE["LaneDetector.detect()\n车道线检测 (CPU)"]
10        PERCEIVE --> SCENE["SceneClassifier.classify()\n场景分类 (NPU/CPU)"]
11        LANE --> OVERLAY["车道线叠加图"]
12        SCENE --> REPORT["Markdown 感知报告\n场景 + 置信度 + 驾驶建议"]
```

```

13     end
14
15     subgraph TAB2_DETAIL["信息 Tab"]
16         SYS["系统信息\n模型状态 / 后端 / 参数"]
17     end
18
19     TAB1 --> TAB1_DETAIL
20     TAB2 --> TAB2_DETAIL

```

两个页签

1. **道路感知**：上传道路图像（或使用示例图片），系统同时执行车道线检测和场景分类。左侧显示输入图像，右侧显示感知报告（车道检测状态、场景标签、置信度进度条、Top-3 预测、驾驶建议），下方显示车道线叠加效果图。
2. **系统信息**：展示当前推理后端（NPU/CPU）、模型文件状态、5 类场景列表及驾驶建议、车道线检测参数（Canny 阈值、Hough 阈值等）。

双后端

与案例 7、案例 8 相同，SceneClassifier 使用懒加载 + 自动后端检测：

```

1 _classifier = None
2
3 def get_classifier():
4     global _classifier
5     if _classifier is None:
6         _classifier = SceneClassifier() # 自动检测 NPU/CPU
7     return _classifier

```

6.2.7. 2.7. 用户手册

2.7.1 部署

1. 将项目代码拷贝到昇腾 310B 设备
2. 运行 `bash setup.sh` 安装依赖并导出模型
3. 启动服务：`python3 app.py`
4. 浏览器打开 `http://<设备IP>:7860`

2.7.2 使用

1. 在「道路感知」页签上传一张包含道路的图像
2. 观察车道线检测结果——绿色线条应覆盖路面上的车道标线
3. 查看右侧的场景分类结果和驾驶建议
4. 如果车道线检测不理想，可以在 `config.py` 中调整参数

2.7.3 调整车道线参数

参数	默认值	调高效果	调低效果
CANNY_LOW/HIGH	50 / 150	减少边缘噪点	检测更多边缘
HOUGH_THRESHOLD	50	只保留强直线	检测弱直线
HOUGH_MIN_LINE_LEN	40	过滤短线段	保留短线段
HOUGH_MAX_LINE_GAP	100	容忍更大断连	不连接断裂线
ROI_TOP	0.6	ROI 更小（忽略远处）	ROI 更大（包含远处）

2.7.4 故障排除

问题	可能原因	解决方法
车道线检测不到	图像中无清晰车道线	使用有标线的道路图像，调整 Canny 阈值
车道线位置偏差大	弯道 / 虚线	Hough 直线模型对弯道敏感，可以降低 HOUGH_THRESHOLD
场景分类随机	未加载训练权重	需要在驾驶场景数据集上训练，或使用案例 8 的模式训练
NPU 初始化失败	CANN 环境未配置	<code>source /usr/local/Ascend/</code> <code>↪ ascend-toolkit/set_env.sh</code>
ATC 转换失败	soc_version 不匹配	<code>npu-smi info</code> 查看版本

2.7.5 如何训练场景分类模型

参考案例 8 的 train.py 模式：1. 准备驾驶场景数据集（按 5 个类别分文件夹存放图像）2. 修改案例 8 的 train.py，将模型替换为 ResNet18，类别数改为 5 3. 训练完成后将 .pth 文件放到 models/resnet18_scene.pth 4. 运行 `python3 prepare_models.py --force` 重新导出 ONNX/OM

2.7.6 与案例 2 的配合

案例 2 提供了完整的 SSD 目标检测 + DeepSORT 跟踪。读者可以将案例 2 的 ssdlite/ 模块集成到本案中，实现：

- 1. 车道线检测（本案，经典 CV）
- 2. 目标检测（案例 2，SSD on NPU）——检测行人、车辆、交通标志

3. 场景分类（本案，ResNet18 on NPU）——判断道路类型

4. 目标跟踪（案例 2，DeepSORT）——跟踪移动物体

这四项共同构成一个完整的智能小车视觉感知栈。

6.3. 3. 源代码结构

```

1 case6/
2 |— app.py                # Gradio Web 界面入口
3 |— lane_detector.py      # 经典 CV 车道线检测
4 |— scene_classifier.py   # NPU/CPU 双后端场景分类
5 |— config.py            # 配置常量
6 |— prepare_models.py     # ONNX 导出 & ATC OM 转换
7 |— setup.sh             # 一键环境安装
8 |— requirements.txt      # Python 依赖列表
9 |— data/
10 |   |— .gitkeep
11 |— models/              # 模型文件目录 (.pth / .onnx / .om)
12 |— README.md            # 快速开始指南

```

模块间的调用关系：

```

1 flowchart TB
2   APP["app.py\nGradio 界面入口"] --> LD["lane_detector.py\nLaneDetector\n•\n  ↳ detect()\n• draw_overlay()"]
3   APP --> SC["scene_classifier.py\nSceneClassifier\n• classify()\n• preprocess\n  ↳ ()"]
4   APP --> CFG["config.py\n• SCENE_CLASSES\n• IMAGE_SIZE\n• 车道线参数"]
5
6   SC --> AR["AscendResource\nacl.init / device / context"]
7   SC --> AM["AscendModel\nacl.mdl.execute()"]
8   SC --> TORCH["torchvision\nresnet18(IMGNET1K_V1)"]
9
10  LD --> CV["cv2\nCanny / Hough / GaussianBlur"]
11
12  PREP["prepare_models.py\n模型转换"] --> TORCH
13  PREP --> CFG
14
15  AR --> ACL["acl (CANN)"]
16  AM --> ACL

```

与之前案例的继承关系：

- AscendResource / AscendModel 沿用案例 8 的同一套模式
- config.py 的结构与案例 7/案例 8 一致（BASE_DIR + os.path.join）
- app.py 的 Gradio 懒加载模式与案例 7/案例 8/案例 9 一致
- prepare_models.py 的 ONNX -> ATC 流程与案例 7/案例 8 一致
- LaneDetector 是本书第一个纯经典 CV 类——无需任何模型文件，纯 OpenCV 实现

- 本项目不需要训练脚本——场景分类头尚未训练，但推理框架完整，参考案例 8 即可完成训练

6.4. 4. 效果演示

6.4.1. 预期效果

功能	预期表现	备注
车道线检测	清晰标线检测率 > 90%	对弯道、虚线、阴影敏感
场景分类 (已训练)	5 类准确率 > 85%	需要训练，基线模型随机
端到端处理时间	车道线 <5ms + 推理 ~12ms (NPU)	总计 ~20ms
车道线检测范围	画面下部 40% 区域	ROI 参数可调

6.4.2. 性能指标

指标	NPU (Ascend 310B)	CPU (PyTorch)
车道线检测	<5 ms	<5 ms
场景分类	10-15 ms	25-40 ms
模型大小 (.om)	~45 MB	N/A

6.4.3. 浏览器中的效果

Gradio 界面在浏览器中的预期布局：

1	智能小车视觉感知			
2	车道线检测（经典 CV）+ 驾驶场景分类（ResNet18 on NPU）			
3				
4	[道路感知] [系统信息]			
5				
6				
7	输入道路图像 (上传或拍摄)	感知结果		
8		车道线检测		
9		- 左车道：是	右车道：是	
10		- Hough 线段数：47		
11		驾驶场景分类		
12		- 城市道路 (urban)		
13		置信度：87.3%		
14				
15				
16				

[illegible]

6.4.4. 如何验证系统正常工作

1. 打开浏览器访问 `http://127.0.0.1:7860`
2. 切换到「道路感知」页签
3. 上传一张包含清晰车道线的道路图像
4. 确认车道线叠加图中出现绿色车道线标记
5. 确认右侧感知报告中显示场景分类结果（如未训练则显示随机结果，这是预期行为）
6. 切换到「系统信息」页签，确认后端状态和参数显示正确
7. 在昇腾设备上，确认后端显示“NPU (Ascend 310B)”

7. 案例 7：智能相册

7.1. 1. 项目简介

本项目基于昇腾 310B 平台，构建一个智能照片管理和检索系统。系统使用 **ResNet50** 卷积神经网络提取每张照片的 2048 维视觉特征向量，通过 **FAISS** 向量检索引擎实现毫秒级的相似图片搜索，利用 **OpenCV** 进行人脸计数实现按人物筛选。

与案例 1（人脸识别）的实时摄像头推理不同，本项目是书中第一个演示**批量离线索引 + 在线检索**模式的案例，帮助读者理解 NPU 如何在”先算后查”的场景中发挥作用。

7.1.1. 三个核心功能

- 1. **照片浏览**：按”全部 / 有人脸 / 无人脸”筛选照片库
- 2. **相似搜索**：上传一张照片，找出视觉上最相似的 12 张
- 3. **批量索引**：一键扫描照片文件夹，提取特征向量，构建 FAISS 索引

7.1.2. 与已有案例的关系

案例	领域	NPU 任务	模式
案例 1	人脸识别	SCRFD 检测 + MobileFaceNet 识别	实时摄像头 -> Flask API
案例 7	智能相册	ResNet50 特征提取	批量索引 + FAISS 检索
案例 8	手势识别	MobileNetV3 分类	实时摄像头 -> Gradio
案例 9	聊天机器人	MiniLM 文本嵌入	文本 RAG + 云端 LLM

7.2. 2. 内容大纲

7.2.1. 2.1. 硬件准备

相比旧版案例 7 中列出的扫描仪、4K 显示器、触摸屏、NAS 网络存储等复杂设备，本项目只需要：

- 昇腾 310B 开发者套件：核心计算单元
- 存储设备：存放照片的 USB 硬盘 / SD 卡 / SSD（照片库需要一定容量）
- 显示屏（可选）：用于浏览器访问 Gradio 界面

硬件架构如图所示：

```
1 flowchart LR
2   subgraph STORAGE["存储"]
3     DISK["USB 硬盘 / SD 卡\n照片库"]
4   end
5
6   subgraph COMPUTE["计算平台"]
7     NPU["昇腾 310B NPU\nResNet50 特征提取 -> OM"]
8     CPU["CPU\nOpenCV 人脸计数 + FAISS 检索"]
9   end
10
11  subgraph OUTPUT["输出"]
12    UI["Gradio Web 界面\n浏览 / 搜索 / 管理"]
13  end
14
15  DISK -->|"读取照片"| CPU
16  CPU -->|"预处理张量"| NPU
17  NPU -->|"2048-dim 特征向量"| CPU
18  CPU -->|"FAISS 索引"| UI
```

为什么不需要 GPU 工作站？ResNet50 的特征提取在 310B NPU 上即可高效完成，单张照片推理仅需 5-10ms。对于个人照片库（几百到数千张），310B 完全能在几十秒内完成全库索引。而且模型权重直接由 PyTorch 官方 CDN 提供，无需额外下载第三方模型文件。

7.2.2. 2.2. 软件环境

操作系统与框架

层级	软件	版本	用途
操作系统	Ubuntu	20.04 / 22.04 LTS	运行环境
CANN	Ascend Toolkit	7.0+	NPU 驱动与 ATC 转换工具
Python	CPython	3.8 ~ 3.10	应用开发语言

层级	软件	版本	用途
深度学习	PyTorch + torchvision	2.0+	ResNet50 模型 / CPU 推理回退
图像处理	OpenCV	4.8+	图像预处理 + 人脸检测
向量搜索	FAISS	1.7+	向量相似度检索
Web	Gradio	4.0+	照片画廊 + 搜索界面

环境准备

在昇腾 310B 设备上，运行一键安装脚本：

```
1 # 安装 Python 依赖并导出 ONNX 模型
2 bash setup.sh
```

setup.sh 会依次完成：1. 安装系统包（python3-dev, python3-pip）2. 安装 Python 依赖（pip3 install -r requirements.txt）3. 运行 prepare_models.py 导出 ResNet50 的 ONNX 模型（如有 CANN 则继续转换为 OM）

如果你希望在开发机上只做 ONNX 导出：

```
1 python3 prepare_models.py --onnx-only
```

Python 依赖说明

包	用途	必需？
gradio	Web 界面框架，内置 gr.Gallery 照片墙组件	是
torch + torchvision	ResNet50 模型定义、CPU 特征提取回退	是
opencv-python	图像缩放、色彩转换、归一化、人脸检测	是
numpy	特征向量操作、L2 归一化	是
faiss-cpu	向量相似度搜索（IndexFlatIP）	是
Pillow	Gradio 图像格式转换	是
onnx	ONNX 模型校验（仅 prepare_models.py）	仅转换
acl (CANN 自带)	NPU 推理，随 CANN 安装	仅推理

requirements.txt 文件中已包含所有 pip 可安装的 Python 依赖。acl 包无需手动安装，它随 CANN 一起部署，Python 通过 `import acl` 调用。

7.2.3. 2.3. 图像特征提取原理

什么是图像特征向量

要让计算机判断两张照片是否“相似”，不能直接比较像素——同一物体在不同光照、角度、背景下，像素值完全不同。正确的做法是：用一个训练好的深度神经网络将每张图像映射为一个固定长度的**特征向量（Embedding）**，使得语义相似的图像在向量空间中距离很近。

1	照片 A（海滩日落）	->	ResNet50	->	[0.12, -0.34, 0.78, ..., 0.05]	(2048-dim)
2	照片 B（海边黄昏）	->	ResNet50	->	[0.11, -0.32, 0.80, ..., 0.04]	(2048-dim)
3					↓	
4					余弦相似度 约等于 0.95	（高度相似!）
5						
6	照片 C（会议室）	->	ResNet50	->	[-0.45, 0.67, -0.12, ..., 0.33]	(2048-dim)
7					↓	
8					与 A 的余弦相似度 约等于 0.12	（不相似）

为什么选择 ResNet50

昇腾 310B NPU 最适合运行固定输入输出形状的卷积神经网络。ResNet50 是计算机视觉领域最经典的特征提取骨干网络之一：

模型	特征维度	模型大小 (OM)	NPU 推理	备注
ResNet50	2048-dim	~95 MB	~8ms	推荐, torchvision 内置
MobileNetV3-Small	576-dim	~10 MB	~5ms	案例 8 已使用, 维度较低
EfficientNet-B0	1280-dim	~20 MB	~10ms	也可用
CLIP ViT-B/32	512-dim	~350 MB	一般	Transformer, OM 转换复杂

选择 ResNet50 的理由：

- 1. **零外部下载**: torchvision.models.resnet50(weights=...) 自动从 PyTorch CDN 获取权重，无需维护第三方模型下载链接
- 2. **经典可迁移**: ResNet 是计算机视觉的”标准答案”，“去掉分类头得到特征向量”的模式适用于几乎所有 CNN 骨干网络
- 3. **维度适中**: 2048 维特征向量在表达能力（越高越好）和存储开销（越低越好）之间取得平衡——10,000 张照片的索引仅占 ~80 MB
- 4. **与案例 8 差异化**: 案例 8 使用 MobileNetV3-Small 做手势分类，本案使用 ResNet50 做特征提取——不同的模型、不同的任务类型

去掉分类头：分类模型 -> 特征提取器

torchvision 的 resnet50 默认输出 1000 类的分类概率。我们只需要图像的“语义表示”，不需要分类结果。做法是将最后一层 fc（全连接层）替换为 nn.Identity()（恒等映射）：

```

1 import torchvision.models as models
2
3 model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)
4 model.fc = torch.nn.Identity() # 去掉分类头 -> 输出 2048-dim 特征向量
5 model.eval()

```

这样模型输入 (1, 3, 224, 224) 的图像张量, 输出 (1, 2048) 的特征向量, 而不是 (1, $\rightarrow$ 1000) 的类别概率。

L2 归一化: 让内积等于余弦相似度

FAISS 的 IndexFlatIP (内积索引) 计算的是向量点积。为了让点积等价于余弦相似度, 需要先将所有特征向量做 L2 归一化 (模长为 1):

```

1 norm = np.linalg.norm(vec)
2 if norm > 0:
3     vec = vec / norm # 归一化后 |vec| = 1, 点积 = 余弦相似度

```

图像预处理流程

```

1 flowchart TD
2     INPUT["输入图像\nBGR (H, W, 3)"] --> RESIZE["cv2.resize\n-> 224×224"]
3     RESIZE --> RGB["cv2.cvtColor\nBGR -> RGB"]
4     RGB --> NORM["归一化\npixel/255 -> (pixel-mean)/std\nImageNet 统计值"]
5     NORM --> TRANSPOSE["维度重排\nHWC -> CHW -> NCHW\n(1, 3, 224, 224)"]

```

预处理代码在 `feature_extractor.py` 的 `FeatureExtractor.preprocess()` 方法中, 与案例 8 的手势识别预处理完全一致。

7.2.4. 2.4. 模型转换与昇腾部署

`torchvision` 提供的 ResNet50 权重是 PyTorch 格式, 需要通过两次转换才能在 310B NPU 上运行。

```

1 flowchart TD
2     subgraph EXPORT["1. ONNX 导出 (开发机 / 昇腾设备)"]
3         TORCH["torchvision\nresnet50(IMAGENET1K_V1)\nmodel.fc = nn.Identity()"]
4         ONNX["resnet50_feature.onnx\n输入 (1,3,224,224)\n输出 (1,2048)"]
5     end
6
7     subgraph CONVERT["2. ATC 转换 (昇腾设备)"]
8         OM["resnet50_feature.om\nAscend 离线模型 ~95MB"]
9     end
10
11    subgraph DEPLOY["3. 推理部署"]
12        APP["app.py -> FeatureExtractor\n-> AscendModel.execute()"]
13    end
14
15    TORCH --> |"torch.onnx.export()\nopset=11"| ONNX

```



```
16 ONNX -->|"atc --framework=5\n--soc_version=Ascend310B4"| OM
17 OM -->|"acl.mdl.execute()"| APP
```

步骤 1: 导出 ONNX

```
1 # 仅导出 ONNX (可在开发机上运行, 不需要昇腾硬件)
2 python3 prepare_models.py --onnx-only
```

这一步会: 1. 从 torchvision 加载预训练的 ResNet50 (ImageNet 权重) 2. 将 fc 层替换为 nn.Identity() 3. 用 torch.onnx.export() 导出为 ONNX 格式 4. 输入形状固定为 (1, 3, 224, 224), 输出形状固定为 (1, 2048)

关键代码见 [prepare_models.py](#) 中的 export_onnx() 函数。注意 dynamic_axes={} 参数——我们刻意禁用了动态形状, 确保与 ATC 转换兼容。

步骤 2: ATC 转换 ONNX -> OM

```
1 # 在昇腾设备上运行
2 python3 prepare_models.py
```

这一步调用 CANN 的 atc 工具:

```
1 atc --model=models/resnet50_feature.onnx \
2     --framework=5 \
3     --output=models/resnet50_feature \
4     --soc_version=Ascend310B4 \
5     --input_format=NCHW \
6     --input_shape=input:1,3,224,224
```

参数说明:

参数	值	说明
--model	ONNX 文件路径	输入的 ONNX 模型
--framework	5	5 = ONNX 格式
--output	输出路径	生成的 .om 文件路径
--soc_version	Ascend310B4	芯片型号
--input_format	NCHW	输入数据排布格式
--input_shape	input:1,3,224,224	固定输入形状

soc_version 通过 npu-smi info 自动检测, 默认值为 Ascend310B4。

与案例 8 的模型转换对比

对比项	案例 8 (手势识别)	案例 7 (智能相册)
模型来源	自己训练 (.pth)	torchvision 预训练
模型架构	MobileNetV3-Small	ResNet50
输出	10 类 logits	2048-dim 特征向量
需要训练?	是 (或下载.pth)	否 (torchvision 直接提供)
外部下载	预训练.pth + HaGRID 数据集	无 (PyTorch CDN 自动拉取)

本案不需要训练脚本——这是案例 8 已经覆盖的内容。本案展示的是另一种常见的边缘 AI 模式：直接使用预训练模型做特征提取，无需微调。

7.2.5. 2.5. 照片索引与向量检索

整体流程

```

1 flowchart TD
2   PHOTOS["照片文件夹\n(几十 ~ 几千张)"] --> SCAN["扫描图像文件\n.jpg .jpeg .\n↔ png .bmp"]
3   SCAN --> LOOP["逐张处理"]
4
5   subgraph PER_PHOTO["每张照片"]
6     READ["cv2.imread()\n读取"] --> FACE["OpenCV Haar Cascade\n人脸计数"]
7     READ --> FEATURE["ResNet50 (NPU)\n提取 2048-dim 特征"]
8     FACE --> META["记录元数据\n{文件名, 人脸数, 路径}"]
9     FEATURE --> ADD["faiss.add()\n加入向量索引"]
10  end
11
12  LOOP --> PER_PHOTO
13  META --> SAVE["持久化\nphoto_index.faiss\nphoto_metadata.json"]
14  ADD --> SAVE

```

PhotoIndex 类设计

`photo_index.py` 中的 `PhotoIndex` 类管理整个照片索引的生命周期：

- `index_photos(photo_dir)` — 扫描目录，逐张提取特征，构建 FAISS 索引
- `search(query_image, k)` — 提取查询图像特征，返回 Top-K 相似照片
- `get_all_photos()` / `get_photos_by_face_count()` — 按条件筛选照片
- `save()` / `load()` — 将 FAISS 索引和元数据持久化到磁盘

为什么用 IndexFlatIP (暴搜)

`faiss.IndexFlatIP` 是 FAISS 提供的最简单的索引——不做任何近似或压缩，直接计算查询向量与库中所有向量的内积。对于个人照片库（通常几百到几千张），暴搜完全够用：

- 在 10,000 张照片的库中搜索一次约需 1-2ms (2048 维 × 10,000 次点积)
- 不需要训练 (IVF 需要)、不损失精度 (PQ 会降低召回率)
- 索引文件大小 = 照片数 × 2048 × 4 字节, 10,000 张约 80 MB

当照片库增长到数万张以上时,可以将 IndexFlatIP 替换为 IndexIVFFlat 或 IndexHNSWFlat
→ , 在精度和速度之间做权衡。

NPU 批量推理的实际情况

OM 模型的输入形状固定为 (1, 3, 224, 224) (batch size = 1), 所以 NPU 模式下的”批量索引”实际上是逐张调用 execute()。对于几百张照片的索引, 总耗时完全可以接受 (每张 ~8ms, 100 张不到 1 秒)。

CPU 回退模式反而可以利用 PyTorch 的动态批处理能力——将多张照片堆叠为 (N, 3, → 224, 224) 的张量, 一次前向传播处理整个批次。这是 OM 模型的一个已知限制, 书中已在多处提及 (参见案例 8、案例 9 中的相关讨论)。

人脸检测：为什么只用 Haar Cascade

本案例使用 OpenCV 内置的 Haar Cascade 分类器做人脸计数, 而不是使用案例 1 中的 SCRFD NPU 模型做人脸识别：

方面	Haar Cascade (本案)	SCRFD (案例 1)
任务	数人脸 -> 几人	检测 + 提取人脸特征 -> 是谁
运行位置	CPU	NPU (OM)
设置成本	零 (OpenCV 自带)	需下载模型 + ATC 转换
单张耗时	<10ms (CPU)	~5ms (NPU) + 前后处理

对于智能相册的”按人数筛选”需求, 知道一张照片里有几个人就足够了。如果需要完整的人脸识别 (标记每个人是谁), 读者可以将案例 1 的 FaceSystem 集成到本案中——两个案例的 AscendResource / AscendModel 基础设施是相同的。

持久化

FAISS 索引通过 faiss.write_index() 保存为二进制文件, 照片元数据保存为 JSON。这与案例 9 的知识库持久化模式完全一致。下次启动时, 如果检测到已有索引文件, 会自动加载, 无需重新索引。

7.2.6. 2.6. Web 界面

本项目使用 Gradio 构建 Web 界面，通过 `gr.Gallery` 实现照片墙展示，无需任何前端代码。

```

1 flowchart TD
2     subgraph UI["Gradio Blocks 界面"]
3         TAB1[" 照片浏览 (Tab 1)"]
4         TAB2[" 相似搜索 (Tab 2)"]
5         TAB3[" 管理 (Tab 3)"]
6     end
7
8     subgraph TAB1_DETAIL["浏览 Tab"]
9         FILTER["Radio\n全部 / 有人脸 / 无人脸"] --> GALLERY1["gr.Gallery\n照片缩  
    ↳ 略图 + 人脸数"]
10    end
11
12    subgraph TAB2_DETAIL["搜索 Tab"]
13        UPLOAD["gr.Image\n上传查询照片"] --> SEARCH["search_similar()"]
14        SEARCH --> GALLERY2["gr.Gallery\nTop-12 相似照片"]
15        SEARCH --> REPORT["Markdown\n相似度分数 + 进度条"]
16    end
17
18    subgraph TAB3_DETAIL["管理 Tab"]
19        FOLDER["Textbox\n照片目录路径"] --> INDEX["index_folder()"]
20        INDEX --> PROGRESS["Progress\n索引进度条"]
21        INDEX --> PREVIEW["Gallery\n索引预览 (前20张)"]
22        INDEX --> STATS["Markdown\n系统信息 / 索引统计"]
23    end
24
25    TAB1 --> TAB1_DETAIL
26    TAB2 --> TAB2_DETAIL
27    TAB3 --> TAB3_DETAIL

```

三个页签

1. **照片浏览**：`gr.Gallery` 以网格形式展示所有已索引的照片，每张照片下方标注文件名和人脸数。通过单选按钮筛选“全部 / 有人脸 / 无人脸”。
2. **相似搜索**：上传一张查询照片，系统提取其特征向量后在 FAISS 索引中搜索最相似的 12 张照片。结果以画廊形式展示，同时以 Markdown 表格列出每张照片的相似度分数和进度条。
3. **管理**：输入照片文件夹路径，点击“开始索引”即可批量处理整个文件夹。`gr.Progress` 组件实时显示索引进度。索引完成后展示前 20 张照片预览和系统统计信息。

双后端切换

`FeatureExtractor` 在初始化时自动检测 NPU 可用性：

```

1 class FeatureExtractor:
2     def _init_backend(self):
3         if os.path.exists(OM_MODEL_PATH):

```

```

4         try:
5             self._acl_resource = AscendResource(NPU_DEVICE_ID)
6             self._om_model = AscendModel(...)
7             self.use_npu = True      # NPU 模式
8             return
9         except Exception:
10            pass
11            self._init_cpu_backend()      # CPU 回退

```

这意味着：- 在昇腾 310B 上 -> 自动使用 OM 模型，NPU 特征提取 - 在没有昇腾硬件的机器上 -> 自动使用 PyTorch ResNet50，CPU 推理 - 同一份代码，无需修改任何配置

懒加载

与案例 8、案例 9 相同，FeatureExtractor 和 PhotoIndex 使用模块级懒加载：

```

1 _extractor = None
2 _photo_index = None
3
4 def get_extractor():
5     global _extractor
6     if _extractor is None:
7         _extractor = FeatureExtractor()
8     return _extractor

```

这避免了 import 时的重操作，只在首次使用时初始化模型。

7.2.7. 2.7. 用户手册

2.7.1 部署

1. 将项目代码拷贝到昇腾 310B 设备
2. 运行 `bash setup.sh` 安装依赖并导出模型
3. (可选) 在「管理」页签中准备测试照片文件夹
4. 启动服务：`python3 app.py`
5. 浏览器打开 `http://<设备IP>:7860`

2.7.2 索引照片

1. 在「管理」页签的“照片目录路径”中输入照片文件夹路径
2. 点击“开始索引”
3. 观察进度条，等待索引完成
4. 索引会自动保存，下次启动无需重新索引

索引速度参考（100 张照片）：

后端	耗时
NPU (Ascend 310B)	~1-2 秒
CPU (PyTorch)	~3-5 秒

2.7.3 浏览和搜索

1. 在「照片浏览」页签查看已索引的照片
2. 使用筛选按钮按人脸数过滤
3. 在「相似搜索」页签上传任意照片，查看相似结果
4. 相似度分数反映两张照片在 ResNet50 特征空间中的余弦距离
 - 90%: 高度相似（同类场景 / 同一物体）
 - 70-90%: 有一定相似性（色彩 / 构图接近）
 - < 70%: 视觉差异较大

2.7.4 故障排除

问题	可能原因	解决方法
索引时找不到照片	目录路径错误	检查路径是否正确，支持绝对路径
照片不显示在画廊中	文件格式不支持	支持.jpg / .jpeg / .png / .bmp / .webp
人脸数始终为 0	Haar Cascade 漏检	正常现象——正面清晰人脸检测率更高
搜索结果不相关	ResNet50 特征侧重场景	它对场景/物体/色彩敏感，对人脸身份不敏感
NPU 初始化失败	CANN 环境未配置	<code>source /usr/local/Ascend/</code> <code>↪ ascend-toolkit/set_env.sh</code>
ATC 转换失败	soc_version 不匹配	<code>npu-smi info</code> 查看版本，脚本会自动检测
ONNX 导出失败	onnx 包未安装	<code>pip install onnx</code>

2.7.5 扩展建议

1. 集成人脸识别：将案例 1 的 FaceSystem（SCRFD + MobileFaceNet）加入本案，实现“按人物搜索”和“面孔聚类”

2. **文本搜索**：接入案例 9 的文本嵌入模型，支持用自然语言描述搜索照片（如“海边的日落”）
3. **大规模索引**：照片超过 10,000 张时，将 IndexFlatIP 替换为 IndexIVFPQ 以压缩索引体积和加速检索
4. **增量索引**：监控照片文件夹变化，自动索引新增照片
5. **更大模型**：将 ResNet50 替换为 ResNet101 或 ConvNeXt，获得更强的特征表达能力（模型更大，转换方式相同）

7.3. 3. 源代码结构

```

1 case7/
2 |— app.py                # Gradio Web 界面入口
3 |— feature_extractor.py  # NPU/CPU 双后端特征提取 (ResNet50)
4 |— photo_index.py       # FAISS 索引管理 + 人脸检测
5 |— config.py            # 配置常量 (模型路径、文件路径、阈值)
6 |— prepare_models.py    # ONNX 导出 & ATC OM 转换
7 |— setup.sh             # 一键环境安装
8 |— requirements.txt      # Python 依赖列表
9 |— data/
10 |   |— .gitkeep
11 |   |— photo_index.faiss # FAISS 索引 (运行时生成)
12 |   |— photo_metadata.json # 照片元数据 (运行时生成)
13 |— models/              # 模型文件目录 (.onnx / .om)
14 |— photos/              # 默认照片目录
15 |— README.md            # 快速开始指南

```

模块间的调用关系：

```

1 flowchart TB
2   APP["app.py\nGradio 界面入口"] --> FE["feature_extractor.py\n↪ nFeatureExtractor\n• extract()\n• preprocess()"]
3   APP --> PI["photo_index.py\nPhotoIndex\n• index_photos()\n• search()\n• ↪ get_all_photos()"]
4   APP --> CFG["config.py\n• FEATURE_DIM\n• IMAGE_SIZE\n• TOP_K_RESULTS"]
5
6   FE --> AR["AscendResource\nnacl.init / device / context"]
7   FE --> AM["AscendModel\nnacl.mdl.execute()"]
8   FE --> TORCH["torchvision\nresnet50(IMAGENET1K_V1)"]
9
10  PI --> FE
11  PI --> FAISS["faiss\nIndexFlatIP"]
12  PI --> CV["cv2\nHaar Cascade"]
13
14  PREP["prepare_models.py\n模型转换"] --> TORCH
15  PREP --> CFG
16
17  AR --> ACL["acl (CANN)"]
18  AM --> ACL

```

与之前案例的继承关系:

- AscendResource / AscendModel 沿用案例 8 的同一套模式 (案例 8 又继承自案例 1)
- config.py 的路径管理方式与案例 9 一致 (BASE_DIR + os.path.join)
- app.py 的 Gradio 模式与案例 8/案例 9 一致 (gr.Blocks + 懒加载)
- prepare_models.py 的 ONNX -> ATC 流程与案例 8 一致
- PhotoIndex 的 FAISS 持久化模式与案例 9 的 KnowledgeBase 一致
- 本项目不需要训练脚本——直接使用 torchvision 预训练权重, 这是与案例 8 最大的区别

7.4. 4. 效果演示

7.4.1. 预期效果

在正常使用条件下, 系统的各功能预期表现:

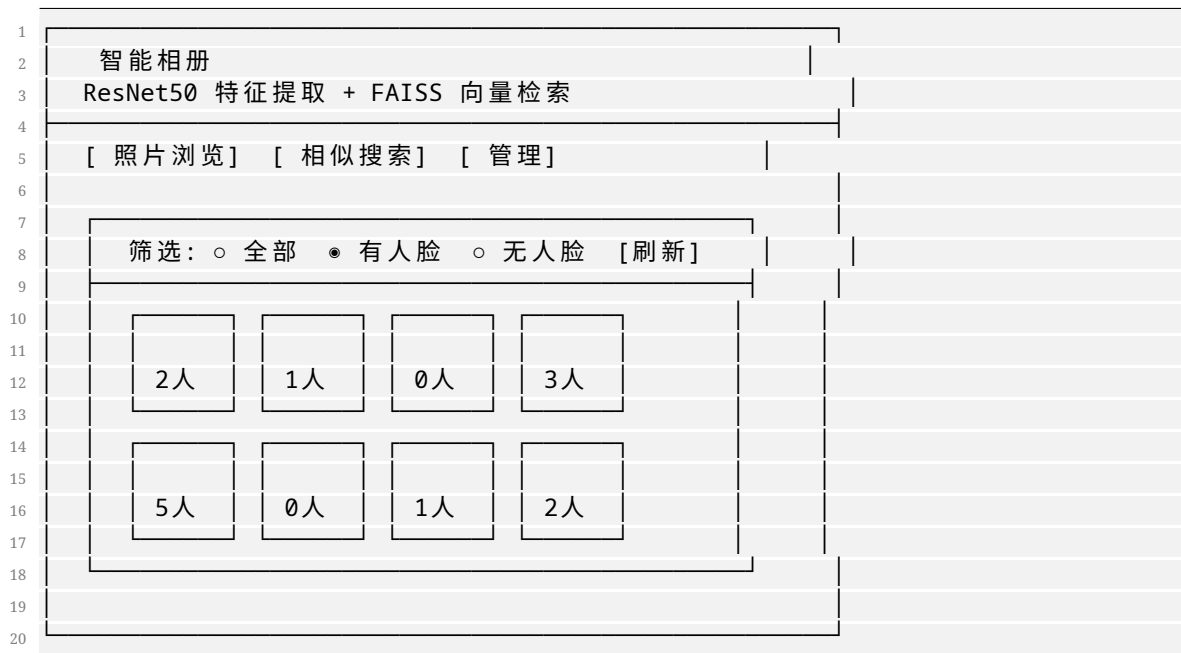
功能	预期表现	备注
照片索引	100 张照片 < 2 秒 (NPU)	含特征提取 + 人脸检测
相似搜索	< 20ms / 次	特征提取 ~8ms + FAISS 搜索 ~2ms
人脸计数	正面清晰人脸检测率 > 85%	Haar Cascade 对侧脸/遮挡敏感
相似结果相关性	同类场景/物体的照片排前面	受 ResNet50 ImageNet 训练域影响

7.4.2. 性能指标

指标	NPU (Ascend 310B)	CPU (PyTorch)
单张特征提取	5-10 ms	20-40 ms
100 张批量索引	1-2 秒	3-5 秒
FAISS 搜索 (1000 张)	~0.5 ms	~0.5 ms
模型大小 (.om)	~95 MB	N/A
FAISS 索引 (10000 张)	~80 MB	~80 MB

7.4.3. 浏览器中的效果

Gradio 界面在浏览器中的预期布局：



7.4.4. 如何验证系统正常工作

1. 打开浏览器访问 <http://127.0.0.1:7860>
2. 切换到「管理」页签，输入包含一些照片的文件夹路径
3. 点击“开始索引”，等待进度条完成
4. 切换到「照片浏览」页签，确认照片显示在画廊中
5. 切换到「相似搜索」页签，上传一张照片，确认返回 12 张相似结果
6. 在「管理」页签中确认系统信息显示正确的后端（NPU 或 CPU）
7. 关闭并重启 `app.py`，确认索引被正确保存和自动加载

8. 案例 8：实时手势识别

8.1. 项目简介

本案例在昇腾 310B 上实现一个实时手势检测系统。系统从 USB 摄像头读取图像, 用 HaGRIDv2 提供的 YOLOv10 手势检测模型完成目标检测, 再把检测框画回原始画面, 通过 WebRTC 推送到远程浏览器。整个流程覆盖了边缘视觉项目中最常见的几件事: 模型从 PyTorch 导出为 ONNX, 使用 ATC 转换为 OM, 通过 AscendCL/PyACL 在 NPU 上推理, 并把推理结果组织成可远程查看的实时视频服务。

本案例采用目标检测路线, 而不是只对裁剪后的单张手部图片做分类。YOLOv10 可以直接处理完整摄像头画面, 同时输出手势类别、置信度和目标框坐标。本章关注的核心问题是: 如何在真实视频流中找到手势、把检测框正确画回原始图像, 并把结果稳定地推送到远程浏览器。

代码位于 samples/case8。其中 hagrid_yolo 是可复用 Python 包, 保存预处理、后处理和推理后端; scripts 保存命令行入口, 包括 ONNX 验证、ATC 转换、OM 推理和 WebRTC 服务; webrtc_app 保存 CANN VENC、DVPP JPEGD 和 V4L2 采集相关适配代码; web 保存浏览器前端; weights 暂时保存 PyTorch 权重和导出脚本, 后续可以单独迁移到模型仓库。

下方架构图说明了本案例的部署关系。PyTorch 到 ONNX 的导出建议在 PC 或 GPU 工作站完成; 310B 侧负责 ATC 转换、OM 推理、摄像头采集和 WebRTC 推流。

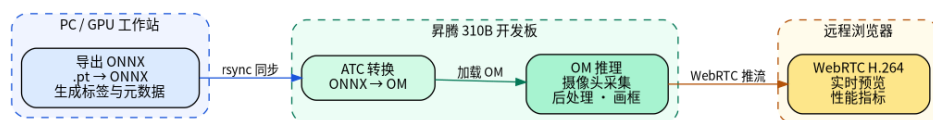


图 1: case8 系统架构。

8.2. 学习目标

完成本案例后, 你将能够:

- 在 GPU 工作站上将 YOLO 格式的 .pt 权重导出为 ONNX 模型, 并生成配套的标签和元数据;
- 在昇腾 310B 上使用 ATC 将 ONNX 转换为 OM 离线模型;
- 使用 PyACL 加载 OM 模型, 编写 NPU 推理的预处理与后处理代码;

- 搭建 WebRTC H.264 推流服务，在浏览器中远程查看实时检测结果；
- 理解 YOLOv10 的无 NMS 推理机制与 HaGRID 数据集的构成。

预备知识：需要熟悉 Python 编程和 Linux 命令行基础操作，了解神经网络的基本概念（卷积、池化、全连接）。本案例使用的 YOLOv10 模型由 HaGRIDv2 官方预训练提供，无需自行训练。完整走通全流程约 2~3 小时。

8.3. 实验环境

本案例只需要一块昇腾 310B 开发板和一个普通 USB 摄像头。摄像头最好支持 MJPG 输出——MJPG 是摄像头的压缩输出格式，相比未压缩的 YUYV 占用的 USB 带宽更少，更容易在 1280x720 及以上分辨率达到 30fps（详见「OpenCV 与 DVPP 采集后端」章节）。显示器不是必需的，远程浏览器可以直接查看 WebRTC 视频流。

310B 运行时需要 CANN、ATC、PyACL、OpenCV、aiortc 和 av 等组件。Python 运行依赖已经写在 samples/case8/requirements.txt 中，可以在 310B 的虚拟环境里安装：

```
1 cd ~/Documents/Ascend310/samples/case8
2 pip install -r requirements.txt
```

ac1 模块由 CANN 提供，不是 pip 安装的普通 Python 包。进入实验目录前通常需要先加载 CANN 环境并激活 Python 虚拟环境：

```
1 source /usr/local/Ascend/ascend-toolkit/set_env.sh
2 conda activate npu
3 cd ~/Documents/Ascend310/samples/case8
```

本教程中的设备示例 hostname 为 313，虚拟环境名为 npu。如果你的开发板名称或环境不同，只需要替换命令中的对应字段。

8.4. HaGRID 手势检测任务

手势识别可以做成分类任务，也可以做成检测任务。分类任务把已经裁剪好的手部图像送入模型，输出一个类别，例如 ok、stop 或 like。这种方式实现简单，但对输入画面要求较高：手需要占据主要区域，背景不能太复杂，一旦出现多只手或手离摄像头较远，分类模型就很难判断。检测任务则直接接收完整图像，输出一个或多个检测框，每个框都带有类别和置信度。对于摄像头实时应用，检测任务更贴近实际场景，因为用户不会总是把手放在画面正中，画面中也经常会出现身体、背景和多只手。

HaGRID 是 **H**and **G**esture **R**ecognition **I**mage **D**ataset 的缩写，是面向手势识别系统的大规模 RGB 图像数据集。初版 HaGRID 于 2022 年首次公开，包含约 552,992 张 FullHD RGB 图像，覆盖 18 类手势，并提供 no_gesture 类来降低误检。HaGRIDv2 进一步扩展到约 1,086,158 张 FullHD RGB 图像，包含 33 类手势和单独的 no_gesture 类，并按照 user_id 划分训练、验

证和测试集。这个划分方式很重要，因为它能减少同一个人同时出现在训练集和测试集中的情况，更接近真实泛化能力评估。

HaGRIDv2 对边缘部署很有价值。它不是只在干净背景下拍摄单只手，而是包含自然室内场景、不同光照、不同距离和不同人群。数据中的手势手和非手势手也可能同时出现，这使得模型不仅要识别手势，还要学会避免把普通手部姿态误判成命令手势。本案例直接使用 HaGRIDv2 官方提供的 YOLOv10 权重，把重点放在部署、推理和实时视频优化，而不是重新训练数据集。

当前 samples/case8/models 中包含 YOLOv10n_gestures、YOLOv10x_gestures、YOLOv10n_hands 和 YOLOv10x_hands 四组模型。其中 YOLOv10n_gestures 是默认模型，标签数为 34，完整类别列表如下（推理时 class_id 即对应此表编号）：

ID	标签	中文名称	手势说明
0	grabbing	抓取	五指弯曲，做抓握动作
1	grip	握紧	五指紧握成拳
2	holy	祈祷	双手合十于胸前
3	point	指向	食指伸出，其余握拢
4	call	打电话	拇指与小指伸出，模拟电话
5	three3	三指展开	拇指、食指、中指伸出展开
6	timeout	暂停	双手呈 T 形
7	xsign	交叉	双手食指交叉成 X 形
8	hand_heart	手指比心	单手拇指与食指交叉成心形
9	hand_heart2	双手比心	双手合拢围成心形
10	little_finger	小指	伸出小拇指
11	middle_finger	中指	伸出中指
12	take_picture	拍照	模拟按下相机快门
13	dislike	踩	拇指向下，表示不喜欢
14	fist	拳头	五指紧握成拳，拳面向前
15	four	四指	伸出四根手指
16	like	点赞	拇指向上，表示喜欢
17	mute	静音	食指竖起放在嘴前
18	ok	OK	拇指与食指成圈，其余三指伸直
19	one	一指	伸出食指，表示数字 1
20	palm	手掌	五指张开，掌心向前
21	peace	剪刀手	食指与中指伸出呈 V 形，掌心向外
22	peace_inverted	反手剪刀手	V 形手势掌心向内
23	rock	摇滚	食指与小指伸出，其余握拢
24	stop	停止	五指张开，掌心向前
25	stop_inverted	反手停止	手背向前，五指张开

ID	标签	中文名称	手势说明
26	three	三指	拇指、食指、中指伸出
27	three2	三指并拢	食指、中指、无名指并拢伸出
28	two_up	两指向上	食指与中指并拢向上伸出
29	two_up_inverted	反手两指	手背向外，食指中指向上
30	three_gun	手枪	拇指与食指伸出成枪形
31	thumb_index	捏合	拇指与食指指尖捏合
32	thumb_index2	展开	拇指与食指展开成 L 形
33	no_gesture	无手势	无特定手势或背景

标签文件存储在 models/<模型名>_labels.txt 中,由 hagrid_yolo/metadata.py 中的 load_labels
→ () 自动加载。48 类模型 (YOLOv10x_hands) 在以上 34 类基础上增加了 14 个左右手区分变体。

YOLOv10n 和 YOLOv10x 的模型输入都是 1,3,640,640，差异来自模型规模和计算量，而不是输入分辨率。摄像头可以采集 640x480、1280x720 或 1920x1080，但进入模型前都会先等比例缩放，再填充到 640×640。

四个 HaGRID YOLOv10 OM 模型的纯推理性能

下表汇总了四个已转换 OM 模型的实测结果。测试在昇腾 310B 设备（示例主机名 313）上执行，命令如下：

```
1 python scripts/infer_om_camera.py \  
2 --benchmark-runs 80 \  
3 --warmup-runs 10 \  
4 --print-model-info
```

输入为脚本生成的全零张量，因此结果只代表 OM 后端推理耗时，不包含摄像头采集、预处理、后处理、画框、颜色转换和 WebRTC 编码。

模型简称	类别数	模型大小	推理平均延迟
n_gestures	34	6.4 MB	18.29 ms
n_hands	34	6.4 MB	18.41 ms
x_gestures	34	64 MB	122.61 ms
x_hands	48	65 MB	124.64 ms

表中模型简称均省略 YOLOv10 前缀，例如 n_gestures 对应 YOLOv10n_gestures.om。
四个模型的选择建议如下：

- YOLOv10n_gestures 是默认模型，速度最快，适合实时手势命令检测。

- YOLOv10n_hands 的标签集合与 n_gestures 相同，适合对比误检和召回表现。
- YOLOv10x_gestures 模型规模明显更大，适合离线精度对比，不适合每帧 30fps 推理。
- YOLOv10x_hands 类别更多，包含若干左右手细分类；延迟最高，适合精度优先场景。

8.5. YOLOv10 与本案例模型

YOLOv10 是一种实时目标检测模型，由清华大学 MIG 课题组于 2024 年 5 月发布。YOLO 系列的基本思想是单阶段检测——一次前向计算同时预测目标框、置信度和类别。YOLOv10 在此基础上重点解决了一个工程问题：推理阶段不再依赖 NMS（非极大值抑制）后处理，同时从模型结构上减少冗余计算。

下图展示了 YOLOv10 的三段式架构（Backbone → Neck → Head）：

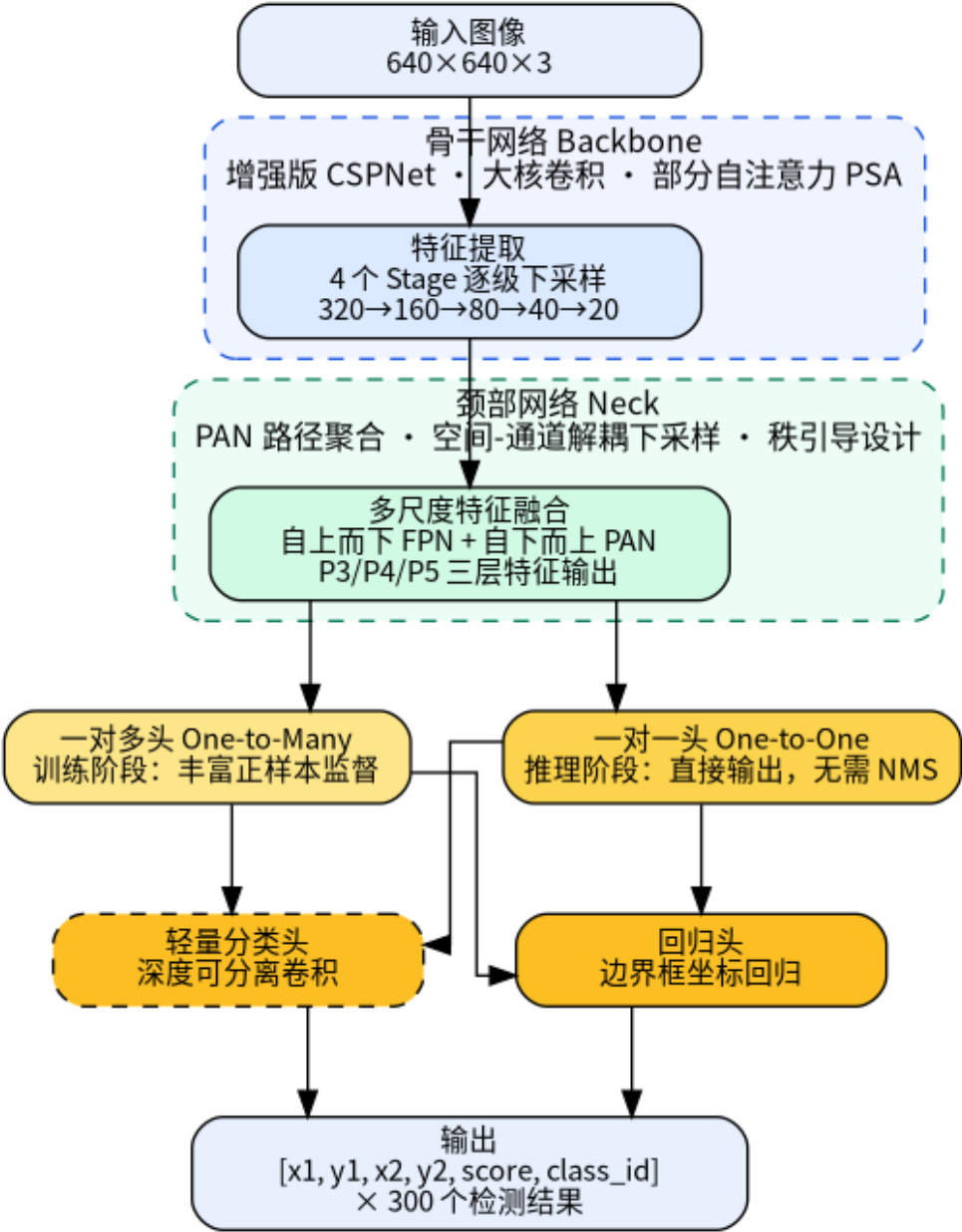


图 2：YOLOv10 总体框架——输入经骨干网络提取特征，颈部网络融合多尺度信息，最终由双重检测头输出检测结果。

骨干网络 (Backbone)

骨干网络基于增强版 CSPNet，通过 4 个 Stage 逐级下采样，提取从浅层纹理到深层语义的多尺度特征：

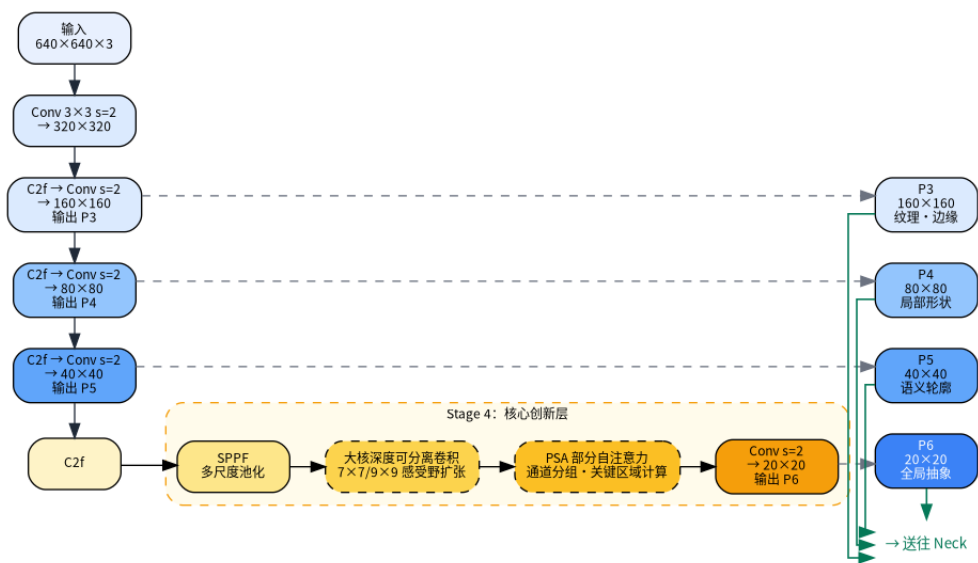


图 3：YOLOv10 骨干网络——Stem + 4 个 Stage 逐级下采样，Stage 4 包含 SPPF、大核卷积和 PSA 三个关键创新。

Stage	输出尺寸	主要模块	提取的特征
Stem	320×320	Conv 3×3, s=2	初始降采样，扩充通道数
Stage 1	160×160	C2f + Conv	纹理、边缘等浅层细节（输出 P3）
Stage 2	80×80	C2f + Conv	局部形状、角点等中层结构（输出 P4）
Stage 3	40×40	C2f + Conv	语义信息，如手部整体轮廓（输出 P5）
Stage 4	20×20	C2f + SPPF + 大核卷积 + PSA	全局抽象特征（输出 P6）

Stage 4 包含了骨干网络的主要改进：

- **SPPF（空间金字塔池化）**：用多个不同尺寸的池化核并行处理同一张特征图，再把结果拼接起来，使模型能同时看到局部细节和更大范围的上下文。
- **大核深度可分离卷积（7×7 或 9×9）**：普通 3×3 卷积每次只能看到相邻像素，要堆很多层才能覆盖大范围。直接把卷积核放大，并用深度可分离的方式（先逐通道做空间卷积，再逐点做通道融合）来控制参数增长。
- **PSA（部分自注意力）**：卷积的局限在于每个像素只能看到周围一小圈，缺乏”全局视野”。自注意力能让每个像素看到整张图，但计算量太大。PSA 的做法是把特征图按通道一分

为二，一半保留卷积结果，另一半只在关键区域内计算自注意力，兼顾全局感知和计算效率。

颈部网络（Neck）

颈部网络采用 PAN（路径聚合网络）结构，通过两条方向相反的路径融合骨干网输出的多尺度特征：

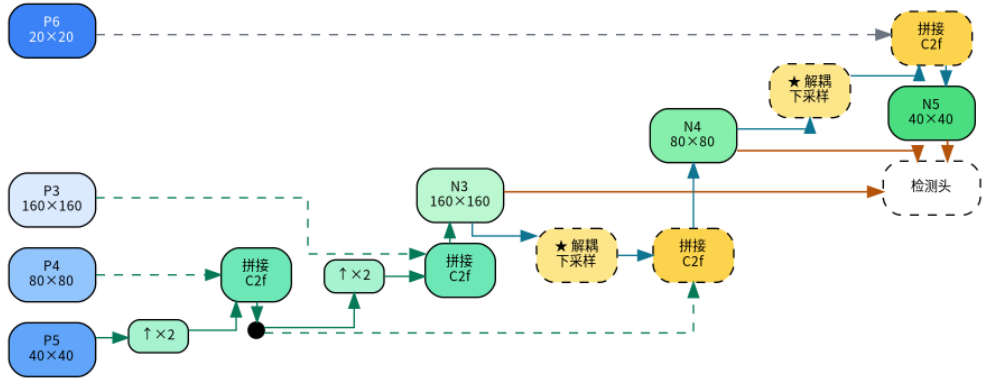


图 4：YOLOv10 颈部网络——FPN 自上而下传递语义信息，PAN 自下而上传递定位信息，★ 标记的模块为空间-通道解耦下采样。

通路	方向	操作	解决的问题
FPN（自上而下）	P5 → P4 → P3	上采样 + 拼接 + C2f	把深层的” 这是什么” 传给浅层，利于小目标检测
PAN（自下而上）	N3 → N4 → N5	解耦下采样 + 拼接 + C2f	把浅层的” 这在哪里” 传给深层，利于精确定位

两条通路中的关键设计：

- **空间-通道解耦下采样**：传统 stride=2 卷积一步完成空间压缩和通道变换，信息损失较大。解耦下采样把这两步分开——先用池化做纯空间降维，再用 1×1 卷积调整通道，信息保留更完整。
- **秩引导模块设计**：不同 Stage 的特征图信息量不同——浅层特征丰富（矩阵的秩较高），用较大的 C2f 模块；深层特征经过多次压缩后信息密度降低（秩较低），用小型 C2f 即可，避免统一配置带来的参数浪费。

检测头 (Head)

检测头是 YOLOv10 与以往 YOLO 版本最大的不同。传统 YOLO 推理后会输出大量重叠的预测框，需要 NMS 去重——NMS 本身较慢，且阈值调不好容易误删正确结果。YOLOv10 的解决方案是**一致性双重分配 (Consistent Dual Assignments)**：

- **训练阶段**：同时使用一对多头（一个真实目标匹配多个预测框，提供丰富监督）和一对一头（一个目标只匹配一个最优框，学习直接输出“最佳答案”）。两个头共享统一的匹配评分公式 $m = s^\alpha \times u^\beta$ （ s 为分类得分， u 为预测框与真实框的 IoU， α 和 β 为平衡系数），保证学习方向一致。
- **推理阶段**：只保留一对一头。因为模型在训练时已学会直接给出最佳预测，推理输出天然没有重叠框，不再需要 NMS。
- **轻量级分类头**：分类分支用深度可分离卷积代替标准卷积，进一步减少参数量。

对本案例来说，更需要理解的是导出后的部署形式：模型接收固定大小的 NCHW 图像张量，输出检测结果，后处理再把检测框映射回摄像头原图。

当前导出的模型输出可以理解为若干行检测结果，每一行至少包含 [x1, y1, x2, y2,
→ score, class_id]。虽然 YOLOv10 论文强调 NMS-free，本案例的 postprocess.py 仍然保留了一次 OpenCV NMS。这不是理论上的必要步骤，而是工程兼容措施：不同导出版本可能产生略有差异的输出，保留 NMS 可以让教程代码在更换模型时更稳健。对当前 HaGRIDv2 YOLOv10 导出模型而言，这一步不是主要性能瓶颈。

8.6. 模型导出与 ATC 转换

模型准备流程见下方流程图。

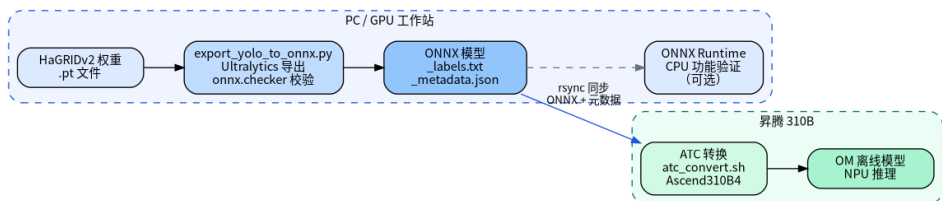


图 5：case8 模型转换流程。

PyTorch 权重到 ONNX 的导出不建议在 310B 上完成。这个步骤依赖 PyTorch、Ultralytics 和 ONNX 工具，更适合放在 PC 或 GPU 工作站上。仓库中 weights/export_yolo_to_onnx.py 就是为这一步准备的，它跟随权重文件放在 weights 目录中，后续可以与样例代码仓库分离。

PC 或 GPU 工作站上的导出环境需要额外安装 PyTorch、Ultralytics 和 ONNX 工具。下面是一组已经验证过的依赖组合：

```
1 pip install numpy==1.26.4 onnx==1.14.1 onnxruntime==1.15.1 opencv-python
   → ==4.8.0.76
```

```

2 pip install torch==2.10.0 torchvision==0.25.0 --extra-index-url https://download
  ↪ .pytorch.org/whl/cu128
3 pip install ultralytics==8.4.60

```

导出 YOLOv10n_gestures 的命令如下：

```

1 cd samples/case8
2 python weights/export_yolo_to_onnx.py \
3   --weights weights/YOLOv10n_gestures.pt \
4   --output-dir models \
5   --imgsz 640 \
6   --batch 1 \
7   --opset 13 \
8   --device cpu

```

脚本会调用 Ultralytics 的 ONNX 导出接口，然后用 onnx.checker 检查模型合法性，并写出标签文件和元数据文件。元数据文件记录输入名、输入形状、输出名、类别名和导出参数。后面的 ATC 脚本会读取这些信息，因此不需要手工猜测输入名是不是 images，也不容易把输入形状写错。

ONNX 可以先用 CPU 做一次功能验证。scripts/infer_onnx_camera.py 默认使用 models ↪ /YOLOv10n_gestures.onnx，直接运行即可：

```

1 python scripts/infer_onnx_camera.py

```

如果通过 SSH 操作，没有图形界面，可以限制帧数并关闭 OpenCV 窗口：

```

1 python scripts/infer_onnx_camera.py --no-window --max-frames 30

```

ONNX 验收时应看到脚本正常打开摄像头，并在退出前打印类似下面的统计信息：

```

1 Processed 30 frames, ... inferences, camera FPS ..., inference FPS ..., avg ONNX
  ↪ latency ... ms

```

这个步骤只用于验证导出模型、标签和后处理流程，不代表最终性能。310B 的 CPU 跑 YOLOv10 会比较慢，尤其是 YOLOv10x。如果 ONNX Runtime 打印 pthread_setaffinity_np ↪ failed，通常是线程亲和性设置与当前系统 CPU 拓扑不匹配。脚本已经默认设置了较保守的线程数，一般不影响模型功能验证。

在 310B 上转换 OM 时，先加载 CANN 环境，再运行：

```

1 SOC_VERSION=Ascend310B4 bash scripts/atc_convert.sh

```

当前 scripts/atc_convert.sh 默认会转换 models 目录下所有 .onnx 文件。转换单个模型时，也可以传入 ONNX 路径和输出前缀：

```

1 SOC_VERSION=Ascend310B4 \
2   bash scripts/atc_convert.sh models/YOLOv10n_gestures.onnx models/
  ↪ YOLOv10n_gestures

```

脚本最终调用的 ATC 命令核心参数如下：

```

1 atc \
2   --framework=5 \

```

```

3 --model=models/YOLOv10n_gestures.onnx \
4 --output=models/YOLOv10n_gestures \
5 --input_format=NCHW \
6 --input_shape=images:1,3,640,640 \
7 --soc_version=Ascend310B4

```

其中 `--framework=5` 表示输入模型是 ONNX, `--input_shape` 指定静态输入形状。对 310B 这类边缘推理设备来说, 静态形状更容易得到稳定性能, 也更容易定位问题。

ATC 成功时会输出:

```

1 ATC run success, welcome to the next use.

```

转换完成后, `models` 目录下应出现同名 `.om` 文件, 例如 `models/YOLOv10n_gestures.om`。如果转换脚本一次处理多个 ONNX 文件, 每个模型都会打印一次 [ATC] model、[ATC] output 和 ATC run success。

有时它还会伴随 W11001 性能警告, 例如 `/model.23/Div_1` 和 `/model.23/Mod` 没有命中高优先级算子信息库。这不是转换失败。对 34 类手势模型来说, 检测头会把 TopK 得到的展平索引还原为候选框索引和类别 id。这个关系可以写成:

```

1 flat_index = box_index * 34 + class_id
2 box_index  = flat_index // 34
3 class_id   = flat_index % 34

```

因此, Div 和 Mod 对应的是输出端的索引解码, 而不是主干网络中的大卷积计算。遇到这种警告时, 应该先 benchmark 生成的 OM 模型, 再决定是否需要重新导出或简化模型图。

8.7. OM 推理与代码解析

OM 摄像头推理入口是 `scripts/infer_om_camera.py`。它的默认模型已经设置为 `models/YOLOv10n_gestures` → `.om`, 所以在 310B 上可以直接运行:

```

1 python scripts/infer_om_camera.py

```

如果要指定摄像头分辨率和阈值, 可以写成:

```

1 python scripts/infer_om_camera.py \
2 --source /dev/video0 \
3 --camera-width 1280 \
4 --camera-height 720 \
5 --camera-fps 30 \
6 --conf 0.25 \
7 --iou 0.45

```

纯模型 benchmark 不需要打开摄像头:

```

1 python scripts/infer_om_camera.py \
2 --benchmark-runs 50 \
3 --warmup-runs 5 \
4 --print-model-info

```

OM benchmark 验收时应看到模型输入、输出和延迟统计。以 YOLOv10n_gestures.om 为例，313 上的实测输出如下：

```
1 [ACL] input[0] size=4915200 shape=(1, 3, 640, 640)
2 [ACL] output[0] size=7200 shape=(1, 300, 6)
3 [OM] benchmark runs=80, avg=18.29 ms, min=18.20 ms, max=18.47 ms
4 [OM] output[0] shape=(1, 300, 6) dtype=float32
```

理解这段程序，关键是理解 hagrid_yolo 包内的三个文件：preprocess.py、detector.py 和 postprocess.py。摄像头读到的原图可能是 1280x720，而模型需要的是 640x640。直接拉伸会改变手的比例，因此代码使用 letterbox：先按比例缩放，再用灰色边填充到正方形。letterbox() 不仅返回填充后的图像，还返回 scale、pad_left 和 pad_top，这些值在后处理时会用来恢复坐标。

preprocess_image() 的输出是模型需要的 NCHW 张量。它先把 OpenCV 的 BGR 图像转换成 RGB，再把 HWC 排布转为 CHW，最后转成 float32 并除以 255：

```
1 padded, info = letterbox(image, imgsz)
2 rgb = cv2.cvtColor(padded, cv2.COLOR_BGR2RGB)
3 tensor = rgb.transpose(2, 0, 1).astype(np.float32) / 255.0
4 tensor = np.expand_dims(tensor, axis=0)
```

detector.py 把预处理、后端推理和后处理连接起来。代码中统计的 latency_ms 只包围 backend.infer(tensor)，所以它表示 OM 后端推理时间，不包括预处理、后处理和线程调度：

```
1 tensor, preprocess_info = preprocess_image(frame, self.imgsz)
2 start_t = time.time()
3 outputs = self.backend.infer(tensor)
4 latency_ms = (time.time() - start_t) * 1000.0
5 detections = decode_detections(outputs[0], frame.shape, preprocess_info, self.
    ↪ conf, self.iou)
```

后处理最容易出错的是坐标映射。模型输出的框坐标属于 letterbox 后的 640x640 图像；要画回原图，必须先减去 padding，再除以缩放比例：

```
1 boxes[:, [0, 2]] = (boxes[:, [0, 2]] - preprocess_info.pad_left) /
    ↪ preprocess_info.scale
2 boxes[:, [1, 3]] = (boxes[:, [1, 3]] - preprocess_info.pad_top) /
    ↪ preprocess_info.scale
```

这也是为什么 WebRTC 推流中看到是原始摄像头分辨率的画面，而不是 640x640 的模型输入。模型输入尺寸只决定推理张量大小；浏览器中的视频清晰度主要由摄像头实际输出、H.264 码率和前端显示尺寸决定。

OM 推理后端在 hagrid_yolo/backends/acl_backend.py 中实现。AclRuntime 负责初始化 ACL、设置 device、创建 context 和 stream；AclModel 负责加载 OM、创建输入输出 dataset、分配 device buffer，并在 infer() 中完成 host 到 device 的输入拷贝、acl.mdl.execute 执行和 device 到 host 的输出拷贝。WebRTC 程序中 OM 推理、VENC 和 JPEG 可能位于同一进程，因此代码接受 ACL_ALREADY_INITIALIZED=100002，并在 WebRTC track 中使用 finalize_on_release ↪ =False，避免某个模块释放时全局 acl.finalize() 影响其他硬件模块。

8.8. WebRTC 远程推流

本地 OpenCV 窗口适合调试，远程查看则需要一个面向实时视频的传输方式。WebRTC 是浏览器原生支持的实时音视频协议，可以使用 H.264 编码，并通过 `RTCPeerConnection.getStats()` 观察接收端码率和帧率。本案例使用 aiortc 在 Python 服务端建立 PeerConnection，同时把 aiortc 默认的 H.264 编码器替换为 Ascend CANN VENC，尽量减少 CPU 编码压力。

下方流程图是 WebRTC 程序的简化流程。

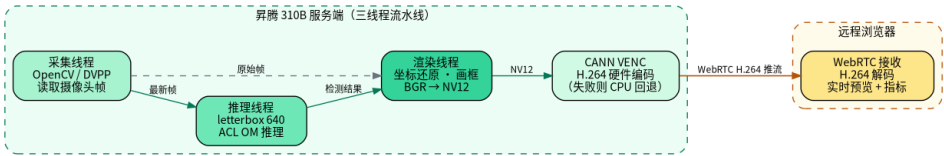


图 6：case8 WebRTC 流水线。

启动服务只需要：

```
1 python scripts/webrtc_om_app.py
```

默认配置使用 `YOLOv10n_gestures.om`、`/dev/video0`、`1280x720`、`30fps`、`MJPEG`、`4000 kbps` H.264 码率、OpenCV 采集后端和每帧推理。服务启动后会打印可访问地址，也可以直接在浏览器中打开：

```
1 WebRTC H.264 app is starting. Open one of these URLs:
2 http://313:8080
```

前端会从 `/models` 获取 `models` 目录下的 OM 模型列表，从 `/health` 获取默认参数和编码器状态，再通过 `/offer` 建立 WebRTC 连接。连接建立后，前端定期读取 `/stats`，把 FPS、NPU 推理时间、总推理时间、采集格式、码率和错误信息显示在页面右侧，而不是画到视频图像里。这样做的好处是视频画面保持干净，性能信息也更容易复制和分析。

`scripts/webrtc_om_app.py` 中的 `YoloOmVideoTrack` 使用三个后台线程组织实时流水线。采集线程不断读取摄像头帧，只保留最新帧；推理线程按 `infer_every_n` 取帧执行 YOLO 推理；渲染线程把最新检测结果画到最新原始帧上，再把 BGR 转成 NV12。WebRTC 的 `recv()` 直接从最新 NV12 图像构造 `PyAV VideoFrame`：

```
1 video_frame = av.VideoFrame.from_ndarray(frame, format="nv12")
```

VENC 接收 NV12 图像并输出 H.264 码流。这样可以减少 aiortc 内部的颜色空间转换。如果 CANN VENC 可用，`/health` 中会看到 `"encoder": "cann-venc-h264"` 和 `"hardware_encode": true`；如果不可用，程序会回退到 CPU `libx264`，也可以用 `--no-hardware-encode` 主动关闭硬件编码。

WebRTC 验收时，浏览器应能打开视频页面并列出 `models` 目录下的 OM 模型。命令行访问 `/health` 时，应看到类似下面的字段：

```
1 "status": "ok"
2 "runtime_target": "ascend-310b"
```



```

3 "transport": "webrtc"
4 "video_codec": "h264"
5 "default_model": "YOLOv10n_gestures.om"

```

这个流水线有一个重要设计：队列长度很短，旧帧会被丢弃。实时视频系统追求的是“最新画面”，不是“每一帧都处理完”。如果推理线程一时跟不上采集线程，保留旧帧只会让画面延迟越来越大。因此本案例宁愿丢旧帧，也要保持远程预览的实时性。

8.9. OpenCV 与 DVPP 采集后端

WebRTC 页面中可以选择 OpenCV 或 DVPP 采集后端。OpenCV 是默认路径，流程是 `cv2.↔ VideoCapture -> BGR -> 推理/画框 -> NV12 -> CANN VENC`。它的优点是稳定、容易调试、兼容大多数 USB 摄像头。缺点是 MJPEG 解码通常由 CPU/OpenCV 完成，如果摄像头实际落到 YUYV 高分辨率模式，采集帧率可能明显下降。

DVPP 后端的思路是绕过 OpenCV 的部分开销，直接用 V4L2 读取 MJPEG，再通过 DVPP JPEGD 解码成 NV12。当前流程仍然需要把 NV12 转为 BGR 做推理和画框，然后再转回 NV12 交给 VENC，因此它还不是全链路零拷贝。它的优势是可能降低 MJPEG 解码的 CPU 压力，适合高分辨率 MJPG 摄像头；限制是对摄像头格式、JPEG bitstream 和 DVPP 初始化更敏感。当前代码选择 `dvpp` 后不会静默回退 OpenCV，如果 V4L2 MJPEG 或 JPEGD 失败，`/offer` 会返回明确错误，前端日志也会显示对应信息。

调试采集性能时，首先应该确认摄像头真实支持哪些模式：

```

1 v4l2-ctl --device=/dev/video0 --list-formats-ext

```

如果同一摄像头显示 YUYV 1280x720 只能到 10fps，而 MJPG 1280x720 可以到 30fps，就应该优先请求 MJPG。程序里设置了宽高和帧率，并不代表摄像头一定按这个模式工作，最终还要看 `/stats` 里的 `actual_fourcc` 和 `capture_fps`。

8.10. 完整实验流程

前面各节已经展开了每一步的原理和代码。本节提供一个精简的操作 checklist，方便读者快速跑通全流程。各项操作的详细解释请回顾对应章节。

在 PC / GPU 工作站上（详见「模型导出与 ATC 转换」）：

```

1 cd samples/case8
2 # 安装导出依赖（一次性）
3 pip install numpy==1.26.4 onnx==1.14.1 onnxruntime==1.15.1 opencv-python
   ↪ ==4.8.0.76
4 pip install torch==2.10.0 torchvision==0.25.0 --extra-index-url https://download
   ↪ .pytorch.org/whl/cu128
5 pip install ultralytics==8.4.60
6 python weights/export_yolo_to_onnx.py \
7   --weights weights/YOLOv10n_gestures.pt --output-dir models \

```

```

8 --imgsz 640 --batch 1 --opset 13 --device cpu
9 # 验证: ls models/*.onnx models/*_labels.txt models/*_metadata.json

```

将 ONNX 同步到 310B 并转换 OM (详见「模型导出与 ATC 转换」):

```

1 # 在 PC 上
2 cd /path/to/Ascend310
3 rsync -av samples/case8/ 313:~/Documents/Ascend310/samples/case8/
4
5 # SSH 到 310B
6 ssh 313
7 conda activate npu
8 source /usr/local/Ascend/ascend-toolkit/set_env.sh
9 cd ~/Documents/Ascend310/samples/case8
10 pip install -r requirements.txt
11 SOC_VERSION=Ascend310B4 bash scripts/atc_convert.sh
12 # 验证: ls models/*.om

```

OM 基准测试与摄像头验证 (详见「OM 推理与代码解析」):

```

1 # 纯模型 benchmark
2 python scripts/infer_om_camera.py --benchmark-runs 50 --warmup-runs 5 --print-
  ↳ model-info
3 # 基准: avg ≈ 18.29ms (YOLOv10n, 具体值因设备而异)
4
5 # 摄像头测试 (有显示器时可去掉 --no-window)
6 python scripts/infer_om_camera.py --no-window --max-frames 60 \
7   --camera-width 1280 --camera-height 720 --camera-fps 30
8 # 验证: 打印 Processed 60 frames ... avg NPU latency ... ms

```

启动 WebRTC 服务 (详见「WebRTC 远程推流」):

```

1 python scripts/webrtc_om_app.py
2 # 浏览器打开终端打印的局域网地址, 如 http://192.168.1.100:8080
3 # 验证: curl http://127.0.0.1:8080/health # 返回 "status": "ok"

```

验收标准汇总:

- OM benchmark 能正常加载模型, 打印输入形状 1,3,640,640 和输出形状 1,300,6;
- 摄像头测试脚本正常退出并打印统计信息;
- WebRTC /health 显示 "runtime_target": "ascend-310b";
- /models 路由列出 models/ 下的全部 .om 文件;
- 浏览器能打开页面并看到带检测框的实时视频。

8.11. 性能分析与优化

实时系统的帧率由最慢环节决定。看到远程 FPS 低时, 不应该只看 NPU 推理时间。一次完整远程显示至少经过采集、预处理、OM 推理、后处理、画框、颜色转换、H.264 编码、网络传输和浏览器解码。NPU 推理时间只有 20ms, 并不意味着端到端一定能达到 50fps。

本案例把关键指标拆开放在 /stats 中。capture_fps 表示摄像头采集速度，infer_fps 表示推理线程实际运行速度，track_fps 表示 WebRTC track 实际送帧速度，npv_latency_ms 表示 OM 后端推理时间，infer_total_ms 表示包含预处理和后处理的完整推理线程耗时，nv12_ms 表示 BGR 转 NV12 耗时。只有把这些指标分开看，才能判断瓶颈在摄像头、模型、颜色转换、编码还是网络。

在 313 上的参考测试中，YOLOv10n_gestures.om、1280x720、MJPG、CANN VENC、OpenCV 后端、infer_every_n=1 时，WebRTC 大约可以达到 27fps，NPU 推理时间约 24ms，完整推理线程耗时约 30ms。把 infer_every_n 改为 2 后，视频可以接近 30fps，但检测结果每两帧更新一次，推理线程约 15fps。这组数字只能作为基线，不同摄像头、CANN 版本和浏览器环境都会改变结果。

四个模型的纯 OM benchmark 显示，YOLOv10x 的单次推理约为 YOLOv10n 的 6.7 倍。实时 WebRTC 场景中还要叠加预处理、后处理、画框和编码，因此默认使用 YOLOv10n_gestures 更稳妥。如果需要比较 x 模型的识别效果，建议先在本地窗口或低帧率 WebRTC 配置中测试，并把 infer_every_n 调大，避免远程画面堆积延迟。

优化时建议先使用 YOLOv10n，不要一开始就用 YOLOv10x。两者输入同样是 640x640，但 YOLOv10x 计算量大得多。然后确认摄像头实际输出是否为 MJPG，以及 capture_fps 是否已经达到目标帧率。如果采集只有 15fps，后面的推理和编码再快也无法得到 30fps。采集正常后，再观察 infer_total_ms 和 nv12_ms。如果推理接近 33ms，infer_every_n=1 就很难稳定超过 30fps；如果只是远程画面模糊，可以提高 H.264 码率，当前默认值是 4000 kbps。

8.12. 常见问题

如果 ATC 报 --host_env_os linux is invalid，说明旧命令中传入了当前 CANN/OPP 组合不接受的参数。当前 scripts/atc_convert.sh 已经不再设置 --host_env_os，只保留 ONNX 转 OM 需要的核心参数。

如果 ATC 成功但出现 W11001，先不要急着改模型。只要已经输出 ATC run success，就先运行 OM benchmark。/model.23/Div_1 和 /model.23/Mod 是检测头末尾的索引解码，通常不是主要耗时。

如果 ONNX Runtime 打印 pthread_setaffinity_np failed，通常不是模型错误，而是线程亲和性设置与系统 CPU 拓扑不匹配。脚本已经提供 --intra-op-threads 和 --inter-op-threads 参数，可以显式限制线程数。

如果 WebRTC 端口被占用，可以换端口启动：

```
1 python scripts/webrtc_om_app.py --port 8081
```

如果选择 DVPP 后端后没有画面，要看前端日志和服务端日志。当前实现不会静默回退 OpenCV。常见关键字包括 Using direct V4L2 MJPEG capture backend for DVPP、DVPP JPEGD decode frame、jpeg_get_image_info failed 和 jpeg_decode_async failed。

如果本地显示器能到 30fps，远程浏览器只有个位数 FPS，瓶颈通常不在模型推理本身，而在推流链路。需要同时观察 `track_fps`、浏览器 `getStats` 中的码率、服务端 `nv12_ms` 和编码器状态。

8.13. 维护建议

为了让本案例长期适合作为教程使用，代码结构应保持清晰。可复用逻辑放在 `hagrid_yolo` 包内，脚本只作为入口；`.pt` -> ONNX 导出脚本继续留在 `weights`，便于后续随权重一起迁移；310B 运行时不要依赖 PyTorch；模型、标签和元数据文件保持同名，例如 `YOLOv10n_gestures.om`、`YOLOv10n_gestures_labels.txt` 和 `YOLOv10n_gestures_metadata.json`。每次新增模型时，都应该先做 ONNX 功能验证，再做 ATC 转换和 OM benchmark。涉及 CANN、ATC、OM、DVPP 的行为必须在真实 310B 上验证，本地文档环境只能做代码编辑、图生成和 Markdown 检查。

8.14. 参考资料

[1] Kapitanov A, Kvanchiani K, Nagaev A, et al. HaGRID – HAnd gesture recognition image dataset[C]// Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV). Waikoloa, HI, USA: IEEE, 2024.

[2] Nuzhdin A, Nagaev A, Sautin A, et al. HaGRIDv2: 1M images for static and dynamic hand gesture recognition[EB/OL]. (2024-12-02). <https://arxiv.org/abs/2412.01508>.

[3] Kapitanov A. HaGRID: HAnd gesture recognition image dataset[EB/OL]. <https://github.com/hukendo>

[4] Wang A, Chen H, Liu L H, et al. YOLOv10: Real-time end-to-end object detection[EB/OL]. (2024-05-23). <https://arxiv.org/abs/2405.14458>.

[5] Wang A. YOLOv10: Real-time end-to-end object detection[EB/OL]. <https://github.com/THU-MIG/yolov10>.

[6] Ultralytics. YOLOv10[EB/OL]. <https://docs.ultralytics.com/models/yolov10/>.

[7] ONNX developers. Open Neural Network Exchange[EB/OL]. <https://github.com/onnx/onnx>.

[8] Microsoft. ONNX Runtime Python API[EB/OL]. https://onnxruntime.ai/docs/api/python/api_summary

[9] 华为技术有限公司. 昇腾 CANN 文档 [EB/OL]. <https://www.hiascend.com/document>.

[10] aiortc contributors. aiortc: WebRTC and ORTC for Python[EB/OL]. <https://github.com/aiortc/aiortc>

[11] W3C. WebRTC: Real-Time Communication Between Browsers[S/OL]. <https://www.w3.org/TR/webrtc>

9. 案例 9：边缘智能聊天机器人

9.1. 1. 项目简介

本项目基于昇腾 310B 平台，构建一个可以在边缘设备上运行的智能聊天机器人。系统集成了文本嵌入、RAG（检索增强生成）、语音交互等 AI 技术，能够在离线环境下回答专业问题，也可接入云端大语言模型提升回复质量。

与云端聊天机器人相比，边缘端部署具有**低延迟、隐私保护、离线可用**三大优势。本项目的核心设计思想是：不强行在昇腾 310B 上运行大语言模型（这在实际硬件条件下既困难又低效），而是将 NPU 用于它最擅长的事情——文本嵌入的快速推理，再通过向量检索和模板/云端 LLM 两级策略生成回复。

项目的源代码可以从[这里](#)下载。

9.2. 2. 内容大纲

9.2.1. 2.1. 硬件准备

- 核心计算单元: 昇腾 310B 开发者套件
- 音频设备:
 - USB 麦克风或 USB 耳机（语音输入）
 - USB 音响或 3.5mm 耳机（语音输出）
 - 也可直接使用电脑自带麦克风和扬声器
- 网络连接:
 - 以太网或 WiFi（仅云端 LLM 模式需要）
- 可选外设:
 - 触摸屏显示器（用于独立运行时的交互）
 - 键盘鼠标（开发调试用）

系统硬件架构

```
1 flowchart LR
2   MIC["语音 USB 麦克风"]
```

```

3  SPK[/ " USB 音响"/]
4  BROWSER[/ " 浏览器<br/>Gradio UI"/]
5  CLOUD[/ " 云端 LLM API<br/>(可选)"/]
6
7  subgraph DEVICE["昇腾310B 开发板"]
8      NPU["NPU<br/>文本嵌入推理"]
9      CPU["CPU<br/>FAISS检索 + 对话管理"]
10     NPU --- CPU
11 end
12
13 MIC -->|"音频流"| DEVICE
14 DEVICE -->|"音频输出"| SPK
15 BROWSER -->|"HTTP/WebSocket<br/>文本/语音数据"| DEVICE
16 DEVICE -->|"HTTP<br/>(可选)"| CLOUD
17 CLOUD -. ->|"LLM 回复"| DEVICE

```

相比原方案中动辄列出“4 麦克风阵列、NVMe SSD、触摸屏、锂电池组”等大量外围硬件，本项目的硬件需求非常简洁：一块昇腾 310B 开发板、一个 USB 麦克风和音响（或一个 USB 耳机），足以运行完整的聊天机器人系统。没有昇腾设备也可以——嵌入模型会自动回退到 CPU 推理。

9.2.2. 2.2. 软件环境

- 操作系统: Ubuntu 20.04 / 22.04 LTS
- CANN 版本: 7.0.RC1 或以上（仅 NPU 推理需要）
- Python 版本: 3.9 或以上
- 核心依赖:
 - gradio — Web 聊天界面
 - sentence-transformers — 嵌入模型（CPU 推理 & 模型参考）
 - faiss-cpu — 向量相似度搜索
 - jieba — 中文分词（文本分块）
 - numpy — 数值计算
- 语音交互（可选）:
 - SpeechRecognition — 语音识别（调用 Google Web Speech API）
 - pyaudio — 麦克风音频采集
 - pyttsx3 — 语音合成（espeak 后端）
- 模型准备（仅开发/转换阶段）:
 - torch + transformers — ONNX 模型导出
 - optimum — HuggingFace 模型导出工具（推荐）
- 云端 LLM（可选）:

– requests — HTTP 调用 OpenAI 兼容 API

环境配置脚本 (setup.sh)

```

1 #!/bin/bash
2 set -e
3
4 echo "=== Ascend 310B 智能聊天机器人 - 环境安装 ==="
5
6 # 1. 系统依赖
7 sudo apt update
8 sudo apt install -y python3-dev python3-pip portaudio19-dev espeak
9
10 # 2. Python包
11 pip3 install gradio sentence-transformers faiss-cpu jieba numpy<2.0
12 pip3 install SpeechRecognition pyaudio pyttsx3
13
14 # 3. 模型准备（下载 + ONNX导出 + ATC转换）
15 python3 prepare_models.py
16
17 echo "安装完成！启动：python3 app.py"

```

与旧方案的关键区别：- 不再安装 **PyTorch / transformers / accelerate / peft** 作为运行时依赖——这些加起来超过 5GB，对于嵌入模型来说完全不需要 - 不再安装 **Redis / FastAPI / uvicorn / websockets**——Gradio 一个库覆盖了 Web 界面和 API - 语音库标注为可选，没有麦克风也不影响核心功能

9.2.3. 2.3. 系统架构设计

本节是理解整个项目设计思路的核心。在动手写代码之前，需要先回答一个关键问题：**昇腾 310B 到底能不能跑大语言模型？**

2.3.1 为什么不在 NPU 上直接跑 LLM

昇腾 310B 的 NPU 通过 OM (Offline Model) 格式执行推理。OM 是一个**静态计算图**——输入张量的形状和数据类型在模型转换时就固定了，运行时不可改变。

而大语言模型的生成过程是**自回归**的：每生成一个 token，要把它拼回输入序列，再跑一次模型。序列长度在逐 token 增长，输入形状在不断变化。这与 OM 的静态图假设根本矛盾。

加上 KV-cache 管理的复杂性、7B 模型至少 14GB 的 FP16 内存需求（而昇腾 310B 通常只有 4-8GB），直接部署 LLM 不切实际。

但这并不意味着 NPU 在 NLP 任务中没用。恰恰相反——**文本嵌入** (Text Embedding) 是 NPU 的“甜点区”：

- 固定输入形状 (256 tokens in, 384 维向量 out)
- 纯前向推理，一次执行完成
- 模型小 (all-MiniLM-L6-v2 约 90MB)，轻松装入 NPU 内存

- 推理速度快（NPU 上约 10ms/条 vs CPU 50-80ms/条）

2.3.2 三层架构设计

基于上述分析，本项目采用三层混合架构，完整程序流程如下：

```

1 flowchart TD
2     %% ===== 入口 =====
3     U[/ " 用户 " /]
4     UI{{ "输入方式?" }}
5
6     U --> UI
7     UI -->| " 文本 " | TXT["输入消息"]
8     UI -->| "语音 语音" | MIC["浏览器录音<br/>gr.Audio"]
9     MIC --> WAV["WAV 音频数据<br/>float32->int16"]
10    WAV --> ASR["SpeechRecognizer<br/>Google Web Speech API"]
11    ASR --> ASR_CHECK{{ "识别成功?" }}
12    ASR_CHECK -->| "是" | TXT
13    ASR_CHECK -->| "否" | ERR(["[失败] 识别错误"])
14    ERR --> U
15
16    %% ===== 对话管理 =====
17    TXT --> DM["DialogueManager<br/>.process_message()"]
18    DM --> INTENT{{ "意图检测<br/>关键词匹配?" }}
19
20    INTENT -->| "问候词" | GREET["GREETING 状态<br/>返回时段问候语"]
21    INTENT -->| "告别词" | BYE["FAREWELL 状态<br/>返回告别语"]
22    INTENT -->| "正常问答" | RAG["进入 RAG 管道"]
23    INTENT -->| "模糊不清" | CLARIFY["CLARIFYING 状态<br/>请用户重述"]
24
25    %% ===== RAG 管道 =====
26    RAG --> ENC["EmbeddingModel.encode()<br/>用户查询 -> 384维向量"]
27    ENC --> NPU_CHECK{{ "NPU 可用?" }}
28    NPU_CHECK -->| "[OK] 是" | TOK["Tokenizer<br/>文本->input_ids/attn_mask"]
29    TOK --> H2D["acl.rt.memcpy H2D<br/>numpy->NPU 内存"]
30    H2D --> ACL_EXEC["acl.mdl.execute<br/>OM 模型推理"]
31    ACL_EXEC --> D2H["acl.rt.memcpy D2H<br/>NPU 内存->numpy"]
32    D2H --> POOL["Mean Pooling<br/>attention_mask 加权均值"]
33    NPU_CHECK -->| "[失败] 否 (回退)" | CPU_ENC["SentenceTransformer<br/>.encode(
    ↳ normalize=True)"]
34    POOL --> NORM["L2 归一化<br/>向量/| 向量|"]
35    CPU_ENC --> NORM
36
37    NORM --> FAISS["FAISS IndexFlatIP<br/>.search(query_vec, k=3)"]
38    FAISS --> FILTER{{ "相似度 >= 阈值<br/>(SIMILARITY_THRESHOLD=0.3)?" }}
39    FILTER -->| "是" | CTX["检索上下文<br/>[{text, score, metadata}]"]
40    FILTER -->| "否" | NO_CTX["无匹配知识<br/>context=[]"]
41
42    %% ===== 回复生成 =====
43    CTX --> GEN{{ "回复模式?" }}
44    NO_CTX --> GEN
45    GEN -->| " 离线模式 " | TPL["模板生成<br/>.generate_template()"]
46    GEN -->| " 云端模式 " | CLOUD_CHECK{{ "API Key 已配置?" }}
47    CLOUD_CHECK -->| "是" | LLM["_generate_cloud()<br/>构造 System Prompt<br/>+
    ↳ RAG上下文 + 历史"]
48    LLM --> API["POST OpenAI-compatible API<br/>requests.post(endpoint)"]
  
```

```
49 API --> API_CHECK{{"API 响应 OK?"}}
50 API_CHECK -->|"是"| LLM_RESP["LLM 回复文本"]
51 API_CHECK -->|"否"| TPL
52 CLOUD_CHECK -->|"否"| TPL
53
54 TPL --> RESP["回复文本"]
55 LLM_RESP --> RESP
56 GREET --> RESP
57 BYE --> RESP
58 CLARIFY --> RESP
59
60 %% ===== 输出 =====
61 RESP --> TTS_CHECK{{"语音输出开启?"}}
62 TTS_CHECK -->|"是"| TTS["TextToSpeech.speak()<br/>pyttsx3 / espeak<br/>异步
    ↳ 线程播放"]
63 TTS_CHECK -->|"否"| CHAT[Gradio Chatbot 展示]
64 TTS --> CHAT
65 CHAT --> HISTORY["存入 ConversationHistory<br/>环形缓冲 (最近10轮)"]
66 HISTORY --> U
```

每一层的职责和设计考量：

层	运行位置	为什么放在这里
语音 I/O	CPU	音频采集/播放是操作系统层面的事，与 NPU 无关
文本嵌入	NPU（主）/ CPU（回退）	NPU 擅长矩阵运算，嵌入模型正好是纯矩阵计算；CPU 作为无 NPU 时的保障
FAISS 检索	CPU	向量搜索是索引结构的遍历，非矩阵运算，CPU+C++ 优化就足够快（<1ms）
回复生成	CPU	模板匹配在 CPU 上是 O(1)；云端 API 走网络，与 NPU 无关

2.3.3 与纯云端/纯边缘方案的对比

维度	纯云端	纯边缘 LLM	本项目（混合）
延迟	网络延迟 0.5-3s	NPU 不可行，CPU 上极慢	嵌入 <50ms + 检索 <1ms + 回复 <10ms（模板）
隐私	数据上传云端	完全本地	完全本地（离线模式）
可用性	依赖网络	始终可用	始终可用（离线模式）

维度	纯云端	纯边缘 LLM	本项目（混合）
回复质量	高（大模型）	取决于模型大小	模板精准/云端高质量
硬件要求	无需 NPU	需大内存 +GPU	升腾 310B 或普通 CPU
维护成本	API 费用	模型更新复杂	只更新知识库文本

9.2.4. 2.4. 文本嵌入模型与昇腾部署

本节详细介绍如何在昇腾 310B 上部署文本嵌入模型，这是整个 RAG 管道的基石。

2.4.1 什么是文本嵌入

文本嵌入（Text Embedding）将一段自然语言文本转换为固定长度的浮点数向量。其核心性质是：语义相近的文本，在向量空间中距离更近。

1	"昇腾310B是一款边缘AI芯片"	->	[0.12, -0.34, 0.08, ..., 0.21]	(384维)
2	"华为Ascend 310B是边缘推理处理器"	->	[0.13, -0.31, 0.06, ..., 0.19]	← 余弦相似 ↪ 度 约等于 0.92
3	"今天天气很好适合出去玩"	->	[-0.45, 0.28, 0.73, ..., -0.55]	← 余弦相似度 约等 ↪ 于 0.03

有了这个性质，“找到知识库里和用户问题最相关的内容”就变成了数学上的向量内积运算。

2.4.2 为什么选择 all-MiniLM-L6-v2

候选模型	参数量	向量维度	模型大小	推理速度 (CPU)
all-MiniLM-L6-v2	22M	384	~90 MB	~10ms
all-mpnet-base-v2	110M	768	~420 MB	~50ms
bge-large-zh-v1.5	326M	1024	~1.3 GB	~150ms
text2vec-large-chinese	326M	1024	~1.3 GB	~150ms

all-MiniLM-L6-v2 是由 Microsoft 的 Sentence-Transformers 团队发布的轻量级嵌入模型。它用知识蒸馏技术将 BERT 的 12 层压缩到 6 层，保留了原始模型约 95% 的嵌入质量，但体积缩小为原来的 1/3。

选择它的原因：- **规格适中**：90MB 的 ONNX 模型对昇腾 310B 的内存没有任何压力 - **固定输入输出**：最大 256 tokens 输入，384 维向量输出，完美匹配 OM 静态图 - **多语言支持**：虽然以英文为主训练，但对中文的嵌入质量在轻量模型中排名靠前 - **生态成熟**：HuggingFace 直接支持，optimum 库可一行代码导出 ONNX

2.4.3 模型转换管线

```

1 flowchart TD
2   HF["HuggingFace<br/>all-MiniLM-L6-v2<br/>PyTorch (~90MB)"]
3   HF -->|"optimum / torch.onnx.export"| ONNX["ONNX 模型 (~90MB)<br/><br/>
   ↳ inputs:<br/> input_ids [batch,256] int64<br/> attention_mask [batch
   ↳ ,256] int64<br/> token_type_ids [batch,256] int64<br/><br/>output:<br/>
   ↳ sentence_embedding [batch,384] float32"]
4   ONNX -->|"atc --framework=5<br/>--soc_version=Ascend310B4"| OM["OM 模型 (~45
   ↳ MB, FP16)<br/>静态计算图"]
5   OM -->|"PyACL 加载 & 执行"| INFER["实时推理<br/><br/>文本 -> Tokenizer ->
   ↳ numpy<br/>-> NPU H2D -> acl.mdl.execute -> D2H<br/>-> Mean Pooling ->
   ↳ L2 归一化<br/>-> 384维向量"]

```

ONNX 导出代码 (prepare_models.py 核心逻辑):

```

1 from transformers import AutoTokenizer, AutoModel
2 import torch
3
4 tokenizer = AutoTokenizer.from_pretrained("sentence-transformers/all-MiniLM-L6-
   ↳ v2")
5 model = AutoModel.from_pretrained("sentence-transformers/all-MiniLM-L6-v2")
6 model.eval()
7
8 # 准备一个dummy输入来确定动态轴
9 dummy = tokenizer("Hello world", padding="max_length",
10                  truncation=True, max_length=256, return_tensors="pt")
11
12 torch.onnx.export(
13     model,
14     (dummy["input_ids"], dummy["attention_mask"],
15      dummy.get("token_type_ids", torch.zeros_like(dummy["input_ids"]))),
16     "models/embedding_model.onnx",
17     input_names=["input_ids", "attention_mask", "token_type_ids"],
18     output_names=["sentence_embedding"],
19     dynamic_axes={
20         "input_ids": {0: "batch_size"},
21         "attention_mask": {0: "batch_size"},
22         "token_type_ids": {0: "batch_size"},
23         "sentence_embedding": {0: "batch_size"},
24     },
25     opset_version=14,
26 )

```

ATC 转换命令:

```

1 atc --model=models/embedding_model.onnx \
2     --framework=5 \
3     --output=models/embedding_model \
4     --input_shape="input_ids:1,256;attention_mask:1,256;token_type_ids:1,256" \
5     --soc_version=Ascend310B4

```

参数说明: `--framework=5`: 5 代表 ONNX 格式 `--input_shape`: 明确指定每个输入的维度, 避免 ATC 推导失败 `--soc_version`: 根据实际芯片选择, `npu-smi info` 可查看

转换完成后, `models/` 目录下会生成 `embedding_model.om` (约 45MB, FP16), 这就是昇腾 NPU 可以直接执行的格式。

2.4.4 NPU 推理封装

昇腾的 Python API(PyACL)提供了底层的设备管理、内存拷贝和模型执行接口。ascend_inference

→ .py 沿用了案例 1 中成熟的 AscendSystem / AscendModel 模式。

AscendSystem — 管理 NPU 设备生命周期：

```
1 class AscendSystem:
2     def __init__(self, device_id=0):
3         ret = acl.init() # 初始化 ACL 运行时
4         ret = acl.rt.set_device(device_id) # 选择 NPU 设备
5         self.context, ret = acl.rt.create_context(device_id) # 创建执行上下文
6         self.stream, ret = acl.rt.create_stream() # 创建任务流
```

初始化顺序是固定的: acl.init -> set_device -> create_context -> create_stream。

释放时严格反向: destroy_stream -> destroy_context -> reset_device -> finalize。

AscendModel — 加载和执行 OM 模型：

```
1 class AscendModel:
2     def _load_model(self):
3         self.model_id, ret = acl.mdl.load_from_file(self.model_path) # 加载 OM
4         self.desc = acl.mdl.create_desc() # 获取模型描述
5         acl.mdl.get_desc(self.desc, self.model_id)
6         self._init_buffers() # 预分配输入/输出内存
7
8     def execute(self, input_data_list):
9         # 1. Host -> Device: 将 numpy 数组拷贝到 NPU 内存
10        for i, data in enumerate(input_data_list):
11            acl.rt.memcpy(dev_ptr, size, host_ptr, size, 1) # 1 = H2D
12
13        # 2. 执行推理
14        acl.mdl.execute(self.model_id, self.input_dataset, self.output_dataset)
15
16        # 3. Device -> Host: 将结果拷回主机
17        for i in range(len(self.output_buffers)):
18            acl.rt.memcpy(host_ptr, size, dev_ptr, size, 2) # 2 = D2H
```

EmbeddingModel — 业务层封装，整合 tokenizer 和 NPU 推理：

```
1 class EmbeddingModel:
2     def encode(self, texts):
3         """输入文本列表，返回归一化的嵌入向量 (N, 384)"""
4         if self._use_npu:
5             return self._encode_npu(texts)
6         else:
7             return self._encode_cpu(texts) # 自动回退
8
9     def _encode_npu(self, texts):
10        for text in texts:
11            encoded = tokenizer(text, padding="max_length",
12                               truncation=True, max_length=256,
13                               return_tensors="np")
14            outputs = self._ascend_model.execute([
15                encoded["input_ids"].astype(np.int64),
16                encoded["attention_mask"].astype(np.int64),
17                encoded["token_type_ids"].astype(np.int64),
18            ])
```

```

19         embedding = mean_pool(outputs[0], attention_mask)
20         embeddings = L2_normalize(embeddings)
21     return embeddings

```

需要特别注意的细节：

1. **数据类型**：PyACL 要求 int64 输入。numpy 默认的 int32 会导致数据错位。
2. **内存对齐**：np.ascontiguousarray() 确保 C-contiguous 内存布局，否则 acl.rt.memcpy 会失败。
3. **Mean Pooling**：Transformer 输出是每个 token 的隐藏状态，需要对其求均值得到句子级向量。注意要用 attention_mask 屏蔽 padding token。

```

1 def _pool_output(self, outputs, attention_mask):
2     # outputs: [batch, seq_len, hidden_size]
3     # mask:    [batch, seq_len]
4     mask = np.expand_dims(attention_mask.astype(np.float32), axis=-1)
5     summed = (token_embeddings * mask).sum(axis=1) # 有效token求和
6     counts = mask.sum(axis=1).clip(min=1)         # 有效token数量
7     return summed / counts                        # 均值

```

4. **L2 归一化**：将向量归一化到单位球面上。归一化后，内积即等价于余弦相似度，这是 FAISS IndexFlatIP 的工作前提。

```

1 @staticmethod
2 def _normalize(embeddings):
3     norms = np.linalg.norm(embeddings, axis=-1, keepdims=True)
4     norms = np.clip(norms, 1e-12, None) # 防止除零
5     return embeddings / norms

```

9.2.5. 2.5. 知识库与向量检索

有了嵌入模型，下一步是构建知识库并实现高效的向量检索。

2.5.1 RAG 的核心思想

RAG (Retrieval-Augmented Generation, 检索增强生成) 的工作流程：

```

1 flowchart LR
2     subgraph OFFLINE["1 离线阶段"]
3         direction LR
4         A["知识库文档"] --> B["中文分块<br/>段落/句子切分"]
5         B --> C["嵌入编码<br/>EmbeddingModel.encode()"]
6         C --> D["存入 FAISS 索引<br/>IndexFlatIP.add()"]
7     end
8
9     subgraph ONLINE["2 在线阶段"]
10        direction LR
11        E["用户提问"] --> F["嵌入编码<br/>-> 384维向量"]
12        F --> G["向量检索<br/>FAISS.search(k=3)"]

```

```

13     G --> H["Top-K 文档块<br/>含相似度分数"]
14 end
15
16 subgraph GENERATE["3 生成阶段"]
17     direction LR
18     H --> I["构造 Prompt<br/>上下文 + 提问"]
19     I --> J["回复生成模块<br/>模板 / 云端LLM"]
20 end
21
22 D -. -> G

```

RAG 解决的核心问题：让 AI 的回答有据可查。没有 RAG 时，模型要么凭空编造（大模型的“幻觉”），要么只能靠训练数据中记住的知识。有了 RAG 后，回答可以被溯源到知识库中的具体文档片段。

2.5.2 中文文本分块策略

知识库文档需要被切成合适大小的“块”（chunk）。块太大，检索精度下降（一个块里混入太多不相关内容）；块太小，语义不完整。

knowledge_base.py 中的分块策略考虑了中文的特点：

```

1 def _split_text(self, text):
2     paragraphs = [p.strip() for p in text.split("\n") if p.strip()]
3     chunks = []
4     for para in paragraphs:
5         if len(para) <= self._chunk_size:      # 短段落直接作为一块
6             chunks.append(para)
7             continue
8         sentences = self._split_sentences(para) # 按句号/问号/感叹号切分
9         current = ""
10        for sent in sentences:
11            if len(current) + len(sent) <= self._chunk_size:
12                current += sent
13            else:
14                chunks.append(current)
15                # 重叠：保留上一块的末尾，避免语义断裂
16                current = current[-self._chunk_overlap:] + sent
17        if current:
18            chunks.append(current)
19    return chunks

```

中文分句不使用 jieba 分词，而是通过正则表达式匹配句末标点（。！？；）。这比用分词器更轻量，且对中文段落的分句准确度足够。

2.5.3 FAISS 向量索引

FAISS（Facebook AI Similarity Search）是 Meta 开源的向量相似度搜索库。核心操作极其简单：

```

1 import faiss
2 import numpy as np

```

```

3
4 # 创建索引
5 dim = 384
6 index = faiss.IndexFlatIP(dim)          # 内积索引
7
8 # 添加向量
9 embeddings = model.encode(documents)    # shape: (N, 384)
10 index.add(embeddings)                  # 加入索引
11
12 # 搜索
13 query_vec = model.encode(["昇腾310B的算力是多少?"])
14 scores, indices = index.search(query_vec, k=3) # 返回最相似的3个

```

IndexFlatIP 的含义：- **Flat**：暴力搜索，不做任何近似。对 N 个文档、384 维向量，复杂度 $O(N \times 384)$ 。- **IP**：Inner Product（内积）。配合 L2 归一化的向量，内积等价于余弦相似度。

对于几千到几万条文档的规模，暴力搜索完全够用（ $<1\text{ms}$ ）。当知识库扩大到百万级别时，FAISS 提供了 IVF（倒排索引）、HNSW（图索引）等近似搜索方案，只需改一行代码即可切换。

2.5.4 知识库管理

KnowledgeBase 类提供了完整的增删查改和持久化：

```

1 kb = KnowledgeBase(embedding_model)
2
3 # 添加知识
4 kb.add_texts(["昇腾310B支持FP16和INT8推理..."], [{"source": "manual"}])
5 kb.add_document("path/to/knowledge.txt") # 自动分块 + 编码 + 入库
6
7 # 检索
8 results = kb.search("什么是CANN?", k=3)
9 # [{"text": "...", "score": 0.87, "metadata": {...}}, ...]
10
11 # 持久化
12 kb.save("data/index.faiss", "data/documents.json")
13 kb.load("data/index.faiss", "data/documents.json")

```

9.2.6. 2.6. 对话管理与响应生成

2.6.1 对话状态机

DialogueManager 维护一个简单的四状态机：

```

1 stateDiagram-v2
2     [*] --> GREETING: 启动 / 首次交互
3     GREETING --> ACTIVE: 用户发言
4     ACTIVE --> ACTIVE: 正常问答<br/>走 RAG 管道
5     ACTIVE --> CLARIFYING: 意图模糊不清
6     CLARIFYING --> ACTIVE: 用户重述/补充
7     ACTIVE --> FAREWELL: 检测到告别词
8     FAREWELL --> [*]: 对话结束
9
10    note right of GREETING: 根据时段返回<br/>不同问候语

```

```

11 note right of ACTIVE: 嵌入->检索->回复
12 note left of FAREWELL: 再见/拜拜/bye

```

状态转换规则：- 首次交互自动进入 GREETING（根据时间段返回不同问候语）- 检测到“再见/拜拜/bye”等关键词进入 FAREWELL - 其他情况保持在 ACTIVE 状态，走完整的 RAG 处理流程

2.6.2 两级回复生成

默认模式：模板 + RAG 检索上下文

查询经过 RAG 检索后，将匹配到的知识片段直接组织成回复：

```

1 def _generate_template(self, query, context):
2     if context:
3         pieces = ["根据我的知识库，以下是相关内容：\n"]
4         for i, item in enumerate(context, 1):
5             pieces.append(f"{i}. {item['text']}\n")
6         pieces.append("\n请问还有什么想了解的吗？")
7         return "".join(pieces)
8     return "抱歉，我在知识库中没有找到相关的信息..."

```

这种方式虽然简单，但结合了 RAG 的检索能力——回复中的每一条信息都直接来自知识库，**零幻觉**。对于昇腾 310B、CANN、边缘计算等专业问题，FAQ + 知识库的模板回复比大模型凭空生成更可靠。

云端增强模式：LLM + RAG

当用户配置了云端 API Key，同样的检索结果会作为 system prompt 注入 LLM：

```

1 def _generate_cloud(self, query, context, history):
2     system_prompt = (
3         "你是一个运行在昇腾310B边缘设备上的AI助手。"
4         "请基于提供的知识库内容回答用户问题。"
5         "如果知识库中没有相关信息，请诚实告知，不要编造。"
6     )
7     # 将检索到的文档拼接为上下文
8     context_block = "\n".join(f"[{i}] {item['text']}"
9                               for i, item in enumerate(context, 1))
10    messages = [
11        {"role": "system", "content": system_prompt},
12        {"role": "system", "content": f"相关知识库内容：{context_block}"},
13        {"role": "user", "content": query},
14    ]
15    # 调用 OpenAI 兼容 API
16    resp = requests.post(endpoint, headers={...}, json={
17        "model": "gpt-3.5-turbo",
18        "messages": messages,
19        "max_tokens": 512,
20        "temperature": 0.7,
21    })

```

兼容所有使用 OpenAI API 格式的服务：- 云端：OpenAI、Azure OpenAI、通义千问、DeepSeek - 本地：Ollama、vLLM、llama.cpp server、LocalAI

2.6.3 对话历史管理

ConversationHistory 是一个固定容量的环形缓冲区：

```

1 class ConversationHistory:
2     def __init__(self, max_turns=10):
3         self._turns = []
4         self._max = max_turns
5
6     def add(self, role, text):
7         self._turns.append(Turn(role=role, text=text, timestamp=time.time()))
8         if len(self._turns) > self._max:
9             self._turns.pop(0) # 超出容量时丢弃最早的
10
11    def get_recent(self, n=4):
12        return self._turns[-n:] # 获取最近n轮

```

保留最近 10 轮对话，云端模式下将最近 4 轮作为上下文发送给 LLM。这样做既维持了多轮对话的连贯性，又控制了 API 调用的 token 消耗。

9.2.7. 2.7. 语音交互系统

语音交互采用浏览器端录音 + 服务端识别的架构。Gradio 的 Audio 组件负责在浏览器中通过 MediaRecorder API 录制音频，然后将 WAV 数据发送到服务端。服务端的 SpeechRecognizer 接收音频文件，调用 Google Web Speech API 完成语音识别。

2.7.1 语音识别

```

1 class SpeechRecognizer:
2     def recognize_from_file(self, audio_path):
3         recognizer = sr.Recognizer()
4         with sr.AudioFile(audio_path) as source:
5             audio = recognizer.record(source)
6         return recognizer.recognize_google(audio, language="zh-CN")

```

Google Web Speech API 的优势是免费、无需注册、中英文识别准确率不错。局限是需要网络连接，且可能有频率限制。在生产环境中可以替换为：- **Vosk** — 离线、支持中文的小型识别模型 - **Whisper.cpp** — OpenAI Whisper 的 C++ 移植，在 CPU 上可运行 - **FunASR** — 阿里达摩院出品，中文识别效果优秀

2.7.2 语音合成

```

1 class TextToSpeech:
2     def speak(self, text):
3         def _run():
4             engine = pyttsx3.init()
5             engine.setProperty("rate", 160) # 语速
6             engine.setProperty("volume", 0.8) # 音量
7             engine.say(text)

```

```

8     engine.runAndWait()
9     threading.Thread(target=_run, daemon=True).start()

```

pyttsx3 在 Linux 上使用 espeak 作为后端，离线可用、零成本。缺点是中文发音偏机械，自然度不及商业 TTS 服务。如需更好的中文语音效果，可替换为 Piper TTS（开源、离线、中文音色更自然）。

2.7.3 浏览器端录音

Gradio 的 `gr.Audio(sources=["microphone"], type="numpy")` 返回 `(sample_rate, audio_array ↪ )`，其中 `audio_array` 是 float32 numpy 数组。需要在服务端将其转换为 WAV 格式，因为 `speech_recognition` 只接受文件路径：

```

1 def voice_input_fn(audio):
2     sr_val, audio_data = audio
3     audio_int16 = (audio_data * 32767).astype(np.int16)
4
5     # 写入临时 WAV 文件
6     with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as tmp:
7         with wave.open(tmp, "wb") as wf:
8             wf.setnchannels(1)
9             wf.setsampwidth(2)
10            wf.setframerate(sr_val)
11            wf.writeframes(audio_int16.tobytes())
12            tmp_path = tmp.name
13
14            text = asr.recognize_from_file(tmp_path)
15            os.unlink(tmp_path)
16            return text, text # 返回识别结果，自动填入聊天输入框

```

9.2.8. 2.8. Web 界面与 API 服务

本项目使用 **Gradio** 构建 Web 界面。Gradio 是 HuggingFace 出品的 Python 库，专为机器学习模型演示而设计。与 Flask/FastAPI 相比，Gradio 的优势在于：不需要写 HTML/CSS/JS 代码，Python 类定义直接映射为 Web 组件，内置 WebSocket 连接管理、队列系统和错误处理。

2.8.1 界面设计

```

1 with gr.Blocks(theme=gr.themes.Soft(), title="Ascend 310B 智能聊天机器人") as
   ↪ demo:
2     with gr.Tabs():
3         with gr.TabItem(" 对话"):
4             chatbot = gr.Chatbot(height=500)
5             msg_box = gr.Textbox(label="输入消息")
6             send_btn = gr.Button("发送", variant="primary")
7             audio_in = gr.Audio(sources=["microphone"], type="numpy")
8
9             # 文本对话
10            msg_box.submit(chat_fn, [msg_box, chatbot], [msg_box, chatbot])

```



```

11         send_btn.click(chat_fn, [msg_box, chatbot], [msg_box, chatbot])
12
13         # 语音输入
14         audio_in.stop_recording(voice_input_fn, audio_in, [status, msg_box])
15
16         with gr.TabItem(" 设置"):
17             cloud_toggle = gr.Checkbox(label="启用云端 LLM")
18             ...

```

gr.Chatbot 组件内置了聊天气泡样式和自动滚动，交互逻辑只需关注 handler 函数的输入输出映射。

2.8.2 事件流

Gradio 的事件驱动模型：

```

1 flowchart TD
2     subgraph TEXT["文本输入路径"]
3         direction LR
4         T1["用户输入文本"] --> T2["msg_box.submit()"]
5         T2 --> T3["chat_fn(message, history)"]
6         T3 --> T4["dialogue_manager<br/>.process_message()"]
7         T4 --> T5["返回<br/>(reply, updated_history)"]
8         T5 --> T6["chatbot 自动刷新"]
9     end
10
11     subgraph VOICE["语音 语音输入路径"]
12         direction LR
13         V1["用户点击录音按钮"] --> V2["audio_in 开始录音"]
14         V2 --> V3["松开触发<br/>stop_recording 事件"]
15         V3 --> V4["voice_input_fn(audio)<br/>WAV 转换 + ASR 识别"]
16         V4 --> V5["识别文本填入<br/>msg_box"]
17     end
18
19     V5 -->|"触发 submit 链"| T2

```

9.2.9. 2.9. 用户手册

2.9.1 系统部署

1. 克隆代码：进入 samples/case9/ 目录
2. 安装环境：bash setup.sh
3. 准备模型：python3 prepare_models.py（自动下载、导出 ONNX、转换为 OM）
4. 启动服务：python3 app.py
5. 访问界面：打开浏览器访问 <http://127.0.0.1:7860>

如果没有昇腾设备，跳过第 3 步的 ATC 转换即可。系统会自动使用 CPU 模式运行，所有功能不受影响。

2.9.2 配置说明

配置项	位置	说明
嵌入模型	config.py	默认 all-MiniLM-L6-v2，可换其他 sentence-transformers 模型
检索数量	config.py: ↪ TOP_K_RETRIEVAL	默认 3，增大可提供更多上下文
相似度阈值	config.py: ↪ SIMILARITY_THRESHOLD	默认 0.3，提高可过滤不相关结果
云端 API	设置面板	支持任何 OpenAI 兼容接口 (Ollama/vLLM/通义千问等)
语音开关	设置面板	关闭后仅文本交互

2.9.3 使用指南

- 文本对话：直接在输入框打字，回车发送
- 语音输入：点击语音按钮开始录音，再次点击停止，自动识别并发送
- 知识问答：询问昇腾 310B、CANN、边缘计算、RAG 等相关问题，系统会从知识库检索后回答
- 云端增强：在设置面板填入 API Key 并启用，回复质量大幅提升
- 扩展知识库：编辑 data/sample_knowledge.txt 添加自己的知识条目，重启服务即可生效

2.9.4 维护与扩展

- 添加自定义知识：在 data/ 目录下创建文本文件，修改 app.py 中 get_knowledge_base() 的加载逻辑
- 切换嵌入模型：修改 config.py 中的 MODEL_NAME，重新运行 prepare_models.py
- 自定义回复模板：编辑 data/sample_faq.json 增加 FAQ 问答对
- 接入离线 ASR：将 voice_io.py 中的 recognize_google 替换为 Vosk 或 Whisper
- 日志监控：终端输出包含请求日志和检索上下文，方便调试

9.3. 3. 源代码结构

```

1 samples/case9/
2 |— app.py                # Gradio Web 界面入口，事件绑定
3 |— ascend_inference.py  # Ascend NPU 推理封装
4 |   |— AscendSystem      # NPU 设备初始化 / 释放
5 |   |— AscendModel       # OM 模型加载 / H2D / execute / D2H
6 |   |— EmbeddingModel    # 文本嵌入业务层 (tokenize -> NPU/CPU -> 归一化)
7 |   |— create_embedding_model() # 工厂函数，自动尝试 NPU -> 回退 CPU
8 |— knowledge_base.py    # RAG 检索引擎
9 |   |— Document          # 文档数据类
10 |   |— KnowledgeBase     # FAISS 索引管理 / 分块 / 检索 / 持久化
11 |— dialogue.py         # 对话管理器
12 |   |— State             # 对话状态枚举
13 |   |— ConversationHistory # 环形缓冲对话历史
14 |   |— DialogueManager   # 意图检测 -> RAG -> 模板/API 回复
15 |— voice_io.py         # 语音交互
16 |   |— SpeechRecognizer  # 语音识别 (Google API / 文件)
17 |   |— TextToSpeech      # 语音合成 (pyttsx3 / espeak)
18 |— config.py           # 全局配置常量
19 |— prepare_models.py    # 模型准备 (ONNX 导出 + ATC 转换)
20 |— setup.sh            # 一键环境安装
21 |— requirements.txt     # Python 依赖清单
22 |— data/
23 |   |— sample_knowledge.txt # 示例知识库 (约 20 条昇腾/边缘计算知识)
24 |   |— sample_faq.json    # 示例 FAQ 问答对 (15 组模式匹配)
25 |— models/             # 模型文件目录
26 |— README.md            # 快速开始指南

```

各模块的调用关系：

```

1 flowchart TB
2   APP["app.py<br/>Gradio Web 入口"]
3
4   subgraph MODULES["核心模块"]
5     EMBED["ascend_inference<br/>EmbeddingModel<br/>文本 -> 384维向量"]
6     KB["knowledge_base<br/>KnowledgeBase<br/>FAISS 索引管理 & 检索"]
7     DIALOG["dialogue<br/>DialogueManager<br/>对话状态机 & 回复生成"]
8     VOICE["voice_io<br/>SpeechRecognizer<br/>TextToSpeech<br/>语音识别 & 合成"]
9   end
10
11   APP --> EMBED
12   APP --> KB
13   APP --> DIALOG
14   APP --> VOICE
15   DIALOG --> EMBED
16   DIALOG --> KB

```

9.4. 4. 效果演示

9.4.1. 基础对话 — 离线模板模式

1 用户：什么是昇腾310B？
2 机器人：根据我的知识库，以下是相关内容：
3 1. 昇腾310B是华为推出的面向边缘计算场景的AI推理处理器...
4 2. 昇腾310B支持FP16和INT8两种计算精度...
5 请问还有什么想了解的吗？

9.4.2. RAG 检索 — 专业问题

1 用户：CANN的ATC工具怎么用？
2 机器人：根据我的知识库，以下是相关内容：
3 1. ATC（Ascend Tensor Compiler）是昇腾的模型转换工具...
4 2. 使用ATC进行模型转换的基本流程是：首先将训练好的模型导出为ONNX格式...
5 请问还有什么想了解的吗？

9.4.3. 语音交互

1 用户：（点击录音按钮）"什么是边缘计算？"
2 机器人：（识别文字 -> RAG检索 -> 返回回复 -> TTS朗读）
3 "根据我的知识库，边缘计算是一种将计算和数据存储从云端推到..."

9.4.4. 性能指标

指标	CPU 模式	NPU 模式	说明
嵌入延迟 (单条)	50-80ms	10-15ms	all-MiniLM-L6-v2, 256 tokens
嵌入延迟 (批量 32 条)	200ms	80ms	知识库批量入库
FAISS 检索 (1000 条)	< 1ms	< 1ms	IndexFlatIP
端到端响应 (模板)	~100ms	~30ms	不含语音
端到端响应 (云端 LLM)	1-3s	1-3s	取决于网络和 API
语音识别延迟	1-2s	1-2s	Google Web Speech API

关键观察: NPU 对嵌入计算有约 4-5x 的加速, 这对批量入库知识库时效果明显。端到端延迟的主要瓶颈在云端 API 和语音识别服务, 而非本地推理。这也印证了架构设计的合理性——NPU 被用在它最擅长的密集矩阵计算上, 而系统的其他部分不受 NPU 限制。

A. 工具箱、FAQ 与参数模板

A.1. A.1 附录总览

本附录聚焦“查得快、用得稳”：常见报错速查、转换参数模板、性能/质量 Checklist、术语字典、推荐资源与社区贡献规范。可作为日常开发随手翻阅的工具章节。

A.2. A.2 常见报错速查

分类	报错/现象	可能原因	排查步骤	解决建议
ATC	E19001: Op Not → Supported	新算子/版本落后	确认 CANN 版本 + onnxsim 简化	升级/替换结构/自 定义算子
ATC	Shape 推断失败	动态维度不明确	检查 --input_shape/动 态参数	固定关键维度或提 供范围
ACL	aclmdlLoadFromFile → failed	权限/路径/模型损 坏	校验文件 hash/权 限	修正权限/重新生 成 OM
Runtime	OOM / alloc 失败	Batch 或分辨率过 大	统计输入分布	降 batch/分桶/复 用内存
运行	推理输出 NAN	数值溢出/量化尺 度错误	Dump 中间 Tensor	调整量化/保留 FP32 层
性能	Timeline 大量 gap	Host 阻塞/小算子	Profiling 分析	合并算子/异步预 取
性能	H2D 高占比 >25%	多次小拷贝	合并缓冲	AIPP 下沉/批量化
精度	Top1 下降 >1%	预处理不匹配	对比 ONNX 输出	统一 Normalize & Layout
精度	mAP 不稳定	阈值或 NMS 误差	调整阈值/比对中 间框	校准 NMS 公式/尺 度
稳定	间歇 Crash	悬空指针/并发访 问	启用 ASAN/日志 回溯	修订生命周期/加 锁
部署	模型加载慢	冷启动/IO 慢	预热/缓存	预加载 + 固态存储

分类	报错/现象	可能原因	排查步骤	解决建议
安全	日志泄露敏感路径	直接 print	grep 审计	结构化日志脱敏

A.3. A.3 模型转换参数模板合集

A.3.1. A.3.1 分类模型 (ResNet)

```

1 atc --model=resnet50.onnx \
2   --framework=5 \
3   --output=resnet50_fp16 \
4   --input_format=NCHW \
5   --input_shape="input:1,3,224,224" \
6   --soc_version=Ascend310B \
7   --precision_mode=allow_fp32_to_fp16 \
8   --log=info

```

A.3.2. A.3.2 YOLO 动态分辨率

```

1 atc --model=yolov5s.onnx \
2   --framework=5 \
3   --output=yolov5s_640_768 \
4   --dynamic_image_size="640,640;768,768" \
5   --input_format=NCHW \
6   --soc_version=Ascend310B \
7   --op_select_implmode=high_performance \
8   --precision_mode=allow_fp32_to_fp16

```

A.3.3. A.3.3 INT8 量化 (示例)

```

1 atc --model=resnet50.onnx \
2   --framework=5 \
3   --output=resnet50_int8 \
4   --input_format=NCHW \
5   --input_shape="input:1,3,224,224" \
6   --soc_version=Ascend310B \
7   --precision_mode=allow_mix_precision \
8   --insert_op_conf=aipp.cfg \
9   --enable_small_channel=true

```

A.4. A.4 性能与质量 Checklist (执行勾项)

性能:

- ☐ Profiling 无明显 Idle gap > 10%
- ☐ H2D + D2H 占比 < 25%
- ☐ Postprocess 占比 < 20%
- ☐ Stream 利用率平衡（无单流饱和）
- ☐ 使用内存池减少频繁 alloc/free

精度：

- ☐ ONNX vs OM Top1 差异 < 0.2%
- ☐ L1 平均误差 < 1e-3 (FP16)
- ☐ NMS 输出框数量与基线差异 < 1 框/图（平均）
- ☐ INT8 校准集覆盖多场景

稳定性：

- ☐ 1h 稳态无 Crash / OOM
- ☐ 温度在安全区间 < 85°C
- ☐ 看门狗重启次数 = 0

安全：

- ☐ 日志无明文密钥
- ☐ 模型文件 hash 校验通过

A.5. A.5 术语表（扩展）

术语	说明
OM	Ascend 离线模型二进制格式
ACL	Ascend 计算语言 API 层
ATC	模型转换/编译工具
AIPP	自动图像预处理模块
Stream	异步任务调度通道
Profiling	性能采样分析工具体系
Fallback	算子未匹配优化实现退回通用实现
Quant Calibration	量化尺度统计过程
Baseline	初始标准对照性能/精度集
Drift	指标随时间未经预期的漂移

A.6. A.6 推荐资源与外部引用

- Ascend 官方文档入口（安装/算子列表/最佳实践）
- CANN Release Notes：版本兼容与已知问题。
- ONNX Operator 列表与语义说明。
- Open Model Zoo / ModelScope：获取预训练模型与许可信息。
- 学术资源：算子融合、低比特量化、蒸馏相关论文列表。

A.7. A.7 贡献指南摘要

流程：Fork -> 新分支 -> 修改/新增 -> 本地 lint & 生成脚本 -> PR（描述动机/影响面/验证方式）。
PR 要求：| 要素 | 说明 | | — | — | | 标题 | 简明说明改动作用 | | 描述 | 背景 + 修改点 + 风险 | | 验证 | 性能/精度/功能截图或数据 | | 回滚 | 若失败如何恢复 | | 关联 Issue | 追踪链接 |

A.8. A.8 FAQ

问题	回答
模型转换慢怎么办？	使用 SSD，关闭调试日志，检查不必要动态 shape。
精度下降如何定位？	离线脚本层级 Dump 比对，逐层二分。
如何减少内存占用？	启用内存池 + 减少中间冗余张量 + 固定 batch。
量化后收益不明显？	检查是否 Compute-bound，或激活分布集中导致尺度相近。
NMS 很慢？	合并小框批量处理/降低候选阈值/考虑 Device 版 NMS。

A.9. A.9 License 与引用

本书内容遵循 Apache 2.0 许可证。引用：> 《昇腾 310B 实战：从入门到精通边缘计算与人工智能》（GitHub: zhouxzh/Ascend310）

A.10. A.10 版本路线回顾

版本	内容	目标
v0.1	结构框架	验证框架可行
v0.3	核心部署链路	形成可用主线
v0.6	案例与工程化	贴近实战
v1.0	全面审校发行	正式发布

A.11. A.11 实践任务

1. 为你的项目添加 1 条本地常见错误记录（含根因与解决）。
2. 复制分类 ATC 模板并改写为检测模型版本（含动态尺寸）。
3. 在术语表补充 3 个任务相关术语（并验证唯一性）。
4. 选取 FAQ 一条，写出更深入排查脚本思路。